# PicoMite
# User Manual

## MMBasic BASIC Interpreter
## Ver 7.0.00

For the
Raspberry Pi Pico
Raspberry Pi Pico 2
Raspberry Pi Pico W
Raspberry Pi Pico 2 W
and
modules using the RP2040 and RP2350 processors

(13 September 2026)
13 October 2026

For updates to this manual and more details on MMBasic go to
http://geoffg.net/picomite.html
and http://mmbasic.com

# About

Peter Mather (matherp on the Back Shed Forum) led the project, ported MMBasic to the Raspberry Pi Pico and wrote the drivers for its hardware features. The MMBasic interpreter and this manual was written by Geoff Graham ( http://geoffg.net ). In addition, many others have supported the project with specialised code, testing and suggestions.

# Support

Support questions should be raised on the Back Shed forum ( http://www.thebackshed.com/forum/Microcontrollers ) where there are many enthusiastic MMBasic users who would be only too happy to help. The developers of the PicoMite firmware are also regulars on this forum.

# Copyright and Acknowledgments

The PicoMite firmware and MMBasic is copyright 2011-2025 by Geoff Graham and Peter Mather 2016-2025.
1-Wire Support is copyright 1999-2006 Dallas Semiconductor Corporation and 2012 Gerard Sexton.
FatFs (SD Card) driver is copyright 2014, ChaN.
WAV, MP3, and FLAC file support is copyright 2019 David Reid.
JPG support is thanks to Rich Geldreich
The pico-sdk is copyright 2021 Raspberry Pi (Trading) Ltd.
TinyUSB is copyright tinyusb.org
LittleFS is copyright Christopher Haster
Thomas Williams and Gerry Allardice for MMBasic enhancements
The VGA driver code was derived from work by Miroslav Nemecek
The CRC calculations are copyright Rob Tillaart
PicoCalc enhancements are improved and collated by Ernst Bokkelkamp

The compiled object code (the .uf2 file) for the PicoMite firmware is free software: you can use or redistribute it as you please. The source code is on GitHub ( https://github.com/UKTailwind/PicoMiteAllVersions ) and can be freely used subject to some conditions (see the header in the source files).

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY, without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

# This Manual

Copyright 2026 Geoff Graham and Peter Mather
The author of this manual is Geoff Graham with considerable input by Peter Mather, Harm de Leeuw, Mick Ames and many others on The Back Shed forum. It is distributed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Australia license (CC BY-NC-SA 3.0)

# Contents


Introduction ..... 10

Firmware Versions and Files ..... 11

Self Contained Computer ..... 11
Embedded Controller ..... 11
Processor Support ..... 11
PicoCalc Support ..... 11
Pico Computer 3 Support ..... 11
File Names ..... 12
Loading the Firmware ..... 13

Serial Console ..... 14

Virtual Serial Port ..... 14
Terminal Emulator ..... 14
The Console ..... 14
Windows 7 and 8.1 ..... 15
Apple Macintosh ..... 15
Linux ..... 15
Android ..... 15

First Steps ..... 16

A Simple Program ..... 16
Flashing a LED ..... 17
Tutorial on Programming in the BASIC Language ..... 17

Hardware Details ..... 18

Third Party Modules ..... 19
WebMite version for the Raspberry Pi Pico W or Pico 2 W ..... 19
CPU Variations ..... 19
PSRAM ..... 20
I/O Pin Limits ..... 20
Power Supply ..... 21
Clock Speed ..... 21
Power Consumption ..... 21

Using MMBasic ..... 23

Commands and Program Input ..... 23
Program Structure ..... 23
Editing the Command Line ..... 23
Shortcut Keys ..... 23
Interrupting A Running Program ..... 24
Setting Options ..... 24
Saved Variables ..... 24
Watchdog Timer ..... 24
PIN Security ..... 24
The Library ..... 25
Program Initialisation ..... 25
MM.STARTUP ..... 26
MM.PROMPT ..... 26
MM.END ..... 26

Calculator Immediate Mode ..... 27
Full Screen Editor ..... 29
Edit Functions ..... 29
Mark Mode ..... 30
Alternate Keys ..... 30
Long Lines ..... 31
Using a Mouse ..... 31
Colour Coded Editor Display ..... 31
File Manager ..... 32
Variables and Expressions ..... 34
Variables ..... 34
Constants ..... 34
OPTION DEFAULT ..... 34
OPTION EXPLICIT ..... 35
DIM and LOCAL ..... 35
STATIC ..... 36
CONST ..... 36
Special Characters in Strings ..... 37
Expressions and Operators ..... 37
Mixing Floating Point and Integers ..... 38
Structures ..... 38
64-bit Unsigned Integers ..... 39
Subroutines and Functions ..... 40
Subroutines ..... 40
Functions ..... 40
Passing Arguments by Reference ..... 41
Passing Arrays ..... 41
Early Exit ..... 42
Recursion ..... 42
Examples ..... 42
Video Output ..... 44
VGA Video ..... 44
HDMI Video ..... 44
VGA/PS2 Reference Design (Raspberry Pi Pico) ..... 46
HDMI/USB Reference Design (Raspberry Pi Pico 2) ..... 46
Keyboard/Mouse/Gamepad ..... 47
PS2 Keyboard on the Raspberry Pi Pico (RP2040) ..... 47
PS2 Keyboard on the Raspberry Pi Pico 2 (RP2350) ..... 47
PS2 Mouse ..... 47
USB Interface ..... 48
USB Hub ..... 48
USB Keyboard ..... 48
USB Mouse ..... 48
USB Gamepad ..... 48
USB Touch Screen ..... 48
USB Serial Devices (COM3 to COM6) ..... 49
USB Flash Drive ..... 49

Configuring the Keyboard ..... 49
Using a Mouse ..... 50
On-Screen Keyboard ..... 50
Program and Data Storage ..... 52
Flash Slots ..... 52
Flash Filesystem ..... 52
SD Cards ..... 53
Combined Chip Select ..... 54
USB Flash Drive ..... 54
MMBasic Support for Flash and SD Card Filesystems ..... 54
XModem Transfer ..... 56
Load and Save Image ..... 57
Example of Sequential I/O ..... 57
Random File I/O ..... 58
Sound Output ..... 60
Pulse Width Modulated (PWM) Signal ..... 60
Filter Circuits ..... 60
VS1053 support ..... 61
MCP48n2 DAC ..... 61
Bluetooth Speakers and Headphones ..... 61
Playing WAV, FLAC, MP3 and MOD Files ..... 62
Generating Sine Waves ..... 62
Using PLAY ..... 62
Utility Commands ..... 63
Specialised Audio Output ..... 63
Using the I/O pins ..... 64
Digital Inputs ..... 64
Analog Inputs ..... 64
Counting Inputs ..... 65
Digital Outputs ..... 65
Pulse Width Modulation ..... 65
Communications Interfaces (Serial, SPI and I$^2$C) ..... 66
Interrupts ..... 66
Special Device Support ..... 68
Infrared Remote Control Decoder ..... 68
Infrared Remote Control Transmitter ..... 69
Measuring Temperature ..... 69
Measuring Humidity and Temperature ..... 70
Real Time Clock Interface ..... 70
Measuring Distance ..... 71
LCD Display ..... 71
Keypad Interface ..... 72
WS2812 Support ..... 73
OV7670 Camera module ..... 73
Display Panels ..... 74
SPI Based Display Panels ..... 74
I$^2$C Based LCD Panels ..... 76

8-bit Parallel LCD Panels ..... 77
Connecting an 8-bit parallel LCD Panel ..... 77
Configuring an 8-bit parallel LCD Panel ..... 79
8 and 9 inch Displays ..... 79
16-bit Parallel LCD Panels ..... 79
RP2350 Extended Display Support ..... 80
VGA222 drivers ..... 81
Backlight Control ..... 81
Touch Support ..... 82
Calibrating the Touch Screen ..... 82
Touch Functions ..... 82
Touch Interrupts ..... 83
LCD Display as the Console Output ..... 83
Example SPI LCD Panel Configuration ..... 83
Graphics Functions ..... 85
Supported Hardware ..... 85
Colours ..... 86
Screen Coordinates ..... 86
Fonts ..... 87
Embedded Fonts ..... 87
Drawing Commands ..... 88
Rotated Text ..... 89
Transparent Text ..... 89
Framebuffers and Layers ..... 89
BLIT and Sprite Commands ..... 90
Load Image ..... 91
Advanced Graphics ..... 91
LCD Graphics Example ..... 91
GUI Integration ..... 93
WiFi and Internet Functions ..... 95
Connecting to a WiFi Network ..... 95
Remote Console Access ..... 96
File Transfer ..... 96
Getting the Time ..... 96
Implementing a Web Server ..... 97
Live Graphical Data in a WEB Page ..... 99
A Complete General Purpose Server ..... 99
A Typical WEB Page ..... 100
Input Fields and Control ..... 100
Implementing a TCP Client ..... 101
Secure Connections (TLS) ..... 102
Using UDP ..... 102
Sending EMails ..... 103
Base 64 Encoding ..... 105
MQTT Client ..... 105
Ping ..... 105
Streaming audio ..... 105

Long Strings ..... 107
Long String Variables ..... 107
Long String Commands ..... 107
Long String Functions ..... 108
Bluetooth Versions ..... 109
PicoMiteBT – Console over Bluetooth ..... 109
PicoMiteBTH – Bluetooth Keyboards and Mice ..... 109
PicoMiteBTH – Bluetooth Speakers and Headphones ..... 110
PicoMiteBTH – Bluetooth Serial Devices (COM7) ..... 111
MMBasic Characteristics ..... 113
Naming Conventions ..... 113
Constants ..... 113
Implementation Characteristics ..... 113
Compatibility ..... 114
Predefined Read Only Variables ..... 115
Detailed Listing ..... 115
Options ..... 122
Detailed Listing ..... 122
Commands ..... 138
Detailed Listing ..... 138
Functions ..... 228
Detailed Listing ..... 228
Obsolete Commands and Functions ..... 247
Detailed Listing ..... 247
Appendix A – Serial Communications ..... 248
I/O Pins ..... 248
Commands ..... 248
The OPEN Command ..... 248
Examples ..... 249
Reading and Writing ..... 249
Interrupts ..... 249
Appendix B – I2C Communications ..... 250
I/O Pins ..... 250
I²C Master Commands ..... 250
I²C Slave Commands ..... 251
Errors ..... 252
7-Bit Addressing ..... 252
System I2C ..... 252
Examples ..... 252
Appendix C – 1-Wire Communications ..... 254
Appendix D – SPI Communications ..... 255
I/O Pins ..... 255
SPI Open ..... 255
Transmission Format ..... 255
Standard Send/Receive ..... 255

Bulk Send/Receive ..... 256
SPI Close ..... 256
Examples ..... 256
Appendix E – Regex Syntax ..... 257
Using Regular expressions with OPTION ESCAPE ..... 258
Appendix F – The PIO Programming Package ..... 259
Introduction to the PIO ..... 259
Availability of PIOs ..... 259
Overview of PIO ..... 259
Programming PIO ..... 260
Configuring PIO ..... 261
FIFO's ..... 263
DMA To and From the FIFOs ..... 265
Appendix G – Sprites ..... 270
Appendix H – Turtle Graphics ..... 272
Appendix I – Special Keyboard Keys ..... 275
Appendix J – Programming in BASIC - A Tutorial ..... 277
Command Prompt ..... 277
Structure of a BASIC Program ..... 277
Comments ..... 278
The PRINT Command ..... 278
Variables ..... 279
Expressions ..... 280
The IF Statement ..... 281
FOR Loops ..... 283
Multiplication Table ..... 284
DO Loops ..... 284
Console Input ..... 285
GOTO and Labels ..... 286
Testing for Prime Numbers ..... 287
Arrays ..... 289
Integers ..... 290
Strings ..... 290
Manipulating Strings ..... 291
Scientific Notation ..... 292
DIM Command ..... 293
Constants ..... 294
Subroutines ..... 294
Functions ..... 296
Local Variables ..... 297
Static Variables ..... 298
Calculate Days ..... 299
Appendix K – Firmware Version Applicability ..... 301
The sixteen firmware images ..... 301
Version labels ..... 301
Conditions ..... 304

Appendix L – Where Each Keyword Is Documented .....304
Appendix M – Companion Manuals.....305

# Introduction



The PicoMite is an operating firmware for all versions of the Raspberry Pi Pico including the Pico, Pico 2, Pico W and Pico 2 W.

It includes a BASIC interpreter (MMBasic) which is a Microsoft BASIC compatible implementation of the BASIC language with floating point, integer and string variables, arrays, long variable names, a built in program editor and many other features.

There are versions of the PicoMite firmware suited for embedded controller applications (such as a heating controller, burglar alarm, etc) as well as versions with VGA/HDMI and keyboard support for building a self contained computer.

Using MMBasic you can control the I/O pins and use communications protocols such as $I^2C$ or SPI to get data from a variety of sensors. You can display data on low-cost colour LCD displays, measure voltages, detect digital inputs and drive output pins to turn on lights, relays, etc. And with the Raspberry Pi Pico W you can access the internet and build a WEB server on this low cost module.

The PicoMite firmware is totally free to download and use.

In summary the features of the PicoMite firmware are:

- **The BASIC interpreter is full featured** with double precision floating point, 64-bit integers and string variables, long variable names, arrays of floats, integers or strings with multiple dimensions, extensive string handling and user defined structures, subroutines and functions. In addition, MMBasic allows the embedding of compiled C programs for high performance functions. The emphasis is on ease of use and development.
- **Support for all Raspberry Pi Pico input/output pins**. These can be independently configured as digital input or output, analog input, frequency or period measurement and counting. Interrupts can be used to notify when an input pin has changed state. PWM outputs can be used to create various sounds, control servos or generate computer-controlled voltages.
- **Support for LCD/OLED display panels** using parallel, SPI and $I^2C$ interfaces allowing the BASIC program to display text and draw lines, circles, boxes, etc in up to 16 million colours. Resistive touch controllers on these panels are also supported allowing them to be used as sophisticated input devices.
- **Support for Internet and WEB protocols using the Raspberry Pi Pico W, Pico 2 W and Pimoroni Pico Plus 2W**. This includes a WEB server using TCP and HTML, accessing other resources using TCP and HTTP. MQTT protocol for connecting via a message broker. NTP protocol for getting the date/time from a time server. Telnet for remote console access and TFTP for fast file transfer.
- **Support for a PS2 or USB keyboard and HDMI or VGA video output**. This includes full support for graphics, audio (sound effects and music), internal program storage, game controllers and more. This converts the Raspberry Pi Pico or into a self-contained computer like the Apple II or Tandy TRS-80 of yesterday. Great for writing games, learning BASIC or just balancing your chequebook
- **Flexible program and data storage**. Programs and data can be read/written from an internal file system created from the Pico's flash memory or to an externally connected SD Card up to 32GB formatted as FAT16 or FAT32. This includes opening files for reading, writing or random access and loading and saving programs.
- **A full screen editor** is built into the firmware and can edit the whole program in one session. It includes advanced features such as colour coded syntax, search and copy, cut and paste to and from a clipboard.
- **Programs can be easily transferred** from a desktop or laptop computer (Windows, Mac or Linux) via the serial console or via an SD card.
- **A comprehensive range of communications protocols** are implemented including $I^2C$, asynchronous serial, RS232, SPI and 1-Wire. These can be used to communicate with many sensors (temperature, humidity, acceleration, etc) as well as for sending data to test equipment.
- **Built in commands** to directly interface with infrared remote controls, the DS18B20 temperature sensor, LCD display modules, battery backed clock, numeric keypads and more.

# Firmware Versions and Files

The PicoMite firmware can be used for two distinctly differing roles depending on the version of the firmware loaded. These roles are; a self-contained computer and an embedded controller:

## Self Contained Computer

Versions with VGA or HDMI video output are intended for use as a self contained computer. These boot up and display the output of the BASIC interpreter on the monitor connected to the video output. They are coupled with a PS2 or USB keyboard and by using the keyboard and video output you can enter and edit a program, run it, set options, etc.

Because the self contained computer starts by displaying the BASIC command prompt they are often called "boot to BASIC" computers. They are simple and fun to use and were popular in the 70s and 80s, for example the Apple II, Tandy TRS-80, Commodore 64 and others.

When a program is running all of its output (text and graphics) is displayed on the video output. The text output is also sent to the serial console. This is a secondary communications channel using the Raspberry Pi Pico's USB connector and is another way of communicating with the MMBasic interpreter using a desktop or laptop computer. For the details of using this see the next chapter: *Serial Console*.

## Embedded Controller

Versions of the firmware without a video output are primarily intended for use as an embedded controller. This is where the Raspberry Pi Pico or Pico 2 is used as the brains inside some device. For example, a burglar alarm, a heating controller, weather station, etc. Quite often they have an attached touch sensitive LCD panel for the user to control the device and observe the output.

There is also a version of the firmware that supports the wireless interface on the Raspberry Pi Pico W (and 2 W) and using this you can create an embedded controller which has a miniature web server running on the Pico and can access the Internet to get the time, send emails, etc.

To enter programs, set options and generally manage the Raspberry Pi Pico as an embedded controller you use the serial console to connect to a desktop or laptop computer. Unlike the self contained computer described above, this is the only way to communicate with the BASIC interpreter so it is important that you can connect to it. For a description of the serial console see the heading *Serial Console* below.

## Processor Support

The PicoMite firmware supports the original RP2040 processors used in the Raspberry Pi Pico and the newer RP2350 used in the Raspberry Pi Pico 2. The firmware is also designed to work with modules produced by other vendors that use the same chips.

While the RP2040 has just one version the RP2350 comes in four sub versions designated the RP2350A, RP2350B, RP2354A and the RP2354B. The RP2350B is the same as the RP2350A except that it has 18 additional I/O pins (pins GP30 to GP47) which are automatically made available in MMBasic. Both of these chips are supported by the same PicoMite firmware and work the same. So, within this manual, all references to the RP2350 apply equally to both the A and B variants and the same firmware can be used.

The RP2354A and the RP2354B are not currently supported (although they may be in the future).

Throughout this manual any references to the Raspberry Pi Pico also includes the Raspberry Pi Pico 2 unless it specifically excluded. If there are differences then the part number of the processor (RP2040 or RP2350) will be used to make the difference obvious.

## PicoCalc Support

The PicoMite firmware supports the PicoCalc offered by clockwork. Just load the correct firmware PicoMiteRP2040, PicoMiteRP2350, WebMiteRP2040, or WebMiteRP2350 to a blank Pico and the PicoCalc will be automatically configured ready for use. The built-in keyboard can be used at the command prompt, in the full screen editor and from a program with INKEY$ and KEYDOWN().

## Pico Computer 3 Support

The firmware will also automatically detect and configure the Pico Computer 3. This is based on the RP2350B so any of the RP2350 versions of the firmware can be used - load the firmware to a Pico Computer 3 which has not previously had a platform configured and, at the next boot, the SD card, I2S audio, system I2C, real time clock, PSRAM, video output and interrupt pins will all be set up automatically ready for use.

PicoMitePC3 is a version of the firmware built for the Pico Computer 3 alone. It is the PicoMiteHDMIUSB (HDMI video with USB keyboards, mice and game controllers, and the full set of HDMI resolutions and modes) with the WiFi and Internet features of the PicoMiteHDMIWEB and the Bluetooth features of the PicoMiteHDMIBTH added - Bluetooth keyboards, mice and game controllers, Bluetooth speakers and headphones, and Bluetooth serial devices - together with the OV7670 camera support of the HDMI versions. It needs the PSRAM fitted to the Pico Computer 3: its program memory is 256K, and the editor, LOAD, AUTOSAVE, LIBRARY and the XModem, YModem and ZModem transfers take their buffers from the PSRAM, so with the PSRAM disabled a program larger than the internal memory cannot be loaded. Like the other Bluetooth host versions it does not support audio through PWM, an MCP48n2 DAC or a VS1053: sound is played through the I2S DAC of the Pico Computer 3 or a Bluetooth speaker. When WiFi and a Bluetooth speaker are both in use the radio gives the speaker priority while sound is playing, so a WiFi transfer slows to a crawl until the sound stops and then recovers at once. OPTION RADIO selects which halves of the wireless chip are used - both, WiFi only, Bluetooth only or neither - so the PicoMitePC3 can be run as a plain HDMI computer, as a WebMite or as a Bluetooth host, with the half not in use costing nothing; with neither the chip is left unpowered, its pins are free and the heartbeat LED is on GP25.

## File Names

There are seventeen firmware files contained in the firmware distribution zip file.

A typical filename for a firmware image looks like this:

PicoMiteRP2350VGAUSBV7.0.00.uf2

Where (in this example):

- **RP2350** is the processor that the firmware is compiled for.
- **VGAUSB** is the feature set supported (VGA and USB).
- **V7.0.00 is the version number**. This will be incremented in future releases.
- **.uf2** is the extension indicating a loadable Raspberry Pi Pico firmware image.

The following table lists the prefix for each firmware file and its associated capabilities.

| **Firmware File Name** For example: PicoMiteRP2040V7.0.00.uf2 | CPU | Touch LCD Panel | Keyboard/Mouse | | Video Output | | WiFi Internet |
|---|---|---|---|---|---|---|---|
| | | | PS2 | USB | VGA | HDMI | |
| PicoMiteRP2040 | RP2040 | ✓ | ✓ | | | | |
| PicoMiteRP2350 | RP2350 | ✓ | ✓ | | | | |
| PicoMiteRP2040MIN | RP2040 | ✓ | ✓ | | | | |
| PicoMiteRP2040USB | RP2040 | ✓ | | ✓ | | | |
| PicoMiteRP2350USB | RP2350 | ✓ | | ✓ | | | |
| PicoMiteRP2040VGA | RP2040 | | ✓ | | ✓ | | |
| PicoMiteRP2350VGA | RP2350 | | ✓ | | ✓ | | |
| PicoMiteRP2040VGAUSB | RP2040 | | | ✓ | ✓ | | |
| PicoMiteRP2350VGAUSB | RP2350 | | | ✓ | ✓ | | |
| PicoMiteHDMI | RP2350 | USB | ✓ | | | ✓ | |
| PicoMiteHDMIUSB | RP2350 | USB | | ✓ | | ✓ | |
| WebMiteRP2040 | RP2040 | ✓ | ✓ | | | | ✓ |
| WebMiteRP2350 | RP2350A | ✓ | ✓ | | | | ✓ |
| PicoMiteRP2350BT | RP2350A | ✓ | ✓ | | | | |
| PicoMiteRP2350BTH | RP2350A | ✓ | | | | | |
| PicoMiteHDMIWEB | RP2350 | USB | | ✓ | | ✓ | ✓ |
| PicoMiteHDMIBTH | RP2350 | USB | | ✓ | | ✓ | |
| PicoMitePC3 | RP2350B | USB | | ✓ | | ✓ | ✓ |

The BT and BTH versions require a Raspberry Pi Pico 2 W. The BT version provides the console over Bluetooth LE instead of USB while the BTH version supports Bluetooth LE and Bluetooth Classic keyboards and mice and can play its sound through a Bluetooth speaker or headphones (with the console remaining on USB). PicoMiteHDMIWEB combines the HDMI self contained computer (with USB keyboard/mouse support) with the WiFi and Internet features of the WebMite. To recover the memory needed by its network stack the HDMIWEB version has reduced HDMI capabilities: its HDMI output is 8-bit colour and offers a reduced set of resolutions - 640x480 (the default), 720x400, 800x600, 848x480, 800x480 and 1024x600 - selected with OPTION RESOLUTION or, at run time, the RESOLUTION command. PicoMiteHDMIBTH is the PicoMiteHDMIUSB with the Bluetooth features of the BTH version added (Bluetooth LE and Bluetooth Classic keyboards and mice, and Bluetooth speakers or headphones). Like the HDMIWEB version it needs an RP2350 module with the Raspberry Pi wireless chip (such as a Pico 2 W) and it has the same reduced HDMI capabilities. PicoMitePC3 combines both: it is the PicoMiteHDMIUSB with the WiFi of the HDMIWEB version and the Bluetooth of the HDMIBTH version, keeping the full HDMI capabilities; it is built for the Pico Computer 3 and needs its PSRAM (see Pico Computer 3 Support).

## Loading the Firmware

The Raspberry Pi Pico and Pico 2 come with their own built in firmware loader that is easy to use.

To load the PicoMite firmware follow these steps:

- Download the PicoMite firmware from http://geoffg.net/picomite.html, unzip the file and identify the firmware which suits your usage (see the previous headings).
- Using a USB cable plug the Raspberry Pi Pico into your computer (Windows, Linux or Mac) **while holding down the white BOOTSEL button** on the top of the module.
- The Raspberry Pi Pico should connect to your computer and create a virtual drive (the same as if you had plugged in a USB memory stick). You can ignore any files that may be on this "drive"
- Copy the firmware file (with the extension .uf2) to this virtual drive.
- When the copy has completed the Raspberry Pi Pico will restart and create a virtual serial port over USB on your computer. See the chapter *Serial Console* below for the details of using this.
- The LED on the Raspberry Pi Pico will blink slowly indicating that the PicoMite firmware with MMBasic is now running (Pico, Pico2, Pico W, Pico2 W compatible boards)

While the virtual drive created by the Raspberry Pi Pico looks like a USB memory stick it is not, the firmware file will vanish once copied and if you try copying any other type of file it will be ignored.

Loading the PicoMite firmware may erase all the flash memory including the current program, any files in drive A: and all saved variables. So make sure that you backup this data before you upgrade the firmware.

It is possible for the flash memory to be corrupted resulting in unusual and unpredictable behaviour. In that case you should download the appropriate firmware listed below and load it onto the Pico as described above. This will reset the Raspberry Pi Pico to its factory fresh state, then you can reload the PicoMite firmware:

| | |
|---|---|
| Raspberry Pi Pico (RP2040) | https://geoffg.net/Downloads/picomite/Clear_Flash.uf2 |
| Raspberry Pi Pico 2 (RP2350) | https://geoffg.net/Downloads/picomite/Clear_Flash_RP2350.uf2 |

# Serial Console

The serial console is a method of connecting your desktop or laptop computer to the Raspberry Pi Pico and the MMBasic console. With access to the console you can enter and edit programs, run them, etc. Most versions of the firmware will automatically create the serial console as a virtual serial port over USB and this chapter describes how that works and how to use it.

For a self contained computer (described above) the serial console is a secondary communications method but when you are using the Raspberry Pi Pico as an embedded controller it is the only communications method available, so it is important that you can connect to it.

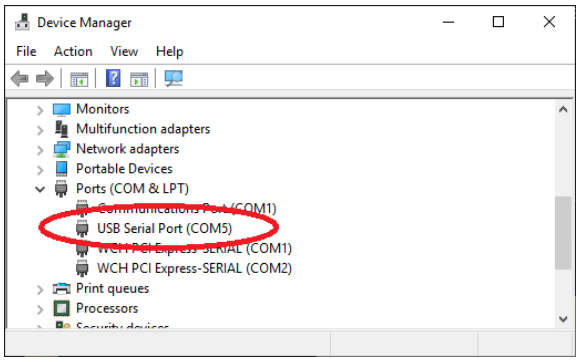
Versions of the PicoMite firmware that support a USB keyboard/mouse cannot create the serial port over USB so, for this firmware, you should refer to the chapter titled *Keyboard/Mouse/Gamepad* for the alternative.

## Virtual Serial Port

The virtual serial port over USB created by the PicoMite firmware uses the CDC (Communication Device Class) protocol and acts like a normal serial port but operating over USB. Windows 10 and 11 includes a driver for this but with other operating systems you may need to load a driver (see the next page).

When you connect the Raspberry Pi Pico's USB connector to your desktop or laptop computer (after loading the PicoMite firmware) the connection will be immediately made.

You should then note the port number created by your computer for the virtual serial connection. In Windows this can be done by starting Device Manager and checking the "Ports (COM & LPT)" entry for a new COM port as shown on the right.




## Terminal Emulator

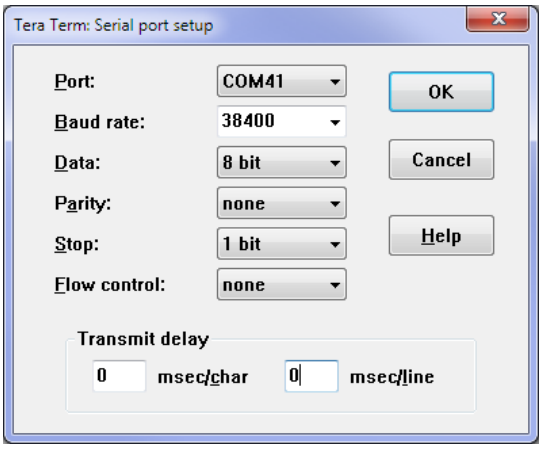
You also need a terminal emulator running on your desktop or laptop computer. This is a program that acts like an old fashioned computer terminal where it will display text received from a remote computer and any key presses will be sent to the remote computer over the serial link. The terminal emulator that you use should support VT100 emulation as this is required by the editor built into the PicoMite firmware.

For Windows users it is recommended that you use Tera Term as this has a good VT100 emulator and is known to work with the XModem and ZModem protocols which you can use to transfer programs to and from the PicoMite. Tera Term can be downloaded from: http://tera-term.en.lo4d.com .

The screen shot on the right shows the setup for Tera Term. Note that the "Port:" setting will vary depending on which USB port your Raspberry Pi Pico was plugged into.

The PicoMite firmware will ignore the baud rate setting so it can be set to any speed (other than 1200 baud which puts the Pico into firmware upgrade mode).

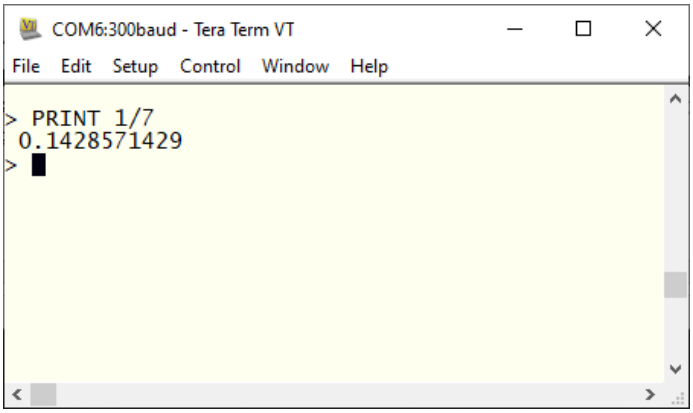
If you are using Tera Term do not set a delay between characters and if you are using Putty set the backspace key to generate the backspace character.




## The Console

Once you have identified the virtual serial port and have connected your terminal emulator to it, you should be able to press return on your keyboard and see the MMBasic prompt, which is the greater than symbol (eg, ">").

This is the console and you use it to issue commands to configure MMBasic, load the BASIC program, edit and run it. MMBasic also uses the console to display any error messages.



## Windows 7 and 8.1

The USB serial port uses the CDC protocol and the drivers for this are standard in Windows 10 and 11 and will load automatically.

The Raspberry Pi Foundation lists Windows 7 or 8.1 as "unsupported" however you can use a tool like Zadig ( https://zadig.akeo.ie ) to install a generic driver for a "usbser" device and that should allow these computers to connect. This post describes the process: https://github.com/raspberrypi/pico-feedback/issues/118

## Apple Macintosh

The Apple Macintosh (OS X) is somewhat easier as it has the device driver and terminal emulator built in. First start the application 'Terminal' and at the prompt list the connected serial devices by typing in:

```
ls /dev/tty.*.
```

The USB to serial converter will be listed as something like `/dev/tty.usbmodem12345`. While still at the Terminal prompt you can run the terminal emulator at 115200 baud by using the command:

```
screen   /dev/tty.usbmodem12345 115200
```

By default the function keys will not be correctly defined for use in the PicoMite's built in program editor so you will have to use the control sequences as defined in the chapter *Full Screen Editor* of this manual. To avoid this you can configure the terminal emulator to generate these codes when the appropriate function keys are pressed.

Documentation for the screen command is here: https://www.systutorials.com/docs/linux/man/1-screen/

## Linux

For Linux see these posts:

https://www.thebackshed.com/forum/ViewTopic.php?TID=14157&PID=175474#175474#175466

and

https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=16312&LastEntry=Y#213664#213594

## Android

For Android devices see this post:

https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=17476&LastEntry=Y#230521#230517

# First Steps

Once you have access to the console and the MMBasic prompt there are a few things that you can do to prove that you have a working computer. All of these commands should be typed at the command prompt (ie, ">").
What you type is shown in bold and the MMBasic output is shown in normal text.

Try a simple calculation:

```
> PRINT 1/7
0.1428571429
```

See how much memory you have:

```
> MEMORY
Program:
   0K ( 0%) Program (0 lines)
 180K (100%) Free

Saved Variables:
  16K (100%) Free

RAM:
   0K ( 0%) 0 Variables
   0K ( 0%) General
 228K (100%) Free 112K (100%) Free
```

What is the current time? Note that the internal clock is reset to midnight on power up.

```
> PRINT TIME$
00:04:01
```

Set the clock to the current time:

```
> TIME$ = "10:45"
```

Check the time again:

```
> PRINT TIME$
10:45:09
```

Count to 20:

```
> FOR a = 1 to 20 : PRINT a; : NEXT a
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

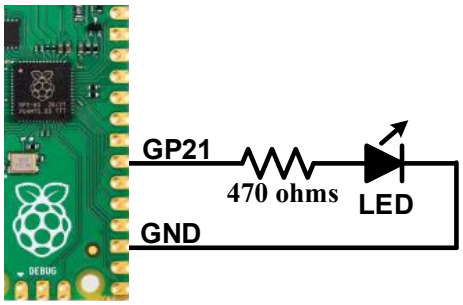
## A Simple Program

To enter a program, you can use the EDIT command which is described later in this manual. However, for the moment, all that you need to know is that anything that you type will be inserted at the cursor, the arrow keys will move the cursor and backspace will delete the character before the cursor.
To get a quick feel for how MMBasic works, try this sequence:

- At the command prompt type `EDIT` followed by the ENTER key.
- The editor should start up and you can enter this line: `PRINT "Hello World"`
- Press the F1 key in your terminal emulator (or CTRL-Q which will do the same thing). This tells the editor to save your program and exit to the command prompt.
- At the command prompt type `RUN` followed by the ENTER key.
- You should see the message: `Hello World`

Congratulations. You have just written and run your first program in BASIC.
If you type EDIT again you will be back in the editor where you can change or add to your program.

## Flashing a LED




Connect a LED to pin GP21 (marked on the underside of the board) and a ground pin as shown in the diagram on the right.

Then use the EDIT command to enter the following program:

```
SETPIN GP21, DOUT
DO
  PIN(GP21) = 1
  PIN(GP21) = 0
LOOP
```

When you have saved and run this program you should be greeted by the LED flashing on and off. It is not a great program but it does illustrate how the PicoMite firmware can interface to the physical world via your programming.

The program itself is simple. The first line sets pin GP21 as an output. Then the program enters a continuous loop where the output of that pin is set high to turn on the LED followed by a short pause (300 milliseconds). The output is then set to low followed by another pause. The program then repeats the loop.

If you leave it this way, the Raspberry Pi Pico will sit there forever with the LED flashing. If you want to change something (for example, the speed of flashing) you can interrupt the program by typing CTRL-C on the console and then edit it as needed. This is the great benefit of MMBasic, it is very easy to write and change a program.

If you want this program to automatically start running every time power is applied, you can use the following command at the command prompt or in the program:

```
OPTION AUTORUN ON
```

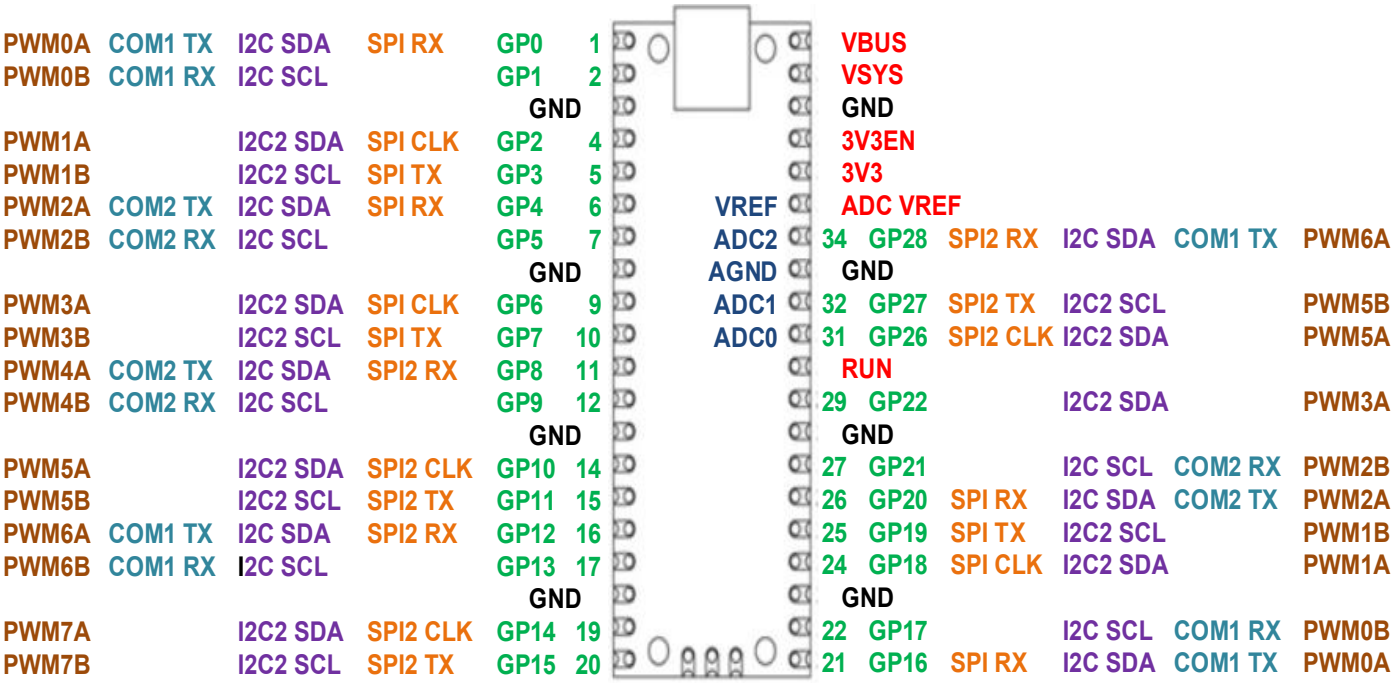
To test this you can remove the power and then re-apply it. The device should start up flashing the LED.

## Tutorial on Programming in the BASIC Language

If you are new to the BASIC programming language now would be a good time to turn to Appendix J (*Programming in BASIC - A Tutorial*) at the end of this manual. This is a comprehensive tutorial on the language which will take you through the fundamentals in an easy to read format with lots of examples.

# Hardware Details

This diagram shows the possible uses within MMBasic for each I/O pin on the Raspberry Pi Pico and Pico 2:




For versions with VGA video output six pins (GP16 to GP21) are reserved for that function. Similarly HDMI versions have eight pins (GP12 to GP19) that are reserved for that function. Refer to the chapter titled *Video Output* for more information.

The version of the firmware with USB keyboard/mouse support also reserves pin 11 (GP8) for the serial console Tx and pin 12 (GP9) for Rx. Refer to the chapter *Keyboard/Mouse/Gamepad* for more information.

The notation is as follows:

| | |
|---|---|
| GP0 to GP28 | Can be used for digital input or output. |
| COM1, COM2 | Can be used for asynchronous serial I/O (UART0 and UART1 pins on the Pico datasheet). |
| I2C, I2C2 | Can be used for I$^2$C communications (I2C0 and I2C1 pins on the Pico datasheet). |
| SPI, SPI2 | Can be used for SPI I/O (see Appendix D). (SPI0 and SPI1 pins on the Pico datasheet). |
| PWMnx | Can be used for PWM output (see the PWM and SERVO commands). |
| GND | Common ground. |
| VBUS | 5V supply directly from the USB port. |
| VSYS | 5V supply used by the SMPS to provide 3.3V. This can be used as a 5V output or input. |
| 3V3EN | Enable 3.3V regulator (low = off, high = enabled). |
| RUN | Reset pin, low will hold the device in reset. |
| ADCn | These pins can be used to measure voltage (analog input). |
| ADC VREF | Reference voltage for voltage measurement. |
| AGND | Analog ground. |

Within the MMBasic program I/O pins can be referred by using the physical pin number (i.e. 1 to 40) or the GP number (i.e. GP0 to GP28). As an example, the following refer to the same pin and operate identically:

```
      SETPIN 32, DOUT
and   SETPIN GP27, DOUT
```

The pin numbers (i.e. 1 to 40) only apply to the standard Pico and Pico 2 form factors, other modules made by third parties could have a different pin numbering scheme. For this reason it is strongly recommended to use only the GP number when referring to an I/O pin.

You can also use string literals and string variables for GP pin numbers (except PORT command and function). For example:

```
CONST outpin$ = "GP27"
SETPIN outpin$, DOUT
PIN(outpin$) = 1
```

In the PicoMite firmware on-chip functions such as the SPI and I2C interfaces are not allocated to fixed pins, unlike (for example) the Micromite. The PicoMite firmware makes extensive use of the SETPIN command, not only to configure I/O pins but also to configure the pins used for interfaces such as serial, SPI, $I^2C$, etc.

Pins must be allocated according to this drawing. For example, the SPI TX can be allocated to pins GP3, GP7 or GP19 but it cannot be allocated to pin GP11 which can only be allocated to the SPI2 channel. Allocations don't have to be in the same "block" so you could, for example, allocate SPI2 TX to pin GP11 and SPI2 RX to pin GP28.

## Third Party Modules

Pins that are not exposed on the Raspberry Pi Pico can still be accessed using MMBasic via their GPn number. This allows MMBasic to be used on other modules that use the RP2040 or RP2350 processors.

On the Raspberry Pi Pico these hidden pins are used for internal functions as follows:

- GP23 is a digital output set to the value of OPTION POWER. (ON=PWM, OFF=PFM).
- GP24 is a digital input, which is high when VBUS is present.
- GP25 is also PWM4B. It is an output connected to the on-board LED.
- GP29 is also ADC3 which is an analog input reading ⅓ of VSYS.

However on third party modules that make them available they can be used as follows:

- GP23: DIGITAL_IN: DIGITAL_OUT, SPI TX, I2C2 SCL, PWM3B
- GP24: DIGITAL_IN: DIGITAL_OUT, SPI2 RX, COM2 TX, I2C SDA, PWM4A
- GP25: DIGITAL_IN: DIGITAL_OUT, COM2 RX, I2C SCL, PWM4B
- GP29: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, COM1 RX, I2C SCL, PWM6B

## WebMite version for the Raspberry Pi Pico W or Pico 2 W

The WebMite and Bluetooth versions (including PicoMiteHDMIWEB, PicoMiteHDMIBTH and PicoMitePC3) also use some GPIO pins for the wireless chip on the Pico W and Pico 2 W:

- GP29 (Input/Output) wireless SPI CLK/ADC mode (ADC3) measures VSYS/3
- GP25 SPI CS (Output) when high also enables GPIO29 ADC pin to read VSYS
- GP24 (Input/Output) wireless SPI data / IRQ
- GP23 (Output) wireless power-on signal

These pins are reserved for the wireless chip and cannot be used by a BASIC program.

Some RP2350 modules connect the wireless chip to different pins. On the RP2350 WiFi and Bluetooth versions (WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMIWEB, PicoMiteHDMIBTH and PicoMitePC3) the pins can be changed with OPTION CYW43 PINS, which lists the power-on, data, chip select and clock pins in that order. For example, the Waveshare RP2350B Pico WiFi board needs:

```
OPTION CYW43 PINS GP36, GP37, GP38, GP39
```

The pins given are then reserved instead of GP23, GP24, GP25 and GP29, which become normal I/O pins. If the wireless chip does not respond on the configured pins the firmware reports “CYW43 radio failed to start on …” when it starts and carries on without WiFi or Bluetooth (the WEB commands then report “WiFi radio not running”). OPTION CYW43 PINS DEFAULT (or OPTION RESET) returns to the standard pins. The RP2040 version of the WebMite always uses the standard pins.

Unlike the standard Raspberry Pi Pico, the on-board LED on the Pico W is not connected to a pin on the RP2040, but instead to a GPIO pin on the wireless chip and cannot be accessed from a BASIC program.

The antenna is on the PCB at the opposite end to the USB connector and should be kept in free space for best performance – don’t put metal under or close by the antenna.

## CPU Variations

The chips released by the Raspberry Pi Foundation for the Pico and Pico 2 are:

RP2040

This is packaged in a 60 pin package and is used in the original Raspberry Pi Pico and many other third party modules. The pinouts and functions of the I/O pins are as documented above.

RP2350A

This is also packaged in a 60 pin package and is used in the Raspberry Pi Pico 2 and third party modules. The pinouts and functions of the I/O pins are the same as the RP2040 and are documented above.

RP2350B

The RP2350B is in a 80 pin package with an additional 18 GPIO pins. The PicoMite firmware supports these additional pins including PWM channels 8-11 (allowing for a maximum of 24 simultaneous PWM outputs). Configuration for the RP2350B is done automatically making all the additional pins and three PIO channels available (the VGA version has two free PIO channels). Pin definitions for the RP2350B should use the GP nomenclature (ie, GP0 to GP47). Pins GP40 to GP47 can be used for analogue input, pins GP26-GP29 do not support analogue input.

Pin usage for pins GP0 to GP29 on the RP2350B are the same as that used for the RP2040 and RP2350A. Pins GP30 to GP47 can be used as follows:

GP30: DIGITAL_IN: DIGITAL_OUT, SPI2 SCK, I2C2 SDA, PWM7A
GP31: DIGITAL_IN: DIGITAL_OUT, SPI2 TX, I2C2 SCL, PWM7B
GP32: DIGITAL_IN: DIGITAL_OUT, COM1 TX, SPI RX, I2C SDA, EXT_PWM8A
GP33: DIGITAL_IN: DIGITAL_OUT, COM1 RX, I2C SCL, PWM8B
GP34: DIGITAL_IN: DIGITAL_OUT, SPI SCK, I2C2 SDA, PWM9A
GP35: DIGITAL_IN: DIGITAL_OUT, SPI TX, I2C2 SCL, PWM9B
GP36: DIGITAL_IN: DIGITAL_OUT, COM2 TX, SPI RX, I2C SDA, PWM10A
GP37: DIGITAL_IN: DIGITAL_OUT, COM2 RX, I2C SCL, PWM10B
GP38: DIGITAL_IN: DIGITAL_OUT, SPI SCK, I2C2 SDA, PWM11A
GP39: DIGITAL_IN: DIGITAL_OUT, SPI TX, I2C2 SCL, PWM11B
GP40: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, COM2 TX, SPI2 RX, I2C SDA, PWM8A
GP41: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, COM2 RX, I2C SCL, PWM8B
GP42: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, SPI2 SCK, I2C2 SDA, PWM9A
GP43: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, SPI2 TX, I2C2 SCL, PWM9B
GP44: DIGITAL_IN: DIGITAL_OUT, COM1TX, ANALOG_IN, SPI2 RX, I2C SDA, PWM10A
GP45: DIGITAL_IN: DIGITAL_OUT, COM1RX, ANALOG_IN, I2C SCL, PWM10B
GP46: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, SPI2 SCK, I2C2 SDA, PWM11A
GP47: DIGITAL_IN: DIGITAL_OUT, ANALOG_IN, SPI2 TX, I2C2 SCL, PWM11B

RP2354A and RP2354B

These are currently not supported by the PicoMite firmware.

## PSRAM

The RP2350 supports PSRAM and some commercial offerings have added 8MB of PSRAM to boards with the 80-pin RP2350B (eg, Pimoroni PGA2350).

The PSRAM is accessed via the same Quad SPI bus used by the flash memory so it is comparatively slow although it is buffered via a cache which mitigates that issue. If PSRAM is present and configured MMBasic will add it to the general purpose RAM pool so programs will have an enormous amount of general RAM to work with, even though it may be a little slower.

To access any PSRAM an additional pin is needed for the chip select function and this is selected using the OPTION PSRAM PIN command. Valid pins for the PSRAM chip select are GP0, GP8, GP19 and GP47.

After power up the contents of the PSRAM are indeterminate - they are not zero. You need to use RAM ERASE to clear it before use. This behaviour is deliberate, so that the contents will survive a reset.

## I/O Pin Limits

The maximum voltage that can be applied to any I/O pin on the Raspberry Pi Pico using the RP2040 processor is 3.6V. The Raspberry Pi Pico 2 using the RP2350 processor can accept 5V while the chip is powered.

As outputs all I/O pins can individually source or sink up to 8mA. At this load the output voltage will sag to about 2.3V. A more practical load is 5mA where the output voltage would typically be 3V. To drive a red LED at 5mA the recommended resistor is 220Ω. Other colours may require a different value.

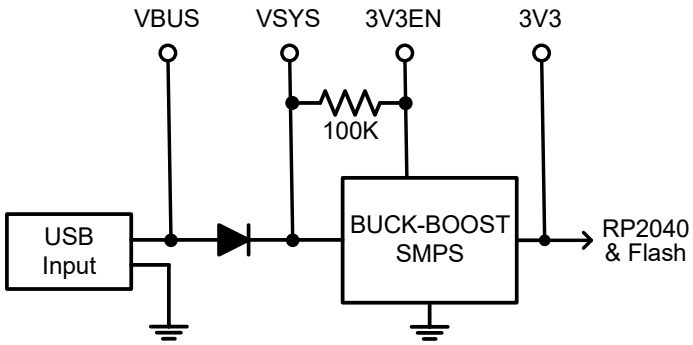
The maximum total I/O current load for the entire chip is 100mA.

## Power Supply

The Raspberry Pi Pico and Pico 2 has a flexible power system.

The input voltage from either the USB or VBUS inputs is connected through a Schottky diode to the buck-boost SMPS (Switch Mode Power Supply) which has an output of 3.3V. The SMPS will accommodate input voltages from 1.8V to 5.5V allowing the device to run from a wide range of power sources including batteries.

External circuitry can be powered by VBUS (normally 5V) or by the 3V3 (3.3V) output which can source up to 300mA.




For embedded controller applications generally an external power source (other than USB) is required and this can be connected to VSYS via a Schottky diode. This will allow the Raspberry Pi Pico to be powered by whichever supply is producing the highest voltage (USB or VSYS). The diodes will prevent feedback into the lower voltage supply.

To minimize power supply noise, it is possible to ground 3V3EN to turn off the SMPS. When shutdown the converter will stop switching, internal control circuitry will be turned off and the load disconnected. You can then power the board via a 3.3V linear regulator feeding into the 3V3 pin.

## Clock Speed

By default the clock speed for the RP2040 used in the Raspberry Pi Pico is 200MHz and for the RP2350 used in the Raspberry Pi Pico 2 it is also 200MHz. The chip manufacturer's recommended maximums are 200MHz for the RP2040 and 150MHz for the RP2350, so on the RP2350 the default speed is already a mild overclock.

Using the OPTION CPUSPEED command, most PicoMite CPUs can be overclocked well above the default and can also be run slower to save power. The permitted range depends on the version: the PicoMite versions accept 48MHz to 420MHz on both the RP2040 and the RP2350, the WebMite versions accept 126MHz to 396MHz, and the Bluetooth versions (PicoMiteRP2350BT and PicoMiteRP2350BTH) accept 200MHz to 396MHz. The command is not available on the VGA and HDMI versions, where the clock speed is set by the video resolution. This option is saved and will be reapplied on power up. When changing the clock speed the PicoMite firmware will be reset then rebooted so the USB connection will be disconnected.

The clock speed can also be changed at any time, without a reboot and without losing the USB connection, by using the CPU SPEED command (PicoMite and WebMite versions only). This change is not saved; on reset or power up the clock speed will return to that set by OPTION CPUSPEED.

VGA/HDMI versions with a video resolution of 640x480 will set the clock speed to 252MHz however this can be changed using OPTION RESOLUTION to 252MHz, 315MHz or 378MHz. At other video resolutions the clock speed is fixed at 252MHz, 283.2MHz, 333MHz, 336MHz, 360MHz, 372MHz or 375MHz depending on the video resolution selected and this cannot be changed. On the HDMI versions the resolution, and with it the clock speed, can be changed at run time without a reboot using the RESOLUTION command.

Nearly all tested Raspberry Pi Picos have worked correctly at 380MHz or more, so overclocking can be useful. If the processor fails to restart at its new clock speed you can reset it by loading Clear_flash.uf2 to reset the Pico to its factory fresh state (see the heading *Loading the Firmware* above).

## Power Consumption

The power consumption is dependent on the clock speed, however at the default clock speed (200MHz for both the RP2040 and the RP2350), the typical power consumption is 25mA. This does not include any current sourced or sunk by the I/O pins or the 3V3 pin:

The power consumption for the WebMite version for the Raspberry Pi Pico W is about 20mA with the WiFi disabled but, when the WiFi is enabled, the power consumption will rise to 40 to 70mA.

# Using MMBasic

## Commands and Program Input

At the command prompt you can enter a command and it will be immediately run. Most of the time you will do this to tell the PicoMite firmware to do something like run a program or set an option. But this feature also allows you to test out commands at the command prompt.

To enter a program the easiest method is to use the EDIT command. This will invoke the full screen program editor which is built into the PicoMite firmware and is described later in this manual. It includes advanced features such as search and copy, cut and paste to and from a clipboard.

You could also compose the program on your desktop computer using something like Notepad and then transfer it to the Raspberry Pi Pico via the XModem, YModem or ZModem protocols (see the XMODEM, YMODEM or ZMODEM command) or by streaming it up the console serial link (see the AUTOSAVE command).

A third and convenient method of writing and debugging a program is to use MMEdit. This is a program running on your Windows computer which allows you to edit your program on your computer then transfer it to the PicoMite with a single click of the mouse. MMEdit was written by Jim Hiley and is free to download and use. To learn more go to: https://geoffg.net/mmedit.html

One thing that you cannot do is use the old BASIC way of entering a program which was to prefix each line with a line number. Line numbers are optional in MMBasic so you can still use them if you wish but if you enter a line with a line number at the prompt MMBasic will simply execute it immediately.

## Program Structure

A BASIC program starts at the first line and continues until it runs off the end of the program or hits an END command - at which point MMBasic will display the command prompt (>) on the console and wait for something to be entered.

A program consists of a number of statements or commands, each of which will cause the BASIC interpreter to do something (the words statement and command generally mean the same and are used interchangeably). Normally each statement is on its own line but you can have multiple statements in the one line separated by the colon character (:). For example.

```
A = 24.6 : PRINT A
```

Appendix J (*Programming in BASIC - A Tutorial*) at the rear of this manual provides a comprehensive tutorial on the language which will take you through the fundamentals in an easy to read format with lots of examples.

## Editing the Command Line

When entering a line at the command prompt the line can be edited using the left and right arrow keys to move along the line, the Delete key to delete a character, the Backspace to delete before the cursor and the Insert key to switch between insert and overwrite modes. Home/End will move to the beginning/end of the line and Home pressed twice will terminate the edit. At any point the Enter key will send the line to MMBasic which will execute it.

The up and down arrow keys will move through a history of previously entered command lines which can be edited and reused.

## Shortcut Keys

The function keys on the keyboard used for the console can be used at the command prompt to automatically enter common commands. These function keys will insert the text followed by the Enter key so that the command is immediately executed:

| | |
|---|---|
| F2 | RUN |
| F3 | LIST |
| F4 | EDIT |
| F5 | Sends ESC sequence to clear the VT100 screen. Also clears the console. |
| F10 | AUTOSAVE |
| F11 | XMODEM RECEIVE |
| F12 | XMODEM SEND |

Function keys F1, and F5 to F9 can be programmed with custom text. See the OPTION FNKey command.

## Interrupting A Running Program

A program is set running by the RUN command. You can interrupt MMBasic and the running program at any time by typing CTRL-C on the console input and MMBasic will return to the command prompt.

## Setting Options

Many options can be set by using commands that start with the keyword OPTION. They are listed in their own section of this manual. For example, on some versions of the firmware you can change the CPU clock speed with the command:

```
OPTION CPUSPEED speed
```

## Saved Variables

Often there is a need to save data that can be recovered when power is restored. For example, program options, calibration settings, etc. This can be done with the VAR SAVE command which will save the variables listed on its command line in non-volatile flash memory. The space reserved for saved variables is 16KB.

These variables can be restored with the VAR RESTORE command which will add all the saved variables to the variable table of the running program. Normally this command is placed near the start of a program so that the variables are ready for use by the program.

This facility is intended for saving calibration data, user selected options and other items which change infrequently. It should not be used for high-speed saves as you may wear out the flash memory. The flash used for the Raspberry Pi Pico has a high endurance but this can be exceeded by a program that repeatedly saves variables. If you do want to save data often you should add a real time clock chip. The RTC commands can then be used to store and retrieve data from the RTC's battery backed memory. See the RTC command for more details.

Saved variables are cleared when a new program is loaded.

## Watchdog Timer

One of the uses for the Raspberry Pi Pico is as an embedded controller. It can be programmed in MMBasic and when the program is debugged and ready for "prime time" the OPTION AUTORUN configuration setting can be turned on. The module will then automatically run its program when power is applied and act as a custom circuit performing some special task. The user need not know anything about what is running inside it.

However, there is the possibility that a fault in the program could cause MMBasic to generate an error and return to the command prompt. This would be of little use in an embedded situation as the device would not have anything connected to the console. Another possibility is that the BASIC program could get itself stuck in an endless loop for some reason. In both cases the visible effect would be the same… the program would stop running until the power was cycled.

To guard against this the watchdog timer can be used. This is a timer that counts down to zero and when it reaches zero the processors will be automatically restarted (the same as when power was first applied), this will occur even if MMBasic was sitting at the command prompt. Following the restart the automatic variable MM.WATCHDOG will be set to true to indicate that the restart was caused by a watchdog timeout.

The WATCHDOG command should be placed in strategic locations in the program to keep resetting the timer and therefore preventing it from counting down to zero. Then, if a fault occurs, the timer will not be reset, it will count down to zero and the program will be restarted (assuming the AUTORUN option is set).

## PIN Security

Sometimes it is important to keep the data and program in an embedded controller confidential. In the PicoMite firmware this can be done by using the OPTION PIN command. This command will set a pin number (which is stored in flash) and whenever the PicoMite firmware returns to the command prompt (for whatever reason) the user at the console will be prompted to enter the PIN number. Without the correct PIN the user cannot get to the command prompt and their only option is to enter the correct PIN or reboot the PicoMite firmware. When it is rebooted the user will still need the correct PIN to access the command prompt.

Because an intruder cannot reach the command prompt they cannot list or copy a program, they cannot change the program or change any aspect of MMBasic or the PicoMite firmware. Once set the PIN can only be removed by providing the correct PIN as set in the first place. If the number is lost the only method of recovery is to reload the PicoMite firmware (which will erase the program and all options).

There are other time consuming ways of accessing the data (such as using a programmer to examine the flash memory) so this should not be regarded as the ultimate security but it does act as a significant deterrent.

## The Library

Using the LIBRARY feature it is possible to create BASIC functions, subroutines and embedded fonts and add them to MMBasic to make them permanent and part of the language. For example, you might have written a series of subroutines and functions that perform sophisticated bit manipulation; these could be stored as a library and become part of MMBasic and perform the same as other built-in functions that are already part of the language. An embedded font can also be added the same way and used just like a normal font.

To install components into the library you need to write and test the routines as you would with any normal BASIC routines. When they are working correctly you can use the LIBRARY SAVE command. This will transfer the routines (as many as you like) to a non-visible part of flash memory where they will be available to any BASIC program but will not show when the LIST command is used and will not be deleted when a new program is loaded or NEW is used. However, the saved subroutines and functions can be called from within the main program and can even be run at the command prompt (just like a built-in command or function).

Some points to note:

- Library routines act exactly like normal BASIC code and can consist of any number of subroutines, functions, embedded C routines and fonts. The only difference is that they do not show when a program is listed and are not deleted when a new program is loaded.
- Library routines can create and access global variables and are subject to the same rules as the main program – for example, respecting OPTION EXPLICIT if it is set.
- When the routines are transferred to the library MMBasic will compress them by removing comments, extra spaces, blank lines and the hex codes in embedded C routines and fonts. This makes the library space efficient, especially when loading large fonts. Following the save the program area is cleared.
- You can use the LIBRARY SAVE command multiple times. With each save the new contents of the program space are appended to the already existing code in the library.
- You can use line numbers in the library but you cannot use a line number on an otherwise empty line as the target for a GOTO, etc. This is because the LIBRARY SAVE command will remove any blank lines.
- You can use READ commands in the library. DATA statements are searched as a single sequence: the main program first and then the library, so when the program's DATA statements are exhausted (or the program has none) reading continues with the DATA statements in the library. To read the library's DATA from the beginning use the RESTORE command before the first READ; used within library code this resets the pointer to the library space.
- Labels belong to the library and the program together: a label must not be defined in both, and the program must not have a subroutine or function with the name of one in the library. Either is reported when the program is run (Duplicate label or Duplicate name).
- The library is saved to program flash memory Slot 3 and this will not be available for storing a program if LIBRARY SAVE is used.
- You can see what is in the library by using the LIBRARY LIST command which will list the contents of the library space.
- The LIBRARY contents can be saved to disk using LIBRARY DISK SAVE fname$ and restored using LIBRARY DISK LOAD fname$

To delete the routines in the library space you use the LIBRARY DELETE command. This will clear the space and return the Flash Slot 3 used by the library back to being available for storage for normal programs. The only other way to delete a library is to use OPTION RESET.

A program can install its own library with LIBRARY LOAD fname$, which reads a BASIC source file and makes it the library. This is intended for programs that are distributed with a library of their own: the library becomes part of the program rather than something that must be installed first. The command must be the first statement in the program and the program is restarted once the library has been installed, so that the library's initialisation code runs before the program proper. Because the command remembers a hash of the source file it will do nothing at all when the library already matches, so it is safe to leave at the top of a program permanently.

## Program Initialisation

The library can also include code that is not contained within a subroutine or function. This code (if it exists) will be run automatically before a program starts running (ie, via the RUN command). This feature can be used

to initialise constants or setup MMBasic in some way. For example, if you wanted to set some constants you could include the following lines in the library code:

```
CONST TRUE = 1
CONST FALSE = 0
```

For all intents and purposes, the identifiers TRUE and FALSE have been added to the language and will be available to any program that is run.

## MM.STARTUP

There may be a need to execute some code on initial power up, perhaps to initialise some hardware, set some options or print a custom start-up banner. This can be accomplished by creating a subroutine with the name MM.STARTUP. When the PicoMite firmware is first powered up or reset it will search for this subroutine and, if found, it will be run once.

For example, if the Raspberry Pi Pico has a real time clock attached, the program could contain the following code:

```
SUB MM.STARTUP
  RTC GETTIME
END SUB
```

This would cause the internal clock within MMBasic to be set to the current time on every power up or reset.

After the code in MM.STARTUP has been run MMBasic will continue with running the rest of the program in program memory. If there is no other code MMBasic will return to the command prompt.

Note that you should not use MM.STARTUP for general setup of MMBasic (like dimensioning arrays, opening communication channels, etc) before running a program. The reason is that when you use the RUN command MMBasic will first clear the interpreter's state ready for a fresh start.

## MM.PROMPT

If a subroutine with this name exists it will be automatically executed by MMBasic instead of displaying the command prompt. This can be used to display a custom prompt, set colours, define variables, etc all of which will be active at the command prompt.

Note that MMBasic will clear all variables and I/O pin settings when a program is run so anything set in this subroutine will only be valid for commands typed at the command prompt (i.e. in immediate mode).

As an example the following will display a custom prompt:

```
SUB MM.PROMPT
  PRINT TIME$ "> ";
END SUB
```

Note that while constants can be defined, they will not be visible because a constant defined inside a subroutine is local to a subroutine. However, DIM will create variables that are global that that should be used instead.

## MM.END

If a subroutine named MM.END exists in the program it will be executed whenever the program ends with an actual or implied END command. It is not executed if the program ends with a Ctrl-C.

The optional parameter ‘noend’ to the END command can be used to block execution of the MM.END subroutine if needed (see the END command for more information),

# Calculator Immediate Mode

The command prompt accepts bare expressions and evaluates them immediately, in the same way you would use a desktop calculator.

The result is printed and also stored in the variable MM.ANSWER, which can be referenced by name in any subsequent expression.

Calculator mode only activates at the interactive prompt. Tokenisation of program files loaded with RUN, EDIT or LOAD is unaffected, so existing programs run identically whether CALCPROMPT is enabled or not.

Three forms of input are recognised:

**1. Expressions starting with a digit, decimal point or open parenthesis**

```
> 4 + 5
 9
> sqr(2)
 1.414213562
> (3 + 5) * 2
 16
> .5 * 100
 50
```

**2. Expressions starting with an identifier followed by an operator**

A variable, named constant or built-in function at the start of the line, followed by an arithmetic, relational or logical operator (symbolic or text), is also evaluated:

```
> x + 3
> pi * 2
> a$ + b$
> x AND y
> x MOD 5
```

A name on its own (not followed by anything) is treated as an unknown command, not as a request to print the variable's value — use ? x for that.

**3. Expressions starting with an operator (implicit MM.ANSWER)**

When the line begins with one of the symbolic operators + - * / \ ^ < >, MMBasic implicitly inserts MM.ANSWER as the left operand. This lets you chain calculations the way you would on a pocket calculator:

```
> 5
 5
> +10
 15
> *3
 45
> ^2
 2025
> -1000
 1025
```

**MM.ANSWER**

MM.ANSWER is a global float variable created on first use. It is set to the most recent numeric or integer result. String results are printed but do not update MM.ANSWER. The variable can be read or written like any other variable; it survives across CALC invocations within a session and is cleared by NEW, RUN or other commands that reset the variable pool.

**Interaction with the *<filename> RUN shortcut**

The * character at the start of a line is also the shorthand for RUN "filename". Calculator mode resolves the ambiguity by checking whether the file exists before deciding:

- If a file matching the name (or the name with .bas appended) exists, *name runs that file as before.

- If no file is found and the rest of the line looks like a calculator expression (contains a digit, dot, parenthesis, or operator), it is evaluated as MM.ANSWER * <expression>.
- If no file is found and the input is just a bare name (e.g. *foo, *sqrt2), MMBasic reports Neither file nor function found rather than letting the expression fail with a misleading "is not declared" message.

This means filenames consisting purely of digits (*32 where 32.bas exists) still work as RUN shortcuts when the file is present.

**Examples**

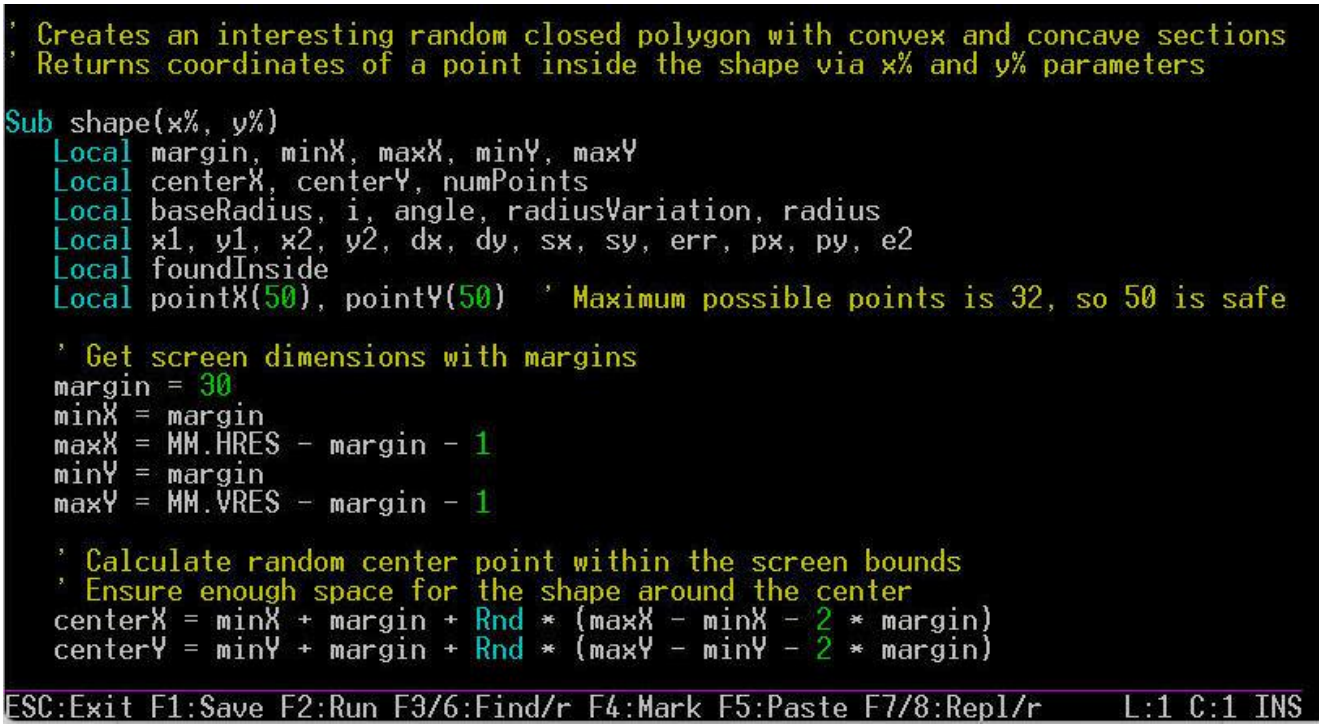
```
> 2 ^ 10
 1024
> *2              ' MM.ANSWER * 2
 2048
> sqr(MM.ANSWER)     ' explicit reference
 45.25483400
> a = MM.ANSWER * 100
> ? a
 4525.483400
```

**Limitations**

- A unary minus or plus at the start of a line is interpreted as MM.ANSWER - n or MM.ANSWER + n. To enter a standalone negative number, type (-5) or 0-5.
- A bare identifier with no operator is reported as an unknown command. To inspect a variable, use ? variableName.
- Text operators (AND, OR, MOD, etc.) work in the middle of an expression but cannot start a line in the implicit-MM.ANSWER form — only symbolic operators trigger that path.
- String results are printed but not stored in MM.ANSWER (which is numeric).
- With OPTION EXPLICIT set, MM.ANSWER is created automatically the first time a CALC expression produces a numeric result, so subsequent operator-led expressions can reference it without needing an explicit DIM.

# Full Screen Editor

An important productivity feature is the built-in full screen editor. When running it looks like this:




The editor operates with:

- The serial console using a VT100 supported terminal emulator (such as Tera Term) on all versions.
- The VGA or HDMI video output (on versions with this feature).
- An attached LCD panel that has been configured with OPTION LCDPANEL CONSOLE.

The editor is a very productive method of writing a program. With the command EDIT you can enter and edit your program then, by pressing the F2 key, you can save and run the program. If your program stops with an error pressing the function key F4 at the command prompt will load the editor and position the cursor at the line that caused the error. This edit/run/edit cycle is very fast.

If you have previously used an editor like Windows Notepad you will find that the operation of this editor is familiar. The arrow keys will move the cursor around in the text, home and end will take you to the beginning or end of the line. Page up and page down will do what their titles suggest. The delete key will delete the character at the cursor and backspace will delete the character before the cursor. The insert key will toggle between insert and overtype modes. About the only unusual key combination is that two home key presses will take you to the start of the program and two end key presses will take you to the end.

The best way to learn how to use the editor is to simply fire it up and experiment.

## Edit Functions

When the editor starts the cursor will be positioned at the last editing position or, if your program had been stopped by an error, the line that caused the error. At the bottom of the screen the status line will list details such as the current cursor position and the common functions supported by the editor.

In more details these functions are:

| | |
|---|---|
| ESC | This will cause the editor to abandon all changes and return to the command prompt with the program memory unchanged. If you have changed the text, you will be asked if you really want to abandon your changes. |
| F1: SAVE | This will save to program memory and return to the command prompt. |
| F2: RUN | This will save to program memory and immediately run it. |
| F3: FIND | This will prompt for the text that you want to search for. When you press enter the cursor will be placed at the start of the first entry found. |

| | |
|---|---|
| F4: MARK | This is described in detail below. |
| F5: PASTE | This will insert (at the current cursor position) the text that had been previously cut or copied (see below). |
| F6: FIND AGAIN | Once you have used the search function you can repeat the search by pressing F6 (shift-F3 also valid). |
| F7: REPLACE | If the cursor is positioned after F3 or F6, this will open a dialog allowing you to directly enter a string into the paste buffer and on pressing Enter, replace the search target string |
| F8: REPLACE AGAIN | If the cursor is positioned after F3 or F6 the system will replace the search target string with the contents of the paste buffer. |
| F9: INSERT | You will be prompted for a file name and when Enter is pressed that file will be inserted at the current cursor position. |
| F10: SAVE CLIPBOARD | You will be prompted for a file name and when Enter is pressed the contents of the clipboard (if any) will be saved to that file. See the mark mode below. |
| F12: BEAUTIFY | This will re-indent the program using a consistent two space indent for the body of multiline structures such as IF…END IF, FOR…NEXT, DO…LOOP, SELECT CASE…END SELECT, SUB…END SUB and FUNCTION…END FUNCTION. Single line IF statements are not changed, comment lines are indented at the current level and in programs that use line numbers the statements are aligned in a column after the longest line number. A single blank line is always placed after each END SUB and END FUNCTION. If the program contains any other blank lines you will be asked "KEEP BLANK LINES? (Y/N)" - answer Y (or press Enter) to keep them, N to remove them or press ESC to cancel. When finished the cursor will be placed at the start of the program. Ctrl-A can also be used to start the beautify. Not available on the PicoMiteMIN. |

## Mark Mode

Pressing the mark key (F4) will put the editor into mark mode which will allow you to cut or copy text to the clipboard or delete or save text.

When in the mark mode the arrow keys are used to mark the text which will be highlighted in reverse video. If you reach the beginning or end of the screen the arrow keys will continue to mark text as the screen scrolls. This also works with page up and page down.

In the mark mode the status line will change to show the functions available in this mode.
These are:

| | |
|---|---|
| ESC | Exit mark mode without changing anything. |
| F4: CUT | Copy the marked text to the clipboard and remove it from the program. |
| F5: COPY | Just copy the marked text to the clipboard. |
| F10: SAVE | Prompt for a filename and when Enter is pressed, save the marked text to the file. |
| DELETE | Delete the marked text leaving the clipboard unchanged. |

Cut or copy is limited to a maximum of 2048 characters.

## Alternate Keys

You can also use control keys instead of the function keys listed above. These control keystrokes are:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| LEFT | Ctrl-S | RIGHT | Ctrl-D | UP | Ctrl-E | DOWN | Ctrl-X |
| HOME | Ctrl-U | END | Ctrl-K | PageUp | Ctrl-P | PageDn | Ctrl-L |
| DEL | Ctrl-] | INSERT | Ctrl-N | F1 | Ctrl-Q | F2 | Ctrl-W |
| F3 | Ctrl-R | F4 | Ctrl-T | F5 | Ctrl-Y | F6 | Ctrl-G |
| F7 | Ctrl-F | F8 | Ctrl-I | F12 | Ctrl-A | | |

## Long Lines

Long lines are handled by horizontal scrolling. A line that is longer than the display width will initially show just its first part, however, when the cursor is moved beyond the right hand margin, the line that the cursor is on will scroll horizontally so that any part of it can be seen and edited. The other lines on the screen are not affected and continue to show their first part. The maximum length of a line is 255 characters.

Continuation lines are also supported but are deprecated - with horizontal scrolling they are no longer needed and they are retained only for compatibility with existing programs. If enabled (OPTION CONTINUATION LINES ON) a space followed by an underscore character at the end of a line indicates that the next line is a continuation and should be concatenated in order for the program to run. The concatenation occurs when the file is saved and, when re-edited, long lines are automatically split up as the file is read into the editor (the line breaks may not be in the same places). The limit of a maximum of 255 characters in a concatenated line still applies.

## Using a Mouse

Versions of the PicoMite firmware that support VGA/HDMI (in both the RP2040 and RP2350 versions) also supports the use of a PS2 or USB mouse in the editor. For the details of connecting a mouse see the heading *Keyboard/Mouse/Gamepad* later in this manual.

If you start the editor with a mouse plugged in and are in video MODE 1 with colour coding enabled you will see a character highlighted with red on a white background. This highlight can be moved using the mouse. Left clicking on the mouse will move the edit cursor to that position (ie, the same as using the cursor keys). Right clicking the mouse is the same as pressing F4 on the keyboard and clicking the scroll wheel is the same as F5.

This means that in the editor's normal mode you can position the mouse cursor and by right clicking the editor will enter mark mode (cut-and-copy) with the cursor starting where the mouse cursor was. Then moving the mouse and then left clicking will highlight the characters from the mark position to the new mouse position. Right clicking (same as F4) will cut the highlighted region to the clipboard while clicking the scroll wheel (same as F5) will copy the highlighted text to the clipboard without deleting it from the text. Both will return the editor to normal mode.

In normal mode the contents of the clipboard can be inserted into the text by moving the mouse to the new position and clicking the scroll wheel (same as F5).

## Colour Coded Editor Display

The editor can colour code the edited program with keywords, numbers and comments displayed in different colours. This feature can be turned on or off at the command prompt with the commands:

```
OPTION COLOURCODE ON
```

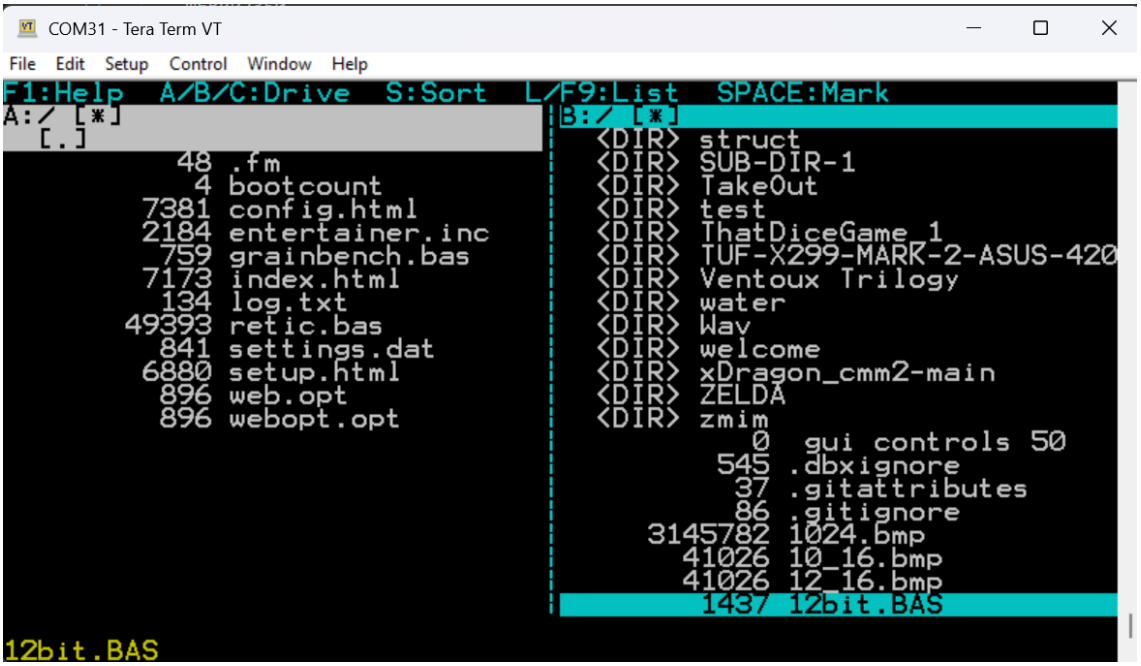
or

```
OPTION COLOURCODE OFF
```

This option is saved in non-volatile memory and will be automatically applied on start-up.

# File Manager

The FM command (available on all versions except the PicoMiteMIN) opens a full screen, two panel file manager in the style of the classic Norton Commander. It works on the serial console using a VT100 terminal emulator, on the VGA/HDMI display and on an LCD panel configured as the console. The display must be at least 50 characters wide by 8 lines; on a narrower display the file manager will try to select a suitable screen mode or font for the session and will restore the original on exit.




Each panel shows the contents of one directory. Pressing A, B or C switches the active panel between the drives: A: is the internal flash filesystem, B: is the SD card and C: is a USB flash drive (RP2350 versions with USB support). The panel header shows the drive, the current path and the active filter, and the highlighted panel is the active one - all file operations apply to it. Files can be marked with the space bar (shown with a leading *) so that copy and delete operations work on several files at once, and each panel can have a wildcard filter (eg, *.BAS) and its own sort order. The status line at the bottom shows the result of the last operation.

Pressing ENTER on a directory opens it; on a .BAS file it saves the file manager's state and runs the program (the file manager restarts automatically with its state restored when the program ends); on an audio file (WAV, FLAC, MP3, MID or MOD, subject to the same hardware requirements as the PLAY command) it plays the file in the background while the file manager remains usable; and on an image file (BMP, JPG or PNG) it displays a full screen preview on a graphical display.

Marked audio files play as a track list. Mark them with the space bar and press ENTER: when every marked item in the panel is an audio file, the file manager plays the marked files in panel order, as the PLAY WAV, FLAC, MP3 and MIDI commands play a directory, with each track starting when the one before it ends. A list holds up to 20 tracks and may mix WAV, FLAC, MP3 and MID files; MOD files cannot be in a list. The status line shows the track that is playing (for example "Playing 2/5 name") and reports when the list has finished. The . or > key skips to the next track and , or < returns to the previous one, as PLAY NEXT and PLAY PREVIOUS do, and F6 or T stops the list. With any other marks (a directory, a program, an image) ENTER acts on the selected item as usual.

Pressing F4 opens the selected file in the full screen editor and exiting the editor returns to the file manager. If a program run from the file manager stops with an error, pressing F4 on that file opens the editor with the cursor on the line that caused the error. Pressing F2 within the editor saves and runs the program as normal.

The file manager remembers its state (paths, selection, sort order and filters) between invocations. The full set of keys is (press F1 inside the file manager for a quick reminder):

| **Action** | **Key(s)** |
|---|---|
| Exit the file manager | ESC |
| Help screen | F1, H or ? |

| | |
|---|---|
| Switch the active panel | TAB, LEFT or RIGHT (Ctrl-S = left, Ctrl-D = right) |
| Move the selection up / down | UP, DOWN (Ctrl-E, Ctrl-X) |
| Page up / page down | PgUp, PgDn (Ctrl-P, Ctrl-L) |
| Top of the drive / end of the list | HOME, END (Ctrl-U, Ctrl-K) |
| Go to the parent directory | Backspace |
| Open / run the selected item | ENTER or F2 |
| Change the drive | A, B or C |
| Go to a path | G |
| Cycle the sort mode (name / datetime / type) | S |
| Type-select by filename prefix | / then type (ENTER opens, ESC cancels) |
| Set / clear the file filter | F9 (F) / F10 (W) |
| Mark or unmark the current item | SPACE |
| Mark all / clear all marks | * / \ |
| Play the marked audio files as a track list | ENTER or F2 (when every marked item is an audio file) |
| Copy the selected or marked items to the other panel | F5 or Y |
| Move the selected item to the other panel | M |
| Delete the selected or marked items | DEL |
| Recursive delete (including directories) | X |
| Rename the selected item | R |
| Duplicate the selected item | D |
| New file (create and edit) | N |
| Make a directory | F11 or K |
| Edit the selected file | F4 or E |
| List the selected file (paged) | F3 or L |
| Stop audio playback | F6 or T |
| Volume down / volume up | F7 or - / F8 or + |
| Previous / next track of a track list | , or < / . or > |

Notes: a file copy overwrites the destination without asking and a directory copy is recursive and works across drives. DEL only deletes empty directories - use X (which asks for two confirmations) to remove a directory and its contents. M moves within the same drive only; to move between drives use copy (F5) then delete (DEL). CTRL-C is ignored while the file manager is active, except in the LIST viewer where it exits the listing.

FM starts by clearing the variables and the other memory left by the last program to run, as EDIT does, so the file manager and its editor always have the memory they need.

# Variables and Expressions

In MMBasic command names, function names, labels, variable names, file names, etc are not case sensitive, so that "Run" and "RUN" are equivalent and "dOO" and "Doo" refer to the same variable.

## Variables

Variables can start with an alphabetic character or underscore and can contain any alphabetic or numeric character, the period (.) and the underscore (_). They may be up to 31 characters long.

A variable name or a label must not be the same as a command, function, one of the nine PIO instructions (IN, OUT, JMP, WAIT, PUSH, PULL, MOV, IRQ, SET) or one of the following keywords: THEN, ELSE, GOTO, GOSUB, TO, STEP, FOR, WHILE, UNTIL, LOAD, MOD, NOT, AND, OR, XOR, AS.
Eg, step = 5 is illegal as STEP is a keyword.

A label must be unique: the same label defined twice, in the program or in both the program and the library, is reported as Duplicate label when the program is run.

MMBasic supports three types of variables:

1. Double Precision Floating Point.
   These can store a number with a decimal point and fraction (eg, 45.386) however they will lose accuracy when more than 14 digits of precision are used. Floating point variables are specified by adding the suffix '!' to a variable's name (eg, i!, nbr!, etc). They are also the default when a variable is created without a suffix (eg, i, nbr, etc).
2. 64-bit Signed Integer.
   These can store positive or negative numbers with up to 19 decimal digits without losing accuracy but they cannot store fractions (i.e. the part following the decimal point). These are specified by adding the suffix '%' to a variable's name. For example, i%, nbr%, etc.
3. A String.
   A string will store a sequence of characters (eg, "Tom"). Each character in the string is stored as an eight-bit number and can therefore have a decimal value of 0 to 255. String variable names are terminated with a '$' symbol (eg, name$, s$, etc). Strings can be up to 255 characters long.

Note that it is illegal to use the same variable name with different types. Eg, using `nbr!` and `nbr%` in the same program would cause an error.

Most programs use floating point variables for arithmetic as these can deal with the numbers used in typical situations and are more intuitive than integers when dealing with division and fractions. So, if you are not bothered with the details, always use floating point.

## Constants

Numeric constants may begin with a numeric digit (0-9) for a decimal constant, &H for a hexadecimal constant, &O for an octal constant or &B for a binary constant. For example &B1000 is the same as the decimal constant 8. Constants that start with &H, &O or &B are always treated as 64-bit unsigned integer constants.

Decimal constants may be preceded with a minus (-) or plus (+) and may be terminated with 'E' followed by an exponent number to denote exponential notation. For example, 1.6E+4 is the same as 16000.

When a constant number is used it will be assumed that it is an integer if a decimal point or exponent is not used. For example, 1234 will be interpreted as an integer while 1234.0 will be interpreted as a floating-point number.

String constants must be surrounded by double quote marks ("). Eg, "Hello World".

## OPTION DEFAULT

A variable can be used without a suffix (i.e. !, % or $) and in that case MMBasic will use the default type of floating point. For example, the following will create a floating-point variable:

```
Nbr = 1234
```

However. the default can be changed with the OPTION DEFAULT command. For example, OPTION DEFAULT INTEGER will specify that all variables without a specific type will be integer. So, the following will create an integer variable:

```
OPTION DEFAULT INTEGER
```

```
Nbr = 1234
```

The default can be set to FLOAT (which is the default when a program is run), INTEGER, STRING or NONE. In the latter all variables must be specifically typed otherwise an error will occur.

The OPTION DEFAULT command can be placed anywhere in the program and changed at any time but good practice dictates that if it is used it should be placed at the start of the program and left unchanged.

## OPTION EXPLICIT

By default MMBasic will automatically create a variable when it is first referenced. So, `Nbr = 1234` will create the variable and set it to the number 1234 at the same time. This is convenient for short and quick programs but it can lead to subtle and difficult to find bugs in large programs. For example, in the third line of this fragment the variable `Nbr` has been misspelt as `Nbrs`. As a consequence, the variable `Nbrs` would be created with a value of zero and the value of `Total` would be wrong.

```
Nbr = 1234
Incr = 2
Total = Nbrs + Incr
```

The OPTION EXPLICIT command tells MMBasic to not automatically create variables. Instead, they must be explicitly defined using the DIM, LOCAL or STATIC commands (see below) before they are used. The use of this command is recommended to support good programming practice. If it is used it should be placed at the start of the program before any variables are used.

## DIM and LOCAL

The DIM and LOCAL commands can be used to define a variable and set its type and are mandatory when the OPTION EXPLICIT command is used.

The DIM command will create a global variable that can be seen and used throughout the program including inside subroutines and functions. However, if you require the definition to be visible only within a subroutine or function, you should use the LOCAL command at the start of the subroutine or function. LOCAL has exactly the same syntax as DIM.

If LOCAL is used to specify a variable with the same name as a global variable, then the global variable will be hidden to the subroutine or function and any references to the variable will only refer to the variable defined by the LOCAL command. Any variable created by LOCAL will vanish when the program leaves the subroutine.

At its simplest level DIM and LOCAL can be used to define one or more variables based on their type suffix or the OPTION DEFAULT in force at the time. For example:

```
DIM nbr%, s$
```

But it can also be used to define one or more variables with a specific type when the type suffix is not used:

```
DIM INTEGER nbr, nbr2, nbr3, etc
```

In this case nbr, nbr2, nbr3, etc are all created as integers. When you use the variable within a program you do not need to specify the type suffix. For example, `MyStr` in the following works perfectly as a string variable:

```
DIM STRING MyStr
MyStr = "Hello"
```

The DIM and LOCAL commands will also accept the Microsoft practice of specifying the variable's type after the variable with the keyword "AS". For example:

```
DIM nbr AS INTEGER, s AS STRING
```

In this case the type of each variable is set individually (not as a group as when the type is placed before the list of variables).

The variables can also be initialised while being defined. For example:

```
DIM INTEGER a = 5, b = 4, c = 3
DIM s$ = "World", i% = &H8FF8F
DIM msg AS STRING = "Hello" + " " + s$
```

The value used to initialise the variable can be an expression including user defined functions.

The DIM or LOCAL commands are also used to define an array and all the rules listed above apply when defining an array. For example, you can use:

```
DIM INTEGER nbr(10), nbr2, nbr3(5,8)
```

When initialising an array the values are listed as comma separated values with the whole list surrounded by brackets. For example:

```
DIM INTEGER nbr(5) = (11, 12, 13, 14, 15, 16)
DIM days(7) AS STRING = ("","Sun","Mon","Tue","Wed","Thu","Fri","Sat")
```

Once created, a complete array can be copied into another array in a single assignment by using the array names with empty brackets. For example:

```
b%() = a%()
```

Both arrays must be the same type (for strings, also the same maximum length) and must hold the same total number of elements. The dimensions may differ so long as the totals match; for example, a 2 x 3 array can be copied into a six element array.

## STATIC

Inside a subroutine or function it is sometimes useful to create a variable which is only visible within the subroutine or function (like a LOCAL variable) but retains its value between calls to the subroutine or function.

You can do this by using the STATIC command. STATIC can only be used inside a subroutine or function and uses the same syntax as LOCAL and DIM. The difference is that its value will be retained between calls to the subroutine or function (i.e. it will not be initialised on the second and subsequent calls).

For example, if you had the following subroutine and repeatedly called it, the first call would print 5, the second 6, the third 7 and so on.

```
SUB Foo
  STATIC var = 5
  PRINT var
  var = var + 1
END SUB
```

Note that the initialisation of the static variable to 5 (as in the above example) will only take effect on the first call to the subroutine. On subsequent calls the initialisation will be ignored as the variable had already been created on the first call.

As with DIM and LOCAL the variables created with STATIC can be float, integers or strings and arrays of these with or without initialisation. The length of the variable name created by STATIC and the length of the subroutine or function name added together cannot exceed 31 characters.

## CONST

Often it is useful to define an identifier that represents a value without the risk of the value being accidently changed - which can happen if variables were used for this purpose (this practice encourages another class of difficult to find bugs).

Using the CONST command you can create an identifier that acts like a variable but is set to a value that cannot be changed. For example:

```
CONST InputVoltagePin = 31
CONST MaxValue = 2.4
```

The identifiers can then be used in a program where they make more sense to the casual reader than simple numbers. For example:

```
IF PIN(InputVoltagePin) > MaxValue THEN SoundAlarm
```

A number of constants can be created on the one line:

```
CONST InputVoltagePin = 31, MaxValue = 2.4, MinValue = 1.5
```

The value used to initialise the constant is evaluated when the constant is created and can be an expression including user defined functions.

The type of the constant is derived from the value assigned to it; so for example, `MaxValue` above will be a floating point constant because 2.4 is a floating point number. The type of a constant can also be explicitly set by using a type suffix (i.e. !, % or $) but it must agree with its assigned value.

A constant defined outside of any subroutine or function is global and can be used throughout the program. A constant defined inside a subroutine or function is local to that routine - it is created when the routine is called, disappears when the routine exits and will hide a global constant of the same name.

## Special Characters in Strings

Special, non-printable characters can be inserted in string constants using the backslash (ie, \) as an escape symbol. To enable this facility the command OPTION ESCAPE must be placed at the start of the program.

These are the valid escape sequences:

| | Hex Value | Description |
|---|---|---|
| \a | 07 | Alert (Beep, Bell) |
| \b | 08 | Backspace |
| \e | 1B | Escape character |
| \f | 0C | Formfeed Page Break |
| \n | 0A | Newline (Line Feed) |
| \r | 0D | Carriage Return |
| \q | 22 | Quote symbol |
| \t | 09 | Horizontal Tab |
| \v | 0B | Vertical Tab |
| \\ | 5C | Backslash |
| \nnn | any | The byte whose value is given by nnn interpreted as a decimal number |
| \&hh | any | The byte whose value is given by hh… interpreted as a hexadecimal number |

For example, the following will print the words Hello and World on separate lines:

```
OPTION ESCAPE
PRINT "Hello\r\nWorld"
```

## Expressions and Operators

MMBasic will evaluate a mathematical expression using the standard mathematical rules. For example, multiplication and division are performed first followed by addition and subtraction. These are called the rules of precedence and are detailed below.

This means that 2 + 3 * 6 will resolve to 20, so will 5 * 4 and also 10 + 4 * 3 – 2.

If you want to force the interpreter to evaluate parts of the expression first you can surround that part of the expression with brackets. For example, (10 + 4) * (3 – 2) will resolve to 14 not 20 as would have been the case if the brackets were not used. Using brackets does not appreciably slow down the program so you should use them liberally if there is a chance that MMBasic will misinterpret your intention.

The following operators, in order of precedence, are implemented in MMBasic. Operators that are on the same level (for example + and -) are processed with a left to right precedence as they occur on the program line.

Arithmetic operators:

| | |
|---|---|
| ^ | Exponentiation (eg, `b^n` means $b^n$) |
| * / \ MOD | Multiplication, division, integer division and modulus (remainder) |
| + - | Addition and subtraction |

Shift operators:

| | |
|---|---|
| x << y x >> y | These operate in a special way. << means that the value returned will be the value of x shifted by y bits to the left while >> means the same only right shifted. They are integer functions and any bits shifted off are discarded and any bits introduced are set to zero. |

Logical operators:

| | |
|---|---|
| NOT INV | invert the logical value on the right (eg, `NOT(a=b)` is `a<>b`)<br>or bitwise inversion of the value on the right (eg, `a = INV b`) |
| <> < > <= =< >= => | Inequality, less than, greater than, less than or equal to, less than or equal to (alternative version), greater than or equal to, greater than or equal to (alternative version) |

| | |
|---|---|
| = | equality |
| AND OR XOR | Conjunction, disjunction, exclusive or |

For Microsoft compatibility the operators AND, OR and XOR are integer bitwise operators. For example, PRINT (3 AND 6) will output the number 2. Because these operators can act as both logical operators (for example, IF a=5 AND b=8 THEN …) and as bitwise operators (eg, y% = x% AND &B1010) the interpreter will be confused if they are mixed in the same expression. So, always evaluate logical and bitwise expressions in separate expressions.

The other logical operations result in the integer 0 (zero) for false and 1 for true. For example the statement PRINT 4 >= 5 will print the number zero on the output and the expression A = 3 > 2 will store +1 in A.

The NOT operator will invert the logical value on its right (it is not a bitwise invert) while the INV operator will perform a bitwise invert. Both of these have the highest precedence so they will bind tightly to the next value. For normal use of NOT or INV the expression to be operated on should be placed in brackets. Eg:

```
IF NOT (A = 3 OR A = 8) THEN …
```

String operators:

| | |
|---|---|
| + | Join two strings |
| <> < > <= =< >= => | Inequality, less than, greater than, less than or equal to, less than or equal to (alternative version), greater than or equal to, greater than or equal to (alternative version) in order of ascii value. |
| = | Equality |

String comparisons respect case. For example "A" is greater than "a".

## Mixing Floating Point and Integers

MMBasic automatically handles conversion of numbers between floating point and integers. If an operation mixes both floating point and integers (eg, `PRINT A% + B!`) the integer will be converted to a floating point number first, then the operation performed and a floating point number returned. If both sides of the operator are integers then an integer operation will be performed and an integer returned.

The one exception is the normal division ("/") which will always convert both sides of the expression to a floating point number and then returns a floating point number. For integer division you should use the integer division operator "\".

MMBasic functions will return a float or integer depending on their characteristics. For example, PIN() will return an integer when the pin is configured as a digital input but a float when configured as an analog input.

If necessary you can convert a float to an integer with the INT() function. It is not necessary to specifically convert an integer to a float but if it was needed the integer value could be assigned to a floating point variable and it will be automatically converted in the assignment.

## Structures

Structures (also known as User-Defined Types) allow you to group related variables of different types together under a single name. This is useful for organizing complex data such as coordinates, records, or any collection of related values. This section provides a brief overview but structures are described in full in the file *MMBasic_Structures_Manual.pdf* included in the firmware download zip file.

As an example of a structure, assume that you need to track a number of alarms. Each alarm has a number of properties; the time of the alarm, its level (1, 2. 3, etc) and the action taken. Using structures enables you to treat the data for an alarm as a single "thing".

For example, to create a structure for an alarm you might use:

```
TYPE StructAlarm
  time AS INTEGER
  level AS INTEGER
  action AS STRING
END TYPE
```

The types of the members can be any of the normal data types in MMBasic (FLOAT, INTEGER or STRING). This new type that you have defined can then be used to define a variable. For example you could define a new variable called `alarm` using:

```
DIM alarm AS StructAlarm
```

You can also define a structure as LOCAL or STATIC within a subroutine of function.

You can access members of the structure using a period (.) between the variable's name and the member's name. For example: `alarm.time`.

You can assign values to members of the structure using normal assignment. ie:

```
alarm.time = EPOCH(NOW)
alarm.level = 3
```

And you can access members of the structure using the same notation:

```
IF alarm.level > 4 THEN ...
```

You can assign one structure to another of the same type:

```
DIM alarm AS StructAlarm
DIM a2 AS StructAlarm
...
alarm = a2
```

You can call a subroutine or function using a structure as one or more of its parameters:

```
MySub alarm, a2
```

And a function can return a structure:

```
FUNCTION MyFun() AS StructAlarm
  MyFun.time = EPOCH(NOW)
  MyFun.level = 1
END FUNCTION
...
alarm = MyFun()
```

You can define an array of a structures as you would any normal array:

```
DIM alarm(nbr) AS StructAlarm
```

And you can easily access the members of each element in the array:

```
IF alarm(i).time < x AND alarm(i).level < y THEN CallSomeSub
```

There are many commands within MMBasic for special handling of structures including saving and loading them from a file, sorting a structure array, resetting the members of a structure and so on. The full details are covered the file *MMBasic_Structures_Manual.pdf* included in the firmware download zip file.

## 64-bit Unsigned Integers

MMBasic supports 64-bit signed integers. This means that there are 63 bits for holding the number and one bit (the most significant bit) which is used to indicate the sign (positive or negative). However it is possible to use full 64-bit unsigned numbers as long as you do not do any arithmetic on the numbers.

64-bit unsigned numbers can be created using the &H, &O or &B prefixes to a number and these numbers can be stored in an integer variable. You then have a limited range of operations that you can perform on these. They are << (shift left), >> (shift right), AND (bitwise and), OR (bitwise or), XOR (bitwise exclusive or), INV (bitwise inversion), = (equal to) and <> (not equal to). Arithmetic operators such as division or addition may be confused by a 64-bit unsigned number and could return nonsense results.

To display 64-bit unsigned numbers you should use the HEX$(), OCT$() or BIN$() functions.

For example, the following 64-bit unsigned operation will return the expected results:

```
X% = &HFFFF0000FFFF0044
Y% = &H800FFFFFFFFFFFF
X% = X% AND Y%
PRINT HEX$(X%, 16)
```

Will display "`800F0000FFFF0044`"

# Subroutines and Functions

A program defined subroutine or function is simply a block of programming code that is contained within a module and can be called from anywhere within your program. It is the same as if you have added your own command or function to the language.

## Subroutines

A subroutine acts like a command and it can have arguments (sometimes called a parameter list). In the definition of the subroutine they look like this:

```
SUB MYSUB arg1, arg2$, arg3
  <statements>
  <statements>
END SUB
```

And when you call the subroutine you can assign values to the arguments. For example:

```
MYSUB  23, "Cat", 55
```

Inside the subroutine `arg1` will have the value `23`, `arg2$` the value of `"Cat"`, and so on. The arguments act like ordinary variables but they exist only within the subroutine and will vanish when the subroutine ends. You can have variables with the same name in the main program and they will be hidden by the arguments defined for the subroutine.

When calling a subroutine you can supply less than the required number of values and in that case the missing values will be assumed to be either zero or an empty string. You can also leave out a value in the middle of the list and the same will happen. For example:

```
MYSUB  23, , 55
```

Will result in `arg2$` being set to the empty string `""`.

Rather than using the type suffix (eg, the $ in arg2$) you can use the suffix AS <type> in the definition of the subroutine argument and then the argument will be known as the specified type, even when the suffix is not used. For example:

```
SUB MYSUB arg1, arg2 AS STRING, arg3
  IF arg2 = "Cat" THEN …
END SUB
```

Inside a subroutine you can define a variable using LOCAL (which has the same syntax as DIM). This variable will only exist within the subroutine and will vanish when the subroutine exits.

## Functions

Functions are similar to subroutines with the main difference being that a function is used to return a value in an expression. The rules for the argument list in a function are similar to subroutines. The only difference is that brackets are required around the argument list when you are calling a function, even if there are no arguments (brackets are optional when calling a subroutine).

To return a value from the function you assign a value to the function's name within the function. If the function's name is terminated with a $, a % or a ! the function will return that type, otherwise it will return whatever the OPTION DEFAULT is set to. You can also specify the type of the function by adding AS <type> to the end of the function definition.

For example:

```
FUNCTION Fahrenheit(C) AS FLOAT
  Fahrenheit = C * 1.8 + 32
END FUNCTION
```

A function can also return a complete array by declaring the array dimensions with the type at the end of the definition (INTEGER and FLOAT arrays only). The result is set by assigning to elements of the function's name. For example:

```
FUNCTION Multiples(n%) AS INTEGER(5)
  LOCAL i%
  FOR i% = 0 TO 5 : Multiples(i%) = n% * i% : NEXT i%
END FUNCTION
```

Such a function can only be used as the source of a whole-array assignment; the destination array must have the same type and total number of elements. For example:

```
DIM m%(5)
m%() = Multiples(3)
```

It cannot be used anywhere else in an expression. Note that reading an element of the function's name inside the function (eg, x = Multiples(i%)) will call the function recursively; if intermediate values must be read back, build the result in a LOCAL array and copy it to the function's name with a whole-array assignment (eg, Multiples() = tmp%()).

The dimensions can be expressions and are evaluated each time the function is called, after its arguments have been set up, so they can depend on the arguments. The result array can also be filled by passing it to any command that takes an array (eg, the MATH commands) rather than by assigning to its elements. For example, this returns the inverse of whatever size matrix it is given:

```
FUNCTION Invr(N()) AS FLOAT(BOUND(N(),1), BOUND(N(),2))
  MATH M_INVERSE N(), Invr()
END FUNCTION
DIM a(3,3), b(3,3) : b() = Invr(a())
```

## Passing Arguments by Reference

If you use an ordinary variable (i.e., not an expression) as the value when calling a subroutine or a function, the argument within the subroutine/function will point back to the variable used in the call and any changes to the argument will also be made to the supplied variable. This is called passing arguments by reference.

For example, you might define a subroutine to swap two values, as follows:

```
SUB Swap a, b
  LOCAL t
  t = a
  a = b
  b = t
END SUB
```

In your calling program you would use variables for both arguments:

```
Swap nbr1, nbr2
```

And the result will be that the values of `nbr1` and `nbr2` will be swapped.

For this to work the type of the variable passed (eg, `nbr1`) and the defined argument (eg, `a`) must be the same (in the above example both default to float).

If you want to use an argument as a general purpose variable inside a subroutine or function (ie, change its value) you should prefix its definition with the keyword BYVAL. This instructs MMBasic to always use the value of the argument, even if it is a variable, and to never point back to the variable used in the call. This is because another user of your routine may unwittingly use a variable in their call and that variable could be "magically" changed by your routine if you did not use BYVAL.

Note that BYVAL only works with simple variables (not arrays or structures).

## Passing Arrays

Single elements of an array can be passed to a subroutine or function and they will be treated the same as a normal variable. For example, this is a valid way of calling the Swap subroutine (discussed above):

```
Swap dat(i), dat(i + 1)
```

This type of construct is often used in sorting arrays.

You can also pass one or more complete arrays to a subroutine or function by specifying the array with empty brackets instead of the normal dimensions. For example, `a()`. In the subroutine or function definition the associated parameter must also be specified with empty brackets. The type (i.e., float, integer or string) of the argument supplied and the parameter in the definition must be the same.

In the subroutine or function the array will inherit the dimensions of the array passed and these must be respected when indexing into the array. If required the dimensions of the array could be passed as additional arguments to the subroutine or function or it could find these via the BOUND() function. The array is passed by reference which means that any changes made to the array within the subroutine or function will also apply to the supplied array.

For example, when the following is run the words "Hello World" will be printed out:

```
DIM MyStr$(5, 5)
MyStr$(4, 4) = "Hello" : MyStr$(4, 5) = "World"
Concat MyStr$()
PRINT MyStr$(0, 0)

SUB Concat arg$()
  arg$(0,0) = arg$(4, 4) + " " + arg$(4, 5)
END SUB
```

A subroutine or function can also return a complete array to its caller. This is described under Functions earlier in this chapter.

## Early Exit

There can be only one END SUB or END FUNCTION for each definition of a subroutine or function. To exit early from a subroutine (i.e., before the END SUB command has been reached) you can use the EXIT SUB command. This has the same effect as if the program reached the END SUB statement. Similarly you can use EXIT FUNCTION to exit early from a function.

## Recursion

Recursion is where a subroutine or function calls itself. You can do recursion in MMBasic but there are a number of issues (these are a direct consequence of the limitations of microcontrollers and the BASIC language):

- There is a fixed limit to the depth of recursion. In the PicoMite firmware this is 50 levels. In practice the processor's stack usually runs out first, particularly in a function: a simple recursive function reaches about 13 levels on an RP2350 and 9 on an RP2040 before the error "Stack overflow" (compiled with OPTION COMPILE ON, a few levels more).
- If you have many arguments to the subroutine or function and many LOCAL variables (especially strings) you could easily run out of memory before reaching the 50 level limit.
- FOR…NEXT loops work correctly when the subroutine or function is called recursively from within them, but any DO…LOOPs will be corrupted.

## Examples

There is often the need for a special command or function to be implemented in MMBasic but in many cases these can be constructed using an ordinary subroutine or function which will then act exactly the same as a built in command or function.

For example, sometimes there is a requirement for a TRIM function which will trim specified characters from the start and end of a string. The following provides an example of how to construct such a simple function in MMBasic.

The first argument to the function is the string to be trimmed and the second is a string containing the characters to trim from the first string. RTrim$() will trim the specified characters from the end of the string, LTrim$() from the beginning and Trim$() from both ends.

```
' trim any characters in c$ from the start and end of s$
Function Trim$(s$, c$)
  Trim$ = RTrim$(LTrim$(s$, c$), c$)
End Function

' trim any characters in c$ from the end of s$
Function RTrim$(s$, c$)
  RTrim$ = s$
  Do While Instr(c$, Right$(RTrim$, 1))
    RTrim$ = Mid$(RTrim$, 1, Len(RTrim$) - 1)
  Loop
End Function

' trim any characters in c$ from the start of s$
Function LTrim$(s$, c$)
  LTrim$ = s$
  Do While Instr(c$, Left$(LTrim$, 1))
```

```
      LTrim$ = Mid$(LTrim$, 2)
    Loop
  End Function
```

As an example of using these functions:

```
  S$ = "   ****23.56700  "
  PRINT Trim$(s$, " ")
```

Will give "****23.56700"

```
  PRINT Trim$(s$, " *0")
```

Will give "23.567"

```
  PRINT LTrim$(s$, " *0")
```

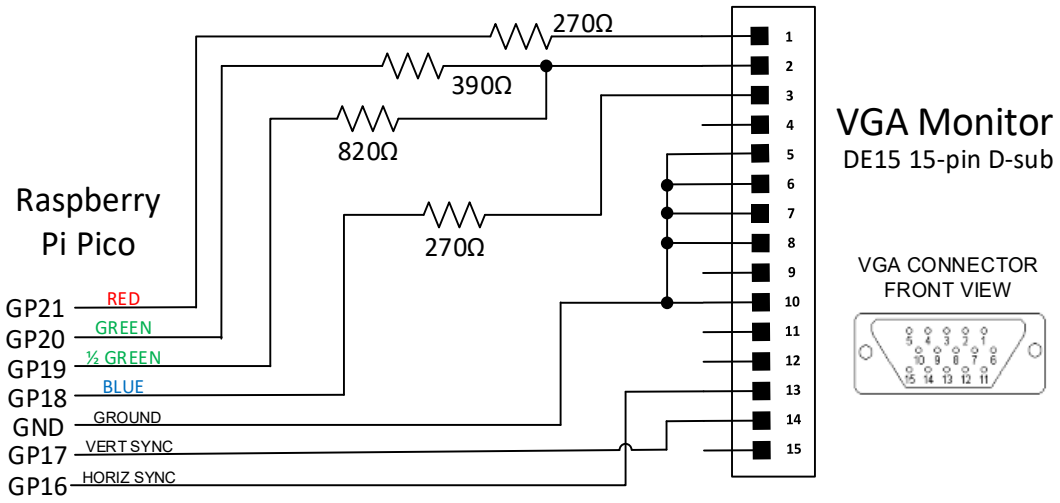
Will give "23.56700"

# Video Output

## VGA Video

For versions of the firmware that support VGA the video output is automatically enabled at startup – there are no options that need to be set.

The following diagram illustrates how to attach the VGA monitor.




The output is in the standard VGA format with a pixel rate of 25.175MHz and a frame rate of 60Hz.

There are two or three modes which can be selected using the MODE command:

MODE 1 Monochrome with a resolution of 640 x 480 (default at startup)

MODE 2 16 colours with a resolution of 320 x 240

MODE 3 16 colours with a resolution of 640 x 480 (RP2350 only)

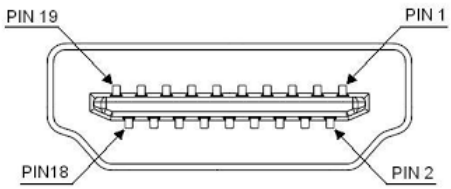
In MODE 2 and 3 the output is 16 colours in the 4-bit RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). In MODE 2 the pixels are duplicated along both the x and y axis giving a 320 x 240 resolution while the monitor still sees a 640 x 480 signal.

The output from MMBasic is written as a bitmap to a framebuffer. The firmware then uses the second CPU in the processor to feed this framebuffer data, a pixel line at a time, via DMA to one of the RP2040's programmable I/O controllers (PIO0) to generate the display. As this runs independently of the main processor there is little or no impact on the speed of MMBasic caused by generating the VGA output.

## HDMI Video

For versions of the firmware that support HDMI video the following table lists the connections to the standard HDMI Type A socket. The HDMI output is automatically enabled at startup – there are no options that need to be set.

HDMI Pin 1: Pin 21 (GP16) via a 220Ω resistor
HDMI Pin 2: Ground
HDMI Pin 3: Pin 22 (GP17) via a 220Ω resistor
HDMI Pin 4: Pin 24 (GP18) via a 220Ω resistor
HDMI Pin 5: Ground
HDMI Pin 6: Pin 25 (GP19 via a 220Ω resistor
HDMI Pin 7: Pin 16 (GP12) via a 220Ω resistor
HDMI Pin 8: Ground
HDMI Pin 9: Pin 17 (GP13) via a 220Ω resistor
HDMI Pin 10: Pin 19 (GP14) via a 220Ω resistor
HDMI Pin 11: Ground
HDMI Pin 12: Pin 20 (GP15) via a 220Ω resistor
HDMI Pin 13: No Connection
HDMI Pin 14: No Connection
HDMI Pin 15: No Connection
HDMI Pin 16: No Connection
HDMI Pin 17: Ground
HDMI Pin 18: +5V via Schottky barrier diode
HDMI Pin 19: No Connection




HDMI Front View

The HDMI signal pins are driven at a high frequency and for this reason care should be taken as follows:

- Keep the signal traces as short as possible.
- Make sure that all signal traces are the same length.
- The 220Ω resistors should preferably be surface mount types.

To generate the DVI/HDMI signal the firmware needs to overclock the RP2350 to as high as 378MHz and most Raspberry Pi Pico 2 modules will not have any trouble at these speeds. However, this cannot be guaranteed, especially with third party modules. An example is the Pimoroni Pico Plus 2 which is marginal at the required speeds and therefore cannot be recommended with the HDMI versions of the PicoMite firmware.

Similar to how VGA is generated, the output of MMBasic is written to a framebuffer which, using the second CPU and DMA, is fed to the HSTX peripheral which in turn generates the parallel video data. The video signal produced is actually DVI (HDMI supports DVI) so this means that audio is not supported on the HDMI output and sophisticated HDMI features such as High Definition Content Protection (HDCP) and Ethernet are also not supported.

HDMI video supports a number of resolutions. To set these you use the following command:

```
OPTION RESOLUTION nn
```

Where 'nn' is one of:

640x480 or 640

1280x720 or 1280

1024x768 or 1024

800x600 or 800

720x400 or 720

848x480 or 848

1024x600

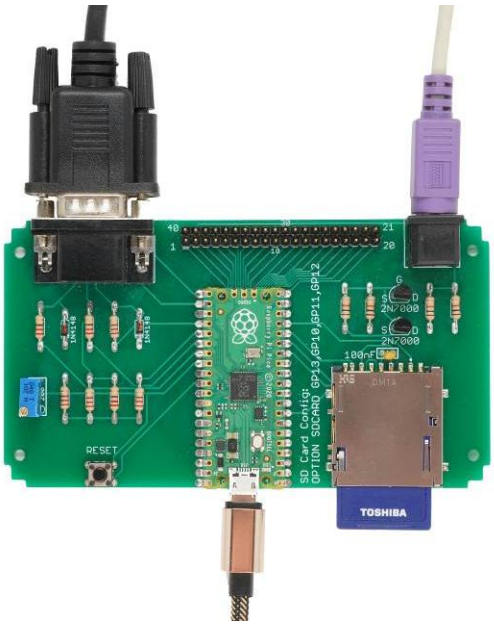
800x480

Note that on the PicoMiteHDMIWEB version the abbreviated form 1024 selects 1024x600 rather than 1024x768 (which that version does not support); write the resolution out in full if a program has to run on both. The resolution can also be changed at run time, without a reboot, using the RESOLUTION command - see the command listing for the details.

Note that the PicoMiteHDMIWEB version uses a reduced HDMI pipeline to recover the memory needed by its network stack: everything is 8-bit (RGB332) colour. The available resolutions, with their fixed clock speeds, are 640x480 (the default, at 252, 315 or 378MHz), 720x400 (283.2MHz), 800x600 (360MHz), 848x480 (336MHz), 800x480 (333MHz) and 1024x600 (252MHz). The 800x600, 848x480 and 800x480 resolutions support only MODE 1 and MODE 2 (the reduced framebuffer cannot hold the higher modes). MODE 4 is not available in any resolution on this version; 640x480 and 720x400 support MODE 1, 2 and 5, and 1024x600 supports MODE 1, 2, 3 and 5. Both OPTION RESOLUTION and the RESOLUTION command accept these. The PicoMitePC3 is not reduced in this way: its network stack lives in the Pico Computer 3's PSRAM and it keeps the full HDMI pipeline of the PicoMiteHDMIUSB.

Each HDMI resolution can operate in a number of modes which are set using the MODE command. Note that many modes reduce the displayed resolution to save memory for other features, this reduction is done by doubling or quadrupling each pixel, however the monitor will always see the resolution (ie, pixel density) set by the OPTION RESOLUTION command. The default is RESOLUTION 640x480 and MODE 1.

## VGA/PS2 Reference Design (Raspberry Pi Pico)




This is an easy to assemble design that implements the VGA output, PS2 keyboard interface and the SD Card socket (this design was featured in *Silicon Chip* magazine).

It uses common thru-hole components and can be assembled in under an hour.

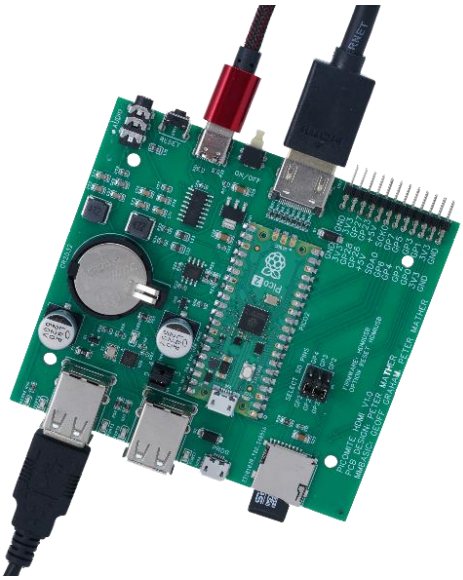
All 40 pins on the Raspberry Pi Pico are routed to a 40-way connector on the rear of the PCB in the same configuration as that used by the Pico. This makes it easy to connect external devices as you can consult the pin out diagram in this manual and then select the corresponding pins on the 40-way connector.

Allowing for the I/O pins reserved for the VGA output, keyboard and SD Card there are 14 I/O pins available for external circuitry.

The board is sized to fit in an Altronics snap-together case 130 x 75 x 28mm (part number H0376).

The construction pack for this design can be downloaded from: https://geoffg.net/picomitevga.html (at the bottom of the page).

## HDMI/USB Reference Design (Raspberry Pi Pico 2)



This is a full featured design based on the Raspberry Pi Pico 2 (using the RP2350 processors) which includes:

- HDMI video output
- Four USB interfaces for keyboard, mouse, game pad, etc.
- High quality audio output for amplified speakers.
- USB interface to the serial console.
- Battery backed real-time clock.
- Micro SD card socket.
- 14 I/O pins made available on the rear panel.
- Sized to fit a Multicomp MCRM2015S or Hammond RM2015S enclosure.

The construction pack for this design can be downloaded from: https://geoffg.net/picomitevga.html (at the bottom of the page).

# Keyboard/Mouse/Gamepad

The PicoMite firmware can accommodate a keyboard and mouse using either the PS2 or USB interfaces. The choice between PS2 and USB is determined by the version of the firmware loaded. See the chapter *Firmware Versions and Files* at the start of this manual.

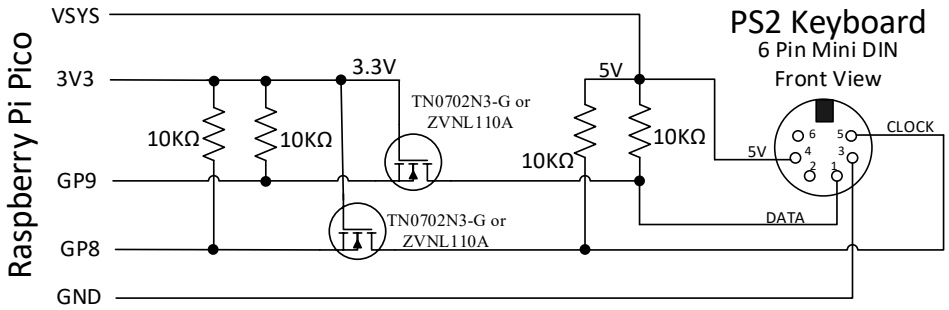
USB versions of the firmware will also support a PS3 or PS4 gamepad with a USB interface. For versions without USB support the WII (CLASSIC) and WII NUNCHUCK commands can be used to specify a gamepad connected using $I^2C$.

A keyboard can be used to input data to the BASIC program or, with a VGA/HDMI video output, be used to create a self contained computer with keyboard and display. Instead of using a video output you can also connected an LCD panel on versions supporting this and display the MMBasic console output on the LCD panel, creating a more compact version of a self contained computer. See the section *LCD Display as the Console Output* for details on how to do this.

## PS2 Keyboard on the Raspberry Pi Pico (RP2040)

The PS2 keyboard clock and data signals operate at 5V but the I/O pins on the RP2040 processors must not be subjected to more than 3.6V. For this reason a level shifter should be employed so that the Raspberry Pi Pico sees signal voltages in the range 0 to 3.3V while the keyboard sees voltages 0 to 5V.

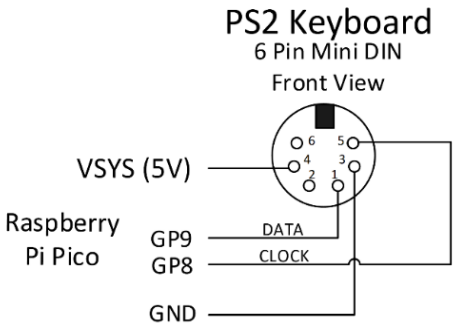
There are many ways that this can be accomplished but the following circuit is a simple and low cost solution:




The recommended MOSFET is a TN0702N3-G or ZVNL110A however the commonly available 2N7000 has been tested and works well.

After it is connected the keyboard must be enabled with the OPTION KEYBOARD command.

## PS2 Keyboard on the Raspberry Pi Pico 2 (RP2350)




The I/O pins on the RP2350 series of microcontrollers can withstand 5V (while powered up) so the keyboard can be directly connected as illustrated with the 5V supplied by the VSYS pin on the Raspberry Pi Pico 2.

The keyboard is enabled with the OPTION KEYBOARD command.

## PS2 Mouse

The I/O pins used for a PS2 mouse must be configured using the commands OPTION MOUSE (at the command prompt) or MOUSE OPEN (within a program).

A PS2 mouse is powered via 5V so on a Raspberry Pi Pico (RP2040) a level shifter will be required for the mouse clock and data pins - this can be the same as the above circuit for a PS2 keyboard. On a Raspberry Pi Pico 2 (RP2350) a level shifter is not required so the mouse can be directly connected.

## USB Interface

Versions of the firmware (both RP2040 or RP2350) with USB support will allow the connection of a keyboard, mouse and/or gamepad, all via the USB interface. To accomplish this the Raspberry Pi Pico's USB port is converted to a USB host (as against its normal mode of USB client). This is possible because the Pico's connector and electronics are USB OTG (On The Go) compliant similar to the connector on many mobile phones.

Because the USB connector is used for other duties the Pico must be powered via 5V applied to the VSYS pin or the VbUS pin if you desire to power an external hub/keyboard from the pico..

To connect a USB device you need a converter cable which has a Micro Male USB plug on one end (for the Pico) and a Type A USB socket at the other end (for the device). A typical example is the Jaycar Cat Nbr WC7725.

Because the USB interface on the Pico has been converted to a USB host you will not have access to the MMBasic serial console. The firmware accommodates this by automatically using pin 11 (GP8) for the serial console Tx and pin 12 (GP9) for Rx and setting the baudrate to 115200 baud. To access this console you will need a USB to serial bridge which provides a TTL serial interface on one side and a USB interface on the other (search for modules using the CP2102 or CH340 chip). If needed OPTION SERIAL CONSOLE can be used to change the pins used for the console.

## USB Hub

The firmware will also support a USB hub via this interface so it is possible to have multiple keyboards or a keyboard plus a mouse plus a gamepad, etc. A maximum of 4 devices may be connected via a hub. These are reference by a channel index number (1 to 4). Use MM.INFO(USB n) to return the device code for any device connected to channel n. By default a keyboard will be allocated to channel 1. A mouse will be allocated to channel 2. The first gamepad will be allocated to channel 3 and a second gamepad to channel 4. In addition to these four channels, up to four USB serial (CDC) devices can be connected - see the heading USB Serial Devices below.

If you use a USB hub it is better to use an unpowered hub (ie, one that is powered by the Raspberry Pi Pico). This is because the USB protocol stack cannot reset the hub and it may be confused if the power on the Pico is cycled without doing the same for the hub. The hub can also be confused if devices are swapped while the hub is powered. If this happens you should cycle the power on the Pico followed by the hub then add the USB devices one by one.

Note that a hub is not required. If you only want to connect one device (for example a keyboard) you can just plug the device (using an adapter cable) directly into the Pico's USB connector.

## USB Keyboard

When a USB keyboard is connected it will be immediately recognised (no configuration required) and MMBasic will allocate it to channel 1 by default– there is nothing extra required.

## USB Mouse

When a USB mouse is connected it will be immediately recognised (no configuration required) and MMBasic will allocate it to channel 2 by default– there is nothing extra required.

## USB Gamepad

One or more PS3 or PS4 controllers or gamepad such as a Super Nintendo SNES Controller with a USB interface can be connected via USB (illustrated on the right).



By default the first gamepad will be allocated to channel 3 and a second gamepad channel 4. Within a program the data from the gamepad can be read using the DEVICE(GAMEPAD) function.

## USB Touch Screen

A USB touch screen (for example, the touch interface built into many HDMI monitors intended for the Raspberry Pi) can be connected and will be immediately recognised - no configuration or calibration is required. When the touch screen is detected the firmware will report it on the console, for example:

Multi-touch Connected on channel 3 (10 contacts, 1024x600)

The touch positions are automatically scaled to the resolution of the current display so a BASIC program can use them directly as screen coordinates. The touch is read using the TOUCH() function exactly as for a touch sensitive LCD panel: TOUCH(X), TOUCH(Y) and TOUCH(DOWN) report the first touch point, TOUCH(X2), TOUCH(Y2), TOUCH(XN n) and TOUCH(YN n) report additional touch points (up to ten) and the gesture functions (TOUCH(TAP), TOUCH(SWIPE), TOUCH(PINCH), etc) are fully supported. See the TOUCH() function in the Functions section of this manual for the details.

A USB touch screen can also be used with the on-screen keyboard (see below) and its multi-touch capability allows the modifier keys (Sh, Ctl, etc) to be held down while another key is tapped.

Touch screens vary in how they report touch and many identify themselves as a combination of a touch digitizer and a mouse. The firmware recognises these and will automatically use the best interface that the screen offers; some will be reported on the console as "USB Touchscreen Connected" and support a single touch point only. No user action is required in either case.

## USB Serial Devices (COM3 to COM6)

Versions of the firmware with USB host support can also communicate with USB serial (CDC) devices. This includes USB to serial adapters based on the CH340, CP210x and FTDI chips as well as devices that present a USB serial interface of their own such as an Arduino, another Raspberry Pi Pico, a GPS receiver, etc.

Up to four USB serial devices can be connected (via a hub) and they are allocated to the serial ports COM3, COM4, COM5 and COM6 in the order that they are plugged in. These are in addition to the two hardware serial ports COM1 and COM2. When a device is connected or removed a message is shown on the console, eg, "USB CDC Device Connected on channel 5 (COM3)" (USB serial devices are reported as channels 5 to 8 which correspond to COM3 to COM6). On the PicoMiteHDMIBTH a Bluetooth serial device can also be used, as COM7 (see the chapter Bluetooth Versions).

The port is opened and used exactly like the hardware serial ports, eg:

```
OPEN "COM3:115200" AS #1
```

The baud rate is passed to the USB device (where the device supports it) and the connection always uses 8 data bits, no parity and one stop bit (the 7BIT, EVEN, ODD, S2 and INV options do not apply). A receive buffer size, an interrupt subroutine and an interrupt trigger level can be specified exactly as for COM1 and COM2 - see Appendix A - Serial Communications. The DTR and RTS signals are asserted while the port is open.

If the USB device is unplugged while the port is open the port will remain open and communications will resume transparently when the device is plugged back in. The CLOSE command closes the port and frees its receive buffer.

## USB Flash Drive

On the RP2350 versions of the firmware with USB support a USB flash drive (memory stick) can also be connected, either directly or via a hub. The drive will be automatically recognised and the message "USB Flash Drive Connected" will be shown on the console.

The drive is accessed as drive C: and works alongside the internal flash filesystem (A:) and the SD card (B:). All of the normal file commands can be used with it (FILES, LOAD, SAVE, RUN, OPEN, COPY, etc) and it can be made the default drive with the command DRIVE "C:" or by simply typing C: at the command prompt. This makes it easy to transfer files between a PC and the PicoMite. The drive must be formatted as FAT16 or FAT32 (exFAT is not supported so large drives, which are usually supplied formatted as exFAT, may need to be reformatted as FAT32).

Note that the Raspberry Pi Pico's USB port operates at USB full speed (12 Mbits/s), which caps the transfer rate at a few hundred KB per second regardless of how fast the flash drive itself is. Reads run close to that limit (typically 300 to 500 KB per second), which is comparable to, and often faster than, an SD card on the same board, so a USB flash drive is well suited to loading programs and reading or streaming large files such as audio. Writes are considerably slower (around 50 KB per second), so for applications that frequently write large amounts of data an SD card will still give much better performance.

A Raspberry Pi Pico in BOOTSEL mode also appears as a USB flash drive and the command FLASH UF2 fname$ will write the UF2 firmware file 'fname$' (stored on the A: or B: drive) directly to it. This allows a PicoMite to program another Raspberry Pi Pico without using a PC.

## Configuring the Keyboard

By default the keyboard configuration will be assumed to be the standard US layout. However the OPTION KEYBOARD command can be used to configure layouts for other countries.

The syntax of the command is:

```
OPTION KEYBOARD language [,capslock] [,numlock] [,repeatstart] [,repeatrate]
```

Where 'language' is a two-character code such as US for the standard keyboard used in the USA, Australia and New Zealand. Other keyboard layouts are United Kingdom (UK), French (FR), German (GR), Belgium (BE), Italian (IT), Brazilian (BR) or Spanish (ES). Belgium (BE) and Brazilian (BR) are not available on the USB and Bluetooth versions.

Note that the non US layouts map some of the special keys present on these keyboards but the corresponding special character will not display as they are not part the standard PicoMite fonts. Instead a standard ASCII character will be used.

The optional parameters 'capslock' and 'numlock' set the initial state of Caps Lock and Num Lock (the default is Caps Lock off and Num Lock on) and 'repeatstart' and 'repeatrate' set the key repeat timing - see OPTION KEYBOARD in the Options listing for the details, which differ between the PS2 and the USB and Bluetooth versions. A keyboard with a numeric keypad built into its letter keys, such as the Raspberry Pi keyboard, needs Num Lock off or keys such as U, I and O will type 4, 5 and 6, for example:

```
OPTION KEYBOARD UK, 0, 0
```

## Using a Mouse

The mouse is especially useful in the MMBasic program editor where it can replicate much of the functionality found in GUI editors such as Notepad in Windows (see the heading *Full Screen Editor* above in this manual). This includes positioning the insert point and copy and paste using the clipboard.

A mouse can also be used in a program where its position can be queried by using the DEVICE() function. As an example, the following program will report any mouse movement.

Note that the mouse is always allocated to channel 2

```
' continuous loop to print on the console any movement
Do
  mx=DEVICE(MOUSE 2, x)
  my=DEVICE(MOUSE 2, y)
  If mx <> tx Or my <> ty Then Print mx, my
  tx = mx : ty = my
Loop
```

A program can also display a cursor on the screen and link it to the mouse using the GUI CURSOR command, and mouse clicks can be read with the CLICK() function which works the same way as the TOUCH() function - see the command and function listings for the details.

## On-Screen Keyboard

On the VGA and HDMI versions with USB support a touch screen can be used to display an on-screen keyboard. This makes it possible to use the PicoMite without a physical keyboard at all – commands can be entered at the command prompt and programs edited in the full screen editor by tapping the keys drawn at the bottom of the screen.

The on-screen keyboard is enabled with the command:

```
OPTION SCREEN KEYBOARD nn [, M]
```

Where 'nn' is the percentage of the screen's vertical resolution (up to 50) that will be reserved for the keyboard at the bottom of the screen. Setting 'nn' to zero disables the feature. This option is saved permanently and the console and full screen editor will automatically adjust to use the remaining screen area.

The keyboard is displayed automatically at the command prompt and in the full screen editor. While a program is running the keyboard is removed so that the program has use of the full screen; a program that wants input from the on-screen keyboard can display it with the command KEYBOARD ON and remove it again with KEYBOARD OFF. Keys tapped on the on-screen keyboard are processed exactly as if they had been typed on a USB keyboard (eg, they can be read with INKEY$ or INPUT and will trigger keyboard interrupts). KEYBOARD OFF can also be used at the command prompt and will keep the keyboard hidden until KEYBOARD ON is entered.

The keyboard provides lower case, upper case (Sh), number/symbol (Sym) and function key (Fn) layouts. The Fn layout includes F1 to F12, Home, End, Page Up, Page Down, Insert and Delete. The Ctl key applies control to the next key (eg, Ctl followed by C generates CTRL-C which will interrupt a running program).

Normally a tap on one of the modifier keys (Sh, Ctl, Sym or Fn) latches it on until the next key is tapped. If the touch screen supports multi-touch the 'M' option can be added to the OPTION SCREEN KEYBOARD

command and the modifier keys will then behave as on a physical keyboard – hold the modifier down with one finger while typing with another.

When the on-screen keyboard is enabled a double tap on the touch screen (two quick taps in the same place) can be used to exit the GUI TEST LCDPANEL and GUI TEST TOUCH commands, which otherwise run until a key is pressed on the console keyboard.

# Program and Data Storage

The BASIC program is held in flash memory and is run from there. When a program is edited via EDIT or loaded via the console it will be saved there. Flash memory is non-volatile so the program will not be lost if the power is lost or the processors is reset.

In addition to this program memory there are three other locations where programs can be saved. These are described in detail below and are Flash Slots, the Flash Filesystem and an attached SD Card

## Flash Slots

There are three of these which can be used to save completely different programs or previous versions of the program you are working on (in case you need to revert to an earlier version). In addition, MMBasic will allow a BASIC program to load and run another program saved to a numbered flash location while retaining all the variables and settings of the original program – this is called chaining and allows for a much larger program to be run than the amount of program memory would normally allow.

To manage these numbered locations in flash you can use the following commands (note that in the following *n* is a number from 1 to 3):

| | |
|---|---|
| FLASH SAVE *n* | Save the program in the program memory to the flash location *n*. |
| FLASH LOAD *n* | Load a program from flash location *n* into the program memory. |
| FLASH RUN *n* | Run a program from flash location *n*, clears all variables but does not erase or change the program held in the main program memory. |
| FLASH LIST | Display a list of all flash locations including the first line of the program. |
| FLASH LIST n [,ALL] | Lists the program held in location n. Use FLASH LIST n,ALL to list without page breaks |
| FLASH ERASE *n* | Erase flash location *n*. |
| FLASH ERASE ALL | Erase all flash locations. |
| FLASH CHAIN *n* | Run the program in flash location *n*, leaving all variables intact. This allows for very large programs, that can be split into two or three parts, to be run. It does not erase or change the program held in the main program memory. |
| FLASH OVERWRITE *n* | Erase flash location *n* and then save the program in the program memory to that location. |
| FLASH DISK LOAD f$ [,O] | Loads a flash slot with a binary file created using LIBRARY DISK SAVE. Overwrites the slot if the optional "O" is specified. |

In addition, the command OPTION AUTORUN can be used to specify a flash program location to be set running when power is applied or the CPU restarted. This option can also be used without specifying a flash location and in that case MMBasic will automatically run the program in program memory.

- It is recommended that you include a comment describing the program as the first line of the program. This will then be displayed by the FLASH LIST command and will help identify the program.
- All BASIC programs saved to flash may be erased if you upgrade (or downgrade) the PicoMite firmware. So make sure that you backup these first.
- The LIBRARY command uses Slot 3 for saving library data therefore only 2 slots will be available if the library feature is used.

## Flash Filesystem

This is an area of the Raspberry Pi Pico's flash memory which is automatically created by the firmware and will look like a normal disk drive to MMBasic. It is called drive A: and data and programs can be read/written using the normal BASIC file commands (SAVE, RUN, OPEN, etc). In addition, sub directories can be created and deleted and long filenames used.

For example, to run a program:

```
RUN "A:/MyProgram.bas"
```

Open a text file for random access:

```
OPEN "A:/data/database.dat" FOR RANDOM as #4
```

This drive is automatically created when the PicoMite firmware is loaded so it will always be available to the BASIC program. It can be used to store programs, images, music, configuration data, log files and much more. Its size varies depending on the amount of flash memory – on a Raspberry Pi Pico with 2MB flash it will be 200 to 500KB, on a Raspberry Pi Pico 2 with 4MB flash it will be a little over 2MB and on a module with 16MB the flash filesystem will be up to 14MB in size. On an RP2350 with 16MB the Flash Filesystem is always 12MB and starts at the same place in every version of the firmware (PicoMite, WebMite, VGA, HDMI and so on), so its contents are kept when changing to another version or when the options are reset with OPTION RESET.

The system will create and maintain the file "BOOTCOUNT" on the Flash Filesystem. This keeps a count of the number of times the device has been restarted and can be read with the function MM.INFO(boot count).

## SD Cards

An SD card socket can be connected to the Raspberry Pi Pico and accessed as drive B:. Like the Flash Filesystem the normal BASIC file commands can be used to save/load programs as well as opening data files for read/write.

Cards up to 32 GB formatted in FAT16 or FAT32 are supported and the files created can also be read/written on personal computers running Windows, Linux or the Mac operating system. The PicoMite firmware uses the SPI protocol to talk to the card and this is not influenced by the card type, so all types (Class 4, 10, UHS-1 etc) are supported. For a description of SPI see: http://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus

The SPI protocol needs to be specifically configured before it can be used. This can be done in one of two ways - by using the "system" SPI port or by directly specifying the I/O pins to use:

System SPI Port

This is a port that will be used for system use (SD card, LCD display and the touch controller on an LCD panel). There are a number of ports and pins that can be used (see the chapter *PicoMite Hardware*) but this example uses SPI on pins GP18, GP19 and GP16 for Clock, MOSI and MISO respectively.

```
OPTION SYSTEM SPI GP18, GP19, GP16
```

Then MMBasic must be told that there is an SD card attached and what pin is used for the Chip Select (CS) signal:

```
OPTION SDCARD GP22
```

Dedicated I/O Pins

Alternatively, where no other devices share the SPI bus with the SD card it can be set up with:

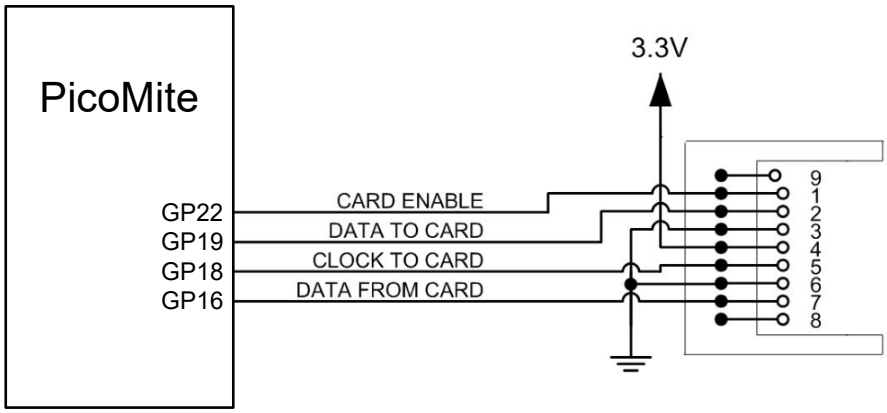
```
OPTION SDCARD  CS_pin, CLK_pin, MOSI_pin, MISO_pin
```

In this case the pins can be assigned completely flexibly and do not need to be capable of SPI operation but SD card performance will be better if valid SPI pins are chosen.

These commands must be entered at the command prompt (not in a program) and will cause the PicoMite firmware to restart. This has the side effect of disconnecting the USB console interface which will need to be reconnected.

When the Raspberry Pi Pico is restarted MMBasic will automatically initialise the SD Card interface. This SPI port will then not be available to BASIC programs (ie, it is reserved). To verify the configuration, you can use the command OPTION LIST to list all options that have been set including the configuration of the SD Card.

The basic circuit diagram for connecting the SD Card connector using these pin allocations is illustrated below.

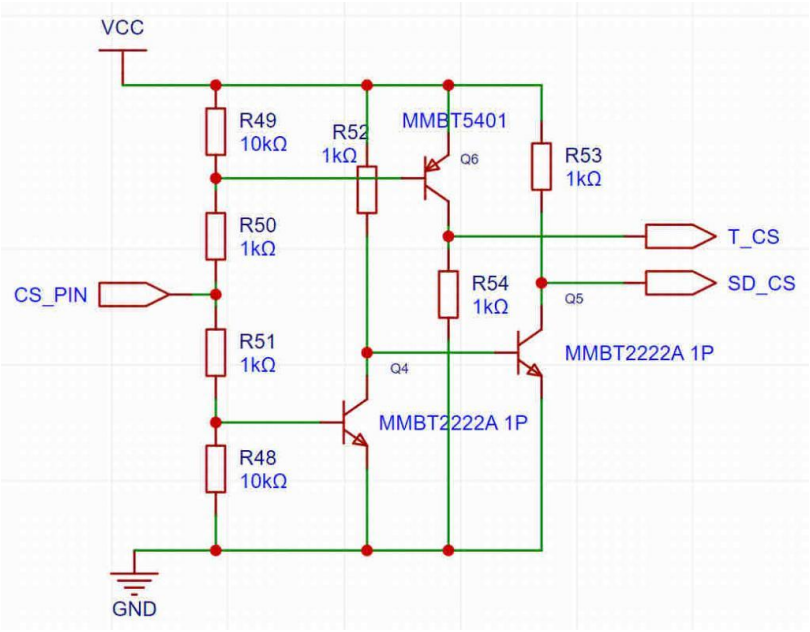


Note that you can use many different configurations using various pin allocations – this is just an example based on the configuration commands listed above.

Care must be taken when the SPI port is shared between a number of devices (SD Card, touch, etc). In this case all the Chip Select signals must configured in MMBasic or alternatively disabled by a permanent connection to 3.3V. If this is not done any floating Chip Select signal lines can cause the wrong controller to respond to commands on the SPI bus.

## Combined Chip Select

The chip select pin used for the SD card and the touch controller on an LCD panel can be combined by using the OPTION SDCARD COMBINED CS command. If this is specified the following circuitry is required to implement the SD chip select:




The firmware uses the touch pin as follows:

TOUCH_CS low: TOUCH_CS low, SD_CS high

TOUCH CS high: SD_CS low: TOUCH_CS high

TOUCH CS set as input (high impedance) TOUCH_CS and SD_CS high.

## USB Flash Drive

On the RP2350 versions with USB support a USB flash drive can be connected and accessed as drive C: using the same file commands as the other drives. See the heading USB Flash Drive in the chapter Keyboard/Mouse/Gamepad for the details, including the format requirement (FAT16/FAT32) and the USB full speed performance limit.

## MMBasic Support for Flash and SD Card Filesystems

The MMBasic support for the Flash Filesystem, SD Cards and USB flash drives is almost identical. This allows programs to use either filesystem with minimal modification. The Flash Filesystem is referred to as drive A: while the SD Card (when connected) is drive B:. The default drive can be set with the DRIVE command and then the drive prefix is not needed.

In the following note that:

- Upper/lowercase characters and spaces are allowed. Neither the file system on the SD Card nor the Flash Filesystem is case sensitive.
- Long file/directory names are supported in addition to the old 8.3 format.
- The maximum file/path length is 63 characters (127 characters for RP2350 versions).
- Any file path that uses the drive letter must be a full path from the root (ie, “A:/mypath/myfile.txt”).
- Directory paths are allowed in file/directory strings. (i.e., OPEN "A:\dir1\dir2\file.txt" FOR …).
- Either forward or back slashes can be used in paths. Eg, \dir\file.txt is the same as /dir/file.txt.
- On startup the active drive (ie, the default) is A: (the Flash Filesystem).
- The current PicoMite firmware time is used for file create and last access times.

- Up to ten files can be simultaneously open, mixed between the A:, B: and C: drives.
- Except for INPUT, LINE INPUT and PRINT the # in #fnbr is optional and may be omitted.

Programs can be loaded from or saved to the Flash Filesystem and SD Cards using these commands.

□ LOAD fname$ [, C] [, R]
Load a BASIC program. The optional suffix ",R" will cause the program to be run after it has been loaded (in this case fname$ must be a string constant). The optional suffix ",C" will crunch the program (removing comments, blank lines and unnecessary spaces) as it is loaded.

□ RUN fname$
Load a BASIC program and run it. fname$ can be a variable.

□ SAVE fname$
Save the current program to the Flash Filesystem or SD Card.

These are the basic commands for reading and writing data.

□ OPEN fname$ FOR mode AS #fnbr
Opens a file for reading or writing. 'fname$' is the file name. 'mode' can be INPUT, OUTPUT, APPEND or RANDOM. '#fnbr' is the file number (1 to 10).

□ PRINT #fnbr, expression [[,; ]expression] … etc
Outputs text to the file opened as #fnbr.

□ INPUT #fnbr, list of variables
Read a list of comma separated data into the variables specified from the file previously opened as #fnbr.

□ LINE INPUT #fnbr, variable$
Read a complete line into the string variable specified from the file previously opened as #fnbr.

□ FLUSH #fnbr
Forces any buffered writes to be written to the Flash Filesystem or SD Card. It is recommended that this command be used regularly where data loss could occur in the event of power loss.

□ CLOSE #fnbr [,#fnbr] …
Close the file(s) previously opened with the file number '#fnbr'.

Basic file and directory manipulation. Most can be done at the command prompt or from within a BASIC program.

□ DRIVE drive$
Sets the active disk drive as 'drive$'. 'drive$' can be "A:", "B:" or "C:" where A is the flash drive, B is the SD Card and C is a USB flash drive on the RP2350 versions with USB support.

□ DRIVE "A:/FORMAT"
Reformat the Flash Filesystem (drive A:) to its initial state.

□ FILES [wildcard]
Search the current directory and list the files/directories found.
Note: Can only be used at the command prompt, not within a program.

□ LIST fname$
List the contents of a program or text file on the console.

□ KILL fname$
Delete a file in the current directory on the current drive.
See the command reference for more details on wildcard deletes.

□ MKDIR dname$
Creates a sub directory in the current directory on the current drive.

□ CHDIR dname$
Change into to the directory $dname. $dname can also be "." For the current directory, ".." (dot dot) for up one directory or "\" for the root directory. The starting point is the current directory on the current drive.

- RMDIR dir$
  Remove, or delete, the directory ‘dir$’ in the current directory on the current drive.
- SEEK #fnbr, pos
  Will position the read/write pointer in a file that has been opened for RANDOM access to the 'pos' byte.
- RENAME fromname$ AS toname$
  Will rename the file fromname$ to have the name toname$ in the current directory on the current drive
- COPY [mode] fromname$ TO toname$
  Will copy the file fromname$ to ~~have~~ the file toname$.
  See the command reference for more details on the optional mode and wildcard copies.

Also there are a number of functions that support the above commands.

- INPUT$(nbr, #fnbr)
  Will return a string composed of ‘nbr’ characters read from a file previously opened for INPUT or RANDOM with the file number ‘#fnbr’. If less than ‘nbr’ characters are available the function will return with what it has (including an empty string if no characters are available).
- DIR$( fspec, type )
  Will search for files and return the names of entries found.
- CWD$
  Will return the current working directory.
- EOF( #fnbr )
  Will return true if the file previously opened for INPUT with the file number ‘#fnbr’ is positioned at the end of the file.
- LOC( #fnbr )
  For an open file this will return the current position of the read/write pointer in the file.
- LOF( #fnbr )
  Will return the current length of the file in bytes.
- MM.INFO(drive)
  Will return the current active drive – ie, "A:" or "B:"
- MM.INFO(free space)
  Will return how much space is left on the active drive
- MM.INFO(disk size)
  Will return the size of the active drive
- MM.INFO(exists file fname$)
  Will return true if the file exists
- MM.INFO(exists dir dirname$)
  Will return true if the directory exists
- MM.INFO(path)
  Will return the file path of the program being run. This allows the user to create path relative references for any resource files needed in a program.

## XModem Transfer

In addition to the standard method of XModem transfer which copies to or from the program memory the PicoMite firmware can also copy to and from a file on the Flash Filesystem or SD Card. The syntax is:

```
XMODEM SEND filename$
XMODEM RECEIVE filename$
```

Where ‘filename$’ is the file to save or send. ‘filename$’ can be a string expression, variable or constant. If it is a constant the string must be quoted (eg,, XMODEM SEND "prbas.bas"). In the case of receiving a file, any file with the same name will be overwritten.

There is also the ZMODEM command, which takes the same forms (ZMODEM SEND filename$ and ZMODEM RECEIVE filename$) and is faster and more reliable, and ZMODEM RECEIVE "*" receives any number of files at once. See the ZMODEM command for details.

## Load and Save Image

An image can be loaded from the Flash Filesystem or SD Card for display on an attached LCD display panel or VGA/HDMI monitor. This can be used to draw a logo or add a background on the display.

The syntax is:

```
     LOAD IMAGE filename$ [, StartX, StartY]  (load a BMP image)
or   LOAD JPG filename$ [, StartX] [, StartY] [, dither] [, xOffset] [, yOffset]
[, scale]
or   LOAD PNG filename$ [, StartX] [, StartY] [, transparent] [, alphacut] [,
xOffset] [, yOffset]    (Pico2/RP2350 only)
```

Where 'filename$' is the image to load and 'StartX'/'StartY' are the coordinates of the top left corner of the image (these are optional and will default to the top left corner of the display if not specified).

The LOAD JPG command accepts four additional optional parameters. 'dither' selects a dithering mode (0 to 7) which can improve the appearance of the image on displays with a limited colour depth. 'xOffset' and 'yOffset' specify a pixel offset into the source image so that only the lower right portion of a large image is displayed. 'scale' reduces the displayed size of the image by pixel averaging: use 2 to display at 50% size (each output pixel is the average of a 2x2 block of source pixels) or 4 to display at 25% size (the average of a 4x4 block). The default is 1 which displays the image at full size.

LOAD PNG accepts 'xOffset' and 'yOffset' in the same way, after its 'transparent' and 'alphacut' parameters (see LOAD PNG in the commands listing). With all three formats an image larger than the display is cropped to fit.

The image must be in the appropriate format (BMP, JPG or PNG) and MMBasic will add the extension to the file name if it is not specified. All types of images are supported including black and white and true colour 24_bit images.

The current image on the video output, virtual LCD or an LCD panel that supports BLIT can be saved to a file using the following command:

```
SAVE IMAGE filename$ [,StartX, StartY, width, height]
```

This will save the image, or part of the image, as a 24-bit true colour BMP file (the extension .BMP) will be added if an extension is not supplied.

## Example of Sequential I/O

In the example below a file is created and two lines are written to the file (using the PRINT command). The file is then closed.

```
OPEN "fox.txt" FOR OUTPUT AS #1
PRINT #1, "The quick brown fox"
PRINT #1, "jumps over the lazy dog"
CLOSE #1
```

You can read the contents of the file using the LINE INPUT command. For example:

```
OPEN "fox.txt" FOR INPUT AS #1
LINE INPUT #1,a$
LINE INPUT #1,b$
CLOSE #1
```

LINE INPUT reads one line at a time so the variable a$ will contain the text "The quick brown fox" and b$ will contain "jumps over the lazy dog".

Another way of reading from a file is to use the INPUT$() function. This will read a specified number of characters. For example:

```
OPEN "fox.txt" FOR INPUT AS #1
ta$ = INPUT$(12, #1)
tb$ = INPUT$(3, #1)
CLOSE #1
```

The first INPUT$() will read 12 characters and the second three characters. So the variable ta$ will contain "The quick br" and the variable tb$ will contain "own".

Files normally contain just text and the print command will convert numbers to text. So in the following example the first line will contain the line "123" and the second "56789".

```
nbr1 = 123 : nbr2 = 56789
OPEN "numbers.txt" FOR OUTPUT AS #1
PRINT #1, nbr1
PRINT #1, nbr2
CLOSE #1
```

You can read the contents of this file using the LINE INPUT command but then you would need to convert the text to a number using VAL().

For example:

```
OPEN "numbers.txt" FOR INPUT AS #1
LINE INPUT #1, a$
LINE INPUT #1, b$
CLOSE #1
x = VAL(a$) : y = VAL(b$)
```

Following this the variable x would have the value 123 and y the value 56789.

## Random File I/O

For random access the file should be opened with the keyword RANDOM. For example:

```
OPEN "filename" FOR RANDOM AS #1
```

To seek to a record within the file you would use the SEEK command which will position the read/write pointer to a specific byte. The first byte in a file is numbered one so, for example, the fifth record in a file that uses 64 byte records would start at byte 257. In that case you would use the following to point to it:

```
SEEK #1, 257
```

When reading from a random access file the INPUT$() function should be used as this will read a fixed number of bytes (i.e. a complete record) from the file. For example, to read a record of 64 bytes you would use:

```
dat$ = INPUT$(64, #1)
```

When writing to the file a fixed record size should be used and this can be easily accomplished by adding sufficient padding characters (normally spaces) to the data to be written. For example:

```
PRINT #1, dat$ + SPACE$(64 - LEN(dat$));
```

The SPACE$() function is used to add enough spaces to ensure that the data written is an exact length (64 bytes in this example). The semicolon at the end of the print command suppresses the addition of the carriage return and line feed characters which would make the record longer than intended.

Two other functions can help when using random file access. The LOC() function will return the current byte position of the read/write pointer and the LOF() function will return the total length of the file in bytes.

The following program demonstrates random file access. Using it you can append to the file (to add some data in the first place) then read/write records using random record numbers. The first record in the file is record number 1, the second is 2, etc.

```
RecLen = 64
OPEN "test.dat" FOR RANDOM AS #1

DO
 abort: PRINT
 PRINT "Number of records in the file =" LOF(#1)/RecLen
 INPUT "Command (r = read, w = write, a = append, q = quit): ", cmd$
 IF cmd$ = "q" THEN CLOSE #1 : END
 IF cmd$ = "a" THEN
    SEEK #1, LOF(#1) + 1
 ELSE
    INPUT "Record Number: ", nbr
    IF nbr < 1 or nbr > LOF(#1)/RecLen THEN PRINT "Invalid record" : GOTO abort
    SEEK #1, RecLen * (nbr - 1) + 1
 ENDIF
 IF cmd$ = "r" THEN
    PRINT "The record = " INPUT$(RecLen, #1)
 ELSE
```

```
      LINE INPUT "Enter the data to be written: ", dat$
      PRINT #1,dat$ + SPACE$(RecLen - LEN(dat$));
   ENDIF
   LOOP
```

Random access can also be used on a normal text file. For example, this will print out a file backwards:

```
OPEN "file.txt" FOR RANDOM AS #1
FOR i = LOF(#1) TO 1 STEP -1
   SEEK #1, i
   PRINT INPUT$(1, #1);
NEXT i
CLOSE #1
```

# Sound Output

The PicoMite firmware can play stereo WAV, FLAC, MP3 or MOD files located on the Flash Filesystem or SD Card and generate precise sine waves using the PLAY command.

Note that the switching power regulator on the Raspberry Pi Pico can cause some noise on the audio output. This can be reduced by disabling the regulator and powering the module via an external linear regulator.

## Pulse Width Modulated (PWM) Signal

The audio is created using PWM outputs so before the PLAY commands can be used the PWM output pins must be allocated as audio outputs:

This is done using the OPTION AUDIO command as follows:

```
OPTION AUDIO PWM-A-PIN, PWM-B-PIN
```

This command should be entered at the command prompt and will be saved, so it only needs to be run once. Both pins must be on the same PWM channel with PWM-A-PIN the left audio channel and PWM-B-PIN the right.

For example:

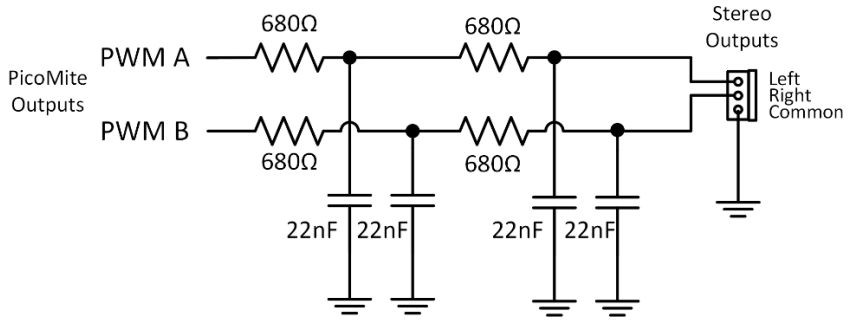
```
OPTION AUDIO GP0, GP1
```

The audio signal is superimposed on a 44KHz square wave (called the carrier wave) as a pulse width modulated (PWM) signal. This means that a low pass filter is required to remove the carrier and recover the audio signal.

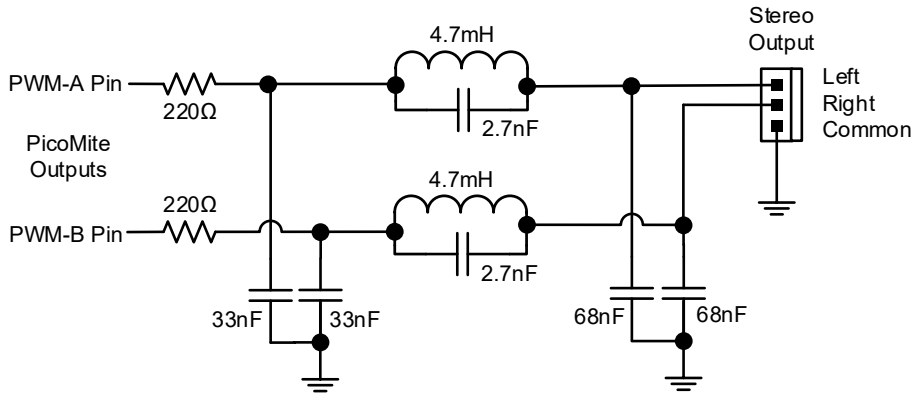
## Filter Circuits

Most low cost amplified speakers (for a personal computer) will not respond to the carrier frequency so they will act as a low pass filter in themselves. Therefore, if you want to keep it simple, you can directly connect the PWM output to an amplified speaker's input and a reasonable sound output should be achieved.

However, the high level of the 44KHz carrier may cause problems for the amplifier (eg, overheating or distortion) so the following filter is recommended. This removes most of the carrier and delivers about 2V peak to peak (0.6V RMS) with reasonable fidelity up to 8KHz (more than enough for most amplified speakers):




Below is a superior circuit producing quality audio with an insignificant amount of the carrier remaining. This is suitable for a more sophisticated HiFi amplifier/speaker configuration. The output is good for 10Hz to 15KHz at about 3V peak to peak (1V RMS) at 1KHz.




Both circuits are designed to feed an amplifier (not directly drive a headphone or speaker) and rely on capacitor coupling into the following amplifier (most have this).

## VS1053 support

The audio output can be generated using a VS1053 CODEC module which is configured using the command

```
OPTION AUDIO VS1053 CLKpin, MOSIpin, MISOpin, XCSpin, XDCSpin, DREQpin,XRSTpin
```

This requires no output filtering and can drive 32Ω headphones direct. It also supports additional playback capabilities.

If a VS1053 codec is used as the audio output device, additional commands are available:

```
PLAY MP3 file$, interrupt
PLAY MIDIFILE file$, interrupt
PLAY MIDI
PLAY MIDI CMD cmd%, data1% [,data2%]
PLAY NOTE ON channel, note, velocity
PLAY NOTE OFF channel, note [,velocity]
PLAY HALT
PLAY CONTINUE track$
PLAY STREAM buffer%(), readpointer%, writepointer%
```

These are explained in more detail in the commands listing section.

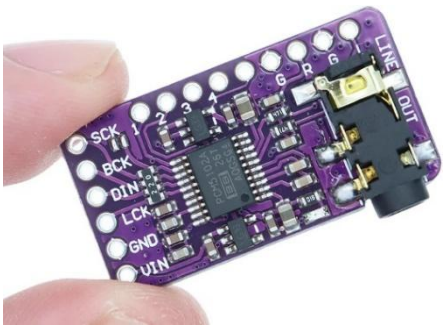
## MCP48n2 DAC

The audio output can also be generated through a connected MCP48n2 DAC (eg, MCP4822) in which case it is configured using the command:

```
OPTION AUDIO SPI CS-PIN, CLK-PIN, MOSI-PIN
```

The DAC does not need a complex low pass filter and a 120Ω resistor connected to the DAC output with the other end of the resistor connected to GND via a 100nF capacitor will be adequate. When a MCP4822 is used the LDAC pin on the DAC should be connected to GND.

## I2S DAC




The audio output can also be generated though an I2S DAC such as the PCM5102A. The DAC must support creating its own master clock as this is not created by the firmware. The I2S DAC on the Pico2 (RP2350A/B) uses PIO2 to generate the output so this will not be available when the I2S DAC is enabled. On the RP2350 WiFi and Bluetooth versions PIO2 is used by the wireless chip and the I2S DAC shares it, so PIO2 is never available on those versions. The I2S DAC on the RP2040 uses PIO 0 which is shared with the VGA PIO on VGA versions, except on the WebMiteRP2040 where it shares PIO 1 with the wireless chip.

The I2S DAC is configured with the command:

```
OPTION AUDIO I2S BCLK-PIN, DIN-PIN
```

The I2S word clock (LRCK) will then be on the next pin to the BCLK. So, for example if BCLK is set to GP0 then LRCK will be on GP1. Both these pins and the DIN pin must be unused when the command is issued. Typically, the DAC module will also require a GND and power pin to be connected (typically 5V)

The I2S DAC will produce CD quality audio with the frequency response perfectly flat from 20Hz to 20KHz. This works best with FLAC files with up to 96000Hz and 24-bit formats have been tested. However MP3 files can also deliver high quality output limited only by the inherent MP3 compression.

## Bluetooth Speakers and Headphones

On the PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3 versions the audio output can be sent to a Bluetooth speaker or headphones. This is selected with the command:

```
OPTION AUDIO BLUETOOTH
```

and the speaker is then connected with the BLUETOOTH CONNECT command. Everything that the PLAY command can produce is sent to the speaker. See the chapter Bluetooth Versions for the details.

Bluetooth and an I2S DAC are the only audio outputs on these versions - PWM, the MCP48n2 DAC and the VS1053 are not available.

## Playing WAV, FLAC, MP3 and MOD Files

The PLAY command can play a WAV, FLAC, MP3 (RP2350 only) or MOD file residing on the Flash Filesystem or SD Card to the sound output. It can be used to provide background music, add sound effects to programs and provide informative announcements.

The commands are:

```
PLAY WAV file$, interrupt
PLAY FLAC file$, interrupt
PLAY MODFILE file$, interrupt
PLAY MP3 file$, interrupt 'RP2350 only
```

'file$' is the name of the audio file to play. It must be on the Flash Filesystem or SD Card and the appropriate extension (eg .WAV) will be appended if missing. The audio will play in the background (ie, the program will continue without pause). 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing.

## Generating Sine Waves

The PLAY TONE command uses the audio output to generate sine waves with selectable frequencies for the left and right channels. This feature is intended for generating attention catching sounds but, because the frequency is very accurate, it can be used for many other applications. For example, signalling DTMF tones down a telephone line or testing the frequency response of loudspeakers.

The syntax of the command is:

```
PLAY TONE left, right, duration, interrupt
```

'left' and 'right' are the frequencies in Hz to use for the left and right channels.

The tone plays in the background (the program will continue running after this command) and 'duration' specifies the number of milliseconds that the tone will sound for. 'duration' is optional and if not specified the tone will continue until explicitly stopped or the program terminates. 'interrupt' (if specified) will be triggered when the duration has finished.

The specified frequency can be from 1 Hz to 20 KHz and is very accurate (it is based on a crystal oscillator). The frequency can be changed at any time by issuing a new PLAY TONE command.

## Using PLAY

It is important to realise that the PLAY command will generate the audio in the background. This allows a program (for example) to play the sound of a bell while continuing with its control function. Without the background facility the whole BASIC program would freeze while the sound was played.

However, generating the audio in the background has some subtle inferences which can trip up newcomers. For example, take the following program:

```
PLAY TONE 500, 500, 2000
END
```

You may expect the 500Hz tone to sound for 2 seconds but in practice it will not make any sound at all. This is because MMBasic will execute the PLAY TONE command (which will start generating the sound in the background) and then it will immediately execute the END command which will terminate the program and the background sound. This will happen so fast that nothing is heard.

Similarly the following program will not work either:

```
PLAY TONE 500, 500, 2000
PLAY TONE 300, 300, 5000
```

This is because the first command will set a 500Hz the tone playing but then the second PLAY command will immediately replace that with a 300Hz tone and following that the program will run off the end terminating the program (and the background audio), resulting in nothing being heard.

If you want MMBasic to wait while the PLAY command is doing its thing you should use suitable PAUSE commands. For example:

```
PLAY TONE 500, 500 : PAUSE 2000
PLAY TONE 300, 300 : PAUSE 5000
PLAY STOP
```

This applies to all versions of the PLAY command including PLAY WAV.

## Utility Commands

There are a number of commands that can be used to manage the sound output:

| | |
|---|---|
| `PLAY PAUSE` | Temporarily halt (pause) the currently playing file or tone. |
| `PLAY RESUME` | Resume playing a file or tone that was previously paused. |
| `PLAY NEXT` | Play the next WAV, MP3 or FLAC file in a directory |
| `PLAY PREVIOUS` | Play the previous WAV, MP3, or FLAC file in a directory |
| `PLAY STOP` | Terminate the playing of the file or tone, same as when the program ends. |
| `PLAY VOLUME L, R` | Set the volume to between 0 and 100 with 100 being the maximum volume. The volume will reset to the maximum level when a program is run. A logarithmic scale is used so that PLAY VOLUME 50,50 should sound half as loud as 100,100. |

## Specialised Audio Output

The PLAY SOUND command will generate an output based on a mixture of sine, square, etc waveforms. See the details in the command listing.

The PLAY BBC SOUND and PLAY BBC ENVELOPE commands are the BBC Micro's SOUND and ENVELOPE statements, with the same parameters, units and behaviour, so the tunes and sound effects of a BBC Micro program carry across with only the PLAY BBC prefix added. They give MMBasic a four channel sequencer: notes are queued on each channel with a duration and play in order without the program having to time them, chords can be started together, and an envelope can shape both the volume and the pitch of a note while it plays. Channels 1 to 3 are square wave tones and channel 0 is noise, as on the BBC. See the details in the command listing.

Pitch is in quarter semitones with 89 being A4 (440Hz) and 53 middle C, so each octave is 48 and the notes of the C major scale from middle C are 53, 61, 69, 73, 81, 89, 97 and 101. Durations are in twentieths of a second. For example, a scale, a chord and a note shaped by an envelope:

```
FOR p = 53 TO 101 STEP 4 : PLAY BBC SOUND 1, -15, p, 5 : NEXT p
PLAY BBC SOUND &H201, -10, 53, 20 : PLAY BBC SOUND &H202, -10, 69, 20 :
PLAY BBC SOUND &H203, -10, 81, 20
PLAY BBC ENVELOPE 1, 1, 4, -4, 4, 10, 10, 10, 126, -2, 0, -10, 120, 100
PLAY BBC SOUND 1, 1, 100, 30
```

Each channel queues up to 16 notes and, when a channel's queue is full, PLAY BBC SOUND waits for a slot, so a loop of PLAY BBC SOUND commands paces itself to the music. MM.INFO(BBC BUFFER) returns how many slots are free, so a program that must not be held up can check before it queues. Adding &H10 to the channel number flushes the queue and stops the current note so that a sound effect is heard at once, which is how a game fires a laser over whatever is playing:

```
PLAY BBC ENVELOPE 2, 1, -8, 0, 0, 20, 0, 0, 126, -8, 0, -20, 126, 100
PLAY BBC SOUND &H11, 2, 220, 4
```

The audio output is released automatically when every channel is silent and every queue is empty, so a tune can be followed by PLAY WAV or PLAY MP3 without a PLAY STOP in between, and MM.INFO(SOUND) returns BBC while it is playing. Envelope definitions survive PLAY STOP and are cleared with the program's variables by RUN.

# Using the I/O pins

The Raspberry Pi Pico has 26 input/output pins which can be controlled from within the BASIC program with 3 of these supporting a high speed ADC (Analog to Digital Converter).

An I/O pin is referred to by its pin number and this can be the number (eg,, 2) or its GP number (eg,, GP1). In addition for all commands using I/O pins other than the PORT command and function you can also use a string variable or string literal of the form “GPn”.

## Digital Inputs

A digital input is the simplest type of input configuration. If the input voltage is higher than 2.5V the logic level will be true (numeric value of 1) and anything below 0.65V will be false (numeric value of 0). The inputs use a Schmitt trigger input so anything in between these levels will retain the previous logic level.

Note that the maximum voltage on the RP2040 (ie, the Raspberry Pi Pico) I/O pins is 3.3V. Level shifting will be required if a device uses 5V levels for signalling. The Raspberry Pi Pico 2 using the RP2350 can tolerate 5V (while powered) so, in this case, level shifting is not required for signals up to 5V.

In your BASIC program you would set the input as a digital input and use the PIN() function to get its level. For example:

```
SETPIN GP4, DIN
IF PIN(GP4) = 1 THEN PRINT "High"
```

The SETPIN command configures pin GP4 as a digital input and the PIN() function will return the value of that pin (the number 1 if the pin is high). The IF command will then execute the command after the THEN statement if the input was high. If the input pin was low the program would just continue with the next line in the program.

The SETPIN command also recognises a couple of options that will connect an internal resistor from the input to either the supply or ground. This is called a "pullup" or "pulldown" resistor and is handy when connecting to a switch as it saves having to install an external resistor to place a voltage across the contacts. Due to a hardware issue with the RP2350 processor it is recommended that an external resistor of 8,2K or less be used if a pulldown is required on that processor.

## Analog Inputs

Pins marked as ADC can be configured to measure the voltage on the pin. The input range is from zero to 3.3V and the PIN() function will return the voltage. For example:

```
> SETPIN 31, AIN
> PRINT PIN(31)
 2.345
>
```

You will need a voltage divider if you want to measure voltages greater than 3.3V. For small voltages you may need an amplifier to bring the input voltage into a reasonable range for measurement.

The measurement uses 3.3V power supply to the CPU as its reference and it is assumed that this is exactly 3.3V. This value can be changed with the OPTION VCC command. In order to get the best possible reading, the analogue input is sampled 10 times. The values are then sorted and the top 2 and bottom 2 discarded and the remaining 6 averaged.

If you want the direct reading from the ADC you can use the raw mode by using the ARAW option to the SETPIN command:

```
SETPIN pinno,ARAW
```

In this case a value between 0 and 4095 will be returned based on a single sample.

The ADC commands provide an alternate method of recording analog inputs and are intended for high speed recording of many readings into an array.

## Counting Inputs

Any four pins can be used as counting inputs to measure frequency, period or just count pulses on the input. The pins used for this function can be configured using the OPTION COUNT command but, if not changed, will default to GP6, GP7, GP8 and GP9.

As an example, the following will print the frequency of the signal on pin GP7:

```
> SETPIN GP7, FIN
> PRINT PIN(GP7)
110374
>
```

In this case the frequency is 110.374 kHz.

By default the gate time is one second which is the length of time that MMBasic will use to count the number of cycles on the input and this means that the reading is updated once a second with a resolution of 1 Hz. By specifying a third argument to the SETPIN command it is possible to specify an alternative gate time between 10 ms and 100000 ms. Shorter times will result in the readings being updated more frequently but the value returned will have a lower resolution. The PIN() function will always scale the returned number as the frequency in Hz regardless of the gate time used.

For example, the following will set the gate time to 10ms with a corresponding loss of resolution:

```
> SETPIN GP7, FIN, 10
> PRINT PIN(GP7)
110300
>
```

For accurate measurement of signals less than 10 Hz it is generally better to measure the period of the signal. When set to this mode the PicoMite firmware will measure the number of milliseconds between sequential rising edges of the input signal. The value is updated on the low to high transition so if your signal has a period of (say) 100 seconds you should be prepared to wait that amount of time before the PIN() function will return an updated value.

The count pins can also count the number of pulses on their input. When a pin is configured as a counter (for example, `SETPIN 7,CIN`) the counter will be reset to zero and PicoMite firmware will then count every transition from a low to high voltage. The counter can be reset to zero again by executing `PIN(7) = 0`.

Counting inputs are accurate up to about 200KHz at the default processors frequency. A minimum pulse width of about 40nS is needed to trigger the counter. The RP2350 also has the option of configuring GP1 as an extremely fast frequency counting pin (see the SETPIN GP1, FFIN command)..

## Digital Outputs

All I/O pins can be configured as a digital output using the DOUT parameter to the SETPIN command. For example:

```
SETPIN GP15, DOUT
```

This means that when an output pin is set to logic low it will pull its output to zero and when set high it will pull its output to 3.3V. In MMBasic this is done with the PIN command. For example `PIN(GP15) = 0` will set pin GP15 to low while `PIN(GP15) = 1` will set it high.

## Pulse Width Modulation

The PWM (Pulse Width Modulation) command allows the PicoMite firmware to generate square waves with a program controlled duty cycle.

By varying the duty cycle you can generate a program controlled voltage output for use in controlling external devices that require an analog input (power supplies, motor controllers, etc). The PWM outputs are also useful for driving servos and for generating a sound output via a small transducer.

RP2040 The PWM outputs consists of up to 8 channels (numbered 0 to 7) with each channel having two outputs (A and B). For each channel the frequency can be selected and for each output a different duty cycle can be set. Up to 16 pins can be configured as PWM outputs using the SETPIN command.

RP2350 The RP2350 supports up to 12 PWM channels numbered 0 to 11) and up to 24 pins can be configured as PWM outputs using the SETPIN command.

See the section on 'Hardware Details' for which pins are allocated to which PWM channel and the PWM command entry for how to start and stop PWM output.

## Communications Interfaces (Serial, SPI and I²C)

These are described in the appendices at the rear of this manual. Before these interfaces can be used the pins that are to be used for the relevant signals must be configured using the SETPIN command.

Some devices such as an SD Card, LCD panels, touch, etc also use SPI or I2C interfaces and the pins used for these must similarly be configured using the OPTION SYSTEM command before they can be used.

## Interrupts

Interrupts are a handy way of dealing with an event that can occur at an unpredictable time. An example is when the user presses a button. In your program you could insert code after each statement to check to see if the button has been pressed but an interrupt makes for a cleaner and more readable program.

When an interrupt occurs MMBasic will execute a defined subroutine and when finished return to the main program. The main program is completely unaware of the interrupt and will carry on as normal.

Any I/O pin that can be used as a digital input can be configured to generate an interrupt using the SETPIN command with up to ten interrupts active at any one time. Interrupts can be set up to occur on a rising or falling digital input signal (or both) and will cause an immediate branch to the specified user defined subroutine. The target can be the same or different for each interrupt. Return from an interrupt is via the END SUB or EXIT SUB commands. Note that no parameters can be passed to the subroutine however within the interrupt calls to other subroutines and functions are allowed.

If two or more interrupts occur at the same time they will be processed in order of the interrupts as defined below. During the processing of an interrupt all other interrupts are disabled until the interrupt subroutine returns. During an interrupt (and at all times) the value of the interrupt pin can be accessed using the PIN() function.

Interrupts can occur at any time but they are disabled during INPUT statements. Also interrupts are not recognised during some long hardware related operations (eg, the TEMPR() function, LCD drawing commands, and SD access commands) although they will be recognised if they are still present when the operation has finished. When using interrupts the main program is completely unaffected by the interrupt activity unless a variable used by the main program is changed during the interrupt.

Because interrupts run in the background they can cause difficult to diagnose bugs. Keep in mind the following factors when using interrupts:

- Interrupts are only checked by MMBasic at the completion of each command, and they are not latched by the hardware. This means that an interrupt that lasts for a short time can be missed, especially when the program is executing commands that take some time to execute. Most commands will execute in under 15µs however some commands such as the TEMPR() function can take up to 200ms so it is possible for an interrupt to occur and vanish within this window and thus not be recognised.
- When inside an interrupt all other interrupts are blocked so your interrupts should be short and exit as soon as possible. For example, never use PAUSE inside an interrupt. If you have some lengthy processing to do you should simply set a flag and immediately exit the interrupt, then your main program loop can detect the flag and do whatever is required.
- The subroutine that the interrupt calls (and any other subroutines or functions called by it) should always be exclusive to the interrupt. If you must call a subroutine that is also used by an interrupt you must disable the interrupt first (you can reinstate it after you have finished with the subroutine).
- Remember to disable an interrupt when you have finished needing it – background interrupts can cause strange and non-intuitive bugs.

In addition to interrupts generated by the change in state of an I/O pin, an interrupt can also be generated by other sections of MMBasic including timers and communications ports and the above notes also apply to them.

The list of all these interrupts (in high to low priority ranking) is:

1. PID control loops
2. ON KEY individual
3. ON KEY general
4. ON PS2
5. PIO RX FIFO

6. PIO TX FIFO
7. PIO RX DMA completion
8. PIO TX DMA completion
9. GUI Int Down
10. GUI Int Up
11. Sprite collision
12. Static object collision (SPRITE STINTERRUPT)
13. WebMite: TCP receive
14. WebMite: MQTT complete
15. WebMite: UDP receive
16. USB Game Controller/USB or PS2 Mouse/Wii controller
17. ADC completion
18. I2C Slave Rx
19. I2C Slave Tx
20. I2C2 Slave Rx
21. I2C2 Slave Tx
22. WAV Finished
23. COM1: Serial Port
24. COM2: Serial Port
25. COM3: to COM6: Serial Ports (USB versions only)
26. COM7: Serial Port (Bluetooth host versions only)
27. IR Receive
28. Keypad
29. Interrupt command/CSub Interrupt
30. I/O Pin Interrupts in order of definition
31. Tick Interrupts (1 to 4 in that order)

As an example: If an ON KEY interrupt occurred at the same time as a COM1: interrupt the ON KEY interrupt subroutine would be executed first and then, when the interrupt subroutine finished, the COM1: interrupt subroutine would then be executed.

# Special Device Support

To make it easier for a program to interact with the external world the PicoMite firmware includes drivers for a number of common peripheral devices.

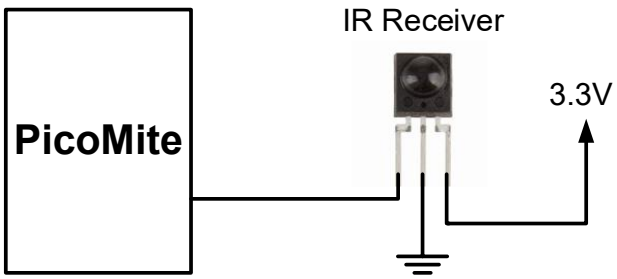
These are:

- Infrared remote control receiver and transmitter
- The DS18B20 temperature sensor and DHT22 temperature/humidity sensor
- LCD display modules
- Numeric keypads
- Battery backed clock
- Ultrasonic distance sensor
- WS2812 RGB LEDs

## Infrared Remote Control Decoder

You can easily add a remote control to your project using the IR command. When enabled this function will run in the background and interrupt the running program whenever a key is pressed on the IR remote control.

It will work with any NEC or Sony compatible remote controls including ones that generate extended messages. Most cheap programmable remote controls will generate either protocol and using one of these you can add a sophisticated flair to your Pico based project.




The NEC protocol is also used by many other manufacturers including Apple, Pioneer, Sanyo, Akai and Toshiba so their branded remotes can be used.

To detect the IR signal you need an IR receiver. NEC remotes use a 38kHz modulation of the IR signal and suitable receivers tuned to this frequency include the Vishay TSOP4838, Jaycar ZD1952 and Altronics Z1611A. Note that the I/O pins on the Raspberry Pi Pico are only 3.3V tolerant and so the receiver must be powered by a maximum of 3.3V. The Raspberry Pi Pico 2 is different and can withstand 5V.

NB: because of a H/W bug in the RP2350 before stepping version A4 it is highly recommended to add a 4K7 pullup resistor to the IR receiver data line when using the RP2350.

Sony remotes use a 40 kHz modulation but receivers for this frequency can be hard to find. Generally 38 kHz receivers will work but maximum sensitivity will be achieved with a 40 kHz receiver.

The IR receiver can be connected to any pin on the Raspberry Pi Pico. This pin must be configured by the program using the command:

```
SETPIN n, IR
```

where *n* is the I/O pin to use for this function.

To setup the decoder you use the command:

```
IR  dev, key, interrupt
```

Where *dev* is a variable that will be updated with the device code and *key* is the variable to be updated with the key code. *Interrupt* is the interrupt subroutine to call when a new key press has been detected. The IR decoding is done in the background and the program will continue after this command without interruption.

This is an example of using the IR decoder connected to the GP6 pin:

```
SETPIN GP6, IR                      ' define the pin to be used
DIM INTEGER DevCode, KeyCode        ' variables used by the decoder
IR DevCode, KeyCode, IRInt          ' start the IR decoder
DO
  ' < body of the program >
LOOP

SUB IRInt                           ' a key press has been detected
```

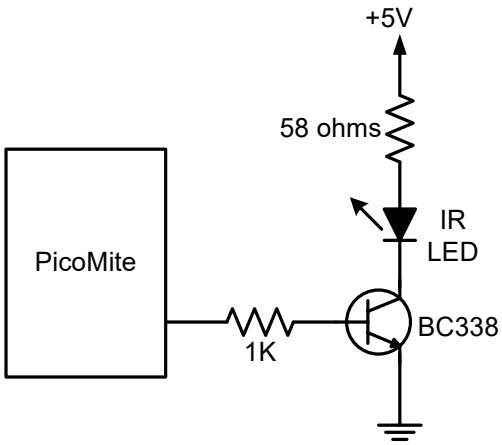
```
    PRINT "Received  device = " DevCode "  key = " KeyCode
END SUB
```

IR remote controls can address many different devices (VCR, TV, etc) so the program would normally examine the device code first to determine if the signal was intended for the program and, if it was, then take action based on the key pressed. There are many different devices and key codes so the best method of determining what codes your remote generates is to use the above program to discover the codes.

## Infrared Remote Control Transmitter

Using the IR SEND command you can transmit a 12 bit Sony infrared remote control signal. This is intended for Raspberry Pi Pico to Raspberry Pi Pico or Micromite communications but it will also work with Sony equipment that uses 12 bit codes. Note that all Sony products require that the message be sent three times with a 26 ms delay between each message.

The circuit on the right illustrates what is required. The transistor is used to drive the infrared LED because the output capability of the Raspberry Pi Pico is limited. This circuit provides about 50 mA to the LED.




To send a signal you use the command:

```
IR SEND pin, dev, key
```

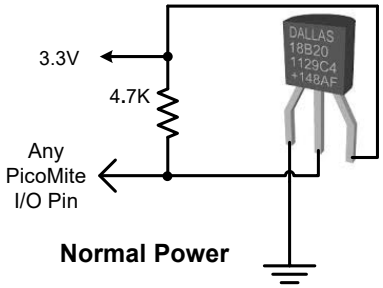
Where pin is the I/O pin used, dev is the device code to send and key is the key code. Any I/O pin on the Raspberry Pi Pico can be used and you do not have to set it up beforehand (IR SEND will automatically do that).

The modulation frequency used is 38 kHz and this matches the common IR receivers (described in the previous page) for maximum sensitivity when communicating between two Raspberry Pi Picos or with a Micromite.

## Measuring Temperature

The TEMPR() function will get the temperature from a DS18B20 temperature sensor. This device can be purchased on eBay for about US$5 in a variety of packages including a waterproof probe version.

The DS18B20 can be powered separately by a 3.3V supply or it can operate on parasitic power from the Raspberry Pi Pico as shown on the right. Multiple sensors can be used but a separate I/O pin and a 4.7K pullup resistor is required for each one.

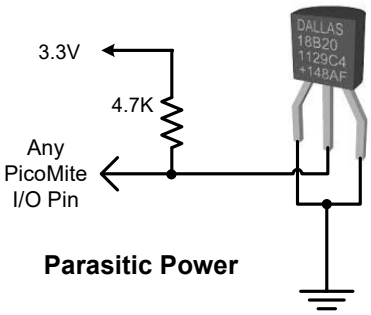



To get the current temperature you just use the TEMPR() function in an expression. For example:

```
PRINT "Temperature: " TEMPR(pin)
```

Where 'pin' is the I/O pin to which the sensor is connected. You do not have to configure the I/O pin, that is handled by MMBasic.

The returned value is in degrees C with a resolution of 0.25 ºC and is accurate to ±0.5 ºC. If there is an error during the measurement the returned value will be 1000.




The time required for the overall measurement is 200ms and the running program will halt for this period while the measurement is being made. This also means that interrupts will be disabled for this period. If you do not want this you can separately trigger the conversion using the TEMPR START command then later use the TEMPR() function to retrieve the temperature reading. The TEMPR() function will always wait if the sensor is still making the measurement.

For example:

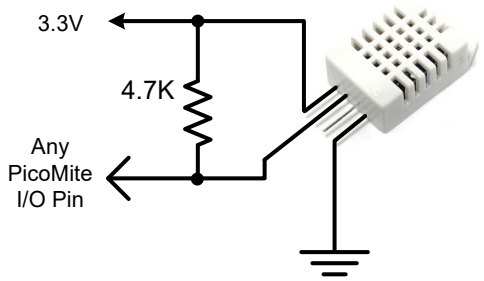
```
TEMPR START GP15
< do other tasks >
PRINT "Temperature: " TEMPR(GP15)
```

The TEMPR START command can also be used to change the resolution of the measurement (from the default 0.25 ºC) and the associated conversion time.

## Measuring Humidity and Temperature

The HUMID command will read the humidity and temperature from a DHT22 humidity/temperature sensor. This device is also sold as the RHT03 or AM2302 but all are compatible and can be purchased on eBay for under $5. The DHT11 sensor is also supported.

The DHT22 must be powered from 3.3V (or up to 5V with the Raspberry Pi Pico 2) and it should have a pullup resistor on the data line as shown. This is suitable for long cable runs (up to 20 meters) but for short runs the resistor can be omitted as the PicoMite firmware also provides an internal weak pullup.




To get the temperature or humidity you use the HUMID command with three arguments as follows:

```
HUMID pin, tVar, hVar [,DHT11]
```

Where 'pin' is the I/O pin to which the sensor is connected. The I/O pin will be automatically configured by MMBasic.

'tVar' is a floating point variable in which the temperature is returned and 'hVar' is a second variable for the humidity. The temperature is returned as degrees C with a resolution of one decimal place (eg, 23.4) and the humidity is returned as a percentage relative humidity (eg, 54.3).

If the optional 'DHT11' parameter is set to 1 then the command will use device timings suitable for that device. In this case the results will be returned with a resolution of 1 degree and 1% humidity

This example demonstrates using the DHT22 to display the current temperature and humidity every second:

```
DIM FLOAT temp, humidity
DO
  HUMID GP15, temp, humidity
  PRINT "The temperature is" temp " and the humidity is" humidity
  PAUSE 1000
LOOP
```

## Real Time Clock Interface

Using the RTC GETTIME command it is easy to get the current time from a PCF8563, DS1307, DS3231 or DS3232 real time clock as well as compatible devices such as the M41T11. These integrated circuits are popular and cheap, will keep accurate time even with the power removed and can be purchased for US$2 to $8 on eBay. Complete modules including the battery can also be purchased on eBay for a little more.

The PCF8563 and DS1307 will keep time to within a minute or two over a month while the DS3231 and DS3232 are particularly precise and will remain accurate to within a minute over a year.

Firmware V6.01.01 or later supports the RV3028 which boasts an accuracy of ±1 ppm (30 seconds in a year).

These chips are $I^2C$ devices and should be connected to the $I^2C$ I/O pins of the Raspberry Pi Pico.

Internal pullup resistors (100KΩ) are applied to the $I^2C$ I/O pins so, in many cases external resistors are not needed.

In order to enable the RTC you first need to allocate the $I^2C$ pins to be used using the command:

```
OPTION SYSTEM I2C  SDApin, SCLpin
```

The time used by the RTC must also be set. That is done with the RTC SETTIME command which uses the format :

```
RTC SETTIME year, month, day, hour, minute, second
```

Note that the hour must be in 24 hour format.

For example, the following will set the real time clock to 4PM on the 10th November 2025:

```
RTC SETTIME 2025, 11, 10, 16, 0, 0
```

To get the time you use the RTC GETTIME command which will read the time from the real time clock chip and set the clock inside the Raspberry Pi Pico. Normally this command will be placed at the beginning of the program or in the subroutine MM.STARTUP so that the time is set on power up.

The command OPTION RTC AUTO ENABLE can also be used to set an automatic update of the TIME$ and DATE$ read only variables from the real time clock chip on boot and every hour.

## Measuring Distance



Using a HC-SR04 ultrasonic sensor and the DISTANCE() function you can measure the distance to a target.

This device can be found on eBay for about US$4 and it will measure the distance to a target from 3cm to 3m. It works by sending an ultrasonic sound pulse and measuring the time it takes for the echo to be returned.

Compatible sensors are the SRF05, SRF06, Parallax PING and the DYP-ME007 (which is waterproof and therefore good for monitoring the level of a water tank). Others that have been reported as working well use the CS100 chip - such as the HC-SR04 and US-025.

In the PicoMite firmware you use the DISTANCE function as follows:

```
d = DISTANCE(trig, echo)
```

The value returned is the distance in centimetres to the target.

Where trig is the I/O pin connected to the "trig" input of the sensor and echo is the pin connected the "echo" output of the sensor. You can also use 3-pin devices and in that case only one pin number is specified.

Note that the maximum voltage on all the Raspberry Pi Pico's I/O pins is 3.3V. Level shifting will be required for this sensor because it uses 5V levels for its echo output. The Raspberry Pi Pico 2 can tolerate 5V (while powered) so, in this case, level shifting is not required.

## LCD Display



The LCD command will display text on a standard LCD module with the minimum of programming effort.

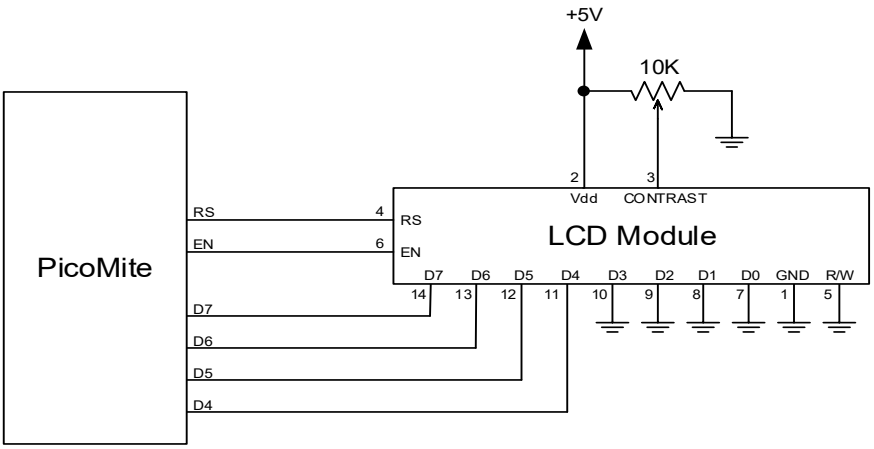
This command will work with LCD modules that use the KS0066, HD44780 or SPLC780 controller chip and have 1, 2 or 4 lines. Typical displays include the LCD16X2 (futurlec.com), the Z7001 (altronics.com.au) and the QP5512 (jaycar.com.au). eBay is another good source where prices can range from $10 to $50.

To setup the display you use the DEVICE LCD INIT command:

```
LCD INIT d4, d5, d6, d7, rs, en
```

d4, d5, d6 and d7 are the numbers of the I/O pins that connect to inputs D4, D5, D6 and D7 on the LCD module (inputs D0 to D3 and R/W on the module should be connected to ground). 'rs' is the pin connected to the register select input on the module (sometimes called CMD or DAT). 'en' is the pin connected to the enable or chip select input on the module.

Any I/O pins on the Raspberry Pi Pico can be used and you do not have to set them up beforehand (the LCD command automatically does that for you). The following shows a typical set up.




To display characters on the module you use the LCD command:

```
LCD line, pos, data$
```

Where 'line' is the line on the display (1 to 4) and 'pos' is the position on the line where the data is to be written (the first position on the line is 1). 'data$' is a string containing the data to write to the LCD display. The characters in 'data$' will overwrite whatever was on that part of the LCD.

The following shows a typical usage where d4 to d7 are connected to pins GP2 to GP5, rs is connected to pin GP6 and en to pin GP7.

```
LCD INIT GP2, GP3, GP4, GP5, GP6, GP7
LCD 1, 2, "Temperature"
LCD 2, 6, STR$(TEMPR(GP15))    ' DS18B20 connected to pin GP15
```

Note that this example also uses the TEMPR() function to get the temperature (described above).

In addition, the firmware supports the widely available LCD displays with the I2C backpack using the PCF8574 series I2C I/O expander chips. See the I2CLCD command for details.

## Keypad Interface

A keypad is a low tech but effective method of entering numeric data. The PicoMite firmware supports either a 4x3 keypad or a 4x4 keypad and the monitoring and decoding of key presses is done in the background. When a key press is detected an interrupt will be issued where the program can deal with it.

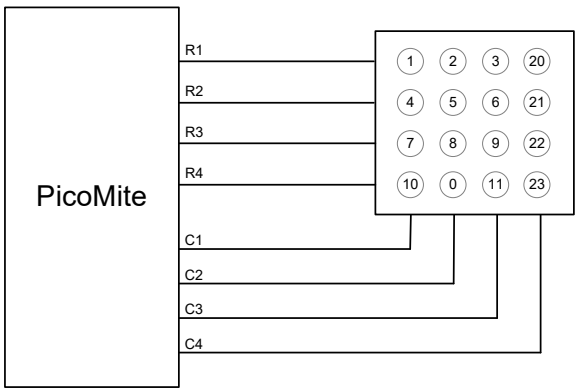
Examples of a 4x3 keypad and a 4x4 keypad are the Altronics S5381 and S5383 (go to www.altronics.com ).

To enable the keypad feature you use the command:

```
KEYPAD  var, int, r1, r2, r3, r4, c1, c2, c3, c4
```

to call when a new key press has been detected. ‘r1’, ‘r2’, ‘r3’ and ‘r4’ are the pin numbers used for the four row connections to the keypad (see the diagram below) and ‘c1’, ‘c2’, ‘c3’ and ‘c4’ are the column connections. ‘c4’ is only used with 4x4 keypads and should be omitted if you are using a 4x3 keypad.

Any I/O pins on the Raspberry Pi Pico can be used and you do not have to set them up beforehand, the KEYPAD command will automatically do that for you.




The detection and decoding of key presses is done in the background and the program will continue after this command without interruption. When a key press is detected the value of the variable var will be set to the number representing the key (this is the number inside the circles in the diagram above). Then the interrupt will be called.

For example:

```
Keypad KeyCode, KP_Int,GP2,GP3,GP4,GP5,GP6,GP7,GP8    ' 4x3 keybd
DO
  < body of the program >
LOOP

SUB KP_Int                              ' a key press has been detected
  PRINT "Key press = " KeyCode
END SUB
```

**See also the extended KEYPAD command in the command description that allows any number of rows and columns with user specified return codes.**

## WS2812 Support

The PicoMite firmware has built in support for the WS2812 multicolour LED chip. This chip needs a very specific timing to work properly and with the DEVICE WS2812 command it is easy to control these devices with minimal effort.

This command will output the required signals needed to drive a chain of WS2812 LED chips connected to the pin specified and set the colours of each LED in the chain. The syntax of the command is:

```
WS2812 type, pin, nbr%, colours%[()]
```

Note that the pin must be set to a digital output before this command is used. The colours%() array should be sized to have at least the same number of elements as the number of LEDs to be driven (nbr%). Each element in the array should contain the colour in the normal RGB888 format (0 - HFFFFFF). Where a single LED is to be driven then colours% should be a simple variable.

Up to 256 WS2812 chips in a string are supported.

'type' is a single character specifying the type of chip being driven as follows:

O = original WS2812

B = WS2812B

S = SK6812

W = SK6812W (RGBW)

As an example:

```
DIM b%(4)=(RGB(red), Rgb(green), RGB(blue), RGB(Yellow), rgb(cyan))
SETPIN GP5, DOUT
WS2812 O, GP5, 5, b%()
```

will output the specified colours to an array of five WS2812 LEDs daisy chained off pin GP5.

It is possible that a WS2812 will not work reliably with the 3.3V output from the Raspberry Pi Pico. In this case there are a number of solutions:

- Use the WS2812B which will work with a 3.3V supply and inputs.
- Use the Raspberry Pi Pico 2 which can tolerate 5V (while powered) so, in this case, level shifting is not required..
- Use a single WS2812 powered from 3.3V as a first stage to buffer the input of the first "real" LED in the string. The minimum supply for the WS2812 is 4V but in many cases it will work at 3.3V.

## OV7670 Camera module

The PicoMite firmware has support for a OV7670 camera module. See the CAMERA command for details

# Display Panels

**NOT AVAILABLE ON HDMI OR VGA VERSIONS**

The PicoMite firmware includes support for many LCD display panels using an SPI, I²C or parallel interface.

These commands must be entered at the command prompt (not in a program) and will cause the PicoMite firmware to restart. This has the side effect of disconnecting the USB console interface which will need to be reconnected. All types of display are configured with the OPTION LCDPANEL command using the various formats described below and can be disabled with OPTION LCDPANEL DISABLE

Note that the maximum voltage on all the Raspberry Pi Pico's I/O pins is 3.3V. Level shifting will be required for displays that use 5V levels for signalling. The Raspberry Pi Pico 2 can tolerate 5V (while powered) so, in this case, level shifting is not required.

## System SPI Bus

The system SPI bus is a dedicated SPI channel which is used by many LCD panels, all touch controllers and also to communicate with a SD Card. If any of these are attached the I/O pins used for the system SPI bus must be defined first.

This is done using this command:

```
OPTION SYSTEM SPI CLK-pin, MOSI-pin, MISO-pin
```

This must be entered at the command prompt and will cause the firmware to restart and disconnect the USB console interface, which will need to be reconnected. This option will be reapplied on startup and the pins will be reserved and not be available for other uses.

A typical example is:

```
OPTION SYSTEM SPI GP18, GP19, GP16
```

Note that the speed of drawing to SPI based displays and accessing SD cards is not affected by the CPU speed.

## SPI Based Display Panels

These display panels are configured using the following commands. All require that the system SPI bus (see above) has been defined first.

In all commands the parameters are:

| | |
|---|---|
| OR | This is the orientation of the display and it can be LANDSCAPE, PORTRAIT, RLANDSCAPE or RPORTRAIT. These can be abbreviated to L, P, RL or RP. The R prefix indicates the reverse or "upside down" orientation. |
| DC | Display Data/Command control pin. |
| RESET | Display Reset pin (when pulled low). |
| CS | Display Chip Select pin (active low). |
| BL | Optional pin used to control the backlight brightness using Pulse Width Modulation (PWM). |
| INVERT | This option will cause the colours to be inverted to compensate for a non standard panel. |

```
OPTION LCDPANEL ILI9341, OR, DC, RESET, CS [,BL] [,INVERT]
```

Initialises a TFT display using the ILI9341 controller. This supports 320 * 240 resolution. Displays using this controller are capable of transparent text and will work with the BLIT and BLIT READ commands.

```
OPTION LCDPANEL ILI9163, OR, DC, RESET, CS [,BL] [,INVERT]
```

Initialises a TFT display using the ILI9163 controller. This supports 128 * 128 resolution.

```
OPTION LCDPANEL ILI9481, OR, DC, RESET, CS [,BL] [,INVERT]
```

Initialises a TFT display using the ILI9481 controller. This supports 480 * 320 resolution.

```
OPTION LCDPANEL ILI9481IPS, OR, DC, RESET,CS [,BL][,INVERT]
```

Initialises an IPS display using the ILI9481 controller. This supports 480 * 320 resolution.

`OPTION LCDPANEL ILI9488, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a TFT display using the ILI9488 controller. This supports 480 * 320 resolution. Note that this controller has an issue with the LCD_SDO (MISO) pin which interferes with the touch controller. For this display to work the LCD_SDO pin must not be directly connected to the System SPI MISO.

`OPTION LCDPANEL ILI9488P, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a TFT display using the ILI9488 controller. This supports 320 * 320 resolution. Note that this controller has an issue with the LCD_SDO (MISO) pin which interferes with the touch controller. For this display to work the LCD_SDO pin must not be directly connected to the System SPI MISO.. This configuration supports the PicoCalc with the display in portrait mode.

`OPTION LCDPANEL ILI9488W, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a TFT display using the ILI9488 controller. This supports the Waveshare 3.5" display as used on their Pico Eval board and the normal 3.5" display adapter.

`OPTION LCDPANEL N5110, OR, DC, RESET, CS [,contrast] [,INVERT]`

Initialises a LCD display using the Nokia 5110 controller. This supports 84 * 48 resolution. An additional parameter 'contrast' may be specified to control the contrast of the display. Try contrast values between &HA8 and &HD0 to suit your display, default if omitted is &HB1

`OPTION LCDPANEL SSD1306SPI, OR, DC, RESET, CS [,offset] [,INVERT]`

Initialises a OLED display using the SSD1306 controller with an SPI interface. This supports 128 * 64 resolution. An additional parameter 'offset' may be specified to control the position of the display. 0.96" displays typically need a value of 0. 1.3" displays typically need a value of 2. Default if omitted is 0.

`OPTION LCDPANEL SSD1331, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a colour OLED display using the SSD1331 controller. This supports 96 * 64 resolution.

`OPTION LCDPANEL ST7735, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a TFT display using the ST7735 controller. This supports 160 * 128 resolution.

`OPTION LCDPANEL ST7735S, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a IPS display using the ST7735S controller. This supports 160 * 80 resolution.

`OPTION LCDPANEL ST7735S_W, OR, DC, RESET, CS [,BL][,INVERT]`

Initialises a Waveshare 128x128 ST7735S display. This supports 128 * 128 resolution.

`OPTION LCDPANEL ST7789, OR, DC, RESET, CS [,BL] [,INVERT]`

Initialises a IPS display using the 7789 controller. This supports 240 * 240 resolution.
NOTE: display boards without a CS pin are not currently supported in the PicoMite firmware unless modified.

`OPTION LCDPANEL ST7789_135, OR, DC, RESET,CS [,BL][,INVERT]`

Initialises a IPS display using the 7789 controller. This supports 240 * 135 resolution.
NOTE: display boards without a CS pin are not currently supported in the PicoMite firmware unless modified.

`OPTION LCDPANEL ST7789_320, OR, DC, RESET,CS [,BL][,INVERT]`

Initialises a IPS display using the 7789 controller. This type supports the 320 * 240 resolution display from Waveshare ( https://www.waveshare.com/wiki/Pico-ResTouch-LCD-2.8 ).
These are capable of transparent text and will work with the BLIT and BLIT READ commands.
NOTE: display boards without a CS pin are not currently supported in the PicoMite firmware unless modified.

```
OPTION LCDPANEL ST7796S, OR, DC, RESET,CS [,BL][,INVERT]
```
Initialises a IPS display using the ST7796S controller. This supports 480 * 320 resolution.
NOTE: In order for transparent test and blit to work properly the diode D1 on the back of the display should be bridged and it is recommended that J1 is also bridged to run on 3.3V

```
OPTION LCDPANEL ST7796SP, OR, DC, RESET,CS [,BL][,INVERT]
```
Initialises a TFT display using the ST7796S controller. This supports 320 * 320 resolution. This configuration supports the PicoCalc with the display in portrait mode.

```
OPTION LCDPANEL ST7365P, OR, DC, RESET,CS [,BL][,INVERT]
```
Initialises a TFT display using the ST7365P controller. This supports 320 * 320 resolution. Displays using this controller are capable of transparent text and will work with the BLIT and BLIT READ commands.

```
OPTION LCDPANEL GC9A01, OR, DC, RESET, CS [,BL] [,INVERT]
```
Initialises a IPS display using the GC9A01 controller. This supports 240 * 240 resolution.

```
OPTION LCDPANEL ST7920, OR, DC, RESET
```
Initialises a LCD display using the ST7920 controller. This supports 128 * 64 resolution. Note this display does not support a chip select so the SPI bus cannot be shared if this display is used.

## I²C Based LCD Panels

All I²C based display controllers use the system I²C pins as per the pinout for the specific device. Other I²C devices can share the bus subject to their addresses being unique.

To setup the system I²C bus use the command:

```
OPTION SYSTEM I2C sdapin, sclpin
```

If an I²C display is configured it will not be necessary to "open" the I²C port for an additional device (I2C OPEN), I2C CLOSE is blocked, and all I²C devices must be capable of 100KHz operation. The I²C bus speed is not affected by changes to the CPU clock speed

These panels are configured using the following commands. In all commands the parameters OR is the orientation of the display and it can be LANDSCAPE, PORTRAIT, RLANDSCAPE or RPORTRAIT. These can be abbreviated to L, P, RL or RP. The R prefix indicates the reverse or "upside down" orientation.

```
OPTION LCDPANEL SSD1306I2C, OR [,offset]
```
Initialises a OLED display using the SSD1306 controller with an I²C interface. This supports 128 * 64 resolution. An additional parameter offset may be specified to control the position of the display. 0.96" displays typically need a value of 0. 1.3" displays typically need a value of 2. Default if omitted is 0.

NB many cheap I²C versions of SSD1306 displays do not implement I²C properly due to a wiring error. This seems to be particularly the case with 1.3" variants.

The SSD1306I2C driver also works with SSD1315 and SH1106 controllers.

```
OPTION LCDPANEL SSD1306I2C32, OR
```
Initialises a OLED display using the SSD1306 controller with an I²C interface. This supports 128 * 32 resolution.

## 8-bit Parallel LCD Panels

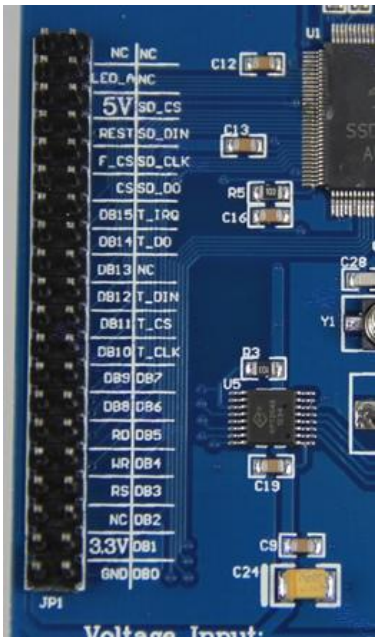


In addition to the SPI and I²C based controllers the PicoMite firmware supports LCD displays using the SSD1963 controller (as illustrated) and the ILI9341 controller.

These use a parallel interface, are available in sizes from 2.8" to 9" and have better specifications than the smaller displays. All these displays have an SD Card socket which is fully supported by MMBasic. On eBay you can find suitable displays by searching for the controller's name (eg SSD1963).

Because they use a parallel interface data can be transferred much faster than an SPI interface resulting in a very quick screen update.

The SSD1963 displays in particular are also much larger, have more pixels and are brighter. MMBasic can drive some of them using 24-bit true colour for a full colour rendition (16 million colours).




The characteristics of these displays are:

- A 2.8, 3.2, 4.3, 5, 7, 8 or 9 inch display
- Resolution of 320 x 240, 480 x 272 pixels (4.3" version) or 800 x 480 pixels (5", 7", 8" or 9" versions).
- An SSD1963 display controller or ILI9341 display controller with a parallel interface (8080 format)
- A touch controller (SPI interface).
- A full sized SD Card socket.

There are a number of different designs using the SSD1963 controller but fortunately most suppliers have standardised on a single connector as illustrated on the right.

It is strongly recommended that any display purchased has a matching connector – this provides some confidence that the manufacturer has followed the standard that the PicoMite firmware is designed to use.

## Connecting an 8-bit parallel LCD Panel

The controller uses a parallel interface while the touch controller and SD Card use an SPI interface. The touch and SD Card features are optional but if they are used they will use the second SPI port (SPI2).

The following table lists the connections required between the display board and the Raspberry Pi Pico to support the 8-bit parallel interface and the LCD display. The touch controller and SD Card interfaces are listed below.

| 8-bit parallel Display | Description | Raspberry Pi Pico |
|---|---|---|
| DB0 | Parallel Data Bus bit 0 | Pin 1/GP0 |
| DB1 | Parallel Data Bus bit 1 | Pin 2/GP1 |
| DB2 | Parallel Data Bus bit 2 | Pin 4/GP2 |
| DB3 | Parallel Data Bus bit 3 | Pin 5/GP3 |
| DB4 | Parallel Data Bus bit 4 | Pin 6/GP4 |
| DB5 | Parallel Data Bus bit 5 | Pin 7/GP5 |
| DB6 | Parallel Data Bus bit 6 | Pin 9/GP6 |
| DB7 | Parallel Data Bus bit 7 | Pin 10/GP7 |
| CS | Chip Select (active low) | Ground (ie, always selected) |
| WR | Write (active low) | Pin 19/GP14 |
| RD | Read (active low) | Pin 20/GP15 |

| | | |
|---|---|---|
| DC | Command/Data | Pin 17/GP13 |
| RESET | Reset the SSD1963 | Pin 21/GP16 |
| LED_A | Backlight control for an unmodified display panel | Configurable see OPTION LCDPANEL |
| 5V | 5V power for the backlight on some displays (most displays use the 3.3V supply for this). | |
| 3.3V | Power supply. | |
| GND | Ground | |

Pins DC, WR, RD, RESET can be allocated to other pins as a block of 4 using the optional parameter *DCpin*. On the RP2350B the optional parameter *DB0pin* can be used to specify the start pin for the 8 or 16 consecutive data pins used by the display.

The following table lists the connections required to support the touch controller interface:

| 8-bit parallel Display | Description | Raspberry Pi Pico |
|---|---|---|
| T_CS | Touch Chip Select | Recommended Pin 24/GP18 |
| T_IRQ | Touch Interrupt | Recommended Pin 25/GP19 |
| T_DIN | Touch Data In (MOSI) | Recommended Pin 15/GP11 |
| T_CLK | Touch SPI Clock | Recommended Pin 14/GP10 |
| T_DO | Touch Data Out (MISO) | Recommended Pin 16/GP12 |

The following table lists the connections required to support the SD Card connector:

| 8-bit parallel Display | Description | Raspberry Pi Pico |
|---|---|---|
| SD_CS | SD Card Chip Select | Recommended Pin 29/GP22 |
| SD_DIN | SD Card Data In (MOSI) | Recommended Pin 15/GP11 |
| SD_CLK | SD Card SPI Clock | Recommended Pin 14/GP10 |
| SD_DO | SD Card Data Out (MISO) | Recommended Pin 16/GP12 |

Where a connection is listed as "Recommended" this is only a suggestion and other pins can be used depending on the hardware configuration. Regardless, the specific pin should be specified in the appropriate OPTION command (see below).

Generally 7 inch and larger displays have a separate pin on the connector (marked 5V) for powering the backlight from a 5V supply. If this pin is not provided the backlight power will be drawn from the 3.3V pin. Note that the power drawn by the backlight can be considerable. For example, a 7 inch display will typically draw 330 mA from the 5V pin.

If the Pico's 3,3V output is used for powering a panel plus its backlight, it may easily require more current than the Pico can supply. Symptoms of a marginal supply could include TOUCH calibration failures or SD access failures. In this case an external 3.3V supply should be used.

The current drawn by the backlight can also cause a voltage drop on the LCD display panel's ground pin which can in turn shift the logic levels as seen by the display controller resulting in corrupted colours or text. An easy way of diagnosing this effect is to reduce the CPU speed to (say) 48MHz. If this fixes the problem it is a strong indication that this is the cause. Soldering power and ground wires direct to the LCD display panel's PCB is one workaround.

Care must be taken with display panels that share the SPI port between a number of devices (SD Card, touch, etc). In this case all the Chip Select signals must configured in MMBasic or disabled by a permanent connection to 3.3V. If this is not done the pin will float causing the wrong controller to respond to commands on the SPI bus.

In the PicoMite firmware either SPI channel can used to communicate with the touch controller and the SD Card interface as defined by the OPTION SYSTEM SPI setting. If this is set, that SPI channel will be unavailable to BASIC programs (which can use the other SPI channel).

## Configuring an 8-bit parallel LCD Panel

To use the display MMBasic must be configured using the OPTION LCDPANEL command which is normally entered at the command prompt. Every time the PicoMite firmware is restarted MMBasic will automatically initialise the display.

The syntax is:

OPTION LCDPANEL controller, orientation [,backlightpin] [,DCpin] [,NORESET] [,INVERT] [,DB0pin]

Where 'controller' can be either:

- SSD1963_4 For a 4.3 inch display
- SSD1963_5 For a 5 inch display
- SSD1963_5A For an alternative version of the 5 inch display if SSD1963_5 does not work
- SSD1963_7 For a 7 inch display
- SSD1963_7A For an alternative version of the 7 inch display if SSD1963_7 does not work.
- SSD1963_8 For 8 inch or 9 inch displays.
- ILI9341_8 For a 2.8" or 3.2" display

'orientation' can be LANDSCAPE, PORTRAIT, RLANDSCAPE or RPORTRAIT. These can be abbreviated to L, P, RL or RP. The R prefix indicates the reverse or "upside down" orientation.

'DCpin' is optional and is the Data/Command pin (previously called the RS pin). If this parameter is omitted the pin assignment will be as above in the table. If it is specified then DC, WR, RD and RESET pins will be assigned sequentially from DC pin.

The optional 'NORESET' parameter can be used to save a pin. In this case the reset pin should be tied high.

The optional 'INVERT' parameter specifies that the colours pallet should be inverted (needed for certain EsatRisng displays)

On the RP2350B processor the optional 'DB0pin' parameter can be used to specify the start pin for the 8 or 16 consecutive data pins used by the display.

This command only needs to be run once. From then on MMBasic will automatically initialise the display on startup or reset. In some circumstances it may be necessary to interrupt power to the LCD panel while the PicoMite firmware is running (eg, to save battery power) and in that case the GUI RESET LCDPANEL command can be used to reinitialise the display.

If the LCD panel is no longer required the command OPTION LCDPANEL DISABLE can be used which will return the I/O pins for general use.

To verify the configuration you can use the command OPTION LIST to list all options that have been set including the configuration of the LCD panel.

To test the display you can enter the command GUI TEST LCDPANEL. You should see an animated display of colour circles being rapidly drawn on top of each other. Press the space bar on the console's keyboard to stop the test. If the on-screen keyboard is enabled (HDMI USB and RP2350 VGA USB versions – see OPTION SCREEN KEYBOARD) you can also stop the test by double tapping the touch screen (two quick taps in the same place).

## 8 and 9 inch Displays

The controller configuration SSD1963_8 has only been tested with the 8 and 9 inch displays made by EastRising (available at www.buydisplay.com). These must be purchased as a TFT LCD panel with 8080 interface, 800x480 pixel LCD, SSD1963 display controller and XPT2046 touch controller. Note that the EastRising panels use a non-standard interface connector pin-out so you will need to refer to their data sheets when connecting these to the Raspberry Pi Pico. A suitable adapter to convert to the standard 40-pin connection can be purchased from: https://www.rictech.nz/micromite-products

## 16-bit Parallel LCD Panels

SSD1963 panels can also be enabled for 16-bit parallel operation. In this case, by default, pins GP0-GP15 are used for the data connections and, by default, pins GP16 to GP19 are used for the control signals DC, WR, RD and RESET.

For systems using the RP2350B only the data pins used can be selected by using the optional DB0pin parameter in the configuration command. To enable 16-bit operation append "_16" to the controller. Eg, SSD1963_4_16. The firmware also supports ILI9341, ILI9486, NT35510 and OTM8009A panels in 16-bit mode using the controller types ILI9341_16, ILI9486_16, and IPS_4_16 (supports both NT35510 and OTM8009A).

Valid 16-bit 'controllers' can be:

- SSD1963_4_16 For a 4.3 inch display
- SSD1963_5_16 For a 5 inch display
- SSD1963_5A_16 For an alternative version of the 5 inch display if SSD1963_5 does not work
- SSD1963_7_16 For a 7 inch display
- SSD1963_7A_16 For an alternative version of the 7 inch display if SSD1963_7 does not work.
- SSD1963_8_16 For 8 inch or 9 inch displays.
- ILI9341_16 For a 2.8" or 3.2" display
- ILI9486_16 Also supports NXP R61529
- IPS_4_16

## RP2350 Extended Display Support

The PicoMite RP2350 versions have between 320K and 364K bytes of heap memory available to the MMBasic programmer. This allows various additional features to be implemented using buffered display drivers.

These are available on the PicoMite RP2350 versions only - PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350BT and PicoMiteRP2350BTH. They are not available on WebMiteRP2350 or on any VGA or HDMI version. The following buffered display drivers are supported:

SPI

| | |
|---|---|
| ILI9341BUFF: | 320 x 240 |
| ST7796SPBUFF: | 320 x 320 |
| ST7365PBUFF: | 320 x 320 |
| ST7796SBUFF: | 480 x 320 |
| ILI9488PBUFF: | 320 x 320 |
| ILI9488BUFF: | 480 x 320 |
| ILI9488WBUFF: | 480 x 320 'Waveshare Pico-Restouch-lcd-3.5 |
| ST7789_320BUFF: | 320 x 240 'Waveshare Pico-Restouch-lcd-2.8 |

Parallel

| | |
|---|---|
| SSD1963_5_BUFF: | 400x240 (8-bit data bus) |
| SSD1963_7_BUFF: | 400x240 (8-bit data bus) |
| SSD1963_5_12BUFF: | 400x240 |
| SSD1963_7_12BUFF: | 400x240 |
| SSD1963_5_16BUFF: | 400x240 |
| SSD1963_7_16BUFF: | 400x240 |

In all cases enabling one of these drivers steals a RGB332 framebuffer from the general memory so in the worst case of a ILI9488BUFF general memory is reduced by 150K bytes (out of 320K to 364K bytes, depending on the version)

To use one of the SPI drivers you first need to allocate the SPI pins to be used:

```
OPTION LCD SPI clkpin, mosipin, misopin
```

These pins are dedicated to the display and must be separate from any pins used for the SYSTEM SPI which may still be required for touch and the sdcard (NB: in the case of the Waveshare boards using this driver will conflict with also using touch and the sdcard as they share the SPI channel). The reason for the dedicated pins is that the screen updates will all take place on the second processor and for maximum performance this cannot share the SPI channel. The LCD SPI pins can be disabled with OPTION LCD SPI DISABLE.

The format for the command to enable a SPI buffered driver is the same as any SPI driver

```
OPTION LCDPANEL controller, orientation, DCpin, RESETpin, CSpin [,BLpin] [,INVERT]
```

The SSD1963 drivers use 8, 12, or 16 data pins as indicated in the driver name.In all cases enabling one of these drivers steals a 400x240 RGB332 framebuffer from the heap memory. When using these drivers you can switch between the buffered driver and a conventional driver using the command MODE 800/400 which switches without rebooting between 800x480 and 400x240 modes. You can use the command OPTION LCD320 ON/OFF to centre a 320x240 image on the 400x240 display.

The format for the command to enable a SDD1963 buffered driver is the same as any SSD1963 driver

```
OPTION LCDPANEL controller, orientation [,backlightpin]  [,DCpin]
[,NORESET] [,INVERT] [,DB0pin]
```

The advantage of the buffered drivers is that the graphics commands in MMBasic just write to a memory buffer and then the update of the physical display takes place in the background on the second processor. This allows the Basic program to continue processing without waiting for the display. Of course, the use of a buffered driver does not significantly change the overall throughput to the physical display. So, continuous writes will have to wait for the display as they would for a standard driver. The big advantage comes where a program has a heavy processing loop where screen writes delay the processing. In this case as long as the overall screen update rate is manageable, the overhead on the processing loop will be drastically reduced.

## VGA222 drivers

The PICOMITE RP2350 and PICOMITEUSB RP2350 also support an enhanced VGA driver which operates in RGB222 mode giving 64 colours. This is configured with the command:

```
OPTION LCDPANEL driver, hsyncpin, bluelpin
```

Valid drivers are: VGA222_640, VGA222_320, VGA222_720, VGA222_360 with 640x480, 320x240, 720x400 and 360x200 VGA222 resolutions. Each driver name may also be written out in full as VGA222_640x480, VGA222_320x240, VGA222_720x400 or VGA222_360x200 - the short and long forms select the same driver.

The OPTION command checks for availability of hsyncpin and hsyncpin+1 (vsync) bluelpin - bluelpin+5 (blue_l, blue_h, green_l, green_h, red_l, red_h and assuming these are all free it sets up to run VGA222 output on the pins specified.

To connect the data pins to the VGA connector, use 390 and 820 ohm resistors from the H and l outputs respectively as per the green channel in the manual (page 30) RGB222 gives a much nicer colour rendition than the RGB121 available on the standard VGA version. Note, there are no modes. The 640x480, 320x240, 720x400 and 360x200 are different drivers and use different amounts of memory (much more in the case of 640x480 and 720x400). To generate the VGA output, the firmware uses PIO0 and PIO1 leaving PIO2 available (unless you also use the I2S audio driver.

## Backlight Control

For the ILI9163, ILI9341, ST7735, ST7735S, SSD1331, ST7789, ILI9481, ILI9488, ILI9488W, ST7789_135 ILI9341_8 and ST7789_320 displays an optional parameter ‘, backlight’ can be added to the end of the configuration parameters which specifies a pin to use to control the brightness of the backlight (LED_A). This will setup a PWM output on that pin with a frequency of 50KHz and an initial duty cycle of 99%.

You can then use the BACKLIGHT command to change the brightness between 0 and 100%. The PWM channel is blocked for normal PWM use and must not conflict with the PWM channel that may be set up for audio.

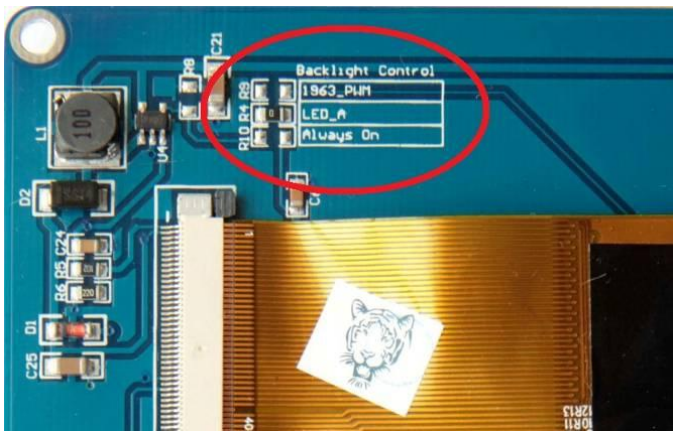
For example:

```
OPTION LCDPANEL ILI9341, OR, DC, RESET, CS, GP11
```

The backlight can then be set to 40% with this command:

```
BACKLIGHT 40
```

Most SSD1963 based LCD panels have three pairs of solder pads on the PCB which are grouped under the heading "Backlight Control" as illustrated on the right.




Normally the pair marked "LED-A" are shorted together with a zero ohm resistor and this allows control of the backlight's brightness with a PWM (pulse width modulated) signal on the LED-A pin of the display panel's main connector.

However, it is better to use the SSD1963 controller to generate this signal as it frees up one I/O pin and allows finer control.

To use the SSD1963 control, the zero ohm resistor should be removed from the pair marked "LED-A" and used to short the nearby pair of solder pads marked "1963-PWM". The brightness can then be set using the BACKLIGHT command via the SSD1963 controller (this is automatic, nothing needs to be configured).

## Touch Support

Many LCD panels are supplied with a resistive touch sensitive panel and associated controller chip. MMBasic fully supports this interface and this allows many of the physical knobs and switches used in a project to be implemented as on-screen controls activated by touch. As an alternative to resistive touch MMBasic also supports capacitive touch based on the FT6336 controller.

In all cases the LCD display panel itself should be configured and tested first, then the touch feature can be configured. The touch controller uses the SPI protocol for communications and for LCD panels using the SPI protocol this has normally been previously done using the OPTION SYSTEM SPI command.

For display panels using I²C or a parallel interface the OPTION SYSTEM SPI command must be separately used to define the system SPI bus for use by the touch controller. This command was discussed near the start of this chapter and defined in detail in the *Options* chapter of this manual.

For example, using the recommended pins for an 8-bit parallel display (described above) you would use:

```
OPTION SYSTEM SPI GP10, GP11, GP12
```

To use the touch facility MMBasic must be told that the touch controller is available on the system SPI bus by using the OPTION TOUCH command. This tells MMBasic what pins are used for the Chip Select and Interrupt signals.

As an example, for a typical ILI9341 display, this sets Chip Select to the GP12 pin and Interrupt to GP11:

```
OPTION TOUCH GP12, GP11
```

This must be entered at the command prompt and will cause the firmware to restart and disconnect the USB console interface, which will need to be reconnected.

When the PicoMite firmware is restarted MMBasic will automatically initialise the touch controller. To verify the configuration, you can use the command OPTION LIST to list all options that have been set including the configuration of the display panel and touch.

Care must be taken when the SPI port is shared between a number of devices (SD Card, touch, etc). In this case all the Chip Select signals must configured in MMBasic or alternatively disabled.

## Calibrating the Touch Screen

Before the touch facility can be used it must be calibrated using the GUI CALIBRATE command.




This command will present a target in the top left corner of the screen (as illustrated). Using a pointy but blunt object (such as a toothpick) press exactly on the centre of the target and hold it down for at least a second. MMBasic will record this location and then continue the calibration by sequentially displaying the target in the other three corners of the screen for touch and calibration.

The calibration routine may warn that the calibration was not accurate. This is just a warning and you can still use the touch feature if you wish but it would be better to repeat the calibration using more care.

Following calibration you can test the touch facility using the GUI TEST TOUCH command. This command will blank the screen and wait for a touch. When the screen is touched a white dot will be placed on the display marking the position of the touch on the screen. If the calibration was carried out successfully the dot should be displayed exactly under the location of the stylus on the screen. To exit the test routine you can press the space bar on the console's keyboard.

## Touch Functions

To detect if and where the screen is touched you can use the following functions in a BASIC program:

- TOUCH(X)
  Returns the X coordinate of the currently touched location or -1 if the screen is not being touched.
- TOUCH(Y)
  Returns the Y coordinate of the currently touched location or -1 if the screen is not being touched.
- TOUCH(DOWN)
  Returns 1 (true) if the screen is currently being touched or 0 (false) if it is not. TOUCH(UP) is the reverse.

Capacitive touch panels (FT6336 or GT911) and USB touch screens can report more than one simultaneous touch point and these can be read using TOUCH(X2), TOUCH(Y2), TOUCH(XN n) and TOUCH(YN n). The

firmware will also detect gestures such as taps, swipes, pinch and rotate (eg, TOUCH(TAP), TOUCH(SWIPE)). See the TOUCH() function in the Functions section of this manual for the full list.

### Touch Interrupts

An interrupt can be set on the IRQ pin number that was specified when the touch facility was configured. To detect touch down the interrupt should be configured as INTL (i.e., high to low). The interrupt can be cancelled with the command SETPIN pin, OFF.

The following program illustrates how the touch interrupt can be used. Whenever the screen is touched it will print on the console the coordinates of that touch. It assumes that the command OPTION TOUCH 7, 15 was used to initially configure the touch function:

```
SETPIN 15, INTL, MyInt    ' assumes that OPTION TOUCH 7, 15 was used
DO : LOOP

SUB MyInt                 ' subroutine called on a touch interrupt
  PRINT TOUCH(X) TOUCH(Y)
END SUB
```

### LCD Display as the Console Output

A PS2 or USB keyboard can be used with an LCD display panel to create a fully self contained computer that will boot direct to the MMBasic prompt. This feature works particularly well with 5 and 7 inch LCD panels using the SSD1963 controller. These can display a lot of text and update rapidly resulting in a user experience that is comparable to a PicoMite based computer using VGA or HDMI video output.

The LCD must be first configured using OPTION LCDPANEL. Then, to enable the console output on the LCD panel, you should use the following command:

```
OPTION LCDPANEL CONSOLE [font [, fc [, bc [, blight]]] [,NOSCROLL]
```

'font' is the default font, 'fc' is the default foreground colour, 'bc' is the default background colour and 'blight' is the default backlight brightness (2 to 100). These settings are saved in flash and are used to configure MMBasic at power up. They are all optional and default to font 2, white, black and 100%.

On displays where the framebuffer cannot be read, the firmware will automatically set the NOSCROLL option. When this is set and output reaches the bottom of the screen the screen is cleared and output continues again at the top. This also applies to the in-built editor. For SPI displays that can read the framebuffer (eg ILI9341) scrolling is very slow so the NOSCROLL parameter can be set to improve output and editing performance.

Note that this isn't necessary if the display is used in portrait mode as H/W scrolling is then used.

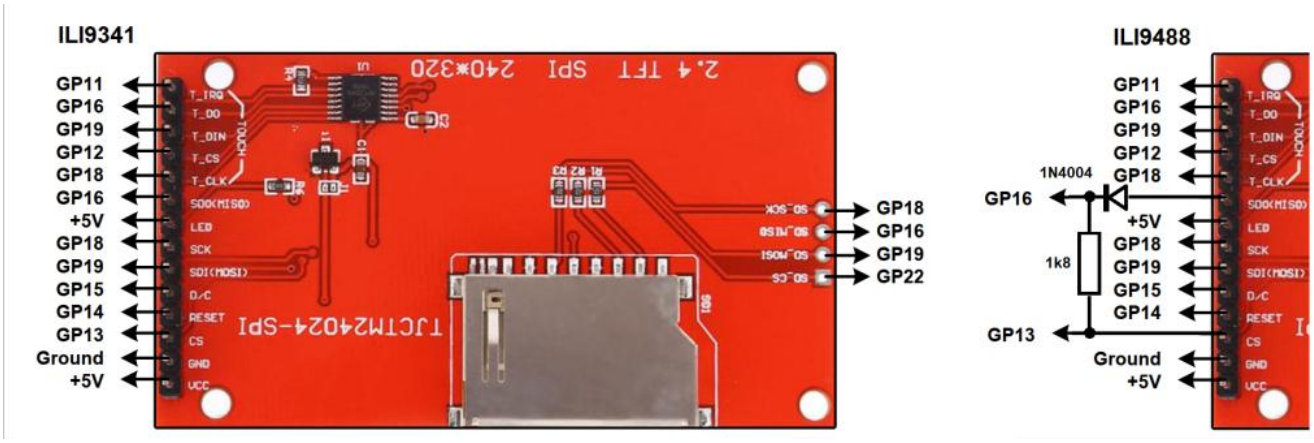
Colour coding in the editor is also turned on by this command (see the chapter *Full Screen Editor*). To disable using the LCD panel as the console the command is OPTION LCDPANEL NOCONSOLE.

### Example SPI LCD Panel Configuration

The following is a summary of how a typical LCD panel using an ILI9341 controller can be connected. This example supports the SD Card socket, the LCD display and the touch interface.

Typical panels can be found on ebay.com and similar sites by searching for the keyword "ILI9341". Make sure that the connections on the rear of the panel resemble that shown below:

The panel should be connected to the Raspberry Pi Pico as illustrated:




To match the above connections the following configuration commands should be entered, one by one at the command prompt:

```
OPTION SYSTEM SPI GP18, GP19, GP16
OPTION LCDPANEL ILI9341, L, GP15, GP14, GP13
OPTION TOUCH GP12, GP11
OPTION SDCARD GP22
```

These commands will be remembered and automatically applied on power up. Note that after each command is entered the firmware will restart, and the USB connection will be lost and must be reconnected.

Next the touch screen should be calibrated with:

```
GUI CALIBRATE
```

You can then test the various components.

The following will draw multiple colourful overlapping circles on the LCD screen which will confirm that the LCD is connected correctly:

```
GUI TEST LCDPANEL
```

The following will test the touch interface. When you touch the LCD screen a dot should appear on the screen at the exact point of the touch.

```
GUI TEST TOUCH
```

If this is not accurate you may have to run the GUI CALIBRATE command a second time taking greater care.

Finally, the following will list the files on the SD Card, if it executes without error you can be assured that the SD Card interface is good.

```
FILES
```

If you run into trouble getting the display to work it is worth disconnecting everything and clear the options with the command OPTION RESET so that you can start with a clean slate. Then reconnect it one stage at a time and configure and test each new stage as you progress. As you do this remember that any device connected to the SPI bus must be configured in MMBasic or its chip select line must be held logic high. This is because the voltage on an unconnected chip select line will float, possibly causing the wrong device to respond to signals intended for another device.

Also note that the ILI9341 controller is very sensitive to static discharge so, if the panel will not respond, it could easily be damaged and it would be worth testing with another panel.

# Graphics Functions

These commands and functions operate on attached LCD panels and VGA/HDMI video outputs.

## Supported Hardware

LCD Panels

The resolution and number of colours supported by an LCD panel is determined by the panel itself and the driver – see the chapter *Display Panels* for the details.

VGA Video

There are a number of modes which can be selected using the MODE command:

OPTION RESOLUTION 640x480

MODE 1 640x480 monochrome with RGB121 tiles, optional layer buffer
MODE 2 320x240 4-bit colour, optional layer buffer (RP2350 only) second optional layer buffer
MODE 3 640x480 4-bit colour, optional layer buffer (RP2350 only)

OPTION RESOLUTION 720x400

MODE 1 720x400 monochrome with RGB121 tiles, optional layer buffer
MODE 2 360x200 4-bit colour, optional layer buffer (RP2350 only) second optional layer buffer
MODE 3 720x400 4-bit colour, optional layer buffer (RP2350 only)

OPTION RESOLUTION 800x600 (RP2350 only)

MODE 1 800x600 monochrome with RGB121 tiles, optional layer buffer
MODE 2 400x300 4-bit colour, two optional layers
MODE 3 800x600 4-bit colour, optional layer buffer

OPTION RESOLUTION 848x480 (RP2350 only)

MODE 1 848x480 monochrome with RGB121 tiles, optional layer buffer
MODE 2 424x240 4-bit colour, two optional layers
MODE 3 848x480 4-bit colour, optional layer buffer

HDMI Video (RP2350 only)

Each HDMI resolution can operate in a number of modes which are set using the MODE command:

OPTION RESOLUTION 640x480

MODE 1 640x480x2-colours with RGB555, optional layer buffer
MODE 2 320x240x16colours and colour mapping to RGB555 palette, two optional layers
MODE 3 640x480x16 colours and colour mapping to RGB555 palette, optional layer buffer
MODE 4 320x240x32768 colours, optional layer buffer
MODE 5 320x240x256 colours and colour mapping to RGB555 palette, two optional layers

OPTION RESOLUTION 720x400

MODE 1 720x400 monochrome with RGB555 tiles, optional layer buffer
MODE 2 360x200 4-bit colour and colour mapping to RGB555 palette, two optional layers
MODE 3 720x400 4-bit colour and colour mapping to RGB555 palette, optional layer buffer
MODE 4 360x200x32768 colours, optional layer buffer
MODE 5 360x200x256 colours and colour mapping to RGB555 palette, two optional layers

OPTION RESOLUTION 800x600 (RP2350 only)

MODE 1 800x600 monochrome with RGB332 tiles, optional layer buffer
MODE 2 400x300 4-bit colour and colour mapping to RGB332 palette, optional layer buffer
MODE 3 800x600 4-bit colour and colour mapping to RGB332 palette, optional layer buffer
MODE 5 400x300x256 colours, optional layer buffer

OPTION RESOLUTION 848x480 (RP2350 only)

MODE 1 848x480 monochrome with RGB332 tiles, optional layer buffer
MODE 2 424x240 4-bit colour and colour mapping to RGB332 palette, optional layer buffer
MODE 3 848x480 4-bit colour and colour mapping to RGB332 palette, optional layer buffer
MODE 5 424x240x256 colours, optional layer buffer

OPTION RESOLUTION 1280x720

MODE 1 1280x720x2-colours with RGB332, optional layer buffer
MODE 2 320x180x16colours and colour mapping to RGB332 palette, two optional layers
MODE 3 640x360x16 colours and colour mapping to RGB332 palette, optional layer buffer
MODE 5 320x180x256 colours, two optional layers

OPTION RESOLUTION 1024x768

MODE 1 1024x768x2-colours with RGB332tiles, optional layer buffer
MODE 2 256x192x16colours and colour mapping to RGB332 palette, two optional layers
MODE 3 512x384x16 colours and colour mapping to RGB332 palette, optional layer buffer
MODE 5 256x192x256 colours, two optional layers

OPTION RESOLUTION 1024x600

MODE 1 1024x600x2-colours with RGB332 tiles, optional layer buffer
MODE 2 256x150x16 colours and colour mapping to RGB332 palette, two optional layers
MODE 3 512x300x16 colours and colour mapping to RGB332 palette, optional layer buffer
MODE 5 256x150x256 colours, two optional layers

OPTION RESOLUTION 800x480 (NB: reduces heap memory available)

MODE 1 800x480x2-colours with RGB332 tiles, optional layer buffer
MODE 2 400x240x16 colours and colour mapping to RGB332 palette, optional layer buffer
MODE 3 800x480x16 colours and colour mapping to RGB332 palette, optional layer buffer
MODE 5 400x240x256 colours, optional layer buffer

## Colours

Colour is specified as a true colour 24 bit number where the top eight bits represent the intensity of the red colour, the middle eight bits the green intensity and the bottom eight bits the blue. The easiest way to generate this number is with the RGB() function which has the form:

```
RGB(red, green, blue)
```

The RGB() function also supports a shortcut where you can specify common colours by naming them. For example, RGB(red) or RGB(cyan). The colours that can be named using the shortcut form are white, black, blue, green, cyan, red, magenta, yellow, brown, white, orange, pink, gold, salmon, beige, lightgrey and grey (or USA spelling gray/lightgray).

MMBasic will automatically translate all colours to the format required by the individual display controller. For example, in the case of the ILI9341 LCD controller, is 64K colours in the 565 format.

The default for commands that require a colour parameter can be set with the COLOUR command (can also be spelt COLOR). This is handy if your program uses a consistent colour scheme, you can then set the defaults and use the short version of the drawing commands throughout your program.

The COLOUR command takes the format: `COLOUR foreground-colour, background-colour`

## Screen Coordinates

All coordinates and measurements on the screen are done in terms of pixels with the X coordinate being the horizontal position and Y the vertical position. The top left corner of the screen has the coordinates X = 0 and Y = 0 and the values increase as you move down and to the right of the screen.

There are four read only variables which provide useful information about the display currently connected:

- MM. HRES
  Returns the width of the display (the X axis) in pixels.
- MM.VRES
  Returns the height of the display (the Y axis) in pixels.

- MM.INFO(FONTHEIGHT)
  Returns the height of the current default font (in pixels). All characters in a font have the same height.
- MM.INFO(FONTWIDTH)
  Returns the width of a character in the current font (in pixels). All characters have the same width.

## Fonts

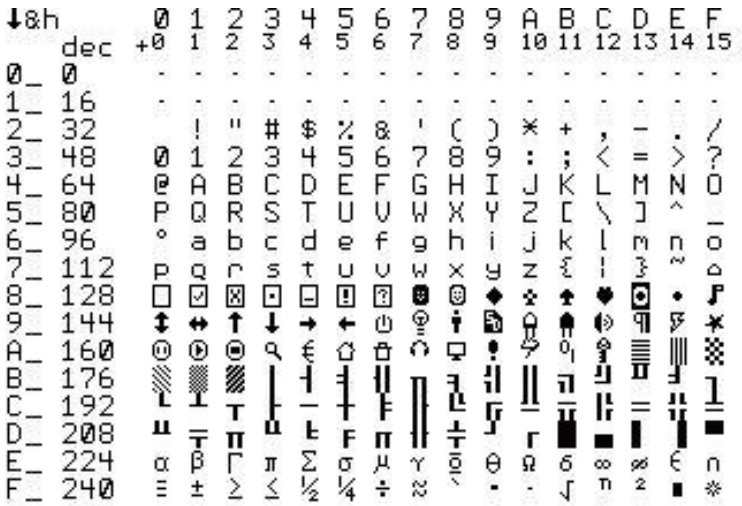
There are eight built in fonts. These are:

| Font Number | Size (width x height) | Character Set | Description |
|---|---|---|---|
| 1 | 8 x 12 | All 95 ASCII characters plus 7F to FF (hex) | Standard font (default on start-up). |
| 2 | 12 x 20 | All 95 ASCII characters | Medium sized font. |
| 3 | 16 x 16 | All 95 ASCII characters | A large font for VGA versions. |
| 3 | 16 x 24 | All 95 ASCII characters | A large font for HDMI versions and LCD panels. |
| 4 | 10x16 | All 95 ASCII characters plus 7F to FF (hex) | A font with extended graphic characters. Suitable for high resolution displays. |
| 5 | 24 x 32 | All 95 ASCII characters | Extra large font, very clear. |
| 6 | 32 x 50 | 0 to 9 plus some symbols | Numbers plus decimal point, positive, negative, equals, degree and colon symbols. Very clear. |
| 7 | 6 x 8 | All 95 ASCII characters | A small font useful when low resolutions are used. |
| 8 | 4 x 6 | All 95 ASCII characters | An even smaller font. |

Note that font 3 has a 16 x 16 pixel size when used with VGA video output but a 16 x 24 size for HDMI and LCD panels.

In all fonts (including font #6) the back quote character (60 hex or 96 decimal) has been replaced with the degree symbol (º).

Font #1 (the default font) and font #4 have an extended character set covering all characters from CHR$(32) to CHR$(255) or 20 to FF (hex) as illustrated on the right.



## Embedded Fonts

If required, additional fonts can be embedded in a BASIC program. These fonts work exactly same as the built in font (i.e. selected using the FONT command or specified in the TEXT command).

The format of an embedded font is:

```
DefineFont #Nbr
    hex [[ hex[…]
    hex [[ hex[…]
END DefineFont
```

It must start with the keyword "DefineFont" followed by the font number (which may be preceded by an optional # character). Any font number in the range of 2 to 5 and 8 to 16 can be specified and if it is the same as a built in font it will replace that font.

The body of the font is a sequence of 8-digit hex words with each word separated by one or more spaces or a new line. The font definition is terminated by an "End DefineFont " keyword. These can be placed anywhere in a program and MMBasic will skip over it. This format is the same as that used by the Micromite.

Additional fonts and information can be found in the Embedded Fonts folder in the PicoMite firmware download. These fonts cover a wide range of character sets including a symbol font (Dingbats) which is handy for creating on screen icons, etc.

## Drawing Commands

There are ten basic drawing commands that you can use within MMBasic programs to draw graphics. Most of these have optional parameters. You can completely leave these off the end of a command or you can use two commas in sequence to indicate a missing parameter. For example, the fifth parameter of the LINE command is optional so you can use this format:

```
LINE 0, 0, 100, 100, , rgb(red)
```

Optional parameters are indicated below by italics, for example: *font*.

In the following commands C is the drawing colour and defaults to the current foreground colour. FILL is the fill colour which defaults to -1 which indicates that no fill is to be used.

The basic drawing commands are:

- CLS *C*
  Clears the screen to the colour C. If C is not specified the current default background colour will be used.
- PIXEL X, Y, *C*
  Illuminates a pixel. If C is not specified the current default foreground colour will be used.
- LINE X1, Y1, X2, Y2, *LW, C*
  Draws a line starting at X1 and Y1 and ending at X2 and Y2.
  LW is the line's width and is only valid for horizontal or vertical lines. It defaults to 1 if not specified or if the line is a diagonal. There is an extended version for diagonal lines (see LINE AAA).
- BOX X, Y, W, H, *LW, C, FILL*
  Draws a box starting at X and Y which is W pixels wide and H pixels high.
  LW is the width of the sides of the box and can be zero. It defaults to 1.
- RBOX X, Y, W, H, *R, C, FILL*
  Draws a box with rounded corners starting at X and Y which is W pixels wide and H pixels high.
  R is the radius of the corners of the box. It defaults to 10.
- CIRCLE X, Y, R, *LW, A, C, FILL*
  Draws a circle with X and Y as the centre and a radius R. LW is the width of the line used for the circumference and can be zero (defaults to 1). A is the aspect ratio which is a floating point number and defaults to 1. For example, an aspect of 0.5 will draw an oval where the width is half the height.
- TEXT X, Y, STRING, *ALIGNMENT, FONT, SCALE, C, BC*
  Displays a string starting at X and Y. ALIGNMENT is 0, 1 or 2 characters (a string expression or variable is also allowed) where the first letter is the horizontal alignment around X and can be L, C or R for LEFT, CENTER or RIGHT aligned text and the second letter is the vertical alignment around Y and can be T, M or B for TOP, MIDDLE or BOTTOM aligned text. The default alignment is left/top. An additional code letter can be used to rotate the text (see below for the details). FONT and SCALE are optional and default to that set by the FONT command. C is the drawing colour and BC is the background colour. They are optional and default to that set by the COLOUR command.
- GUI BITMAP X, Y, BITS, *WIDTH, HEIGHT, SCALE, C, BC*
  Displays the bits in a bitmap starting at X and Y. HEIGHT and WIDTH are the dimensions of the bitmap as displayed on the LCD panel and default to 8x8. SCALE, C and BC are the same as for the TEXT command. The bitmap can be an integer or a string variable or constant and is drawn using the first byte as the first bits of the top line (bit 7 first, then bit 6, etc) followed by the next byte, etc. When the top line has been filled the next line of the displayed bitmap will start with the next bit in the integer or string.
- POLYGON n, xarray%(), yarray%() [, bordercolour] [, fillcolour]
  Draws a filled or outline polygon with n xy-coordinate pairs in xarray%() and yarray%(). If 'fillcolour' is omitted then just the polygon outline is drawn. If 'bordercolour' is omitted then it will default to the current default foreground colour.
- ARC x, y, r1, [r2], a1, a2 [, c]
  Draws an arc of a circle with a given colour and width between two radials (defined in degrees). Parameters for the ARC command are the x and y coordinates of the centre of the arc, the inner and outer radii, the start and end angles of the arc and the colour of the arc. The zero degrees reference is at the 12 o'clock position

## Rotated Text

As described above the alignment of the text in the TEXT command can be specified by using one or two characters in a string expression for the third parameter of the command. In this string you can also specify a third character to indicate the rotation of the text.

This character can be one of:

N for normal orientation

V for vertical text with each character under the previous running from top to bottom.

I the text will be inverted (i.e. upside down)

U the text will be rotated counter clockwise by 90º

D the text will be rotated clockwise by 90º

As an example, the following will display the text "LCD Display" vertically down the left hand margin of the display panel and centred vertically:

```
TEXT 0, 250, "LCD Display", "LMV", 5
```

Positioning is relative to the top left corner of the character when viewed normally so inverted 100,100 will have the top left pixel of the first character at 100,100 and the text will then be above y=101 and to the left of x=101. Similarly, “R” in the alignment string is viewed from the perspective of the character in whatever orientation it is in (not the screen).

## Transparent Text

The VGA or HDMI video output or LCD displays using the SSD1963, ST7796S, ILI9341, ST7789_320, or ILI9488 with MISO connected are capable of transparent text.

In this case the TEXT command will allow the use of -1 for the background colour. This means that the text is drawn over the background with the background image showing through the gaps in the letters.

## Framebuffers and Layers

All variants of the firmware can create one or two in memory framebuffers and one or two layer buffers (this is memory dependent). These are areas of memory with the same width and height as the main display. In the case of HDMI and VGA displays they will have the same colour depth as the current mode. In the case of LCD displays, they will have 4-bits per pixel (16 colours).

Depending on the version of the firmware and current display mode, creation of framebuffers or layer buffers will either use pre-allocated memory or allocate memory from user memory.

Framebuffers can be used to construct image data that can be copied to the physical display. Layer buffers are typically used to create partial images that can sit on top of a background display image and provide an efficient method of moving display elements over a static background.

All standard graphics drawing commands can be used on a framebuffer or layer buffer in the same way as if writing to the physical display. The FRAMEBUFFER WRITE command is used to direct the destination of the graphic output using a *code*.

The *code* is a single character which can be:

N The physical output device.

F The framebuffer.

2 A second framebuffer (RP2350 only)

L The layerbuffer

T A second layerbuffer (RP2350 only)

The basic framebuffer commands are:

```
FRAMEBUFFER CREATE  ' code F
FRAMEBUFFER LAYER   ' code L
FRAMEBUFFER CLOSE
FRAMEBUFFER WRITE code
FRAMEBUFFER COPY code1, code2 [,B]
```

See the detail command descriptions for addition framebuffer commands.

In the case of VGA and HDMI versions of the firmware, depending on the display mode and CPU speed (>=252MHz), layers are automatically applied on top of the main display image as it is output to the screen. In

the case of LCD displays the FRAMEBUFFER MERGE command is used to create the final image from a framebuffer and a layer buffer. The game PETSCII robots shows how this technique can be used to great effect.

The automatic application of a layer buffer is implemented in VGA versions mode 2 and mode 3 (RP2350 only) as well as HDMI modes 2, 3, 4, and 5. Two layer buffers are only available on the RP2350, and only in VGA mode 2 and HDMI modes 2 and 5. The second layer is not applied at the HDMI resolutions 800x600, 848x480 and 800x480, nor in mode 5 on the PicoMiteHDMIWEB version at 640x480 and 720x400. Where the second layer is not applied the T buffer simply acts as another framebuffer.

## BLIT and Sprite Commands

In previous versions of the firmware the blit and sprite commands were synonyms for the same functionality. In release 6.00.00 onwards they are separate commands. The distinction is that BLIT is a simple memory operation copying to and from a display or memory to a display or memory. Sprites are more complex and allow the programmer to display elements over a background and then move them over the background without corrupting the background image. In addition, the programmer can use the sprite functionality to detect collisions between sprites and between a sprite and the edges of the display.

Sprites cannot be used unless the display supports reading from its framebuffer and blit functionality is also limited unless this is the case. Sprites are enabled for all versions of the firmware when used on a an in-memory framebuffer and VGA and HDMI versions of the firmware directly with the screen.

Sprites are always stored as RGB121 nibbles with 2 pixels to a byte. In contrast BLIT buffers are stored as RGB888 values and so can be used with full colour LCD displays. Of course this comes at the expense of significantly greater memory usage.

See the SPRITE command and function together with Appendix G for more information on using sprites.

If the display is capable of transparent text the BLIT command allows a portion of the image currently showing on the display to be copied to a memory buffer and later copied back to the display. This is useful when something needs to be drawn over the background and later removed without damaging the image in the background. Examples include a game where a character is moving about in front of a landscape or the moving needle of a photorealistic gauge.

The available standard blit commands are:

```
BLIT READ #b, x, y, w, h
BLIT WRITE #b, x, y [,mode]
BLIT LOAD #b, f$, x, y, w, h
BLIT CLOSE #b
```

#b is the buffer number in the range of 1 to 64. x and y are the coordinates of the top left corner and w and h are the width and height of the image. READ will copy the display image to the buffer, WRITE will copy the buffer to the display and CLOSE will free up the buffer and reclaim the memory used. LOAD will load an image file into the buffer.

BLIT LOAD and BLIT WRITE will work on any display while BLIT and BLIT READ will only work on displays capable of transparent text (i.e. using the SSD1963, ILI9341, ST7789_320, or ILI9488 with MISO connected) as well as VGA and HDMI displays and any in-memory framebuffers

These commands can be used to copy a portion of the display to another location (by copying to a buffer then writing somewhere else) but a simpler method is to use an alternative version of the BLIT command as follows:

```
BLIT x1, y1, x2, y2, w, h
```

BLIT RESIZE src, dst, sx, sy, sw, sh, dx, dy, dw, dh

This command scales a source region to a destination region using nearest-neighbour sampling (fast integer scaling).

Source coordinates and size are rigorously validated and must be fully within 0..MM.HRES-1 and 0..MM.VRES-1 (otherwise Error 21).

Destination coordinates are clipped to the visible display area. If no destination pixels remain after clipping, no drawing occurs.

If either src or dst is N (display buffer), the current mode must be RGB121 (mode 2 or mode 3).

To preserve correctness with same-buffer overlap, a temporary source copy is used only when source and destination overlap; non-overlap uses direct read/write.

This will copy a portion of the image at x1/y1 to the location x2/y2. w and h specify the width and height of the image to be copied. The source and destination areas can overlap and the BLIT command will perform the copy correctly.

This form of the BLIT command is particularly useful for creating graphs that can scroll horizontally or vertically as new data is added.

In addition, the firmware provides BLIT MEMORY, BLIT COMPRESSED, BLIT FRAMEBUFFER, and BLIT MERGE commands. These advanced commands can be used to help code games with hundreds of display elements such as the port to MMBasic of PETSCII robots.

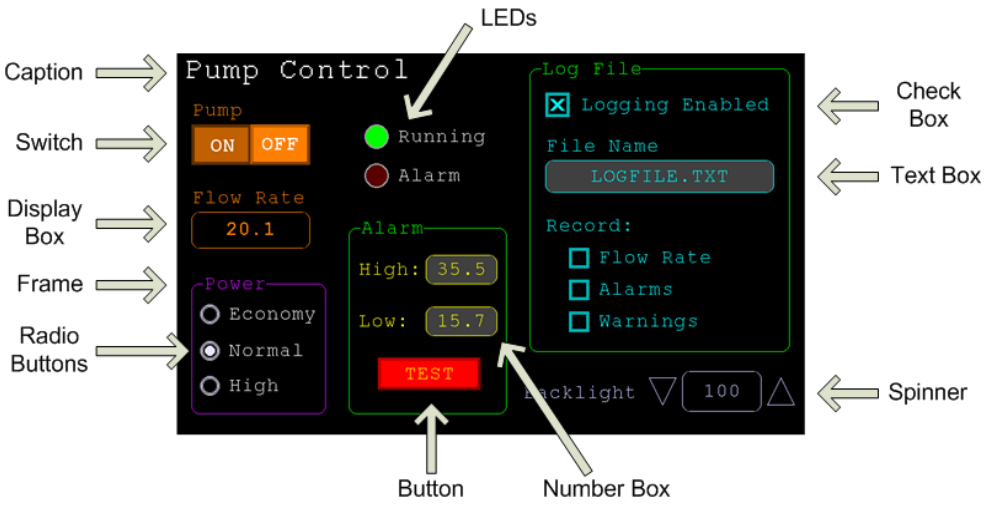
## Load Image

The LOAD IMAGE and LOAD JPG commands can be used to load an image from the Flash Filesystem or SD Card and display it on the LCD display. This can be used to draw a logo or add an ornate background to the graphics drawn on the display.

## Advanced Graphics

NOT AVAILABLE IN VGA/HDMI AND WEBMITE RP2040 VERSIONS

The PicoMite firmware includes a suite of advanced graphics functions to make it easy for a programmer to create touch sensitive control elements on an LCD panel as illustrated. These include on screen switches, buttons, indicator lights, keyboard, etc.




MMBasic will draw the control and animate it (ie, a switch will appear to depress when touched). All that the BASIC programmer needs to do is invoke a single command to specify the basic details of the control.

These functions make it easy to create a control panel to manage any control functions like a lathe, motor controller, heating system, small industrial process and so on.

The Advanced Graphics functions are described in detail in the document *Advanced Graphics Functions.pdf* which is included in the firmware download file.

## 3D Engine

NOT AVAILABLE IN WEBMITE VERSIONS

The 3D Engine includes ten commands for manipulating 3D images including setting the camera, creating, hiding, rotating, etc. See the document *3D_Graphics_User_Manual.pdf* in the PicoMite firmware download for a full description of these commands and how to use them.

## LCD Graphics Example

As an example of using the simple graphics commands the following program will draw a simple digital clock on an ILI9341 based LCD display. The program will terminate and return to the command prompt if the display screen is touched.

First the display and touch options must be configured by entering the commands listed at the beginning of this chapter. The exact format of these will depend on how you have connected the display panel.

Then enter and run the program:

```
CONST DBlue = RGB(0, 0, 128)                ' A dark blue colour
COLOUR RGB(GREEN), RGB(BLACK)               ' Set the default colours
FONT 1, 3                                   ' Set the default font

BOX 0, 0, MM.HRes-1, MM.VRes/2, 3, RGB(RED), DBlue

DO
  TEXT MM.HRes/2, MM.VRes/4, TIME$, "CM", 1, 4, RGB(CYAN), DBlue
  TEXT MM.HRes/2, MM.VRes*3/4, DATE$, "CM"
```

```
    IF TOUCH(X) <> -1 THEN END
  LOOP
```

This program starts by defining a constant with a value corresponding to a dark blue colour and then sets the defaults for the colours and the font. It then draws a box with red walls and a dark blue interior.

Following this the program enters a continuous loop where it performs three functions:

1. Displays the current time inside the previously drawn box. The string is drawn centred both horizontally and vertically in the middle of the box. Note that the TEXT command overrides both the default font and colours to set its own parameters.
2. Draws the date centred in the lower half of the screen. In this case the TEXT command uses the default font and colours previously set.
3. Checks for a touch on the screen. This is indicated when the TOUCH(X) function returns something other than -1. In that case the program will terminate.

The screen display should look like this (the font used in this illustration is different):



# GUI Integration

Versions of the firmware with GUI controls support advanced graphic controls such as on screen buttons, switches, check boxes, etc. These are described in the separate manual Advanced Graphics Functions.pdf. This chapter summarises how the various input devices (touch, mouse, keyboard and push buttons) integrate with these controls so that the same program can run unchanged on hardware with different input devices.

The graphic controls respond to a touch on a touch sensitive LCD panel, a click of the left mouse button and a synthetic click generated by the GUI CLICK command - all three are treated identically. The current state can be read at any time with the CLICK() or TOUCH() functions and the GUI INTERRUPT command arms interrupt routines that are called when any of these sources registers a down or an up event:

GUI INTERRUPT TouchDown [, TouchUp]

'TouchDown' is the subroutine called when the screen is touched or clicked and the optional 'TouchUp' subroutine is called on release. GUI INTERRUPT 0 disables the interrupts.

To use a mouse, display a cursor and link it to the mouse with GUI CURSOR ON followed by GUI CURSOR LINK MOUSE. The cursor then tracks the mouse and the left mouse button acts exactly like a touch at the cursor position - see the GUI CURSOR command for the details.

Without a mouse the cursor can be driven from anything the program can read. For example, a keyboard handler using INKEY$ can move the cursor with GUI CURSOR x, y and activate the control under it with GUI CLICK. The GUI CLICK PIN command goes one step further and designates an I/O pin (eg, a push button or a joystick fire button) as a click source which is polled in the background, so it works even while a modal control such as a message box is displayed.

On the VGA and HDMI versions with USB support a touch screen can also display the on-screen keyboard (see OPTION SCREEN KEYBOARD) so text can be entered without a physical keyboard.

In summary, the mechanisms for each input device are:

| **Input device** | **Mechanism** |
|---|---|
| Touch screen | Acts on the controls directly. Use GUI INTERRUPT TouchDown [, TouchUp] for events and read the state with TOUCH() or CLICK(). |
| Mouse | GUI CURSOR ON then GUI CURSOR LINK MOUSE. The left button clicks the control under the cursor and fires the same GUI INTERRUPT routines. |
| Keyboard / joystick / program logic | Move the cursor with GUI CURSOR x, y (eg, from an INKEY$ loop) and click with GUI CLICK. |
| Push button on an I/O pin | GUI CLICK PIN pin [,INV] - the pin then acts exactly like the left mouse button. |

The input devices available for driving the GUI controls on each version of the firmware are:

| **Firmware version** | **GUI controls** | **Touch** | **Mouse** | **Keyboard** |
|---|---|---|---|---|
| PicoMiteRP2040 | yes | LCD panel | PS/2 | PS/2 |
| PicoMiteRP2350 | yes | LCD panel | PS/2 | PS/2 |
| PicoMiteRP2040USB | yes | LCD panel or USB | USB | USB |
| PicoMiteRP2350USB | yes | LCD panel or USB | USB | USB |
| PicoMiteRP2350VGA | yes | - | PS/2 | PS/2 |

| | | | | |
|---|---|---|---|---|
| PicoMiteRP2350VGAUSB | yes | USB touch screen | USB | USB |
| PicoMiteHDMI | yes | - | PS/2 | PS/2 |
| PicoMiteHDMIUSB | yes | USB touch screen | USB | USB |
| PicoMiteHDMIWEB | yes | USB touch screen | USB | USB |
| WebMiteRP2350 | yes | LCD panel | PS/2 | PS/2 |
| PicoMiteRP2350BT * | yes | LCD panel | PS/2 | PS/2 |
| PicoMiteRP2350BTH * | yes | LCD panel | Bluetooth or PS/2 | Bluetooth |
| PicoMiteHDMIBTH * | yes | USB touch screen | USB or Bluetooth | USB or Bluetooth |
| PicoMitePC3 * | yes | USB touch screen | USB or Bluetooth | USB or Bluetooth |
| PicoMiteRP2040MIN | no | - | - | - |
| PicoMiteRP2040VGA | no | - | - | - |
| PicoMiteRP2040VGAUSB ** | no | - | - | - |
| WebMiteRP2040 | no | - | - | - |

* The Bluetooth versions are special builds: on the BT version the console is provided over Bluetooth LE and the input devices are PS/2; the BTH version supports Bluetooth LE and Bluetooth Classic keyboards and mice with the console on USB (it does not support a PS/2 keyboard). The HDMIBTH version adds the BTH version's Bluetooth support to the HDMIUSB version. The PC3 version has both the WiFi of the HDMIWEB version and the Bluetooth of the HDMIBTH version.

** The versions marked 'no' do not include the graphic controls but may still support the same input devices for general use. For example, the RP2040 VGA USB version supports a USB keyboard, mouse and touch screen and provides CLICK(DOWN), CLICK(UP), CLICK(X) and CLICK(Y), but the graphic controls, GUI CURSOR and GUI CLICK are not available.

# WiFi and Internet Functions

**WEBMITE VERSION ONLY**

This chapter provides an overview of the WiFi and Internet features implemented in the WebMite version for the Raspberry Pi Pico W and the Raspberry Pi Pico 2 W.

These functions are complex and, as a result, a few points should be noted:

- Implementing the Internet protocols uses a lot of the resources of the processor (particularly RAM) so the WebMite firmware should only be used where WiFi and Internet connectivity are important and some performance impact can be tolerated compared to the standard Raspberry Pi Pico.
- This manual describes how the WebMite interfaces with the WiFi and Internet protocols and provides some simple examples but it does not describe HTML, TCP and the many other protocols and conventions involved. They can be confusing for the newcomer who will need to consult some of the many primers available on the web.
- While processing complex Internet protocols the WebMite firmware can get confused and hang or throw an error and return to the command prompt. To allow for this, programs should use the WATCHDOG feature to recover from such situations. It is also recommended that from time to time (eg, once a week) the program should force a reboot using CPU RESTART to ensure that the protocol stacks are reset.

## Connecting to a WiFi Network

Before you do anything, you need to enable the WiFi feature on the WebMite and this is done with the following command entered at the command prompt:

```
OPTION WIFI ssid, password
```

Where 'ssid' is the name of the WiFi network and 'password' is the security password used to gain access to the network. Both are strings and if string constants are used they should be quoted as shown in this example:

```
OPTION WIFI "MyNetwork", "secret"
```

The command can also set the device name, a static IP address and the WiFi country code. The full syntax is:

```
OPTION WIFI ssid$, password$ [, hostname$] [, country$]
```

or, to use a static IP address instead of DHCP:

```
OPTION WIFI ssid$, password$, hostname$, ipaddress$, mask$, gateway$ [, country$]
```

'hostname$' is an optional name for the device on the network (if it is omitted a name is created from the unique device ID). 'country$' is an optional two letter ISO country code (eg, "GB", "DE", "AU") which sets the WiFi regulatory domain. The default is "XX" (worldwide) which restricts the radio to the most conservative set of channels and transmit power - setting the local country code is recommended and is often required to connect to European routers which may use channels 12 or 13. The WEB CONNECT command accepts the same parameters for connecting from within a program.

When this is entered the WebMite will restart which means that you will lose the USB connection and access to the console. If you reconnect quickly, you will see the following message:

```
Connecting to WiFi...
Connected 192.168.1.52
```

The address given in the last line is the IP address given to the WebMite by the router and will vary depending on the network. If the WebMite cannot connect you will see this message:

```
Connecting to WiFi...
failed to connect.
```

If the connection fails you should check the configuration of your WiFi router:

- The WiFi security must be WPA-PSK with either TKIP or AES encryption (or both).
- The WiFi access must have a password set.
- DHCP must be configured.

This command only needs to be entered once and will be remembered every time your WebMite is restarted. You can check its setting with the `OPTION LIST` command. When connected you can also check the IP address allocated as follows:

```
> PRINT MM.Info(ip address)
192.168.1.52
```

With some routers it can take some time or a couple of attempts to connect, so if you are using OPTION AUTORUN ON, it would be worth inserting something like this at the very start of your program:

```
DO WHILE MM.INFO(IP ADDRESS) = "0.0.0.0"
  IF TIMER > 5000 THEN CPU RESTART
LOOP
```

This would wait 5 seconds for a connection and restart if still not connected.

## Remote Console Access

You can remotely connect to the console of the WebMite via WiFi using Telnet. This is handy if the device is in a location that is difficult to access. Once connected via Telnet you can do everything that you would normally do via the USB console including running the editor.

To enable this feature, enter the following at the command prompt: `OPTION TELNET CONSOLE ON`

This only needs to be entered once and will be remembered every time your WebMite is restarted. It will also cause the WebMite to restart so you will need to reconnect to the USB console.

A program can find out whether anyone is connected by using the function MM.INFO(TELNET). This returns 1 while a Telnet client is attached to the console and 0 at all other times. It reports the connection rather than the setting, so it will return 0 when nothing is connected even though OPTION TELNET CONSOLE is enabled.

Tera Term ( http://tera-term.en.lo4d.com ) supports Telnet and is recommended for this task as it has a good VT100 emulation and supports the XModem file transfer protocol.

## File Transfer

Files can be transferred to and from the WebMite via XModem or TFTP. XModem is supported by Tera Term and will operate over Telnet the same as it does over a direct serial connection. However TFTP is much faster and more reliable than XModem so it is the recommended method of transferring files to and from the WebMite.

A TFTP server on the WebMite is automatically enabled when you are connected to a WiFi network so nothing is needed there. You do need a TFTP client for your PC and many different implementations are available for Windows, Mac OS and Linux. Note that many tutorials on TFTP discuss setting up a TFTP server - this is not needed, you only need a client.

For Windows a TFTP client is included in the operating system but it needs to be enabled via the Control Panel. To do this select:

```
Control Panel -> Programs and Features -> Turn Windows features on or off
```

Then scroll down the list and tick `TFTP Client`.

To send a file from your PC to the WebMite run the following in a Command or Power Shell window on your Windows PC ('IP-Addr' is the IP address of the WebMite):

```
TFTP -i IP-Addr PUT filename
```

And to copy a file from the WebMite to your PC:

```
TFTP -i IP-Addr GET filename
```

To list the functions of the TFTP client use the following:

```
TFTP -h
```

An alternative and simple graphical Windows TFTP client is: http://www.3iii.dk/linux/dd-wrt/tftp2.exe

## Getting the Time

A common first step in a program is to get the time/date and set the clock in the WebMite. This action also confirms that the WebMite can reach the internet.

Getting the time is done with the `WEB NTP` command as follows:

```
WEB NTP [timeoffset [, NTPserver$]]
```

Where 'timeoffset' is the time zone that you are in and "NTPserver$" is the name or IP address of the time server to use. This last parameter is optional and if left out the firmware will use a public timeserver pool. If the 'timeoffset' parameter is also omitted the WebMite's clock will be set to UTC.

This is a typical example for a device in the Los Angeles time zone:

```
> WEB NTP -8
ntp address 27.124.125.251
got ntp response: 08/03/2023 05:34:57
```

If the WebMite does not have access to the Internet, you will get an error. This can be trapped using the `ON ERROR SKIP` command and a suitable action taken (ie, reboot or display a message for the operator).

For example:

```
ON ERROR SKIP 3
WEB NTP -8
IF MM.ERRNO THEN WEB NTP -8
IF MM.ERRNO THEN PRINT "Failure to connect to the Internet" : CPU RESTART
```

The reason why we try the WEB NTP command twice is in case the first attempt failed due to some timing error (this can happen) – however, it should be successful on the second attempt.

## Implementing a Web Server

A popular requirement is to setup a web server which will display data collected by the WebMite in a user friendly web page.

The first step is to configure the server function using this command at the command prompt:

```
OPTION TCP SERVER PORT nn
```

Where 'nn' is the port number to use (normally 80 for a web page). Typically the command will be:

```
OPTION TCP SERVER PORT 80
```

As with the other OPTION commands listed above this only needs to be entered once and will be remembered every time the WebMite is restarted. It will also cause the WebMite to restart and if you reconnect quickly, you will see the following (with a different IP address):

```
Starting server at 192.168.1.52 on port 80
```

The above step configured the WebMite firmware to support a TCP server. In your program you need to start the server running with the following command:

```
WEB TCP INTERRUPT InterruptSub
```

Where 'InterruptSub' is the name of your subroutine that will be called whenever a request is received by the TCP server. This subroutine can use the command `WEB TCP READ` to read the incoming request from the remote client and the command `WEB TRANSMIT PAGE` can be used to send the requested web page.

For example, below is the full program to implement a simple web page (don't forget to use the OPTION TCP SERVER command first):

```
1  DIM buff%(4096/8)
2  WEB TCP INTERRUPT WebInterrupt
3  DO
4     '<do some processing here>'
5  LOOP
6
7  SUB WebInterrupt
8     LOCAL a%, p%, t%, s$
9     FOR a% = 1 To MM.INFO(MAX CONNECTIONS)
10       WEB TCP READ a%, buff%()
11       p% = LINSTR(buff%(),"GET")
12       t% = LINSTR(buff%(),"HTTP")
13       If (p% <> 0) And (t% > p%) Then
14             WEB TRANSMIT PAGE a%,"index.html"
15       ENDIF
16    NEXT a%
17 END SUB
```

The following describes this program in detail:

Line 1 First a 4K byte buffer is created for the incoming request from the client. This example uses the long string commands in MMBasic for handling the data and this is the buffer for this (see the next chapter titled *Long Strings* for a description of long strings). The size of this buffer will limit the amount of data received from the client.

Line 2 This starts the server running and specifies the interrupt subroutine for handling incoming requests as `WebInterrupt`.

Line 4 This is your main program loop and would be normally collecting data, switching outputs, etc.

Line 7 This is the subroutine that is called every time a request is made by the remote client browser.

Line 9 Loop through all the incoming connections (there could be a number of simultaneous connections).

Line 10 Read the incoming message into the long string buffer.

Lines 11 & 12 Get the locations of key words in the message.

Line 13 Check that the keywords are present and in the correct order.

Line 14 Send the page. This is a file called index.html which is located in the default directory on the internal flash filesystem or SD card. It is formatted in html which means that it can contain tags such as `<h1>This is a heading</h1>`. See *A Typical WEB Page* below for more.

## Inserting Data in the Web Page

Usually you need to insert data generated by the BASIC program in the web page that is transmitted. This is easily done by inserting the name of the BASIC variable surrounded by curly brackets (ie, { and }) into the text of the web page.

For example, if your program had a variable called `CurrentTemp` which had the value of 24 and represented the current temperature, the following web page: `The temperature is {CurrentTemp}` would display in the client's browser as: `The temperature is 24`.

The identifier between the curly brackets can be a float, integer or string, an array element and even an expression (ie, A + B). You can also use functions, so if you wanted to format a floating point number with the correct number of decimal places, etc you could use the formatting function Str$(). Note that an error in the expression will cause a corresponding error when the `WEB TRANSMIT PAGE` command is executed.

If you need to use an opening curly bracket in your web page you can use two as a pair (ie, {{) and that will be changed to a single opening bracket. A closing curly bracket without an opening partner will be untouched.

## Sending Multiple Pages

When a remote client requests data without specifying a page it will send the request as `GET / HTTP` with the forward slash representing the default page for the server (which is normally index.html). The example above did not bother checking for this, it just sent the same page for all requests.

However, if your page contains clickable links such as `<a href="page2.html">Next page</a>` and the user clicked on this link the remote client will then send another request containing `GET /page2.html HTTP`.

This can be easily accommodated by examining the requested data between the GET and HTML keywords. For example, replace line 15 in the above example with the following (s$ is the file requested):

```
s$ = LGetStr$(buff%(), p% + 4, t% - p% - 5)
IF s$ = "/" THEN
   WEB TRANSMIT PAGE a%,"index.html"
ELSE IF s$ = "/page2.html" THEN
   WEB TRANSMIT PAGE a%,"page2.html"
ENDIF
```

this can be extended to as many pages as you need.

## Sending an Image

You can insert an image in your web page using the following html code `<img src="pix.jpg"/>`. When the remote browser reads this it will send the following request `GET /pix.jpg HTTP`. You can then send the requested image using the code shown in bold:

```
s$ = LGetStr$(buff%(), p% + 4, t% - p% - 5)
IF s$ = "/" THEN
   WEB TRANSMIT PAGE a%,"index.html"
ELSE IF s$ = "/page2.html" THEN
   WEB TRANSMIT PAGE a%,"page2.html"
ELSE IF s$ = "/pix.jpg" THEN
   WEB TRANSMIT FILE a%,"pix.jpg","image/jpeg"
ENDIF
```

Note that `pix.jpg` must be a jpeg image residing in the default directory of the internal flash filesystem or SD card. The WebMite is not a fast server so small and simple images are preferred.

The "image/jpeg" parameter is known as a MIME type and there are many different types, other common image types are image/bmp, image/png, and image/gif.

### Page Not Found (404) Response

If a remote client requests a page or a file which is not supported by your program you can use the WEB TRANSMIT CODE command to send a 404 error as follows:

```
s$ = LGetStr$(buff%(), p% + 4, t% - p% - 5)
IF s$ = "/" THEN
   WEB TRANSMIT PAGE a%,"index.html"
ELSE IF s$ = "/page2.html" THEN
   WEB TRANSMIT PAGE a%,"page2.html"
ELSE IF s$ = "/pix.jpg" THEN
   WEB TRANSMIT FILE a%,"pix.jpg","image/jpeg"
ELSE
   WEB TRANSMIT CODE a%, 404
ENDIF
```

## Live Graphical Data in a WEB Page

Serving numbers and text in a Web page is useful but often you would like to also include graphical elements such as pi charts, line graphs, historical trends, etc derived from the data collected by the program  This can be done in a roundabout manner by configuring the WebMite with a virtual display panel, then drawing on that display using the standard drawing commands (pixel, line, circle, etc) and saving that as a BMP image.  This file can then be included in the web page as an image.  In more detail, this process is as follows:

First a virtual display needs to be configured.  There are two that you can use, VIRTUAL_C is a 320x240 pixel image with 16 colours and VIRTUAL_M is a 640x480 pixel monochrome image.  For example:

```
OPTION LCDPANEL VIRTUAL_C
```

As with the other OPTION commands this must be entered at the command prompt, will cause a restart and only needs to be entered once.

Then, in your program you can draw images and text on this “display” using the commands described in the chapter *Graphics Commands and Functions*.  For example:

```
CIRCLE 100, 100, 50, 1, 1, RGB(red), RGB(blue)
LINE 10, 10, 200, 200, 1, RGB(yellow)
```

When you are finished, save this image:

```
SAVE COMPRESSED IMAGE "graph.bmp"
```

Within the web page that you are serving you can insert this image using the following html code:

```
<img src="graph.bmp"/>
```

Finally, in your BASIC program, you must arrange for this file to be sent when the web page is loaded by the remote browser as described above (see *Sending an Image*).  For example, insert this in the ELSE IF chain described above:

```
ELSE IF s$ = "/graph.bmp" THEN
  WEB TRANSMIT FILE a%, "graph.bmp", "image/bmp"
```

Your BASIC program will need to update this file as new data is recorded but, as this only needs to be done when the remote browser requests the image, it should not cause any excessive wear on the flash memory.

## A Complete General Purpose Server

The above pages have described the individual components that make up a web server.  Below is an example of a complete and integrated general purpose server that can handle most requests made by a browser.  It works by examining the extension of the file requested then uses the appropriate WEB TRANSMIT command to send the requested data.  It can be used as a drop in module for any project that needs the WebMite to be a web server.

```
WEB TCP INTERRUPT WebInterrupt
DO
   '<do some processing here>'
LOOP

' sub to handle all web server requests
SUB WebInterrupt
  LOCAL a%, p1%, p2%, file$, buff%(4096/8)
  FOR a% = 1 To MM.INFO(MAX CONNECTIONS)
    WEB TCP READ a%, buff%()
    P1% = LINSTR(buff%(),"GET")
    P2% = LINSTR(buff%(),"HTTP")
    If (p1% <> 0) And (p2% <> 0) And (p2% > p1%) Then
```

```
        file$ = LCASE$(LGetStr$(buff%(), p1% + 4, p2% - p1% - 5))
        IF file$ = "/" THEN file$ = "/index.html"
        ON ERROR SKIP
        OPEN file$ FOR INPUT AS #1       ' check that the file exists
        IF MM.ERRNO THEN WEB TRANSMIT CODE a%, 404 : CONTINUE FOR
        CLOSE #1
        SELECT CASE RIGHT$(file$, 4)
          CASE "html", ".htm", ".txt"
            WEB TRANSMIT PAGE a%, file$
          CASE ".bmp", ".png", ".gif"
            WEB TRANSMIT FILE a%, file$, "image/" + RIGHT$(file$, 3)
          CASE ".jpg", "jpeg"
            WEB TRANSMIT FILE a%, file$, "image/jpeg"
          CASE ".ico"
            WEB TRANSMIT FILE a%, file$, "image/vnd.microsoft.icon"
        END SELECT
      ENDIF
    NEXT a%
  END SUB
```

Note that all files must be stored in the root of the flash filesystem or SD card and their names must use lowercase only (the flash filesystem is case sensitive).

## A Typical WEB Page

The WEB page to be transmitted via the WEB TRANSMIT PAGE command must be constructed according to the html standard. It can be as simple as a single line of text with no formatting, ie:

```
The temperature is {CurrentTemp}
```

Or you can include some simple formatting:

```
<title>WebMite</title>
<h2>Temperature Monitor</h2>
The temperature is {CurrentTemp}
```

Or you can send a complex page. Typically these have head and body sections and delineate text into paragraphs and use the break tag for spacing. This is the skeleton of such a page:

```
<html>
<head>
<title>WebMite</title>
</head>
<body>
<h1>This is a heading</h1>
<br>
<p>The temperature is {CurrentTemp}</p>
</body>
</html>
```

There are many resources on the Internet offering tutorials in html for beginners. A typical example is http://www.simplehtmlguide.com/. There are also numerous WISWIG HTML editors available. For example: https://onlinehtmleditor.dev/

## Input Fields and Control

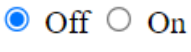
Using HTML input fields you can place buttons, check boxes, radio buttons, etc on your WEB page which allow the user to send requests to the WebMite. This way the user can remotely turn things on/off, set control parameters and much more - all from the web page served by the WebMite.

To do this you need to insert an *HTML form* in the web page containing one or more input fields. There are many types of input fields to choose from (see https://www.w3schools.com/tags/att_input_type.asp ) but in our simple example we will use two radio buttons to turn off and on a fictitious device.

The following code needs to be inserted in the default web page (ie, index.html):

```
<form method='get'>
    <input name='RB' type='radio' value='OFF' {offb$} onClick='this.form.submit()'> Off
    <input name='RB' type='radio' value='ON' {onb$} onClick='this.form.submit()'> On
</form>
```

This will create two radio buttons which will look like this in a browser:



Note that the above HTML code contains two BASIC variables (offb$ and onb$) which will be substituted by the WEB TRANSMIT PAGE command. These variables control how the button is displayed. If they are set to an empty string the button will display as unchecked or, if it is set to the string "checked", the button will display as checked. These variables should be initialised when the program starts running.

When the user clicks on the first button the browser will send a request containing: GET /?RB=OFF HTML and when the second is clicked the request will be: GET /?RB=ON HTML

In the ELSE IF chain in the TCP interrupt subroutine we can act on these requests:

```
ELSE IF s$ = "/?RB=OFF"
    ' <insert code here to turn off the device>
    offb$ = "checked" : onb$ = ""
    WEB TRANSMIT PAGE a%, "index.html"
ELSE IF s$ = "/?RB=ON"
    ' <insert code here to turn on the device>
    offb$ = "" : onb$ = "checked"
    WEB TRANSMIT PAGE a%, "index.html"
```

Essentially all that this code does is turn off or on the device as requested, sets the variables onb$ and offb$ to reflect the new state of the buttons and then resends the whole web page back to the browser.

This example has glossed over the many details involved and you can get extremely complicated if you wished with multiple inputs involving buttons, text input, drop down lists, password fields, requests for file uploads and much more. However, to do this, you will have to get deeper into HTML coding. For an example of this see the *Garden Watering Controller* project at: https://geoffg.net/retic.html

## Implementing a TCP Client

The WebMite can also act as a TCP client to request data from a remote server. This is managed with several commands. The first is:

```
WEB OPEN TCP CLIENT Domain$, PortNumber
```

This opens a TCP connection to Domain$ (for example "openweathermap.org") using the specified PortNumber (normally 80 for a web page).

With the connection open you can send one or more requests using this command:

```
WEB TCP CLIENT REQUEST query$, inbuf [, timeout]
```

The request to be sent is 'query$' and the response will be saved in 'inbuf' which is normally a long string variable such as buff%(4096/8). The size of this buffer (in bytes) will limit the amount of data received from the server and should be increased if more data is expected.

'timeout' is optional and is the timeout in milliseconds.

If you are accessing a website 'query$' can be something as simple as "GET / HTTP" which will retrieve the default page for that website. Your program will then be responsible for picking out the data that you want from within the response.

Finally, you close the connection with:

```
WEB CLOSE TCP CLIENT
```

Two further commands are available for more advanced use.

```
WEB TCP CLIENT READ inbuf [, timeout]
```

reads data sent by the server without first sending a request. It is typically used to read a server's greeting (such as the SMTP 220 banner) before issuing any commands, which avoids the WebMite being treated as an 'early talker' by servers that deliberately delay their greeting.

```
WEB TCP CLIENT WRITE data%() [, timeout]
```

sends the entire contents of a long string to the server in a single operation, without waiting for a reply. Because it has no 255 character limit and avoids a round trip to the server for every block of data it is much faster than WEB TCP CLIENT REQUEST when sending large amounts of data such as an email attachment.

The following example is a complete program (using these commands) to get the current temperature for the city of Paris from openweathermap.com. For this to work you need a (free) account with Open Weather Map ( https://openweathermap.org ) which will include an API key, this is a 32 digit hex number that allows you to make the query. This number should be substituted for the dummy key in the first line.

```
CONST Key = "nnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnn"
CONST Query = "GET /data/2.5/weather?q=Paris,fra&APPID="+Key+Chr$(13)+Chr$(10)
DIM buff%(4096/8)

WEB OPEN TCP CLIENT "api.openweathermap.org", 80
```

```
WEB TCP CLIENT REQUEST Query, buff%()
WEB CLOSE TCP CLIENT
temp = VAL(JSON$(buff%(), "main.temp"))
PRINT "Current temperature in Paris is:" temp - 273
```

The JSON() function is used to extract the value that we want from the JSON (JavaScript Object Notation) formatted response.

When this program is run you should see something like:

```
Connected
Current temperature in Paris is: 13.54
```

## Secure Connections (TLS)

All WebMite versions support TLS (Transport Layer Security), which allows the WebMite to connect to secure (https) servers. A secure connection is opened with one of:

WEB OPEN TLS CLIENT Domain$, PortNumber [, timeout]
WEB OPEN TLS STREAM Domain$, PortNumber [, timeout]

These take the same parameters as their TCP equivalents but negotiate an encrypted connection - for a web server the port number is normally 443 instead of 80. Once open, the connection is used exactly as a plain TCP connection (ie, with WEB TCP CLIENT REQUEST or WEB TCP CLIENT STREAM and WEB CLOSE TCP CLIENT). For example, the weather example above needs only one changed line to use a secure connection:

WEB OPEN TLS CLIENT "api.openweathermap.org", 443

By default the connection is encrypted but the identity of the server is not verified. To also verify the server, load a bundle of trusted root certificates (in PEM format) from the flash filesystem or SD card with:

WEB TLS CA file$

After this every TLS connection requires the server to present a certificate signed by one of the trusted authorities, otherwise the connection will fail. Verification checks the certificate's expiry dates so the clock must be set first (use WEB NTP). A suitable bundle of the common root certificates (cacert.pem) can be downloaded from https://curl.se/docs/caextract.html. Verification can be turned off again with WEB TLS NOVERIFY.

TLS is also used automatically by the MQTT client when port 8883 (the standard secure MQTT port) is specified in WEB MQTT CONNECT.

## Using UDP

User Datagram Protocol (UDP) is a fast and lightweight protocol used to send messages between devices. It can be unreliable because it doesn't guarantee that data packets will be delivered or arrive in order, however it is often used for time sensitive applications that prioritize speed over reliability.

Setting up a UDP server is similar to setting up a TCP server. Firstly UDP is enabled with the following option entered at the command prompt (this will also cause a restart):

```
OPTION UDP SERVER PORT port_nbr
```

In your program you need to start the server running with the following command:

```
WEB UDP INTERRUPT InterruptSub
```

Where 'InterruptSub' is the name of your subroutine that will be called whenever a message is received by the UDP server. When this happens the IP address of the sending device will be held in the read only variable MM.ADDRESS$ and the received message in MM.MESSAGE$.

You can send a UDP message using:

```
WEB UDP send ip_address$, port_nbr, message$
```

The following demonstration uses two WebMites configured with OPTION WIFI on the same network.

Before starting both WebMites must be configured with OPTION UDP SERVER PORT 77.

On the first WebMites make a note of its IP address and enter the following program:

```
WEB UDP interrupt myint
Do
   '<do some processing here>'
Loop

' interrupt subroutine
Sub myint
   Print "Received " + mm.message$ + " from " + mm.address$
   pause 100
```

```
   WEB UDP send mm.address$, 77, " WebMite #1 message"
End Sub
```

Then run the program, which will initially sit in a loop doing nothing.

On the second WebMite enter the following program substituting the IP address with the one from the first WebMite. Then run the program:

```
WEB UDP interrupt myint
WEB UDP send "192.168.1.127", 77, "Starting UDP echo"
Do
   '<do some processing here>'
Loop

' interrupt subroutine
Sub myint
   Print "Received " + mm.message$ + " from " + mm.address$
   pause 100
   WEB UDP send mm.address$, 77, "WebMite #2 message"
End Sub
```

The result will be a very fast ping pong of UDP messages between the two WebMites.

## Sending EMails

When you have a remote device like the WebMite it is useful for it to be able to send emails to raise the alarm over faults, report on its status and so on. The WebMite can do this using the SMTP protocol to connect to a server which will then relay the email to its destination.

The following example uses the free SMTP relay service offered by SMTP2GO which allows for 1000 emails per month (plenty for the WebMite). Note that they do not accept registration requests from people with a generic free email address (eg, xxx@gmail.com).

To get started you need to create a free login at SMTP2GO ( https://www.smtp2go.com/ ), register a Verified Sender and create an associated username and password. Both then need to be converted to Base64 encoded strings (the following website will do this for you: https://www.base64encode.org ).

The Base64 encoded username should be used to replace the nnnnnnnnnn string in the first line of the following program while the Base64 encoded password should be used to replace the xxxxxxxxxxx in the second line. The other four lines at the start of the program should also be replaced with your data:

```
CONST userBase64$ =  "nnnnnnnnnnn"
CONST paswdBase64$ = "xxxxxxxxxxx"
CONST mailfrom$ =    "from@server.com"
CONST mailto$ =      "to@server.com"
CONST subject$ =     "Test EMail"
CONST message$ =     "Test of SMTP2GO"

CONST cr = Chr$(13)+Chr$(10)
DIM buff%(4096/8), body$

body$ = "From: " + mailfrom$ + cr + "To: " + mailto$ + cr
body$ = body$ + "Subject: " + subject$ + cr + cr
body$ = body$ + message$ + cr + "." + cr

WEB OPEN TCP CLIENT "mail.smtp2go.com", 2525
WEB TCP CLIENT READ buff%(), 30000
WEB TCP CLIENT REQUEST "EHLO picomite" + cr, buff%()
WEB TCP CLIENT REQUEST "AUTH LOGIN" + cr, buff%()
WEB TCP CLIENT REQUEST userBase64$ + cr, buff%()
WEB TCP CLIENT REQUEST paswdBase64$ + cr, buff%()
WEB TCP CLIENT REQUEST "MAIL FROM:<" + mailfrom$ + ">" + cr, buff%()
WEB TCP CLIENT REQUEST "RCPT TO:<" + mailto$ + ">" + cr, buff%()
WEB TCP CLIENT REQUEST "DATA" + cr, buff%()
WEB TCP CLIENT REQUEST body$, buff%()
WEB CLOSE TCP CLIENT
IF LINSTR(buff%(), "250") = 0 THEN
  PRINT "Email send failed"
Else
  Print "Email sent OK"
EndIf
```

The program reads the server's greeting with WEB TCP CLIENT READ before sending anything; some servers will not respond, or will reject the session, if the client speaks first. The email addresses are enclosed in angle brackets (< >) as required by the SMTP standard.

Note that the `mailfrom$` email address used in the above program MUST be the same as that used when you registered the Verified Sender with SMTP2GO. If they are not the same SMTP2GO will reject the email (this is an anti spam precaution).

These days using an SMTP relay service is complicated by variations in the SMTP protocol used by each vendor and the various protections in place to reduce the amount of spam. This example is specific to the SMTP protocol as used by SMTP2GO however other services can also be used but the program must be modified to accommodate their own version of the SMTP protocol.

Sending an EMail with an Attachment

Files such as an image or a data log can be attached to an email by sending it as a MIME multipart message. An attachment is binary and usually much larger than the 255 character limit of WEB TCP CLIENT REQUEST, so it is Base64 encoded and streamed to the server with WEB TCP CLIENT WRITE. This command sends a complete LONGSTRING in a single operation and is far faster than sending the data as many small requests.

Connect and authenticate exactly as in the previous example. Then, instead of sending the message body, build the whole message (including the encoded file) in a LONGSTRING and send it:

```
' add these to the declarations at the start of the program
CONST bd$ = "=_picomite_boundary_="
CONST fname$ = "A:/test.jpg"
DIM msg%(8800/8), h$, enc$, chunk$, fsize, k, n

' connect and authenticate as before, then after sending DATA:
WEB TCP CLIENT REQUEST "DATA" + cr, buff%()

LongString Clear msg%()
h$ =      "From: " + mailfrom$ + cr + "To: " + mailto$ + cr
h$ = h$ + "Subject: " + subject$ + cr + "MIME-Version: 1.0" + cr
h$ = h$ + "Content-Type: multipart/mixed; boundary=" + Chr$(34) + bd$ +
Chr$(34) + cr + cr
h$ = h$ + "--" + bd$ + cr + "Content-Type: text/plain" + cr + cr
LongString Append msg%(), h$
LongString Append msg%(), message$ + cr
h$ =      "--" + bd$ + cr + "Content-Type: image/jpeg" + cr
h$ = h$ + "Content-Transfer-Encoding: base64" + cr
h$ = h$ + "Content-Disposition: attachment; filename=" + Chr$(34) + "test.jpg"
+ Chr$(34) + cr + cr
LongString Append msg%(), h$

OPEN fname$ FOR INPUT AS #1
fsize = LOF(#1)
DO WHILE fsize > 0
  k = MIN(189, fsize)              ' 189 bytes encode to 252 Base64 characters
  chunk$ = INPUT$(k, #1)
  n = MATH(BASE64 ENCODE chunk$, enc$)
  IF LLEN(msg%()) + LEN(enc$) + 2 > 8700 THEN
    WEB TCP CLIENT WRITE msg%()    ' flush when the buffer is nearly full
    LongString Clear msg%()
  ENDIF
  LongString Append msg%(), enc$ + cr
  fsize = fsize - k
LOOP
CLOSE #1

LongString Append msg%(), "--" + bd$ + "--" + cr
WEB TCP CLIENT WRITE msg%()        ' send the remainder of the message
WEB TCP CLIENT REQUEST "." + cr, buff%()   ' the "." ends DATA; server replies
250
WEB CLOSE TCP CLIENT
```

Each block of 189 bytes from the file becomes 252 Base64 characters. The LONGSTRING msg%() is sent with WEB TCP CLIENT WRITE whenever it is nearly full and once more at the end, before the terminating '.' is sent with WEB TCP CLIENT REQUEST so that the server's final acknowledgement can be read. WEB TCP CLIENT READ and WEB TCP CLIENT WRITE require firmware that includes these commands.

## Base 64 Encoding

Base64 is system for converting binary data to a text string that only uses ASCII characters (ie, there are no control characters). It's designed to make it easy to send binary data over the internet using protocols which do not accept binary data and many protocols require its use.

The MATH BASE64 encode/decode function will do the encoding and decoding for you (see the detailed function list for the full syntax). A typical use is in the above program for sending emails. Rather than use an external service to convert the username and password to Base64 you can do this in the program using this function.

For example:

```
DIM n, userBase64$, paswdBase64$
CONST user$ = "MyUserName"        ' user name in plain text
CONST paswd$ = "MyPassword"       ' password in plain text
...
' encode the user name and password
n = MATH(BASE64 ENCODE user$, userBase64$)
n = MATH(BASE64 ENCODE paswd$, paswdBase64$)
...
```

## MQTT Client

MQTT is a protocol that allows clients like the WebMite to post or retrieve messages on a server (also called a MQTT broker). This is rather like a bulletin board or web based forum where people post messages which others can later read at their leisure – the main difference is that MQTT is designed for machine to machine communications.

A typical example could be a battery powered WebMite which is monitoring the water level in a dam. Twice a day it would power up, make the measurement, post the result on a MQTT broker and power down again. A client program (perhaps on a PC) could later read these messages, display the results and graph them.

There are many free brokers available, use Google to search for "free MQTT broker".

The WebMite uses five commands to post or retrieve messages:

```
WEB MQTT CONNECT  Connect to an MQTT broker.
WEB MQTT PUBLISH  Publish content to an MQTT topic (ie, post a message).
WEB MQTT SUBSCRIBE      Subscribe to an MQTT topic (ie, retrieve messages).
WEB MQTT UNSUBSCRIBE    Unsubscribe from an MQTT topic.
WEB MQTT CLOSE    Close the MQTT connection.
```

## Ping

The WebMite will respond to a ping message so you can check if it is alive and accessible. If it is connected to the public Internet a free service like https://uptimerobot.com/ can be used to alert you if it has stopped running.

## Streaming audio

The WEB OPEN TCP STREAM and WEB TCP CLIENT STREAM commands can together with the PLAY STREAM command very simply implement a basic internet radio capability. This is demonstrated in the code below which receives the UK program ClassicFM. NB: a VS1053 audio codec is required for this program.

```
Option escape
Option default none
' create the request for the radio site (ClassicFM)
Dim a$="ice-the.musicradio.com"
Dim q$="GET "
Inc q$,"/ClassicFMMP3"
Inc q$," HTTP/1.1\r\n"
Inc q$,"Host: "
Inc q$,a$
Inc q$,"\r\nConnection: close\r\n\r\n"

'create a circular buffer for reading the internet stream and
```

```
'read and write pointers
Dim buff%(4095),w%,r%

' Configure the VS1053 and tell it to play from the circular buffer
Play stream buff%(), r%, w%

' Open the internet radio site
WEB open tcp stream a$,80

' Send the request to start the stream using the circular buffer specified
WEB TCP CLIENT STREAM q$, buff%(), r%, w%

'sit back and listen
Do : Pause 500: Loop
```

# Long Strings

Long Strings are a set of commands and functions that allow MMBasic to manipulate strings of unlimited length and are particularly useful when dealing with data sent via WiFi and the Internet. Standard strings in MMBasic are limited to a maximum length of 255 characters. Long strings duplicate these functions but will work with strings of any length limited only by the amount of available RAM.

## Long String Variables

Variables for holding long strings must be defined as integer arrays. The long string routines do not keep numbers in these arrays but just use them as blocks of memory for holding long strings.

When creating these arrays they should be defined as single dimensioned integer arrays with the number of elements set to the number of characters required for the maximum string length divided by eight. The reason for dividing by eight is that each integer in an MMBasic array occupies eight bytes.

The following is an example of declaring three long string variables which will be used to hold up to 2048 characters in each:

```
CONST MaxLen = 2048
DIM INTEGER Str1(MaxLen/8), Str2(MaxLen/8), Str3(MaxLen/8)
```

These will contain empty strings when created (ie, their length will be zero). When these variables are passed to the long string functions they should be entered as the variable name followed by empty brackets. For example:

```
LONGSTRING COPY Str1(), Str2()
```

Long string variables can be passed as arguments to user defined subroutines and functions. For example:

```
Sub MySub longarg() AS INTEGER
  PRINT  "Long string length is" LLEN(longarg())
END SUB
```

And it could be called like this:

```
MySub str1()
```

## Long String Commands

These are documented in detail in the *Commands* and *Functions* sections of this manual. The commands are:

| | |
|---|---|
| LONGSTRING AES128 ENCRYPT/DECRYPT | Encrypts or decrypts a long string |
| LONGSTRING APPEND array%(), string$ | Append an ordinary string to a long string |
| LONGSTRING BASE64 ENCODE/DECODE | Encodes or decodes a long string using base 64 |
| LONGSTRING CLEAR array%() | Clear (ie, set to empty) a long string |
| LONGSTRING COPY dest%(), src%() | Copy a long string |
| LONGSTRING CONCAT dest%(), src%() | Concatenate two long strings |
| LONGSTRING LCASE array%() | Convert a long string to lowercase |
| LONGSTRING LEFT dest%(), src%(), nbr | Get the left nbr characters from a long string |
| LONGSTRING LOAD array%(), nbr, string$ | Copy characters to a long string |
| LONGSTRING MID dest%(), src%(), start, nbr | Get characters from the middle of a long string |
| LONGSTRING PRINT [#n,] src%() [;] | Print a long string |
| LONGSTRING REPLACE array%() , string$, start | Replace characters in a long string |
| LONGSTRING RESIZE addr%(), nbr | Set the length of a long string |
| LONGSTRING RIGHT dest%(), src%(), nbr | Get the right nbr characters from a long string |
| LONGSTRING SETBYTE addr%(), nbr, data | Set a byte in a long string |
| LONGSTRING TRIM array%(), nbr | Trim characters from the left of a long string |
| LONGSTRING UCASE array%() | Convert a long string to uppercase |
| LMID(array%(),start [,num])=string$ | Insert/replace text in a longstring |

## Long String Functions

| | |
|---|---|
| r = LGETBYTE(array%(), n) | Return the value of a byte in a long string |
| r$ = LGETSTR$(array%(), start, length) | Returns part of a long string as a normal string. |
| r = LINSTR(array%(), search$ [,start] [,size]) | Returns the position of a string in a long string |
| r = LLEN(array%()) | Returns the length of a long string |
| r= LINPUT(array%(),fnbr,nbr) | Reads nbr bytes from a file returning the number read |
| MATH(BASE64 ENCODE/DECODE) | Encodes or decodes data using base 64 |

# Bluetooth Versions

Four versions of the firmware use the Bluetooth radio of the Raspberry Pi wireless chip. PicoMiteRP2350BT provides the MMBasic console over Bluetooth (the PicoMite acts as a Bluetooth device) while PicoMiteRP2350BTH allows Bluetooth keyboards, mice and game controllers to be used and can play its sound through a Bluetooth speaker or headphones and use a Bluetooth serial device such as a GPS receiver (the PicoMite acts as a Bluetooth host). Both require a Raspberry Pi Pico 2 W and are otherwise the same as the standard PicoMite RP2350 version including support for LCD panels, touch and the GUI controls. PicoMiteHDMIBTH adds the Bluetooth features of the PicoMiteRP2350BTH to the PicoMiteHDMIUSB (HDMI video with USB keyboards, mice and game controllers) and everything that this chapter says about the PicoMiteRP2350BTH also applies to it. PicoMitePC3, built for the Pico Computer 3, adds the same Bluetooth features to the PicoMiteHDMIWEB's WiFi, and everything said here about the PicoMiteRP2350BTH applies to it too.

On the BT and BTH versions the CPU speed is restricted to the range 200MHz to 396MHz (below 200MHz the Bluetooth stack cannot reliably keep up); on the PicoMiteHDMIBTH and PicoMitePC3 it is set by the HDMI resolution. Pairing information is stored with the saved options so that paired devices reconnect automatically after a power cycle. OPTION RESET (or a firmware update that resets the options) clears the stored pairings and any devices must then be paired again. On the PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3 versions the BLUETOOTH FORGET command also deletes them, either all of them or those of one device. On a module that connects the wireless chip to different pins from the Pico 2 W, set them with OPTION CYW43 PINS.

## PicoMiteBT – Console over Bluetooth

On this version the USB connector is used only for power; the console is provided wirelessly over Bluetooth LE using the Nordic UART Service. This makes it possible to program and control a PicoMite that is completely sealed or embedded where a USB cable cannot be attached.

The PicoMite advertises itself as PicoMite-XXXX where XXXX is derived from the processor's unique ID. Pairing is automatic and no PIN is required. Any Bluetooth LE serial terminal application can connect - for example "Serial Bluetooth Terminal" on Android or "Bluefruit Connect" on iOS.

On a personal computer the supplied bridge program ble_bridge (included in the firmware download as ble_bridge.exe for Windows; other systems can run the Python script ble_bridge.py) connects to the PicoMite over Bluetooth and makes the console available as a local network connection:

ble_bridge [device-name] [--port n]

Then connect Tera Term using File -> New connection -> TCP/IP with host 127.0.0.1, service Telnet and port 5555 (or the port specified with --port). Each keypress is sent immediately and CTRL-C, the function keys and the full screen editor all work as normal (make sure that Tera Term's local echo and local edit modes are off). The bridge keeps running when the PicoMite restarts (eg, after an OPTION command) and reconnects automatically.

There is also a backup console on a serial port, for when the Bluetooth link cannot be used. By default (after the firmware is loaded or after OPTION RESET) the console is also available on GP8 (Tx) and GP9 (Rx) at 115200 baud, alongside the Bluetooth console - connect a USB to serial adapter to these pins. It can be moved to other pins with OPTION SERIAL CONSOLE, keeping the B parameter (for example OPTION SERIAL CONSOLE GP0, GP1, B), because without it the Bluetooth console is not used. OPTION SERIAL CONSOLE DISABLE frees the pins and leaves only the Bluetooth console.

## PicoMiteBTH – Bluetooth Keyboards and Mice

On this version the console remains on the USB port as normal but the firmware acts as a Bluetooth LE host so that wireless keyboards, mice and game controllers using Bluetooth LE (also called HID over GATT) can be used, and so can the older Bluetooth Classic keyboards together with the touchpad or mouse built into one (see below). Note that this version does not support a PS2 keyboard (a PS2 mouse is still supported) or audio output using PWM, an MCP48n2 DAC or a VS1053 - sound is played through a Bluetooth speaker or an I2S DAC. The PicoMiteHDMIBTH and PicoMitePC3 also support USB keyboards, mice and game controllers in the same way as the PicoMiteHDMIUSB (but not PS2 devices).

Pairing is automatic: put the keyboard (or mouse) into its pairing mode close to the PicoMite and it will be discovered, paired (normally no PIN is required) and connected - the message "Bluetooth Keyboard Connected" is shown on the console. Only devices nearby (roughly within the same room) are considered so a neighbour's devices will not connect accidentally. Once paired, the device reconnects automatically whenever it or the PicoMite is switched on.

A few keyboards ask for a passkey while pairing. The PicoMite then shows a six digit number on the console, for example “Bluetooth keyboard pairing: type 123456 on the keyboard being paired, then Enter”, and this number should be typed on that keyboard (not on one that is already connected) followed by the Enter key, within about 30 seconds.

A connected keyboard behaves exactly like a USB or PS2 keyboard: it can be used at the command prompt and in the full screen editor, the layout is set with OPTION KEYBOARD and it can be read in a program with INKEY$ and KEYDOWN(). A Bluetooth mouse is read with the DEVICE(MOUSE) function and can be used with the GUI controls, and a game controller is read with DEVICE(GAMEPAD).

Bluetooth Classic keyboards (the older kind, including many low cost keyboards with a built-in touchpad) are not paired automatically. Put the keyboard into its pairing mode and search for it with BLUETOOTH SCAN, which lists it as a ‘Keyboard’:

```
> BLUETOOTH SCAN
Scanning...
54:46:6E:AD:87:50  -65 dBm  Keyboard  "Bluetooth 3.0 Keyboard"
1 device found
```

Then connect to it using the name shown by the scan (or its address):

```
BLUETOOTH CONNECT "Bluetooth 3.0 Keyboard"
```

If the keyboard asks for a passkey the six digit number is shown on the console as described above. The keyboard is then remembered and reconnects automatically whenever it wakes up or is switched on, also after the PicoMite restarts. Only the remembered keyboard is allowed to connect, and connecting another Classic keyboard replaces it. Its keys work in the same way as a Bluetooth LE keyboard’s, its Caps, Num and Scroll Lock LEDs follow the lock keys, and its touchpad or mouse is read with DEVICE(MOUSE 2, ...) in the same way as a Bluetooth LE mouse. A Bluetooth LE keyboard, a Bluetooth Classic keyboard and a Bluetooth speaker can all be connected at the same time.

Many low cost keyboards sold for tablets have an operating system mode which is selected with an Fn key combination or a switch (see the keyboard’s instructions). Set such a keyboard to its Windows or PC mode. In its iOS mode Home, End, Page Up and Page Down are sent as Cmd with an arrow key, the function keys send iPad shortcuts and the touchpad works differently.

BLUETOOTH STATUS shows whether a keyboard is connected (with a separate line for a Bluetooth Classic keyboard) and MM.INFO$(BLUETOOTH KEYBOARD) returns its address (or an empty string if none is connected).

## PicoMiteBTH – Bluetooth Speakers and Headphones

The PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3 versions can also send their audio output to a Bluetooth speaker or headphones (anything that supports the A2DP profile, which is almost every Bluetooth speaker). Everything that the PLAY command can produce - tones, sounds, WAV, FLAC, MP3 and MOD files, arrays, samples and BBC sounds - is played through the speaker, and a Bluetooth keyboard or mouse continues to work at the same time. On the PicoMitePC3, which also has WiFi, the radio gives the speaker priority while sound is playing: a WiFi transfer slows to a crawl until the sound stops, and WiFi recovers at once.

First select Bluetooth as the audio output. This command is entered at the command prompt and is saved, so it only needs to be run once (the PicoMite will then restart):

```
OPTION AUDIO BLUETOOTH
```

Then put the speaker into its pairing mode and search for it with BLUETOOTH SCAN. This lists the nearby Bluetooth Classic devices with their address, signal strength and name, and marks the audio devices and keyboards:

```
> BLUETOOTH SCAN
Scanning...
41:42:A3:1C:07:5E  -58 dBm  Audio  "BT Speaker"
1 device found
```

Connect to the speaker using the name shown by the scan (or its address):

```
BLUETOOTH CONNECT "BT Speaker"
```

The speaker is now paired and remembered. From then on it is reconnected automatically when the PicoMite starts and when the speaker is switched off and on again or comes back into range (the PicoMite keeps trying, at intervals of up to a minute), so BLUETOOTH CONNECT is normally needed only once. Only one speaker

is remembered - connecting to another replaces it. When the speaker connects or disconnects by itself the message “Bluetooth speaker connected” or “Bluetooth speaker disconnected” is shown at the command prompt.

If the speaker asks for a PIN when it pairs, 0000 is used. A speaker that needs a different PIN (its instructions will say, some car radios use 1234) is connected with the PIN added, for example:

```
BLUETOOTH CONNECT "Car Radio", "1234"
```

BLUETOOTH DISCONNECT disconnects the speaker and stops it being reconnected until the next BLUETOOTH CONNECT or restart. BLUETOOTH FORGET deletes the stored pairings, BLUETOOTH STATUS shows the state of the connections, and MM.INFO$(BLUETOOTH SPEAKER) returns the address of the connected speaker (or an empty string) so that a program can check that it is there.

The audio stream to the speaker starts as soon as the speaker connects. To save power it is suspended when nothing has been played for 60 seconds and it is restarted automatically by the next sound, which is then delayed slightly while the stream starts. The time can be changed by adding it to the option, for example OPTION AUDIO BLUETOOTH 10 (1 to 250 seconds; this does not restart the PicoMite). OPTION AUDIO BLUETOOTH 0 keeps the stream running all the time, which avoids the delay but uses more power and keeps the speaker awake.

If no speaker is connected the PLAY commands still work normally and take the same time, but the sound is discarded.

Bluetooth speakers buffer the audio, so the sound is heard a fraction of a second (typically 0.2 seconds, depending on the speaker) after it is played. This is not noticeable with music but sound effects in a game will lag slightly behind the action.

Encoding the audio for Bluetooth takes about 12% of the processor’s time at 200MHz while the stream is running, so a program runs slightly slower then. While the stream is running the PicoMite also listens less often for keyboards and mice, so a keyboard that has gone to sleep may take a little longer to reconnect.

## PicoMiteBTH – Bluetooth Serial Devices (COM7)

The PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3 versions can also connect to a Bluetooth serial device, which then becomes the serial port COM7. This can be a Bluetooth Classic device with a serial port (the Serial Port Profile, often called SPP), for example a Bluetooth GPS receiver, or a Bluetooth LE device with a serial service: the Nordic UART Service (used by many modules and by the console of the PicoMiteRP2350BT), the service of HM-10 modules or Microchip’s Transparent UART.

Make the device discoverable (see its instructions) and search for it with BLUETOOTH SCAN. A Bluetooth LE serial device is listed as ‘BLE serial’ and a Bluetooth Classic serial device without a type:

```
> BLUETOOTH SCAN
Scanning...
28:CD:C1:1A:A7:D3  -59 dBm  BLE serial  "PicoMite-500D"
00:19:01:2D:DC:BF  -81 dBm              "XGPS150-2DDCBF"
2 devices found
```

Then connect to it using the name shown by the scan (or its address):

```
BLUETOOTH CONNECT "XGPS150-2DDCBF"
```

For a Bluetooth Classic device the PicoMite first checks that it has a serial port (one that has not is treated as a speaker) and then pairs with it - if the device asks for a PIN, 0000 is used unless a PIN is given to BLUETOOTH CONNECT (many serial modules use 1234, for example BLUETOOTH CONNECT "HC-05", "1234"). A Bluetooth LE device is used without pairing unless it asks for it. The device is remembered. Only one serial device can be connected at a time and connecting another replaces the remembered one, but a serial device, a Bluetooth LE keyboard, a Bluetooth Classic keyboard and a speaker can all be connected at the same time.

COM7 is then opened and used like the USB serial ports COM3 to COM6, for example:

```
OPEN "COM7:" AS #1
```

or, for a GPS receiver:

```
OPEN "COM7:" AS GPS
```

A baud rate can be given but it is ignored, as are the 7BIT, EVEN, ODD, S2 and INV options. A receive buffer size, an interrupt subroutine and an interrupt trigger level can be specified exactly as for COM1 and COM2 - see Appendix A - Serial Communications. PRINT # ends each line with a carriage return and a line feed.

While COM7 is open the remembered device is reconnected automatically whenever it is not connected - when the port is opened, and when the device is switched off and on again or comes back into range - and

communications resume transparently. A Bluetooth Classic device is retried at intervals of up to a minute and a Bluetooth LE device is reconnected as soon as it is seen. When the device connects or disconnects by itself the message “Bluetooth serial device connected (COM7)” or “Bluetooth serial device disconnected (COM7)” is shown at the command prompt. CLOSE closes the port but leaves the device connected, and while the port is closed the device is not reconnected.

BLUETOOTH DISCONNECT SERIAL disconnects the device and stops it being reconnected until COM7 is next opened or the next BLUETOOTH CONNECT. BLUETOOTH STATUS shows the state of the connection and the number of bytes received and sent, and MM.INFO$(BLUETOOTH SERIAL) returns the address of the connected device (or an empty string) so that a program can wait for it to connect.

Some Bluetooth GPS receivers have a mode switch for use with an iPhone or iPad which hides their serial port - set it for other devices (Android or PC). A Bluetooth LE device that is switched off may take some seconds to be shown as disconnected, as this is only noticed when the link times out.

# MMBasic Characteristics

## Naming Conventions

Command names, function names, labels, variable names, etc are not case sensitive, so that "Run" and "RUN" are equivalent and "dOO" and "Doo" refer to the same variable.

The type of a variable can be specified in the DIM command or by adding a suffix to the end of the variable's name. For example the suffix for an integer is '%' so if a variable called nbr% is automatically created it will be an integer. There are three types of variables:

1. Floating point. These can store a number with a decimal point and fraction (eg, 45.386) and also very large numbers. However, they will lose accuracy when more than 14 significant digits are stored or manipulated. The suffix is '!' and floating point is the default when a variable is created without a suffix
2. 64-bit integer. These can store numbers with up to 19 decimal digits without losing accuracy but they cannot store fractions (i.e. the part following the decimal point). The suffix for an integer is '%'
3. Strings. These will store a string of characters (eg, "Tom"). The suffix for a string is the '$' symbol (eg, name$, s$, etc) Strings can be up to 255 characters long.

Variable names and labels can start with an alphabetic character or underscore and can contain any alphabetic or numeric character, the period (.) and the underscore (_). They may be up to 31 characters long. A variable name or a label must not be the same as a command or a function or one of the following keywords: THEN, ELSE, TO, STEP, FOR, WHILE, UNTIL, MOD, NOT, AND, OR, XOR, AS. Eg, step = 5 is illegal.

It is important to note that the alphanumeric name of the variable must be unique so you cannot have both a variable s$ and s%. As a result, if a variable is declared without a type suffix, it can be used with the relevant suffix in a program and will still reference the same variable.

For file names see the section *MMBasic Support for Flash and SD Card Filesystems*.

## Constants

Numeric constants may begin with a numeric digit (0-9) for a decimal constant, &H for a hexadecimal constant, &O for an octal constant or &B for a binary constant. For example &B1000 is the same as the decimal constant 8. Constants that start with &H, &O or &B are always treated as 64-bit integer constants.

Decimal constants may be preceded with a minus (-) or plus (+) and may be terminated with 'E' followed by an exponent number to denote exponential notation. For example 1.6E+4 is the same as 16000. If the decimal constant contains a decimal point or an exponent, it will be treated as a floating point constant; otherwise it will be treated as a 64-bit integer constant.

String constants are surrounded by double quote marks ("). Eg, "Hello World".

## Implementation Characteristics

Maximum program - see table. Note that MMBasic tokenises the program when it is stored in flash so the final size in flash might vary from the plain text size.

Maximum length of a command line is 255 characters.

Maximum length of a variable name or a label is 31 characters.

Maximum number of dimensions - see table.

Maximum number of arguments to commands that accept a variable number of arguments is 50.

Maximum number of nested FOR…NEXT loops is 20.

Maximum number of nested DO…LOOP commands is 20.

Maximum number of nested GOSUBs is 50.

Maximum number of nested multiline IF…ELSE…ENDIF commands is 20.

Maximum number of user defined subroutines and functions – see table. Labels are limited only by memory.

Maximum number of interrupt pins that can be configured: 10

Numbers are stored and manipulated as double precision floating point numbers or 64-bit signed integers. The range of floating point numbers is 1.797693134862316e+308 to 2.225073858507201e-308.

The range of 64-bit integers (whole numbers) that can be manipulated is ± 9223372036854775807.

Maximum string length is 255 characters.

Maximum line number is 65000.

Maximum number of background pulses launched by the PULSE command is 5.

Maximum number of global variables and constants - see table.

Maximum number of local variables - see table
The maximum number of files that can be listed by the FILES command is 1000.
The maximum length of a filename/path is 63 characters (RP2040) or 127 characters (RP2350).
Characteristics vs firmware features:

| | Maximum Program size | Maximum frequency | Maximum numbers of subroutines or functions | Maximum Array Dimensions | Default number of global variables | Default number of local variables* |
|---|---|---|---|---|---|---|
| PicoMite RP2040 | 120Kb | 420MHz | 256 | 6 | 256 | 240 |
| PicoMiteMIN RP2040** | 112Kb | 420MHz | 256 | 6 | 256 | 240 |
| PicoMiteUSB RP2040 | 120Kb | 420MHz | 256 | 6 | 256 | 240 |
| PicoMiteVGA RP2040 | 100Kb | 378MHz | 256 | 6 | 240 | 240 |
| PicoMiteVGAUSB RP2040 | 96Kb | 378MHz | 256 | 6 | 240 | 240 |
| WebMite RP2040 | 88Kb | 396MHz | 256 | 6 | 240 | 192 |
| PicoMite RP2350 | 208Kb | 420MHz | 512 | 5 | 576 | 192 |
| PicoMiteUSB RP2350 | 208Kb | 420MHz | 512 | 5 | 576 | 192 |
| PicoMiteVGA RP2350 | 168Kb | 378MHz | 512 | 5 | 544 | 192 |
| PicoMiteVGAUSB RP2350 | 160Kb | 378MHz | 512 | 5 | 544 | 192 |
| PicoMiteHDMI RP2350 | 160Kb | 378MHz | 512 | 5 | 544 | 192 |
| PicoMiteHDMIUSB RP2350 | 152Kb | 378MHz | 512 | 5 | 544 | 192 |
| WebMite RP2350 | 208Kb | 396MHz | 512 | 5 | 576 | 192 |
| PicoMiteBT RP2350 | 208Kb | 396MHz | 512 | 5 | 576 | 192 |
| PicoMiteBTH RP2350 | 208Kb | 396MHz | 512 | 5 | 576 | 192 |
| PicoMiteHDMIWEB RP2350 | 156Kb | 378MHz | 512 | 5 | 544 | 192 |
| PicoMiteHDMIBTH RP2350 | 180Kb | 378MHz | 512 | 5 | 544 | 192 |
| PicoMitePC3 RP2350B | 256Kb | 378MHz | 512 | 5 | 544 | 192 |

* The ratio of the number of global to local variables can be changed.
See OPTION LOCAL VARIABLES and MM.INFO(MAX VARS)

** The PicoMiteMIN is a cut down version of the standard PicoMite RP2040 intended to keep the image size to a minimum so that the A: drive is maximised when only 2Mbyte of flash are available. The Functionalities removed compared to the standard PicoMite are: Structures, Draw3D command and function, GUI, Turtle Graphics, Regular expression support, Help command, Audio VS1053 and SPI support, SSD1963 support, ILI9163, SSD1331, ST7735 but not ST7735S, ILI9481IPS, ILI9481, ILI9488P, GC9A01, Virtual displays, Editor F12, FileManager.

## Compatibility

MMBasic implements a large subset of Microsoft's GW-BASIC. There are numerous differences due to physical and practical considerations but most standard BASIC commands and functions are essentially the same. An online manual for GW-BASIC is available at http://www.antonis.de/qbebooks/gwbasman/index.html and this provides a more detailed description of the commands and functions.

MMBasic also implements a number of modern programming structures documented in the ANSI Standard for Full BASIC (X3.113-1987) or ISO/IEC 10279:1991. These include SUB/END SUB, the DO WHILE … LOOP, the SELECT…CASE statements and structured IF . THEN … ELSE … ENDIF statements.

# Predefined Read Only Variables

These variables are set by MMBasic and cannot be changed by the running program. Note that they do not do anything on their own, they must be printed or assigned to a variable.

For example:

```
> PRINT MM.VER
6.0002
>
```

or, in a program:

```
If MM.VER < 6.0002 Then Error "Needs version 6.00.02 or greater"
```

| | |
|---|---|
| MM.VER | Returns the version number of the firmware as a floating point number in the form aa.bbcc where aa is the major version number, bb is the minor version number and cc is the revision number. For example version 6.03.00 would return 6.03 and version 6.03.01 will return 6.0301. |
| MM.ADDRESS$ | WIFI VERSIONS ONLY<br>This variable returns the IP address of the sender of the last UDP datagram or MQTT packet received |
| MM.CMDLINE$ | This function returns any command line arguments passed to the current program when an MMBasic program runs.<br>see RUN and * commands for details.<br>The function will return an empty string for programs run from the editor or using OPTION AUTORUN. |
| MM.CODE | RP2350 VERSIONS ONLY Returns the number of free slots remaining in the G-code buffer used by the STEPPER command. See the file Stepper_Reference.pdf for the details. |
| MM.DEVICE$ | A string representing the device or platform that MMBasic is running on. |
| MM.DISPLAY | Returns 1 if a physical display is configured, otherwise 0 |
| MM.ERRNO<br>MM.ERRMSG$<br>MM.ERRLINE | If a statement caused an error which was ignored these variables will be set accordingly. MM.ERRNO is a number where non zero means that there was an error and MM.ERRMSG$ is a string representing the error message that would have normally been displayed on the console. They are reset to zero and an empty string by RUN, ON ERROR IGNORE or ON ERROR SKIP.<br>MM.ERRLINE is the number of the program line that caused the error. It is zero if the error did not occur in the program (eg, at the command prompt or in the library) and it is reset to zero along with the other two variables. |
| MM.FLAGS | Returns the value of the system flags register |
| MM.FONTHEIGHT<br>MM.FONTWIDTH | Returns the height or width of the current font in pixels |
| MM.HRES<br>MM.VRES | Integers representing the current horizontal and vertical resolution of the VGA/HDMI video output or the LCD display panel (if configured) in pixels. |

| | |
|---|---|
| MM.HEIGHT<br>MM.WIDTH | Returns the number of characters across the physical display with the current font or the number of characters down the display with the current font |
| MM.HPOS<br>MM.VPOS | Returns the current horizontal and vertical position (in pixels) following the last graphics or print command. |
| MM.SUPPLY | For modules with a pin allocated to measure the incoming supply voltage such as the Pico and Pico2, MM.SUPPLY will return the voltage. |
| MM.INFO()<br>MM.INFO$() | These two versions can be used interchangeably but good programming practice would require that you use the one corresponding to the returned datatype. |
| MM.INFO$(AUTORUN) | Returns the setting of the OPTION AUTORUN command |
| MM.INFO(ADC) | Returns the number of the buffer currently ready to read when using ADC RUN (1 or 2). Returns 0 if nothing ready. |
| MM.INFO(ADC DMA) | Returns true (1) if the ADC DMA is active. |
| MM.INFO(BATTERY) | PicoCalc only: Returns the battery level |
| MM.INFO(BBC BUFFER) | Returns the number of free slots in the PLAY BBC SOUND queue for channel n (0 to 3), or in the emptiest of the four channels if n is omitted. |
| MM.INFO(BIOS) | PicoCalc only: Returns the bios version |
| MM.INFO$(BLUETOOTH device) | BLUETOOTH HOST VERSIONS ONLY<br>Returns the address of the connected Bluetooth keyboard (or mouse), speaker or serial device if 'device' is KEYBOARD, SPEAKER or SERIAL, or an empty string if there is none. |
| MM.INFO(BOOT) | Tells you the reason for the last restart of the Pico<br>Returns:<br>Restart - the device has been restarted with CPU RESTART or an OPTION command<br>S/W Watchdog - the device has been restarted by a software watchdog timeout<br>H/W Watchdog - the device has been restarted by a hardware watchdog timeout<br>Firmware update - the device has been restarted following a firmware update<br>Power On - the device has been powered up<br>Brown Out - RP2350 only: the core supply fell below the brown-out threshold<br>Reset Switch - the device has been restarted using the reset switch<br>Unknown code &Hn - unknown reason - please report the code and version RP2040/2350 |
| MM.INFO(BOOT COUNT) | Returns the number of times the Pico has been restarted since the flash drive was last formatted. |
| MM.INFO(CHARGING) | PicoCalc only: Returns the charging status |
| MM.INFO$(CALLTABLE) | Returns the base address of the MMBasic call table which contains pointers to each MMBasic function. |
| MM.INFO$(CPUSPEED) | Returns the CPU speed as a string. |

| | |
|---|---|
| MM.INFO$(LCDPANEL) | Returns the name of the configured LCD panel or a blank string. In the case of VGA or HDMI versions returns the current mode. |
| MM.INFO(LCD320) | Returns true if the display is capable of 320x240 operation using the OPTION LCD320 command |
| MM.INFO$(COMPILE) | RP2350 VERSIONS ONLY<br>Returns what the last RUN did with the compiler (see OPTION COMPILE), for example COMPILED 3 REUSED 1 STMTS 3227 RAN 714382 MISS 25707 CODE 680945 SIZE 132348 OF 966656. COMPILED (NONE at the command prompt) and REUSED count the compiles, and the runs that found the program already compiled, since power on. STMTS is the number of statements compiled, RAN the number of statements the last run executed and CODE how many of those ran as compiled code, so CODE divided by RAN is the share that ran compiled. SIZE gives the space used, of the space available. MISS is internal. Returns OFF when the compiler is off, or TEXT: and the reason when the program ran entirely on the interpreter. |
| MM.INFO$(KEYBOARD) | USB versions with GUI controls: returns “On” if the on-screen keyboard is currently displayed under program control (see the KEYBOARD command and OPTION SCREEN KEYBOARD), otherwise “Off”. |
| MM.INFO$(SDCARD) | Returns the status of the SD Card. Valid returns are:<br>DISABLED, NOT PRESENT, READY, and UNUSED |
| MM.INFO$(CURRENT) | Returns the name of the current program when loaded from a file or NONE if called after a NEW, AUTOSAVE, XMODEM or EDIT Command. |
| MM.INFO$(PATH) | Returns the path of the current program or NONE if called after a NEW or EDIT Command. |
| MM.INFO(DISK SIZE) | Returns the capacity of the Flash Filesystem or SD Card, whichever is the active drive, in bytes |
| MM.INFO$(DRIVE) | Returns the active drive “A:” or “B:” |
| MM.INFO(EXISTS FILE fname$) | Returns 1 if the file specified exists, returns -1 if fname$ is a directory, otherwise returns 0. |
| MM.INFO(EXISTS DIR dirname$) | Returns a Boolean indicating whether the directory specified exits. |
| MM.INFO(FREE SPACE) | Returns the free space on the Flash Filesystem or SD Card whichever is the active drive. |
| MM.INFO(FILESIZE file$) | Returns the size of file$ in bytes or 0 if not found. |
| MM.INFO$(MODIFIED file$) | Returns the date/time that file$ was modified, Empty string if not found. |
| MM.INFO$(SYSTEM I2C) | Returns “I2C”, “I2C2”, or “Not set” depending on the status of OPTION SYSTEM I2C |
| MM.INFO(FCOLOUR) | Returns the current foreground colour. |
| MM.INFO(BCOLOUR) | Returns the current background colour. |
| MM.INFO(FONT) | Returns the number of the currently active font. |
| MM.INFO(FONT ADDRESS n) | Returns the address of the memory location with the address of FONT n . |
| MM.INFO(FONT POINTER n) | Returns a POINTER to the start of FONT n in memory. |

| | |
|---|---|
| MM.INFO(FONTHEIGHT)<br>MM.INFO(FONTWIDTH) | Integers representing the height and width of the current font (in pixels). |
| MM.INFO(FLASH) | Reports which flash slot the program was loaded from if applicable. |
| MM.INFO(FLASHTOP) | Returns the address of the first byte of flash memory above the area used by the firmware, options, saved variables, flash slots and program storage. |
| MM.INFO(FLASH ADDRESS n) | Returns the address of image slot n: a flash slot for 1 to 3 or, on an RP2350 with PSRAM, RAM slot n-3 for 4 to 8 (see FLASH LOAD IMAGE). |
| MM.INFO(HEAP) | Returns the amount of MMbasic Heap memory free. MMBasic heap is used for strings, arrays and various other temporary and permanent buffers (eg, audio) |
| MM.INFO(HPOS)<br>MM.INFO(VPOS) | The current horizontal and vertical position (in pixels) following the last graphics or print command. |
| MM.INFO(ID) | Returns the unique ID of the Pico. |
| MM.INFO$(IP ADDRESS) | Returns the IP address of the WebMite |
| MM.INFO(MAX GP) | Returns the highest valid GPno on the chip |
| MM.INFO(MAX VARS) | Returns the total number of variables available in the current version |
| MM.INFO$(MODBUFF ADDRESS) | Returns the address in memory of the buffer used for storing MOD files |
| MM.INFO$(OPTION option) | Returns the current value of a range of options that affect how a program will run. “option” can be one of AUTORUN, AUDIO, BASE, BREAK, CONSOLE, DEFAULT, EXPLICIT, KEYBOARD, ANGLE, HEIGHT, WIDTH, FLASH SIZE, AUTOREFRESH or SSID (WebMite only) |
| MM.INFO(HEAPEND)<br>MM.INFO(STACKPEAK)<br>MM.INFO(CHEAPMAX)<br>MM.INFO(CHEAPTOTAL)<br>MM.INFO(TLSHEAP)<br>MM.INFO(TLSHEAPMAX)<br>MM.INFO(TLSHEAPERR) | These report internal memory-allocator values and exist to help debug the firmware itself. They are not part of the supported MMBasic language and may change or be withdrawn without notice, so programs should not rely on them. MM.INFO(HEAPEND) is available on all versions; the others are on the WiFi versions only (MM.INFO(STACKPEAK) on WebMiteRP2350, PicoMiteHDMIWEB and PicoMitePC3 only). |
| MM.INFO$(PIN pinno) | Returns the status of I/O pin 'pinno'. Valid returns are:<br>OFF, DIN, DOUT, AIN, etc |
| MM.INFO(PINNO GPnn) | Returns the physical pin number for a given GP number. GPnn can be an unquoted string (GP01), a string literal(“GP01”) or a string variable. Ie, A$=”GP01”: MM.INFO(PINNO A$) . |
| MM.INFO(PIO RX DMA) | Indicates whether the PIO RX DMA channel is busy |
| MM.INFO(PIO TX DMA) | Indicates whether the PIO TX DMA channel is busy |
| MM.INFO$(PLATFORM) | Returns the string previously set with OPTION PLATFORM. |
| MM.INFO(PROGRAM) | Returns the address in memory of the currently running program |
| MM.INFO(PSRAM SIZE) | Returns the size of any PSRAM if available. The RP2040 always returns 0. |
| MM.INFO(PS2) | Reports the last raw message received on the PS2 interface if enabled. |

| | |
|---|---|
| MM.INFO(PWM COUNT) | Returns the number of PWM channels supported by the chip |
| MM.INFO(PWM DUTY C%, n%) | Returns the current duty cycle in clock counts of PWM channel C%,N% Where N%=0 for A and 1 for B |
| MM.INFO(PWM SLICE pinno) | This returns the PWM slice for a given pin (0-7 RP2040, 0-11 RP2350) |
| MM.INFO$(SOUND) | Returns the current activity on the audio output: OFF, TONE, SOUND, BBC, WAV, FLAC, MP3, MOD, MIDI, STREAM, ARRAY or SAMPLE, or PAUSED followed by the name of the paused activity (e.g. PAUSED BBC) |
| MM.INFO(SPI SPEED | Returns the actual speed of the SYSTEM SPI or an error if not set |
| MM.INFO(STACK) | Returns the C stack pointer. Complex or recursive Basic code may result in the error "Stack overflow, expression too complex at depth %" This will occur when the stack is below &H 2003f800. Monitoring the stack will allow the programmer to identify simplifications to the Basic code to avoid the error. |
| MM.INFO(SYSTEM HEAP) | Returns the free space on the System Heap. |
| MM.INFO(SYSTICK) | Returns the current value of the system 24-bit systick timer which runs at the CPU clock speed |
| | VGA AND HDMI VERSIONS ONLY |
| MM.INFO(TILE HEIGHT) | Returns the current setting of the tile height. |
| MM.INFO(TRACK) | Returns the name of the FLAC, MP3, WAV or MIDI file currently playing on the audio output. |
| MM.INFO$(TOUCH) | Returns the status of the Touch controller. Valid returns are: “Disabled”, “Not calibrated”, and “Ready”. |
| MM.INFO(USB n) | Return the device code for any device connected to channel ‘n’ which is a number from 1 to 4. The returned device code can be:<br>0=not in use, 1=keyboard, 2=mouse, 128=ps4, 129=ps3, 130=SNES/Generic, 131=xbox, 132=unknown, 133=touch screen, 134=touch screen (pointer mode)<br>By default a connected keyboard will be allocated to channel 1, a mouse the channel 2, and gamepads to channel 3 and then channel 4. If 2 or more keyboards or mice are connected or 3 or more gamepads then the additional devices will be allocated to the highest available channel. |
| MM.INFO(USB VID n) | Returns the VID of the USB device on channel n |
| MM.INFO(USB PID n) | Returns the PID of the USB device on channel n |
| MM.INFO(VARCNT) | Returns the number of variables in use in the MMBasic program. |
| MM.INFO$(LINE) | Returns the current line number as a string. LIBRARY returned if in the Library and UNKNOWN if not in a program. Assists in diagnostics while unit testing. |
| MM.INFO(UPTIME) | Returns the time in seconds since booted as a floating point number. |
| MM.INFO(VALID CPUSPEED speed%) | Returns 1 if ‘speed%’ is valid for OPTION CPUSPEED ‘speed%’ |

| | |
|---|---|
| MM.INFO(VERSION) | The version number of the firmware (MM.VER converts to this) |
| MM.INFO(WRITEBUFF) | Returns the address in memory of the current buffer used for drawing commands. |
| MM.INFO(SCROLL) | RP2350 versions with a buffered LCD panel: returns the current vertical scroll offset of the display in pixels. |
| MM.INFO(SCREENBUFF) | RP2350 versions with a buffered LCD panel: returns the address in memory of the screen buffer. |
| MM.INFO(TCP PORT)<br>MM.INFO(UDP PORT)<br><br>MM.INFO(MAX CONNECTIONS)<br><br>MM.INFO(TCP REQUEST n) | WIFI VERSIONS ONLY<br>Returns the TCP port set as a server or 0 if not set<br>Returns the UDP port set as a server or 0 if not set<br><br>Returns the maximum number of simultaneous connections supported by the TCP server. Typically used to step through the connections in a WEB TCP INTERRUPT subroutine.<br><br>Returns true if connection number 'n' (1 to MM.INFO(MAX CONNECTIONS)) has a request pending for the TCP server. |
| MM.INFO(TCPIP STATUS)<br>MM.INFO(WIFI STATUS) | WIFI VERSIONS ONLY<br>Returns the TCPIP status of the connection<br>Returns the WIFI status of the connection.<br>Valid returns are:<br>0 WiFi is down<br>1 Connected to WiFi<br>2 Connected to WiFi, but no IP address (TCPIP STATUS only)<br>3 Connected to WiFi with an IP address (TCPIP STATUS only)<br>-1 Connection failed<br>-2 No matching SSID found (could be out of range, or down)<br>-3 Authentication failure |
| MM.INFO(TELNET) | WIFI VERSIONS ONLY<br>Returns 1 if a Telnet client is currently connected to the console, otherwise 0. This reports the connection, not the setting, so it will return 0 whenever nothing is connected even though OPTION TELNET CONSOLE is set to ON or ONLY. |
| MM.MESSAGE$ | WIFI VERSIONS ONLY<br>Returns the contents of the last UDP datagram received or last MQTT packet received |
| MM.TOPIC$ | WIFI VERSIONS ONLY<br>Returns the topic of the last MQTT packet received |
| MM.ONEWIRE | Following a 1-Wire reset function this integer variable will be set to indicate the result of the operation: 0 = Device not found, 1 = Device found, 2=Device timeout. |
| MM.I2C | Following an I2C write or read command this integer variable will be set to indicate the result of the operation as follows:<br>0 = The command completed without error.<br>1 = Received a NACK response<br>2 = Command timed out |

| | |
|---|---|
| MM.PERSISTENT | Returns a value saved with the command SAVE PERSISTENT |
| MM.PS2 | Returns the last code received on the PS2 interface if enabled. |
| MM.USB | On versions with USB support: returns the last code received from a USB keyboard (the USB equivalent of MM.PS2 which is not available on these versions). |
| MM.WATCHDOG | An integer which is true if MMBasic was restarted as the result of a Watchdog timeout (see the WATCHDOG command) otherwise false. |

# Options

This table lists the various option commands which can be used to configure MMBasic and change the way it operates. Options that are marked as permanent will be saved in non-volatile memory and automatically restored when the PicoMite firmware is restarted. Options that are not permanent will be reset on start-up, reset and in many cases when a program is run and/or exited.

Many OPTION commands will force a restart of the PicoMite firmware and that will cause the USB console interface to be reset. The program in memory will not be lost as it is held in non-volatile flash memory.

| | Permanent? | |
|---|---|---|
| OPTION ANGLE RADIANS \| DEGREES | | This command switches trig functions between degrees and radians. Acts on SIN, COS, TAN, ATN, ATAN2, MATH ATAN3, ACOS, ASIN |
| OPTION AUDIO PWMnApin, PWMnBpin<br>or<br>OPTION AUDIO DISABLE | ✓ | NOT BLUETOOTH HOST VERSIONS Configures one of the PWM channels as an audio output.<br>'PWMnApin' is the left audio channel, 'PWMnBpin' is the right. Both pins must belong to the same audio channel.<br>Example, OPTION AUDIO GP18, GP19 would use PWM1A and PWM1B on pins 24 and 25 respectively.<br>This option prevents use of these pins in the BASIC program.<br>The audio output is generated using PWM so a low pass filter is necessary on the output. The audio output from the Raspberry Pi Pico is very noisy. Using OPTION POWER and/or supplying power via a separate 3.3V linear regulator can reduce this.<br>This command must be run at the command prompt (not in a program). |
| OPTION AUDIO SPI CSpin, CLKpin, MOSIpin<br>or<br>OPTION AUDIO DISABLE | ✓ | NOT PICOMITEMIN OR BLUETOOTH HOST VERSIONS<br>Configures the audio output to be directed to a MCP48n2 DAC connected to the specified pins. The LDAC pin on the DAC should be connected to GND. |
| OPTION AUDIO VS1053 CLKpin, MOSIpin, MISOpin, XCSpin, XDCSpin, DREQpin, XRSTpin<br>or<br>OPTION AUDIO DISABLE | ✓ | NOT PICOMITEMIN OR BLUETOOTH HOST VERSIONS<br>Configures the audio output to be directed to a VS1053 CODEC. This allows MP3 and MIDI playback in addition to the other formats supported and also supports real-time MIDI output. See the PLAY command for more details |
| OPTION AUDIO I2S BCLKpin, DINpin<br>or<br>OPTION AUDIO DISABLE | ✓ | Configures the audio output to be directed to an I2S DAC connected to the specified pins. The LRCK pin on the DAC should be connected to the next consecutive GPIO pin to BCLKpin. |
| OPTION AUDIO BLUETOOTH [idle]<br>or<br>OPTION AUDIO DISABLE | ✓ | BLUETOOTH HOST VERSIONS ONLY Directs the audio output to a Bluetooth speaker or headphones, which are connected with the BLUETOOTH CONNECT command. No I/O pins are used. See the chapter Bluetooth Versions for the details.<br>'idle' is the time in seconds (0 to 250) after the last sound before the audio stream to the speaker is suspended. The stream starts as soon as the speaker connects, and once suspended it is restarted automatically by the next sound, which is delayed slightly while it starts. 0 means that the stream is never suspended. The default is 60 seconds. |

| | | |
|---|---|---|
| | | If Bluetooth is not already the audio output the PicoMite will restart after this command. If it is, the command just changes 'idle'.<br>This command must be run at the command prompt (not in a program). |
| OPTION AUTOREFRESH OFF \| ON | | NOT VGA OR HDMI VERSIONS Black and white displays can only be updated a full screen at a time. By using OPTION AUTOREFRESH OFF/ON you can control whether a write command immediately updates the display or not. If AUTOREFRESH is OFF the REFRESH command can be used to trigger the write. This applies to the following displays: N5110, SSD1306I2C, SSD1306I2C32, SSD1306SPI and ST7920 |
| OPTION AUTORUN ON [,NORESET]<br>or<br>OPTION AUTORUN n [,NORESET]<br>or<br>OPTION AUTORUN OFF | ✓ | Instructs MMBasic to automatically run a program on power up or restart.<br>ON will cause the current program in program memory to be run.<br>Specifying 'n' will cause that location in flash memory to be run. 'n' must be in the range 1 to 3.<br>Specifying the optional parameter "NORESET" will maintain AUTORUN even if the program causes a system error (by default this will cause the firmware to cancel any OPTION AUTORUN setting).<br>OFF will disable the autorun option and is the default for a new program.<br>Entering the break key (default CTRL-C) at the console will interrupt the running program and return to the command prompt. |
| OPTION BASE 0 \| 1 | | Set the lowest value for array subscripts to either 0 or 1.<br>This must be used before any arrays are declared and is reset to the default of 0 on power up. |
| OPTION BACKLIGHT KBD n<br>OPTION BACKLIGHT LCD n | ✓ | PICOCALC ONLY<br>Sets the keyboard backlight level 0-100%<br>Sets the display backlight level 0-100% |
| OPTION BAUDRATE nn | | Set the baudrate of the serial console (if it is configured). |
| OPTION BREAK nn | | Set the value of the break key to the ASCII value 'nn'. This key is used to interrupt a running program.<br>The value of the break key is set to CTRL-C key at power up and when a program is run but it can be changed to any keyboard key using this command in a program (for example, OPTION BREAK 4 will set the break key to the CTRL-D key). Setting this option to zero will disable the break function entirely. |
| OPTION CASE LOWER \| UPPER \| TITLE | ✓ | Change the case used for listing command and function names when using the LIST command. The default is TITLE but the old standard of MMBasic can be restored using OPTION CASE UPPER. |
| OPTION CDC POLL RATE n | ✓ | Sets the polling interval of the USB serial (CDC) console in microseconds. 'n' can be 50 to 2000 (the default is 1000). A smaller value makes the console more responsive at the cost of increased CPU overhead. This setting is not saved and reverts to the default on reset or power up. |

| | | |
|---|---|---|
| OPTION COLOURCODE ON<br>or<br>OPTION COLOURCODE OFF | ✓ | Turn on or off colour coding for the editor's output. Keywords will be in cyan, comments in yellow, etc. The default is OFF.<br>The keyword COLORCODE (USA spelling) can also be used.<br>This will work on VGA/HDMI video and the serial console using a terminal emulator with VT100 emulation (eg, Tera Term). This command must be run at the command prompt (not in a program). |
| OPTION COMPILE ON<br>or<br>OPTION COMPILE SHADOW<br>or<br>OPTION COMPILE OFF | ✓ | RP2350 VERSIONS ONLY Controls the compiler. With ON, RUN compiles the program (and its library, if there is one) into a stream of native operations and runs that. Statements that the compiler cannot take run on the interpreter as before, interleaved with the compiled ones, so a program does not need to be changed and gives exactly the same results.<br>SHADOW compiles in the same way but also works out every compiled value again with the interpreter, and stops at the first statement where the two disagree. It is slower than ON and is meant for checking a program. OFF (the default) interprets the program only.<br>The setting takes effect at the next RUN and is saved with the other options. A program is compiled again only when it, or its library, has changed since the last compile, so the first RUN after an edit takes a little longer. MM.INFO$(COMPILE) reports what the last RUN compiled and how much of the program ran compiled, and OPTION PROFILING ON lists the statements that ran on the interpreter.<br>With PSRAM the compiled program is kept in PSRAM reserved by the firmware; without PSRAM it is kept in flash after the program memory. |
| OPTION CONSOLE output | | Specifies where print statements will output. Valid settings are BOTH (i.e. SCREEN and SERIAL), SERIAL, SCREEN, NONE. This is a temporary option that is reset when a program exits. |
| OPTION CONTINUATION LINES ON/OFF | ✓ | Enables or disables the use of continuation lines in the editor and with the LIST command. Line continuation is indicated by a space followed by an underscore character at the end of a line. When enabled the editor will automatically split lines as it reads them from file and add the continuation characters required. When exiting the editor, the continuation characters are removed before saving. While in the editor the user can create long lines by adding their own continuation characters. This makes using a small screen as a console much easier. |
| OPTION CPUSPEED speed | ✓ | NOT VGA OR HDMI VERSIONS<br>Change the CPU clock speed.<br>'speed' is the CPU clock in KHz. The permitted range depends on the version: 48000 to 420000 on the PicoMite versions, 126000 to 396000 on the WebMite versions, and 200000 to 396000 on the Bluetooth versions. Speeds above 200MHz (150MHz on the RP2350) are regarded as overclocking as those are the chip manufacturer's specified maximums.<br>The default speed is 200000 on both the RP2040 and the RP2350.<br>This command must be run at the command prompt (not in a program).<br>See also the CPU SPEED command which changes the clock speed immediately, without saving it or restarting the firmware, and can be used in a program. |
| OPTION COUNT pin1, pin2, pin3, pin4 | ✓ | Specifies which pins are to be used as Count inputs. By default these are GP6, GP7, GP8 and GP9. The SETPIN command defines the Counter mode.<br>This command must be run at the command prompt (not in a program). |

| | | |
|---|---|---|
| OPTION CYW43 PINS wl_on, wl_d, wl_cs, wl_clk<br>or<br>OPTION CYW43 PINS DEFAULT | ✓ | RP2350 WIFI AND BLUETOOTH VERSIONS ONLY Specifies the pins used to connect to the wireless chip, for modules that do not connect it in the same way as the Raspberry Pi Pico 2 W (GP23, GP24, GP25 and GP29). 'wl_on' is the chip's power-on pin, 'wl_d' its data pin, 'wl_cs' its chip select and 'wl_clk' its clock. For example, the Waveshare RP2350B Pico WiFi board needs OPTION CYW43 PINS GP36, GP37, GP38, GP39. DEFAULT returns to the standard pins.<br>The pins must be unused. 'wl_d' and 'wl_clk' must both be within GP0 to GP31 or both within GP16 to GP47. If an I2S DAC is configured it shares a PIO with the wireless chip, so its pins must be within the same range.<br>The configured pins are reserved and cannot be used by a program. If the wireless chip does not respond on them the firmware reports this when it starts and runs without WiFi or Bluetooth. OPTION RESET also returns to the standard pins.<br>This command must be run at the command prompt (not in a program) and will cause a restart. |
| OPTION DEFAULT FLOAT \| INTEGER \| STRING \| NONE | | Used to set the default type for a variable which is not explicitly defined.<br>If OPTION DEFAULT NONE is used then all variables must have their type explicitly defined or the error "Variable type not specified" will occur.<br>When a program is run the default is set to FLOAT for compatibility with Microsoft BASIC and previous versions of MMBasic. |
| OPTION DEFAULT COLOURS foreground [,background] | ✓ | Set the default foreground and background colours for both the monochrome and colour modes. The colour must be one of the following: white, yellow, lilac, brown, fuchsia, rust, magenta, red, cyan, green, cerulean, midgreen, cobalt, myrtle, blue and black. A numeric value cannot be used. The default is white, black.<br>If background is omitted it defaults to black. |
| OPTION DEFAULT MODE n | ✓ | VGA AND HDMI VERSIONS ONLY This sets the default display mode on boot. This command must be run at the command prompt (not in a program). |
| OPTION DISK SAVE fname$<br>OPTION DISK LOAD fname$ | ✓ | NOT PICOMITEMIN These commands let the user save and restore the complete set of options defined to and from a disk file. The file could then be transferred to a host computer using XMODEM allowing additional devices to be easily configured or options recovered after a firmware upgrade |
| OPTION DISPLAY lines [,chars] | ✓ | Set the characteristics of the display terminal used for the console. Both the LIST and EDIT commands need to know this information to correctly format the text for display.<br>'lines' is the number of lines on the display and 'chars' is the width of the display in characters. The default is 24 lines x 80 chars and when changed this option will be remembered even when the power is removed. Maximum values are 100 lines and 240chars.<br>This will send an ESC sequence to set the VT100 terminal to the matching size. TerraTerm, Putty and MMCC respond to this sequence and set the terminal width (if the option is enabled in the terminal setup).<br>**This option is not available if an LCD display is being used as the console.** |
| OPTION ESCAPE | | Enables the ability to insert escape sequences into string constants. See the section *Special Characters in Strings*. |

| | | |
|---|---|---|
| OPTION EXPLICIT | | Placing this command at the start of a program will require that every variable be explicitly declared using the DIM, LOCAL or STATIC commands before it can be used in the program.<br>This option is disabled by default when a program is run. If it is used it must be specified before any variables are used. |
| OPTION FAST AUDIO ON\|OFF | | When using the PLAY SOUND command, changes to sounds, volumes, or frequencies can cause audible clicks in the output. The firmware attempts to mitigate this by ramping the volume down on the channel's previous output before changing the output and ramping it back up again. This significantly improves the audio output but at the expense of a short delay in the PLAY SOUND command (worst case 3mSec). This delay can be avoided using OPTION FAST AUDIO ON in a program. The audible clicks may then re-appear but this is at the programmer's discretion.<br>This is a temporary option that is reset to OFF whenever a program is run. |
| OPTION FNKey string$ | ✓ | Define the string that will be generated when a function key is pressed at the command prompt. 'FNKey' can be F1, and F5 thru to F9. Setting string$ to "" erases the saved function and restores the original function of the key.<br>Example:<br>OPTION F8 "RUN "+chr$(34)+"myprog" +chr$(34)+chr$(13)+chr$(10).<br>This command must be run at the command prompt (not in a program). |
| OPTION GPS | ✓ | Configures the PicoMite to interface with a GPS module. This integration allows the PicoMite to automatically parse NMEA data streams in the background, making location, time, and speed data available to the user program via the `GPS()` function.<br>This is described in detail in the file *Option_GPS_User_Manual.pdf* included in the firmware download zip file. |
| OPTION GUI CONTROLS n | ✓ | <u>GUI CONTROLS VERSIONS ONLY</u><br>Sets the maximum number of advanced graphic controls (see Advanced Graphics Functions.pdf) that can be defined. 'n' can be 0 to 200. Setting 'n' to zero disables the graphic controls and recovers the memory used to manage them. The option is saved and the firmware restarts. |
| OPTION HEARTBEAT ON/OFF [HEARTBEATpin] | ✓ | Enables or disables the output of the heartbeat LED.<br>In the case of the Pico-W the heartbeat is on a pin controlled by the CWY43 chip.<br><u>NOT WEBMITE VERSION</u><br>By default, for RP2350A chips the heartbeat is enabled on GP25.<br>If it is disabled the program can control the LED via GP25.<br>For RP2350B chips the heartbeat is not enabled except for WebMiteRP2350 and HDMIWEB firmware.<br>If the heartbeat is disabled then the command can be used both to enable it and optionally specify the pin to use (default GP25) |
| | ✓ | <u>HDMI VERSIONS ONLY</u><br>Set the I/O pins used for the HDMI video output. This is only required to suit nonstandard PCB layouts. |

| | | |
|---|---|---|
| OPTION HDMI PINS clockpositivepin, d0positivepin, d1positivepin, d2positivepin | | The positive HDMI signal pins are set according to 'nbr' below. Valid values are 0-7 and the pins must not overlap for each channel. If 'nbr' is an even number the negative output is on physical pin+1, if 'nbr' is odd it will be on physical pin-1.<br>nbr HSTX Nbr Physical Pin<br>0 HSTX0 GP12<br>1 HSTX1 GP13<br>2 HSTX2 GP14<br>3 HSTX3 GP15<br>4 HSTX4 GP16<br>5 HSTX5 GP17<br>6 HSTX6 GP18<br>7 HSTX7 GP19<br>The default is: OPTION HDMI PINS 2, 0, 6, 4<br>Which means that:<br>CK+ and CK- are allocated to GP14 and GP15<br>D0+ and D0- are allocated to GP12 and GP13<br>D1+ and D1- are allocated to GP18 and GP19<br>D2+ and D2- are allocated to GP16 and GP17 |
| OPTION KEYBOARD nn [,capslock] [,numlock] [,repeatstart] [,repeatrate]<br>or<br>OPTION KEYBOARD DISABLE | ✓ | PS2 KEYBOARD VERSIONS ONLY Configure a PS2 keyboard. This can be used for console input and any characters typed will be available via any commands that read from the console. 'nn' is a two character code defining the keyboard layout: US for the standard layout used in the USA, Australia and New Zealand, UK for the United Kingdom, FR for France, GR for Germany, IT for Italy, ES for Spain, BE for Belgium and BR for Brazil. The optional parameters 'capslock' and 'numlock' are true/false integers that set the initial state of the keyboard (default is 0 and 1). The optional parameters 'repeatstart' and 'repeatrate' set the time before a held key first repeats and the rate of subsequent repeats: 'repeatstart' is 0 to 3 indicating 250mSec, 500mSec, 750mSec and 1000mSec (default is 1) and 'repeatrate' is 0 to 31 indicating 33mSec to 500mSec as per the PS2 keyboard specification (default is 12 or 100mSec). This command must be entered at the command line and will cause a reboot. The setting can be removed with OPTION KEYBOARD DISABLE. |
| OPTION KEYBOARD nn [,capslock] [,numlock] [,repeatstart] [,repeatrate] | ✓ | USB AND BLUETOOTH KEYBOARD VERSIONS ONLY Set the layout and the initial state of a USB or Bluetooth keyboard. 'nn' is a two character code: US, UK, FR, GR, IT or ES. BE and BR are not available on these versions. The optional parameters 'capslock' and 'numlock' are true/false integers that set the state of Caps Lock and Num Lock when a USB keyboard is connected (default is 0 and 1); a Bluetooth keyboard always starts with both off. A keyboard with a numeric keypad built into its letter keys, such as the Raspberry Pi keyboard, needs 'numlock' set to 0 or keys such as U, I and O will type 4, 5 and 6. The optional parameters 'repeatstart' and 'repeatrate' set the time before a held key first repeats (100 to 2000mSec, default 600mSec) and the time between subsequent repeats (25 to 2000mSec, default 150mSec); if they are omitted the current setting is kept. These can also be set with OPTION KEYBOARD REPEAT. OPTION KEYBOARD DISABLE is not available. This command must be entered at the command line and will cause a reboot. |
| OPTION KEYBOARD I2C | ✓ | PS2 KEYBOARD VERSIONS ONLY Configures support for the Solderparty bbq20 mini I2C keyboard.<br>Note: OPTION SYSTEM I2C must be set before executing this command |

| | | |
|---|---|---|
| OPTION KEYBOARD PINS clockpin, datapin | ✓ | NON-USB VERSIONS ONLY Allows the user to select the pins to be used for connecting a PS2 keyboard. The default is pin 11 (GP8) and pin 12 (GP9).<br>The PS2 keyboard must be disabled (OPTION KEYBOARD DISABLE) |
| OPTION PS2 PINS clockpin, datapin | ✓ | NON-USB VERSIONS ONLY An alternative spelling of OPTION KEYBOARD PINS - the two are interchangeable. |
| OPTION KEYBOARD REPEAT repeatstart , repeatrate | ✓ | USB AND BLUETOOTH KEYBOARD VERSIONS ONLY<br>Sets the time for the first repeat of a key that is held down (100-2000) and subsequent repeats (25-2000) in milliseconds. |
| OPTION LCD SPI clkpin, mosipin, misopin<br>or<br>OPTION LCD SPI DISABLE | ✓ | RP2350 PICOMITE VERSIONS ONLY The firmware supports separating out the SPI pins used to drive a LCD display from those used for touch and/or the SDcard. By keeping these separate, this allows the firmware to use the second processor to work with the display with no risk of impacting the other functions. Setting the LCD SPI pins is a pre-requisite for using buffered drivers with the RP2350 PicoMite. The pins specified must be valid SPI pins for the functions clk, mosi (tx), and miso (rx). Where SYSTEM SPI pins are also specified, these must be on a different SPI channel.<br>OPTION LCD SPI DISABLE releases the pins. |
| OPTION LOCAL KEYBOARD ON or OPTION LOCAL KEYBOARD OFF | ✓ | RP2350 PICOMITE VERSIONS ONLY Enables or disables the local matrix keyboard fitted to devices such as the PicoCalc. The PS2 keyboard must be disabled (OPTION KEYBOARD DISABLE) before this option can be set. This command must be run at the command prompt and will cause a reboot. |
| OPTION LCDPANEL | ✓ | NOT VGA OR HDMI VERSIONS<br>Configures an LCD panel on versions that accept a connected LCD. |
| OPTION LCDPANEL VIRTUAL_C<br>or<br>OPTION LCDPANEL VIRTUAL_M | | Configures a virtual LCD panel without a physically connected panel.<br>VIRTUAL _C = Colour, 4bit, 320 x 240<br>VIRTUAL _M = Monochrome, 640 x 480<br>Using this feature a program can draw graphical images on this virtual panel and then save them as a BMP file. Useful for creating a graphic image for export without an attached display |
| OPTION LCDPANEL options<br>or<br>OPTION LCDPANEL DISABLE | | NOT VGA OR HDMI VERSIONS Configures the PicoMite firmware to work with an attached LCD panel.<br>See the chapter *LCD Displays* for the details.<br>This command must be run at the command prompt (not in a program). |
| OPTION LCDPANEL CONSOLE [font [, fc [, bc [, blight]]] [,NOSCROLL]<br>or<br>OPTION LCDPANEL NOCONSOLE | | NOT VGA OR HDMI VERSIONS Configures the LCD display panel for use as the console output. The LCD must support transparent text (i.e. the SSD1963_x, ILI9341 or ST7789_320 controllers).<br>'font' is the default font, 'fc' is the default foreground colour, 'bc' is the default background colour. These parameters are optional and default to font 1, white, black and 100%. These settings are applied at power up.<br>The optional NOSCROLL command changes the firmware such that when outputting to the last line of the display rather than the display scrolling it is cleared and output continues at the top of the display. This |

| | | |
|---|---|---|
| OPTION LCDPANEL USER hres, vres | | allows displays that don't support reading to be used as a console device and:<br>Note that for displays other than the SSD1963 scrolling for any console output is very slow so it is recommended to use the NOSCROLL option for these displays. This setting is saved in flash and will be automatically applied on startup. To disable it use the OPTION LCDPANEL NOCONSOLE command.<br>This command must be run at the command prompt.<br><br>NOT VGA OR HDMI VERSIONS Configures a user written display driver in MMBasic. See the file "User Display Driver.txt" in the PicoMite firmware distribution for a description of how to write the driver. |
| OPTION LCDPANEL CONSOLE [font [, fc [,bc]]<br><br>or<br><br>OPTION LCDPANEL NOCONSOLE | ✓ | VGA AND HDMI VERSIONS ONLY<br>Changes the default font used on the VGA or HDMI display.<br>'fc' is the foreground colour and 'bc' is the background colour.<br>Disables the console output to the VGA/HDMI display. This option is permanent, both print output and console output will be disabled and only graphics commands will output to the VGA screen. If output is required to be temporarily disabled in a program use the OPTION CONSOLE command<br>For SSD1963 based displays in landscape and SPI displays in portrait the firmware uses H/W scrolling to improve display console performance. |
| OPTION LCD320 ON/OFF | | NOT VGA OR HDMI VERSIONS<br>This enables or disables 16-bit LCD displays in 320x240 mode allowing things like games on these larger LCD displays. In the case of 800x480 displays the 320x240 image is scaled by 2 and occupies the screen area 80,0 to 719,479<br>In the case of 480x272 displays the 320x240 image is windowed and occupies the screen area 80,16 to 399,255 |
| OPTION LEGACY ON<br>or<br>OPTION LEGACY OFF | | This will turn on or off compatibility mode with the graphic commands used in the original Colour Maximite. The commands COLOUR, LINE, CIRCLE and PIXEL use the legacy syntax and all drawing commands will accept colours in the range of 0 to 7. Notes:<br>• Keywords such as RED, BLUE, etc are not implemented so they should be defined as constants if needed.<br>• Refer to the Colour Maximite MMBasic Language Manual for the syntax of the legacy commands. This can be downloaded from https://geoffg.net/OriginalColourMaximite.html . |
| OPTION LIST | | This will list the settings of any options that have been changed from their default setting and are the permanent type. OPTION LIST also shows the version number and which firmware is loaded. This command must be run at the command prompt (not in a program). |
| OPTION LOCAL VARIABLES n | | This command sets the number of the total variables (MM.INFO(MAX VARS) to assign as local variables. In general, this should not be needed but, for example, if you want to assign a lot of global constants and don't need many local variables then it could be used to change the balance. It applies to the program that sets it: RUN, NEW and LOAD return to the default split, while CHAIN keeps it for the chained program. It can not be called if any variables have already been defined. |

| | | |
|---|---|---|
| OPTION LOGGING ON<br>or<br>OPTION LOGGING OFF | | When set to ON every error message is appended to the file log.txt on the A: drive (the file is created if necessary). This can help when debugging an unattended system.<br>The default is OFF. Like OPTION EXPLICIT the setting is cleared when a program starts running, so a program that wants its errors logged should include OPTION LOGGING ON near its start. The setting is not saved. |
| OPTION MILLISECONDS ON\|OFF | | This enables or disables a millisecond output in the TIME$ function.<br>Ie, HH:MM:SS.mmm<br>The milliseconds counter is set to zero whenever the time is updated using the TIME command, WEB NTP command or RTC GETTIME command. Default is OFF. |
| OPTION MODBUFF ENABLE/DISABLE [sizeinK] | ✓ | Creates or removes an area of flash memory used for loading and playing .MOD files. If enabled then a mod buffer is created with a size of 128Kbytes. This can be overridden with "sizeinK".<br>Note that this option reserves part of the Flash Filesystem (ie, it shrinks the Flash Filesystem). The default is disabled.<br>Note: This option is not required on an RP2350 with PSRAM enabled. In this case the MOD file will be loaded to space in the PSRAM.<br>Note: On an RP2350 with 16MB of flash the mod buffer is always present and fixed at 1Mbyte (it does not shrink the Flash Filesystem), and this option cannot be used. |
| OPTION MOUSE CLKpin, DATApin | ✓ | <u>NON-USB VERSIONS ONLY</u><br>Set the pins to be used to connect a PS2 mouse. Using this command the mouse is automatically configured on boot and you can set up interrupts and read values with no additional commands. This is different from the MOUSE OPEN which only connects to a mouse while the program is running. The PS2 mouse MUST be is disabled. |
| OPTION MOUSE DISABLE | ✓ | Disables the automatic connection to a PS2 mouse and frees up the pins for normal usage. |
| OPTION MOUSE SENSITIVITY f! | ✓ | <u>USB VERSIONS ONLY</u><br>By default, the firmware operates a USB mouse in boot mode. This means it returns 8-bit x and y positions and the state of the standard 3-buttons. By setting OPTION MOUSE SENSITIVITY the firmware will command the mouse to operate with its full capabilities. This means that it will try and interpret the full USB report received from the mouse including the wheel position, all buttons and the full resolution x/y position information. To do this the firmware reads the mouse's HID report descriptor when it is plugged in and decodes whatever report layout the mouse declares - any number of buttons, any width of x and y movement, padding in any position and reports which use report IDs. If the mouse does not declare usable x and y movement, or will not leave boot mode, the firmware automatically falls back to the standard boot report.<br>Setting 'f$' will enable the full mouse report and scale the x/y positions by 'f!'<br>The firmware cannot support all types of mouse, so if this causes problems with a specific mouse reset back to boot mode by using OPTION MOUSE SENSITIVITY 0. |

| | | |
|---|---|---|
| | | Enabling this option with a value of 1.0 using a standard Microsoft Basic Optical mouse has been fully tested and will enable the wheel to be used in a program |
| OPTION NOCHECK ON/OFF | | This command, when set ON, disables the standard checking for interrupts and ctrl-C at the end of every command. Setting it to ON allows time critical processing to take place without risk of interruption. However, the command should be used carefully or the program may only be stopped with a H/W reset. |
| OPTION PICO ON/OFF | ✓ | NOT WIFI VERSIONS<br>When set to OFF pins GP23, GP24 and GP29 are not set up for normal Pico use and are immediately available to use. Default ON for RP2350A and RP2040, OFF for RP2350B |
| OPTION PICO PLUS 2 ON<br>or<br>OPTION PICO PLUS 2 OFF | | RP2350B VERSIONS ONLY<br>Configures the firmware for the Pimoroni Pico Plus 2 board with a single command. ON sets the heartbeat to the board's LED pin and enables the PSRAM fitted to the board (chip select GP47), then saves the options and restarts. OFF disables the heartbeat and the PSRAM.<br>This command will generate an error on RP2350A based boards (eg, the Raspberry Pi Pico 2). |
| OPTION PIN nbr | | Set 'nbr' as the PIN (Personal Identification Number) for access to the console prompt. 'nbr' can be any non zero number of up to eight digits.<br>Whenever a running program tries to exit to the command prompt for whatever reason MMBasic will request this number before the prompt is presented. This is a security feature as without access to the command prompt an intruder cannot list or change the program in memory or modify the operation of MMBasic in any way. To disable this feature enter zero for the PIN number (i.e. OPTION PIN 0).<br>A permanent lock can be applied by using 99999999 for the PIN number. If a permanent lock is applied or the PIN number is lost the only way to recover is to reload the PicoMite firmware.<br>This command must be run at the command prompt (not in a program). |
| OPTION PLATFORM name$ | ✓ | Allows a user to identify a particular hardware configuration that can then be used in programs to control the program's operation.<br>'name$' can be up to 31 characters long. This is a permanent option.<br>MM.INFO$(PLATFORM) returns this string.<br>For example, this can be used on a particular hardware configuration:<br>`OPTION PLATFORM "GameMite"`<br>Then programs that might run on this or other platforms can use:<br>`IF MM.INFO$(PLATFORM) = "GameMite" THEN …` |
| OPTION POWER PFM \| PWM | ✓ | Changes operation of the 3.3V supply switch mode power supply.<br>By default this runs in PFM mode. PWM gives better noise performance but is less power-efficient. Note that under heavy load the system will run in PWM mode regardless of this setting.<br>Not available on the WiFi and Bluetooth versions while GP23 is the wireless chip's power-on pin, as it is on the Pico W and Pico 2 W (on the WebMiteRP2040 it always is). |
| OPTION PROFILING ON | ✓ | Enables or disables the execution profiler. When enabled, counters record how often each statement executes and a [PERF] report listing the |

| | | |
|---|---|---|
| or<br>OPTION PROFILING OFF<br>or<br>OPTION PROFILING ON, SAMPLE [, entries] | | most executed commands and the busiest subroutines/functions is printed when the program ends. Profiling has no overhead when disabled.<br>If the report shows that most of the time goes in a few subroutines or functions, the mmb2csub tool can convert them into CSUBs, which run many times faster (typically 10 to 35 times for loops and arithmetic). See the separate manual mmb2csub.pdf.<br>The option must be in the program, as RUN turns profiling off, and the report is printed by END: not when a program runs off its last line or is stopped with CTRL-C. The report also gives local_peak, the most local variables alive at once, of the number available.<br>OPTION PROFILING ON, SAMPLE also samples where the processor is about 10,000 times a second, and the report adds the program lines that took the most samples, as [PCSLINE] line samples (a line of -1 is in the library). 'entries' sets the size of the sample tables, from 256 to 65536 (default 4096).<br>With OPTION COMPILE ON the statement count and the list of commands cover only the statements that ran on the interpreter, so the list shows exactly what was not compiled, busiest first.<br>See the separate manual option-profiling-cache.pdf for the details. |
| OPTION PSRAM PIN n<br>or<br>OPTION PSRAM PIN DISABLE | ✓ | RP2350 VERSIONS ONLY Enable/disable PSRAM support.<br>'n' is the PSRAM chip select (CS) pin and can be GP0, GP8, GP19, or GP47.<br>Typically GP47 is used for Pimoroni boards. Default is disabled.<br>After power up the contents of the PSRAM are indeterminate, use RAM ERASE to clear it before use. |
| OPTION RADIO BOTH \| WEB \| BLUETOOTH \| NONE | ✓ | PICOMITEPC3 ONLY Selects which halves of the wireless chip the next boot brings up.<br>BOTH (the default) starts the WiFi and the Bluetooth host. WEB leaves the Bluetooth host out: no Bluetooth stack, no firmware download and no polling, so the BLUETOOTH commands and OPTION AUDIO BLUETOOTH refuse. BLUETOOTH leaves the WiFi down: no network join and no TCP, TFTP or UDP servers, and the WEB commands refuse. NONE leaves the chip unpowered and unused, its four pins are free for general use and the heartbeat LED is on GP25.<br>The stored WiFi network and the Bluetooth pairings are kept, so a setting can be changed back at any time. The Bluetooth host cannot be switched off while the audio is on Bluetooth (OPTION AUDIO). The setting is saved and the firmware restarts.<br>eg. OPTION RADIO WEB |
| OPTION RESET | ✓ | Reset all saved options to their default values.<br>This command must be run at the command prompt (not in a program). |
| OPTION RESET cfg<br>or<br>OPTION RESET LIST | ✓ | Reset all options to default values for the configuration 'cfg'.<br>OPTION RESET LIST will list all available configurations.<br>This command must be run at the command prompt (not in a program). |
| OPTION RESOLUTION nn [,cpuspeedinKhz] | | VGA AND HDMI VERSIONS ONLY<br>For firmware with HDMI video set the video resolution to 'nn'.<br>Where 'nn' is: |

| | | |
|---|---|---|
| | | 640x480 or 640<br>720x400 or 720<br>800x600 or 800 (RP2350 only)<br>848x480 or 848 (RP2350 only)<br>1280x720 or 1280 (HDMI only)<br>1024x768 or 1024 (HDMI only)<br>1024x600 (HDMI only)<br>800x480 (HDMI only)<br>On the PicoMiteHDMIWEB version 1280x720 and 1024x768 are not available and the abbreviated form 1024 selects 1024x600, so write the resolution out in full if a program has to run on more than one HDMI version. For 640x480 the display frequency can be set to 60Hz (252Mhz or 378MHz) or 75Hz (315MHz) by appending 'cpuspeedinKHz' to the command (ie, 252000, 378000 or 315000).<br>Each VGA and HDMI resolution can operate in a number of modes which are set using the MODE command.<br>Note that the 800x600, 848x480 and 800x480 resolutions reduce both the maximum program size and the variable space available to the Basic programs<br>See also the RESOLUTION command which changes the resolution immediately without saving it or restarting the firmware.<br>On the PicoMiteHDMIWEB version everything is 8-bit colour and the choices are 640x480 (or 640) [,cpuspeedinKHz], 720x400 (or 720), 800x600 (or 800), 848x480 (or 848), 800x480 and 1024x600 (or 1024). 640x480 takes the optional 'cpuspeedinKHz' (252000, 315000 or 378000); each of the other resolutions runs at a fixed clock speed. The 800x600, 848x480 and 800x480 resolutions support only MODE 1 and MODE 2. The default is 640x480 at 315000 (75Hz). |
| OPTION RTC AUTO ENABLE \| DISABLE | ✓ | Enable auto-load time$ & date$ from RTC on boot & every hour. If enabled and the RTC does not respond then any running program will abort with an error. At the command prompt an information message will be output.<br>This command must be run at the command prompt (not in a program). |
| OPTION SCREEN KEYBOARD nn [, M] | | <u>RP2350 VGA USB AND HDMI USB VERSIONS ONLY</u><br>Enable an on-screen keyboard operated using a USB touch screen. 'nn' is the percentage of the screen's vertical resolution (up to 50) that will be reserved for the keyboard at the bottom of the screen. Set 'nn' to zero to disable the feature. The console and the full screen editor automatically adjust to use the remaining screen area.<br>The keyboard is displayed automatically at the command prompt and in the full screen editor. While a program is running it is removed; the program can display it using the KEYBOARD ON command.<br>Append ', M' if the touch screen supports multi-touch. The modifier keys (Sh, Ctl, Sym and Fn) will then act as on a physical keyboard (hold the modifier with one finger while typing with another). Without it a tap on a modifier key latches it on for the next key press.<br>This option is saved permanently. |
| OPTION SDCARD CSpin [,CLKpin, MOSIpin, MISOpin] | ✓ | Specify or disable the I/O pins to use for the SD Card interface. |

| | | |
|---|---|---|
| or<br>OPTION SDCARD DISABLE | | If the optional pins are not specified the SD Card will use the pins specified by OPTION SYSTEM SPI.<br>Note: The pins specified by OPTION SYSTEM SPI must be a valid set of hardware SPI pins (SPI or SPI2), however, the pins specified by OPTION SDCARD can be any pins. The pins specified by OPTION SYSTEM SPI and OPTION SDCARD cannot be the same.<br>This command must be run at the command prompt (not in a program). |
| OPTION SDCARD COMBINED CS | ✓ | NOT VGA OR HDMI VERSIONS This specifies that the touch chip select pin is also used for the SDcard. In this case external circuitry is needed to implement the SD chip select. See "SD Cards" in the chapter "Program and Data Storage". |
| OPTION SERIAL CONSOLE uartapin, uartbpin [,B]<br><br>OPTION SERIAL CONSOLE DISABLE | ✓ | Specify that the console be accessed via a hardware serial port (instead of virtual serial over USB).<br>'uartapin' and 'uartbpin' can be any valid pair of rx and tx pins for either COM1 or COM2. The order that they are specified is not important. The speed defaults to 115200 baud but can be changed with OPTION BAUDRATE. Adding the "B" parameter means output will go to "B"oth the serial port and the USB.<br>Revert to the normal the USB console.<br>These commands must be run at the command prompt (not in a program).<br>On the PicoMiteRP2350BT version the serial console is enabled by default on GP8 (Tx) and GP9 (Rx) as a backup to the Bluetooth console. On this version "B" means both Bluetooth and the serial port; without it the console is on the serial port only and the Bluetooth console is not used. OPTION SERIAL CONSOLE DISABLE leaves just the Bluetooth console. |
| OPTION SYMBOLS OFF | | Placing this command at the start of a program will cause it to run without the symbol tables that MMBasic normally builds when a program is run. These tables speed up access to variables, subroutines and functions but they use some memory (typically 3KB to 5KB) and a large program written for an earlier version of MMBasic may need that memory. If such a program runs out of memory the error message will suggest this option.<br>It must be on a line by itself before any other command except for comment lines and a LIBRARY LOAD command (which can come before or after it). It cannot be used at the command prompt. |
| OPTION SYSTEM I2C sdapin, sclpin [,SLOW/FAST] | ✓ | Specify the $I^2C$ port and pins for use by system devices (LCD panel, and RTC).<br>The PicoMite firmware uses a specific $I^2C$ port for system devices, leaving the other for the programmer. This command specifies which pins are to be used, and hence which of the $I^2C$ ports is to be used.<br>The pins allocated to the SYSTEM I2C will not be available for other MMBasic SETPIN settings but can be used for additional $I^2C$ devices using the standard I2C command. Note: I2C(2) OPEN and I2C(2) CLOSE are not available in this case.<br>By default the $I^2C$ port is opened at a speed of 400KHz and with a 100mSec timeout. The $I^2C$ frequency can be set using the optional third parameter which can take the values FAST = 400KHz or SLOW = 100KHz.<br>This command must be run at the command prompt (not in a program). |

| | | |
|---|---|---|
| OPTION SYSTEM SPI CLKpin, MOSIpin, MISOpin<br>or<br>OPTION SYSTEM SPI DISABLE | ✓ | NOT VGA OR HDMI VERSIONS Specify or disable the SPI port and pins for use by system devices (SD Card, LCD panel, etc).<br>The PicoMite firmware uses a specific hardware SPI port for system devices, leaving the other for the user. This command specifies which pins are to be used, and hence which of the SPI ports is to be used. The pins allocated to the SYSTEM SPI will not be available to other MMBasic commands.<br>This command must be run at the command prompt (not in a program). |
| OPTION TAB 2 \| 3 \| 4 \| 8 | ✓ | Set the spacing for the tab key. Default is 2. |
| OPTION TCP SERVER PORT n | ✓ | WIFI VERSIONS ONLY<br>Launches a TCP server on port 'n' during every restart of the WebMite. Typically HTTP servers use port 80.<br>USE "OPTION TCP SERVER PORT 0" to disable<br>When the server is running it can respond to up to MM.INFO(MAX CONNECTIONS) |
| OPTION TELNET CONSOLE OFF\|ONLY\|ON | ✓ | WIFI VERSIONS ONLY<br>Configures the handling the console over Telnet.<br>ON = Console output sent to USB and Telnet<br>ONLY= Console output sent to Telnet only<br>OFF = Console output sent to USB only |
| OPTION TFTP OFF\|ON | ✓ | WIFI VERSIONS ONLY<br>Enables or disables the TFTP server. Default is on. |
| OPTION TOUCH T_CS pin, T_IRQ pin [, Beep]<br>or<br>OPTION TOUCH DISABLE | ✓ | NOT VGA OR HDMI VERSIONS<br>Configures MMBasic for the touch sensitive feature of an attached LCD panel.<br>'T_CS pin' and 'T_IRQ pin' are the I/O pins to be used for chip select and touch interrupt respectively (any free pins can be used). The remaining pins are connected to those specified using the OPTION SYSTEM SPI command.<br>'Beep' is an optional pin which can be connected to a small buzzer/beeper to generate a "click" or beep sound when an Advanced Graphics control is touched (ie, radio button, switch, etc). This is described in *Advanced Graphics Functions.pdf*.<br>This command must be run at the command prompt (not in a program). |
| OPTION TOUCH FT6336 IRQpin, RESETpin [,BEEPpin] [,sensitivity]<br>or<br>OPTION TOUCH GT911 IRQpin, RESETpin [,BEEPpin] [,sensitivity] | ✓ | NOT VGA OR HDMI VERSIONS<br>Enables touch support for the FT6336 or GT911 capacitive touch controllers. 'sensitivity' is a number between 0 and 255 - defaults to 50, lower is more sensitive (FT6336 only).<br>SDA and SCK should be connected to valid I2C pins and set up with OPTION SYSTEM I2C. The FT6336 supports up to two simultaneous touch points and the GT911 up to five. See also the TOUCH function which includes multi-touch and gesture detection. |

| | | |
|---|---|---|
| OPTION VCC voltage | | Specifies the voltage (Vcc) supplied to the Raspberry Pi Pico.<br>When using the ADC pins to measure voltage the PicoMite firmware uses the voltage on the pin marked VREF as its reference. This voltage can be accurately measured using a DMM and set using this command for more accurate measurement.<br>The parameter is not saved and should be initialised in the program. The default if not set is 3.3. |
| OPTION UDP SERVER PORT n | ✓ | WIFI VERSIONS ONLY<br>Sets up a listening socket on the port specified. Any UDP datagrams received on that port will be processed and the contents saved in MM.MESSAGE$. The IP address of the sender will be stored in MM.ADDRESS$. Note: If the UDP datagram is longer than 255 characters then any extra is discarded.<br>USE "OPTION UDP SERVER PORT 0" to disable |
| OPTION VGA PINS HSYNCpin, BLUEpin | ✓ | VGA VERSIONS ONLY<br>Changes the pins used for VGA display output allowing more flexibility in PCB design or wiring. “HSYNCpin” defines the start of a pair of contiguous GP numbered pins that are connected to HSYNC and VSYNC<br>“BLUEpin” defines the start of four contiguous GP numbered pins that are connected to BLUE, GREEN_LSB, GREEN_MSB, and RED. |
| OPTION WEB MESSAGES ON/OFF | | WIFI VERSIONS ONLY<br>Disable informational web messages when set to OFF. Default is ON |
| OPTION WIFI ssid$, passwd$ [,name$] [, country$]<br>Or<br>OPTION WIFI ssid$, passwd$ [, name$] [, ipaddress$, mask$, gateway$] [, country$] | ✓ | WIFI VERSIONS ONLY<br>Configures the firmware to automatically connect to a WiFi network on restart.<br>'ssid$' is the name of the network and 'password$' is the access password. Both are strings and if string constants are used they should be quoted.<br>Optionally a name for the device can be specified 'name$', otherwise a name is created from the unique device ID.<br>Optionally, a static IP address, IP mask, and gateway address can be specified as 'ipaddress$', 'mask$', 'gateway$'.<br>Optionally a 2-letter ISO country code 'country$' can be specified to set the WiFi regulatory domain. The default is "XX" (worldwide), which restricts the device to the most conservative subset of channels and transmit power. Setting the local country code is recommended and is often required to associate with EU routers (for example AVM Fritzbox) which may use channels 12 or 13. If 'country$' is omitted the previously stored value is preserved; "" or "WW" both select worldwide. The country code is a regulatory setting and must match the country in which the device is operated.<br>Supported country codes:<br>AU AT BE BR CA CH CL CN CO CZ DE DK EE ES FI FR GB GR HK HU IL IN IS IT JP KE KR LI LT LU LV MT MX MY NG NL NO NZ PE PH PL PT SE SG SI SK TH TR TW US ZA XX or |

| | | |
|---|---|---|
| | | WW (worldwide, default)<br><br>eg. OPTION WIFI "mysid","mypassword","myPico","DE"<br><br>eg. OPTION WIFI "mysid","mypassword","myPico",<br>"192.168.1.111","255.255.255.0","192.168.1.1","DE" |

# Commands

Square brackets indicate that the parameter or characters are optional.

Where a keyword contains COLOUR or GREY the American spellings COLOR and GRAY are accepted as well - for example COLOR, MAP GRAYSCALE, OPTION DEFAULT COLORS and MM.INFO(FCOLOR). Both spellings mean the same thing throughout and only the British spelling is listed below.

| | |
|---|---|
| ‘ (single quotation mark) | Starts a comment and any text following it will be ignored. Comments can be placed anywhere on a line. |
| *file | The star/asterisk command is a shortcut for RUN that may only be used at the MMBasic prompt. eg,<br>* RUN<br>*foo RUN "foo"<br>*"foo bar" RUN "foo bar"<br>*foo –wombat RUN "foo", "--wombat"<br>*foo "wom" RUN "foo", CHR$(34) + "wom" + CHR$(34)<br>*foo --wom="bat" RUN "foo","--wom=" + CHR$(34) + "bat" + CHR$(34)<br>String expressions are not supported/evaluated by this command; any arguments provided are passed as a literal string to the RUN command. |
| ? (question mark) | Shortcut for the PRINT command. |
| /*<br>*/ | Start and end of multiline comments. /* and */ must be the first non-space characters at the start of a line and have a space or end-of-line after them (i.e. they are MMBasic commands). Multi-line comments cannot be used inside subroutines and functions. Any characters after */ on a line are also treated as a comment. |
| A: , B: or C: | Shortcut for DRIVE “A:” ,DRIVE “B:” or DRIVE “C:” at the command prompt |
| ADC | The ADC commands provide an alternate method of recording analog inputs and are intended for high speed recording of many readings into an array. |
| ADC OPEN freq, n_channels [,interrupt] | This allocates up to 4 ADC channels from the ADC0-ADC3 group for use and sets them to be converted at the specified frequency.<br>The range of pins are GP26, GP27, GP28, and GP29 for the RP2040 and RP2350A or GP40, GP41, GP42, GP43 on the RP2350B. If the number of channels is one then it will always be GP26 (ADC0) used, if two then GP26 and GP27 (ADC0 and ADC1) are used, etc. Sampling of multiple channels is sequential (there is only one ADC). The pins are locked to the function when ADC OPEN is active<br>The maximum total frequency is CPUspeed/96 (eg, 520KHz if all four channels are to be sampled with the CPU set at 200MHz). Note that a aggregate sampling frequency over 500Khz is overclocking the ADC.<br>The optional interrupt parameter specifies an interrupt to call when the conversion completes. If not specified then conversion will be blocking |
| ADC FREQUENCY freq | This changes the sampling frequency of the ADC conversion without having to close and re-open |
| ADC CLOSE | Releases the pins to normal usage |

| | |
|---|---|
| ADC START array1!() [,array2!()] [,array3!()] [,array4!()] [,C1min] [,C1max] [,C2min] [,C2max] [,C3min] [,C3max] [,C4min] [,C4max] | This starts conversion into the specified arrays. The arrays must be floating point and the same size. The size of the arrays defines the number of conversions. Start can be called repeatedly once the ADC is OPEN<br>'Cxmin' and 'Cxmax' will scale the readings. For example, C1min=200 and C1max=100 will create values ranging from 200 to 100 for equivalent voltages of 0 - 3.3. If the scaling is not used the results are returned as a voltage between 0 and OPTION VCC (defaults to 3.3V).<br>If ADC OPEN specified an interrupt the arrays must be global (or STATIC), not LOCAL, as they are filled after the command has returned. |
| ADC RUN array1%(),array2%) | Runs the ADC continuously in double buffered mode. The ADC first fills array1% and then array2% and then back to array1% etc. If more than one ADC channel is specified in the ADC OPEN command the data are interleaved. The data is returned as packed 8-bit values (Use MEMORY UNPACK to convert to a normal array). MM.INFO(ADC) will return the number of the buffer currently available for reading (1 or 2).<br>The arrays must be global (or STATIC), not LOCAL, as they are filled in the background. |
| ARRAY ADD in(), value ,out() | This adds (or appends for strings) the value 'value' to every element of the matrix in() and puts the answer in out(). Works for arrays of any dimensionality of strings and both integer and float (can convert between integer and float). Setting num to 0 or "" is optimised and is a fast way of copying an entire array. in() and out() can be the same array. |
| ARRAY INSERT targetarray(), [d1] [,d2] [,d3] [,d4] [,d5] , sourcearray() | This is the opposite of ARRAY SLICE, has a very similar syntax, and allows you, for example, to substitute a single vector into an array of vectors with a single instruction or a one dimensional array of strings into a two dimensional array of strings. The arrays can be numerical or strings and 'sourcearray' and 'destinationarray' must be the same (NB: can convert between integers and floats for numerical arrays).<br><br>eg,<br>`OPTION BASE 1`<br>`DIM targetarray(3,4,5)`<br>`DIM sourcearray(4)=(1,2,3,4)`<br>`ARRAY INSERT targetarray(), 2, , 3, sourcearray()`<br>Will set elements 2,1,3 = 1 and 2,2,3 = 2 and 2,3,3 = 3 and 2,4,3 = 4 |
| ARRAY SET value, array() | Sets all elements in array() to the value 'value'. Value can be a number or a string and 'array' must be the same (NB: can convert between integers and floats). Note this is the fastest way of clearing an array by setting it to zero or an empty string. |
| ARRAY SLICE sourcearray(), [d1] [,d2] [,d3] [,d4] [,d5] destinationarray() | This command copies a specified set of values from a multi-dimensional array into a single dimensional array. It is much faster than using a FOR loop. The slice is specified by giving a value for all but one of the source array indices and there should be as many indices in the command, including the blank one, as there are dimensions in the source array. The arrays can be numerical or strings and 'sourcearray' and 'destinationarray' must be the same (NB: can convert between integers and floats for numerical arrays).<br>eg,<br>`OPTION BASE 1`<br>`DIM a(3,4,5)`<br>`DIM b(4)`<br>`ARRAY SLICE a(), 2, , 3, b()`<br>Will copy the elements 2,1,3 and 2,2,3 and 2,3,3 and 2,4,3 into array b() |

| | |
|---|---|
| ARC x, y, r1, [r2], a1, a2 [, c] | Draws an arc of a circle with a given colour and width between two radials (defined in degrees). Parameters for the ARC command are:<br>x: X coordinate of centre of arc<br>y: Y coordinate of centre of arc<br>r1: inner radius of arc<br>r2: outer radius of arc - can be omitted if 1 pixel wide<br>a1: start angle of arc in degrees<br>a2: end angle of arc in degrees<br>c: Colour of arc (if omitted it will default to the foreground colour)<br>Zero degrees is at the 12 o'clock position. |
| ASTRO | RP2350 VERSIONS ONLY Calculates the position of a celestial object using the manual time and location set by the LOCATION command.<br>This is one of a set of commands that performs high-precision astronomical calculations suitable for telescope pointing or navigation and is described in detail in the file *GPS_Astro_Reference.pdf* included in the firmware download zip file. |
| AUTOSAVE<br>or<br>AUTOSAVE CRUNCH<br>or<br>AUTOSAVE APPEND<br>or<br>AUTOSAVE N | Enter automatic program entry mode. This command will take lines of text from the console serial input and save them to program memory.<br>This is one way of transferring a BASIC program into the Raspberry Pi Pico. The program to be transferred can be pasted into a terminal emulator and this command will capture the text stream and store it into program memory. It can also be used for entering a small program directly at the console input.<br>This mode is terminated by entering Control-Z or F1 which will then cause the received data to be transferred into program memory overwriting the previous program. Use F2 to exit and immediately run the program.<br>The CRUNCH option instructs MMBasic to remove all comments, blank lines and unnecessary spaces from the program before saving. This can be used on large programs to allow them to fit into limited memory. CRUNCH can be abbreviated to the single letter C.<br>If the incoming program is too large for program memory the transfer stops with the error 'Not enough memory'. Any text still being sent is then echoed at the command prompt, which can scroll that message out of view, so the reliable sign that a transfer ran out of room is that it finished without the usual 'Saved nnn bytes' report. The CRUNCH option will often make the difference.<br>The APPEND option will leave the existing program intact and append the new data from the serial input to the end of it.<br>The N option runs autosave as normal but does not echo the data back to the console. It waits for the first character to be sent and then uses a timer that checks for greater than 100mSec between characters. If it sees this delay it switches back to normal input mode and outputs the message<br>"Enter ctrl-Z, F1, or F2 to exit"<br>You can then type additional characters to be saved or use one of the normal exits.<br>At any time, this command can be aborted by Control-C which will leave program memory untouched. |

| | |
|---|---|
| BACKLIGHT n [,DEFAULT]<br>BACKLIGHT n [,FreqInHz] | NOT VGA OR HDMI VERSIONS<br>Sets the display backlight, valid values are 0 to 100. If DEFAULT is specified then the firmware will automatically set the backlight to that level on power-up. This is particularly useful for battery operation where reducing the backlight level can significantly increase battery life.<br>Some circuits are too slow to use the default backlight PWM frequency which is chosen to avoid interference with audio. In which case a user frequency can be specified. This is a temporary option and will need setting on reboot. |
| BEZIER x%(),y%() [,n] [,colour] | Draws a Bezier curve with an unlimited number of control points. 'x%()' and 'y%()' are single dimension integer arrays holding the coordinates of the control points. The Bezier curve always starts at the first control point. The arrays must have the same dimensionality.<br>The optional parameter 'n' defines how many control points to use for the plot. If omitted the size of 'x%()' and 'y%()' determine the number. N must be >=2 and <= the array size.<br>The optional parameter 'colour' defines the colour to plot the curve (defaults to white). |
| BIT(var%, bitno) = value | Sets a specific bit (0-63) in an integer variable. 'value' can be 0 or 1.<br>See also the BIT function |
| BITBANG | Replaced by the command DEVICE. For compatibility BITBANG can still be used in programs and will be automatically converted to DEVICE |
| BITSTREAM<br><br>BITSTREAM pin1, count1, array1() [, mode1] [,pin2, count2, array2()] [,mode2] [,logic] | The `BITSTREAM` command generates extremely accurate bit sequences on one or two GPIO pins. Each timing value in the array specifies the duration (in microseconds) before the pin state toggles. This enables precise generation of protocols like IR remote control signals, PS/2 keyboard/mouse communication, and other timing-critical waveforms.<br><br>`pin1`, `pin2` are the GPIO pin number(s) for the output. Can use GP notation (e.g., GP15) or physical pin numbers.<br>count1`, `count2` specify the number of timing values to use from the arrays (1 to 10000).<br>array1()`, `array2()` are numeric array containing timing values in microseconds. Each value specifies the time before the next pin toggle.<br>`mode1`, `mode2` specify the output mode (optional, defaults to 0)<br>0 = Push-pull (driven high and low),1 = Open-collector (driven low, high-impedance for high)<br>`logic` is a special function for same-pin mode (optional, defaults to 0, only valid when pin1 = pin2) 0 = XOR (toggle on any event, original behavior), 1 = AND (output HIGH only when both logical states are HIGH), 2 = OR (output HIGH when either logical state is HIGH)<br><br>See the manual BITSTREAM_User_Manual for more information and examples of how to use the bitstream command to create a PS2 output and drive NEC IR transmit |
| BLIT | BLIT is a simple memory operation copying to and from a display or memory to a display or memory.<br>Notes:<br>• 32 buffers are available ranging from #1 to #32.<br>• When specifying the buffer number the # symbol is optional.<br>• All other arguments are in pixels. |

| | |
|---|---|
| BLIT READ [#]b, x, y, w, h | BLIT READ will copy a portion of the display to the memory buffer '#b'. The source coordinate is 'x' and 'y' and the width of the display area to copy is 'w' and the height is 'h'. When this command is used the memory buffer is automatically created and sufficient memory allocated. This buffer can be freed and the memory recovered with the BLIT CLOSE command. |
| BLIT WRITE [#]b, x, y [,mode] | BLIT WRITE will copy the memory buffer '#b' to the display. The destination coordinate is 'x' and 'y'.<br>The optional 'mode' parameter defaults to 0 and specifies how the stored image data is changed as it is written out. It is the bitwise AND of the following values:<br>&B001 = mirrored left to right<br>&B010 = mirrored top to bottom<br>&B100 = don't copy transparent pixels |
| BLIT LOAD[BMP] [#]b, fname$ [,x] [,y] [,w] [,h] | BLIT LOAD will load a blit buffer from a 24-bit bmp image file. x,y define the start position in the image to start loading and w,h specify the width and height of the area to be loaded. This command will work on most display panels (not just panels using the ILI9341 controller).<br>eg,<br>BLIT LOAD #1,"image1", 50,50,100,100<br>will load an area of 100 pixels square with the top left had corner at 50,50 from the image image1.bmp |
| BLIT CLOSE [#]b | BLIT CLOSE will close the memory buffer '#b' to allow it to be used for another BLIT READ operation and recover the memory used. |
| | <u>NOT VGA OR HDMI VERSIONS</u> |
| BLIT MERGE colour, x, y, w, h | Copies an area of the framebuffer defined by the ‘x’ and ‘y’ pixel coordinates of the top left and with a width of ‘w’ and height ‘h’ to the LCD display. As part of the copy it will overlay the LCD display with pixels from the layer buffer that aren’t set to the ‘colour’ specified. The colour is specified as a number between 0 and 15 representing:<br>Black, Blue, Myrtle, Cobalt, Midgreen, Cerulean, green, cyan, red, magenta, rust, fuchsia, brown, lilac, yellow and white<br>Requires both a framebuffer and a layer buffer to have been created to operate. Will automatically wait for frame blanking before starting the copy on ILI9341, ST7789_320 and ILI9488 displays |
| BLIT FRAMEBUFFER from, to, xin, yin, xout, yout, width, height [,colour] | Copies an area of a specific ‘from’ framebuffer N, F, L, T or 2 to another different ‘to’ framebuffer N, F, L, T or 2 (T and 2 are only available on RP2350 VGA and HDMI builds). ‘xin’ and ‘yin’ define the top left of the area of ‘width’ and ‘height’ on the source framebuffer to be copied. ‘xout’ and ‘yout’ define the top left of the area on the target framebuffer to receive the copy. The optional parameter colour defines a pixel colour on the source which will not be copied. If omitted all pixels are copied. The colour is specified as a number between 0 and 15 representing:<br>Black, Blue, Myrtle, Cobalt, Midgreen, Cerulean, green, cyan, red, magenta, rust, fuchsia, brown, lilac, yellow and white<br>Requires both a framebuffer and a layer buffer to have been created to operate. Will automatically wait for frame blanking before starting the copy on ILI9341, ST7789_320 and ILI9488 displays |
| BLIT MEMORY address, x, y [,col] | Copies an area of memory treated as a packed array of colour nibbles to the current graphical output as specified by FRAMEBUFFER WRITE. The colour is specified as a number between 0 and 15 representing:<br>Black, Blue, Myrtle, Cobalt, Midgreen, Cerulean, green, cyan, red, magenta, rust, fuchsia, brown, lilac, yellow and white |

| | |
|---|---|
| | The first word of the area of memory starting at 'address%' must contain the width and height of the area to be copied as 16-bit integers with the width as the bottom 16 bits. The address must be aligned to a word boundary (divisible by 4).<br>If the optional parameter 'col' is specified then that specific colour is not copied.<br>If the top bit of either the width or height is set to 1 then the colour data is treated as compressed (the remaining 15 bits are used as the width and/or height). The compression algorithm is simple, each byte contains a count in the bottom nibble (1-15) and a colour in the top nibble (0-15). In the event that more than 15 pixels are the same colour additional bytes are used for that colour. |
| BLIT MEMORY332 address, x, y [,col] | <u>ONLY ON RGB332 (256 COLOUR) DISPLAYS</u><br>Acts the same as BLIT MEMORY but for RGB332 (8-bit, 256 colour) displays, for example HDMI MODE 5 and the buffered RGB332 LCD panels. The image data is one byte per pixel in RGB332 format (3 bits red, 3 bits green and 2 bits blue).<br>The data is run-length compressed: each run is a pair of bytes, a count from 1 to 255 followed by the RGB332 colour value, and a run may continue from the end of one row onto the next. Runs longer than 255 pixels of the same colour use additional pairs.<br>The first word of the area of memory starting at 'address' contains the width and height of the image as 16-bit integers with the width as the bottom 16 bits (the top bit of each is ignored). The data is written to the current graphical output as specified by FRAMEBUFFER WRITE.<br>If the optional parameter 'col' (0 to 255) is specified then that RGB332 colour is treated as transparent and is not copied. |
| BLIT COMPRESSED address%, x, y [,col] | Acts the same as BLIT MEMORY but assumes the data is compressed and ignores the top bit in the width and height |
| BLIT FLASH from, to, xin, yin, xout, yout, width, height [,colour] | Copies an area of a specific 'from' image slot to a 'to' framebuffer N, F, L, T or 2 (T and 2 are only available on RP2350 VGA and HDMI builds). Slots 1 to 3 are the flash slots; on an RP2350 with PSRAM, slots 4 to 8 are the RAM slots 1 to 5 (see FLASH LOAD IMAGE). 'xin' and 'yin' define the top left of the area of 'width' and 'height' on the image to be copied.<br>'xout' and 'yout' define the top left of the area on the target framebuffer to receive the copy.<br>The optional parameter colour defines a pixel colour in the image which will not be copied. If omitted all pixels are copied. The colour is specified as a number between 0 and 15 representing:<br>Black, Blue, Myrtle, Cobalt, Midgreen, Cerulean, green, cyan, red, magenta, rust, fuchsia, brown, lilac, yellow and white |

| | |
|---|---|
| BLIT RESIZE src, dst, sx, sy, sw, sh, dx, dy, dw, dh [,transparent] | Scales a rectangular source region into a destination rectangle using fast nearest-neighbour sampling.<br>`src` Source buffer selector. Supported: `L`, `F`, `N` (VGA/HDMI builds), `2` (second framebuffer, VGA/HDMI builds), `T` (top layer, RP2350 VGA/HDMI builds).<br>`dst` \|Destination buffer selector. Supported: `L`, `F`, `N` (VGA/HDMI builds), `2` (second framebuffer, VGA/HDMI builds), `T` (top layer, RP2350 VGA/HDMI builds).<br>`sx`, `sy` Top-left source coordinate<br>`sw`, `sh` Source width and height (must be > 0<br>‘dx`, `dy` Top-left destination coordinate<br>‘dw`, `dh` Destination width and height (must be > 0)<br>`transparent` Optional RGB121 pixel value to skip while writing destination. Range `-1` to `15`, default `-1` (disabled). |
| BLIT x1, y1, x2, y2, w, h | Copy one section of the display screen to another part of the display.<br>The source coordinate is 'x1' and 'y1'. The destination coordinate is 'x2' and 'y2'. The width of the screen area to copy is 'w' and the height is 'h'.<br>All arguments are in pixels.<br>If the output is to an LCD panel it must be either the SSD19863, ILI9341_8, ILI9341, ILI9488 (if MISO connected), or ST7789_320 controllers. |
| BLUETOOTH SCAN [seconds] | <u>BLUETOOTH HOST VERSIONS ONLY</u><br>Searches for nearby Bluetooth Classic devices such as speakers, headphones, keyboards and serial devices (a Bluetooth GPS receiver, for example), and for Bluetooth LE serial devices, and lists the address, signal strength and name of each one found. Devices that describe themselves as audio devices are marked ‘Audio’, keyboards ‘Keyboard’ and Bluetooth LE serial devices ‘BLE serial’. A device is only found while it is discoverable, which for most speakers and keyboards means while they are in pairing mode.<br>‘seconds’ is how long to search for (2 to 60 seconds, the default is 10). The names are fetched after the search so the command can take a few seconds longer than this.<br>Bluetooth LE keyboards, mice and game controllers are not listed - they are paired automatically (see the chapter Bluetooth Versions). |

| | |
|---|---|
| BLUETOOTH CONNECT name$ [, pin$]<br>BLUETOOTH CONNECT addr$ [, pin$] | BLUETOOTH HOST VERSIONS ONLY<br>Pairs with and connects to a Bluetooth speaker or headphones, a Bluetooth Classic keyboard (one that BLUETOOTH SCAN marked 'Keyboard') or a Bluetooth serial device, which becomes the serial port COM7. 'name$' is the name of the device as listed by the last BLUETOOTH SCAN (upper and lower case are treated as the same). 'addr$' is its address in the form "AA:BB:CC:DD:EE:FF", which can be used without a scan. If the scan found two devices with the same name the address must be used.<br>'pin$' is the PIN for a speaker or a Bluetooth Classic serial device that asks for one when it pairs (the older way of pairing, from before Bluetooth 2.1). It is 1 to 16 characters long and is needed only when the device does not accept 0000, which is used when 'pin$' is not given. The device's instructions give its PIN (some car radios and serial modules use 1234). A PIN cannot be given for a keyboard or a Bluetooth LE device.<br>The command waits up to 30 seconds for the device to answer and then shows the format of the audio stream that has been agreed. Only one speaker can be connected at a time. For a keyboard it waits up to 60 seconds, as a passkey may have to be typed on it (see the chapter Bluetooth Versions).<br>The speaker is remembered: when OPTION AUDIO BLUETOOTH is set it is reconnected automatically after a restart and when it is switched off and on again, so this command is normally needed only once. Connecting to another speaker replaces the remembered one. A Classic keyboard is remembered in the same way and reconnects by itself.<br>A serial device is a Bluetooth Classic device with a serial port (the Serial Port Profile), for example a Bluetooth GPS receiver, or a Bluetooth LE device that BLUETOOTH SCAN marked 'BLE serial'. It is remembered in the same way and is reconnected automatically while COM7 is open. Only one serial device can be connected at a time and connecting another replaces the remembered one. See the chapter Bluetooth Versions. |
| BLUETOOTH DISCONNECT<br>BLUETOOTH DISCONNECT SERIAL | BLUETOOTH HOST VERSIONS ONLY<br>Disconnects the speaker. It is not reconnected automatically until the next BLUETOOTH CONNECT or until the PicoMite is restarted. The pairing is kept (see BLUETOOTH FORGET).<br>BLUETOOTH DISCONNECT SERIAL disconnects the serial device on COM7 instead. It is not reconnected automatically until COM7 is opened again or the next BLUETOOTH CONNECT, and its pairing is kept. |
| BLUETOOTH STATUS | BLUETOOTH HOST VERSIONS ONLY<br>Lists the state of the Bluetooth connections: whether a Bluetooth LE keyboard (or mouse or game controller) is connected, the Bluetooth Classic keyboard, the serial device on COM7 (with the number of bytes received and sent), the address of the connected speaker and whether audio is streaming to it (or the remembered speaker if it is not connected), and the format of the audio stream.<br>The lines that follow are diagnostic information which may be useful when reporting a problem. |

| | |
|---|---|
| BLUETOOTH FORGET [addr$] | BLUETOOTH HOST VERSIONS ONLY<br>Deletes stored Bluetooth pairing information. With no argument the pairings of every device are deleted - keyboards, mice, game controllers, the remembered Classic keyboard, the remembered serial device and the remembered speaker. 'addr$' deletes only those of the device with that address (as listed by BLUETOOTH STATUS or returned by MM.INFO$(BLUETOOTH device)). A connected device that is forgotten is disconnected.<br>This can be used to clear a pairing that has stopped working, after which the device can be paired again. A forgotten speaker, Classic keyboard or serial device is no longer reconnected automatically. OPTION RESET also deletes all the pairings. |
| BOX x, y, w, h [, lw] [,c] [,fill] | Draws a box on the display with the top left hand corner at 'x' and 'y' with a width of 'w' pixels and a height of 'h' pixels.<br>'lw' is the width of the sides of the box and can be zero. It defaults to 1.<br>'c' specifies the colour and defaults to the default foreground colour if not specified. 'fill' is the fill colour. It can be omitted or set to -1 in which case the box will not be filled.<br>All parameters can be expressed as arrays and the software will plot the number of boxes as determined by the dimensions of the smallest array. 'x' and 'y', must both be arrays or be single variables /constants otherwise an error will be generated. 'w', 'h' , 'lw', 'c', and fill can be either arrays or single variables/constants.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates |
| BYTE(var$, byteno)=value | Sets a specific byte in a string to an integer value. 'value' can be in the range 0-255. The byteno can be between 1 and the current length of the string variable. This is the equivalent of MID$(var$,byteno,1)=CHR$(value) but executes much faster.<br>See also the BYTE function. |
| CALL usersubname$ [,usersubparameters,..] | This is an efficient way of programmatically calling user defined subroutines (see also the CALL() function). In many cases it can allow you to get rid of complex SELECT and IF THEN ELSEIF ENDIF clauses and is processed in a much more efficient way.<br>The "usersubname$" can be any string or variable or function that resolves to the name of a normal user subroutine (not an in-built command). The "usersubparameters" are the same parameters that would be used to call the subroutine directly. A typical use could be writing any sort of emulator where one of a large number of subroutines should be called depending on some variable. It also allows a way of passing a subroutine name to another subroutine or function as a variable. |

| | |
|---|---|
| CAMERA | NOT VGA, HDMIWEB OR HDMIBTH VERSIONS<br>Command supporting the OV7670 camera module.<br>On the HDMI versions (PicoMiteHDMI, HDMIUSB and PC3) the camera is supported when the screen is in MODE 4 (320x240) and the captured image is written directly to the framebuffer.<br>For the 2-megapixel OV2640 and the 5-megapixel OV5640 cameras (RP2350 versions only) see the separate entries that follow.<br>Except on the HDMI versions (which require MODE 4) a display is not needed to use the camera. CAMERA CAPTURE JPEG writes directly to a file and CAMERA CHANGE (motion detection) does not draw to the screen, so both work with no display connected. Only the simple CAMERA CAPTURE command needs a display and it must be one with at least RGB565 colour.<br>If the camera is not responding correctly (for example it is not connected, or a signal is wired to the wrong pin) the capture commands will report an error naming the signal that stalled rather than waiting indefinitely. |
| CAMERA OPEN XLKpin, PLKpin, HSpin, VSCpin, RETpin, D0pin [, QVGA] | This initialises the camera, It outputs a 12MHz clock on XLK (PWM) and checks that it is correctly receiving signals on PLK, VS, and HS. By default the camera is set to a resolution of 160x120 (QQVGA). On RP2350-based versions the resolution can be increased to 320x240 by adding the optional QVGA parameter.<br>Enable OPTION SYSTEM I2C in the PicoMite firmware and wire SCL and SDA to the relevant pins (may be labelled SIOC and SIOD on the camera module). These connections must have a pullup to 3.3V - 2K7 recommended)<br>Other pins are wired as per the OPEN command. (NB: VS may be labelled VSYNC, HS may be labelled HREF, PLK may be labelled PCLK, RET may be labelled RESET and XLK may be XCLK on your module)<br>D0pin defines the start of a range of 8 contiguous pins (eg,GP0 - GP7). On the RP2350 the eight data pins are read in a single operation so they must not straddle the boundary between GP31 and GP32. |
| CAMERA CAPTURE [scale, [x , y]] | This captures a picture from the camera (RGB565) and displays it on an LCD screen. A display with at least RGB565 colour must be connected and enabled in order for the command to work - an SPI LCD panel (ILI9341 and ST7789_320 recommended) or an SSD1963 panel. Displays with fewer colours (for example 4-bit, RGB222, RGB332 or monochrome panels) will report an error; CAMERA CAPTURE JPEG and CAMERA CHANGE can still be used as neither needs a display.<br>Scale defaults to 1 and x,y each to 0<br>By default a 160x120 image is output on the LCD with the top left at 0,0 on the LCD. Setting scale to 2 will fill a 320x240 display with the image. Setting the x and y parameters will offset the top left of the image on the LCD.<br>If the camera was opened with the QVGA option a 320x240 image is captured. On the HDMI versions (which require MODE 4) the image is written directly to the framebuffer instead of to an SPI LCD.<br>Update rate in a continuous loop is 7FPS onto the display at 1:1 scale and 5FPS scaled to 320x240.<br>Assuming the display has MISO wired it is then possible to save the image to disk using the SAVE IMAGE command. |
| CAMERA CLOSE | Closes the camera subsystem and frees up all the pins allocated in the OPEN command. |
| CAMERA CHANGE image%(),change! [,scale [,x ,y]] | The camera firmware is also able to detect motion in the camera's field of view using the command. It does this by operating the camera in YUV mode rather than RGB. This has the advantage that the intensity information and colour |

| | |
|---|---|
| | information are separated and just one byte is needed for a 256-level greyscale image which is ideal for detecting movement.<br>image% is an array of size 160x120 bytes (DIM image%(160*120/8-1)<br>If the camera was opened with the QVGA option the array must be sized for 320x240, i.e. DIM image%(320*240/8-1). On the HDMI versions the image is captured into the MODE 4 framebuffer.<br>On calling the command it holds a packed 8-bit greyscale image.<br>The change! variable returns the percentage the image has changed from the previous time the command was called.<br>Optionally if "scale" is set then the image delta is output to the screen i.e. the difference between the previous image and this one. As in the CAPTURE command the delta image can be scaled and positioned as required. If the scale parameter is omitted then the LCD is not updated by this command. |
| CAMERA TEST tnum | Enables or disables a test signal from the camera. tnum=2 generates colourbars and tnum=0 sets back to the visual input. |
| CAMERA REGISTER reg%, data% | Sets the register "reg%" in the camera to the value "data%". When used the command will report to the console the previous value and automatically confirms that the new value has been set as requested. The colour rendition of the camera as initialised is reasonable but could probably be improved further by tuning the various camera registers. |
| Camera (OV2640) | RP2350 versions only. The OV2640 is a 2-megapixel camera, selected by giving OV2640 as the first parameter of CAMERA OPEN. It is self-clocked (needs no master clock from the PicoMite), provides a 320x240 colour preview, and can capture full-colour JPEG stills up to 1600x1200 using its built-in JPEG engine.<br>CAMERA CLOSE and CAMERA REGISTER are used exactly as for the OV7670; CAMERA TEST is not available on the OV2640. |
| CAMERA OPEN OV2640, PWDNpin, DCLKpin, HSpin, VSpin, RETpin, D0pin | Initialises the OV2640. As the module is self-clocked the first parameter is the PWDN pin (driven low to power the sensor up) rather than a clock output; DCLK is the camera's pixel-clock output.<br>Enable OPTION SYSTEM I2C and wire SCL and SDA (may be labelled SIOC and SIOD) with pullups to 3.3V (2K7 recommended). DCLK may be labelled PCLK, HS may be HREF and VS may be VSYNC on your module.<br>D0pin is the start of a range of 8 contiguous pins. The OV2640 always uses 320x240 for the colour preview. On the RP2350 the eight data pins are read in a single operation so they must not straddle the boundary between GP31 and GP32. |
| CAMERA CAPTURE [scale [, x, y]] | Captures the 320x240 RGB565 preview image, exactly as for the OV7670 CAMERA CAPTURE above (scale and x,y position it on an SPI LCD; on the HDMI versions it is written to the MODE 4 framebuffer). For a higher-quality full-colour still use CAMERA CAPTURE JPEG. |
| CAMERA CAPTURE JPEG fname$ [, resolution [, quality]] | Captures a full-colour image using the camera's hardware JPEG engine and saves it to the file fname$ (.jpg is added if no extension is given).<br>resolution selects the image size: VGA (640x480, the default), SVGA (800x600), XGA (1024x768), SXGA (1280x1024) or UXGA (1600x1200 - the full 2 megapixels).<br>quality is HIGH, MEDIUM (the default) or LOW, trading file size against image quality.<br>During the capture the display framebuffer is used as working memory, so the screen is disturbed; the next CAMERA CAPTURE restores the preview. The saved file can be displayed using LOAD JPG. |

| | |
|---|---|
| CAMERA CHANGE image%(), change! [, scale [, x, y]] | Motion detection with the OV2640, working as for the OV7670 CAMERA CHANGE above but staying in the 320x240 colour preview mode and using image brightness as the change measure. Size the array DIM image%(320*240/8-1). A common use is to loop on CAMERA CHANGE and trigger CAMERA CAPTURE JPEG when change! exceeds a threshold. |
| Camera (OV5640) | RP2350 versions only. The OV5640 is a 5-megapixel camera, selected by giving OV5640 as the first parameter of CAMERA OPEN. It is self-clocked (needs no master clock from the PicoMite), provides a 320x240 RGB565 colour preview and can save JPEG images at up to 2592x1944 (the full 5 megapixels). The camera clock is automatically adapted to the CPU speed (378, 315 and 252MHz are supported).<br>CAMERA CLOSE is used exactly as for the OV7670. CAMERA REGISTER works with the OV5640's 16-bit register addresses; CAMERA TEST is not available on the OV5640. |
| CAMERA OPEN OV5640, PWDNpin, DCLKpin, HSpin, VSpin, RETpin, D0pin | Initialises the OV5640. The module is wired exactly as for the OV2640 above: the first parameter is the PWDN pin (driven low to power the sensor up), DCLK is the camera's pixel-clock output and D0pin is the start of a range of 8 contiguous pins. On the RP2350 the eight data pins are read in a single operation so they must not straddle the boundary between GP31 and GP32.<br>Enable OPTION SYSTEM I2C and wire SCL and SDA (may be labelled SIOC and SIOD) with pullups to 3.3V (2K7 recommended). The OV5640 always uses 320x240 for the colour preview. |
| CAMERA CAPTURE [scale [, x, y]] | Captures the 320x240 RGB565 preview image, exactly as for the OV2640 CAMERA CAPTURE above (scale and x,y position it on an SPI LCD; on the HDMI versions it is written to the MODE 4 framebuffer). |
| CAMERA CAPTURE JPEG fname$ [, resolution [, quality]] | Captures a full-colour image using the camera's hardware JPEG engine and saves it to the file fname$ (.jpg is added if no extension is given).<br>resolution selects the image size: VGA (640x480, the default), XGA (1024x768), SXGA (1280x960), UXGA (1600x1200), QXGA (2048x1536) or 5MP (2592x1944 - the full 5 megapixels).<br>quality is HIGH, MEDIUM (the default) or LOW, trading file size against image quality. At the largest sizes the quality is automatically limited so that the image fits the capture buffer.<br>The white balance of the saved image is locked to the current preview setting, so for best colour allow the preview to run briefly (e.g. one CAMERA CAPTURE) before taking the JPEG. During the capture the display framebuffer is used as working memory, so the screen is disturbed; the next CAMERA CAPTURE restores the preview. The saved file can be displayed using LOAD JPG. |
| CAMERA CHANGE image%(), change! [, scale [, x, y]] | Motion detection with the OV5640, working exactly as for the OV2640 CAMERA CHANGE above (320x240 colour preview mode, image brightness as the change measure). Size the array DIM image%(320*240/8-1). |
| CAT S$, N$ | Concatenates the strings by appending N$ to S$. This functionally the same as `S$ = S$ + N$` but operates somewhat faster. |
| CHAIN fname$ [cmdline$] | Allows a program to run another program with the variable space preserved – the command is recommended to be used in top level program and not from within a subroutine. If the optional 'cmdline$ parameter is specified this is passed to the chained program in MM.CMDLINE$ |

| | |
|---|---|
| CHDIR dir$ | Change the current working directory on the default drive to ‘dir$’<br>The special entry “..” represents the parent of the current directory and “.” represents the current directory. "/" is the root directory. |
| CIRCLE x, y, r [,lw] [, a] [, c] [, fill] | Draw a circle centred at 'x' and 'y' with a radius of 'r' on the display output. ‘lw’ is optional and is the line width (defaults to 1).<br>'c' is the optional colour and defaults to the current foreground colour if not specified. The optional 'a' is a floating point number which will define the aspect ratio. If the aspect is not specified the default is 1.0 which gives a standard circle. 'fill' is the fill colour and can be omitted or set to -1 in which case the box will not be filled.<br>All parameters can be expressed as arrays and the software will plot the number of circles as determined by the dimensions of the smallest array. 'x',<br>'y' and 'r' must all be arrays or all be single variables/constants otherwise an error will be generated. 'lw', 'a', 'c', and fill can be either arrays or single variables/constants.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates. |
| CLEAR | Delete all variables and recover the memory used by them.<br>See ERASE for deleting specific array variables. |
| CLOSE [#]fnbr [,[#]fnbr] … | Close the file(s) previously opened with the file number ‘#fnbr’. The # is optional. Also see the OPEN command. |
| CLS [colour] | Clears the LCD panel's screen. Optionally 'colour' can be specified which will be used for the background colour when clearing the screen. |
| CMM2 LOAD<br>or<br>CMM2 RUN | RP2350 VERSIONS ONLY Loads and or runs a program from disk using the CMM2 program loading mechanism. This includes an aggressive crunching of the program and supports #INCLUDE files and #DEFINE text replacement. This can be used for compatibility with CMM2 programs or to allow structuring programs into separate modules. It is important to note that if used all editing of programs must be offline or direct to and from disk as the source files cannot be reconstructed from the version loaded by these commands. |
| COLOUR fore [, back]<br>or<br>COLOR fore [, back] | Sets the default colour for commands (PRINT, etc) that display on the on the attached LCD panel. 'fore' is the foreground colour, 'back' is the background colour. The background is optional and if not specified will default to any previously set background colour or black if not previously changed. |
| COLOUR MAP inarray%(), outarray%() [,colourmap%()] | This command generates RGB888 colours in outarray% from colour codes (0-15) in inarray%. If the optional colourmap% parameter is used this must be 16 elements long). In this case the values in inarray% are mapped to the colours for that index value in colourmap% |
| CONFIGURE cfg<br><br>CONFIGURE LIST | Configures a board as per the “cfg” specified (equivalent of OPTION RESET).<br><br>Lists all the various configurations available for the firmware version. |

| | |
|---|---|
| CONST id = expression<br>[, id = expression] … etc | Create a constant identifier which cannot be changed once created.<br>'id' is the identifier which follows the same rules as for variables. The identifier can have a type suffix (!, %, or $) but it is not required. If it is specified it must match the type of 'expression'. 'expression' is the value of the identifier and it can be a normal expression (including user defined functions) which will be evaluated when the constant is created.<br>A constant defined outside a sub or function is global and can be seen throughout the program. A constant defined inside a sub or function is local to that routine and will hide a global constant with the same name. |
| CONTINUE | Resume running a program that has been stopped by an END statement, an error, or CTRL-C. The program will restart with the next statement following the previous stopping point.<br>Note that it is not always possible to resume the program correctly – this particularly applies to complex programs with graphics, nested loops and/or nested subroutines and functions. |
| CONTINUE DO<br>or<br>CONTINUE FOR | Skip to the end of a DO/LOOP or a FOR/NEXT loop. The loop condition will then be tested and if still valid the loop will continue with the next iteration. |
| COPY fname1$ TO fname2$<br><br>COPY fname$ TO dirname$ | Copy a file from ‘fname1$’ to ‘fname2$’. Both are strings.<br>A directory path can be used in both 'fname$' and 'fname$'. If the paths differ the file specified in 'fname$' will be copied to the path specified in 'fname2$' with the file name as specified. The filenames can include the drive specification in the case that you are copying to and or from the non-active drive (see the DRIVE command)<br>Wildcard copy. The bulk copy is triggered if fname$ contains a '*' or a '?' character. dirname$ must be a valid directory name and should NOT end in a slash character. If the fname$ does not contain a wildcard then just the single file will be copied.<br>The firmware will block copying a file to itself. |
| COPY A2A fname1$, fname2$ COPY A2B fname1$, fname2$ COPY B2A fname1$, fname2$ COPY B2B fname1$, fname2$ | Copies a single file directly between the two drives without needing a drive letter in either name: A2A copies within drive A:, A2B copies from A: to B:, B2A copies from B: to A: and B2B copies within drive B:. Both file names are strings and may include a path. A2A and B2B report an error if the source and destination resolve to the same file. These are a shorthand for the normal COPY command with drive-qualified names. |
| CPU RESTART | Will force a restart of the processors.<br>This will clear all variables and reset everything (eg, timers, COM ports, I2C, etc) similar to a power up situation but without the power up banner.<br>If OPTION AUTORUN has been set the program in the specified flash location or program memory will restart. |
| CPU SLEEP n | Will cause the processors to sleep for ‘n’ seconds. Note that the CPU does not have a true low power sleep so the power saving is limited. |

| | |
|---|---|
| CPU SPEED n | NOT VGA OR HDMI VERSIONS<br>Changes the CPU clock speed to 'n' KHz immediately and without restarting the firmware. 'n' has the same range as OPTION CPUSPEED (eg, 48000 to 420000 for the PicoMite). The console and COM serial ports, the ADC and the audio output are automatically re-timed to suit the new clock speed and USB is unaffected, so the USB console connection is retained.<br>Unlike OPTION CPUSPEED the new speed is not saved; on reset or power up the clock speed will return to that set by OPTION CPUSPEED. The command can be used in a program (eg, to reduce the clock speed, and with it the power consumption, when full speed is not needed).<br>The command cannot be used while a PIO is in use as any PIO timing programmed in BASIC would become invalid, and the speed cannot exceed 384000 if PSRAM is enabled.<br>On the VGA/HDMI versions the CPU clock is set by the video resolution; on the HDMI versions it can be changed at run time using the RESOLUTION command. |
| CSUB name [type [, type] …]<br>hex [[ hex[…]<br>hex [[ hex[…]<br>END CSUB | Defines the binary code for an embedded machine code program module written in C or ARM assembler. The module will appear in MMBasic as the command 'name' and can be used in the same manner as a built-in command.<br>Multiple embedded routines can be used in a program with each defining a different module with a different 'name'.<br>The first 'hex' word is a 32 bit word which is the offset in bytes from the start of the CSUB to the entry point of the embedded routine (usually the function main()). The following hex words are the compiled binary code for the module. These are automatically programmed into MMBasic when the program is saved. Each 'hex' must be exactly eight hex digits representing the bits in a 32-bit word and be separated by one or more spaces or new lines. The command must be terminated by a matching END CSUB. Any errors in the data format will be reported when the program is run. During execution MMBasic will skip over any CSUB commands so they can be placed anywhere in the program.<br>The type of each parameter can be specified in the definition. For example:<br>`CSUB MySub integer, integer, string`<br>This specifies that there will be three parameters, the first two being integers and the third a string.<br>Note:<br>• Up to ten arguments can be specified on the RP2040 and up to sixteen on the RP2350 ('arg1', 'arg2', etc).<br>• If a variable or array is specified as an argument the C routine will receive a pointer to the memory allocated to the variable or array and the C routine can change this memory to return a value to the caller. In the case of arrays, they should be passed with empty brackets eg, arg(). In the CSUB the argument will be supplied as a pointer to the first element of the array.<br>• Constants and expressions will be passed to the embedded C routine as pointers to a temporary memory space holding the value. |
| DATA constant[,constant].. | Stores numerical and string constants to be accessed by READ.<br>In general string constants should be surrounded by double quotes ("). An exception is when the string consists of just alphanumeric characters that do not represent MMBasic keywords (such as THEN, WHILE, etc). In that case quotes are not needed.<br>Numerical constants can also be expressions such as 5 * 60. |

| | |
|---|---|
| DATE$ = "DD-MM-YY[YY]"<br>or<br>DATE$ = "DD/MM/YY[YY]"<br>or<br>DATE$ =”YYYY-MM-DD”<br>or<br>DATE$=”YYYY/MM/DD” | Set the date of the internal clock/calendar.<br>DD, MM and YY are numbers, for example: DATE$ = "28-7-2014"<br>With OPTION RTC AUTO ENABLE the PicoMite firmware starts with the DATE$ programmed in RTC.<br>Without OPTION RTC AUTO ENABLE the PicoMite firmware starts with the date set to "01-01-2024" on power up. |
| DEFINEFONT #Nbr<br>hex [[ hex[…]<br>hex [[ hex[…]<br>END DEFINEFONT | This will define an embedded font which can be used alongside or to replace the built in font(s) used on an attached LCD panel. These work exactly same as the built in fonts (i.e. selected using the FONT command or specified in the TEXT command). See the *Embedded Fonts* folder in the PicoMite firmware distribution zip file for a selection of embedded fonts and a full description of how to create them.<br>'#Nbr' is the font's reference number (from 1 to 16). It can be the same as a built in font and in that case it will replace the built in font. Each 'hex' must be exactly eight hex digits and be separated by spaces or new lines from the next.<br>• Multiple lines of 'hex' words can be used with the command terminated by a matching END DEFINEFONT.<br>• Multiple embedded fonts can be used in a program with each defining a different font with a different font number.<br>• During execution MMBasic will skip over any DEFINEFONT commands so they can be placed anywhere in the program.<br>• Any errors in the data format will be reported when the program is saved. |
| DEVICE BITSTREAM | See BITSTREAM command |
| DEVICE CAMERA | See CAMERA command |
| DEVICE GAMEPAD | See GAMEPAD command |
| DEVICE HUMID | See HUMID command |
| DEVICE KEYPAD | See KEYPAD command |
| DEVICE MOUSE | See MOUSE command |
| DEVICE LCD | See LCD command |
| DEVICE SERIALTX pinno, baudrate, ostring$ | Outputs 'ostring$' as a serial data stream on 'pinno'. 'baudrate' can be between 110 and 230400 (230400 may need CPU to be overclocked).<br>Note that the program will halt and interrupts ignored during transmission. |
| DEVICE SERIALRX pinno, baudrate, istring$, timeout_in_ms, status% [,nbr] [,terminators$] | Inputs serial data on ‘pinno’. ‘baudrate’ can be between 110 and 230400 (230400 may need CPU to be overclocked).<br>‘status%’ returns:<br>-1 = timeout (Note: use LEN(istring$) to see number of chars received)<br>2 = number of characters requested satisfied<br>3 = terminating character satisfied<br>‘nbr’ specifies the number of characters to be received before the command returns. ‘terminators$’ specifies one or more single characters that can be used to terminate reception.<br>The program will halt and interrupts ignored while this command is executing. |
| DEVICE WII | See WII command |

| DEVICE WS2812 | See WS2812 command |
|---|---|
| DIM [type] decl [,decl]..<br>where 'decl' is:<br>var [length] [type] [init]<br>'var' is a variable name with optional dimensions<br>'length' is used to set the maximum size of the string to 'n' as in LENGTH n<br>'type' is one of FLOAT or INTEGER or STRING (the type can be prefixed by the keyword AS - as in AS FLOAT)<br>'init' is the value to initialise the variable and consists of:<br>= <expression><br>For a simple variable one expression is used, for an array a list of comma separated expressions surrounded by brackets is used.<br><br>Examples:<br>DIM nbr(50)<br>DIM INTEGER nbr(50)<br>DIM name AS STRING<br>DIM a, b$, nbr(100), strn$(20)<br>DIM a(5,5,5), b(1000)<br>DIM strn$(200) LENGTH 20<br>DIM STRING strn(200) LENGTH 20<br>DIM a = 1234, b = 345<br>DIM STRING strn = "text"<br>DIM x%(3) = (11, 22, 33, 44) | Declares one or more variables (i.e. makes the variable name and its characteristics known to the interpreter).<br>When OPTION EXPLICIT is used (as recommended) the DIM, LOCAL or STATIC commands are the only way that a variable can be created. If this option is not used then using the DIM command is optional and if not used the variable will be created automatically when first referenced.<br>The type of the variable (i.e. string, float or integer) can be specified in one of three ways:<br>By using a type suffix (i.e. !, % or $ for float, integer or string). For example:<br>`DIM nbr%, amount!, name$`<br>By using one of the keywords FLOAT, INTEGER or STRING immediately after the command DIM and before the variable(s) are listed. The specified type then applies to all variables listed (i.e. it does not have to be repeated). For example:<br>`DIM STRING first_name, last_name, city`<br>By using the Microsoft convention of using the keyword "AS" and the type keyword (i.e. FLOAT, INTEGER or STRING) after each variable. If you use this method the type must be specified for each variable and can be changed from variable to variable.<br>For example:<br>`DIM amount AS FLOAT, name AS STRING`<br>Floating point or integer variables will be set to zero when created and strings will be set to an empty string (i.e. ""). You can initialise the value of the variable by using an equals symbol (=) and an expression following the variable definition. For example:<br>`DIM STRING city = "Perth", house = "Brick"`<br>The initialising value can be an expression (including other variables) and will be evaluated when the DIM command is executed. See the chapter *Defining and Using Variables* for more examples of the syntax.<br>As well as declaring simple variables the DIM command will also declare arrayed variables (i.e. an indexed variable with a number of dimensions). Following the variable's name the dimensions are specified by a list of numbers separated by commas and enclosed in brackets. For example:<br>`DIM array(10, 20)`<br>Each number specifies the index range in each dimension. Normally the indexing of each dimension starts at 0 but the OPTION BASE command can be used to change this to 1.<br>The above example specifies a two dimensional array with 11 elements (0 to 10) in the first dimension and 21 (0 to 20) in the second dimension. The total number of elements is 231 and because each floating point number requires 8 bytes a total of 1848 bytes of memory will be allocated.<br>A dimension can hold a single element. With the default OPTION BASE 0 this is written as DIM a(0) and with OPTION BASE 1 as DIM a(1). This matters when the size of an array is calculated while the program is running and the calculation can produce one element, and it allows the MATH matrix commands to produce a single row, a single column, or a 1x1 result.<br>Strings will default to allocating 255 bytes (i.e. characters) of memory for each element and this can quickly use up memory when defining arrays of strings. In that case the LENGTH keyword can be used to specify the amount of memory to be allocated to each element and therefore the maximum length of the string that can be stored. This allocation ('n') can be from 1 to 255 characters. |

| | |
|---|---|
| | For example: `DIM STRING s(5, 10)` will declare a string array with 66 elements consuming 16,896 bytes of memory while:<br>`DIM STRING s(5, 10) LENGTH 20`<br>Will only consume 1,386 bytes of memory. Note that the amount of memory allocated for each element is n + 1 as the extra byte is used to track the actual length of the string stored in each element.<br>If a string longer than 'n' is assigned to an element of the array an error will be produced. Other than this, string arrays created with the LENGTH keyword act exactly the same as other string arrays. This keyword can also be used with non-array string variables but it will not save any memory.<br>In the above example you can also use the Microsoft syntax of specifying the type after the length qualifier. For example:<br>`DIM s(5, 10) LENGTH 20 AS STRING`<br>The length parameter can be used for simple (non arrayed) strings with the following limitations, For the RP2040, if length is greater than 9 it will be ignored and the default 255 bytes allocated. For the RP2350, if the length is greater than 15 it will be ignored and the default 255 bytes allocated. If the length is less than or equal to the limit, the string is contained in the variable header and 256 bytes are saved. This is likely to be particularly useful for short constant strings<br>Arrays can also be initialised when they are declared by adding an equals symbol (=) followed by a bracketed list of values at the end of the declaration. For example:<br>`DIM INTEGER nbr(4) = (22, 44, 55, 66, 88)`<br>or `DIM s$(3) = ("foo", "boo", "doo", "zoo")`<br>Note that the number of initialising values must match the number of elements in the array including the base value set by OPTION BASE. If a multi dimensioned array is initialised then the first dimension will be initialised first followed by the second, etc.<br>Also note that the initialising values must be after the LENGTH qualifier (if used) and after the type declaration (if used). |
| DO<br><statements><br>LOOP | This structure will loop forever; the EXIT DO command can be used to terminate the loop or control must be explicitly transferred outside of the loop by commands like GOTO or EXIT SUB (if in a subroutine). |
| DO WHILE expression<br><statements><br>LOOP | Loops while 'expression' is true (this is equivalent to the older WHILE-WEND loop). If, at the start, the expression is false the statements in the loop will not be executed, not even once. |
| DO UNTIL expression<br><statements><br>LOOP | Loops until 'expression' is true. If, at the start, the expression is true the statements in the loop will not be executed, not even once. |
| DO<br><statements><br>LOOP UNTIL expression | Loops until the expression following UNTIL is true. Because the test is made at the end of the loop the statements inside the loop will be executed at least once, even if the expression is true. |
| DO<br><statements><br>LOOP WHILE expression | Loops until the expression following WHILE is false. Because the test is made at the end of the loop the statements inside the loop will be executed at least once, even if the expression is false. |

| | |
|---|---|
| DRAW3D | NOT PICOMITEMIN OR WEBMITE<br>The 3D engine includes commands for manipulating 3D images including setting the camera, creating, hiding, rotating, etc.<br>See the document "*3D_Graphics_User_Manual.pdf*" in the PicoMite firmware download for a full description. |
| DRIVE drive$ | Sets the active disk drive as 'drive$'. 'drive$' can be "A:", "B:" or "C:" where A is the flash drive, B is the SD Card and C is a USB flash drive on the RP2350 versions with USB support<br>On RP2350 versions with USB support "C:" selects a USB flash drive - see the heading USB Flash Drive in the chapter Keyboard/Mouse/Gamepad. Note the transfer rate is limited by the USB full speed (12 Mbits/s) interface. |
| EDIT<br>or<br>EDIT fname$<br>or<br>EDIT FILE fname$ | Invoke the full screen editor.<br>If a filename is specified the editor will load the file from the current disk (A: or B:) to allow editing and on exit with F1 or F2 save it to the disk. If the file does not exist it is created on exit. The current program stored in flash memory is not affected. If editing an existing file a backup with .bak appended to the filename is also created on exit.<br>If fname$ includes an extension other than .bas then colour coding will be temporarily turned off during the edit.<br>If no extension is specified the firmware will assume .bas<br>Editing a file from disk allows non-Basic files such as html or sprite files to be edited without corruption during the tokenising process that happens when stored to flash.<br>EDIT and EDIT fname$ can only be invoked at the command prompt.<br>If you wish to edit a file in a program you can use the EDIT FILE fname$ command. The command must be used in the top level program and not from within a subroutine.<br>EDIT FILE fname$ differs from EDIT fname$ in that it will automatically save the entire variable space to the A: drive and restores it on exit. The command will fail if there is not enough free space on the A: drive. In the case of an RP2350 with PSRAM the variable space will be saved to a reserved area in the PSRAM and the A: drive is not used.<br>See the chapter *Full Screen Editor* for details of how to use the editor. |
| ELSE | Introduces an optional default condition in a multiline IF statement.<br>See the multiline IF statement for more details. |
| ELSEIF expression THEN<br>or<br>ELSE IF expression THEN | Introduces an optional secondary condition in a multiline IF statement.<br>See the multiline IF statement for more details. |
| END [noend]<br>or<br>END cmd$ | End the running program and return to the command prompt. If a subroutine named MM.END exists in the program it will be executed whenever the program ends with an actual or implied END command. It is not executed if the program ends with the break character (ie, Ctrl-C).<br>The optional parameter 'noend' can be used to block execution of the MM.END subroutine eg, "END noend"<br>if 'cmd$' is specified then it will be executed as though at the command prompt after the program finishes. Note: if "END cmd$" is used but a subroutine MM.END exists it will be executed and cmd$ ignored. |
| END CSUB | Marks the end of a C subroutine. See the CSUB command.<br>Each CSUB must have one and only one matching END CSUB statement. |

| | |
|---|---|
| END FUNCTION | Marks the end of a user defined function. See the FUNCTION command.<br>Each function must have one and only one matching END FUNCTION statement. Use EXIT FUNCTION if you need to return from a function from within its body. |
| ENDIF<br>or<br>END IF | Terminates a multiline IF statement.<br>See the multiline IF statement for more details. |
| END SUB | Marks the end of a user defined subroutine. See the SUB command.<br>Each sub must have one and only one matching END SUB statement. Use EXIT SUB if you need to return from a subroutine from within its body. |
| END TYPE | NOT PICOMITEMIN Marks the end of a user defined structure. See the TYPE command.. |
| ERASE variable [,variable].. | Deletes global variables and frees up the memory allocated to them. This will work with arrayed variables and normal (non array) variables. Arrays can be specified using empty brackets (eg, `dat()`) or just by specifying the variable's name (eg, `dat`).<br>Use CLEAR to delete all variables at the same time (including arrays). |
| ERROR [error_msg$] | Forces an error and terminates the program. This is normally used in debugging or to trap events that should not occur.<br>'error_msg$' is optional and is the message to display on the console. |
| EXECUTE command$ | This executes the Basic command "command$". Use should be limited to basic commands that execute sequentially for example the GOTO statement will not work properly<br>Things that are tested and work OK include GOSUB, Subroutine calls, other simple statements (like PRINT and simple assignments)<br>Multiple statements separated by : are not allowed and will error<br>The command sets an internal watchdog before executing the requested command and if control does not return to the command, like in a GOTO statement, the timer will expire. In this case you will get the message "Command timeout".<br>You cannot call EXECUTE from within code that has been EXECUTE'd.<br>RUN is a special case and will cancel the timer allowing you to use the command to chain programs if required. |
| EXIT DO<br>EXIT FOR<br>EXIT FUNCTION<br>EXIT SUB | EXIT DO provides an early exit from a DO..LOOP<br>EXIT FOR provides an early exit from a FOR..NEXT loop.<br>EXIT FUNCTION provides an early exit from a defined function.<br>EXIT SUB provides an early exit from a defined subroutine.<br>The old standard of EXIT on its own (exit a do loop) is also supported. |
| FILES [fspec$] [,sort] | Lists files in any directories on the default Flash Filesystem or SD Card.<br>'fspec$' (if specified) can contain a path and search wildcards in the filename. Question marks (?) will match any character and an asterisk (*) will match any number of characters. If omitted, all files will be listed.<br>For example:<br>* Find all entries<br>*.TXT Find all entries with an extension of TXT<br>E*.* Find all entries starting with E<br>X?X.* Find all three letter file names starting and ending with X |

| | |
|---|---|
| | mydir/* Find all entries in directory mydir<br>NB: putting wildcards in the pathname will result in an error<br>'sort' specifies the sort order as follows:<br>size by ascending size<br>time by descending time/date<br>name by file name (default if not specified)<br>type by file extension |
| FILL x, y, fillcolour [,bordercolour] | Fills and area of a display with a colour.<br>When used without the optional 'bordercolour' the command will read the colour at position 'x','y' in the display and then fills the area from that point where the current colour is the same with the new colour 'fillcolour'.<br>If the optional 'bordercolour' is specified, the 'fillcolour' will replace whatever colour is already there until it hits the 'bordercolour' specified.<br>Note it will be slow on TFT displays using unbuffered drivers. |
| FLAG(n%)=value | Sets a bit in a system flag register. N% can be between 0 and 63 (i.e. 64 flag bits are available). Value can be 0 or 1.<br>See also the FLAGS command and FLAG function and MM.FLAGS |
| FLAGS=value% | Sets all bits in the system flag register to the value specified.<br>See also the FLAG command and the FLAGS function and MM.FLAGS |
| FLASH | Manages the storage of programs in the flash memory. Up to three programs can be stored in the flash memory and retrieved as required. Note that these saved programs will be erased with a firmware upgrade.<br>One of these flash memory locations can be automatically loaded and run when power is applied using the OPTION AUTORUN n command. In the following 'n' is a number 1 to 3. |
| FLASH LIST | Displays a list of all flash locations including the first line of the program. |
| FLASH LIST n [,all] | List the program saved to slot n. Use ALL to list without page breaks. |
| FLASH ERASE n | Erase a flash program location. |
| FLASH ERASE ALL | Erase all flash program locations. |
| FLASH SAVE n | Save the current program to the flash location specified. |
| FLASH LOAD n | Load a program from the specified flash location into program memory. |
| FLASH RUN n | Runs the program in flash location n, clear all variables. Does not change the program memory. |
| FLASH CHAIN n | Runs the program in flash location n, leaving all variables intact (allows for a program that is much bigger than the program memory). Does not change the program memory. NB: if the chained program uses the READ command it must call RESTORE before the first read. |
| FLASH OVERWRITE n | Erase a flash program location and then save the current program to the flash location specified. |
| FLASH DISK LOAD n, fname$ [,O[VERWRITE]] | Loads the contents of file fname$ into flash slot n as a binary image. The file can be created using LIBRARY DISK SAVE. Also, any file created externally |

| | |
|---|---|
| FLASH LOAD IMAGE n, filename$ [,O/OVERWRITE] | with data required by a program can be loaded and accessed using commands like PEEK and MEMORY COPY using the address of the flash slot.<br>If the optional parameter OVERWRITE (or O) is specified the content of the flash slot will be overwritten without an error being raised.<br><br>This command loads a bmp file specified into image slot n in RGB121 format. Slots 1 to 3 are the flash slots. On an RP2350 with PSRAM, slots 4 to 8 are the RAM slots 1 to 5: they load faster and wear no flash, but are cleared at every reset, so a program loads them at every start. BLIT FLASH, TILEMAP and MM.INFO(FLASH ADDRESS) take the same slot number, so an image moves between flash and RAM by changing that one number.<br>The slot should have been previously erased (RAM ERASE for a RAM slot) or specifying the optional parameter O or OVERWRITE will force an erase.<br>Note that the firmware will set the first two words in the slot to the width of the saved image and the height of the saved image.<br>The image size need not be the same dimensions as the current display. |
| FLASH UF2 fname$ | <u>RP2350 USB VERSIONS ONLY</u><br>Writes the UF2 firmware file 'fname$' (stored on the A: or B: drive, .uf2 is added if no extension is given) directly to a Raspberry Pi Pico connected to the USB port in BOOTSEL mode (a Pico in BOOTSEL mode appears as a USB flash drive on drive C:). This allows a PicoMite to program another Raspberry Pi Pico without using a PC.<br>See also the heading USB Flash Drive in the chapter Keyboard/Mouse/Gamepad. |
| FLASH MODBUFF LOAD fname$ | Loads the .MOD file 'fname$' into the mod buffer (created by OPTION MODBUFF, or the fixed 1MB buffer of an RP2350 with 16MB of flash) so that it can be played with PLAY MODFILE without being read from a drive each time. The file is read from the current drive. |
| FM | <u>NOT PICOMITEMIN</u><br>Opens the full screen, two panel file manager for managing the files on the A:, B: and C: drives. Files can be browsed, sorted, filtered, copied, moved, renamed and deleted; BASIC programs can be run, edited, listed and created; audio files can be played, singly or as a track list of marked files, and image files displayed. Press ESC to exit and F1 for a summary of the keys.<br>See the chapter File Manager for the details. |
| FLUSH [#]fnbr | Causes any buffered writes to a file previously opened with the file number '#fnbr' to be written to disk. The # is optional. Using this command ensures that no data is lost if there is a power cut after a write command. |
| FONT [#]font-number, scaling | This will set the default font for displaying text on an LCD panel or the video output.<br>Fonts are specified as a number. For example, #2 (the # is optional). See the chapter *Graphics Commands and Functions* for details of the available fonts.<br>'scaling' can range from 1 to 15 and will multiply the size of the pixels making the displayed character correspondingly wider and higher. Eg, a scale of 2 will double the height and width. |
| FOR counter = start TO finish [STEP increment] | Initiates a FOR-NEXT loop with the 'counter' initially set to 'start' and incrementing in 'increment' steps (default is 1) until 'counter' is greater than 'finish'.<br>The 'increment' can be an integer or a floating point number. Note that using a floating point fractional number for 'increment' can accumulate rounding errors in 'counter' which could cause the loop to terminate early or late. |

| | |
|---|---|
| | 'increment' can be negative in which case 'finish' should be less than 'start' and the loop will count downwards.<br>See also the NEXT command. |
| FRAME | RP2350 VERSIONS ONLY<br>The frame command supports the programmer to create text based user interfaces see the manual 'FRAME_User_Manual.pdf' |
| FRAMEBUFFER | NOT VGA OR HDMI VERSIONS<br>The Framebuffer command allow you to allocate some of the variable memory to either a framebuffer, a second display layer, or both and then use these in interesting ways to both avoid tearing artefacts and/or play graphics objects over the background display. |
| FRAMEBUFFER CREATE | Creates a framebuffer "F" with a RGB121 colour space and resolution to match the configured SPI colour display |
| FRAMEBUFFER LAYER | Creates a framebuffer "L" with a RGB121 colour space and resolution to match the configured SPI colour display |
| FRAMEBUFFER WRITE where/where$ | Specifies the target for subsequent graphics commands.<br>"where" can be N, F, or L where N is the actual display. A string variable can be used or a literal |
| FRAMEBUFFER CLOSE [which] | Closes a framebuffer and releases the memory. The optional parameter "which" can be F or L. If omitted closes both. |
| FRAMEBUFFER COPY from, to [,b] | Does a highly optimised full screen copy of one framebuffer to another.<br>"from" and "to" can be N, F, or L where N is the physical display.<br>You can only copy from N on displays that support BLIT and transparent text.<br>The firmware will automatically compress or expand the RGB resolution when copying to and from unmatched framebuffers.<br>Of course copying from RGB565 to RGB121 loses information but for many applications (eg, games) 16 colour levels is more than enough.<br>When copying to an LCD display the optional parameter "b" can be used (FRAMEBUFFER COPY F/L, N, B). This instructs the firmware to action the copy using the second CPU in the Raspberry Pi Pico and control returns immediately to the Basic program |
| FRAMEBUFFER WAIT | Pauses processing until the LCD display enters frame blanking. Implemented for ILI9341, ST7789_320 and ILI9488 displays. Used to reduce artefacts when writing to the screen |
| FRAMEBUFFER MERGE [colour] [,mode] [,updaterate] | Copies the contents of the Layer buffer and Framebuffer onto the LCD display omitting all pixels of a particular colour. Preconditions for the command are that FRAMEBUFFER and LAYERBUFFER are both created<br>FRAMEBUFFER MERGE - writes the contents of the framebuffer to the physical display overwriting any pixels in the framebuffer that are set in the layerbuffer (not zero)<br>FRAMEBUFER MERGE col - writes the contents of the framebuffer to the physical display overwriting any pixels in the framebuffer that are in the layerbuffer not set to the transparent colour "col". The colour is specified as a number between 0 and 15 representing:<br>0:BLACK,1:BLUE,2:MYRTLE,3:COBALT,4:MIDGREEN,5:CERULEAN,6:GREEN,7:CYAN,8:RED,9:MAGENTA,10:RUST,11:FUCHSIA,12:BROWN,13:LILAC,14:YELLOW,15:WHITE |

| | |
|---|---|
| | FRAMEBUFFER MERGE col,B - as above except that the transfer to the physical display takes place on the second CPU and control returns to Basic immediately<br>FRAMEBUFFER MERGE col,R [,updaterate] - sets the second CPU to continuously update the physical display with the merger of the two buffers.<br>Automatically sets FRAMEBUFFER WRITE F if not F or L already set. By default the screen will update as fast as possible (At 200MHz an ILI9341 in SPI mode updates about 13 times a second, in 8-bit parallel mode the ILI9341 achieves 27 FPS)<br>If "updaterate" is set then the screen will update to the rate specified in milliseconds (unless that is less than the fastest achievable on the display)<br>NB: FRAMEBUFFER WRITE cannot be set to N while continuous merged update is active.<br>FRAMEBUFFER MERGE col,A - aborts the continuous updates<br>In addition deleting either the layerbuf or framebuffer, ctrl-C, or END will abort the automatic update as well. |
| FRAMEBUFFER SYNC | Waits for the latest background merge or background copy on the second CPU to complete to allow drawing without tearing |
| FRAMEBUFFER | <u>VGA AND HDMI VERSIONS ONLY</u><br>The Framebuffer command allow you to allocate some of the variable memory to framebuffers, layer buffers, or both and then use these in interesting ways to both avoid tearing artefacts and/or play graphics objects over the background display. |
| FRAMEBUFFER CREATE | Creates a framebuffer “F” with a colour space and resolution to match the current display mode |
| FRAMEBUFFER CREATE 2 | RP2350 only: Creates a second framebuffer “2” with a colour space and resolution to match the current display mode. In modes 1, 2 and 5 it is placed in the pre-allocated video memory that the layer buffer “L” would otherwise use, provided that memory is large enough and the layer has not already been created; otherwise, and always in modes 3 and 4, it is allocated from variable memory, which on boards with PSRAM may be PSRAM. It is intended for use with FRAMEBUFFER MERGE as the alternative to a live layer. |
| FRAMEBUFFER LAYER [colour] | Creates a layer buffer “L” with a colour space and resolution to match the current display mode. The optional parameter colour is specified as a number 0-15 (modes 2 and 3), RGB888 colour (mode 4) or 0-255 (mode 5) and specifies a colour which is ignored when the layer is applied to the display. In display modes where automatic layer application is not supported a layer buffer acts as another framebuffer. If framebuffer “2” already occupies the pre-allocated video memory the layer is allocated from variable memory instead, and if that would place it in PSRAM the command fails because PSRAM is too slow for the display driver. |
| FRAMEBUFFER LAYER TOP [colour] | RP2350 only: Creates a second layer buffer “T” with a colour space and resolution to match the current display mode. The optional parameter colour is specified as a number 0-15 (modes 2 and 3), 0-255 (mode 5) and specifies a colour which is ignored when the layer is applied to the display. In display modes where automatic 2nd layer application is not supported acts as another framebuffer. |
| FRAMEBUFFER WRITE where/where$ | Specifies the target for subsequent graphics commands.<br>"where" can be N, F, 2, T, or L where N is the actual display. A string variable can be used |

| | |
|---|---|
| FRAMEBUFFER CLOSE [which] | Closes a framebuffer and releases the memory. The optional parameter "which" can be F, 2, T or L. If omitted closes all. |
| FRAMEBUFFER COPY from, to [,b] | Does a highly optimised full screen copy of one framebuffer to another. "from" and "to" can be N, F, 2, T, or L where N is the physical display. If the optional parameter 'b' is specified pauses processing until the Monitor enters frame blanking. |
| FRAMEBUFFER WAIT | Pauses processing until the next frame blanking starts |
| FRAMEBUFFER MERGE [colour] [,b] | RP2350 only: Copies framebuffer "2" over framebuffer "F" into the physical display "N" in a single pass. Pixels in "2" that are set to the optional transparent colour show the "F" pixel instead. The colour is specified as a number 0-15 (modes 2 and 3), an RGB888 colour (mode 4) or 0-255 (mode 5) and defaults to 0. In mode 1 the two buffers are combined. The layer buffers "L" and "T" are not involved and continue to be applied automatically over the result.<br>If the optional parameter 'b' is specified the merge waits until the Monitor enters frame blanking before it starts, so the display is rewritten ahead of the scan and tearing is avoided. This is the alternative to a live layer for programs that draw sprites over a background: the display only changes when MERGE is commanded. Framebuffer "2" and the layer buffer "L" share the same pre-allocated video memory when it is large enough for them; whichever is created first uses it and the other is allocated from variable memory. |
| FUNCTION xxx (arg1 [,arg2, …]) [AS <type>}<br>  <statements><br>  <statements><br>  xxx = <return value><br>END FUNCTION | Defines a callable function. This is the same as adding a new function to MMBasic while it is running your program.<br>'xxx' is the function name and it must meet the specifications for naming a variable. The type of the function can be specified by using a type suffix (i.e. xxx$) or by specifying the type using AS <type> at the end of the functions definition. For example:<br>`FUNCTION xxx (arg1, arg2) AS STRING`<br>'arg1', 'arg2', etc are the arguments or parameters to the function (the brackets are always required, even if there are no arguments). An array is specified by using empty brackets. i.e. arg3(). The type of the argument can be specified by using a type suffix (i.e. arg1$) or by specifying the type using `AS <type>` `(i.e. arg1 AS STRING).`<br>The argument can also be another defined function or the same function if recursion is to be used (the recursion stack is limited).<br>To set the return value of the function you assign the value to the function's name. For example:<br>`FUNCTION SQUARE(a)`<br>`  SQUARE = a * a`<br>`END FUNCTION`<br>Every definition must have one END FUNCTION statement. When this is reached the function will return its value to the expression from which it was called. The command EXIT FUNCTION can be used for an early exit.<br>You use the function by using its name and arguments in a program just as you would a normal MMBasic function. For example:<br>`PRINT SQUARE(56.8)`<br>When the function is called each argument in the caller is matched to the argument in the function definition. These arguments are available only inside the function. |

| | |
|---|---|
| | Functions can be called with a variable number of arguments. Any omitted arguments in the function's list will be set to zero or a null string.<br>Arguments in the caller's list that are a variable and have the correct type will be passed by reference to the function. This means that any changes to the corresponding argument in the function will also be copied to the caller's variable and therefore may be accessed after the function has ended. The argument can be prefixed with BYVAL which will prevent this mechanism and cause only the value to be used. Alternatively, the prefix BYREF instructs MMBasic that a reference is required and an error will be generated if that cannot be done.<br>Arrays are passed by specifying the array name with empty brackets (eg, arg()) and are always passed by reference and must be the correct type.<br>You must not jump into or out of a function using commands like GOTO. Doing so will have undefined side effects including ruining your day.<br>A function can also return a whole array by declaring the array dimensions with the type at the end of the definition (INTEGER and FLOAT arrays only). For example:<br>`FUNCTION fred (arg1, arg2) AS INTEGER(6)`<br>Inside the function the result is set by assigning to elements of the function's name (eg, fred(i) = 44). Note that reading an element of the function's name inside the function will call the function recursively - if intermediate values must be read back, build the result in a LOCAL array and copy it with fred() = tmp%().<br>Such a function can only be used as the source of a whole-array assignment and the destination array must have the same type and total number of elements. For example:<br>`DIM b%(6) : b%() = fred(1, 2)`<br>It cannot be used anywhere else in an expression, so something like PRINT fred(1, 2) will generate an error.<br>The dimensions can be expressions and are evaluated each time the function is called, after its arguments have been set up, so they can depend on the arguments. The result array can also be filled by passing it to any command that takes an array (eg, the MATH commands) rather than by assigning to its elements. For example, this returns the inverse of whatever size matrix it is given:<br>`FUNCTION Invr(N()) AS FLOAT(BOUND(N(),1), BOUND(N(),2))`<br>`  MATH M_INVERSE N(), Invr()`<br>`END FUNCTION`<br>`DIM a(3,3), b(3,3) : b() = Invr(a())` |
| GAMEPAD COLOUR channel, colour | USB VERSIONS ONLY Changes the colour of the display panel on a PS4 controller on USB channel 'channel'. 'colour' is set as a standard RGB888 value e.g. RGB(RED) |
| GAMEPAD HAPTIC channel left, right | USB VERSIONS ONLY Causes the left and right vibration motors to operate on a PS4 controller on USB channel 'channel'. 'left' and 'right' must be a number between 0 (off) and 255 (maximum). |
| GAMEPAD INTERRUPT ENABLE channel, int [,mask] | USB VERSIONS ONLY Enables interrupts on the button presses on a USB game controller. The optional parameter 'mask' defines which of the switches will trigger the interrupt (defaults to all). 'mask' is a bitmap corresponding to the output of the DEVICE(GAMEPAD channel,B) function. |
| GAMEPAD INTERRUPT DISABLE channel | Disables interrupts from the gamepad on the channel specified |

| | |
|---|---|
| GAMEPAD MONITOR or<br>GAMEPAD MONITOR SILENT | Use GAMEPAD MONITOR before plugging in a gamepad and when plugged in it will show the before and after report with each change of buttons. GAMEPAD MONITOR SILENT turns the same monitoring on but suppresses the printed report |
| GAMEPAD CONFIGURE vid,pid,i0,c0,i1,c1,i2,c2,i3,c3,i4,c4,i5,c5,i6,c6,i7,c7,i8,c8,i9,c9,i10,c10,i11,c11,i12,c12,i13,c13,i14,c14,i15,c15 | Use to configure a gamepad that isn't supported by the firmware. Run the command before plugging in the gamepad. All 34 parameters are mandatory. In each case the i/c parameters define the index into the report and the bit number at that index for the data that corresponds to the relevant bit. See DEVICE(GAMEPAD n,B) for more information on bit usage (0-15) |
| GOSUB target | Initiates a call to the subroutine at 'target', which can be a line number or a label. The subroutine must be terminated with RETURN. Up to 50 GOSUBs can be nested.<br>GOSUB is a legacy command; defined subroutines (SUB…END SUB) are recommended instead for new programs. |
| GOTO target | Branches program execution to the target, which can be a line number or a label. |
| GUI BITMAP x, y, bits [, width] [, height] [, scale] [, c] [, bc] | Displays the bits in a bitmap on a VGA/HDMI monitor or LCD panel starting at 'x' and 'y' on an attached device.<br>'height' and 'width' are the dimensions of the bitmap as displayed on the device and default to 8x8.<br>'scale' is optional and defaults to that set by the FONT command.<br>'c' is the drawing colour and 'bc' is the background colour. They are optional and default to the current foreground and background colours.<br>The bitmap can be an integer or a string variable or constant and is drawn using the first byte as the first bits of the top line (bit 7 first, then bit 6, etc) followed by the next byte, etc. When the top line has been filled the next line of the displayed bitmap will start with the next bit in the integer or string.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates. |
| GUI CALIBRATE<br>or<br>GUI CALIBRATE a,b,c,d,d | <u>NOT VGA OR HDMI VERSIONS</u><br>This command is used to calibrate the touch feature on an LCD panel. It will display a series of targets on the screen and wait for each one to be precisely touched.<br>The command can also be used with five arguments which specify the calibration values and in this case the calibration will be done without displaying any targets or requiring an input from the user. To discover the values use the OPTION LIST **after** calibrating the display normally. Note that these values are specific to that display and can vary considerably. |
| GUI CLICK<br>GUI CLICK x, y | <u>GUI CONTROLS VERSIONS ONLY</u><br>Synthesises a click at the cursor position so that the graphic controls (and the CLICK() and TOUCH() functions) can be driven from BASIC code - for example from GPIO push buttons, a joystick or an automation script - without a mouse or touch screen.<br>GUI CLICK generates a momentary click (button down then up) at the current cursor position; GUI CLICK x, y moves the cursor to 'x', 'y' first. Any GUI INTERRUPT routines fire exactly as they would for a real touch or mouse click. |

| | |
|---|---|
| GUI CLICK DOWN<br>GUI CLICK UP | GUI CLICK DOWN holds the synthetic button down at the cursor position and GUI CLICK UP releases it. Combined with GUI CURSOR x, y these can be used to implement drag operations. |
| GUI CLICK PIN pin [,INV]<br>GUI CLICK PIN OFF | GUI CLICK PIN designates an I/O pin as a physical click button: when the pin is pulled low (or high if INV is specified) it acts exactly like the left mouse button. The pin is polled in the background so it works even while the program is blocked in a modal control (eg, a message box). GUI CLICK PIN OFF releases the pin.<br>GUI CLICK, GUI CLICK x, y, GUI CLICK DOWN and GUI CLICK UP cannot be used inside an interrupt routine. Not available on the RP2040 VGA versions. |
| GUI CURSOR ON [n [,x, y [,colour]]] | <u>GUI CONTROLS VERSIONS ONLY</u><br>Displays and manipulates a cursor (eg, a mouse pointer) on the screen. The cursor sits above all other graphics and nothing can overwrite it.<br>GUI CURSOR ON enables and displays the cursor. 'n' is the cursor number: 0 is a mouse type pointer (the default), 1 is a cross hair and 2 is a user defined cursor previously loaded with GUI CURSOR LOAD. 'x' and 'y' specify the initial position (the centre of the screen if omitted) and 'colour' can be used to override the default white colour of cursors 0 and 1. On an LCD panel the display controller must support reading back the image (eg, ILI9341) otherwise an error will be generated. |
| GUI CURSOR x, y | GUI CURSOR x, y moves the cursor to 'x', 'y'. If the cursor is hidden it is not displayed, just its stored location is updated. |
| GUI CURSOR OFF | GUI CURSOR OFF turns off the cursor (this also unlinks the mouse). |
| GUI CURSOR HIDE | GUI CURSOR HIDE hides the cursor but maintains its position and GUI CURSOR SHOW will show a hidden cursor again at its stored position. |
| GUI CURSOR SHOW | |
| GUI CURSOR COLOUR colour | GUI CURSOR COLOUR (or COLOR) changes the colour of cursor numbers 0 and 1. It does not affect a loaded cursor which has its colours specified by the cursor designer. |
| GUI CURSOR LOAD fname$ | GUI CURSOR LOAD loads cursor number 2 from the file 'fname$' (a .spr extension is added if none is given) and selects it. The file uses the same layout and colour coding as a sprite (see Appendix G) except that the header line is: width, height, xoffset, yoffset. The two offsets define the position of the cursor's active point within the image - the mouse pointer (cursor 0) has offsets 0,0 (the tip of the arrow) and the cross hair (cursor 1) has 7,7 (its centre). The maximum size is 24x24 pixels. |
| GUI CURSOR LINK MOUSE<br>GUI CURSOR UNLINK MOUSE | GUI CURSOR LINK MOUSE links the cursor to the mouse so that it automatically tracks the mouse movement. GUI CURSOR UNLINK MOUSE breaks the link. A mouse must be connected (USB) or configured (PS2 - see the MOUSE OPEN command) before the cursor can be linked. |
| GUI TEST LCDPANEL | Will test a display device (LCD, VGA, etc). It will continuously draw an animated display of colour circles on the display.<br>The test is stopped by a keypress on the console keyboard or, if the on-screen keyboard is enabled (see OPTION SCREEN KEYBOARD), by a double tap on the touch screen. |

| | |
|---|---|
| GUI RESET LCDPANEL | NOT VGA OR HDMI VERSIONS<br>Will reinitialise the configured LCD panel. Initialisation is automatically done when the PicoMite firmware starts up but in some circumstances it may be necessary to interrupt power to the LCD panel (eg, to save battery power) and this command can then be used to reinitialise the display. |
| GUI TEST TOUCH | WITH TOUCH SUPPORT ONLY<br>Will test the display touch feature on an LCD panel.<br>The screen will be cleared and MMBasic will wait for a touch which will cause a white dot to be placed on the display marking the exact touch position on the screen. Any character entered at the console will terminate the test. If the on-screen keyboard is enabled (see OPTION SCREEN KEYBOARD) a double tap (two quick taps in the same place) will also terminate the test. |
| HELP searchtext | NOT PICOMITEMIN The help command looks for a file "help.txt" on the A: drive. This can be a user written or community developed file and must be in a particular format.<br>For each help entry the first line must be a search string prefixed with a ~ character. This is used for the help functionality to find an entry and is not displayed. The 'searchtext' can contain ? for single character substitution or * for multiple character (or none) substitution.<br>Following, the search string the next line would typically give the syntax of a particular command of function. Any subsequent lines would be further explanation.<br>e.g.<br>*~COLOR*<br>*COLOR fore [, back]*<br>*Sets the default color for commands (PRINT, etc)*<br>*that display on the on the attached LCD panel.*<br>*'fore' is the foreground colour, 'back' is the background colour.*<br>*The background is optional and if not specified will default to black.*<br>The command will return all entries that match the 'searchtext' and these will be paged to match the console device. Various versions of help.txt are available on https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=17865 |
| HUMID pin, tvar, hvar [,DHT11] | Returns the temperature and humidity using the DHT22 sensor. Alternative versions of the DHT22 are the AM2303 or the RHT03 (all are compatible).<br>'pin' is the I/O pin connected to the sensor. Any I/O pin may be used.<br>'tvar' is the variable that will hold the measured temperature and 'hvar' is the same for humidity. Both must be present and both must be floating point variables.<br>For example: `HUMID 3, TEMP!, HUMIDITY!`<br>Temperature is measured in ºC and the humidity is percent relative humidity. Both will be measured with a resolution of 0.1. If an error occurs (sensor not connected or corrupt signal) both values will be 1000.0.<br>Normally the DHT22 should powered by 3.3V to keep its output below 3.6V for the Raspberry Pi Pico (the Pico 2 does not have this issue) and the signal pin of should be pulled up by a 1K to 10K resistor (4.7K recommended) to 3.3V.<br>The optional DHT11 parameter modifies the timings to work with the DHT11. Set to 1 for DHT11 and 0 or omit for DHT22. |

| | |
|---|---|
| I2C | More detail is in Appendix B |
| I2C OPEN speed, timeout | Enables the first I²C module in master mode. 'speed' is the clock speed (in KHz) to use and must be one of 100, 400 or 1000.<br>'timeout' is a value in milliseconds after which the master send and receive commands will be interrupted if they have not completed. The minimum value is 100. A value of zero will disable the timeout (though this is not recommended). |
| I2C WRITE addr, option, sendlen, senddata [,senddata ..] | Send data to the I²C slave device. 'addr' is the slave's I²C address.<br>'option' can be 0 for normal operation or 1 to keep control of the bus after the command (a stop condition will not be sent at the completion of the command)<br>'sendlen' is the number of bytes to send. 'senddata' is the data to be sent - this can be specified in various ways (all values sent will be between 0 and 255).<br>Notes:<br>• The data can be supplied as individual bytes on the command line. Example: `I2C WRITE &H6F, 0, 3, &H23, &H43, &H25`<br>• The data can be in a one dimensional array specified with empty brackets (i.e. no dimensions). 'sendlen' bytes of the array will be sent starting with the first element. Example: `I2C WRITE &H6F, 0, 3, ARRAY()`<br>The data can be a string variable (not a constant). Example: `I2C WRITE &H6F, 0, 3, STRING$` |
| I2C READ addr, option, rcvlen, rcvbuf [,rcvbuf ..] | Get data from the I2C slave device. 'addr' is the slave's I2C address.<br>'option' can be 0 for normal operation or 1 to keep control of the bus after the command (a stop condition will not be sent at the completion of the command)<br>'rcvlen' is the number of bytes to receive.<br>'rcvbuf' is the destination for the received data. This may be supplied in any one of three forms:<br>A list of individual numeric variables, one for each byte received. The number of variables must equal 'rcvlen' and array elements such as a(3) are allowed.<br>A one dimensional array of numbers specified with empty brackets. Received bytes will be stored in sequential elements of the array starting with the first.<br>Example: I2C READ &H6F, 0, 3, ARRAY()<br>A string variable. Received bytes will be stored as sequential characters and the string will be set to 'rcvlen' characters. |
| I2C CHECK addr | Will set the read only variable MM.I2C to 0 if a device responds at the address 'addr'. MM.I2C will be set to 1 if there is no response. |
| I2C CLOSE | Disables the master I²C module. This command will also send a stop if the bus is still held. |
| I2C SLAVE | See Appendix B |
| I2C2 | The same set of commands as for I2C (above) but applying to the second I²C channel. |
| I2CLCD | This series of commands controls HD44780-compatible character LCD displays via a PCF8574 I2C I/O expander module. These adapter modules are commonly found as daughter boards on 16x2 and 20x4 character LCDs, providing I2C control using only two wires (plus power). |
| I2CLCD INIT address | Initialize the LCD at the specified I2C address. The address is typically &H27 or &H3F for most modules. PCF8574 uses &H20-&H27, PCF8574A uses |

| | |
|---|---|
| | &H38-&H3F. Requires OPTION SYSTEM I2C to be configured first. A degree symbol (°) is automatically created at character code 0. |
| I2CLCD CLOSE | Close the LCD and turn off the backlight. |
| I2CLCD CLEAR | Clear the display and return cursor to home position. |
| I2CLCD BACKLIGHT state | Turn the backlight on (1) or off (0). |
| I2CLCD CURSOR state [, BLINK] | Control cursor visibility. State can be ON/OFF or 1/0. Optional BLINK parameter (or 1) enables cursor blinking. |
| I2CLCD CREATECHAR code, d0, d1, d2, d3, d4, d5, d6, d7 | Define a custom character at code (0-7). The 8 data bytes (d0-d7) define the 5x8 pixel pattern, with each byte representing one row (values 0-31, only lower 5 bits used). |
| I2CLCD CMD byte [, byte ...] | Send raw command byte(s) directly to the LCD controller. |
| I2CLCD DATA byte [, byte ...] | Send raw data byte(s) directly to the LCD controller. |
| I2CLCD line, position, string$ | Display text on line (1-4) starting at position (1-40). |
| I2CLCD line, Cn, string$ | Display centered text on line (1-4). Cn specifies display width: C8, C16, C20, or C40.<br><br>NB:<br>Hardware Configuration:<br>The PCF8574 I/O expander must be wired to the LCD using the standard configuration:<br>P0 = RS (Register Select)<br>P1 = RW (Read/Write)<br>P2 = EN (Enable)<br>P3 = Backlight control<br>P4 = D4<br>P5 = D5<br>P6 = D6<br>P7 = D7<br>This is the default wiring used by virtually all LCD I2C backpack modules.<br>Prerequisites:<br>The SYSTEM I2C interface must be configured before using this command:<br>OPTION SYSTEM I2C sda_pin, scl_pin |
| IF expr THEN stmt [: stmt]<br>or<br>IF expr THEN stmt ELSE stmt | Evaluates the expression ‘expr' and performs the statement following the THEN keyword if it is true or skips to the next line if false. If there are more statements on the line (separated by colons (:) they will also be executed if true or skipped if false. The ELSE keyword is optional and if present the statement(s) following it will be executed if 'expr' resolved to be false.<br>The ‘THEN statement’ construct can be also replaced with:<br>GOTO linenumber \| label’.<br>This type of IF statement is all on one line. |

| | |
|---|---|
| IF expression THEN<br><statements><br>[ELSEIF expression THEN<br><statements>]<br>[ELSE<br><statements>]<br>ENDIF | Multiline IF statement with optional ELSE and ELSEIF cases and ending with ENDIF. Each component is on a separate line.<br>Evaluates 'expression' and performs the statement(s) following THEN if the expression is true or optionally the statement(s) following the ELSE statement if false. The ELSEIF statement (if present) is executed if the previous condition is false and it starts a new IF chain with further ELSE and/or ELSEIF statements as required. One ENDIF is used to terminate the multiline IF. |
| INC var [,increment] | Increments the variable "var" by either 1 or, if specified, the value in increment. "increment" can be negative which will decrement.<br>This is functionally the same as `var = var + increment` but is processed much faster |
| INPUT ["prompt$";] var1 [,var2 [, var3 [, etc]]] | Will take a list of values separated by commas (,) entered at the console and will assign them to a sequential list of variables.<br>For example, if the command is: INPUT a, b, c<br>And the following is typed on the keyboard: 23, 87, 66<br>Then a = 23 and b = 87 and c = 66<br>The list of variables can be a mix of float, integer or string variables. The values entered at the console must correspond to the type of variable.<br>If a single value is entered a comma is not required (however that value cannot contain a comma).<br>'prompt$' is a string constant (not a variable or expression) and if specified it will be printed first. Normally the prompt is terminated with a semicolon (;) and in that case a question mark will be printed following the prompt. If the prompt is terminated with a comma (,) rather than the semicolon (;) the question mark will be suppressed. |
| INPUT #nbr,<br>list of variables | Same as above except that the input is read from a serial port or file previously opened for INPUT as 'nbr'. See the OPEN command. |
| INTERRUPT [myint] | This command triggers a software interrupt. The interrupt is set up using INTERRUPT 'myint' where 'myint' is the name of a subroutine that will be executed when the interrupt is triggered.<br>Use INTERRUPT 0 to disable the interrupt<br>Use INTERRUPT without parameters to trigger the interrupt.<br>Note: the interrupt can also be triggered from within a CSUB |
| IR dev, key , int<br>or<br>IR CLOSE | Decodes NEC or Sony infrared remote control signals.<br>An IR Receiver Module is required to sense the IR light and demodulate the signal. It can be connected to any pin however this pin must be configured in advanced using the command: `SETPIN n, IR`<br>The IR signal decode is done in the background and the program will continue after this command without interruption. 'dev' and 'key' should be numeric variables and their values will be updated whenever a new signal is received ('dev' is the device code transmitted by the remote and 'key' is the key pressed).<br>'int' is a user defined subroutine that will be called when a new key press is received or when the existing key is held down for auto repeat. In the interrupt subroutine the program can examine the variables 'dev' and 'key' and take appropriate action.<br>The IR CLOSE command will terminate the IR decoder.<br>Note that for the NEC protocol the bits in 'dev' and 'key' are reversed. For example, in 'key' bit 0 should be bit 7, bit 1 should be bit 6, etc. This does not affect normal use but if you are looking for a specific numerical code provided |

| | |
|---|---|
| | by a manufacturer you should reverse the bits. This describes how to do it: http://www.thebackshed.com/forum/forum_posts.asp?TID=8367<br>'dev' and 'key' must be global (or STATIC), not LOCAL, as they are set in the background.<br>See the chapter *Special Hardware Devices* for more details. |
| IR SEND pin, dev, key | Generate a 12-bit Sony Remote Control protocol infrared signal.<br>'pin' is the I/O pin to use. This can be any I/O pin which will be automatically configured as an output and should be connected to an infrared LED. Idle is low with high levels indicating when the LED should be turned on.<br>'dev' is the device being controlled and is a number from 0 to 31, 'key' is the simulated key press and is a number from 0 to 127.<br>The IR signal is modulated at about 38KHz and sending the signal takes about 25mS during which program execution is paused. |
| IRETURN | Returns from an interrupt routine. An interrupt routine is normally a defined subroutine which is terminated with END SUB, however an interrupt can also be specified as a line number or a label and in that case the interrupt code must be terminated with IRETURN. |
| KEYBOARD ON<br>or<br>KEYBOARD OFF | RP2350 VGA USB AND HDMI USB VERSIONS ONLY<br>Display (ON) or remove (OFF) the on-screen keyboard configured with OPTION SCREEN KEYBOARD.<br>The on-screen keyboard is automatically displayed at the command prompt and in the full screen editor and is removed while a program is running. A program can use KEYBOARD ON to display the keyboard and accept input from it; keys tapped on it are read as normal keystrokes (eg, with INKEY$ or INPUT).<br>KEYBOARD OFF entered at the command prompt will keep the on-screen keyboard hidden until KEYBOARD ON is entered. |
| KEYPAD var, int, r1, r2, r3, r4, c1, c2, c3 [, c4]<br>or<br>KEYPAD CLOSE | Monitor and decode key presses on a 4x3 or 4x4 keypad.<br>Monitoring of the keypad is done in the background and the program will continue after this command without interruption. 'var' should be a numeric variable and its value will be updated whenever a key press is detected.<br>'int' is a user defined subroutine that will be called when a new key press is received. In the interrupt subroutine the program can examine the variable 'var' and take appropriate action.<br>r1, r2, r3 and r4 are pin numbers used for the four row connections to the keypad and c1, c2, c3 and c4 are the column connections. c4 is optional and is only used with 4x4 keypads. This command will automatically configure these pins as required.<br>On a key press the value assigned to 'var' is the number of a numeric key (eg, '6' will return 6) or 10 for the * key and 11 for the # key. On 4x4 keypads the number 20 will be returned for A, 21 for B, 22 for C and 23 for D.<br>The KEYPAD CLOSE command will terminate the keypad function and return the I/O pin to a not configured state.<br>'var' must be global (or STATIC), not LOCAL, as it is set in the background.<br>See the section *Special Hardware Devices* for more details. |
| KEYPAD keymap!(), var!,int, startcolpin, nocols, startrowpin, norows | RP2350 VERSIONS ONLY<br>Configure and use a keypad with any number of rows and columns as well as allocate user defined specific return codes to each key.<br>'keymap!()' is a two dimensional array of floats that contains the user supplied mapping for every key. The array dimensions must match the number of columns and rows specified. |

| | |
|---|---|
| | 'var!' is the float variable that will contain the value returned. 'int' is the interrupt routine called when a key is pressed.<br>'keymap!()' and 'var!' must be global (or STATIC), not LOCAL, as they are used in the background.<br>'startcolpin' is the pin that begins the range of contiguous GP pins used for the column scan while 'nocols' is the number of pins in the range. 'startrowpin' is the pin that begins 'norows' of contiguous GP pins used for the row scan.<br>Example: With a keypad consisting of 3 columns and 4 rows (ie, 123 / 456 / 789 / *0#). Connect GP1 to the left column, GP2 to the middle and GP3 to the right, then the 4 rows from top to bottom to GP4 to GP7.<br>The program could be:<br>`OPTION BASE 1`<br>`DIM keymap(3,4)=(1,4,7,10, 4,5,6,0, 3,6,9,11)`<br>`KEYPAD keymap(), keyret, myint, GP1, 3, GP4, 4`<br>`DO`<br>`  ' main program processing loop`<br>`LOOP`<br>`SUB myint`<br>`  PRINT keyret`<br>`END SUB` |
| KILL file$ [,all] | Deletes the file specified by 'file$'. Any extension must be specified.<br>Bulk erase is triggered if fname$ contains a '*' or a '?' character. If the optional 'all' parameter is used then you will be prompted for a single confirmation. If 'all' is not specified you will be prompted on each file. |
| LCD INIT d4, d5, d6, d7, rs, en<br>or<br>LCD line, pos, text$<br>or<br>LCD CLEAR<br>or<br>LCD CLOSE | Display text on an LCD character display module. This command will work with most 1-line, 2-line or 4-line LCD modules that use the KS0066, HD44780 or SPLC780 controller (however this is not guaranteed).<br>The LCD INIT command is used to initialise the LCD module for use. 'd4' to 'd7' are the I/O pins that connect to inputs D4 to D7 on the LCD module (inputs D0 to D3 should be connected to ground). 'rs' is the pin connected to the register select input on the module (sometimes called CMD). 'en' is the pin connected to the enable or chip select input on the module. The R/W input on the module should always be grounded. The above I/O pins are automatically set to outputs by this command.<br>When the module has been initialised data can be written to it using the LCD command. 'line' is the line on the display (1 to 4) and 'pos' is the character location on the line (the first location is 1). 'text$' is a string containing the text to write to the LCD display.<br>'pos' can also be C8, C16, C20 or C40 in which case the line will be cleared and the text centred on a 8 or 16, 20 or 40 line display. For example:<br>`LCD 1, C16, "Hello"`<br>LCD CLEAR will erase all data displayed on the LCD and LCD CLOSE will terminate the LCD function and return all I/O pins to the not configured state.<br>See the chapter *Special Hardware Devices* for more details. |
| LCD CMD d1 [, d2 [, etc]]<br>or<br>LCD DATA d1 [, d2 [, etc]] | These commands will send one or more bytes to an LCD display as either a command (LCD CMD) or as data (LCD DATA). Each byte is a number between 0 and 255 and must be separated by commas. The LCD must have been previously initialised using the LCD INIT command (see above).<br>These commands can be used to drive a non standard LCD in "raw mode" or they can be used to enable specialised features such as scrolling, cursors and custom character sets. You will need to refer to the data sheet for your LCD to find the necessary command and data values. |

| | |
|---|---|
| LET variable = expression | Assigns the value of 'expression' to the variable. LET is automatically assumed if a statement does not start with a command. For example:<br>`Var = 56`<br>A whole array can be copied into another array by using the array names with empty brackets. For example:<br>`b%() = a%()`<br>Both arrays must be the same type (for strings, also the same maximum length) and must hold the same total number of elements; the dimensions may differ (eg, a 2 x 3 array can be copied into a six element array). The source can also be a user defined function that returns an array (see the FUNCTION command). |
| | The library is a special segment of program memory that can contain program code such as subroutines, functions and CFunctions. These routines are not visible to the programmer but are available to the running program and act the same as the built in commands and functions in MMBasic.<br>Any code in the library that is not contained within a subroutine or function will be executed immediately before a program is run. This can be used to initialise constants, set options, etc. See the heading *The Library* in this manual for a full explanation.<br>The library is stored in program memory Flash Slot 3 which will then not be available for saving a program (slots 1 to 2 will still be available). |
| LIBRARY SAVE<br>or | LIBRARY SAVE will take whatever is in normal program memory, compress it (remove redundant data such as comments) and append it to the library area (main program memory is then empty). The code in the library will not show in LIST or EDIT and will not be deleted when a new program is loaded or NEW is used. |
| LIBRARY DELETE<br>or | LIBRARY DELETE will remove the library and return Flash Slot 3 for normal use (OPTION RESET will do the same). |
| LIBRARY LIST<br>or<br>LIBRARY LIST ALL<br>or | LIBRARY LIST will list the contents of the library. Use ALL to list without page confirmations. |
| LIBRARY DISK SAVE fname$<br>or<br>LIBRARY DISK LOAD fname$<br>or | LIBRARY DISK SAVE fname$ will save the current library as a binary file allowing a subsequent call to LIBRARY DISK LOAD fname$ to restore the library. Together, these allow libraries specific for individual programs to be stored and restored easily and distributed. Other than using version specific functionality in the library (WEB, VGA, GUI) libraries can be shared between versions. |
| LIBRARY LOAD fname$ [, fname$ ...] [, O] [, RAM] | NOT PICOMITEMIN LIBRARY LOAD fname$ will read a BASIC source file from the filesystem, compress it and install it as the library, replacing whatever was there. Unlike LIBRARY SAVE it may be used inside a running program, which allows a program to install its own library rather than requiring the user to do it beforehand; it must be the first statement in the program, because a library initialises itself before the program's first line is executed. A hash of the source is remembered, so if the library already holds that exact file the command does nothing and no flash is written – a program may therefore use it on every run at no cost. If a different library is already installed MMBasic will ask for confirmation before replacing it; the option O (or OVERWRITE) suppresses the question. After a library has been installed the program is restarted automatically so that the new library is initialised.<br>More than one filename may be given, separated by commas. The files are read in the order listed and concatenated into a single library, which lets the parts of a library be kept in separate source files and assembled on the board. A bare O, OVERWRITE or RAM anywhere in the list is the option of that name rather than a filename; a filename is always a string expression, so a |

| | |
|---|---|
| | quoted "O" is a filename. Up to eight files, and nothing is written until every one of them has been read, so a mistyped name in the last file leaves the existing library untouched.<br>RAM asks for the library to go into RAM slot 5 (image slot 8) in PSRAM instead of flash, which is possible on an RP2350 with OPTION PSRAM. While it is there it takes the place of the flash library, which is left exactly as it was, so a game can bring a large kernel of its own without disturbing the routines the user keeps in flash. It lasts until END or the next RUN, so a program does this at every start; the file's hash is kept in the slot and a repeat load of the same file is immediate. No question is asked and O is not needed. Without PSRAM the RAM keyword is ignored and the library goes to flash as described above. |
| LINE x1, y1, x2, y2 [,[-] LW [, C]] | Draws a line on the display starting at the coordinates 'x1' and 'y1' and ending at 'x2' and 'y2'.<br>'LW' is the line's width. It defaults to 1 if not specified. For lines that have a defined width and the x1 and y1 coordinate define the top-left pixel of the thick line. i.e. the line is to the right of the specified position or below it on the screen.<br>If width is given as a -ve value then width will apply to lines in all directions and they are centred on the given origin and destination coordinates.<br>'C' is an integer representing the colour and defaults to the current foreground colour.<br>All parameters can be expressed as arrays and the software will plot the number of lines as determined by the dimensions of the smallest array. 'x1', 'y1', 'x2', and 'y2' must all be arrays or all be single variables /constants otherwise an error will be generated. 'lw' and 'c' can be either arrays or single variables/constants. |
| LINE AA x1, y1, x2, y2 [, LW [, C]] | Draws a line with anti-aliasing . The parameters are as per the LINE command above. However this version will use variable intensity values of the specified colour to reduce the "staggered" quality of diagonal lines. In addition this version can draw diagonal lines of any width. Note that it does not accept arrays as parameters. |
| LINE GRAPH x(), y(), colour | This command generates a line graph of the coordinate pairs specified in "x()" and "y()". The graph will have n-1 segments where there are n elements in the x and y arrays. |
| LINE INPUT [prompt$,] string-variable$ | Reads an entire line from the console input into 'string-variable$'.<br>'prompt$' is a string constant (not a variable or expression) and if specified it will be printed first. A question mark is not printed unless it is part of 'prompt$'. Unlike INPUT, this command will read a whole line, not stopping for comma delimited data items. |
| LINE INPUT #nbr, string-variable$ | Same as above except that the input is read from a serial communications port or a file previously opened for INPUT as 'nbr'. See the OPEN command. |

| | |
|---|---|
| LINE PLOT ydata() [,nbr] [,xstart] [,xinc] [,ystart] [,yinc] [,colour] | Plots a line graph from an array of y-axis data points.<br>'ydata()' is an array of floats or integers to be plotted<br>'nbr 'is the number of line segments to be plotted - defaults to the lesser of the array size and MM.HRES-2 if omitted<br>'xstart' is the x-coordinate to start plotting - defaults to 0<br>'xinc' is the increment along the x-axis to plot each coordinate - defaults to 1<br>'ystart' is the location in ydata to start the plot - defaults to the array start. NB: respects OPTION BASE<br>'yinc' is the increment to the index into ydata to add for each point to be plotted<br>'colour' is the colour to draw the line |
| LIST [fname$]<br>or<br>LIST ALL [fname$] | List a program on the console.<br>LIST on its own will list the program with a pause at every screen full.<br>LIST ALL will list the program without pauses. This is useful if you wish to transfer the program to a terminal emulator on a PC that has the ability to capture its input stream to a file. I<br>f the optional 'fname$' is specified then that file on the Flash Filesystem or SD Card will be listed. |
| LIST PINS | Lists the current status of all pins on the processor |
| LIST SYSTEM I2C | Lists a map of all I2C devices connected to the system I2C bus |
| LIST COMMANDS<br>or<br>LIST FUNCTIONS | Lists all valid commands or functions |
| LIST VARIABLES [s%()] | Lists all global variables and constants and, if invoked in a subroutine, the variable used by that subroutine and any subroutines that called it. If the optional parameter s$() is used the variables will be listed to s%() treated as a longstring (see LONGSTRING command). |
| LMID(array%(),start [,num])=string$ | inserts 'string$' into the longstring 'array%()' at position 'start' replacing 'num' existing characters. If 'num' isn't specified then it is calculated from the length of 'string$'.<br>'num' can be 0 in which case 'string$' is inserted at the position specified. |
| LOAD file$ [,C] [,R] | Loads a program called 'file$' from the Flash Filesystem or SD Card into program memory.<br>If the optional suffix ,R is added the program will be immediately run without prompting (in this case 'file$' must be a string constant). The RUN command does the same thing and allows a string variable to be used.<br>LOAD with the ,R suffix is also the only form that may be used inside a running program, which is how one program can chain to another. Without it the command is rejected with 'Invalid in a program'.<br>If the optional suffix ,C is added MMBasic will remove all comments, blank lines and unnecessary spaces from the program as it is loaded. This can be used on large programs to allow them to fit into program memory. The file on the disk is not changed, however the program held in memory (as shown by LIST or EDIT) will be the crunched version and error messages will report the crunched line numbers. The ,C and ,R suffixes may both be used, in either order.<br>If an extension is not specified ".BAS" will be added to the file name. |

| | |
|---|---|
| LOAD CONTEXT [KEEP] | Restores the variable space to the previously saved state and optionally preserves the stored variables to allow a second LOAD if required.<br>See also SAVE CONTEXT |
| LOAD DATA fname$, address | Loads the file ‘fname$ as binary data and writes it to the memory address specified.<br>Note that there is no validation of the address so the command must be used with care. |
| LOAD IMAGE/BMP fname$ [,x] [,y] [,mode] [,ximage] [,yimage] | Loads a BMP file from ‘fname$’ and writes it to the current output device (display or framebuffer) starting at ‘x’, ‘y’ (defaults to 0,0).<br>The optional ‘mode’ parameter specifies if the output is to be dithered.<br>Bits 0 and 1 specify the output format and bit 2 specifies the type of dithering to use.<br>By default mode is set to -1 which indicates no dithering should be applied.<br>DITHER_FLOYD_STEINBERG_RGB121 0<br>DITHER_FLOYD_STEINBERG_RGB222 1<br>DITHER_FLOYD_STEINBERG_RGB332 2<br>DITHER_ATKINSON_RGB121 4<br>DITHER_ATKINSON_RGB222 5<br>DITHER_ATKINSON_RGB332 6<br>‘ximage’ and ‘yimage’ specify where in the image file to start writing to the display (defaults to 0,0) If an extension is not specified “.BMP” will be added to the file name.<br>All types of the BMP format are supported including black and white and true colour 24-bit images. |
| LOAD JPG file$ [, x] [, y] [,mode] [,ximage] [,yimage] [,scale] | Loads a JPG file from ‘fname$’ and writes it to the current output device (display or framebuffer) starting at ‘x’, ‘y’ (defaults to 0,0).<br>The optional ‘mode’ parameter specifies if the output is to be dithered.<br>Bits 0 and 1 specify the output format and bit 2 specifies the type of dithering to use.<br>By default mode is set to -1 which indicates no dithering should be applied.<br>DITHER_FLOYD_STEINBERG_RGB121 0<br>DITHER_FLOYD_STEINBERG_RGB222 1<br>DITHER_FLOYD_STEINBERG_RGB332 2<br>DITHER_ATKINSON_RGB121 4<br>DITHER_ATKINSON_RGB222 5<br>DITHER_ATKINSON_RGB332 6<br>‘ximage’ and ‘yimage’ specify where in the image file to start writing to the display (defaults to 0,0)<br>‘scale’ reduces the displayed size of the image by pixel averaging: use 2 for 50% (each output pixel is the average of a 2x2 block of source pixels) or 4 for 25% (a 4x4 block). The default is 1 which displays the image at full size.<br>If an extension is not specified “.JPG” will be added to the file name.<br>Progressive jpg images are not supported. |
| LOAD PNG fname$ [, x] [, y] [,transparent] [,alphacut] [,ximage] [,yimage] | <u>RP2350 VERSIONS ONLY</u><br>Loads and displays a png file 'fname'<br>If no extension is specified .png will be automatically added to the filename.<br>The file must be in RGBA8888 format, which is the normal default, or an indexed (palette) format; interlaced files are not supported. If specified 'x' and 'y' indicate where on the display or framebuffer the image will appear.<br>The optional parameter 'transparent' (defaults to 0) specifies one of the colour codes (0-15) which will be allocated to pixels in the png file with an alpha value less than 'alphacut' (defaults to 20). If 'transparent' is set to -1 the png |

| | |
|---|---|
| | image is written with pixels with an alpha value less than 'alphacut' missed completely.<br>'ximage' and 'yimage' specify where in the image file to start writing to the display (defaults to 0,0). An image larger than the display is cropped to fit, and as the image is decoded a line at a time a large image needs little memory. |
| LOCAL variable [, variables]<br>See DIM for the full syntax. | Defines a list of variable names as local to the subroutine or function. This command uses exactly the same syntax as DIM and will create variables that will only be visible within the subroutine or function. They will be automatically discarded when the subroutine or function exits. |
| LOCATION | RP2350 VERSIONS ONLY Sets a manual location (latitude/longitude) and time for use with the ASTRO command.<br>This is one of a set of commands that performs high-precision astronomical calculations suitable for telescope pointing or navigation and is described in detail in the file *GPS_Astro_Reference.pdf* included in the firmware download zip file. |
| LONGSTRING | The LONGSTRING commands allow for the manipulation of strings longer than the normal MMBasic limit of 255 characters.<br>Variables for holding long strings must be defined as single dimensioned integer arrays with the number of elements set to the number of characters required for the maximum string length divided by eight. The reason for dividing by eight is that each integer in an MMBasic array occupies eight bytes. Note that the long string routines do not check for overflow in the length of the strings. If an attempt is made to create a string longer than a long string variable's size the outcome will be undefined. |
| LONGSTRING AES128 ENCRYPT/DECRYPT CBC/ECB/CTR key$/key[!/%](), in%(), out%() [,iv$/iv[!/%]()] | Encrypts or decrypts the longstring in in%() putting the answer in out%()<br>For CBC and CTR modes the encryption will generate a random initialisation vector and prepend out%() with the IV. If an explicit IV is specified this will be used instead of the random vector and this will be prepended to out%()<br>For CBC and CTR decryption the firmware assumes that the first 16 bytes of in%() are the initialisation vector.<br>In the case where you want to transmit a message without IV you can use LONGSTRING RIGHT to remove the IV before sending the message. In this case the recipient must know the IV as well as the key and create a complete longstring before using DECRYPT. This can be done by using LONGSTRING CONCAT to add the incoming message to a longstring containing the IV. |
| LONGSTRING APPEND array%(), string$ | Append a normal MMBasic string to a long string variable. 'array%()' is a long string variable while 'string$' is a normal MMBasic string expression. |
| LONGSTRING BASE64 ENCODE/DECODE in%(), out%() | This BASE64 encodes or decodes the longstring in in%() placing the answer in out%(). The array used as the output must be big enough relative to the input and the direction. Encoding increases length by 4/3 and decoding decreases it by 3/4 |
| LONGSTRING CLEAR array%() | Will clear the long string variable 'array%()'. i.e. it will be set to an empty string. |
| LONGSTRING COPY dest%(), src%() | Copy one long string to another. 'dest%()' is the destination variable and 'src%()' is the source variable. Whatever was in 'dest%()' will be overwritten. |
| LONGSTRING CONCAT dest%(), src%() | Concatenate one long string to another. 'dest%()' is the destination variable and 'src%()' is the source variable. 'src%()' will the added to the end of 'dest%()' (the destination will **not** be overwritten). |

| | |
|---|---|
| LONGSTRING LCASE array%() | Will convert any uppercase characters in ‘array%()’ to lowercase. ‘array%()’ must be long string variable. |
| LONGSTRING LEFT dest%(), src%(), nbr | Will copy the left hand 'nbr' characters from ‘src%()’ to ‘dest%()’ overwriting whatever was in ‘dest%()’. i.e. copy from the beginning of ‘src%()’. ‘src%()’ and ‘dest%()’ must be long string variables. 'nbr' must be an integer constant or expression. |
| LONGSTRING LOAD array%(), nbr, string$ | Will copy 'nbr' characters from ‘string$’ to the long string variable ‘array%()’ overwriting whatever was in ‘array%()’. |
| LONGSTRING MID dest%(), src%(), start, nbr | Will copy 'nbr' characters from ‘src%()’ to ‘dest%()’ starting at character position 'start' overwriting whatever was in ‘dest%()’. i.e. copy from the middle of ‘src%()’. 'nbr' is optional and if omitted the characters from 'start' to the end of the string will be copied ‘src%()’ and ‘dest%()’ must be long string variables. 'start' and 'nbr' must be integer constants or expressions. |
| LONGSTRING PRINT [#n,] src%() [;] | Prints the longstring stored in ‘src%()’ to the file or COM port opened as ‘#n’. If ‘#n’ is not specified the output will be sent to the console. Add a semi-colon to suppress CR/LF. |
| LONGSTRING REPLACE array%() , string$, start | Will substitute characters in the normal MMBasic string ‘string$’ into an existing long string ‘array%()’ starting at position ‘start’ in the long string. |
| LONGSTRING RESIZE addr%(), nbr | Sets the size of the longstring to ‘nbr’. This overrides the size set by other longstring commands so should be used with caution. Typical use would be in using a longstring as a byte array. |
| LONGSTRING RIGHT dest%(), src%(), nbr | Will copy the right hand 'nbr' characters from ‘src%()’ to ‘dest%()’ overwriting whatever was in ‘dest%()’. i.e. copy from the end of ‘src%()’. ‘src%()’ and ‘dest%()’ must be long string variables. 'nbr' must be an integer constant or expression. |
| LONGSTRING SETBYTE addr%(), nbr, data | Sets byte ‘nbr’ to the value “data”, ‘nbr’ respects OPTION BASE |
| LONGSTRING TRIM array%(), nbr | Will trim ‘nbr’ characters from the left of a long string. ‘array%()’ must be a long string variables. 'nbr' must be an integer constant or expression. |
| LONGSTRING UCASE array%() | Will convert any lowercase characters in ‘array%()’ to uppercase. ‘array%()’ must be long string variable. |
| LOOP [WHILE expression \| UNTIL expression] | Terminates a program loop: see DO. |

| | |
|---|---|
| MANDELBROT [maxiter]<br><br>MANDELBROT DRAW [maxiter]<br><br>MANDELBROT PAN dx, dy<br><br>MANDELBROT ZOOM factor<br><br>MANDELBROT CENTRE x, y<br><br>MANDELBROT RESET | <u>VGA AND ALL RP2350 VERSIONS ONLY</u><br>Draws the Mandelbrot set over the full screen using a fast fixed point renderer with a 16 colour palette (points inside the set are drawn in black). The view (centre position and zoom level) is remembered so a sequence of commands can be used to explore the set.<br>MANDELBROT (or MANDELBROT DRAW) draws the set at the current view. 'maxiter' is the optional maximum number of iterations calculated for each pixel (1 to 10000, default 64) - larger values reveal more detail when zoomed in but take longer to draw.<br>MANDELBROT PAN moves the view by 'dx', 'dy' pixels and redraws.<br>MANDELBROT ZOOM zooms the view by 'factor' (greater than 1 zooms in, less than 1 zooms out) and redraws.<br>MANDELBROT CENTRE (or CENTER) re-centres the view on the pixel coordinates 'x', 'y' of the last drawn image and redraws.<br>MANDELBROT RESET returns to the default view showing the complete set (without redrawing it).<br>Drawing can be interrupted with CTRL-C. |
| MAP<br><br>MAP( n ) = rgb%<br><br>MAP MAXIMITE<br><br>MAP GREYSCALE<br><br>MAP SET<br><br>MAP RESET | <u>HDMI VERSIONS ONLY</u><br>The MAP commands allow the programmer to set the colours used in 4 or 8-bit colour modes. Each value in the 4 or 8-bit colour pallet can be set to an independent 24-bit colour (ie, RGB555 format). See the MAP function for more information<br>This will assign the 24-bit colour 'rgb% to all pixels with the 4 or 8-bit colour value of 'n'. The change is activated after the MAP SET command.<br>This will set the colour map to the colours implemented in the original Colour Maximite.<br>This will set the colour map to 16 or 32 levels of grey (depending on the MODE). MAP GRAYSCALE is also valid.<br>This will cause MMBasic to update the colour map (set using MAP(n)=rgb%) during the next frame blanking interval.<br>This will reset the colour map to the default colours |
| MAP<br><br>MAP( n ) = rgb%<br><br>MAP SET<br><br>MAP RESET | <u>RP2350 PICOMITE VERSIONS WITH A BUFFERED DISPLAY DRIVER ONLY</u><br>The MAP commands allow the programmer to set the colours used in the 8-bit RGB332 colour mode. Each value in the 8-bit colour pallet can be set to an independent 24-bit colour (ie, RGB888 format). See the MAP function for more information<br>This will assign the 24-bit colour 'rgb% to all pixels 8-bit colour value of 'n'. The change is activated after the MAP SET command.<br>This will cause MMBasic to update the colour map (set using MAP(n)=rgb%) during the next frame blanking interval.<br>This will reset the colour map to the default colours |

VGA VERSIONS ONLY

**MAP**

The MAP commands allow the programmer to select the colours used in 4-bit colour modes. Each value in the 4-bit colour pallet can be set to one of the 16 available colours . See the MAP function for more information.

**MAP( n ) = rgb%**

This will assign the 24-bit colour 'rgb% to all pixels with the 4-bit colour value of 'n'. The RGB value is converted to one of the available 16 VGA RGB121 colours as set by the resistor network. The change is activated after the MAP SET command.

**MAP MAXIMITE**

This will set the colour map to the colours implemented in the original Colour Maximite.

**MAP SET**

This will cause MMBasic to update the colour map (set using MAP(n)=rgb%) during the next frame blanking interval.

**MAP RESET**

This will reset the colour map to the default colours which in 4-bit mode are:

| 'n' | Colour | Value |
|---|---|---|
| 15 | WHITE | RGB(255, 255, 255) |
| 14 | YELLOW | RGB(255, 255, 0) |
| 13 | LILAC | RGB(255, 128, 255) |
| 12 | BROWN | RGB(255, 128, 0) |
| 11 | FUCHSIA | RGB(255, 64, 255) |
| 10 | RUST | RGB(255, 64, 0) |
| 9 | MAGENTA | RGB(255, 0, 255) |
| 8 | RED | RGB(255, 0, 0) |
| 7 | CYAN | RGB(0, 255, 255) |
| 6 | GREEN | RGB(0, 255, 0) |
| 5 | CERULEAN | RGB(0, 128, 255) |
| 4 | MIDGREEN | RGB(0, 128, 0) |
| 3 | COBALT | RGB(0, 64, 255) |
| 2 | MYRTLE | RGB(0, 64, 0) |
| 1 | BLUE | RGB(0, 0, 255) |
| 0 | BLACK | RGB(0, 0, 0) |

**MATH**

The math command performs many simple mathematical calculations that can be programmed in BASIC but there are speed advantages to coding looping structures in the firmware and there is the advantage that once debugged they are there for everyone without re-inventing the wheel. Note: 2 dimensional maths matrices are always specified DIM matrix(n_columns, n_rows) and of course the dimensions respect OPTION BASE. Quaternions are stored as a 5 element array w, x, y, z, magnitude.

**MATH RANDOMIZE [n]**

Seeds the Mersenne Twister algorithm.
If n is not specified the seed is the time in microseconds since boot
The Mersenne Twister algorithm gives a much better random number than the C-library inbuilt function. NB: the RP2350 includes a H/W random number generator.

**Simple array arithmetic**

**MATH SET nbr, array()**

See ARRAY SET

**MATH SCALE in(), scale ,out()**

This scales the matrix in() by the scalar scale and puts the answer in out(). Works for arrays of any dimensionality of both integer and float and can

| | |
|---|---|
| | convert between. Setting b to 1 is optimised and is the fastest way of copying an entire array. |
| MATH ADD in(), num ,out() | See ARRAY ADD |
| MATH INTERPOLATE in1(), in2(), ratio, out() | This command implements the following equation on every array element:<br>out = (in2 - in1) * ratio + in1<br>Arrays can have any number of dimensions and must be distinct and have the same number of total elements. The command works with both integer and floating point arrays in any mixture |
| MATH WINDOW in(), minout, maxout, out() [,minin!, maxin!] | This command takes the "in" array and scales it between "minout" and "maxout" returning the answer in "out". Optionally, it can also return the minimum and maximum floating point values found in the original data ("minin!" and "maxin!"). Note: "minout" can be greater than "maxout" and in this case the data will be both scaled and inverted.<br>This command can massively simplify scaling data for plotting etc.<br>Example:<br>DIM IN(2)=(1,2,3)<br>DIM OUT(2)<br>MATH WINDOW IN(),7,3,OUT(),LOW,HIGH<br>Will return OUT(0)=7, OUT(1)=5,OUT(2)=3,LOW=1,HIGH=3 |
| MATH CLAMP in(), lo, hi, out() | Limits every element of 'in()' to the range 'lo' to 'hi' and puts the result in 'out()'. Anything below 'lo' becomes 'lo', anything above 'hi' becomes 'hi' and everything else is copied unchanged. The two arrays must have the same number of elements but can be integer or floating point in any combination; when the output array is an integer array the limits are rounded to whole numbers. An error is reported if 'lo' is greater than 'hi'.<br>Note that this is not the same as MATH WINDOW, which rescales the data to fit the range given rather than clipping it.<br>Example:<br>DIM v(3) = (-8,2,40,11)<br>DIM w(3)<br>MATH CLAMP v(),0,10,w()<br>Will return w(0)=0, w(1)=2, w(2)=10, w(3)=10 |
| MATH SLICE sourcearray(), [d1] [,d2] [,d3] [,d4] [,d5] , destinationarray() | See ARRAY SLICE |
| MATH INSERT targetarray(), [d1] [,d2] [,d3] [,d4] [,d5] , sourcearray() | See ARRAY INSERT |
| MATH POWER inarray(), power, outarray() | Raises each element in 'inarray()' to the 'power' defined and puts the output in 'outarray()' |
| MATH SHIFT inarray%(), nbr, outarray%() [,U] | This command does a bit shift on all elements of inarray%() and places the result in outarray%() (may be the same as inarray%()). nbr can be between -63 and 63. Positive numbers are a left shift (multiply by power of 2). Negative number are a right shift. The optional parameter ,U will force an unsigned shift. |
| **Matrix arithmetic** | |

| | |
|---|---|
| MATH M_INVERSE array!(), inversearray!() | This returns the inverse of array!() in inversearray!(). The array must be square and you will get an error if the array cannot be inverted (determinant=0). array!() and inversearray!() cannot be the same. |
| MATH M_PRINT array() | Quick mechanism to print a 2D matrix one row per line. |
| MATH M_TRANSPOSE in(), out() | Transpose matrix in() and put the answer in matrix out(), both arrays must be 2D but need not be square. If not square then the arrays must be dimensioned in(m,n) out(n,m) |
| MATH M_MULT in1(), in2(), out() | Multiply the arrays in1() and in2() and put the answer in out()c. All arrays must be 2D but need not be square. If not square then the arrays must be dimensioned in1(m,n) in2(p,m) ,out(p,n) |
| **Vector arithmetic** | |
| MATH V_PRINT array() [,hex] | Quick mechanism to print a small array on a single line. 'hex' will print in hex. |
| MATH V_NORMALISE inV(), outV() | Converts a vector inV() to unit scale and puts the answer in outV()<br>(sqr(x*x + y*y +....)=1<br>There is no limit on number of elements in the vector |
| MATH V_MULT matrix(), inV(), outV() | Multiplies matrix() and vector inV() returning vector outV(). The vectors and the 2D matrix can be any size but must have the same cardinality. |
| MATH V_CROSS inV1(), inV2(), outV() | Calculates the cross product of two three element vectors inV1() and inV2() and puts the answer in outV() |
| MATH V_ROTATE x, y, a, xin(), yin(), xout(), yout() | This command rotates the coordinate pairs in "xin()" and "yin()" around the centre point defined by "x" and "y" by the angle "a" and puts the results in "xout()" and "yout()". NB: the input and output arrays can be the same and the rotation angle is, by default, in radians but this can be changed using the OPTION ANGLE command. |
| **Quaternion arithmetic** | |
| MATH Q_INVERT inQ(), outQ() | Invert the quaternion in inQ() and put the answer in outQ() |
| MATH Q_VECTOR x, y, z, outVQ() | Converts a vector specified by x , y, and z to a normalised quaternion vector outVQ() with the original magnitude stored |
| MATH Q_CREATE theta, x, y, z, outRQ() | Generates a normalised rotation quaternion outRQ() to rotate quaternion vectors around axis x,y,z by an angle of theta. Theta is specified in radians. |
| MATH Q_EULER yaw, pitch, roll, outRQ() | Generates a normalised rotation quaternion outRQ() to rotate quaternion vectors as defined by the yaw, pitch and roll angles<br>With the vector in front of the "viewer" yaw is looking from the top of the vector and rotates clockwise, pitch rotates the top away from the camera and roll rotates around the z-axis clockwise.<br>The yaw, pitch and roll angles default to radians but respect the setting of OPTION ANGLE |
| MATH Q_MULT inQ1(), inQ2(), outQ() | Multiplies two quaternions inQ1() and inQ2() and puts the answer in outQ() |

| | |
|---|---|
| MATH Q_ROTATE , RQ(), inVQ(), outVQ() | Rotates the source quaternion vector inVQ() by the rotate quaternion RQ() and puts the answer in outVQ() |
| MATH C_ADD array1(), array2(), array3()<br>MATH C_SUB array1(), array2(), array3()<br>MATH C_MUL array1(), array2(), array3()<br>MATH C_DIV array1(), array2(), array3()<br>MATH C_AND array1(), array2(), array3()<br>MATH C_OR array1(), array2(), array3()<br>MATH C_XOR array1(), array2(), array3() | These commands do cell by cell operations on arrays.<br>array1 and array2 are the parameters and array3 is the output. All arrays must be the same size and the same type (float or integer).<br>There are no restrictions on the number of dimensions and no restrictions on using the same array twice or even three times in the parameters.<br>eg, MATH C_MUL a%(),a%(),a%()<br>will square all the values in the array a%() |
| MATH FFT signalarray!(), FFTarray!() | Performs a fast fourier transform of the data in “signalarray!”. "signalarray" must be floating point and the size must be a power of 2 (eg, s(1023) assuming OPTION BASE is zero)<br>"FFTarray" must be floating point and have dimension 2*N where N is the same as the signal array (eg, f(1,1023) assuming OPTION BASE is zero)<br>The command will return the FFT as complex numbers with the real part in f(0,n) and the imaginary part in f(1,n) |
| MATH FFT INVERSE FFTarray!(), signalarray!() | Performs an inverse fast fourier transform of the data in “FFTarray!”. "FFTarray" must be floating point and have dimension 2*N where N must be a power of 2 (eg, f(1,1023) assuming OPTION BASE is zero) with the real part in f(0,n) and the imaginary part in f(1,n).<br>"signalarray" must be floating point and the single dimension must be the same as the FFT array.<br>The command will return the real part of the inverse transform in "signalarray". |
| MATH FFT MAGNITUDE signalarray!(),magnitudearray!() | Generates magnitudes for frequencies for the data in “signalarray!”<br>"signalarray" must be floating point and the size must be a power of 2 (eg, s(1023) assuming OPTION BASE is zero). "magnitudearray" must be floating point and the size must be the same as the signal array<br>The command will return the magnitude of the signal at various frequencies according to the formula:<br>frequency at array position N = N * sample_frequency / number_of_samples |
| MATH FFT PHASE signalarray!(), phasearray!() | Generates phases for frequencies for the data in “signalarray!”.<br>"signalarray" must be floating point and the size must be a power of 2 (eg, s(1023) assuming OPTION BASE is zero). "phasearray" must be floating point and the size must be the same as the signal array<br>The command will return the phase angle of the signal at various frequencies according to the formula above. |
| MATH SENSORFUSION type ax, ay, az, gx, gy, gz, [mx, my, mz, ] pitch, roll, yaw [,p1] [,p2] | Type can be MAHONY or MADGWICK<br>Ax, ay, and az are the accelerations in the three directions and should be specified in units of standard gravitational acceleration.<br>Gx, gy, and gz are the instantaneous values of rotational speed which should be specified in radians per second.<br>Mx, my, and mz are the magnetic fields in the three directions and should be specified in nano-Tesla (nT). They can be omitted in which case the fusion will may slow drift but allows sensors like the MPU-6050 to be used.<br>Care must be taken to ensure that the x, y and z components are consistent between the three inputs. So , for example, using the MPU-9250 the correct input will be ax, ay,az, gx, gy, gz, my, mx, -mz based on the reading from the sensor. |

| | |
|---|---|
| | Pitch, roll and yaw should be floating point variables and will contain the outputs from the sensor fusion.<br>The SENSORFUSION routine will automatically measure the time between consecutive calls and will use this in its internal calculations.<br>The Madgwick algorithm takes an optional parameter p1. This is used as beta in the calculation. It defaults to 0.5 if not specified<br>The Mahony algorithm takes two optional parameters p1, and p2. These are used as Kp and Ki in the calculation. If not specified these default to 10.0 and 0.0 respectively.<br>A fully worked example of using the code is given on the BackShed forum at: https://www.thebackshed.com/forum/ViewTopic.php?TID=13459&PID=166962#166962 |
| MATH SINC x_in(), y_in(), n , m, window, freq, x_out(), y_out()<br><br>MATH SINC x_in(), y_in(), n, window, freq, x_out(), y_out() | Apply a windowed sinc filter to smooth or interpolate coordinate data.<br>A sinc filter is an ideal low-pass filter that removes high-frequency noise while preserving the signal shape. It is particularly good for resampling (interpolation).<br>Parameters:<br>x_in(), y_in() — Input coordinate arrays (float)<br>n — Number of input points<br>m — Number of output points (optional). If omitted or m=n: Smoothing mode (output size n). If m!=n: Interpolation/Resampling mode (output size m)<br>window — Filter kernel size (must be odd; even values are incremented). Typical values: 15 to 101. Larger values give a sharper cutoff but are slower.<br>freq — Normalized cutoff frequency (0.0 < freq <= 0.5). 0.5 is the Nyquist frequency (no filtering). Typical values: 0.1 (heavy smoothing) to 0.4 (light smoothing).<br>x_out(), y_out() — Output coordinate arrays (must be large enough to hold output) |
| MATH PID INIT channel, pid_params!(), callback | This command sets up a PID controller that can work automatically in the background. Up to 8 PID controllers can run simultaneously (channels 1 to 8)<br>'callback' is a MMbasic subroutine which is called at the rate defined by the sample time. See the MATH(PID …) function for details of what should be included in the subroutine.<br>The pid_params!() array must be dimensioned for all the listed elements, including the controller memory parameters (DIM pid_params!(13)) and be initialised with the required settings.<br>'pid_params!()' must be global (or STATIC), not LOCAL, as it is used and updated in the background.<br>PID configuration<br>Element 0 = Kp<br>Element 1 = Ki<br>Element 2 = Kd<br>Element 3 = tau ' Derivative low-pass filter time constant<br>Element 4 = limMin 'Output limits<br>Element 5 = limMax<br>Element 6 = limMinInt 'Integrator limits<br>Element 7 = limMaxInt<br>Element 8 = T 'Sample time (in seconds)<br>Controller "memory" |

| | |
|---|---|
| MATH PID START channel<br><br>MATH PID STOP channel | Element 9 = integrator<br>Element 10 = prevError<br>Element 11 = differentiator<br>Element 12 = prevMeasurement<br>Element 13 = out<br><br>Starts a previously initialised PID controller on the channel specified<br><br>Stops a previously initialised PID controller on the channel specified and deletes the internal data structures<br>See https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=17263<br>For an example of setting up and running a PID controller |
| MATH AES128 ENCRYPT/DECRYPT CBC/ECB/CTR key$/key(), in$/in(), out$/out() [,iv$/iv()] | This command encrypts or decrypts the data in 'in' and puts the answer in 'out' using the AES128 encryption method specified<br>The parameters can each be either a string, integer array, or float array with any mix possible<br>The key must be 16 elements long (16*8=128bits), in and out must be a multiple of 16 elements long. In the case of out being specified as a string (e.g. out$), the string variable must exist and should be set to empty (DIM out$="")<br>The maximum number of elements in 'in' and 'out' is limited by memory when defined as arrays. Strings for encrypting are limited to 240bytes (EBR) and 224bytes (CTR and CBC).<br>For CBC and CTR encryption you can optionally specify an initialisation vector 'iv'. 'iv' must be 16 elements long (16*8=128bits). If an initialisation vector is not specified encryption will generate a random initialisation vector.<br>In both cases the output is prepended with the IV.<br>For CBC and CTR, decryption requires that the first 16 elements of the input are the initialisation vector.<br>In the case where you want to transmit a message without IV you should remove the IV before sending the message using standard MMBasic manipulations. In this case the recipient must know the IV as well as the key and create a complete message before using DECRYPT by prepending the IV to the incoming message. |
| MEMORY | List the amount of memory currently in use. For example:<br>`Program:`<br>`  0K ( 0%) Program (0 lines)`<br>`180K (100%) Free`<br><br>`Saved Variables:`<br>`  16K (100%) Free`<br><br>`RAM:`<br>`  0K ( 0%) 0 Variables`<br>`  0K ( 0%) General`<br>`228K (100%) Free`<br><br>Notes:<br>• Memory usage is rounded to the nearest 1K byte.<br>• General memory (RAM) is used by arrays, strings, serial I/O buffers, etc. |

| | |
|---|---|
| MEMORY SET address, byte, numberofbytes<br><br>MEMORY SET BYTE address, byte, numberofbytes<br><br>MEMORY SET SHORT address, short, numberofshorts<br><br>MEMORY SET WORD address, word, numberofwords<br><br>MEMORY SET INTEGER address, integervalue ,numberofintegers [,increment]<br><br>MEMORY SET FLOAT address, floatingvalue ,numberofloats [,increment] | This command will set a region of memory to a value.<br>BYTE = One byte per memory address.<br>SHORT = Two bytes per memory address.<br>WORD = Four bytes per memory address.<br>FLOAT = Eight bytes per memory address.<br>'increment' is optional and controls the increment of the 'address' pointer as the operation is executed. For example, if increment=3 then only every third element of the target is set. The default is 1. |
| MEMORY COPY sourceaddress, destinationaddress, numberofbytes [,sourceincrement][,destination increment]<br><br>MEMORY COPY INTEGER sourceaddress, destinationaddress, numberofintegers [,sourceincrement][,destination increment]<br><br>MEMORY COPY FLOAT sourceaddress, destinationaddress, numberoffloats [,sourceincrement][,destination increment] | This command will copy one region of memory to another.<br>COPY INTEGER and FLOAT will copy eight bytes per operation.<br>'sourceincrement' is optional and controls the increment of the 'sourceaddress' pointer as the operation is executed. For example, if sourceincrement=3 then only every third element of the source will be copied. The default is 1.<br>'destinationincrement' is similar and operates on the 'destinationaddress' pointer. |
| MEMORY PRINT #]fnbr , nbr, address%/array()<br><br>MEMORY INPUT [#]fnbr , nbr, address%/array() | These commands save or read 'nbr' of data bytes from or to memory from or to an open disk file.<br>The memory to be saved can be specified as an integer array in which case the nbr of bytes to be saved or read is checked against the array size. Alternatively, a memory address can be used in which case no checking can take place and user errors could result in a crash of the firmware.. |
| MEMORY PACK source%()/sourceaddress%, dest%()/destaddress%, number, size<br><br>MEMORY UNPACK source%()/sourceaddress%, | Memory pack and unpack allow integer values from one array to be compressed into another or uncompressed from one to the other.<br>The two arrays are always normal integer arrays but the packed array can have 2, 4, 8, 16 or 64 values "packed into them. Thus a single integer array element could store 2 off 32-bit words, 4 off 16 bit values, 8 bytes, 16 nibbles, or 64 booleans (bits). |

| | |
|---|---|
| dest%()/destaddress%, number, size | "number specifies the number of values to be packed or unpacked and "size" specifies the number of bits (1,4,8,16,or 32)<br>Alternatively, memory address(es) can be used in which case no checking can take place and user errors could result in a crash of the firmware. |
| MKDIR dir$ | Make, or create, the directory 'dir$' on the default Flash Filesystem or SD Card. |
| MID$( str$, start, num) = str2$ | The 'num' characters in 'str$', beginning at position 'start', are replaced by the characters in 'str2$'. 'num' can be set be 0 i.e. insert the new string at the position specified. If num isn't specified it defaults to the length of 'str2$' |
| MODE 1<br>or<br>MODE 2<br>Or<br>MODE 3 (RP2350 only) | <u>VGA VERSIONS ONLY</u><br>VGA video supports a number of resolutions (see OPTION RESOLUTION). This command will select the mode 'n' depending on the resolution:<br><u>OPTION RESOLUTION 640 x 480</u><br>MODE 1 640 x 480 x 2 colours (monochrome). Default at startup. Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB121. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and do not use user memory but both can be used for creating images and copying to the display screen (N)<br>MODE 2 320 x 240 x 16 colours. RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available. The hardware is limited to the 16 colours defined by the resistor network<br>MODE 3 640 x 480 x 16 colours. RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available. The hardware is limited to the 16 colours defined by the resistor network<br><u>OPTION RESOLUTION 720 x 400</u><br>MODE 1 720 x 400 x 2 colours (monochrome). Default at startup. Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB121. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and do not use user memory but both |

| | |
|---|---|
| | can be used for creating images and copying to the display screen (N)<br>MODE 2 360 x 200 x 16 colours.<br>RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available In the case of VGA, the hardware is limited to the 16 colours defined by the resistor network<br>MODE 3 720 x 400 x 16 colours.<br>RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available. In the case of VGA, the hardware is limited to the 16 colours defined by the resistor network<br><u>OPTION RESOLUTION 800 x 600 (RP2350 only)</u><br>MODE 1 800 x 600 x 2 colours (monochrome). Default at startup.<br>Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB121. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and do not use user memory but both can be used for creating images and copying to the display screen (N)<br>MODE 2 400 x 300 x 16 colours.<br>RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available The hardware is limited to the 16 colours defined by the resistor network<br>MODE 3 800 x 600 x 16 colours.<br>RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the |

| | |
|---|---|
| | main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available, The hardware is limited to the 16 colours defined by the resistor network<br>OPTION RESOLUTION 848 x 480 (RP2350 only)<br>MODE 1 848 x 480 x 2 colours (monochrome). Default at startup. Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB121. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and do not use user memory but both can be used for creating images and copying to the display screen (N)<br>MODE 2 424 x 240 x 16 colours. RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available The hardware is limited to the 16 colours defined by the resistor network<br>MODE 3 848 x 480 x 16 colours. RGB121 format (i.e. 1 bit for red, 2 bits for green, and 1 bit for blue). A framebuffer (F) can be created. This has no impact on the display and does not use user memory but can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This also does not use user memory. any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 16 available, The hardware is limited to the 16 colours defined by the resistor network |
| MODE n | HDMI VERSIONS ONLY<br>HDMI video supports a number of resolutions (see OPTION RESOLUTION). This command will select the mode 'n' depending on the resolution:<br>OPTION RESOLUTION 640 x 480<br>MODE 1 640 x 480 x 2-colours (monochrome). Default at startup. Use the TILE command as normal. Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB555. A framebuffer (F) and a layer buffer (L) can be created. These can be used for creating images and copying to the display screen (N)<br>MODE 2 320 x 240 x 16 colours. A framebuffer (F) can be created. This can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. Any pixels written to the layer |

buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours. A second layer buffer (T) can also be created; it is applied on top of the first layer.

MODE 3 640 x 480 x 16 colours.
Colour mapping to RGB555 palette. A framebuffer (F) can be created. It can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. Any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through.

MODE 4 320 x 240 x 32768 colours.
This is full RGB555 allowing good quality colour images to be displayed. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and can be used for creating images and copying to the display screen (N). Only one can be created

MODE 5 320 x 240 x 256 colours.
A framebuffer (F) can be created. This has no impact on the display. It can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This does not use user memory. Any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-255: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 256 available. Each of the 256 colours can be mapped to any RGB555 colour. A second layer buffer (T) can also be created; it is applied on top of the first layer.

<u>OPTION RESOLUTION 720 x 400</u>

MODE 1 720 x 400 x 2-colours (monochrome). Default at startup. Use the TILE command as normal. Tiles width is fixed at 8 pixels. Tile height defaults to 12 pixels but can be from 8 to MM.VRES. Tiles colours are specified using the standard RGB888 notation. This is converted to RGB555. A framebuffer (F) and a layer buffer (L) can be created. These can be used for creating images and copying to the display screen (N)

MODE 2 360 x 200 x 16 colours.
A framebuffer (F) can be created. This can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. Any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours. A second layer buffer (T) can also be created; it is applied on top of the first layer.

MODE 3 720 x 400 x 16 colours.
Colour mapping to RGB555 palette. A framebuffer (F) can be

created. It can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. Any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-15: defaults to 0) which does not show allowing the main display buffer to show through.

MODE 4 360 x 200 x 32768 colours.
This is full RGB555 allowing good quality colour images to be displayed. A framebuffer (F) and a layer buffer (L) can be created. These have no impact on the display and can be used for creating images and copying to the display screen (N). Only one can be created

MODE 5 360 x 200 x 256 colours.
A framebuffer (F) can be created. This has no impact on the display. It can be used for creating images and copying to the display screen (N). In addition a layer buffer can be created. This does not use user memory. Any pixels written to the layer buffer will automatically appear on the display sitting on top of whatever may be in the main display buffer. A colour can be specified (0-255: defaults to 0) which does not show allowing the main display buffer to show through. Map functionality is available to override the default colours of the 256 available. Each of the 256 colours can be mapped to any RGB555 colour. A second layer buffer (T) can also be created; it is applied on top of the first layer.

OPTION RESOLUTION 800 x 600 (NB: reduces heap memory available)

MODE 1 800 x 600 x 2-colours with RGB332 tiles (use the TILE command as normal)

MODE 2 400 x 300 x 16colours with optional layer and colour mapping to RGB332 palette

MODE 3 800 x 400 x 16 colours with optional layer and colour mapping to RGB332 palette

MODE 5 400 x 300 x 256 colours with optional layer (no memory usage)

OPTION RESOLUTION 848 x 480 (NB: reduces heap memory available)

MODE 1 848 x 480 x 2-colours with RGB332 tiles (use the TILE command as normal)

MODE 2 424 x 240 x 16colours with optional layer and colour mapping to RGB332 palette

MODE 3 848 x 480 x 16 colours with optional layer and colour mapping to RGB332 palette

MODE 5 424 x 240 x 256 colours with optional layer (no memory usage)

OPTION RESOLUTION 1280 x 720

MODE 1 1280 x 720 x 2-colours with RGB332 tiles (use the TILE command as normal)

MODE 2 320 x 180 x 16colours with 2 optional layers and colour mapping to RGB332 palette

MODE 3 640 x 360 x 16 colours with optional layer and colour mapping to RGB332 palette

MODE 5 320 x 180 x 256 colours with 2 optional layers (no memory usage)

| | |
|---|---|
| | OPTION RESOLUTION 1024 x 768<br>MODE 1 1024 x 768 x 2 colours with RGB332 tiles (use the TILE command as normal)<br>MODE 2 256 x 192 x 16 colours with 2 optional layers and colour mapping to RGB332 palette<br>MODE 3 512 x 384 x 16 colours<br>MODE 5 256 x 192 x 256 colours with 2 optional layers, and colour mapping to RGB332 palette<br><br>OPTION RESOLUTION 1024 x 600<br>MODE 1 1024 x 600 x 2 colours with RGB332 tiles (use the TILE command as normal)<br>MODE 2 256 x 150 x 16 colours with 2 optional layers and colour mapping to RGB332 palette<br>MODE 3 512 x 300 x 16 colours with optional layer and colour mapping to RGB332 palette<br>MODE 5 256 x 150 x 256 colours with 2 optional layers and colour mapping to RGB332 palette<br><br>OPTION RESOLUTION 800 x 480 (NB: reduces heap memory available)<br>MODE 1 800 x 480 x 2 colours with RGB332 tiles (use the TILE command as normal)<br>MODE 2 400 x 240 x 16 colours with optional layer and colour mapping to RGB332 palette<br>MODE 3 800 x 480 x 16 colours with optional layer and colour mapping to RGB332 palette<br>MODE 5 400 x 240 x 256 colours with optional layer and colour mapping to RGB332 palette<br>PicoMiteHDMIWEB: to save the memory needed by its network stack this version always uses 8-bit (RGB332) colour and has its own reduced set of modes. MODE 4 is not available in any resolution. The six resolutions and the modes each supports are:<br>640x480 - MODE 1 (640x480x2 colours with RGB332 tiles), MODE 2 (320x240x16 colours) and MODE 5 (320x240x256 colours). MODE 3 is not available. The second layer buffer (T) is applied in MODE 2 but not in MODE 5.<br>720x400 - MODE 1 (720x400x2 colours with RGB332 tiles), MODE 2 (360x200x16 colours) and MODE 5 (360x200x256 colours). MODE 3 is not available. The second layer buffer (T) is applied in MODE 2 but not in MODE 5.<br>800x600, 848x480 and 800x480 - MODE 1 and MODE 2 only, as listed above for these resolutions. The reduced framebuffer cannot hold MODE 3 or MODE 5.<br>1024x600 - MODE 1, MODE 2, MODE 3 and MODE 5, exactly as listed above. |
| MODE 400<br>or<br>MODE 800 | RP2350 PICOMITE VERSIONS ONLY<br>Swaps between 400x240 and 800x480 modes for SSD1963 buffered drivers |
| MOUSE | For all variants of the command. In the case of USB firmware ‘channel’ is the USB port that the mouse is connected to (1-4). See MM.INFO(USB n) for more information. For PS2 firmware ‘channel’ is fixed at the value 2 |

| | |
|---|---|
| MOUSE INTERRUPT ENABLE channel, int | 'int' is a user defined subroutine that will be called when the left mouse button is pressed. |
| MOUSE INTERRUPT DISABLE channel | Disables an interrupt on the left mouse button |
| MOUSE SET channel, x-coord, y-coord [, wheel-count] | Sets the current position that will be returned by the mouse x, y and optionally wheel positions |
| MOUSE OPEN channel, CLKpin, DATApin | NON-USB VERSIONS ONLY<br>Opens a connection to a PS2 mouse connected to the two specified pins. This command can be used in a program to configure the mouse while the program is running as against OPTION MOUSE which permanently configures the mouse.<br>Channel is included for compatibility with USB mouse functionality and must be set to 2. If a mouse is not connected you will get an error and the command can be called again once the mouse is connected |
| MOUSE CLOSE channel | Closes access to the mouse and restores the pins to normal use. The command will error if OPTION MOUSE has been set. |
| NEW | Clears the program memory and all variables including saved variables. |
| NEXT [counter-variable] [, counter-variable], etc | NEXT comes at the end of a FOR-NEXT loop; see FOR.<br>The 'counter-variable' specifies exactly which loop is being operated on. If no 'counter-variable' is specified the NEXT will default to the innermost loop. It is also possible to specify multiple counter-variables as in:<br>`NEXT x, y, z` |
| ON ERROR ABORT<br>or<br>ON ERROR IGNORE<br>or<br>ON ERROR SKIP [nn]<br>or<br>ON ERROR CLEAR<br>Or<br>ON ERROR RESTART | This controls the action taken if an error occurs while running a program and applies to all errors discovered by MMBasic including syntax errors, wrong data, missing hardware, etc.<br>ON ERROR ABORT will cause MMBasic to display an error message, abort the program and return to the command prompt. This is the normal behaviour and is the default when a program starts running.<br>ON ERROR RESTART will cause MMbasic to execute a Hardware Reset and, of course, if you have OPTION AUTORUN set the program will then restart cleanly<br>ON ERROR IGNORE will cause any error to be ignored.<br>ON ERROR SKIP will ignore an error in a number of commands (specified by the number 'nn') executed following this command. 'nn' is optional, the default if not specified is one. After the number of commands has completed (with an error or not) the behaviour of MMBasic will revert to ON ERROR ABORT.<br>If an error occurs and is ignored/skipped the read only variable MM.ERRNO will be set to non zero and MM.ERRMSG$ will be set to the error message that would normally be generated. These are reset to zero and an empty string by ON ERROR CLEAR. They are also cleared when the program is run and when ON ERROR IGNORE and ON ERROR SKIP are used.<br>ON ERROR IGNORE can make it very difficult to debug a program so it is strongly recommended that only ON ERROR SKIP be used. |
| ON KEY target<br>or<br>ON KEY ASCIIcode, target | The first version of the command sets an interrupt which will call 'target' user defined subroutine whenever there is one or more characters waiting in the serial console input buffer. |

| | |
|---|---|
| | Note that all characters waiting in the input buffer should be read in the interrupt subroutine otherwise another interrupt will be automatically generated as soon as the program returns from the interrupt.<br>The second version allows you to associate an interrupt routine with a specific key press. This operates at a low level for the serial console and if activated the key does not get put into the input buffer but merely triggers the interrupt. It uses a separate interrupt from the simple ON KEY command so can be used at the same time if required.<br>In both variants, to disable the interrupt use numeric zero for the target, i.e.:<br>ON KEY 0. or ON KEY ASCIIcode, 0 |
| ON PS2 target | This triggers an interrupt whenever the PicoMite firmware sees a message from the PS2 interface.<br>Use MM.info(PS2) to report the raw message received. This allows the programmer to trap both keypress and release.<br>See https://wiki.osdev.org/PS/2_Keyboard for the scan codes (Set 2). |
| ONEWIRE RESET pin<br>or<br>ONEWIRE WRITE pin, flag, length, data [, data…]<br>or<br>ONEWIRE READ pin, flag, length, data [, data…] | Commands for communicating with 1-Wire devices.<br>ONEWIRE RESET will reset the 1-Wire bus<br>ONEWIRE WRITE will send a number of bytes<br>ONEWIRE READ will read a number of bytes<br>'pin' is the I/O pin (located in the rear connector) to use. It can be any pin capable of digital I/O.<br>'flag' is a combination of the following options:<br>1 - Send reset before command<br>2 - Send reset after command<br>4 - Only send/recv a bit instead of a byte of data<br>8 - Invoke a strong pullup after the command (the pin will be set high and open drain disabled)<br>'length' is the length of data to send or receive<br>'data' is the data to send or variable to receive. The data to send (WRITE) or the destination for received data (READ) may be supplied in any one of three forms:<br>• **A list of individual items.** The number of items must equal the length parameter. For WRITE each item is a constant, variable or expression; for READ each item is a numeric variable to receive one value. Array elements such as a(3) are allowed.<br>• **An array**, specified with empty brackets, e.g. array(). It may be a floating-point or integer array and must be large enough to supply (WRITE) or hold (READ) the specified number of elements, which are taken from / stored into consecutive elements starting at the first.<br>• **A string variable.** For WRITE the first length characters are sent (the string must be at least that long); for READ the received bytes are stored as sequential characters and the string is set to that length.<br>See also *Appendix C*. |
| ONESHOT triggerPin, sense, outputPin, prePulseDelay, pulseWidth [, quiescentPeriod] | Will configure a background one-shot pulse generator.<br>When the selected trigger edge is detected on 'triggerPin', MMBasic will stars a delay of 'prePulseDelay' microseconds, then toggle 'outputPin'. After 'pulseWidth' microseconds it then toggles 'outputPin' again. This creates a pulse in the opposite sense to the output pin's idle state.<br>An optional 'quiescentPeriod' (microseconds) can be supplied. If present, ONESHOT will ignore retriggers for that period after the pulse completes. |

| | |
|---|---|
| | 'sense' selects the trigger edge transition and can be POSITIVE (also RISING) or NEGATIVE (also FALLING). Triggering is edge based (transition), not level-state based.<br>Timing is interrupt driven and non-blocking. Delay and pulse width are specified in microseconds. |
| ONESHOT DISABLE | ONESHOT DISABLE stops background operation and cancels pending timing events. |
| ONESHOT R triggerPin, sense, outputPin, prePulseDelay, pulseWidth [, quiescentPeriod] | ONESHOT R uses the same parameters as ONESHOT but in this case is retriggerable. In retriggerable mode:<br>If a trigger arrives during pre-pulse delay, the pre-pulse delay is restarted from that trigger. Triggers are not queued. During pre-pulse delay, each new valid trigger replaces the previous pending delay and restarts timing. This means the pulse starts only after a quiet gap at least as long as `prePulseDelay` following the most recent valid trigger.<br>If triggers continue to arrive more frequently than `prePulseDelay`, the output transition can be deferred indefinitely until triggers stop.<br>If a trigger arrives during the pulse, the pulse width timer is restarted from that trigger (the output stays in its active pulse state).<br>If a trigger arrives during quiescent period, a new cycle is started (pre-pulse delay then pulse). |
| OPEN fname$ FOR mode AS [#]fnbr | Opens a file for reading or writing.<br>‘fname’ is the filename with an optional extension separated by a dot (.). Long file names with upper and lower case characters are supported. The file systems on both the SD Card and the flash drives are NOT case sensitive.<br>A directory path can be specified with the backslash as directory separators. The parent of the current directory can be specified by using a directory name of “..” (two dots) and the current directory with “.” (a single dot).<br>For example: OPEN ".\dir1\dir2\filename.txt" FOR INPUT AS #1<br>‘mode’ is INPUT, OUTPUT, APPEND or RANDOM.<br>INPUT will open the file for reading and throw an error if the file does not exist. OUTPUT will open the file for writing and will automatically overwrite any existing file with the same name.<br>APPEND will also open the file for writing but it will not overwrite an existing file; instead any writes will be appended to the end of the file. If there is no existing file the APPEND mode will act the same as the OUTPUT mode (i.e. the file is created then opened for writing).<br>RANDOM will open the file for both read and write and will allow random access using the SEEK command. When opened the read/write pointer is positioned at the end of the file. If the file does not exist , it will be created.<br>‘fnbr’ is the file number (1 to 10). The # is optional. Up to 10 files can be open simultaneously and can be on either or both the A: and B: drives. The INPUT, LINE INPUT, PRINT, WRITE and CLOSE commands as well as the EOF() and INPUT$() functions all use ‘fnbr’ to identify the file being operated on.<br>See also ON ERROR and MM.ERRNO for error handling. |

| | |
|---|---|
| OPEN comspec$ AS [#]fnbr | Will open a serial communications port for reading and writing. Two ports are available (COM1: and COM2:) and both can be open simultaneously; the versions with USB host support add COM3: to COM6: for USB serial devices and the Bluetooth host versions COM7: for a Bluetooth serial device. For a full description with examples see *Appendix A*.<br>Using 'fnbr' the port can be written to and read from using any command or function that uses a file number. |
| OPEN comspec$ AS GPS [,timezone_offset] [,monitor] | Will open a serial communications port for reading from a GPS receiver. The data can then be retrieved using the GPS() function.<br>This is described in detail in the file *Option_GPS_User_Manual.pdf* included in the firmware download zip file. |
| OPTION | See the section *Options* earlier in this manual. |
| PAUSE delay | Halt execution of the running program for 'delay' ms. This can be a fraction. For example, 0.2 is equal to 200 µs. The maximum delay is 2147483647 ms (about 24 days).<br>Note that interrupts will be recognised and processed during a pause. |
| PIN( pin ) = value | For a 'pin' configured as digital output this will set the output to low ('value' is zero) or high ('value' non-zero). You can set an output high or low before it is configured as an output and that setting will be the default output when the SETPIN command takes effect.<br>See the function PIN() for reading from a pin and the command SETPIN for configuring it. Refer to the chapter *Using the I/O pins* for a general description of the PicoMite firmware's input/output capabilities. |
| PIO | The processors chip in the Raspberry Pi Pico with the RP2040 processors contains a programmable I/O system with two identical PIO devices (pio%=0 or pio%=1) acting like specialised CPU cores.<br>The Raspberry Pi Pico 2 with the RP2350 processors has three PIO devices.<br>See the Appendix F for a more detailed description of programming PIOs.<br>The PIO instructions are treated as MMBasic commands.<br>Valid commands are:<br>jmp (<cond>) <target><br>wait <polarity> gpio <gpio_num><br>wait <polarity> pin <pin_num><br>wait <polarity> irq (prev \| next) <irq_num> (rel)<br>wait <polarity> jmppin (+ <pin_offset>)<br>in <source>, <bit_count><br>out <destination>, <bit_count><br>push (iffull)<br>push (iffull) block<br>push (iffull) noblock<br>pull (ifempty)<br>pull (ifempty) block<br>pull (ifempty) noblock<br>mov <destination>, (op) <source><br>irq (prev \| next) <irq_num> (rel)<br>irq (prev \| next) set <irq_num> (rel)<br>irq (prev \| next) nowait <irq_num> (rel)<br>irq (prev \| next) wait <irq_num> (rel) |

| | |
|---|---|
| | irq (prev \| next) clear <irq_num> (rel)<br>set <destination>, <value><br><br>RP2350 only<br>mov rxfifo[y], isr<br>mov rxfifo[<index>], isr<br>mov osr, rxfifo[y] NB<br>mov osr, rxfifo[<index>] NB: RP2350 only<br><br>Valid directives:<br>.program name<br>.line n/next<br>.label name<br>.wrap target<br>.wrap<br>.side set n<br>.end program [list]<br><br>PIO commands are used between a 'PIO ASSEMBLE n' command and a '.end program' directive. Other directives will also appear between these. |
| PIO assemble pio,linedata$ | This command will assemble and load text based PIO assembler code including labels for jumps.<br>Use: PIO assemble pio,".program anything" to initialise the assembler.<br>Use: PIO assemble pio,".side_set n [opt] [pindirs]" if using side set. This is mandatory in order to correctly construct the op-codes if one or more side set pins are used.<br>It does not load the pinctrl register as this is specific to the state-machine.<br>Also note the "opt" parameter changes the op-code on instructions that have a side parameter.<br>Use: PIO assemble pio,".line n" to assemble starting from a line other than 1 - this is optional.<br>Use: PIO assemble pio,".end program [list]" to terminate the assembly and program the pio. The optional parameter LIST causes a hex dump of the op-codes to the terminal.<br>Use: PIO assemble pio,"label:" to define a label. This must appear as a separate command.<br>Use: PIO assemble "'.wrap target" to specify where the program will wrap to. See PIO(.wrap target) for how to use this.<br>Use: PIO assemble ".wrap" to specify where the program should wrap back to from ".wrap target" . See PIO(.wrap) for how to use this.<br>Use: PIO assemble pio "instruction [parameters]" to define the actual PIO instructions that will be converted to machine code. |
| PIO DMA RX pio, sm, nbr, data%() [,completioninterrupt] [,transfersize] [,loopbackcount]<br>PIO DMA TX pio, sm, nbr, data%() [,completioninterrupt] [,transfersize] [,loopbackcount] | Sets up DMA transfers from PIO to MMBasic memory.<br>pio specifies which of the two pio instances to use (0 or 1).<br>sm specifies which of the state machine to use (0-3).<br>nbr specifies how many 32-bit words to transfer. See below for the special case of setting nbr to zero.<br>data%() is the array that will either supply or receive the PIO data. |

| | |
|---|---|
| | The optional parameter completioninterrupt is the name of a MMBasic subroutine that will be called when the DMA completes and in the case of DMA_OUT the FIFO has been emptied.<br>If the optional interrupt is not used then the status of the DMA can be checked using the functions:<br>MM.INFO(PIO RX DMA)<br>MM.INFO(PIO TX DMA)<br>The optional parameter transfersize allows the user to override the normal 32-bit transfers and select 8, 16, or 32.<br>The optional parameter loopbackcount specifies how many data items are to be read or written before the DMA starts again at the beginning of the buffer.<br>The parameter must be a power of 2 between 2 and 32768.<br>Due to a limitation in the RP2040/RP2350 if loopbackcount is used the MMBasic array must be aligned in memory to the number of bytes in the loop.<br>Thus if the array is 64 integers long which is 512 bytes then the array must be aligned to a 512byte boundary in memory.<br>All MMBasic arrays are aligned to a 256 byte boundary but to create an array which is guaranteed to be aligned to a 512 byte boundary or greater the PIO MAKE RING BUFFER command must be used.<br>If loopbackcount is set then "nbr" can be set to 0. In this case the transfer will run continuously repeatedly filling the buffer until explicitly stopped.<br>If 'nbr' and 'loopbackcount; are both specified and identical the DMA will auto restart on completion (TX only) |
| PIO DMA RX OFF<br>PIO DMA TX OFF | The array must be global (or STATIC), not LOCAL, as the DMA uses it after the command has returned. |
| PIO INTERRUPT pio, sm [,RXinterrupt] [,TXinterrupt] | Aborts a running DMA.<br>Sets Basic interrupts for PIO activity.<br>Use the value 0 for RXinterrupt or TXinterrupt to disable an interrupt.<br>Omit values not needed.<br>The RX interrupt triggers whenever a word has been "pushed" by the PIO code into the specified FIFO. The data MUST be read in the interrupt to clear it.<br>The TX interrupt triggers whenever the specified FIFO has been FULL and the PIO code has now "pulled" it |
| PIO INIT MACHINE pio%, statemachine%, clockspeed [,pinctrl] [,execctrl] [,shiftctrl] [,startinstruction] [,sideout [,setout] [,outout] | Initialises PIO 'pio%' with state machine 'statemachine%'. 'clockspeed' is the clock speed of the state machine in kHz. The first four optional arguments are variables holding initialising values of the state machine registers and the address of the first instruction to execute (defaults to zero). These decide how the PIO will operate.<br>sideout, setout, and outout can be set to 0 (default) or 1 to specify if pins defined in pinctrl should be initialised as inputs (0) or outputs (1) |
| PIO EXECUTE pio, state_machine, instruction% | Immediately executes the instruction on the pio and state machine specified. |
| PIO WRITE pio, state_machine, count, data0 [,data1..] | Writes the data elements to the pio and state machine specified. The write is blocking so the state machine needs to be able to take the data supplied<br>NB: this command will probably need additional capability in future releases |

| | |
|---|---|
| PIO WRITEFIFO a,b,c,d | Writes to one of the 4 individual FIFO registers.<br>'a' is the pio (0 or 1), 'b' id the state machine (0...3), 'c' is the FIFO register *0…3), 'd' is the data% (32 bit integer value). |
| PIO READ pio, state_machine, count, data%[()] | Reads the data elements from the pio and state machine specified. The read is non-blocking so the state machine needs to be able to supply the data requested. When count is one then an integer can be used to receive the data, otherwise an integer array should be specified.<br>In the event you read when no data is available -1 will be returned. |
| PIO START pio, statemachine | Start a given state machine on pio. |
| PIO STOP pio, statemachine | Stop a given state machine on pio. |
| PIO CLEAR pio | This stops the pio specified on all statemachines and clears the control registers and interrupt flags for the statemachines PINCTRL, EXECTRL, and SHIFTCTRL to defaults. |
| PIO PROGRAM pio | Specifies that the next lines will be PIO assembler commands for the PIO 'pio' until a '.end program' directive |
| PIO PROGRAM pio,array%() | Programs the entire pio program memory with the data in array%(). See Appendix F. |
| PIO PROGRAM LINE pio, line, instruction | Programs just the specified line in a PIO program. |
| PIO SYNC pio, statemachines, [,prevstatemachines] [,nextstatemachines] | Synchronises PIO state machine clocks<br>'pio' specifies the reference PIO for the command<br>'statemachines', 'prevstatemachines' and 'nextstatemachines' are bitmaps (0 to 15) specifying which state machines' clocks should be synced.<br>So to specify that all of PIO0 and PIO1 state machines should be synced you could use:<br>PIO SYNC 0,15,,15<br>or<br>PIO SYNC 1,15,15 |
| PIO SET BASE 0/16 | PIO commands can only work with 32 GPIO ports. For the RP2350B this command tells the system to use GP0-GP31 (0) or GP16-GP47 (16) |
| PIO CONFIGURE pio, sm, clock [,startaddress] [,sidesetbase] [,sidesetno] [,sidesetout] [,setbase] [,setno] [,setout] [,outbase] [,outno] [,outout] [,inbase] [,jmppin] [,wraptarget] [,wrap] [,sideenable] [,sidepindir] [,pushthreshold] [,pullthreshold] [,autopush] [,autopull] [,inshiftdir] [,outshiftdir] [,joinrxfifo] [,jointxfifo] [,joinrxfifoget] [,joinrxfifoput] | The parameters in this command are essentially the same as you would use in the PIO INIT command plus the helper functions PINCTRL, SHIFTCTRL and EXECCTRL but combined into a single command. This is required because the Pico sdk does some very clever processing behind the scenes to handle the RP2350B<br>'sidesetbase', 'setbase', 'outbase', 'inbase' and 'jmppin' are pin definitions. You can specify these as either a GPno or a pin number (e.g. GP3 or 5). In all cases specify the actual pin. So if PIO SET BASE is set to 16 for that PIO then values GP16 to GP47 are valid. If PIO SET BASE is not set or is set to 0 then pins GP0 to GP31 are valid. They all default to the base set except 'jmppin' (defaults to -1) which needs to be explicitly set if you want to use a 'jmppin' as this triggers setting the required status bit<br>'clock' is the desired PIO clock speed in Hz<br>'startaddress' is the PIO statement that will start execution - defaults to 0<br>'sidesetno', 'setno' and 'outno' specify the number of pins that can be used for those functions - default to 0<br>'sidesetout', 'setout' and 'outout' specify if those pins should be configured as outputs (1=yes, 0=no) - default to 0 |

| | |
|---|---|
| | 'wraptarget and 'wrap' are in the range 0-31 and default to 0 and 31<br>'inshiftdir' and 'outshiftdir' default to 1 - shift out of output shift register to right and shift input shift register to right (data enters from left).<br>All other parameters are booleans that can enable a specific function - 1 to enable 0 to disable - all default to 0.<br>Simple example:<br>*'PIO Configure pio, sm, clock, startaddress,*<br>*'sidesetbase, sidesetno, sidesetout,*<br>*'setbase, setno, setout, outbase, outno, outout, inbase,*<br>*'jmppin, wraptarget, wrap, sideenable, sidepindir,*<br>*'pushthreshold, pullthreshold, autopush, autopull, inshiftdir, outshiftdir,*<br>*'joinrxfifo, jointxfifo, joinrxfifoget, joinrxfifoput*<br>*PIO assemble 1*<br>*.program test*<br>*.line 0*<br>*.wrap target*<br>*Set pins,1*<br>*Set pins,0*<br>*.wrap*<br>*.end program*<br>*SetPin gp45,pio1*<br>*PIO set base 1,16*<br>*PIO configure 1,0,1000000,,,,,gp45,1,1,,,,,,Pio(.wrap target),Pio(.wrap)*<br>*PIO start 1,0*<br>*Do*<br>*Loop*<br><br>Although the PIO CONFIGURE command has many parameters, it is very easy to use if you adopt this simple approach: Copy the comment lines in the example into your program. For each parameter substitute your required value or delete the parameter leaving the commas intact.<br>Once all substitutions are done delete any trailing commas<br>Then assuming the line will be too long for the editor delete the CRs one by one starting at the end of the second last line and moving upwards.<br>In this way you will have a valid command that is easy to input and edit.<br>NB: You can also use continuation lines to make the editing easier (see OPTION CONTINUATION LINES) |
| PIXEL x, y [,c] | Set a pixel on a video output or an attached LCD panel to a colour.<br>'x' is the horizontal coordinate and 'y' is the vertical coordinate of the pixel. 'c' is a 24 bit number specifying the colour. 'c' is optional and if omitted the current foreground colour will be used.<br>All parameters can be expressed as arrays and the software will plot the number of pixels as determined by the dimensions of the smallest array. 'x' and 'y' must both be arrays or both be single variables /constants otherwise an error will be generated. 'c' can be either an array or a single variable or constant.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates. |
| PLAY | This command will generate a variety of audio outputs.<br>See the OPTION AUDIO command for setting the I/O pins to be used for the output. The audio is a pulse width modulated signal (PWM) so a low pass filter is required to remove the carrier frequency. |
| PLAY TONE left, right [,dur] [,interrupt] | Generates two separate frequencies on the sound output left and right channels.<br>'left' and 'right' are the frequencies in Hz to use for the left and right channels. The tone plays in the background (the program will continue running after this |

| | |
|---|---|
| | command) and 'dur' specifies the number of milliseconds that the tone will sound for. If the duration is not specified the tone will continue until explicitly stopped or the program terminates.<br>'interrupt' is an optional subroutine which will be called when the play terminates.<br>The frequency can be from 1Hz to 20KHz and is very accurate (it is based on a crystal oscillator). The frequency can be changed at any time by issuing a new PLAY TONE command. |
| PLAY FLAC file$ [, interrupt] | Will play a FLAC file on the sound output.<br>'file$' is the FLAC file to play (the extension of .flac will be appended if missing). The sample rate can be up to 48kHz in stereo (96kHz if the Pico is overclocked)<br>The FLAC file is played in the background. 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing.<br>If file$ is a directory on the B: drive the Pico will play all of the files in that directory in turn. |
| PLAY WAV file$ [, interrupt] | Will play a WAV file on the sound output.<br>'file$' is the WAV file to play (the extension of .wav will be appended if missing). The WAV file must be PCM encoded in mono or stereo with 8 or 16-bit sampling. The sample rate can be up to 48kHz in stereo (96kHz if the Pico is overclocked).<br>The WAV file is played in the background. 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing. |
| PLAY ARRAY l%(), r%(), freq [,start] [,end] [,terminationinterrupt] | Outputs data to the audio output using data in one or two arrays:<br>This requires packed arrays ;l%' and 'r%' (could be the same array for left and right) of 16 bit values in the range -32768 to 32767 and outputs the values in the array at the sample frequency specified.<br>If the optional parameter 'start' is specified then the arrays are output from packed location 'start'<br>If the optional 'end' is specified then the arrays are output until the packed location 'end' is reached<br>If the optional parameter 'terminationinterrupt' is specified then the subroutine specified will execute once the last array element has been output.<br>It is important to understand the 'freq' parameter. This is the rate at which each array element is output. So for example, if the array contained 10 cycles of a sine wave with 18000 total samples then setting the frequency to 1800 would output a 1Hz sine wave for 10 seconds. Setting the frequency to 18000 would output a 10Hz sine wave for 1 second.<br>The arrays must be global (or STATIC), not LOCAL, as they are read while the sound plays. |
| PLAY MP3 file$ [, interrupt] | Will play a MP3 file on the sound output (RP2350 ONLY).<br>'file$' is the MP3file to play (the extension of .mp3 will be appended if missing). The sample rate can be up to 48kHz.<br>The MP3 file is played in the background. 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing.<br>If file$ is a directory on the B: drive the Pico will play all of the files in that directory in turn. |
| PLAY MODFILE file$ [,interrupt] | Will play a MOD file on the sound output.<br>'file$' is the MOD file to play (the extension of .mod will be appended if missing). |

| | |
|---|---|
| | The MOD file is played in the background and will play continuously in a loop. If the optional 'interrupt' is specified This will be called when the file has played once through the sequence and playback will then be terminated. This command will preferentially use space in PSRAM if enabled for the file buffer (RP2350 only). In this case a modbuffer does not need to be enabled with the OPTION command, nor on an RP2350 with 16MB of flash, which has a fixed 1MB mod buffer. |
| PLAY MODSAMPLE Samplenum, channel [,volume] | Plays a specific sample in the mod file on the channel specified. The volume is optional and can be between 0 and 64. This command can only be used when there is a mod file already playing and allows sound effects to be output whilst the background music is still playing. |
| PLAY LOAD SOUND array%() | Loads a 1024 element array comprising 4096 16-bit values between 0 and 4095. This provides the data for any arbitrary waveform that can be played by the PLAY SOUND command. You can use the MEMORY PACK command to create the arrays from a normal 4096 element integer array.<br>The array must be global (or STATIC), not LOCAL, as it is read while sounds play. |
| PLAY SOUND soundno, channelno, type [,frequency] [,volume] | Play a series of sounds simultaneously on the audio output.<br>'soundno' is the sound number and can be from 1 to 4 allowing for four simultaneous sounds on each channel.<br>'channelno' specifies the output channel. It can be L (left speaker), R (right speaker), B (both speakers).<br>type' specifies the wave form to be used. It can be S (Sine wave), Q (square wave) , T (triangular wave) , W (saw tooth) , O (Null output), P (periodic noise), N (random noise) or U (user defined using PLAY LOAD SOUND).<br>'frequency' is the frequency from 1 to 20000 (Hz) and it must be specified except when type is O. In Type U mode, this parameter can also accept decimal values. For example, all of the following will play back the waveform at its original pitch:<br>Sample rate of 4000, frequency = 1<br>Sample rate of 8000, frequency = 2<br>Sample rate of 16000, frequency = 4<br>'volume' is optional and must be between 1 and 25. It defaults to 25<br>When PLAY SOUND is called all other audio usage will be blocked and will remain blocked until PLAY STOP is called. Output can be stopped temporarily using PLAY PAUSE and PLAY RESUME.<br>Calling SOUND on an already running 'soundno' will immediately replace the previous output. Individual sounds are turned off using type “O”<br>Running 4 sounds simultaneously on both channels of the audio output consumes about 23% of the CPU. |
| PLAY PAUSE<br>PLAY RESUME<br>PLAY STOP | PLAY PAUSE will temporarily halt the currently playing file or tone.<br>PLAY RESUME will resume playing a sound that was paused.<br>PLAY STOP will terminate the playing of the file or tone. When the program terminates for whatever reason the sound output will also be automatically stopped. |
| PLAY VOLUME left, right | Will adjust the volume of the audio output.<br>'left' and 'right' are the levels to use for the left and right channels and can be between 0 and 100 with 100 being the maximum volume. There is a linear relationship between the specified level and the output. The volume defaults to maximum when a program is run. |

| | |
|---|---|
| PLAY NEXT<br><br>PLAY PREVIOUS | Stops playback of the current audio file and starts the next one in the directory<br><br>Stops playback of the current audio file and starts the previous one in the directory |
| PLAY SAMPLE left%(), right%(), freq, attack, decay, sustain, release [, freqR, attackR, decayR, sustainR, releaseR] [, interrupt]<br>or<br>PLAY RELEASE | RP2350 VERSIONS ONLY.<br>Plays a looped single-cycle waveform from integer arrays with an ADSR envelope. The optional right-channel parameter set allows independent stereo frequency and ADSR control.<br>Use PLAY RELEASE to enter the release phase and finish playback.<br>The arrays must be global (or STATIC), not LOCAL, as they are read while the sound plays. |
| PLAY BBC SOUND channel, amplitude, pitch, duration | The BBC Micro's SOUND statement. Queues a note on one of four channels: channels 1 to 3 are square-wave tones and channel 0 is a noise source. The notes queued on a channel play in order, each for its duration, so a tune can be written as a run of PLAY BBC SOUND commands and the program carries on while it plays.<br>'channel' is the BBC channel word &HSFC. C (bits 0 and 1) is the channel, 0 to 3. Adding &H10 flushes the channel's queue and stops its current note so that the new note is heard at once. Adding &H100, &H200 or &H300 holds the note until that many other channels are waiting with the same value at the head of their queues, and then they all start together, so a three note chord uses &H2xx on all three channels.<br>'amplitude' is -15 (loudest) to 0 (silent, a rest), or 1 to 16 to use an envelope defined with PLAY BBC ENVELOPE. 'pitch' is 0 to 255 in quarter semitones, 89 being A4 (440Hz) and 53 middle C; on channel 0 only its low three bits are used and they select one of eight noises: 0 to 2 are periodic (a buzzy, pitched tone) at high, medium and low rates, 4 to 6 are white noise at the same three rates, and 3 and 7 are periodic and white clocked by channel 1's tone rather than at a fixed rate, so a note on channel 1 tunes the noise and a pitch envelope on channel 1 bends it (channel 1 need not be audible; an amplitude of 0 still sets its pitch). 'duration' is 0 to 255 in twentieths of a second and 255 means until the channel is flushed or PLAY STOP is used.<br>Each channel queues up to 16 notes. MM.INFO(BBC BUFFER) returns the number of free slots, so a program can keep ahead of the music without ever waiting. When a channel's queue is full the command waits for a slot, as the BBC did, so a loop of PLAY BBC SOUND commands paces itself to the music (Ctrl-C works while it waits but MMBasic interrupts are held off until it returns). The audio output is released automatically when every channel is silent and every queue is empty, as it is when a PLAY TONE ends, and the next PLAY BBC SOUND starts it again. The output is mono and PLAY VOLUME applies. The engine's state (608 bytes) is taken from the MMBasic heap by the first PLAY BBC command and freed when the program's variables are cleared. See the Sound Output chapter for the pitch table and examples. |

| | |
|---|---|
| PLAY BBC ENVELOPE n, T, PI1, PI2, PI3, PN1, PN2, PN3, AA, AD, AS, AR, ALA, ALD | The BBC Micro's ENVELOPE statement. Defines envelope 'n' (1 to 16) for use as the amplitude of a PLAY BBC SOUND. It shapes the volume of the note (attack, decay, sustain and release) and can bend its pitch while it plays. Definitions survive PLAY STOP and are cleared with the program's variables by RUN, NEW and loading a program, so a program defines its envelopes at the start.<br>'T' is the length of each step in hundredths of a second (bits 0 to 6, 0 is treated as 1); add 128 to stop the pitch envelope repeating. 'PI1', 'PI2' and 'PI3' (-128 to 127) are the pitch change per step, in quarter semitones, in each of three pitch sections and 'PN1', 'PN2' and 'PN3' (0 to 255) are the number of steps in each section. When the three sections are done the pitch returns to the note's own and the sections repeat, unless 128 was added to T.<br>'AA', 'AD', 'AS' and 'AR' (-127 to 127) are the volume change per step during the attack, decay, sustain and release phases and 'ALA' and 'ALD' (0 to 126) are the levels the attack rises to and the decay falls to (126 is as loud as amplitude -15). The attack runs from 0 to ALA, the decay from there to ALD, the sustain lasts until the note's duration ends and the release then runs until the level reaches 0, at which point the note ends. A release of 0 never ends the note, so it will sound until the channel is flushed. |
| | **VS1053 SPECIFIC PLAY COMMANDS** |
| PLAY MP3 file$ [, interrupt] | Will play a MP3 file on the sound output.<br>'file$' is the MP3 file to play (the extension of .mp3 will be appended if missing). The sample rate should be 44100Hz stereo.<br>The MP3 file is played in the background. 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing.<br>If file$ is a directory on the B: drive the Pico will play all of the files in that directory in turn. |
| PLAY HALT | This command works when a MP3 file is playing. It stops playback and records the current file position to allow playback to be resumed from the same point. This command is specifically designed to support for mp3 audio books |
| PLAY CONTINUE track$ | Resumes playback of the MP3 track specified. "track$" will be the name of the file that was playing when halted with all file attributes removed<br>eg,<br>PLAY MP3 "B:/mp3/mymp3.mp3"<br>sometime later<br>PLAY HALT<br>later again<br>PLAY CONTINUE "mymp3" |
| PLAY MIDIFILE file$ [, interrupt] | Will play a MIDI file on the sound output.<br>'file$' is the MIDI file to play (the extension of .mid will be appended if missing).<br>The MIDI file is played in the background. 'interrupt' is optional and is the name of a subroutine which will be called when the file has finished playing.<br>If file$ is a directory on the B: drive the Pico will play all of the files in that directory in turn. |
| PLAY MIDI | Initiates the real-time midi mode. In this mode midi instructions can be sent to the VS1053 to select which instruments to play on which channels, turn notes on, and turn them off in real timer |
| PLAY MIDI CMD cmd%, data1%, data2% | Sends a midi command when in real time midi mode. An example would be to allocate an instrument to a channel. Eg,<br>PLAY MIDI CMD &B11000001,4 ‘set channel 1 to instrument 4 |

| | |
|---|---|
| PLAY MIDI TEST n | Plays a MIDI test sequence, n=0 to 3, 0 = normal realtime, the others play note and instrument samples |
| PLAY NOTE ON channel%, note%, velocity% | Turns on the note on the channel specified when in real time MIDI mode |
| PLAY NOTE OFF channel%, note% [,velocity%] | Turns off the note on the channel specified when in real time MIDI mode |
| PLAY STREAM buffer%(), readpointer%, writepointer% | Sends data to the VS1053 CODEC from the circular buffer “buffer%”. This command initiates a background output stream where the VS1053 is sent anything in the buffer between the readpointer and the write pointer, updating the readpointer as it goes. Can be used for arbitrary waveform output.<br>‘buffer%()’, ‘readpointer%’ and ‘writepointer%’ must be global (or STATIC), not LOCAL, as they are used in the background. |
| POKE BYTE addr%, byte<br>or<br>POKE SHORT addr%, short%<br>or<br>POKE WORD addr%, word%<br>or<br>POKE INTEGER addr%, int%<br>or<br>POKE FLOAT addr%, float!<br>or<br>POKE VAR var, offset, byte<br>or<br>POKE VARTBL, offset, byte | Will set a byte or a word within the Pico’s memory space. Where more than one byte is written the address must be exactly divisible by the number of bytes: 2, 4, or 8 otherwise an error will be reported.<br>POKE BYTE will set the byte (i.e. 8 bits) at the memory location 'addr%' to 'byte'. 'addr%' should be an integer.<br>POKE SHORT will set the short integer (i.e. 16 bits) at the memory location 'addr%' to 'word%'. 'addr%' and short%' should be integers.<br>POKE WORD will set the word (i.e. 32 bits) at the memory location 'addr%' to 'word%'. 'addr%' and 'word%' should be integers.<br>POKE INTEGER will set the MMBasic integer (i.e. 64 bits) at the memory location 'addr%' to int%'. 'addr%' and int%' should be integers.<br>POKE FLOAT will set the word (i.e. 64 bits) at the memory location 'addr%' to 'float!'. 'addr%' should be an integer and 'float!' a floating point number.<br>POKE VAR will set a byte in the memory address of 'var'. 'offset' is the ±offset from the address of the variable. An array is specified as var().<br>POKE VARTBL was removed in V7.00.00: the variables are no longer kept in one table, so an offset into it means nothing. Use POKE VAR to change a variable's value, and PEEK(VARHEADER var) for the address of its header. |
| POKE DISPLAY command [,data1] [,data2] [,datan] | This command will send commands and associated data to the display controller for a connected display. This allows the programmer to change parameters of how the display is configured. eg, POKE DISPLAY &H28 will turn off an SSD1963 display and POKE DISPLAY &H29 will turn it back on again.<br>Works for all displays except the ST7790. |
| POKE DISPLAY HRES n<br>POKE DISPLAY VRES n | These commands change the stored value of MM.HRES and MM.VRES allowing the programmer to configure non-standard displays. |
| POLYGON n, xarray%(), yarray%() [, bordercolour] [, fillcolour]<br>POLYGON n(), xarray%(), yarray%() [, bordercolour()] [, fillcolour()] | Draws a filled or outline polygon with ‘n’ xy-coordinate pairs in ‘xarray%()’ and ‘yarray%()’. If ‘fillcolour’ is omitted then just the polygon outline is drawn. If ‘bordercolour’ is omitted then it will default to the current default foreground colour.<br>If the last xy-coordinate pair is not the same as the first the firmware will automatically create an additional xy-coordinate pair to complete the polygon. The size of the arrays should be at least as big as the number of x,y coordinate pairs.<br>'n' can be an array and the colours can also optionally be arrays as follows: |

| | |
|---|---|
| POLYGON n(), xarray%(), yarray%() [, bordercolour] [, fillcolour] | POLYGON n(), xarray%(), yarray%() [, bordercolour()] [, fillcolour()]<br>POLYGON n(), xarray%(), yarray%() [, bordercolour] [, fillcolour]<br>The elements of 'array n()' define the number of xy-coordinate pairs in each of the polygons. eg, DIM n(1)=(3,3) would define that 2 polygons are to be drawn with three vertices each. The size of the n array determines the number of polygons that will be drawn unless an element is found with the value zero in which case the firmware only processes polygons up to that point. The x,y-coordinate pairs for all the polygons are stored in 'xarray%()' and 'yarray%()'. The 'xarray%()' and 'yarray%()' parameters must have at least as many elements as the total of the values in the n array.<br>Each polygon can be closed with the first and last elements the same. If the last element is not the same as the first the firmware will automatically create an additional x,y-coordinate pair to complete the polygon. If fill colour is omitted then just the polygon outlines are drawn.<br>The colour parameters can be a single value in which case all polygons are drawn in the same colour or they can be arrays with the same cardinality as 'n'. In this case each polygon drawn can have a different colour of both border and/or fill.<br>For example, this will draw 3 triangles in yellow, green and red:<br>`DIM c%(2)=(3,3,3)`<br>`DIM x%(8)=(100,50,150,100,50,150,100,50,150)`<br>`DIM y%(8)=(50,100,100,150,200,200,250,300,300)`<br>`DIM fc%(2)=(rgb(yellow),rgb(green),rgb(red))`<br>`POLYGON c%(),x%(),y%(),fc%(),fc%()` |
| PORT(start, nbr [,start, nbr]…) = value | Set a number of I/O pins simultaneously (i.e. with one command).<br>'start' is an I/O pin number and the lowest bit in 'value' (bit 0) will be used to set that pin. Bit 1 will be used to set the pin 'start' plus 1, bit 2 will set pin 'start'+2 and so on for 'nbr' number of bits. I/O pins used must be numbered consecutively and any I/O pin that is invalid or not configured as an output will cause an error. The start/nbr pair can be repeated if an additional group of output pins needed to be added.<br>For example; `PORT(15, 4, 23, 4) = &B10000011`<br>Will set eight I/O pins. Pins 15 and 16 will be set high while 17, 18, 23, 24 and 25 will be set to a low and finally 26 will be set high.<br>This command can be used to conveniently communicate with parallel devices like LCD displays. Any number of I/O pins (and therefore bits) can be used from 1 to the number of I/O pins on the chip.<br>NB: If the pins are defined using the GPn syntax then the firmware will ignore invalid pins so `PORT(GP0, 8) = &B10000011` Will set eight I/O pins. Pins GP0 and GP1 and GP7 will be set high while GP2,GP3,GP4,GP5 and GP6 will be set to a low.<br>See the PORT function to simultaneously read from a number of pins. |
| PRINT expression [[,; ]expression] … etc | Outputs text to the serial console followed by a carriage return/newline pair. Multiple expressions can be used and must be separated by either a:<br>• Comma (,) which will output the tab character<br>• Semicolon (;) which will not output anything (it is just used to separate expressions).<br>• Nothing or a space which will act the same as a semicolon.<br>A semicolon (;) or a comma (,) at the end of the expression list will suppress the output of the carriage return/newline pair at the end of a print statement.<br>When printed, a number is preceded with a space if positive or a minus (-) if negative but is not followed by a space. Integers (whole numbers) are printed without a decimal point while fractions are printed with the decimal point and |

| | |
|---|---|
| | the significant decimal digits. Large or small floating point numbers are automatically printed in scientific number format.<br>The function TAB() can be used to space to a certain column and the STR$() function can be used to justify or otherwise format strings. |
| PRINT #nbr, expression [[,; ]expression] … etc | Same as above except that the output is directed to a serial communications port or a file opened for OUTPUT or APPEND with a file number of 'nbr'. See the OPEN command. |
| PRINT #GPS, expression [[,; ]expression] … etc | Outputs a NMEA string to an opened GPS device. The string must start with a $ character and end with a * character. The checksum is automatically calculated and appended to the string together with the CR/LF characters. |
| PRINT @(x [, y]) expression<br>or<br>PRINT @(x, [y], m) expression | Works on terminal console on an attached computer or VGA/HDMI video or the display if OPTION LCDPANEL CONSOLE is enabled.<br>Same as the standard PRINT command except that the cursor is positioned at the coordinates x, y expressed in pixels. If y is omitted the cursor will be positioned at "x" on the current line.<br>Example: PRINT @(150, 45) "Hello World"<br>The @ function can be used anywhere in a print command.<br>Example: PRINT @(150, 45) "Hello" @(150, 55) "World"<br>The @(x,y) function can be used to position the cursor anywhere on or off the screen. For example, PRINT @(-10, 0) "Hello" will only show "llo" as the first two characters could not be shown because they were off the screen.<br>The @(x,y) function will automatically suppress the automatic line wrap normally performed when the cursor goes beyond the right screen margin.<br>If 'm' is specified the mode of the video operation will be as follows:<br>m = 0 Normal text (white letters, black background)<br>m = 1 The background will not be drawn (ie, transparent)<br>m = 2 The video will be inverted (black letters, white background)<br>m = 5 Current pixels will be inverted (transparent background) |
| PULSE pin, width | Will generate a pulse on 'pin' with duration of 'width' ms. 'width' can be a fraction. For example, 0.01 is equal to 10µs and this enables the generation of very narrow pulses.<br>The generated pulse is of the opposite polarity to the state of the I/O pin when the command is executed. For example, if the output is set high the PULSE command will generate a negative going pulse.<br>Notes:<br>• 'pin' must be configured as an output.<br>• For a pulse of less than 3 ms the accuracy is ± 1 µs.<br>• For a pulse of 3 ms or more the accuracy is ± 0.5 ms.<br>• A pulse of 3 ms or more will run in the background. Up to five different and concurrent pulses can be running in the background and each can have its time changed by issuing a new PULSE command or it can be terminated by issuing a PULSE command with zero for 'width'. |
| PWM channel, frequency, [dutyA] [,dutyB][,phase][,defer] | There are 8 separate PWM frequencies available (channels 0 to 7) and up to 16 outputs with individually controlled duty cycle. You can output on either PWMnA or PWMnB or both for each channel - no restriction. Duty cycles are specified as a percentage and you can use a negative value to invert the output (-100.0 <= duty <=100.0).<br>To use just channel B use the syntax: 'PWM channel, frequency, , dutyB' - note the double comma before the required duty cycle. |

| | |
|---|---|
| | Minimum frequency = (cpuspeed + 1) / (2^24) Hz. Maximum speed is OPTION CPUSPEED/4. At very fast speeds the duty cycles will be increasingly limited.<br>Phase is a parameter that causes the waveforms to be centred such that a wave form with a shorter duty cycle starts and ends equal times from a longer one. Use 1 to enable this mode and 0 (or omit) to run as normal<br>The parameter 'deferredstart' when set to 1 configures the PWM channels ~~as~~ but does not start the output running. They can ~~the~~ be started using the PWM SYNC command. This can be used to avoid any undesirable startup artefacts<br>The PWM command is also capable of driving servos as follows:<br>`PWM 1,50,(position_as_a_percentage * 0.05 + 5)` |
| PWM SYNC s0 [,s1][,s2][,s3][,s4][,s5][,s6][,s7] | This initiates the PWM on channels where a deferred start was defined or just syncs existing running channels. However, the power comes in the ability to offset the channels one to another (defined as a percentage of the time period as per the duty cycle - can be a float)<br>You can use an offset of -1 to omit a channel from the synch |
| PWM channel, OFF | Stop output on 'channel'. |
| RAM | <u>RP2350 VERSIONS WITH PSRAM ENABLED ONLY</u><br>The RAM command allows access to up to 5 RAM program slots (similar to flash slots). RAM slots survive a H/W and software reset but not a power cycle. |
| RAM LIST | Displays a list of all ram locations including the first line of the program. |
| RAM LIST n [,all] | List the program saved to slot n. Use ALL to list without page breaks. |
| RAM ERASE n | Erase a ram program location. |
| RAM ERASE ALL | Erase all ram program locations. |
| RAM SAVE n | Save the current program to the ram location specified. |
| RAM LOAD n | Load a program from the specified ram location into program memory. |
| RAM RUN n | Runs the program in ram location n, clear all variables. Does not change the program memory. |
| RAM CHAIN n | Runs the program in ram location n, leaving all variables intact (allows for a program that is much bigger than the program memory). Does not change the program memory. NB: if the chained program uses the READ command it must call RESTORE before the first read. |
| RAM OVERWRITE n | Erase a ram program location and then save the current program to the ram location specified. |
| RAM FILE LOAD n, fname$ [,O[VERWRITE]] | Loads the MMBasic file fname$ into the ram slot specified.<br>If the optional parameter OVERWRITE (or O) is specified the content of the flash slot will be overwritten without an error being raised. |

| | |
|---|---|
| RANDOMIZE nbr | Seed the random number generator with 'nbr'.<br>On the RP2040 the random number generator is seeded with zero at power up and will generate the same sequence of random numbers each time. To generate a different random sequence each time you must use a different value for 'nbr' (the TIMER function is handy for that).<br>This command does nothing on the RP2350 which has a hardware random generator that does not require seeding. |
| RAY | RP2350 VERSIONS EXCEPT WEBMITERP2350<br>Command to allow the development of games using raycasting techniques. See the separate manual 'Raycaster_User_Manual.pdf' |
| RBOX x, y, w, h [, r] [,c] [,fill] | Draws a box with rounded corners on the video output or attached LCD panel starting at 'x' and 'y' which is 'w' pixels wide and 'h' pixels high.<br>'r' is the radius of the corners of the box. It defaults to 10.<br>'c' specifies the colour and defaults to the default foreground colour if not specified. 'fill' is the fill colour. It can be omitted or set to -1 in which case the box will not be filled.<br>All parameters can be expressed as arrays and the software will plot the number of boxes as determined by the dimensions of the smallest array. 'x', 'y', 'w', and 'h' must all be arrays or all be single variables /constants otherwise an error will be generated. 'r', 'c', and 'fill' can be either arrays or single variables/constants.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates. |
| REDIM [PRESERVE] array1(dimensions) [, array2(dimensions]…[,arrayn( dimensions] | This resizes the array(s) specified with the dimensions given.<br>If the optional sub-command PRESERVE is specified the existing data will be copied into the new array.<br>The new array can be bigger or smaller than the original.<br>In the case of string arrays, the original LENGTH specified is preserved.<br>Note that for multi-dimensional arrays only the last dimension can be changed if PRESERVE is used.<br>When using PRESERVE there must be enough memory available for both the original array and its changed version to exist simultaneously. The memory allocated to the original array is freed as the command exits. |
| READ variable[, variable].. | Reads values from DATA statements and assigns these values to the named variables. Variable types in a READ statement must match the data types in DATA statements as they are read.<br>Arrays can be used as variables (specified with empty brackets, eg, a()) and in that case the size of the array is used to determine how many elements are to be read. If the array is multidimensional then the leftmost dimension will be the fastest moving.<br>DATA statements are searched in the main program first and then in the library, so when the program's DATA is exhausted (or the program has none) reading continues with any DATA statements in the library.<br>See also DATA and RESTORE. |
| READ SAVE<br>or<br>READ RESTORE | READ SAVE will save the virtual pointer used by the READ command to point to the next DATA to be read. READ RESTORE will restore the pointer that was previously saved.<br>This enables subroutines to READ data and then restore the read pointer so as not to disturb other parts of the program that may be reading the same data statements. These commands can be nested. |

| | |
|---|---|
| REFRESH | NOT VGA OR HDMI VERSIONS Initiates an update of the screen for e-ink black and white displays.<br>These can only be updated a full screen at a time and if OPTION AUTOREFRESH is OFF this command can be used to trigger the write.<br>This command works with the following displays: N5110, SSD1306I2C, SSD1306I2C32, SSD1306SPI, ST7920. |
| REM string | REM allows remarks to be included in a program.<br>Note the Microsoft style use of the single quotation mark (‘) to denote remarks is also supported and is preferred. |
| RENAME old$ AS new$ | Rename a file or a directory from ‘old$’ to ‘new$’. Both are strings.<br>A directory path can be used in both 'old$' and 'new$'. If the paths differ the file specified in 'old$' will be moved to the path specified in 'new$' with the file name as specified. |
| RESOLUTION nn [,cpuspeedinKHz] | HDMI VERSIONS ONLY<br>Changes the video resolution to ‘nn’ immediately and without restarting the firmware. ‘nn’ is one of the resolutions supported by OPTION RESOLUTION:<br>640x480 or 640<br>720x400 or 720<br>800x600 or 800<br>848x480 or 848<br>800x480<br>1024x768 or 1024<br>1024x600<br>1280x720 or 1280<br>The CPU clock speed is automatically changed to that required by the new resolution and the display restarts in MODE 1 (use the MODE command to select another mode). For 640x480 the display frequency can be set to 60Hz (252MHz or 378MHz) or 75Hz (315MHz) by appending ‘cpuspeedinKHz’ to the command (ie, 252000, 378000 or 315000) as per OPTION RESOLUTION.<br>The change is not saved; on reset or power up the resolution will return to that set by OPTION RESOLUTION.<br>The 800x600, 848x480 and 800x480 resolutions need a larger framebuffer than the other resolutions and this can only be allocated when the firmware starts. These resolutions can therefore only be selected with this command if the resolution set by OPTION RESOLUTION also uses the larger framebuffer; otherwise the command will report an error and OPTION RESOLUTION (which restarts the firmware) must be used. Switching away from these resolutions is always possible.<br>If the new resolution requires a different clock speed the command cannot be used while a PIO is in use.<br>On the PicoMiteHDMIWEB version everything is 8-bit colour and the choices are RESOLUTION 640 (or 640x480) - the factory default - 720 (or 720x400), 800 (or 800x600), 848 (or 848x480), 800x480 and 1024 (or 1024x600). For 640x480 the optional ‘cpuspeedinKHz’ (252000, 315000 or 378000) selects the display frequency exactly as on the other HDMI versions; each of the other resolutions runs at a fixed clock speed. The 800x600, 848x480 and 800x480 resolutions support only MODE 1 and MODE 2. The display restarts in MODE 1. Because every PicoMiteHDMIWEB resolution uses the same framebuffer pool, any of them may be selected with this command without a reboot. |

| | |
|---|---|
| RESTORE [line] | Resets the line and position counters for the READ statement.<br>If 'line' is specified the counters will be reset to the beginning of the specified line. 'line' can be a line number or a label or a variable with these values.<br>If 'line' is not specified the counters will be reset to the start of the program, or to the start of the library when RESTORE is executed within library code. |
| RETURN | Concludes a subroutine called by GOSUB and returns to the statement after the GOSUB.<br>GOSUB/RETURN are legacy commands; defined subroutines (SUB…END SUB) are recommended instead for new programs. |
| RMDIR dir$ | Remove, or delete, the directory 'dir$' on the default selected drive. |
| RTC GETTIME<br><br>RTC SETTIME year, month, day, hour, minute, second<br><br>RTC SETREG reg, value<br>RTC GETREG reg, var | RTC GETTIME will get the current date/time from a PCF8563, DS1307, DS3231 or RV3028 real time clock and set the internal MMBasic clock. The date/time can then be retrieved with the DATE$ and TIME$ functions.<br>RTC SETTIME will set the time in the clock chip. 'hour' must use 24 hour notation. 'year' can be two or four digits. The RTC SETTIME command will also accept a single string argument in the format of *dd/mm/yy hh:mm*. This means the date/time could be entered by the user using a GUI FORMATBOX with the DATETIME2 format (see *Advanced Graphics Functions.pdf*).<br>The RTC SETREG and GETREG commands can be used to set or read the contents of registers within the real time clock chip. 'reg' is the register's number, 'value' is the number to store in the register and 'var' is a variable that will receive the number read from the register. These commands are not necessary for normal operation but they can be used to manipulate special features of the chip (alarms, output signals, etc). They are also useful for storing temporary information in the chip's battery backed RAM.<br>These chips are $I^2C$ devices and must be connected to the two $I^2C$ pins as specified by OPTION SYSTEM I2C with appropriate pullup resistors.<br>Also see the command OPTION RTC AUTO ENABLE. |
| RUN<br>or<br>RUN [file$] [, cmdline$] | Run a program.<br>If 'file$' is not supplied then run the program currently held in program memory.<br>If 'file$' is supplied then run the named file from the Flash or SD Card filesystem. If 'file$' does not contain a '.BAS' extension then one will be automatically added.<br>If 'cmdline$' is supplied then pass its value to the MM.CMDLINE$ constant of the program when it runs. If 'cmdline'$ is not supplied then an empty string value is passed to MM.CMDLINE$.. Notes:<br>• Both 'file$' and 'cmdline$' may be supplied as string expressions.<br>• Use FLASH RUN n to run a program stored in a Flash Slot. |
| SAVE file$ | Saves the program in the current working directory of the Flash Filesystem or SD Card as 'file$'. Example: SAVE "TEST.BAS"<br>If an extension is not specified ".BAS" will be added to the file name.<br>See also FLASH SAVE *n* for saving to a Flash Slot. |
| SAVE CONTEXT [CLEAR] | Saves the variable space and optionally clears it - command should be used in top level program and not from within a subroutine. This saves the entire variable space to the A: drive. The command will fail if there is not enough free space on the A: drive. In the case of an RP2350 with PSRAM the variable space will be saved to a reserved area in the PSRAM and the A: drive is not used. Variables that had to be placed in PSRAM because the processor's own |

| | |
|---|---|
| | memory was full are saved too; in the rare case that there are too many of them the command fails with “Not enough room to save the context”.<br>See also LOAD CONTEXT |
| SAVE DATA fname$, address, nBytes | Writes ‘nBytes’ from memory address ‘address’ to the file fname$.<br>Note that there is no validation of the address so care should be taken that the address is valid within the Pico address map. |
| SAVE IMAGE file$ [,x, y, w, h]<br>or<br>SAVE COMPRESSED IMAGE file$ [,x, y, w, h] | Save the current image on the video output or LCD panel as a BMP file. Any LCD panel must be capable of being read, for example, a ILI9341 based panel or a VIRTUAL_M or VIRTUAL_C panel.<br>'file$' is the name of the file. If an extension is not specified “.BMP” will be added to the file name. The image is saved as a true colour 24-bit image.<br>‘x’, ‘y’, ‘w’ and ‘h’ are optional and are the coordinates (‘x’ and ‘y’ are the top left coordinate) and dimensions (width and height) of the area to be saved. If not specified, the whole screen will be saved. Note that ‘width’, if used, must be a multiple of 2.<br>SAVE COMPRESSED IMAGE will work the same except that RLE compression will be used to reduce the file size.. |
| SAVE PERSISTENT n% | Saves the value n% in a special memory location that will survive a watchdog reset or a physical reset but not a power cycle<br>See also MM.PERSISTENT or MM.INFO(PERSISTENT) |
| SEEK [#]fnbr, pos | Will position the read/write pointer in a file that has been opened on the Flash Filesystem or SD Card for RANDOM access to the 'pos' byte.<br>The first byte in a file is numbered one so SEEK #5,1 will position the read/write pointer to the start of the file opened as #5. |
| SELECT CASE value<br>CASE testexp [[, testexp] …]<br><statements><br><statements><br>CASE test-n [[, test-n] …]<br><statements><br><statements><br>CASE ELSE<br><statements><br><statements><br>END SELECT | Executes one of several groups of statements, depending on the value of an expression. 'value' is the expression to be tested. It can be a number or string variable or a complex expression.<br>'testexp' (or test-n) is the value that is to be compared against. It can be:<br>• A single expression (i.e. 34, "string" or PIN(4)*5) to which it may equal<br>• A range of values in the form of two single expressions separated by the keyword "TO" (i.e. 5 TO 9 or "aa" TO "cc")<br>• A comparison starting with the keyword "IS" (which is optional). For example: IS > 5, IS <= 10.<br>When a number of test expressions (separated by commas) are used the CASE statement will be true if any one of these tests evaluates to true.<br>If 'value' cannot be matched with a 'testexp' it will be automatically matched to the CASE ELSE. If CASE ELSE is not present the program will not execute any <statements> and continue with the code following the END SELECT. When a match is made the <statements> following the CASE statement will be executed until END SELECT or another CASE is encountered when the program will then continue with the code following the END SELECT.<br>An unlimited number of CASE statements can be used but there must be only one CASE ELSE and that should be the last before the END SELECT.<br>Example:<br>`SELECT CASE nbr%`<br>`CASE 4, 9, 22, 33 TO 88`<br>`statements`<br>`CASE IS < 4, IS > 88, 5 TO 8`<br>`statements`<br>`CASE ELSE`<br>`statements` |

| | |
|---|---|
| | END SELECT<br>Each SELECT CASE must have one and one only matching END SELECT statement. Any number of SELECT…CASE statements can be nested inside the CASE statements of other SELECT…CASE statements. |
| SERVO channel [positionA] [,positionB] | Control a standard servo.<br>'positionA' and 'positionB' can be between -20 and 120 and will generate a 50Hz signal between 800uSec and 2.2mSec<br>As with the PWM command the pins must be set up with SETPIN n,PWM<br>To use just channel B use the syntax: SERVO channel,,position.<br>Note the two commas to indicate that channel A is not being set.<br>Refer to the pinout to give the channel and sub-channel (A or B) for each pin.<br>Examples:<br>90° servo: 0 = 0° and 90 = 90°<br>180° servo: 0 = 0° and 90 = 180°<br>Note: With values < 0 or > 90, the current may increase when the servo reaches its end position.<br>360° servo: Speed 50 = Stop, Left = L->H:51-100, Right = L->H:49-0. |
| SETPIN pin, cfg [, option] | Will configure an external I/O pin. Refer to the chapter *Using the I/O pins* for a general description of the Pico's input/output capabilities.<br>'pin' is the I/O pin to configure, 'cfg' is the mode that the pin is to be set to and 'option' is an optional parameter. 'cfg' is a keyword and can be any one of the following:<br>OFF — Not configured or inactive<br>AIN — Analog input (i.e. measure the voltage on the input).<br>ARAW — Fast analog input returning a value between 0 and 4095.<br>DIN — Digital input<br>If 'option' is omitted the input will be high impedance<br>If 'option' is the keyword "PULLUP" or "PULLDOWN" a constant current of about 50µA will be used to pull the input pin up or down to 3.3V. Due to a bug in the RP2350 chips it is recommended that a pulldown be implemented using a 8.2K or less resistor.<br>FIN — Frequency input<br>'option' can be used to specify the gate time (the length of time used to count the input cycles). It can be any number between 10 ms and 100000 ms. The PIN() function will always return the frequency correctly scaled in Hz regardless of the gate time used. If 'option' is omitted the gate time will be 1 second.<br>The pins can be GP6, GP7, GP8 or GP9 (can be changed with OPTION COUNT).<br>PIN — Period input<br>'option' can be used to specify the number of input cycles to average the period measurement over. It can be any number between 1 and 10000. The PIN() function will always return the average period of one cycle correctly scaled in ms regardless of the number of cycles used for the average. If 'option' is omitted the period of just one cycle will be used.<br>The pins can be GP6, GP7, GP8 or GP9 (can be changed with OPTION COUNT).<br>CIN — Counting input<br>'option' can be used to specify which edge triggers the count and if any pullup or pulldown is enabled<br>2 specifies a falling edge with pullup,<br>3 specifies that both a falling and rising edge will trigger a |

| | |
|---|---|
| | count with no pullup applied,<br>5 specifies both edges but with a pullup applied.<br>If 'option' is omitted a rising edge will trigger the count.<br>Due to a bug in the RP2350 chips pulldown is not recommended.<br>The pins can be GP6, GP7, GP8 or GP9 (can be changed with OPTION COUNT).<br>DOUT Digital output<br>'option' is not used in this mode.<br>The functions PIN() and PORT() can also be used to return the value on one or more output pins. See the function PIN() for reading inputs and the statement PIN()= for setting an output. See the command below if an interrupt is configured. |
| SETPIN pin, cfg, target [, option] | Will configure 'pin' to generate an interrupt according to 'cfg'. Any I/O pin capable of digital input can be configured to generate an interrupt with a maximum of ten interrupts configured at any one time.<br>'cfg' is a keyword and can be any one of the following:<br>OFF Not configured or inactive<br>INTH Interrupt on low to high input<br>INTL Interrupt on high to low input<br>INTB Interrupt on both (i.e. any change to the input)<br>'target' is a user defined subroutine which will be called when the event happens. Return from the interrupt is via the END SUB or EXIT SUB commands. 'option' is the same as used in SETPIN pin DIN (above).<br>This mode also configures the pin as a digital input so the value of the pin can always be retrieved using the function PIN().<br>Refer to the chapter *Using the I/O pins* for a general description. |
| SETPIN GP25, DOUT \| HEARTBEAT | NOT WIFI VERSIONS<br>This version of SETPIN controls the on-board LED.<br>If it is configured as DOUT then it can be switched on and off under program control.<br>If configured as HEARTBEAT then it will flash 1s on, 1s off continually while powered. This is the default state and will be restored to this when the user program stops running. |
| SETPIN p1[, p2 [, p3]], device | These commands are used for the pin allocation for special devices.<br>Pins must be chosen from the pin designation diagram and must be allocated before the devices can be used. Note that the pins (eg, rx, tx, etc) can be declared in any order and that the pins can be referred to by using their pin number (eg, 1, 2) or GP number (eg, GP0, GP1).<br>Note that on the WebMite version:<br>• SPI1 and SPI2 are not available on GP20 to GP28<br>• COM1 and COM2 are not available on P20 to GP28<br>• I2C is not available on pin 34 (GP28)<br>• The following are not available; GP29, GP25, GP24 and GP23 |
| SETPIN rx, tx, COM1 | Allocate the pins to be used for serial port COM1.<br>Valid pins are RX: GP1, GP13 or GP17<br>TX: GP0, GP12, GP16 or GP28 |

| | |
|---|---|
| SETPIN rx, tx, COM2 | Allocate the pins to be used for serial port COM2.<br>Valid pins are RX: GP5, GP9 or GP21<br>TX: GP4, GP8 or GP20 |
| SETPIN rx, tx, clk, SPI | Allocate the pins to be used for SPI port SPI.<br>Valid pins are RX: GP0, GP4, GP16 or GP20<br>TX: GP3, GP7 or GP19<br>CLK: GP2, GP6 or GP18 |
| SETPIN rx, tx, clk, SPI2 | Allocate the pins to be used for SPI port SPI2.<br>Valid pins are RX: GP8, GP12 or GP28<br>TX: GP11, GP15 or GP27<br>CLK: GP10, GP14 or GP26 |
| SETPIN sda, scl, I2C | Allocate the pins to be used for the I²C port I2C.<br>Valid pins are SDA: GP0, GP4, GP8, GP12, GP16, GP20 or GP28<br>SCL: GP1, GP5, GP9, GP13, GP17 or GP21 |
| SETPIN sda, scl, I2C2 | Allocate the pins to be used for the I²C port I2C2.<br>Valid pins are SDA: GP2, GP6, GP10, GP14, GP18, GP22 or GP26<br>SCL: GP3, GP7, GP11, GP15, GP19 or GP27 |
| SETPIN pin, PWM[nx] | Allocate pin to PWMnx<br>'n' is the PWM number (0 to 7, or 0 to 11 on the RP2350B) and 'x' is the channel (A or B). n and x are optional.<br>The setpin can be changed until the PWM command is issued. At that point the pin becomes locked to PWM until PWMn,OFF is issued. |
| SETPIN pin, IR | Allocate pins for InfraRed (IR) communications (can be any pin). |
| SETPIN pin, PIOn | Reserve pin for use by a PIO0, PIO1 or PIO2 (RP2350 only) (see *Appendix F* for PIO details). |
| SETPIN GP1, FFIN [,gate] | <u>RP2350 VERSIONS ONLY</u><br>Sets GP1 as a fast frequency input.<br>Inputs up to the CPU speed /2 can be recorded.<br>'gate' can be used to specify the gate time (the length of time used to count the input cycles). It can be any number between 10 ms and 100000 ms. The PIN() function will always return the frequency correctly scaled in Hz regardless of the gate time used. If 'option' is omitted the gate time will be 1 second.<br>The function uses PWM channel 0 to do the counting so it is incompatible with any other use of that PWM channel. |
| SETTICK period, target [, nbr] | This will setup a periodic interrupt (or "tick").<br>Four tick timers are available ('nbr' is 1, 2, 3 or 4). 'nbr' is optional and if not specified timer number 1 will be used.<br>The time between interrupts is 'period' milliseconds and 'target' is the interrupt subroutine which will be called when the timed event occurs.<br>The period can range from 1 to 2147483647 ms (about 24 days).<br>These interrupts can be disabled by setting 'period' to zero<br>(i.e. SETTICK 0, 0, 3 will disable tick timer number 3). |

| | |
|---|---|
| SETTICK PAUSE, target [, nbr]<br>or<br>SETTICK RESUME, target [, nbr] | Pause or resume the specified timer. When paused the interrupt is delayed but the current count is maintained. |
| SLEW dRA1, dDec1, flipRA, flipDec, dRA2, dDec2, mountRA, mountDec, RAs, DECs, LST | RP2350 VERSIONS ONLY<br>Calculates the moves required to slew a telescope mount from its current position to a target computed by the STAR or ASTRO commands, handling a meridian flip if one is needed. The first six arguments are variables that receive the results (the first move, the flip moves and the second move); 'mountRA' and 'mountDec' describe where the mount is currently pointing, 'RAs' and 'DECs' are the target position and 'LST' is the local sidereal time from the LOCATION command.<br>This is one of a set of commands that performs high-precision astronomical calculations suitable for telescope pointing or navigation and is described in detail in the file GPS_Astro_Reference.pdf included in the firmware download zip file. |
| SORT array() [,indexarray()] [,flags] [,startposition] [,elementstosort] | This command takes an array of any type (integer, float or string) and sorts it into ascending order in place.<br>It has an optional parameter 'indexarray%()'. If used this must be an integer array of the same size as the array to be sorted. After the sort this array will contain the original index position of each element in the array being sorted before it was sorted. Any data in the array will be overwritten. This allows connected arrays to be sorted.<br>The 'flag' parameter is optional and valid flag values are:<br>bit0: 0 (default if omitted) normal sort - 1 reverse sort<br>bit1: 0 (default) case dependent - 1 sort is case independent (strings only).<br>bit2: 0 (default) normal sort - 1 empty strings go to the end of the array<br>The optional 'startposition' defines which element in the array to start the sort. Default is 0 (OPTION BASE 0) or 1 (OPTION BASE 1)<br>The optional 'elementstosort' defines how many elements in the array should be sorted. The default is all elements after the 'startposition'.<br>Any of the optional parameters may be omitted so, for example, to sort just the first 50 elements of an array you could use:<br>`SORT array(), , , ,50`<br>Example:<br>The array city$() might contain the names of world cities and can be easily sorted into increasing alphabetical order with the command: SORT city$()<br>The SORT command will work with strings, floats and integers however the array to be sorted must be single dimensioned.<br>Often data is held in multiple arrays, for example, the name of each city might be held in the array city$(), the population held in the array pop%() and the size of the city held in area!(). The same index would refer to the name, population and the area of the city.<br>Sorting and accessing this data is a little more complex but it can be done relatively easily using an optional parameter to the sort command as follows:<br>SORT array(), indexarray%()<br>indexarray%() must be a single dimension integer array of the same size as the array being sorted. Following the sort indexarray%() will contain the corresponding index to the original data before it was sorted. (anything previously in indexarray%() will be overwritten). |

| | |
|---|---|
| | To access the sorted data you would first copy the array holding the main key to a temporary array and sort that while specifying indexarray%(). After the sort indexarray%() can be used to index the original arrays.<br>For example:<br>`DIM city$(100),pop%(100),area!(100),sindex%(100),t$(100)`<br>`FOR i = 0 to 100`<br>`  t$(i) = city$(i)  ' temporary copy of the keys`<br>`NEXT i`<br>`SORT t$(), sindex%()  ' sort the temporary array,`<br>`FOR i = 0 to 100`<br>`  k = sindex%(i)  ' index to the original array`<br>`  PRINT city$(k),pop%(k),area!(k)' print in sorted order`<br>`NEXT i` |
| SPI OPEN speed, mode, bits<br>or<br>SPI READ nbr, data [, data…]<br>or<br>SPI WRITE nbr, data [, data…]<br>or<br>SPI CLOSE | Communications via an SPI channel. See Appendix D for the details.<br>'nbr' is the number of data items to send or receive.<br>The data to send (WRITE) or the destination for the received data (READ) may be supplied in any one of three forms:<br>A list of individual items. The number of items must equal 'nbr'. For WRITE each item is a constant, variable or expression; for READ each item is a numeric variable to receive one value. Array elements such as a(3) are allowed.<br>An array specified with empty brackets, eg, array(). It may be a floating point or integer array and must be large enough to supply (WRITE) or hold (READ) 'nbr' elements, which are taken from / stored into consecutive elements starting at the first.<br>A string variable. For WRITE the first 'nbr' characters are sent (the string must be at least that long); for READ the received bytes are stored as sequential characters and the string is set to that length.<br>SPI WRITE discards any data returned by the slave and SPI READ sends zeros while reading the data from the slave. |
| SPI2 | The same set of commands as for SPI (above) but applying to the second SPI channel. |
| SPRITE | See Appendix G and the separate SPRITE_User_Manual pdf |
| STAR | RP2350 VERSIONS ONLY Calculates the position of a celestial object using the current time and location from a connected GPS module.<br>This is one of a set of commands that performs high-precision astronomical calculations suitable for telescope pointing or navigation and is described in detail in the file *GPS_Astro_Reference.pdf* included in the firmware download zip file. |
| STATIC variable [, variables]<br>See DIM for the full syntax. | Defines a list of variable names which are local to the subroutine or function. These variables will retain their value between calls to the subroutine or function (unlike variables created using the LOCAL command).<br>This command uses exactly the same syntax as DIM. The only difference is that the length of the variable name created by STATIC and the length of the subroutine or function name added together cannot exceed 31 characters.<br>Static variables can be initialised to a value. This initialisation will take effect only on the first call to the subroutine (not on subsequent calls). |
| STEPPER | RP2350 VERSIONS ONLY This command provides a comprehensive system for controlling up to 3 stepper motor axes (X, Y, Z) with support for G-code execution, acceleration planning, and hardware limit switches. |

| | |
|---|---|
| | This is described in detail in the file *Stepper_Reference.pdf* included in the firmware download zip file. |
| STRUCT | NOT PICOMITEMIN This command allows the manipulation of structures (also known as a User-Defined Types) which allow related variables of different types to be grouped under a single name.<br>This is described in full in the file *MMBasic_Structures_Manual.pdf* included in the firmware download zip file. |
| SUB xxx (arg1 [,arg2, …])<br><statements><br><statements><br>END SUB | Defines a callable subroutine. This is the same as adding a new command to MMBasic while it is running your program.<br>'xxx' is the subroutine name and it must meet the specifications for naming a variable.<br>'arg1', 'arg2', etc are the arguments or parameters to the subroutine. An array is specified by using empty brackets. i.e. arg3(). The type of the argument can be specified by using a type suffix (i.e. arg1$) or by specifying the type using `AS <type> (i.e. arg1 AS STRING).`<br>Arguments in the caller's list that are a variable and have the correct type will be passed by reference to the subroutine. This means that any changes to the corresponding argument in the subroutine will also be copied to the caller's variable and therefore may be accessed after the subroutine has ended. The argument can be prefixed with BYVAL which will prevent this mechanism and cause only the value to be used. Alternatively, the prefix BYREF instructs MMBasic that a reference is required and an error will be generated if that cannot be done.<br>Arrays are passed by specifying the array name with empty brackets (eg, arg()) and are always passed by reference and must be the correct type.<br>Every definition must have one END SUB statement. When this is reached the program will return to the next statement after the call to the subroutine. The command EXIT SUB can be used for an early exit.<br>You use the subroutine by using its name and arguments in a program just as you would a normal command. For example: `MySub a1, a2`<br>When the subroutine is called each argument in the caller is matched to the argument in the subroutine definition. These arguments are available only inside the subroutine. Subroutines can be called with a variable number of arguments. Any omitted arguments in the subroutine's list will be set to zero or a null string.<br>Brackets around the argument list in both the caller and the definition are optional. |
| SYNC time% [,period]<br>or<br>SYNC | The SYNC command allows the user to implement very precisely timed repeated actions (1-2 microseconds accuracy).<br>To enable this the command is first called with the parameter time%. This sets up a repeating clock for time% microseconds. The optional parameter 'period' modifies the time and can be "U" for microseconds, "M" for milliseconds or "S" for seconds.<br>Once the clock is set up the program is synchronised to it using the SYNC command without parameters. This waits for the clock period to expire. For periods below 2 ms this is non-interruptible. Above 2 ms the program will respond to Ctrl-C but not any MMBasic interrupts.<br>Typical use is to set the clock outside of a loop and then at the top of the loop call the SYNC command without parameters. This means the contents of the loop will be executed exactly once for each clock period set. For example, the following would drive a servo with the required precise 50Hz timing: |

| | |
|---|---|
| | SYNC 20, M<br>DO<br>  SYNC<br>  PULSE GP0,n<br>LOOP |
| TEMPR START pin [, precision] [,timeout] | This command can be used to start a conversion running on a DS18B20 temperature sensor connected to 'pin'. Normally the TEMPR() function alone is sufficient to make a temperature measurement so usage of this command is optional. For more detail see the section *Measuring Temperature*.<br>This command will start the measurement on the temperature sensor. The program can then attend to other duties while the measurement is running and later use the TEMPR() function to get the reading. If the TEMPR() function is used before the conversion time has completed the function will wait for the remaining conversion time before returning the value.<br>Any number of these conversions (on different pins) can be started and be running simultaneously.<br>'precision' is the resolution of the measurement and is optional. It is a number between 0 and 3 meaning:<br>0 = 0.5ºC resolution, 100 ms conversion time.<br>1 = 0.25ºC resolution, 200 ms conversion time (this is the default).<br>2 = 0.125ºC resolution, 400 ms conversion time.<br>3 = 0.0625ºC resolution, 800 ms conversion time.<br>The optional timeout parameter overrides the conversion times above to allow for slow devices. |
| TEXT x, y, string$ [,alignment$] [, font] [, scale] [, c] [, bc] | Displays a string on the video output or attached LCD panel starting at 'x' and 'y'.<br>‘string$’ is the string to be displayed. Numeric data should be converted to a string and formatted using the Str$() function.<br>' alignment$' is a string expression or string variable consisting of 0, 1 or 2 letters where the first letter is the horizontal alignment around 'x' and can be L, C or R for LEFT, CENTER, RIGHT and the second letter is the vertical alignment around 'y' and can be T, M or B for TOP, MIDDLE, BOTTOM. The default alignment is left/top.<br>For example. “CM” will centre the text vertically and horizontally.<br>The 'alignment$' string can be a constant (eg, “CM”) or it can be a string variable. For backwards compatibility with earlier versions of MMBasic the string can also be unquoted (eg, CM).<br>A third letter can be used in the alignment string to indicate the rotation of the text. This can be 'N' for normal orientation, 'V' for vertical text with each character under the previous running from top to bottom, 'I' the text will be inverted (i.e. upside down), 'U' the text will be rotated counter clockwise by 90º and 'D' the text will be rotated clockwise by 90º<br>'font' and 'scale' are optional and default to that set by the FONT command.<br>'c' is the drawing colour and 'bc' is the background colour. They are optional and default to the current foreground and background colours.<br>See the chapter *Graphics Commands and Functions* for a definition of the colours and graphics coordinates. |

| | |
|---|---|
| TILE x, y [,foreground] [,background] [,nbr_tiles_wide] [,nbr_tiles_high]<br><br>TILE HEIGHT n | <u>VGA AND HDMI VERSIONS ONLY, IN MODE 1 ONLY</u><br>Sets the colour for one or more tiles on the screen.<br>When in monochrome mode by default the video screen is split up into 80x40 tiles each 8x12 pixels. This matches font 1 and allows full colour coding in the editor in monochrome mode.<br>Each tile can have a different foreground and background named colour assigned to it from the following: white, yellow, lilac, brown, fuchsia, rust, magenta, red, cyan, green, cerulean, midgreen, cobalt, myrtle, blue and black.<br>'x' and 'y' are the coordinates of the start block (0-79, 0-39)<br>'foreground ' and 'background' are the new colours selected.<br>'nbr_tiles_wide' and 'nbr_tiles_high' are the number of tiles to change.<br>The change is instant and does not affect the text or graphics currently displayed in the tiles (just the colours).<br>Sets the height of the tiles. 'n' can be between 12 and 480 (RP2040) or between 8 and 480 (RP2350) |
| TIME$ = "HH:MM:SS"<br>or<br>TIME$ = "HH:MM"<br>or<br>TIME$ = "HH" | Sets the time of the internal clock. MM and SS are optional and will default to zero if not specified. For example TIME$ = "14:30" will set the clock to 14:30 with zero seconds.<br>With OPTION RTC AUTO ENABLE the PicoMite firmware starts with the TIME$ programmed in RTC. Without OPTION RTC AUTO ENABLE the firmware starts with TIME$="00:00:00" |
| TILEMAP | TILEMAP is a powerful command for writing games. See the separate TILEMAP_User_Manual.pdf for more details |
| TIMER = msec | Resets the timer to a number of milliseconds. Normally this is just used to reset the timer to zero but you can set it to any positive number.<br>See the TIMER function for more details. |
| TMC22xx pin, chip, address, current, holdpct, microsteps [,rsense] [,stealthchop]<br><br>TMC22xx SGTHRS address, value<br><br>TMC22xx TCOOLTHRS address, value<br><br>TMC22xx CLOSE | <u>RP2350 VERSIONS ONLY</u><br>Configures Trinamic TMC2208/TMC2209 stepper motor driver chips via their single wire UART interface. 'pin' is the I/O pin connected to the driver's UART input, 'chip' is 2208 or 2209 and 'address' is the driver's address (0 for the TMC2208, 0 to 3 for the TMC2209). 'current' is the motor run current in mA (up to 2000), 'holdpct' is the hold current as a percentage of the run current and 'microsteps' is 1, 2, 4, 8, 16, 32, 64, 128 or 256. 'rsense' is the sense resistor value in ohms (default 0.11) and 'stealthchop' selects quiet StealthChop (1, the default) or SpreadCycle (0) operation.<br>TMC22xx SGTHRS sets the TMC2209's StallGuard threshold (0 to 255) and TMC22xx TCOOLTHRS sets its CoolStep/StallGuard velocity threshold - both can be used at any time for tuning. TMC22xx CLOSE releases the UART pin.<br>Normally used together with the STEPPER command and described in more detail in the file Stepper_Reference.pdf included in the firmware download zip file. |
| TRACE ON<br>or<br>TRACE OFF<br>or<br>TRACE LIST nn | TRACE ON/OFF will turn on/off the trace facility. This facility will print the number of each line (counting from the beginning of the program) in square brackets as the program is executed. This is useful in debugging programs.<br>TRACE LIST will list the last 'nn' lines executed in the format described above. MMBasic is always logging the lines executed so this facility is always available (ie, it does not have to be turned on). |
| TRIANGLE X1, Y1, X2, Y2, X3, Y3 [, C [, FILL]] | Draws a triangle on the attached video output or LCD display panel with the corners at X1, Y1 and X2, Y2 and X3, Y3. 'C' is the colour of the triangle and |

| | |
|---|---|
| | defaults to the current foreground colour. 'FILL' is the fill colour and defaults to no fill (it can also be set to -1 for no fill).<br>All parameters can be expressed as arrays and the software will plot the number of triangles as determined by the dimensions of the smallest array unless X1 = Y1 = X2 = Y2 = X3 = Y3 = -1 in which case processing will stop at that point 'x1', 'y1', 'x2', 'y2', 'x3',and 'y3' must all be arrays or all be single variables /constants otherwise an error will be generated 'c' and 'fill' can be either arrays or single variables/constants. |
| TRIANGLE SAVE [#]n, x1,y1,x2,y2,x3,y3<br><br>TRIANGLE RESTORE [#]n | Saves a triangular area of the screen to buffer #n.<br><br>Restores a saved triangular region of the screen and deletes the saved buffer. |
| TURTLE | NOT PICOMITEMIN OR WEBMITE The firmware implements a full turtle graphics engine. See Appendix H |
| TYPE struct-name<br>struct-member1, struct-member2<br>…<br>END TYPE | NOT PICOMITEMIN Define a structure (also known as a User-Defined Type) which allows related variables of different types to be grouped under a single name.<br>This is described in full in the file *MMBasic_Structures_Manual.pdf* included in the firmware download zip file. |
| UPDATE FIRMWARE | Causes the PicoMite firmware to enter the firmware update mode (the same as applying power while holding down the BOOTSEL button). This command is only available at the command prompt. |
| VAR SAVE var [, var]…<br>or<br>VAR RESTORE<br>or<br>VAR CLEAR | VAR SAVE will save one or more variables to non-volatile flash memory where they can be restored later (normally after a power interruption).<br>'var' can be any number of numeric or string variables and/or arrays. Arrays are specified by using empty brackets. For example: var()<br>The VAR SAVE command can be used repeatedly. Variables that had been previously saved will be updated with their new value and any new variables (not previously saved) will be added to the saved list for later restoration.<br>VAR RESTORE will retrieve the previously saved variables and insert them (and their values) into the variable table.<br>VAR CLEAR will erase all saved variables.<br>This command is normally used to save calibration data, options, and other data which does not change often but needs to be retained across a power interruption. Normally the VAR RESTORE command is placed at the start of the program so that previously saved variables are restored and immediately available to the program when it starts. Notes:<br>• The storage space available to this command is 16KB.<br>• Using VAR RESTORE without a previous save will have no effect and will not generate an error.<br>• If, when using RESTORE, a variable with the same name already exists its value will be overwritten.<br>• Saved arrays must be declared (using DIM) before they can be restored.<br>• Be aware that string arrays can rapidly use up all the memory allocated to this command. The LENGTH qualifier can be used when a string array is declared to reduce the size of the array (see the DIM command). This is not needed for ordinary string variables.<br>• The saved variables will be automatically cleared by a firmware upgrade, by the NEW command or when a new program is loaded via AUTOSAVE, XMODEM, etc. |

| | |
|---|---|
| WATCHDOG timeout<br>or<br>WATCHDOG OFF<br>or<br>WATCHDOG HW timeout<br>or<br>WATCHDOG HW OFF | Starts the watchdog timer which will automatically restart the processors when it has timed out. This can be used to recover from some event that disabled the running program (such as an endless loop or a programming or other error that halts a running program). This can be important in an unattended control situation.<br>The timeout can either be processed in the system timer interrupt (WATCHDOG command) or as a true CPU/hardware watchdog (WATCHDOG HW command).<br>If the hardware watchdog is used the timer has a maximum of 8.3 seconds. No such limitation exists for the software watchdog.<br>'timeout' is the time in milliseconds (ms) before a restart is forced. This command should be placed in strategic locations in the running BASIC program to constantly reset the watchdog timer (to 'timeout') and therefore prevent it from counting down to zero.<br>If the timer count does reach zero (perhaps because the BASIC program has stopped running) the PicoMite firmware will be automatically restarted and the automatic variable MM.WATCHDOG will be set to true (i.e. 1) indicating that an error occurred. On a normal startup MM.WATCHDOG will be set to false (i.e. 0). Note that OPTION AUTORUN must be specified for the program to restart.<br>WATCHDOG OFF can be used to disable the watchdog timer (this is the default on a reset or power up). The timer is also turned off when the break character (CTRL-C) is used on the console to interrupt a running program. |
| WII [CLASSIC] OPEN [,interrupt]<br><br><br><br><br><br>WII [CLASSIC] CLOSE | Opens a WII Classic controller and implements background polling of the device. The Wii Classic must be wired to the pins specified by OPTION SYSTEM I2C which is a prerequisite.<br>Open attempts to talk to the Wii Classic and will return an error if not found. If found the firmware will sample the Wii data in the background at a rate of 50Hz. If an optional user interrupt is specified this will be triggered if any of the buttons changes (both on and off)<br>See the DEVICE function for how to read data from the Wii Classic.<br><br>CLOSE will stop the background polling and disable any interrupt specified |
| WII NUNCHUCK OPEN [,interrupt]<br><br><br><br><br><br>WII NUNCHUCK CLOSE | Opens a WII Nunchuck controller and implements background polling of the device. The Wii Nunchuck must be wired to the pins specified by OPTION SYSTEM I2C which is a prerequisite.<br>Open attempts to talk to the Wii Nunchuck and will return an error if not found. If found the firmware will sample the Wii data in the background at a rate of 50Hz. If an optional user interrupt is specified this will be triggered if either of the buttons changes (both on and off)<br>See the DEVICE function for how to read data from the Wii Nunchuck.<br><br>CLOSE will stop the background polling and disable any interrupt specified |
| WEB<br><br>WEB CONNECT [ssid$, passwd$, [name$] [,ipaddress$, mask$, gateway$] [,country$]] | <u>WIFI VERSIONS ONLY</u><br>The WEB commands are used to manage the Internet capability of the WebMite.<br><br>This command, with no optional parameters, will connect to the default network if possible (as previously set with OPTION WIFI) or with the optional parameters will connect to the network specified and also set up the OPTION WIFI for future use.<br>An optional two letter ISO country code can be given as the last parameter to set the WiFi regulatory domain - see OPTION WIFI for the details. |

| | |
|---|---|
| WEB MQTT CONNECT addr$, port, user$, passwd$ [, interrupt] | Connect to an MQTT Broker.<br>'addr$' is the IP address, 'port' is the port number to use, 'user$' is the user name, 'passwd$' is the account's password and 'interrupt' is optional and if specified is the subroutine to call when a message is received.<br>WEB CONNECT does not disconnect from a previously connected network so should only be used where nothing has been previously set up or where a previously configured network is not active or a previously configured network has failed to connect on boot (no parameters)<br>On the RP2350 WebMite, specifying port 8883 (the standard secure MQTT port) automatically makes the connection using TLS. Any verification mode set with WEB TLS CA also applies to this connection. |
| WEB MQTT PUBLISH topic$, msg$, [,qos] [,retain] | Publish content to an MQTT broker topic.<br>'topic$' is the topic name and 'msg$' is the message/<br>'qos' is the optional quality of service with values of 0, 1 or 2 (default is 1). |
| WEB MQTT SUBSCRIBE topic$ [,qos] | Subscribe to an MQTT broker topic.<br>'topic$' is the topic name and 'qos' is the optional quality of service with values of 0, 1 or 2 (default is 1). |
| WEB MQTT UNSUBSCRIBE topic$ | Unsubscribe from an MQTT broker topic.<br>'topic$' is the topic name. |
| WEB MQTT CLOSE | Close a persistent MQTT Connection. |
| WEB NTP [timeoffset [, NTPserver$]] [,timeout]]] | Get the date/time from an NTP server and set the internal WebMite date/time clock.<br>' timeoffset' is the local time zone, and if omitted the date/time will be set to GMT. ' NTPserver$' is the timeserver to use and if omitted will default to an international timeserver pool. 'timeout' is the optional time out in milliseconds and defaults to 5000. |
| WEB OPEN TCP CLIENT address$, port [,timeout] | Opens a TCP client connection to a WEB server.<br>'address$' is a string and is the address of the server to connect to. It can be either a URL (eg, "api.openweathermap.org") or an IP address (eg, "192.168.1.111"). 'port' is the number of the port to use.<br>Used with WEB TCP CLIENT REQUEST to interrogate the server.<br>Note that one CLIENT connection is allowed. |
| WEB OPEN TCP STREAM address$, port [,timeout] | Opens a TCP client connection to a WEB server like WEB OPEN TCP CLIENT but connects the WEB TCP CLIENT STREAM receiver logic rather than the logic for WEB TCP CLIENT REQUEST.<br>'address$' is a string and is the address of the server to connect to. It can be either a URL (eg, "api.openweathermap.org") or an IP address (eg, "192.168.1.111"). 'port' is the number of the port to use.<br>Note that one CLIENT connection is allowed. |

| | |
|---|---|
| WEB OPEN TLS CLIENT address$, port [,timeout] | Opens a secure (TLS) client connection to a remote server, eg, an https web server (normally port 443). The parameters and use are the same as WEB OPEN TCP CLIENT: use WEB TCP CLIENT REQUEST to communicate with the server and WEB CLOSE TCP CLIENT to close the connection. 'timeout' is the optional connection and handshake timeout in milliseconds (default 5000).<br>If 'address$' is a domain name (not a numeric IP address) it is also sent to the server during the handshake so that servers hosting several web sites will return the correct certificate.<br>By default the connection is encrypted but the identity of the server is not verified - see WEB TLS CA. |
| WEB OPEN TLS STREAM address$, port [,timeout] | Opens a secure (TLS) connection like WEB OPEN TLS CLIENT but connects the streaming receiver logic for use with WEB TCP CLIENT STREAM (see WEB OPEN TCP STREAM). |
| WEB PING address$ [, count [, timeout [, avgvar]]] | Pings a remote host using ICMP echo requests, printing a line for each reply or timeout followed by a summary.<br>‘address$’ is a hostname or a dotted IP address. ‘count’ is the number of pings to send (default 4, maximum 100). ‘timeout’ is the time in milliseconds to wait for each reply (default 1000, minimum 100, maximum 10000).<br>‘avgvar’ is an optional floating-point variable which receives the average round trip time in milliseconds, or -1 if no replies were received, allowing a program to test connectivity without parsing the printed output. |
| WEB SCAN [array%()] | Scans for all available wifi connections. If ‘array%()’ is specified the output will be stored in a Longstring, otherwise output will be to the console. The command can be used whether ot not an network connection is already active. |
| WEB TCP CLIENT READ buff%() [,timeout] | Wait for and read the next data sent by the remote server opened with WEB OPEN TCP CLIENT, without first transmitting anything.<br>'buff%()' is an integer array which will receive the response as a LONGSTRING. The size of this buffer will limit the amount of data received from the server.<br>'timeout' is the optional time out in milliseconds and defaults to 5000. If no data is received before the timeout an error will occur.<br>This is typically used to read a server's unsolicited greeting (for example the SMTP 220 banner) before sending the first command, so that the WebMite is not treated as an 'early talker' by servers that delay or withhold their greeting.<br>On a TLS connection a server that sends its greeting immediately (for example SMTP over TLS) does so during WEB OPEN TLS CLIENT, before control returns; that greeting is consumed there and cannot be read with this command, so send the first command (eg, EHLO) directly. |
| WEB TCP CLIENT REQUEST request$, buff%() [,timeout] | Send a request to the remote server opened with WEB OPEN TCP CLIENT and wait for an answer.<br>'request$' is a string and is the request to be sent to the server. 'buff%()' is an integer array which will receive the response as a LONGSTRING. The size of this buffer will limit the amount of data received from the server.<br>'timeout' is the optional time out in milliseconds and defaults to 5000.<br>If the request times out an error will occur, otherwise the received data will be saved in the LONGSTRING 'buff%()'. If the received data is a JSON string then the JSON$() function can be used to parse it. |
| WEB TCP CLIENT STREAM command$, buffer%(), readpointer%, writepointer% | Connects to a server previously opened with WEB OPEN TCP STREAM.<br>'command$' is a string and is the request to be sent to the server. |

| | |
|---|---|
| | 'buffer%()' is an integer array which will receive the ongoing responses and acts as a circular buffer of bytes received.<br>The firmware maintains the parameter 'writepointer%' as the data from the server arrives.<br>'readpointer%' should be maintained by the Basic program as it removes data from the circular buffer.<br>If 'writepointer%' catches up with 'readpointer%' then 'readpointer%' will be incremented to stay one byte ahead and incoming data will be lost.<br>This command is designed to be compatible with the PLAY STREAM command to allow the implementation of streaming internet audio.<br>'buffer%()', 'readpointer%' and 'writepointer%' must be global (or STATIC), not LOCAL, as they are used in the background. |
| WEB TCP CLIENT WRITE data%() [,timeout] | Send the entire contents of a LONGSTRING to the remote server opened with WEB OPEN TCP CLIENT, without waiting for a reply.<br>'data%()' is an integer array holding the data to be sent as a LONGSTRING. Unlike WEB TCP CLIENT REQUEST there is no 255 character limit and the data is sent in as many TCP segments as required.<br>'timeout' is the optional time out in milliseconds and defaults to 10000.<br>This is much faster than sending a large payload as many small WEB TCP CLIENT REQUEST commands because there is no per request round trip to the server. It is useful for streaming large amounts of data such as a Base64 encoded email attachment. Use a following WEB TCP CLIENT REQUEST or WEB TCP CLIENT READ to read the server's response. |
| WEB CLOSE TCP CLIENT | Closes the connection to the remote server opened with WEB OPEN TCP CLIENT. This must be done before another open is attempted. |
| WEB TCP INTERRUPT InterruptSub | Start the TCP server running. 'InterruptSub' is the subroutine to call when a request is made of the TCP server (ie, an interrupt).<br>Note that the OPTION WIFI command must have been used first followed by the OPTION TCP SERVER PORT command to enable the TCP server. |
| WEB TLS CA file$ | WEB TLS CA loads a bundle of trusted root certificates (PEM format) from the file 'file$' and turns on server certificate verification for all subsequent TLS connections (including MQTT over TLS). The clock must be set first (eg, with WEB NTP) as verification checks the certificate expiry dates. A suitable bundle of the common root certificates (cacert.pem) can be downloaded from https://curl.se/docs/caextract.html. |
| WEB TLS NOVERIFY | WEB TLS NOVERIFY removes the loaded certificates and returns to the default in which connections are encrypted but the server is not verified. |
| WEB TCP READ cb%, buff%() | Read the data from a potential TCP connection' cb%'. ' buff%()' is an array to receive any data from that connection as a longstring. The size of this buffer will limit the amount of data received from the remote client. If there is nothing received on that connection this will return an empty string (ie, `LLEN(buff%())=0`).<br>If there is data that has been received then the BASIC program must respond with one of the WEB TRANSMIT commands in order to respond and close the connection. |
| WEB TCP SEND cb%, data%()<br>WEB TCP CLOSE cb% | These two commands allow more flexibility in using the TCP server. Unlike WEB TRANSMIT PAGE or WEB TRANSMIT FILE, WEB TCP SEND does not create any sort of header, nor does it close the TCP connection after transmission. It just sends exactly what is in the LONGSTRING data%() and it is up to the Basic programmer to close the connection when appropriate. |

| | |
|---|---|
| WEB TRANSMIT CODE cb%, nnn% | Send a numerical response to the open TCP connection 'cb%' and then closes the connection.<br>Typical use would be `TRANSMIT CODE cb%, 404` to indicate page not found. |
| WEB TRANSMIT FILE cb%, filename$, content-type$ | Constructs an HTTP 1.1 header with the ’content-type$’ as specified, sends it and then sends the contents of the file to the open TCP connection cb% and on completion, closes the connection.<br>’content-type$’ is a MIME type expressed as a string. Eg, "image/jpeg" |
| WEB TRANSMIT PAGE cb%, filename$ [,buffersize] | Constructs an HTTP 1.1 header, sends it and then sends the contents of the file to the open TCP connection cb% and on completion closes the connection.<br>MMBasic will substitute current values for any MMBasic variables or expressions defined in the file inside curly brackets eg, {myvar%}. Variables can be simple, array elements or expressions.<br>An opening curly bracket can be included in the output by using {{.<br>By default the command allocates a buffer the size of the file + 4096 bytes to build the page to transmit. However, if the page is complex and includes many MMBasic variables that yield text bigger than the variable name it is possible that the buffer will not be big enough. In this case the user can specify the extra space required (defaults to 4096 if not specified) |
| WEB UDP INTERRUPT intname | Sets up a BASIC interrupt routine that will be triggered whenever a UDP datagram is received. The contents will be saved in MM.MESSAGE$. The IP address of the sender will be stored in MM.ADDRESS$.<br>WEB UDP INTERRUPT 0 cancels the interrupt. |
| WEB UDP SEND addr$, port, data$ | Used to send a datagram to a remote receiver. In this case the IP address must be specified and can be either a numeric address (eg, "192.168.1.147") or a normal text address (eg, "google.com"). The port number of the receiver must also be specified and the message itself. The SEND command can be used as a response to an incoming message or stand-alone. |
| WS2812 type, pin, nbr, value%[()] | This command will drive one or more WS2812 LED chips connected to 'pin'. Note that the pin must be set to a digital output before this command is used.<br>'type' is a single character specifying the type of chip being driven:<br>O = original WS2812<br>B = WS2812B<br>S = SK6812<br>W =SK6812W (RGBW)<br>‘nbr’ is the number of LEDs in the chain (1 to 256). The 'value%()' array should be an integer array sized to have exactly the same number of elements as the number of LEDs to be driven.<br>For the first three variants each element in the array should contain the colour in the normal RGB888 format (i.e. 0 to &HFFFFFF).<br>For type W use a RGBW value (0-&HFFFFFFFF).<br>If only one LED is connected then a single integer should be used for ‘value%’ (ie, not an array). |
| XMODEM SEND<br>or<br>XMODEM SEND file$<br>or<br>XMODEM RECEIVE | Transfers a BASIC program to or from a remote computer using the XModem protocol. The transfer is done over the USB console connection.<br>The ZMODEM command does the same job faster, and recovers from transmission errors.<br>XMODEM SEND will send the current program held in the PicoMite's program memory to the remote device. |

| | |
|---|---|
| or<br>XMODEM RECEIVE file$<br>or<br>XMODEM CRUNCH | XMODEM RECEIVE will accept a program sent by the remote device and save it into the PicoMite's the program memory overwriting the program currently held there.<br>In both cases you can also specify 'file$' which will transfer the data to/from a file on the Flash Filesystem or SD Card. If the file already exists it will be overwritten when receiving a file.<br>Note that the data is buffered in RAM which limits the maximum transfer size. This command also creates a backup of the program in flash memory which will be automatically retrieved if the CPU is reset of the power is lost.<br>The CRUNCH option works like RECEIVE but will remove all comments, blank lines and unnecessary spaces from the program before saving. This can be used on large programs to allow them to fit into limited memory.<br>SEND, RECEIVE and CRUNCH can be abbreviated to S, R and C.<br>The XModem protocol requires a cooperating software program running on the remote computer and connected to its serial port. It has been tested on Tera Term running on Windows and it is recommended that this be used.<br>After running the XMODEM command in MMBasic select:<br>File -> Transfer -> XMODEM -> Receive/Send<br>from the Tera Term menu to start the transfer.<br>The transfer can take up to 15 seconds to start and if the XMODEM command fails to establish communications it will return to the MMBasic prompt after 60 seconds and leave the program memory untouched.<br>Download Tera Term from http://ttssh2.sourceforge.jp/ |
| YMODEM SEND<br>or<br>YMODEM SEND file$<br>or<br>YMODEM RECEIVE<br>or<br>YMODEM RECEIVE file$<br>or<br>YMODEM CRUNCH | RP2350 NON-USB VERSIONS ONLY<br>This command mimics XMODEM but transfers files much faster (>10x) over a CDC serial link<br>YMODEM is different from XMODEM in that the message carries the filename.<br>This means that when you send a file from the Pico to a computer the receiving program will use automatically use the same name as you send.<br>In case the file already exists the receiver will automatically increment a version number on the name of the file received.<br>fred1.bas, fred2.bas etc.<br>When sending from memory with YMODEM S there are two possibilities.<br>In the case that the file was loaded from the A: or B: drives it will contain a hidden filename and this will be used.<br>In the case of a file created through the editor or autosaved the receiving file will automatically have the name FILEn.DAT<br>It is the responsibility of the receiving program to specify where the file will be saved.<br>In the case of teraterm this is an option you set on the general setting screen "File transfer folder"<br>When receiving a file on the Pico, you have complete freedom to specify the name and directory in the normal way<br>NB: YMODEM is very sensitive to the quality of the connection and the USB implementation on the host. If it doesn't work for you continue to use XMODEM<br>The ZMODEM command does the same job and also recovers from transmission errors. |
| ZMODEM SEND<br>or<br>ZMODEM SEND file$<br>or<br>ZMODEM RECEIVE | Transfers a BASIC program or a file to or from a remote computer using the ZModem protocol. It works like XMODEM but is much faster, and it recovers from transmission errors: every block of data is checked and if any is lost or damaged the receiver asks for it again from that point, so a poor connection slows a transfer down rather than stopping it. The transfer is made over the console, either the USB console or a serial console. |

| | |
|---|---|
| or<br>ZMODEM RECEIVE file$<br>or<br>ZMODEM RECEIVE "*"<br>or<br>ZMODEM CRUNCH | ZMODEM SEND will send the program held in program memory. If the program was loaded from the Flash Filesystem or SD Card the remote computer is given the name of that file, otherwise it is called FILE.DAT.<br>ZMODEM RECEIVE will accept a program sent by the remote computer and save it into program memory, overwriting the program currently held there.<br>In both cases you can also specify 'file$' which will transfer the data to or from a file on the Flash Filesystem or SD Card. When sending, the remote computer is given the name of the file without its drive or directory. When receiving, the name sent by the remote computer is ignored and the data is saved in 'file$', overwriting any file of that name. If the remote computer sends several files only the first is received, unless 'file$' is "*" (see below). ZMODEM SEND file$ and ZMODEM RECEIVE file$ can also be used in a program.<br>ZMODEM RECEIVE "*" will accept any number of files and save each one in the current directory under the name given by the remote computer (without any directory), overwriting any file of that name. A directory can be put in front of the *, for example ZMODEM RECEIVE "B:/downloads/*". In Tera Term select all the files to be sent in the ZMODEM Send dialog; with lrzsz list them all, for example sz *.bas. When the transfer is complete the number of files and bytes received is reported. If an error stops the transfer part way through, the files received before it are complete and the error message says how many there were.<br>The CRUNCH option works like RECEIVE but will remove all comments, blank lines and unnecessary spaces from the program before saving it.<br>SEND, RECEIVE and CRUNCH can be abbreviated to S, R and C.<br>A program received into program memory is buffered in RAM, which limits its size. If it is too large the transfer stops with "Not enough memory" and the program in memory is left as it was.<br>The ZModem protocol requires a cooperating program on the remote computer. On Windows Tera Term is recommended: after running the ZMODEM command select File -> Transfer -> ZMODEM -> Send or Receive from the Tera Term menu. On Linux and macOS use sz (to send) and rz (to receive) from the lrzsz package, run with the serial port as their input and output, for example from a terminal program such as minicom.<br>If the remote computer does not start the transfer within 60 seconds the command returns to the MMBasic prompt with an error. If the remote computer cancels the transfer the error is "Cancelled by remote".<br>Over the USB console a 512 KB file takes around 2 seconds on the RP2350 and 3 seconds on the RP2040 with Tera Term. Over a serial console the speed depends on the baud rate (see OPTION BAUDRATE). Some USB to serial adapters (eg, the CH340) lose data sent to the computer at 921600 baud; ZModem recovers, but the transfer is faster at 750000 baud. |

# Functions

Note that the functions related to communications functions ($I^2C$, 1-Wire, and SPI) are not listed here but are described in the appendices at the end of this document.

Square brackets indicate that the parameter or characters are optional.

| | |
|---|---|
| ABS( number ) | Returns the absolute value of the argument 'number' (i.e. any negative sign is removed and a positive number is returned). |
| ACOS( number ) | Returns the inverse cosine of the argument 'number' in radians. |
| ASC( string$ ) | Returns the ASCII code (i.e. byte value) for the first letter in ‘string$’. |
| ASIN( number ) | Returns the inverse sine value of the argument 'number' in radians. |
| ATN( number ) | Returns the arctangent of the argument 'number' in radians. |
| ATAN2( y, x ) | Returns the arc tangent of the two numbers x and y as an angle expressed in radians.<br>It is similar to calculating the arc tangent of y / x, except that the signs of both arguments are used to determine the quadrant of the result. |
| BASE$( base, number [, chars]) | Returns a string giving the value of 'number' in the number base 'base'. 'base' can be from 2 to 36 with the digits above nine represented by the letters A to Z.<br>'chars' is optional and specifies the number of characters in the string with zero as the leading padding character(s).<br>BIN$(), OCT$() and HEX$() are shortcuts for BASE$() with a base of 2, 8 and 16 respectively. Use BASE$() for any other base, for example BASE$(12, 100) will return 84. |
| BIN$( number [, chars]) | Returns a string giving the binary (base 2) value for the 'number'.<br>'chars' is optional and specifies the number of characters in the string with zero as the leading padding character(s). |
| BIN2STR$(type, value [,BIG]) | Returns a string containing the binary representation of 'value'.<br>'type' can be:<br>INT64 signed 64-bit integer converted to an 8 byte string<br>UINT64 unsigned 64-bit integer converted to an 8 byte string<br>INT32 signed 32-bit integer converted to a 4 byte string<br>UINT32 unsigned 32-bit integer converted to a 4 byte string<br>INT16 signed 16-bit integer converted to a 2 byte string<br>UINT16 unsigned 16-bit integer converted to a 2 byte string<br>INT8 signed 8-bit integer converted to a 1 byte string<br>UINT8 unsigned 8-bit integer converted to a 1 byte string<br>SINGLE single precision floating point number converted to a 4 byte string<br>DOUBLE double precision floating point number converted to a 8 byte string<br>By default the string contains the number in little-endian format (i.e. the least significant byte is the first one in the string). Setting the third parameter to ‘BIG’ will return the string in big-endian format (i.e. the most significant byte is the first one in the string). In the case of the integer conversions, an error |

| | |
|---|---|
| | will be generated if the 'value' cannot fit into the 'type' (eg, an attempt to store the value 400 in a INT8).<br>This function makes it easy to prepare data for efficient binary file I/O or for preparing numbers for output to sensors and saving to flash memory.<br>See also the function STR2BIN |
| BIT(var%, bitno) | 'returns the value of a specific bit (0-63) in an integer variable (0 or 1).<br>See also the BIT command |
| BOUND(array() [,dimension] | This returns the upper limit of the array for the dimension requested.<br>The dimension defaults to one if not specified. Specifying a dimension value of 0 will return the current value of OPTION BASE.<br>Requesting a dimension that the array does not have will generate an error, as will using BOUND() on a variable that is not an array. Note that zero is a valid upper limit: an array declared with a single element under OPTION BASE 0 (for example DIM a(0)) has an upper limit of zero.<br>For example:<br>DIM myarray(44,45)<br>BOUND(myarray(),2) will return 45 |
| BYTE(var$, byteno) | Returns the integer value of a specific byte in a string (0-255). This is the equivalent of ASC(MID$(var$,byteno,1)) but operates much faster.<br>See also the BYTE command |
| CALL(userfunname$,<br>[,userfunparameters,..]) | This is an efficient way of programmatically calling user defined functions. (See also the CALL command). In many cases it can be used to eliminate complex SELECT and IF THEN ELSEIF ENDIF clauses and is processed in a much more efficient manner.<br>'userfunname$' can be any string or variable or function that resolves to the name of a normal user function (not an in-built command).<br>'userfunparameters' are the same parameters that would be used to call the function directly.<br>A typical use for this command could be writing any sort of emulator where one of a large number of functions should be called depending on a some variable. It also provides a method of passing a function name to another subroutine or function as a variable. |
| CHOICE(condition,<br>ExpressionIfTrue,<br>ExpressionIfFalse) | This function allows you to do simple either/or selections more efficiently and faster than using IF THEN ELSE ENDIF clauses.<br>The condition is anything that will resolve to nonzero (true) or zero (false).<br>The expressions are anything that you could normally assign to a variable or use in a command and can be integers, floats or strings.<br>Examples:<br>PRINT CHOICE(1, "hello","bye") will print "Hello"<br>PRINT CHOICE (0, "hello","bye") will print "Bye"<br>a=1 : b=1 : PRINT CHOICE (a=b, 4, 5) will print 4 |
| CHR$( number ) | Returns a one-character string consisting of the character corresponding to the ASCII code (i.e. byte value) indicated by argument 'number'. |
| CLICK(DOWN)<br>or<br>CLICK(UP)<br>or<br>CLICK(X) | NOT RP2040 VGA, RP2040 WEBMITE OR PICOMITEMIN<br>Reports mouse click activity in the same way as the TOUCH() function reports touch (this function is compatible with the Colour Maximite 2). A click of the left mouse button and a touch on a touch sensitive LCD panel are treated identically, so CLICK() effectively substitutes for the TOUCH() function and the same program can be driven by either a mouse or a touch screen. |

| | |
|---|---|
| or<br>CLICK(Y)<br>or<br>CLICK(REF)<br>or<br>CLICK(LASTREF)<br>or<br>CLICK(LASTX)<br>or<br>CLICK(LASTY) | CLICK(DOWN) returns 1 (true) if the screen is currently being clicked on (the left mouse button is down) or touched. CLICK(UP) returns 1 (true) if it is not.<br>CLICK(X) and CLICK(Y) return the coordinates of the current click or -1 if the screen is not being clicked on.<br>CLICK(LASTX) and CLICK(LASTY) return the coordinates of the last location that was clicked on.<br>CLICK(REF) returns the reference number of the graphic control that is currently being clicked on, or zero if none. CLICK(LASTREF) returns the reference number of the last control that was clicked on. The graphic controls are described in Advanced Graphics Functions.pdf.<br>On the RP2040 VGA version CLICK is only available when USB is supported and is limited to CLICK(DOWN), CLICK(UP), CLICK(X) and CLICK(Y). |
| CINT( number ) | Round numbers with fractional portions up or down to the next whole number or integer.<br>For example, 45.47 will round to 45<br>45.57 will round to 46<br>-34.45 will round to -34<br>-34.55 will round to -35<br>See also INT() and FIX(). |
| COS( number ) | Returns the cosine of the argument 'number' in radians. |
| CWD$ | Returns the current working directory on the Flash Filesystem or SD Card. Invalid for exFAT format.<br>The format is: `A:/dir1/dir2`. |
| DATE$ | Returns the current date based on MMBasic's internal clock as a string in the form "DD-MM-YYYY". For example, "28-07-2012".<br>The internal clock/calendar will keep track of the time and date including leap years. To set the date use the command DATE$ =. |
| DATETIME$(n) | Returns the date and time corresponding to the epoch number 'n' (number of seconds that have elapsed since midnight GMT on January 1, 1970).<br>The format of the returned string is "dd-mm-yyyy hh:mm:ss". Use the text NOW to get the current datetime string, ie, DATETIME$(NOW) |
| DAY$(date$) | Returns the day of the week for a given date as a string.<br>For example, "Monday", "Tuesday" etc.<br>'date$' is a string and its format can be DD-MM-YY or DD-MM-YYYY or YYYY-MM-DD. You can also use NOW to get the day for the current date, eg, PRINT DAY$(NOW) |
| DEG( radians ) | Converts 'radians' to degrees. |
| DEVICE(GAMEPAD channel, funct) | Returns data from a USB PS3 or PS4 controller.<br>'funct' is a 1 or 2 letter code indicating the information to return as follows:<br>LX the position of the analog left joystick x axis<br>LY the position of the analog left joystick y axis<br>RX the position of the analog right joystick x axis<br>RY the position of the analog right joystick y axis<br>GX the reading from the X axis gyro (where supported)<br>GY the reading from the Y axis gyro (where supported)<br>GZ the reading from the Z axis gyro (where supported)<br>AX the reading from the X axis accelerometer (where supported)<br>AY the reading from the Y axis accelerometer (where supported) |

| | |
|---|---|
| | AZ the reading from the Z axis accelerometer (where supported)<br>L the position of the analog left button<br>R the position of the analog right button<br>B a bitmap of the state of all the buttons. A bit will be set to 1 if the button is pressed.<br>T the ID code of the controller<br>The button bitmap is as follows:<br>BIT 0 Button R/R1<br>BIT 1 Button start/options<br>BIT 2 Button home<br>BIT 3 Button select/share<br>BIT 4 Button L/L1<br>BIT 5 Button down cursor<br>BIT 6 Button right cursor<br>BIT 7 Button up cursor<br>BIT 8 Button left cursor<br>BIT 9 Right shoulder button 2/R2<br>BIT 10 Button x/triangle<br>BIT 11 Button a/circle<br>BIT 12 Button y/square<br>BIT 13 Button b/cross<br>BIT 14 Left should button 2/L2<br>BIT 15 Touchpad |
| DEVICE(MOUSE channel, funct) | Returns data from a mouse connected via 'channel'.<br>A PS2 mouse is always allocated channel 2. Normally a USB mouse is also allocated to channel 2 but this can vary. See MM.INFO(USB n) for more information.<br>'funct' is a 1 letter code indicating the information to return as follows:<br>X the X coordinate (0 to MM.HRES-1)<br>Y the Y coordinate (0 to MM.VRES-1)<br>L the state of the left mouse button<br>R the state of the right mouse button<br>M the state of the middle mouse button (wheel click)<br>D 1 if there has been a double click of the left mouse button<br>W gives the delta of the wheel position since the last call (PS2 mouse or USB mouse with OPTION MOUSE SENSITIVITY set)<br>B the state of all buttons as a bitmap (USB mouse only) |
| DEVICE(WII [CLASSIC] funct) | Returns data from a Wii Classic controller.<br>'funct' is a 1 or 2 letter code indicating the information to return as follows:<br>LX the position of the analog left joystick x axis<br>LY the position of the analog left joystick y axis<br>RX the position of the analog right joystick x axis<br>RY the position of the analog right joystick y axis<br>L the position of the analog left button<br>R the position of the analog right button<br>B a bitmap of the state of all the buttons. A bit will be set to 1 if the button is pressed.<br>T the ID code of the controller - should be hex &HA4200101<br>The button bitmap is as follows: |

| | |
|---|---|
| | BIT 0 Button R<br>BIT 1 Button start<br>BIT 2 Button home<br>BIT 3 Button select<br>BIT 4 Button L<br>BIT 5 Button down cursor<br>BIT 6 Button right cursor<br>BIT 7 Button up cursor<br>BIT 8 Button left cursor<br>BIT 9 Button ZR<br>BIT 10 Button x<br>BIT 11 Button a<br>BIT 12 Button y<br>BIT 13 Button b<br>BIT 14 Button ZL |
| DEVICE(NUNCHUCK funct) | Returns data from a Nunchuck controller.<br>'funct' is a 1 or 2 letter code indicating the information to return as follows:<br>AX the x axis acceleration<br>AY the y axis acceleration<br>AZ the z axis acceleration<br>JX the position of the joystick x axis<br>JY the position of joystick y axis<br>C the state of the C button<br>Z the state of the Z button<br>T the ID code of the controller - should be hex &HA4200000 |
| DIR$( fspec, type )<br>or<br>DIR$( fspec )<br>or<br>DIR$( ) | Will search the default Flash Filesystem or SD Card for files and return the names of entries found.<br>'fspec' is a file specification using wildcards the same as used by the FILES command. Eg, "*.*" will return all entries, "*.TXT" will return text files. Note that the wildcard *.* does not find files or folders without an extension.<br>'type' is the type of entry to return and can be one of:<br>ALL Search for all files and directories<br>DIR Search for directories only<br>FILE Search for files only (the default if 'type' is not specified)<br>The function will return the first entry found. To retrieve subsequent entries use the function with no arguments. i.e. DIR$( ). The return of an empty string indicates that there are no more entries to retrieve.<br>This example will print all the files in a directory:<br>`f$ = DIR$("*.*", FILE)`<br>`DO WHILE f$ <> ""`<br>`PRINT f$`<br>`f$ = DIR$()`<br>`LOOP`<br>You must change to the required directory before invoking this command. |
| DISTANCE( trigger, echo )<br>or<br>DISTANCE( trig-echo ) | Measure the distance to a target using the HC-SR04 ultrasonic distance sensor.<br>Four pin sensors have separate trigger and echo connections. 'trigger' is the I/O pin connected to the "trig" input of the sensor and 'echo' is the pin connected to the "echo" output of the sensor. |

| | |
|---|---|
| | Three pin sensors have a combined trigger and echo connection and in that case you only need to specify one I/O pin to interface to the sensor.<br>Note that the HC-SR04 is a 5V device so level shifting will be required on Pico (RP2040) processors but not on Pico 2 (RP2350) processors.<br>The I/O pins are automatically configured by this function and multiple sensors can be used on different I/O pins.<br>The value returned is the distance in centimetres to the target or -1 if no target was detected or -2 if there was an error (i.e. sensor not connected). |
| Draw3D(arg) | NOT PICOMITEMIN OR WEBMITE<br>The DRAW3D function can be used to determine the boundaries of a 3D object and could be used to define an area to be erased before re-drawing.<br>See the document "*3D_Graphics_User_Manual.pdf*" in the PicoMite firmware download for a full description. |
| EOF( [#]fnbr ) | Will return true if the file previously opened on the Flash Filesystem or SD Card for INPUT with the file number '#fnbr' is positioned at the end of the file.<br>The # is optional. Also see the OPEN, INPUT and LINE INPUT commands and the INPUT$ function. |
| EPOCH(DATETIME$) | Returns the epoch number (number of seconds that have elapsed since midnight GMT on January 1, 1970) for the supplied DATETIME$ string.<br>The format for DATETIME$ is "dd-mm-yyyy hh:mm:ss", "dd-mm-yy hh:mm:ss", or "yyyy-mm-dd hh:mm:ss",. Use NOW to get the epoch number for the current date and time, i.e. PRINT EPOCH(NOW) |
| EVAL( string$ ) | Will evaluate 'string$' as if it is a BASIC expression and return the result.<br>'string$' can be a constant, a variable or a string expression. The expression can use any operators, functions, variables, subroutines, etc that are known at the time of execution. The returned value will be an integer, float or string depending on the result of the evaluation.<br>For example: S$ = "COS(RAD(30)) * 100" : PRINT EVAL(S$)<br>Will display: 86.6025 |
| EXP( number ) | Returns the exponential value of 'number', i.e. e^x where x is 'number'. |
| FIELD$( string1, nbr, string2 [, string3] ) | Returns a particular field in a string with the fields separated by delimiters. Note that a space character cannot be used as a delimiter.<br>'nbr' is the field to return (the first is nbr 1). 'string1' is the string to search and 'string2' is a string holding the delimiters (more than one can be used). The space character may not be used as a delimiter.<br>'string3' is optional and if specified will include characters that are used to quote text in 'string1' (ie, quoted text will not be searched for a delimiter).<br>For example:<br>S$ = "foo, boo, zoo, doo"<br>r$ = FIELD$(s$, 2, ",")<br>will result in r$ = "boo". While:<br>s$ = "foo, 'boo, zoo', doo"<br>r$ = FIELD$(s$, 2, ",", "'")<br>will result in r$ = "boo, zoo". |
| FIX( number ) | Truncate a number to a whole number by eliminating the decimal point and all characters to the right of the decimal point.<br>For example 9.89 will return 9 and -2.11 will return -2. |

| | |
|---|---|
| | The major difference between FIX() and INT() is that FIX() provides a true integer function (i.e. does not return the next lower number for negative numbers as INT() does). This behaviour is for Microsoft compatibility.<br>See also CINT() . |
| FLAG(n%) | Returns the value (0 or 1) of the bit n% (0-63) in the system flag register.<br>See also MM.FLAGS and the FLAG and FLAGS commands |
| FORMAT$( nbr [, fmt$] ) | Will return a string representing 'nbr' formatted according to the specifications in the string 'fmt$'.<br>The format specification starts with a % character and ends with a letter. Anything outside of this construct is copied to the output as is.<br>The structure of a format specification is:<br>% [flags] [width] [.precision] type<br>Where 'flags' can be:<br>- Left justify the value within a given field width<br>0 Use 0 for the pad character instead of space<br>+ Forces the + sign to be shown for positive numbers<br>space Causes a positive value to display a space for the sign. Negative values still show the – sign<br>'width' is the minimum number of characters to output, less than this the number will be padded, more than this the width will be expanded.<br>'precision' specifies the number of fraction digits to generate with an e, or f type or the maximum number of significant digits to generate with a g type and defaults to 4 digits. If specified, the precision must be preceded by a dot (.).<br>'type' can be one of:<br>g Automatically format the number for the best presentation.<br>f Format the number with the decimal point and following digits<br>e Format the number in exponential format<br>If uppercase G or F is used the exponential output will use an uppercase E. If the format specification is not specified "%g" is assumed.<br>Examples: format$(45) will return 45<br>format$(45, "%g") will return 45 |
| GETSCANLINE | <u>VGA AND HDMI VERSIONS ONLY</u><br>This will report on the line that is currently being drawn on the VGA/HDMI monitor in the range of 0 to 525. This is irrespective of the current MODE.<br>Using this to time updates to the screen can avoid timing effects caused by updates while the screen is being updated.<br>The first visible line will return a value of 0. Any line number above 479 is in the frame blanking period. |
| GPS() | The GPS function will return data from a serial communications channel opened as GPS using OPEN GPS or the OPTION GPS commands.<br>This function is described in detail in the file *Option_GPS_User_Manual.pdf* included in the firmware download zip file.<br>The function GPS(VALID) should be checked before the data is accessed to ensure that the returned value is valid. |
| HEX$( number [, chars]) | Returns a string giving the hexadecimal (base 16) value for the 'number'.<br>'chars' is optional and specifies the number of characters in the string with zero as the leading padding character(s). |

| | |
|---|---|
| INKEY$ | Checks the console input buffer and, if there is one or more characters waiting in the queue, will remove the first character and return it as a single character in a string.<br>If the input buffer is empty this function will immediately return with an empty string (i.e. ""). |
| INPUT$(nbr, [#]fnbr) | Will return a string composed of 'nbr' characters read from a file or serial communications port opened as 'fnbr'. This function will return as many characters as are in the file or receive buffer up to 'nbr'. If there are no characters available it will immediately return with an empty string.<br>#0 can be used which refers to the console's input buffer.<br>The # is optional. Also see the OPEN command. |
| INSTR( [start-position,] string-searched$, string-pattern$ [,size] ) | Returns the position at which 'string-pattern$' occurs in 'string-searched$', beginning at 'start-position'. If 'start-position' is not provided it will default to 1.<br>Both the position returned and 'start-position' use 1 for the first character, 2 for the second, etc.<br>The function returns zero if 'string-pattern$' is not found.<br>If the optional parameter "size" is specified the "string-pattern" is treated as a regular expression. See *Appendix E* for the details. |
| INT( number ) | Truncate an expression to the next whole number less than or equal to the argument. For example 9.89 will return 9 and -2.11 will return -3.<br>This behaviour is for Microsoft compatibility, the FIX() function provides a true integer function. See also CINT() . |
| JSON$(array%(), string$) | WIFI VERSIONS ONLY Returns a string representing a specific item out of the JSON input stored in the longstring array%(). Note that many JSON data sets are quite large and may be too big to parse with the memory available.<br>Examples (taken from api.openweathermap.org):<br>JSON$(a%(), "name")<br>JSON$(a%(), "coord.lat")<br>JSON$(a%(), "weather[0].description")<br>JSON$(a%(),"list[4].weather[0].description |
| KEYDOWN(n) | Return the decimal ASCII value of the keyboard key that is currently held down or zero if no key is down. This function applies to USB keyboards, PS2 keyboards (in MMBasic 6.02.01 or later) and the PicoCalc's built-in keyboard (in MMBasic 6.03.02 or later).<br>Note that PS2 keyboards do not support the modifier keys in the same way as USB keyboards. PS2 keyboards do not distinguish between left and right ALT or CTRL keys and MMBasic cannot process the Windows and/or function select keys (if they exist).<br>On the PicoCalc the built-in keyboard reports multiple simultaneous key presses in the same way. The keyboard applies shift and caps lock itself, so KEYDOWN() returns the shifted character (65 rather than 97 for the 'a' key with shift held). KEYDOWN(7) reports left shift, right shift, CTRL and ALT only as the PicoCalc has no GUI or right ALT keys, and note that SHIFT-ENTER also reports as ALT. KEYDOWN(8) reports caps lock only as there is no NUM LOCK or SCROLL LOCK key.<br>The decimal values for the function and arrow keys are listed in Appendix I.<br>This function will report multiple simultaneous key presses and the parameter 'n' is the number of the keypress to report. KEYDOWN(0) will return the number of keys being pressed |

| | |
|---|---|
| | For example, if "c", "g" and "p" are pressed simultaneously KEYDOWN(0) will return 3, KEYDOWN(1) will return 99, KEYDOWN(2) will return 103, etc. The keys do not need to be pressed simultaneously and will report in the order pressed. Taking a finger off a key will promote the next key pressed to #1.<br>The first key ('n' = 1) is entered in the keyboard buffer (accessible using INKEY$) while keys 2 to 6 can only be accessed via this function. Using this function will clear the console input buffer.<br>KEYDOWN(7) will give any modifier keys that are pressed. These keys do not add to the count in keydown(0)<br>The return value is a bitmask as follows:<br>lalt = 1, lctrl = 2, lgui = 4, lshift = 8, ralt = 16, rctrl = 32, rgui = 64, rshift = 128<br>KEYDOWN(8) will give the current status of the lock keys. These keys do not add to the count in keydown(0)<br>The return value is a bitmask as follows:<br>caps_lock = 1, num_lock = 2, scroll_lock = 4<br>Note that some keyboards will limit the number of active keys that they can report on. |
| LCASE$( string$ ) | Returns ‘string$’ converted to lowercase characters. |
| LCOMPARE(array1%(), array2%()) | Compare the contents of two long string variables ‘array1%()’ and ‘array2%()’. The returned is an integer and will be -1 if ‘array1%()’ is less than ‘array2%()’. It will be zero if they are equal in length and content and +1 if ‘array1%()’ is greater than ‘array2%()’. The comparison uses the ASCII character set and is case sensitive. |
| LEFT$( string$, nbr ) | Returns a substring of ‘string$’ with ‘nbr' of characters from the left (beginning) of the string. |
| LEN( string$ ) | Returns the number of characters in 'string$'. |
| LGETBYTE(array%(), n) | Returns the numerical value of the 'n'th byte in the LONGSTRING held in 'array%()'. This function respects the setting of OPTION BASE in determining which byte to return.<br>LGETBYTE treats the LONGSTRING as a plain array of bytes and therefore honours OPTION BASE when interpreting 'n': with OPTION BASE 0 the first byte is number 0, while with OPTION BASE 1 the first byte is number 1.<br>Note that LINSTR and LGETSTR$ always work with 1-based positions (the first character is position 1) irrespective of OPTION BASE. Consequently, when OPTION BASE 0 is in effect, a position returned by LINSTR (or passed to LGETSTR$) refers to the byte that LGETBYTE would read at that position minus one. Subtract 1 from a LINSTR result before passing it to LGETBYTE under OPTION BASE 0 (under OPTION BASE 1 the values line up directly). |
| LGETSTR$(array%(), start, length) | Returns part of a long string stored in ‘array%()’ as a normal MMBasic string. The parameters start and length define the part of the string to be returned.<br>The 'start' position is always measured from 1 (the first character is position 1) regardless of the OPTION BASE setting, and so matches the position returned by LINSTR. Unlike LGETBYTE, LGETSTR$ does not honour OPTION BASE. |
| LINSTR(array%(), search$ [,start] [,size])) | Returns the position of a search string in a long string.<br>The returned value is an integer and will be zero if the substring cannot be found. ‘array%()’ is the string to be searched and must be a long string |

| | |
|---|---|
| | variable. ‘search$’ is the substring to look for and it must be a normal MMBasic string or expression (not a long string). The search is case sensitive.<br>Normally the search will start at the first character in ' array%()' but the optional third parameter allows the start position of the search to be specified.<br>If the optional parameter ‘size’ is specified the ‘search$’ is treated as a regular expression. See *Appendix E* for the details.<br>The position returned is always 1-based (the first character is position 1) and is not affected by OPTION BASE. It can be passed directly to LGETSTR$, but note that LGETBYTE indexes bytes according to OPTION BASE (see LGETBYTE). |
| LLEN(array%()) | Returns the length of a long string stored in ‘array%()’. |
| LINPUT(array%(),fnbr,nbr) | This reads ‘nbr’ bytes from a file open as ‘fnbr’ into the LONGSTRING ‘array%()’.<br>The function returns the number of bytes actually read so if you are near the end of the file the number may be less than the number requested.<br>It only works with file I/O and not serial or console I/O |
| LOC( [#]fnbr ) | For a file on the Flash Filesystem or SD Card opened as 'fnbr' this will return the current position of the read/write pointer in the file. Note that the first byte in a file is numbered 1.<br>For a serial communications port opened as 'fnbr' this function will return the number of bytes received and waiting in the receive buffer to be read. #0 can be used which refers to the console's input buffer.<br>The # is optional. |
| LOF( [#]fnbr ) | For a file on the Flash Filesystem or SD Card opened as 'fnbr' this will return the current length of the file in bytes.<br>For a serial communications port opened as 'fnbr' this function will return the space (in characters) remaining in the transmit buffer. Note that when the buffer is full MMBasic will pause when adding a new character and wait for some space to become available so this function can be used to avoid this.<br>The # is optional. |
| LOG( number ) | Returns the natural logarithm of the argument 'number'. |
| MAP( n ) | <u>VGA, HDMI AND RP2350 PICOMITE VERSIONS ONLY</u><br>Returns the 24-bit RGB value for the index ‘n’ in the colour map table.<br>See the MAP command. This allows the Basic programmer to use a colour specified by the MAP command<br>e.g<br>MAP(8) = RGB(100,100,100)<br>MAP SET<br>Pixel x,y,map(8)<br><br>NB: for VGA all colours set by the map command will be converted to the nearest RGB121 colour as determined by the VGA resistor network. For HDMI displays colours will be converted to the nearest RGB555 colour (640x480 resolution) or RGB332 colour (1024x768 or 1280x720 resolution). For PIcoMite RP2350 buffered drivers displays colours will be converted to the nearest RGB332 colour. |
| MATH | The math function performs many simple mathematical calculations that can be programmed in Basic but there are speed advantages to coding looping |

| | |
|---|---|
| | structures in C and there is the advantage that once debugged they are there for everyone without re-inventing the wheel. |
| **Simple functions** | |
| MATH(ATAN3 x,y) | Returns ATAN3 of x and y |
| MATH(COSH a) | Returns the hyperbolic cosine of a |
| MATH(LOG10 a) | Returns the base 10 logarithm of a |
| MATH(SINH a) | Returns the hyperbolic sine of a |
| MATH(TANH a) | Returns the hyperbolic tan of a |
| MATH(CRCn data [,length] [,polynome] [,startmask] [,endmask] [,reverseIn] [,reverseOut] | Calculates the CRC to n bits (8, 12, 16, 32) of "data". "data" can be an integer or floating point array or a string variable. "Length" is optional and if not specified the size of the array or string length is used. The defaults for startmask, endmask reverseIn, and reversOut are all zero. reverseIn, and reversOut are both Booleans and take the value 1 or 0. The defaults for polynomes are CRC8=&H07, CRC12=&H80D, CRC16=&H1021, crc32=&H04C11DB7<br>eg, for crc16_CCITT use MATH(CRC16 array(), n,, &HFFFF) |
| MATH(RAND) | Returns a random number 0.0 <= n < 1.0 using the "Mersenne Twister algorithm. If not seeded with MATH RANDOMIZE the first usage seeds with the time in microseconds since boot. NB: the RP2350 includes a H/W random number generator. |
| **Simple Statistics** | |
| MATH(CHI a()) | Returns the Pearson's chi-squared value of the two dimensional array a()) |
| MATH(CHI_p a()) | Returns the associated probability in % of the Pearson's chi-squared value of the two dimensional array a()) |
| MATH(CROSSING array() [,level] [,direction] [,confirm]) | This returns the array index of the first sample at which the values in the array cross the "level" in the direction specified, where the crossing is sustained for "confirm" consecutive samples. level defaults to 0. direction defaults to 1 (valid values are -1 or 1). confirm defaults to 1; values greater than 1 reject brief noise spikes by requiring that many consecutive samples to remain past the level. Returns -1 if no qualifying crossing is found; a crossing within the final (confirm-1) samples cannot be confirmed and is not reported. |
| MATH(CORREL a(), a()) | Returns the Pearson's correlation coefficient between arrays a() and b() |
| MATH(MAX a() [,index%]) | Returns the maximum of all values in the a() array, a() can have any number of dimensions. If the integer variable is specified then it will be updated with the |

| | |
|---|---|
| | index of the maximum value in the array. This is only available on one-dimensional arrays |
| MATH(MEAN a()) | Returns the average of all values in the a() array, a() can have any number of dimensions |
| MATH(MEDIAN a()) | Returns the median of all values in the a() array, a() can have any number of dimensions |
| MATH(MIN a(), [index%]) | Returns the minimum of all values in the a() array, a() can have any number of dimensions. If the integer variable is specified then it will be updated with the index of the minimum value in the array. This is only available on one-dimensional arrays. |
| MATH(SD a()) | Returns the sample standard deviation of all values in the a() array, a() can have any number of dimensions |
| MATH(SUM a()) | Returns the sum of all values in the a() array, a() can have any number of dimensions |
| **Vector Arithmetic** | |
| MATH(MAGNITUDE v()) | Returns the magnitude of the vector v(). The vector can have any number of elements |
| MATH(DOTPRODUCT v1(), v2()) | Returns the dot product of two vectors v1() and v2(). The vectors can have any number of elements but must have the same cardinality |
| **Matrix Arithmetic** | |
| MATH(M_DETERMINANT array!()) | Returns the determinant of the array. The array must be square. |

| | |
|---|---|
| **Creation**<br>complex% = MATH(C_CPLX r!, i!)<br>complex% = MATH(C_POLAR radius!, angle!)<br>**Floating returns**<br>real! = MATH(C_REAL complex%)<br>imag! = MATH(C_IMAG complex%)<br>arg! = MATH(C_ARG complex%)<br>mod! = MATH(C_MOD complex%)<br>phase! = MATH(C_PHASE complex%)<br>**Unary functions**<br>complex1% = MATH(C_CONJ complex2%)<br>complex1% = MATH(C_SIN complex2%)<br>complex1% = MATH(C_COS complex2%)<br>complex1% = MATH(C_TAN complex2%)<br>complex1% = MATH(C_ASIN complex2%)<br>complex1% = MATH(C_ACOS complex2%)<br>complex1% = MATH(C_ATAN complex2%)<br>complex1% = MATH(C_SINH complex2%) | MMBasic supports a full range of functions to allow the manipulation of complex numbers. In this implementation complex numbers have a 32-bit real and 32-bit imaginary part and to make this work in MMBasic, it uses integers (64-bit) to hold these. |

| | |
|---|---|
| complex1% = MATH(C_COSH complex2%)<br>complex1% = MATH(C_TANH complex2%)<br>complex1% = MATH(C_ASINH complex2%)<br>complex1% = MATH(C_ACOSH complex2%)<br>complex1% = MATH(C_ATANH complex2%)<br>complex1% = MATH(C_PROJ complex2%)<br>**Basic Arithmetic**<br>complex1% = MATH(C_ADD complex2%,complex3%)<br>complex1% = MATH(C_SUB complex2%,complex3%)<br>complex1% = MATH(C_MUL complex2%,complex3%)<br>complex1% = MATH(C_DIV complex2%,complex3%)<br>complex1% = MATH(C_POW complex2%,complex3%)<br>complex1% = MATH(C_AND complex2%,complex3%)<br>complex1% = MATH(C_OR complex2%,complex3%)<br>complex1% = MATH(C_XOR complex2%,complex3%) | |
| MATH(PID channel, setpoint!, measurement)) | This function must be called in the PID callback subroutine for the 'channel' specified and returns the output of the controller function.<br>The 'setpoint' value is the desired state that the controller is trying to achieve. The 'measurement' is the current value of the real world.<br>https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=17263<br>For an example of setting up and running a PID controller |
| MATH(BASE64 ENCODE/DECODE in$/in(), out$/out()) | Returns the length of out$/out(). This base64 encodes or decodes the data in 'in' and puts the result in 'out'. Where arrays are used as the output they must be big enough relative to the input and the direction. Encryption increases length by 4/3 and decryption decreases it by 3/4. |
| MAX( arg1 [, arg2 [, …]] )<br>or<br>MIN( arg1 [, arg2 [, …]] ) | Returns the maximum or minimum number in the argument list.<br>Note that the comparison is a floating point comparison (integer arguments are converted to floats) and a float is returned. |
| MID$( string$, start )<br>or<br>MID$( string$, start, nbr ) | Returns a substring of 'string$' beginning at 'start' and continuing for 'nbr' characters. The first character in the string is number 1.<br>If 'nbr' is omitted the returned string will extend to the end of 'string$' |
| OCT$( number [, chars]) | Returns a string giving the octal (base 8) representation of 'number'.<br>'chars' is optional and specifies the number of characters in the string with zero as the leading padding character(s). |
| PEEK(BYTE addr%)<br>or<br>PEEK(SHORT addr%)<br>or<br>PEEK(WORD addr%)<br>or<br>PEEK(INTEGER addr%)<br>or<br>PEEK(FLOAT addr%)<br>or | NB: addresses are rounded down to match the datatype requested (e.g. PEEK(SHORT or give a error if not aligned e.g PEEK(SP<br>Will return a byte or a word within the processor's virtual memory space.<br>BYTE will return the byte (8-bits) located at 'addr%'<br>SHORT will return the short integer (16-bits) located at 'addr%'<br>WORD will return the word (32-bits) located at 'addr%'<br>INTEGER will return the integer (64-bits) located at 'addr%'<br>FLOAT will return the floating point number (64-bits) located at 'addr%' |

| | |
|---|---|
| PEEK(VARADDR var)<br>or | VARADDR will return the address (32-bits) of the variable 'var' in memory. An array is specified as var(). |
| PEEK(VARHEADER var)<br>or | VARHEADER will return the address (32-bits) of the variable 'var' header in memory. An array is specified as var(). This will be the address of the first byte of the variable name. |
| PEEK(CFUNADDR cfun)<br>or | CFUNADDR will return the address (32-bits) of the CFunction 'cfun' in memory. This address can be passed to another CFunction which can then call it to perform some common process. |
| PEEK(VAR var, ±offset)<br>or | VAR, will return a byte in the memory allocated to 'var'. An array is specified as var(). |
| PEEK( VARTBL, ±offset)<br>or | VARTBL was removed in V7.00.00: the variables are no longer kept in one table. Use PEEK(VARHEADER var) for the address of a variable's header and PEEK(VAR var, ±offset) to read its bytes. |
| PEEK( PROGMEM, ±offset) | PROGMEM, will return a byte in the memory allocated to the program. Note that there is a comma after the keyword PROGMEM.<br>Note that 'addr%' should be an integer. |
| PEEK(BP n%) | PEEK(bp n%) ' returns the byte at address n% and increments n% to point to the next byte. |
| PEEK(SP n%) | PEEK(sp n%) ' returns the short at address n% and increments n% to point to the next short. |
| PEEK(WP n%) | PEEK(wp n%) ' returns the word at address n% and increments n% to point to the next word. |
| PI | Returns the value of pi. |
| PIN( pin ) | Returns the value on the external I/O 'pin'. Zero means digital low, 1 means digital high and for analogue inputs it will return the measured voltage as a floating point number.<br>Frequency inputs will return the frequency in Hz. A period input will return the period in milliseconds while a count input will return the count since reset (counting is done on the positive rising edge). The count input can be reset to zero by resetting the pin to counting input (even if it is already so configured).<br>When a pin is configured as an output this function will return the value of the output setting (ie, high or low). Also see the SETPIN and PIN() = commands. Refer to the chapter *Using the I/O pins* for a general description of the PicoMite's input/output capabilities. |
| PIN( BOOTSEL ) | Returns the state of the boot select switch allowing it to be used as a user input in a program. |
| PIN( TEMP ) | Returns the temperature of the RP2040/RP2350 chip (see the data sheet for the details). |
| PIO(DMA RX POINTER)<br>PIO(DMA TX POINTER) | Returns the current data item being written or read by the PIO. |
| PIO (SHIFTCTRL push_threshold [,pull_threshold] [,autopush] [,autopull] [,in_shiftdir] | helper function to calculate the value of shiftctrl for the INIT MACHINE command . |

| | |
|---|---|
| [,out_shiftdir] [,fjoin_tx] [,fjoin_rx]) | |
| PIO (PINCTRL no_side_set_pins [,no_set_pins] [,no_out_pins] [,IN base] [,side_set_base] [,set_base][, out_base]) | helper function to calculate the value of pinctrl for the INIT MACHINE command. Note: The pin parameters must be formatted as GPn. |
| PIO (EXECCTRL jmp_pin ,wrap_target, wrap [,side_pindir] [,side_en]) | helper function to calculate the value of execctrl for the INIT MACHINE command |
| PIO(READFIFO a, b, c) | Read from a PIO FIFO<br>'a' is the pio (0 or 1), 'b' id the state machine (0...3), 'c' is the FIFO register *0…3) |
| PIO (FDEBUG pio) | returns the value of the FSDEBUG register for the pio specified |
| PIO (FSTAT pio) | returns the value of the FSTAT register for the pio specified |
| PIO (FLEVEL pio) | returns the value of the FLEVEL register for the pio specified<br>PIO(FLEVEL pio) |
| PIO(FLEVEL pio ,sm, DIR) | dir can be RX or TX. Returns the level of the specific fifo |
| PIO(.WRAP)<br>PIO(.WRAP TARGET) | returns the location of the .wrap directive in PIO ASSEMBLE<br>returns the location of the .wrap target directive in PIO ASSEMBLE.<br>These can be used in the PIO(EXECCTRL function as follows:<br>PIO (EXECCTRL jmp_pin PIO(.WRAP TARGET), PIO(.WRAP) [,side_pindir] [,side_en]) |
| PIO(NEXT LINE) | Returns the next unused PIO instruction slot after a block of PIO instructions terminated by END PROGRAM |
| PIXEL( x, y) | Returns the colour of a pixel on the video output or LCD display. 'x' is the horizontal coordinate and 'y' is the vertical coordinate of the pixel.<br>If an LCD display is used it must use one of the SSD1963, ILI9341, ILI9488, or ST7789_320 controllers. |
| PORT(start, nbr [,start, nbr]…) | Returns the value of a number of I/O pins in one operation.<br>'start' is an I/O pin number and its value will be returned as bit 0. 'start'+1 will be returned as bit 1, 'start'+2 will be returned as bit 2, and so on for 'nbr' number of bits. I/O pins used must be numbered consecutively and any I/O pin that is invalid or not configured as an input will cause an error. The start/nbr pair can be repeated up to 25 times if additional groups of input pins need to be added.<br>This function will also return the output state of a pin configured as an output.<br>This can be used to conveniently communicate with parallel devices like memory chips. Any number of I/O pins (and therefore bits) can be used from 1 to the number of I/O pins on the chip.<br>NB: If the pins are defined using the GPn syntax then the firmware will ignore invalid pins so `PORT(GP0, 8)` Will read eight I/O pins GP0 ,GP1 ,GP2, GP3, GP4, GP5, GP6 and GP7 .<br>See the PORT command to simultaneously output to a number of pins. |

| | |
|---|---|
| PULSIN( pin, polarity )<br>or<br>PULSIN( pin, polarity, t1 )<br>or<br>PULSIN( pin, polarity, t1, t2 ) | Measures the width of an input pulse from 1µs to 1 second with 0.1µs resolution.<br>'pin' is the I/O pin to use for the measurement, it must be previously configured as a digital input. 'polarity' is the type of pulse to measure, if zero the function will return the width of the next negative pulse, if non zero it will measure the next positive pulse.<br>'t1' is the timeout applied while waiting for the pulse to arrive, 't2' is the timeout used while measuring the pulse. Both are in microseconds (µs) and are optional. If 't2' is omitted the value of 't1' will be used for both timeouts. If both 't1' and 't2' are omitted then the timeouts will be set at 100000 (i.e. 100ms).<br>This function returns the width of the pulse in microseconds (µs) or -1 if a timeout has occurred. The measurement is accurate to ±0.5% and ±0.5µs.<br>Note that this function will cause the running program to pause while the measurement is made and interrupts will be ignored during this period. |
| RAD( degrees ) | Converts 'degrees' to radians. |
| RGB(red, green, blue)<br>or<br>RGB(shortcut) | Generates an RGB true colour value.<br>'red', 'blue' and 'green' represent the intensity of each colour. A value of zero represents black and 255 represents full intensity.<br>'shortcut' allows common colours to be specified by naming them. The colours that can be named are white, black, blue, green, cyan, red, magenta, yellow, brown, white, orange, pink, gold, salmon, beige, lightgrey and grey (or USA spelling gray/lightgray). For example, RGB(red) or RGB(cyan). |
| RIGHT$( string$, number-of-chars ) | Returns a substring of ‘string$’ with ‘number-of-chars’ from the right (end) of the string. |
| RND( number )<br>or<br>RND | Returns a random (RP2350) or pseudo-random (RP2040) number in the range of 0 to 0.999999. The 'number' value is ignored if supplied.<br>See also the RANDOMIZE command (RP2040 only). |
| SGN( number ) | Returns the sign of the argument 'number', +1 for positive numbers, 0 for 0, and -1 for negative numbers. |
| SIN( number ) | Returns the sine of the argument 'number' in radians. |
| SPACE$( number ) | Returns a string of blank spaces 'number' characters long. |
| SPI ( data )<br>or<br>SPI2 ( data ) | Send and receive data using an SPI channel.<br>A single SPI transaction will send data while simultaneously receiving data from the slave. ‘data’ is the data to send and the function will return the data received during the transaction. ‘data’ can be an integer or a floating point variable or a constant. |
| SPRITE() | See Appendix G and the separate SPRITE_User_Manual pdf |
| SQR( number ) | Returns the square root of the argument 'number'. |
| STR$( number )<br>or<br>STR$( number, m )<br>or<br>STR$( number, m, n )<br>or | Returns a string in the decimal (base 10) representation of 'number'.<br>If 'm' is specified sufficient spaces will be added to the start of the number to ensure that the number of characters before the decimal point (including the negative or positive sign) will be at least 'm' characters. If 'm' is zero or the number has more than 'm' significant digits no padding spaces will be added. |

| | |
|---|---|
| STR$( number, m, n, c$ ) | If 'm' is negative, positive numbers will be prefixed with the plus symbol and negative numbers with the negative symbol. If 'm' is positive then only the negative symbol will be used.<br>'n' is the number of digits required to follow the decimal place. If it is zero the string will be returned without the decimal point. If it is negative the output will always use the exponential format with 'n' digits resolution. If 'n' is not specified the number of decimal places and output format will vary automatically according to the number.<br>'c$' is a string and if specified the first character of this string will be used as the padding character instead of a space (see the 'm' argument). Examples:<br>STR$(123.456) will return "123.456"<br>STR$(-123.456) will return "-123.456"<br>STR$(123.456, 1) will return "123.456"<br>STR$(123.456, -1) will return "+123.456"<br>STR$(123.456, 6) will return " 123.456"<br>STR$(123.456, -6) will return " +123.456"<br>STR$(-123.456, 6) will return " -123.456"<br>STR$(-123.456, 6, 5) will return " -123.45600"<br>STR$(-123.456, 6, -5) will return " -1.23456e+02"<br>STR$(53, 6) will return " 53"<br>STR$(53, 6, 2) will return " 53.00"<br>STR$(53, 6, 2, "*") will return "****53.00" |
| STR2BIN(type, string$ [,BIG]) | Returns a number equal to the binary representation in 'string$'.<br>'type' can be:<br>INT64 converts 8 byte string representing a signed 64-bit integer to an integer<br>UINT64 converts 8 byte string representing an unsigned 64-bit integer to an integer<br>INT32 converts 4 byte string representing a signed 32-bit integer to an integer<br>UINT32 converts 4 byte string representing an unsigned 32-bit integer to an integer<br>INT16 converts 2 byte string representing a signed 16-bit integer to an integer<br>UINT16 converts 2 byte string representing an unsigned 16-bit integer to an integer<br>INT8 converts 1 byte string representing a signed 8-bit integer to an integer<br>UINT8 converts 1 byte string representing an unsigned 8-bit integer to an integer<br>SINGLE converts 4 byte string representing single precision float to a float<br>DOUBLE converts 8 byte string representing single precision float to a float<br>By default the string must contain the number in little-endian format (i.e. the least significant byte is the first one in the string). Setting the third parameter to 'BIG' will interpret the string in big-endian format (i.e. the most significant byte is the first one in the string).<br>This function makes it easy to read data from binary data files, interpret numbers from sensors or efficiently read binary data from flash memory chips.<br>An error will be generated if the string is the incorrect length for the conversion requested<br>See also the function BIN2STR$ |
| STRING$( nbr, ascii )<br>or | Returns a string 'nbr' bytes long consisting of either the first character of string$ or the character representing the ASCII value 'ascii' which is an integer or float number in the range of 0 to 255. |

| | |
|---|---|
| STRING$( nbr, string$ ) | |
| STRUCT( arg, …) | NOT PICOMITEMIN Returns data from a structure (also known as a User-Defined Type) which allows related variables of different types to be grouped under a single name.<br>This is described in full in the file *MMBasic_Structures_Manual.pdf* included in the firmware download zip file. |
| TAB( number ) | Outputs spaces until the column indicated by 'number' has been reached on the console output. |
| TAN( number ) | Returns the tangent of the argument 'number' in radians. |
| TEMPR( pin [,timeout]) | Return the temperature measured by a DS18B20 temperature sensor connected to 'pin' (which does not have to be configured).<br>The returned value is degrees C with a default resolution of 0.25ºC. If there is an error during the measurement the returned value will be 1000.<br>The time required for the overall measurement is 200ms and interrupts will be ignored during this period.<br>The optional parameter timeout can be used to override the default (200mSec) to allow for slow devices.<br>Alternatively the TEMPR START command can be used to start the measurement and your program can do other things while the conversion is progressing. When this function is called the value will then be returned instantly assuming the conversion period has expired. If it has not, this function will wait out the remainder of the conversion time before returning the value.<br>The DS18B20 can be powered separately by a 3V to 5V supply or it can operate on parasitic power from the Raspberry Pi Pico.<br>See the chapter *Special Hardware Devices* for more details. |
| TIME$ | Returns the current time based on MMBasic's internal clock as a string in the form "HH:MM:SS" in 24 hour notation. For example, "14:30:00".<br>To set the current time use the command TIME$ = . |
| TIMER | Returns the elapsed time in milliseconds (eg, 1/1000 of a second) since reset.<br>The timer is reset to zero on power up or a CPU restart and you can also reset it by using TIMER as a command. If not specifically reset it will continue to count up forever (it is a 64 bit number and therefore will only roll over to zero after 200 million years). |
| TOUCH(X)<br>or<br>TOUCH(Y)<br>or<br>TOUCH(DOWN)<br>or<br>TOUCH(UP)<br>or<br>TOUCH(X2)<br>or<br>TOUCH(Y2)<br>or<br>TOUCH(XN n) | NOT RP2040 VGA Will return the X or Y coordinate of the location currently touched on an LCD panel. If the screen is not being touched the function will return -1.<br>TOUCH(DOWN) returns 1 (true) if the screen is currently being touched and TOUCH(UP) returns 1 (true) if it is not.<br>On panels with a capacitive touch controller (FT6336 or GT911) and on USB touch screens TOUCH(X2) and TOUCH(Y2) return the position of a second touch location or -1 if no second location is touched.<br>TOUCH(XN n) and TOUCH(YN n) return the X or Y coordinate of the ‘n’th touch location (n = 1 to 10) or -1 if that location is not being touched. n = 1 is the same as TOUCH(X)/TOUCH(Y) and n = 2 is the same as TOUCH(X2)/TOUCH(Y2). TOUCH(XN 0) returns the number of locations currently being touched. The maximum number of simultaneous touches depends on the touch controller: one for resistive panels, two for the FT6336, five for the GT911 and up to ten for USB touch screens. |

| | |
|---|---|
| or<br>TOUCH(YN n) | |
| | WITH TOUCH SUPPORT ONLY<br>Detect gestures made on the touch panel. The firmware continuously monitors the touch input and when it recognises a gesture it latches it until it is read. Reading the function that matches the latched gesture returns 1 and clears it. Reading a function that does not match returns 0 and leaves the gesture latched, so a program can test for a number of different gestures in turn without losing an event. Gestures are not available on the RP2040 versions of the WebMite and VGA firmware.<br>Single finger gestures (all touch panels): |
| TOUCH(TAP) | TOUCH(TAP) - a quick tap (touch and release within a quarter of a second without moving). |
| TOUCH(DTAP) | TOUCH(DTAP) - a double tap (two quick taps in the same place within half a second). |
| TOUCH(HOLD) | TOUCH(HOLD) - a long press (the touch held still for at least half a second). This is recognised while the touch is still down. |
| TOUCH(SWL)<br>TOUCH(SWR)<br>TOUCH(SWU)<br>TOUCH(SWD)<br>TOUCH(SWIPE) | TOUCH(SWL), TOUCH(SWR), TOUCH(SWU), TOUCH(SWD) - a swipe to the left, right, up or down (a quick movement of at least about 15% of the screen's shorter dimension).<br>TOUCH(SWIPE) - returns the direction of a swipe as a number (0 = none, 1 = left, 2 = right, 3 = up, 4 = down) and clears it. This is convenient for use with a SELECT CASE statement. |
| | Two finger gestures (touch controllers that can report two simultaneous touch points - see TOUCH(XN n) above): |
| TOUCH(EXPAND)<br>TOUCH(CONTRACT) | TOUCH(EXPAND) and TOUCH(CONTRACT) - the distance between the two touch points increased or decreased by at least 30% (ie, zoom in or zoom out). |
| TOUCH(PINCH) | TOUCH(PINCH) - returns 0 = none, 1 = expand, 2 = contract and clears it. |
| TOUCH(CW)<br>TOUCH(CCW) | TOUCH(CW) and TOUCH(CCW) - the line between the two touch points rotated by at least 15 degrees clockwise or anticlockwise. |
| TOUCH(ROTATE) | TOUCH(ROTATE) - returns 0 = none, 1 = clockwise, 2 = anticlockwise and clears it. |
| TOUCH(TTAP) | TOUCH(TTAP) - a quick tap with two fingers.<br>Apart from TOUCH(HOLD) a gesture is recognised when the finger (or fingers) is lifted from the screen. |
| TRIM$(source$ [,mask$] [,where/where$]) | This function can remove characters from the beginning or end of a string or both<br>'source$' is the input string<br>'mask$; is a string containing a list of character to be removed. If omitted it defaults to space<br>'where/where$' can be L, R, or B or a string beginning with L, R, or B to specify characters should be removed from the left of the source,<br>the right of the source or both. If omitted defaults to L |
| UCASE$( string$ ) | Returns 'string$' converted to uppercase characters. |
| VAL( string$ ) | Returns the numerical value of the 'string$'. If 'string$' is an invalid number the function will return zero.<br>This function will recognise the &H prefix for a hexadecimal number, &O for octal and &B for binary. |

# Obsolete Commands and Functions

These commands and functions are mostly included to assist in converting programs written for Microsoft BASIC. For new programs the corresponding modern commands in MMBasic should be used.

These commands and functions are deprecated and should be avoided in new programs, but they are a permanent part of MMBasic and will not be removed - existing programs that use them will continue to run.

| | |
|---|---|
| BITBANG | Replaced by the command DEVICE. For compatibility BITBANG can still be used in programs and will be automatically converted to DEVICE |
| DEVICE CAMERA | Changed to the CAMERA command |
| DEVICE GAMEPAD | Changed to the GAMEPAD command |
| DEVICE HUMID | Changed to the HUMID command |
| DEVICE KEYPAD | Changed to the KEYPAD command |
| DEVICE MOUSE | Changed to the MOUSE command |
| DEVICE LCD | Changed to the LCD command |
| DEVICE WII | Changed to the WII command |
| DEVICE WS2812 | Changed to the WS2812 command |
| GOSUB target | Initiates a subroutine call to the target, which can be a line number or a label. The subroutine must end with RETURN.<br>New programs should use defined subroutines (i.e. SUB…END SUB). |
| IF condition THEN linenbr | For Microsoft compatibility a GOTO is assumed if the THEN statement is followed by a number. A label is invalid in this construct.<br>New programs should use: IF condition THEN GOTO linenbr \| label |
| IRETURN | Returns from an interrupt when the interrupt destination was a line number or a label.<br>New programs should use a user defined subroutine as an interrupt destination. In that case END SUB or EXIT SUB will cause a return from the interrupt. |
| ON nbr GOTO \| GOSUB target[,target, target,..] | ON either branches (GOTO) or calls a subroutine (GOSUB) based on the rounded value of 'nbr'; if it is 1, the first target is called, if 2, the second target is called, etc. Target can be a line number or a label.<br>New programs should use SELECT CASE. |
| POS | For the console, returns the current cursor position in the line in characters. |
| RETURN | RETURN concludes a subroutine called by GOSUB and returns to the statement after the GOSUB. |

# Appendix A
# Serial Communications

Two serial interfaces are available for asynchronous serial communications. They are labelled COM1: and COM2:.

The versions with USB host support also provide COM3: to COM6: for USB serial devices (see USB Serial Devices in the chapter Keyboard/Mouse/Gamepad) and the Bluetooth host versions provide COM7: for a Bluetooth serial device (see the chapter Bluetooth Versions).

## I/O Pins

Before a serial interface can be used the I/O pins must be defined using the following command for the first channel (referred as COM1):

SETPIN rx, tx, COM1

Valid pins are RX: GP1, GP13 or GP17

TX: GP0, GP12, GP16 or GP28

And the following command for the second channel (referred to as COM2):

SETPIN rx, tx, COM2

Valid pins are RX: GP5, GP9 or GP21

TX: GP4, GP8 or GP20

TX is data from the Raspberry Pi Pico and RX is data to it.

Note that on the WebMite version COM1 and COM2 are not available on GP20 to GP28

The signal polarity is standard for devices running at TTL voltages. Idle is voltage high, the start bit is voltage low, data uses a high voltage for logic 1 and the stop bit is voltage high. These signal levels allow you to directly connect to devices like GPS modules (which generally use TTL voltage levels).

## Commands

After being opened the serial port will have an associated file number and you can use any commands that operate with a file number to read and write to/from it. A serial port can be closed using the CLOSE command.

The following is an example:

```
SETPIN GP13, GP16, COM1     ' assign the I/O pins for the first serial port
OPEN "COM1:4800" AS #5      ' open the first serial port with a speed of 4800 baud
PRINT #5, "Hello"           ' send the string "Hello" out of the serial port
dat$ = INPUT$(20, #5)       ' get up to 20 characters from the serial port
CLOSE #5                    ' close the serial port
```

## The OPEN Command

A serial port is opened using the command:

```
OPEN comspec$ AS #fnbr
```

'fnbr' is the file number to be used. It must be in the range of 1 to 10. The # is optional.

'comspec$' is the communication specification and is a string (it can be a string variable) specifying the serial port to be opened and optional parameters. The default is 9600 baud, 8 data bits, no parity and one stop bit.

It has the form `"COMn: baud, buf, int, int-trigger, EVEN, ODD, S2, 7BIT"` where:

- 'n' is the serial port number for either COM1: or COM2:.
- 'baud' is the baud rate. This can be any number from 1200 to 921600. Default is 9600.
- 'buf' is the receive buffer size in bytes (default size is 256). The transmit buffer is fixed at 256 bytes.
- 'int' is interrupt subroutine to be called when the serial port has received some data.
- 'int-trigger' is the number of characters received which will trigger an interrupt.

All parameters except the serial port name (COMn:) are optional. If any one parameter is left out then all the following parameters must also be left out and the defaults will be used.

Five options can be added to the end of 'comspec$'. These are:

- 'S2' specifies that two stop bits will be sent following each character transmitted.
- EVEN specifies that an even parity bit will be applied, this will result in a 9-bit transfer unless 7BIT is set.

- ODD specifies that an odd parity bit will be applied,  this will result in a 9-bit transfer unless 7BIT is set
- 7BIT specifies that there a 7bits of data. This is normally used with EVEN or ODD
- INV specifies that the output signals will be inverted and input assumed to be inverted

## Examples

Opening a serial port using all the defaults:

```
OPEN "COM1:" AS #2
```

Opening a serial port specifying only the baud rate (4800 bits per second):

```
OPEN "COM1:4800" AS #1
```

Opening a serial port specifying the baud rate (9600 bits per second) and receive buffer size (1KB):

```
OPEN "COM2:9600, 1024" AS #8
```

The same as above but with two stop bits enabled:

```
OPEN "COM2:9600, 1024, S2" AS #8
```

An example specifying everything including an interrupt, an interrupt level, and two stop bits:

```
OPEN "COM2:19200, 1024, ComIntLabel, 256, S2" AS #5
```

## Reading and Writing

Once a serial port has been opened you can use any command or function that uses a file number to read from and write to the port.  Data received by the serial port will be automatically buffered in memory by MMBasic until it is read by the program and the INPUT$() function is the most convenient way of doing that.  When using the INPUT$() function the number of characters specified will be the maximum number of characters returned but it could be less if there are less characters in the receive buffer.  In fact the INPUT$() function will immediately return an empty string if there are no characters available in the receive buffer.

The LOC() function is also handy; it will return the number of characters waiting in the receive buffer (i.e. the maximum number characters that can be retrieved by the INPUT$() function).  Note that if the receive buffer overflows with incoming data the serial port will automatically discard the oldest data to make room for the new data.

The PRINT command is used for outputting to a serial port and any data to be sent will be held in a memory buffer while the serial port is sending it.  This means that MMBasic will continue with executing the commands after the PRINT command while the data is being transmitted.  The one exception is if the output buffer is full and in that case MMBasic will pause and wait until there is sufficient space before continuing.  The LOF() function will return the amount of space left in the transmit buffer and you can use this to avoid stalling the program while waiting for space in the buffer to become available.

If you want to be sure that all the data has been sent (perhaps because you want to read the response from the remote device) you should wait until the LOF() function returns 256 (the transmit buffer size) indicating that there is nothing left to be sent.

Serial ports can be closed with the CLOSE command.  This will wait for the transmit buffer to be emptied then free up the memory used by the buffers and cancel the interrupt (if set).  A serial port is also automatically closed when commands such as RUN and NEW are issued.

## Interrupts

The interrupt subroutine (if specified) will operate the same as a general interrupt on an external I/O pin (see the chapter *Using the I/O pins* for a description).

When using interrupts you need to be aware that it will take some time for MMBasic to respond to the interrupt and more characters could have arrived in the meantime, especially at high baud rates.  For example, if you have specified the interrupt level as 200 characters and a buffer of 256 characters then quite easily the buffer will have overflowed by the time the interrupt subroutine can read the data.  In this case the buffer should be increased to 512 characters or more.

# Appendix B
# I²C Communications

There are two I²C channels. They can operate in master or slave mode.

## I/O Pins

Before the I²C interface can be used the I/O pins must be defined using the following command for the first channel (referred as I2C):

SETPIN sda, scl, I2C

Valid pins are SDA: GP0, GP4, GP8, GP12, GP16, GP20 or GP28
SCL: GP1, GP5, GP9, GP13, GP17 or GP21

Note that on the WebMite version I2C SDA is not available on GP28

And the following command for the second channel (referred to as I2C2):

SETPIN sda, scl, I2C2

Valid pins are SDA: GP2, GP6, GP10, GP14, GP18, GP22 or GP26
SCL: GP3, GP7, GP11, GP15, GP19 or GP27

When running the I²C bus at above 100 kHz the cabling between the devices becomes important. Ideally the cables should be as short as possible (to reduce capacitance) and the data and clock lines should not run next to each other but have a ground wire between them (to reduce crosstalk).

If the data line is not stable when the clock is high, or the clock line is jittery, the I²C peripherals can get "confused" and end up locking the bus (normally by holding the clock line low). If you do not need the higher speeds then operating at 100 kHz is the safest choice.

The command I2C CHECK addr can be used to check if a device is present at the address 'addr'. This will set the read only variable MM.I2C to 0 if a device responds or 1 if there is no response.

## I²C Master Commands

There are four commands that can be used for the first channel (I2C) in master mode as follows.

The commands for the second channel (I2C2) are identical except that the command is I2C2

**I2C OPEN speed, timeout**

Enables the I²C module in master mode. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.

'speed' is the clock speed (in KHz) to use and must be either 100, 400 or 1000.

'timeout' is a value in milliseconds after which the master send and receive commands will be interrupted if they have not completed. The minimum value is 100. A value of zero will disable the timeout (though this is not recommended).

**I2C WRITE addr, option, sendlen, senddata [,senddata ..]**

Send data to the I²C slave device. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.

'addr' is the slave's I²C address.

'option' can be 0 for normal operation or 1 to keep control of the bus after the command (a stop condition will not be sent at the completion of the command)

'sendlen' is the number of bytes to send.

'senddata' is the data to be sent - this can be specified in various ways (all data sent will be sent as bytes with a value between 0 and 255):

- The data can be supplied as individual bytes on the command line.
  Example: I2C WRITE &H6F, 0, 3, &H23, &H43, &H25
- The data can be in a one dimensional array specified with empty brackets (i.e. no dimensions). 'sendlen' bytes of the array will be sent starting with the first element. Example: I2C WRITE &H6F, 0, 3, ARRAY()
- The data can be a string variable (not a constant).
  Example: I2C WRITE &H6F, 0, 3, STRING$

| | |
|---|---|
| I2C READ addr, option, rcvlen, rcvbuf [,rcvbuf ..] | Get data from the I2C slave device. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.<br>'addr' is the slave's I2C address.<br>'option' can be 0 for normal operation or 1 to keep control of the bus after the command (a stop condition will not be sent at the completion of the command)<br>'rcvlen' is the number of bytes to receive.<br>'rcvbuf' is the destination for the received data. This may be supplied in any one of three forms:<br>A list of individual numeric variables, one for each byte received. The number of variables must equal 'rcvlen' and array elements such as a(3) are allowed.<br>A one dimensional array of numbers specified with empty brackets. Received bytes will be stored in sequential elements of the array starting with the first.<br>Example: I2C READ &H6F, 0, 3, ARRAY()<br>A string variable. Received bytes will be stored as sequential characters and the string will be set to 'rcvlen' characters. |
| I2C CLOSE | Disables the master I²C module and returns the I/O pins to a "not configured" state. This command will also send a stop if the bus is still held. |

## I²C Slave Commands

| | |
|---|---|
| I2C SLAVE OPEN addr, send_int, rcv_int | Enables the I²C module in slave mode. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.<br>'addr' is the slave I²C address.<br>'send_int' is the subroutine to be invoked when the module has detected that the master is expecting data.<br>'rcv_int is the subroutine to be called when the module has received data from the master. Note that this is triggered on the first byte received so your program might need to wait until all the data is received. |
| I2C SLAVE WRITE sendlen, senddata [,senddata ..] | Send the data to the I²C master. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.<br>This command should be used in the send interrupt (ie in the 'send_int' subroutine when the master has requested data). Alternatively, a flag can be set in the interrupt subroutine and the command invoked from the main program loop when the flag is set.<br>'sendlen is the number of bytes to send.<br>'senddata' is the data to be sent. This can be specified in various ways, see the I2C WRITE commands for details. |
| I2C SLAVE READ rcvlen, rcvbuf, rcvd | Receive data from the I²C master device. The I2C command refers to channel 1 while the command I2C2 refers to channel 2 using the same syntax.<br>This command should be used in the receive interrupt (ie in the 'rcv_int' subroutine when the master has sent some data). Alternatively a flag can be set in the receive interrupt subroutine and the command invoked from the main program loop when the flag is set.<br>'rcvlen' is the maximum number of bytes to receive.<br>'rcvbuf' is the variable to receive the data. This can be specified in various ways, see the I2C READ commands for details.<br>'rcvd' is a variable that, at the completion of the command, will contain the actual number of bytes received (which might differ from 'rcvlen'). |

| | |
|---|---|
| I2C SLAVE CLOSE | Disables the slave I²C module and returns the external I/O pins to a "not configured" state. They can then be configured using SETPIN. |

## Errors

Following an I²C write or read the automatic variable MM.I2C will be set to indicate the result as follows:

0 = The command completed without error.
1 = Received a NACK response
2 = Command timed out

## 7-Bit Addressing

The standard addresses used in these commands are 7-bit addresses (without the read/write bit). MMBasic will add the read/write bit and manipulate it accordingly during transfers.

Some vendors provide 8-bit addresses which include the read/write bit. You can determine if this is the case because they will provide one address for writing to the slave device and another for reading from the slave. In these situations you should only use the top seven bits of the address. For example: If the read address is 9B (hex) and the write address is 9A (hex) then using only the top seven bits will give you an address of 4D (hex).

Another indicator that a vendor is using 8-bit addresses instead of 7-bit addresses is to check the address range. All 7-bit addresses should be in the range of 08 to 77 (hex). If your slave address is greater than this range then probably your vendor has provided an 8-bit address.

## System I2C

MMbasic can configure pins to be permanently in I2C mode using OPTION SYSTEM I2C sdapin, sckpin. If this is done the I2C master read and write commands can be used without opening or closing the I2C port. This is used specifically for devices like a realtime clock chip (e.g DS3231) which can then be checked automatically (OPTION RTC AUTO ENABLE). Any devices connected to the system I2C pins can be checked using the command LIST SYSTEM I2C.

## Examples

As an example of a simple communications where the Raspberry Pi Pico is the master, the following program will read and display the current time (hours and minutes) maintained by a PCF8563 real time clock chip connected to the second I2C channel:

```
DIM AS INTEGER RData(2)              ' this will hold received data
SETPIN GP6, GP7, I2C2                ' assign the I/O pins for I2C2
I2C2 OPEN 100, 1000                  ' open the I2C channel
I2C2 WRITE &H51, 0, 1, 3             ' set the first register to 3
I2C2 READ &H51, 0, 2, RData()        ' read two registers
I2C2 CLOSE                           ' close the I2C channel
PRINT "Time is " hex$(RData(1),2) ":" hex$(RData(0),2) 'display the BCD
coded data
```

This is an example of communications between two Raspberry Pi Picos. The master will send the characters ‘A’ to ‘Z’ and after each transmission issue a query to the slave and print the response. The slave will read the byte sent by the master and print it. In response to the query from the master it will send the current time.

First the master:

```
SETPIN GP2, GP3, I2C2
I2C2 OPEN 100, 1000
FOR i = 65 to 90
  a$ = CHR$(i)
  I2C2 WRITE &H50, 0, 1, a$
  PAUSE 500
  I2C2 READ &H50, 0, 8, a$
  PRINT a$
  PAUSE 500
NEXT i
```

Then the slave:

```
SETPIN GP2, GP3, I2C2
I2C2 SLAVE OPEN &H50, tint, rint
DO : LOOP
```

```
SUB rint
  LOCAL count, a$
  I2C2 SLAVE READ 10, a$, count
  PRINT LEFT$(a$, count)
END SUB

SUB tint
  LOCAL a$ = Time$
  I2C2 SLAVE WRITE LEN(a$), a$
END SUB
```

# Appendix C
# 1-Wire Communications

The 1-Wire protocol was developed by Dallas Semiconductor to communicate with chips using a single signalling line. This implementation was written for MMBasic by Gerard Sexton.

There are three commands that you can use:

| | |
|---|---|
| ONEWIRE RESET pin | Reset the 1-Wire bus |
| ONEWIRE WRITE pin, flag, length, data [, data…] | Send a number of bytes |
| ONEWIRE READ pin, flag, length, data [, data…] | Get a number of bytes |

Where:

pin - The I/O pin to use. It can be any pin capable of digital I/O.

flag - A combination of the following options:

1 - Send reset before command

2 - Send reset after command

4 - Only send/recv a bit instead of a byte of data

8 - Invoke a strong pullup after the command (the pin will be set high and open drain disabled)

length - Length of data to send or receive

data - Data to send or variable to receive. This may be supplied in any one of three forms:

A list of individual items. The number of items must equal 'length'. For WRITE each item is a constant, variable or expression; for READ each item is a numeric variable to receive one value. Array elements such as a(3) are allowed.

An array specified with empty brackets, eg, array(). It may be a floating point or integer array and must be large enough to supply (WRITE) or hold (READ) 'length' elements, which are taken from / stored into consecutive elements starting at the first.

A string variable. For WRITE the first 'length' characters are sent (the string must be at least that long); for READ the received bytes are stored as sequential characters and the string is set to that length.

The automatic variable MM.ONEWIRE returns true if a device was found

After the command is executed, the I/O pin will be set to the not configured state unless flag option 8 is used.

When a reset is requested the automatic variable MM.ONEWIRE will return true if a device was found. This will occur with the ONEWIRE RESET command and the ONEWIRE READ and ONEWIRE WRITE commands if a reset was requested (flag = 1 or 2).

The 1-Wire protocol is often used in communicating with the DS18B20 temperature measuring sensor and to help in that regard MMBasic includes the TEMPR() function which provides a convenient method of directly reading the temperature of a DS18B20 without using these functions.

# Appendix D
# SPI Communications

The Serial Peripheral Interface (SPI) communications protocol is used to send and receive data between integrated circuits. The Raspberry Pi Pico acts as the master (i.e. it generates the clock).

## I/O Pins

Before an SPI interface can be used the I/O pins for the channel must be allocated using the following commands. For the first channel (referred as SPI) it is:

SETPIN rx, tx, clk, SPI

| | | |
|---|---|---|
| Valid pins are | RX: | GP0, GP4, GP16 or GP20 |
| | TX: | GP3, GP7 or GP19 |
| | CLK: | GP2, GP6 or GP18 |

And the following command for the second channel (referred to as SPI2) is:

SETPIN rx, tx, clk, SPI2

| | | |
|---|---|---|
| Valid pins are | RX: | GP8, GP12 or GP28 |
| | TX: | GP11, GP15 or GP27 |
| | CLK: | GP10, GP14 or GP26 |

TX is data from the Raspberry Pi Pico and RX is data to it.

Note that on the WebMite version SPI1 and SPI2 are not available on GP20 to GP28

## SPI Open

To use the SPI function the SPI channel must be first opened.

The syntax for opening the first SPI channel is (use SPI2 for the second channel):

```
SPI OPEN speed, mode, bits
```

Where:

- ‘speed’ is the requested speed for the SPI clock. Any value can be requested, the firmware will select the next speed that can be achieved that is equal or slower than the speed requested. The speed that will actually be set is CPUspeed/2 divided by 1 to 256.
- 'mode' is a single numeric digit representing the transmission mode – see Transmission Format below.
- 'bits' is the number of bits to send/receive. This can be any number in the range of 4 to 16 bits.
- It is the responsibility of the program to separately manipulate the CS (chip select) pin if required.

## Transmission Format

The most significant bit is sent and received first. The format of the transmission can be specified by the 'mode' as shown below. Mode 0 is the most common format.

| Mode | Description | CPOL | CPHA |
|---|---|---|---|
| 0 | Clock is active high, data is captured on the rising edge and output on the falling edge | 0 | 0 |
| 1 | Clock is active high, data is captured on the falling edge and output on the rising edge | 0 | 1 |
| 2 | Clock is active low, data is captured on the falling edge and output on the rising edge | 1 | 0 |
| 3 | Clock is active low, data is captured on the rising edge and output on the falling edge | 1 | 1 |

For a more complete explanation see: http://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus

## Standard Send/Receive

When the first SPI channel is open data can be sent and received using the SPI function (use SPI2 for the second channel). The syntax is:

```
received_data = SPI(data_to_send)
```

Note that a single SPI transaction will send data while simultaneously receiving data from the slave. ‘data_to_send’ is the data to send and the function will return the data received during the transaction. ‘data_to_send’ can be an integer or a floating point variable or a constant.

If you do not want to send any data (i.e. you wish to receive only) any number (eg, zero) can be used for the data to send. Similarly if you do not want to use the data received it can be assigned to a variable and ignored.

## Bulk Send/Receive

Data can also be sent in bulk (use SPI2 for the second channel):

```
SPI WRITE nbr, data1, data2, data3, … etc
SPI WRITE nbr, string$
SPI WRITE nbr, array()
```

In the first method 'nbr' is the number of data items to send and the data is the expressions in the argument list (i.e. 'data1', data2' etc). The data can be an integer or a floating point variable or a constant. The data can also be an expression or an array element such as a(3).

In the second or third method listed above the data to be sent is contained in the 'string$' or the contents of 'array()' (which must be a single dimension array of integer or floating point numbers). The string length, or the size of the array must be the same or greater than nbr. Any data returned from the slave is discarded.

Data can also be received in bulk (use SPI2 for the second channel):

```
SPI READ nbr, var1, var2, var3, … etc
SPI READ nbr, string$
SPI READ nbr, array()
```

Where 'nbr' is the number of data items to be received. In the first method each received value is stored in one of the listed numeric variables (the number of variables must equal 'nbr' and array elements such as a(3) are allowed). In the second method the received data is stored in 'string$' as sequential characters and the string is set to 'nbr' characters. In the third method the received items are stored in consecutive elements of 'array()' which must be a single dimension float or integer array at least 'nbr' elements long. This command sends zeros while reading the data from the slave.

## SPI Close

If required the first SPI channel can be closed as follows (the I/O pins will be set to inactive):

```
SPI CLOSE
```

Use SPI2 for the second channel.

## Examples

The following example shows how to use the SPI port for general I/O. It will send a command 80 (hex) and receive two bytes from the slave SPI device using the standard send/receive function:

```
PIN(10) = 1 : SETPIN 10, DOUT     ' pin 10 will be used as the enable signal
SETPIN GP20, GP3, GP2, SPI        ' assign the I/O pins
SPI OPEN 5000000, 3, 8            ' speed is 5MHz and the data size is 8 bits
PIN(10) = 0                       ' assert the enable line (active low)
junk = SPI(&H80)                  ' send the command and ignore the return
byte1 = SPI(0)                    ' get the first byte from the slave
byte2 = SPI(0)                    ' get the second byte from the slave
PIN(10) = 1                       ' deselect the slave
SPI CLOSE                         ' and close the channel
```

The following is similar to the example given above but this time the transfer is made using the bulk send/receive commands:

```
OPTION BASE 1                     ' our array will start with the index 1
DIM data%(2)                      ' define the array for receiving the data
SETPIN GP20, GP3, GP2, SPI        ' assign the I/O pins
PIN(10) = 1 : SETPIN 10, DOUT     ' pin 10 will be used as the enable signal
SPI OPEN 5000000, 3, 8            ' speed is 5MHz, 8 bits data
PIN(10) = 0                       ' assert the enable line (active low)
SPI WRITE 1, &H80                 ' send the command
SPI READ 2, data%()               ' get two bytes from the slave
PIN(10) = 1                       ' deselect the slave
SPI CLOSE                         ' and close the channel
```

# Appendix E
# Regex Syntax

The alternate forms of the INSTR() and LINSTR() functions can take a regular expression as the search pattern.

The alternate form of the functions are:

**INSTR([start],text$, search$ [,size])**

**LINSTR(text%(),search$ [,start] [,size]**

In both cases specifying the size parameter causes the firmware to interpret the search string as a regular expression. The size parameter is a floating point or integer variable that is used by the firmware to return the size of a matching string. As implemented in MMBasic you need to apply the returned ***start*** and ***size*** values to the **MID$** function to extract the matched string. e.g.

**IF start THEN match$=MID$(text$,start,size) ELSE match$="" ENDIF**

The syntax of regular expressions can vary slightly with the various implementations. This appendix is a summary of the syntax and supported operations used in the MMBasic implementation.

NB: the use of regular expression syntax with OPTION ESCAPE is best avoided otherwise confusion is very likely!

Anchors

^ Start of string
$ End of string
\b Word Boundary
\B Not a word boundary

Quantifiers

* 0 or more (not escaped)
+ 1 or more
? 0 or 1
{3} Exactly 3
{3,} 3 or more
{,5} 5 or less
{3,5} 3,4 or 5

Groups and Ranges

(a|b|c) a or b or c
(…) group
[abc] Range (a or b or c)
[^abc] Not (a or b or c)
[a-q] lower case letters a to q
[A-Q] upper case letters A to Q
[0-7] Digits from 0 to 7

Character Classes

\w digits,and letters plus _
\W non alpha
\s white space \t \f \r \n \v spaces
\S non white space
\d digits
\D non digits
\xXX hex encoded byte

Escapes Required to Match Normal Characters

\^ to match ^ (caret)
\. to match . (dot)
* to match * (asterisk)
\$ to match $ (dollar)
\[ to match [ (left bracket)
\\ to match \ (backslash)
\? to match ? (question mark)
\{ to match { (left curly bracket)
\} to match } (right curly bracket)
\| to match | (pipe)
\( to match ( (left bracket)
\) to match ) (right bracket)
\ + to match + (plus)

Matching rules

Non special characters match themselves

. matches any character

A word boundary is at start or end of a string or where a \w Character has a \W Character adjacent to it.

Limitations

Anchors within a group are not supported.

e.g. (^hello) or (hello$) will not match "hello" at beginning or end of the line as expected.

However, anchors outside the group are ok. e.g. ^(hello) or (hello)$ will match

Example expression to match an IP Address.

```
"[\d]+\.[\d]+\.[\d]+\.[\d]+"
```

## Using Regular expressions with OPTION ESCAPE

When a regular expression is literally embedded in the INSTR or LINSTR function then any syntax using the '\' escape character is handled correctly without further escaping the backslash, whether OPTION ESCAPE is in use or not. e.g.

```
? instr("test123","\d",size)
```

The INSTR / LINSTR functions temporarily turn off OPTION ESCAPE while the regex expression is read.

However, if the Regex expression is passed as a string variable and OPTION ESCAPE is enabled when you assign the regex expression to the variable, you must escape any backslash in the regex expression. e.g.

```
s$="\\d"
? instr("test123",s$,size)
```

# Appendix F
# The PIO Programming Package

## Introduction to the PIO

The RP2040 and RP2350 have many built in peripherals such as PWM, UART, ADC, SPI. Using PIOs it is possible to add specialised functions/peripherals such as high speed serial data interfaces and bit streams.

PIOs can be thought of as cut-down, highly specialised CPU cores. The RP2040 contains two PIO blocks while the RP2350 has three blocks. MMBasic refers to them as PIO0, PIO1 and PIO2 in line with the Raspberry Pi documentation. The PIOs run completely independently of the main system and of each other and run extremely fast, with a throughput of up to 32 bits during every clock cycle.

PIOs implement state machines. Before a state machine can execute its program, the program needs to be written to PIO memory, and the state machine needs to be configured.

This appendix describes the support MMBasic can give in using PIOs. It does not contain an explanation of how to write PIO state machine programs. For better understanding how these work refer to the following thread "PIO explained PICOMITE" on the thebackshed.com forum:
https://www.thebackshed.com/forum/ViewTopic.php?FID=16&TID=15385

## Availability of PIOs

| PicoMite Platform | I2S audio | PIO0 | PIO1 | PIO2 |
|---|---|---|---|---|
| 2040 | | ✓ | ✓ | |
| 2040 | yes | X | ✓ | |
| 2350 | | ✓ | ✓ | ✓ |
| 2350 | yes | ✓ | ✓ | X |
| 2040 VGA | | X | ✓ | |
| 2040 VGA | yes | X | ✓ | |
| 2350 VGA | | X | ✓ | ✓ |
| 2350 VGA | yes | X | ✓ | X |
| 2350 HDMI | | ✓ | ✓ | ✓ |
| 2350 HDMI | yes | ✓ | ✓ | X |
| WebMite2040 | | ✓ | X | |
| WebMite2040 | yes | ✓ | X | |
| 2350 WiFi/BT | | ✓ | ✓ | X |
| 2350 WiFi/BT | yes | ✓ | ✓ | X |

✓ = AVAILABLE X = NOT AVAILABLE

2350 WiFi/BT means WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMIWEB, PicoMiteHDMIBTH and PicoMitePC3. On these PIO2 is used by the wireless chip and the I2S DAC shares it. On the WebMiteRP2040 the wireless chip uses PIO1 and the I2S DAC shares that.

## Overview of PIO

A single PIO block has four independent state machines. All four state machines share a single 32 instruction program area of flash memory. This memory has write-only access from the main system, but has four read ports, one for each state machine, so that each can access it independently at its own speed. Each state machine has its own program counter.

Each state machine also has two 32-bit "scratchpad" registers, X and Y, which can be used as temporary data stores.

I/O pins are accessed via an input/output mapping module that can access 32 pins (but limited to 30 for the RP2040). All state machines can access all the pins independently and simultaneously.

MMBasic can write data into the input end of a 4-word 32-bit wide TX FIFO buffer. The state machine can then use PULL to move the output word of the FIFO into the OSR (Output Shift Register). It can also use OUT to shift 1-32 bits at a time from the OSR into the output mapping module or other destinations. AUTOPULL can be used to automatically pull data until the TX FIFO is empty or reaches a preset level.

MMBasic can read data from the output end of a 4-word 32-bit wide RX FIFO buffer. The state machine can

then use IN to shift 1-32 bits of data at a time from the input mapping module into the ISR (Input Shift Register). It can then use PUSH to move the contents of the ISR into the FIFO.  AUTOPUSH can be used to automatically push data until the RX FIFO is full or reaches a preset level.

The FIFO buffers can be reconfigured to form a single direction 8-word 32-bit FIFO in a single direction. The buffers allow data to be passed to and from the state machines without either the system or the state machine having to wait for the other.

Each of the four state machines in the PIO has four registers associated with it:

- CLKDIV is the clock divider, which has a 16-bit integer divider and an 8-bit fractional divider. This sets how fast the state machine runs. It divides down from the main system clock.
- EXECCTRL holds information controlling the translation and execution of the program memory
- SHIFTCTRL controls the arrangement and usage of the shift registers
- PINCTRL controls which and how the GPIO pins are used.

The four state machines of a PIO have shared access to its block of 8 interrupt flags. Any state machine can use any flag. They can set, reset or wait for them to change. In this way they can be made to run synchronously if required. The lower four flags are also accessible to and from the main system, so the PIO can be controlled from MMBasic or pass interrupts back.

DMA can be used to pass information to and from the PIO block via its FIFO from the RP2040's memory

A PIO has nine possible programming instructions, but there can be many variations on each one. For example, Mov can have up to 8 sources, 8 destinations, 3 process operations during the copy, with optional delay and/or side set operations!

- Jmp  Jump to an absolute address in program memory if a condition is true (or instantly).
- Wait  Stall operation of the state machine until a condition is true.
- In  Shift a number of bits from a source into the ISR.
- Out  Shift a number of bits out of the OSR to a destination.
- Push  Push the contents of the ISR into the RX FIFO as a single 32-bit word.
- Pull  Load a 32-bit word from the TX FIFO into the OSR.
- Mov  Copy date from a source to a destination.
- Irq  Set or clear an interrupt flag.
- Set  Immediately write data to a destination.

Instructions are all 16-bit and contain both the instruction and all data associated with it. All instructions operate in 1 clock cycle, but it is possible to introduce a delay of several idle clock cycles between an instruction and the next.

Additionally, there is a facility called "side-set" which allows a value to be written to some pre-defined output pins while an instruction is being read from memory. This is transparent to the program.

## Programming PIO

PicoMite firmware programs the PIO state machine memory using one of 3 methods. Each method will be explained with an example of the exact same program that toggles one of the GPIO pins of the Raspberry Pi Pico. Which GPIO pin is toggled, is determined by the configuration, not in the program itself.

### PIO ASSEMBLE

This command is used to use the build in assembler to generate the program from mnemonics, then write it directly into PIO memory.

```
PIO ASSEMBLE 1,".program test"          'a program has to have a name
PIO ASSEMBLE 1,".line 0"                'start the program at line 0
PIO ASSEMBLE 1,"SET PINDIRS,1"          'SET the GPIO line to output
PIO ASSEMBLE 1,"label:"                 'define a label called "label"
PIO ASSEMBLE 1,"SET PINS,1"             'SET the GPIO pin high
PIO ASSEMBLE 1,"SET PINS,0"             'SET the GPIO pin low
PIO ASSEMBLE 1,"JMP label"              'JuMP to "label"
PIO ASSEMBLE 1,".end program list"      'end program, list=show result
```

The assembler also allows a better readable format like this

```
PIO ASSEMBLE 1,".program test"    'a program has to have a name
.line 0                           'start the program at line 0
```

```
   SET PINDIRS,1                  'SET the GPIO line to output
.LABEL label:                     'define a label called "label"
   SET PINS,1                     'SET the GPIO pin high
   SET PINS,0                     'SET the GPIO pin low
   JMP label                      'JuMP to "label"
.end program list                 'end program, list=show result
```

### PIO PROGRAM LINE

This command can be used to program 16bit values to individual lines (locations) in the PIO memory.

```
PIO PROGRAM LINE 1,0,&hE081             'SET pin output
PIO PROGRAM LINE 1,1,&hE001             'SET pin high
PIO PROGRAM LINE 1,2,&hE000             'SET pin low
PIO PROGRAM LINE 1,3,&h0001             'JMP to line 1
```

### PIO PROGRAM

This command writes all 32 lines in one PIO from an array. This is useful once a PIO program is debugged. It is extremely compact.

```
DIM a%(7)=(&h0001E0000E001E081,0,0,0,0,0,0,0)
PIO PROGRAM 1,a%()
```

## Configuring PIO

The PicoMite firmware can configure each state machine individually. Configuration allows 2 state machines to run the exact same program lines (eg, an SPI interface) but operate with different GPIO pins and at different speeds. There are several configuration fields.

### FREQUENCY

PicoMite firmware contains a default configuration for each configuration field, except for the frequency. The frequency is set by a 16 bit CLKDIV divider from the ARM clock. Example: when OPTION CPUSPEED 126000 is set the PIO can run at speeds between 126MHz and 1.922kHz (126000000 / 65536). Be aware that higher CPU speeds (overclocking) directly impact the state machine frequency.

### PIN CONTROL

PicoMite firmware defaults the GPIO pins for use by MMBasic. For the PIO to take ownership of a GPIO pin MMBasic needs to assign it to PIO as below.

SETPIN GPxx,PIOx (eg, SETPIN gp0,pio1)

A state machine can SET the state of a pin (SET is a state machine instruction) but can also output serial data to one or more GPIO pins using the OUT instruction. Or read serial data using the IN instruction. And GPIO pins can be set as a side effect of any state machine instruction (SIDE SET). For each method of interfacing, different pins can be mapped to the state machine.

It is important to understand is that these instructions work on consecutive pins. This means that there is a range of pins that can be controlled, starting at the lowest GPx pin number (eg, GP0), and pins next to it can be included (up to 5 pins in total). So GP0,GP1,GP2 is a valid set of IO pins. GP0,GP1,GP6 is not. Consider this when designing a PIO application.

Assigning GPIO pins to a state machine uses the PIO helper function PINCTRL:

PIO(PINCTRL a,b,c,d,e,f,g)

- a/ the number of SIDE SET pins (0...5), SIDE SET can write 5 pins at once
- b/ the number of SET pins (0...5), SET can write 5 pins at once
- c/ the number of OUT pins (0...31), OUT can write 32 pins at once
- d/ the lowest pin for IN pins (GP0.....GP31) IN can read up to 32 pins at once
- e/ the lowest pin for SIDE SET (GP0.....GP31)
- f/ the lowest pin for SET (GP0.....GP31)
- g/ the lowest pin for OUT (GP0.....GP31)

Ranges for the different functions (SET/OUT/SIDESET/IN) can overlap, be identical, or adjacent.

### EXECUTE CONTROL

The execute control register EXECCTRL configures the program flow. There is a field that connects a GPIO pin to a conditional jump (JMP instruction), and fields that hold the line address of the main program loop begin (.WRAP TARGET) and end (.WRAP).

If we want the program flow to change in response of a GPIO pin state, a JMP PIN is used. The specific pin is

assigned in the execute control configuration (there can only be 1 pin per state machine) and the JMP happens only when the pin is high).

The state machine program starts at the beginning and runs until it reaches the end. In above demo program, the program loops from the end to beginning using a (unconditional) JMP instruction. An alternative way to using the JMP instruction is defining the beginning of the loop (WRAP TARGET = line 1) and end of the loop (WRAP = line 2) and configure the state machine to only execute these instructions in between. The JMP instruction in line 3 is obsolete when WRAP/WRAP TARGET is used.

PIO(EXECCTRL a,b,c)

| | |
|---|---|
| a/ the GPIO pin for conditional JMP | (eg, GP0) |
| b/ the WRAP TARGET line number | (eg, 1) |
| c/ the WRAP line number | (eg, 2) |

## SHIFT CONTROL

The IN and OUT instructions shift data from the FIFO register to the GPIO pins. In between MMBasic and the PIO, 32bit words can be communicated. Since both the ARM cores and the PIO microcontrollers operate independently, the data is exchanged through FIFO's. The ARM (MMBasic) puts data in the FIFO, PIO reads it. This uses the TX FIFO. The other way around uses the RX FIFO. The FIFO's are normally 4 words deep but can be configured to a single 8 word deep RX or TX FIFO.

The PIO can "shift" data IN the RX FIFO from the MSB side, or from the LSB side. That is set with the IN SHIFTDIR bit. Similar the OUT SHIFTDIR bit for OUT data. The autopull and autopush flags in combination with the pull and push thresholds determine when FIFO is replenished.

In RP2350 the FIFO's can also be used as individual registers, allowing more flexible communication between MMBasic and the state machines. This is achieved by FJOIN_RX_GET and FJOIN_RX_PUT in the SHIFTCTRL register.

PIO(SHIFTCTRL a,b,c,d,e,f,g,h)

| | |
|---|---|
| a/ push threshold | (1..31 bits shifted IN in ISR before autoPUSH) |
| b/ pull threshold | (1..31 bits are shifted OUT from OSR before autoPULL) |
| c/ autopush | (1 = allow automatic push) |
| d/ autopull | (1 = allow automatic pull) |
| e/ IN-shiftdir | (1 = shift MSB, 0 = shift LSB) |
| f/ OUT-shiftdir | (1 = shift MSB, 0 = shift LSB) |
| g/ fjoin_tx | (join TX and RX fifo to 1 RX fifo) |
| h/ fjoin_rx | (join TX and RX fifo to 1 TX fifo) |
| i/ fjoin_rx_get | (1=ARM write individual RX FIFO registers, 2350 only) |
| j/ fjoin_rx_put | (1=ARM read individual RX FIFO registers, 2350 only) |

## WRITING THE STATE MACHINE CONFIGURATION

A state machine configuration is written using the command:

PIO INIT MACHINE a,b,c,d,e,f,g

| | |
|---|---|
| a/ the PIO | (0,1 or 2(2350 only)) |
| b/ the state machine number | (0...3) |
| c/ frequency | (CPUSPEED/65536...CPUSPEED in Hz) |
| d/ pincontrol value | (PIO(PINCTRL .....)) |
| e/ execute control value | (PIO(EXECCTRL.....)) |
| f/ shiftcontrol value | (PIO(SHIFTCTRL.....)) |
| g/ start address | (0....31, the line at which the state machine starts executing, can be a label) |

## WRITING THE STATE MACHINE CONFIGURATION IN COMPACT FORM

A state machine configuration is configured and written using this single command:

PIO CONFIGURE a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z,aa,bb,cc

| | |
|---|---|
| a/ pio | (0,1 or 2(2350 only)) |
| b/ sm | (0...3) |
| c/ frequency | (CPUSPEED/65535 up to CPUSPEED in Hz) |
| d/ startaddress | (0..31) |
| e/ sidesetbase | (SIDE SET range start with GPx) |
| f/ sidesetno | (0..5...impact on DELAY range) |
| g/ sidesetout | (1=automatically make above defined pins output) |
| h/ setbase | (SET range start with GPy) |
| i/ setno | (0...5) |

| | |
|---|---|
| j/ setout | (1=automatically make above defined pins output) |
| k/ outbase | (OUT/MOV range start with GPz) |
| l/ outno | (0...5) |
| m/ outout | (1=automatically make above defined pins output) |
| n/ inbase | (IN range starts at GPxx) |
| o/ jmppin | (Conditional JMP pin is GPyy) |
| p/ wraptarget | (0..31 = line number for WRAP TARGET) |
| q/ wrap | (0..31 = line number fro WRAP) |
| r/ sideenable | (1 = only update pins explicit when side set is in instruction) |
| s/ sidepindir | (1 = side set changes pin direction, not pin state) |
| t/ pushthreshold | (1..31 bits shifted IN in ISR before autoPUSH) |
| u/ pullthreshold | (1..31 bits are shifted OUT from OSR before autoPULL) |
| v/ autopush | (1 = allow automatic push) |
| w/ autopull | (1 = allow automatic pull) |
| x/ inshiftdir | (1 = shift MSB→LSB, 0 = shift LSB→MSB direction) |
| y/ outshiftdir | (1 = shift MSB→LSB, 0 = shift LSB→MSB direction) |
| z/ joinrxfifo | (1 = RX fifo is 8 deep, TX fifo = 0 |
| aa/ jointxfifo | (1 = TX fifo is 8 deep, RX fifo = 0 |
| bb/ joinrxfifoget | (1 = RX fifo changes to 4 readable registers (2350 only) |
| cc/ joinrxfifoput | (1 = RX fifo changes to 4 writable registers (2350 only) |

### STARTING AND STOPPING A STATE MACHINES

Once the PIO is configured, you can start and stop the state machine using:

```
PIO START a,b
PIO STOP  a,b
```

| | |
|---|---|
| a/ the PIO number | (0,1 or 2(2350 only)) |
| b/ the state machine | (0...3) |

Note that when stopping a state machine, it stops right where it is. To restart the state machine it is advisable to PIO INIT MACHINE first.

### EXAMPLE PROGRAM 1

A complete PIO implementation that toggles a GPIO pins can be implemented in MMBasic as shown below. Connect a buzzer to GP0, and hear the audio tone generated by the PIO.

```
'disconnect ARM from GP0
setpin gp0,pio1                  'use GP0 as output pin for PIO 1

'pio program used
'0 E081   'SET pin output
'1 E001   'SET pin high
'2 E000   'SET pin low
'3 0001   'jmp 1

'program pio 1 using an array to write the program in PIO memory, and start
Dim a%(7)=(&h0001E000E001E081,0,0,0,0,0,0,0)
PIO program 1,a%()

'configure pio 1 statemachine 0
p=Pio(pinctrl 0,1,,,,gp0,)   'define SET uses 1 pin, and that is GP0
f=3100                      '3051 Hz is lowest frequency CPUSPEED 200000
PIO init machine 1,0,f,p     'use default for execctrl, shiftctrl, start
                                 'address(=0)

'start the PIO 1 state machine 0
PIO start 1,0
```

Note that the MMBasic program ends, but the sound on the buzzer continues. PIO is independent of the ARM CPU and continues until it is stopped. Entering the MMBasic editor will stop the PIO.

## FIFO's

MMBasic and the PIO exchange information using FIFO's. The PIO's PUSH data into the RX FIFO (MMBasic is the receiver), or PULL data from the TX FIFO (MMBasic is the transmitter).

When PIO is fetching data from the FIFO the data is transferred to the OSR (Output Shift Register), from there is can be processed. The PIO can push the data from the ISR (Input shift register) into the FIFO. Additionally, the PIO has 2 registers X and Y that can be used for storage, or counting. PIO cannot add or subtract or compare.

Data flow:

MMBasic -> FIFO -> OSR -> PIO (or pins)
PIO (or pins) -> ISR -> FIFO -> MMBasic

MMBasic can write data into the TX FIFO and read data from the RX FIFO using:

PIO READ a,b,c,d
PIO WRITE a,b,c,d

| | |
|---|---|
| a/ PIO number | (0,1 or 2(2350 only)) |
| b/ state machine number | (0...3) |
| c/ number of 32 bit words | (1...4) |
| d/ integer variable name | (i.e. variable% or array%()) |

PIO CLEAR clears all the PIO FIFO's, as does PIO START and PIO INIT MACHINE.

The MMBAsic program doesn't need to wait for data in the FIFO to appear since the RX FIFO can be assigned an interrupt. The MMBasic interrupt routine can fetch the data from the FIFO.

Similar for TX interrupt in which case MMBasic gets an interrupt when data is needed for the TX FIFO.

PIO INTERRUPT a,b,c,d

| | |
|---|---|
| a/ PIO | (0,1 or 2(2350 only)) |
| b/ state machine | (0...3) |
| c/ Name of RX interrupt handler | (i.e. "myRX_Interrupt" or 0 to disable) |
| d/ Name of TX interrupt handler | (i.e. "myTX_Interrupt" or 0 to disable) |

## EXAMPLE PROGRAM 2

Below program explains many of the above presented MMbasic functions and commands. The program reads a NES controller (SPI) connected to the Raspberry Pi Pico. The NES controller consists of a HEF4021 shift register connected to 8 push button switches.

Program uses: **wrap** and **wrap target**, IN, side set and delay, PUSH, PIO READ. GP0 and GP1 are in SET for pin direction, and in side set for compact code.

The wiring is as defined in the code:

```
'disconnect ARM from GP0/1/2
  setpin gp0,pio1                 'clock out
  setpin gp1,pio1                 'load out
  setpin gp2,pio1                 'data in

'PIO program

PIO assemble 1,".program NES"     'a program needs a name
.side set 2                       'use 2 bits for side set, 3 for delay
.line 0                           'start code at line 0
   SET pindirs,&b11               'set GP0,GP1 output, side GP0,GP1 low
.wrap target                      'wrap target = top of the loop
   IN null,32 side 2              'set ISR to 0, GP1 high (load), GP0 low
   SET X,7 side 0                 'set X counter to 7, GP0,GP1 low
.label loop:                      'inner loop label
   IN pins,1 side 0               'shift 1 databit in, keep GP0,GP1 low
   JMP X-- loop side 1            'jmp to loop, dec. X, GP0 'high(clock)
   PUSH side 0 [7]                'now X=0, PUSH result into FIFO, delay 7
.wrap"                            'end outer loop, repeat
.end program list                 'end of program, list result

'configure pio1
  p=Pio(pinctrl 2,2,,gp2,gp0,gp0,)'GP0,GP1 out (SET and SIDESET), GP2 IN
  f=1e5                           '100kHz
  s=PIO(shiftctrl 0,0,0,0,0,0)    'shift in from LSB for IN (and OUT)
  e=PIO(execctrl gp0,PIO(.wrap target),PIO(.wrap))  'wrap and wrap target
```

```
'write the configuration
  PIO init machine 1,0,f,p,e,s,0  'start at line 0

'start the pio1 code
  PIO start 1,0


'Check the read data in MMBasic and print
  dim d%
  do
    pio read 1,0,1,d%
    print bin$(d%)
    pause 200
  loop
  END
```

## DMA To and From the FIFOs

The way that DMA works is as follows:

When reading from the FIFO the DMA controller waits on data being in the FIFO and when it appears transfers that data into processor memory. Each time it reads it increments the pointer into the processor memory so that it can, for example, incrementally fill an array as each and every data item is made available.

When writing to the FIFO the DMA controller writes data from processor memory to the FIFO automatically waiting whenever the FIFO is full. Thus, data can be prepared in an array and the DMA controller will stream that data to the PIO FIFO as fast as the PIO program requires it.

DMA can transfer a 32-bit word, a 16-bit short, or an 8-bit byte and when setting up DMA you need to tell it the size of the transfer and how many transfers to make. Because each transfer will increment the memory pointer by 1,2, or 4 bytes MMBasic must deal with the data packed into memory rather than the 64-bits used for MMbasic integers and floats. Luckily MMBasic implements two commands MEMORY PACK and MEMORY UNPACK to do this very efficiently but it could equally be done using standard BASIC arithmetic.

The DMA can be configured to repeatedly loop data into or out of a section of memory (a ring buffer)

The commands are:

PIO DMA RX a, b, c, d, e, f, g
PIO DMA TX a, b, c, d, e, f, g

| | | |
|---|---|---|
| Where: | a = pio | (0,1 or 2(2350 only)) |
| | b = state machine | (0...3) |
| | c = nbr | (number of words to be transferred) |
| | d = data%() | (integer array name) |
| | e = completioninterrupt | (where to go when done, optional) |
| | f = transfersize | (8 =16 =32, optional) |
| | g = loopbackcount | (used data%() as a ring buffer, optional, loopbackcount = 2^n) |

The DMA will start the state machine automatically and there is no need for a PIO START command. But, before starting the transfer make sure a fresh PIO INIT MACHINE is done, so the state machine starts at the required start address.

When a ring buffer is used, it requires special preparation:

PIO MAKE RING BUFFER a, b

| | | |
|---|---|---|
| Where: | a = name of integer buffer | |
| | b = size of the array in bytes | |

Example :

```
DIM packed%                              'watch out: NO brackets()
PIO MAKE RING BUFFER packed%,4096
```

packed% will then be an integer array holding 4096/8 = 512 integers

This can then be used by the DMA for a loopbackcounter with DMA of 1024 32-bit words, 2048 16-bit shorts or 4096 8-bit bytes.

EXAMPLE PROGRAM 3

This program brings everything together and uses DMA to read 128 samples from the PIO RX FIFO. For the

demonstration, GP0 to GP5 are outputs of 3 PWMS, and are, at the same time, sampled by the PIO as a 6 channel logic analyser or oscilloscope. The 128 samples are sent to the serial port as waveforms.

This Logic Analyzer program also demonstrates PIO DMA RX, MEMORY UNPACK, the use of buffers. It uses PWM's to generate a test signal on gp0..gp5 for demo purpose. These same pins are read by the logic analyzer and output to the console.

To use this Logic Analyzer normally comment out first 14 lines.

```
'generate a 50Hz 3 phase test signal to demonstrate the DMA on 6 GPIO pins.
SetPin gp0,pwm  'CH 0a
SetPin gp1,pwm  'CH 0b
SetPin gp2,pwm  'CH 1a
SetPin gp3,pwm  'CH 1b
SetPin gp4,pwm  'CH 2a
SetPin gp5,pwm  'CH 2b

Fpwm = 50: PW = 100 / 3
PWM 0, Fpwm, PW, PW - 100, 1, 1
PWM 1, Fpwm, PW, PW - 100, 1, 1
PWM 2, Fpwm, PW, PW - 100, 1, 1
PWM sync 0, 100/3, 200/3

'------------------------------ LA code PIO -------------------------
'PIO code to sample GP0..GP6 as elementary logic analyser
PIO clear 1

'in this program the PIO reads GP0..GP5 brute force
'and pushes data into FIFO. The clock speed determines the
'sampling rate. There are 2 instructions per cycle
'taking 10000/2 / 50 = 100 samples per 50Hz cycle.

PIO assemble 1,".program push"
   .line 0
   .wrap target
   IN pins,6                'get 6 bits from GP0..GP5
   PUSH block               'push data when FIFO has room
   .wrap
.end program list

'configuration
f=1e4                                   'PIO run at 10kHz
p=Pio(pinctrl 0,0,0,gp0,,,)             'IN base = GP0
e=Pio(execctrl gp0,PIO(.wrap target),PIO(.wrap)) 'wrap 1 to 0, gp0 is
                                                  'default
s=Pio(shiftctrl 0,0,0,0,0,0)      'shift in through LSB, out is not used

'write the configuration, speed 10kHz (data in FIFO 10us after edge GP0)
PIO init machine 1,0,f,p,e,s,0          'start address = 0

'------------------------- LA code MMBasic ----------------------------
'define memory buffers
Dim a$(1)=("_","-")                     'characters for the printout
length%=64                              'size of the packed array
Dim data%(2*length%-1)                  'array to put the 32 bit samples
                                        'FIFO format
Dim packed%(length%-1)                  'DMA array to pack 32 bit samples
                                        'in 64 bit integers

'let the DMA machine run, and repeat at will
Do
  PIO DMA RX 1,0,2*length%,packed%(),ReadyInt
  print "press any key to restart sampling"
  do:loop while inkey$=""
Loop
End

'--------------------------------SUBS MMBasic -------------------------
```

```
Sub ReadyInt
'stop the PIO and re-init for next run
PIO stop 1,0
PIO init machine 1,0,f,p,e,s,0          'start address = 0

'get the data from the packed DMA buffer and unpack to original 32 bit format
Memory unpack packed%(),data%(),2*length%,32

'Serial output as if logic analyzer traces
For j=0 To 5
  mask%=2^j
  For i=0 To 2*length%-1
    If i<106 Then Print a$(((data%(i) And mask%)=mask%));
  Next i
  Print : Print
Next j
End Sub
```

## EXAMPLE PROGRAM 4

This program runs on RP2350 only and demonstrates the use of the FIFO's as individual registers. The PIO of the RP2350 has specific instructions to support this MOV RXFIFO[y],ISR and MOV OSR,RXFIFO[y].

MMBasic can read/write the 4 individual FIFO registers by use of:

PIO WRITEFIFO a, b, c, d
PIO READFIFO( a, b, c)

| | |
|---|---|
| a/ pio | (0,1 or 2(2350 only)) |
| b/ state machine | (0...3) |
| c/ nbr | (FIFO register 0…3) |
| d/ data% | (32 bit integer value) |

The program configures the FIFO for individual reads, then writes some values in these register. It starts the PIO that updates 2 of the 4 individual FIFO registers. Then MMBasic reads the values to show only 2 register have changes, and no data shifted (as would happen in RP2040 FIFO).

```
'only for RP2350 assembler. this does not work on RP2040

pio clear 1

pio assemble 1,".program test"
.line 0
set y,4                 'just a value 4 in Y
mov isr,y               'copy Y to isr
jmp y--,next            'y=y-1 always to next
.label next             'label
mov rxfifo[y],isr       'should program 4 in fifo [3]
mov isr,y               'copy Y to isr
mov rxfifo[2],isr       'should program 3 in fifo [2]
jmp 0                   'repeat
.end program

f=1e6 '1MHz

'PIO(EXECCTRL a,b,c)
e=pio(execctrl gp0,0,31)            'default value, not actually changed

'PIO(SHIFTCTRL a,b,c,d,e,f,g,h,i,j)
sr=pio(shiftctrl 0,0,0,0,0,0,0,0,0,1) 'read individual RX, RX=4 deep
sw=pio(shiftctrl 0,0,0,0,0,0,0,0,1,0) 'write individual RX, RX=4 deep

'PIO(PINCTRL a,b,c,d,e,f,g)
p=pio(pinctrl 0,0,0,gp0,gp0,gp0,gp0) 'defaultvalue , not actually changed

'test the use of FIFO as individual registers
'fill the fifo with pre-determined values
```

```
pio init machine 1,0,f,p,e,sw,0   'init machine for writing to RX fifo
for i=0 to &h3                    'write the 4 RXFIFO registers
 pio writefifo 1,0,i,&h100*(i+1)  'write values &h100, &h200, &h300, &h400
next

'check the values are written correctly
print "3 RXFIFO registers before running the program"
pio init machine 1,0,f,p,e,sr,0   'init machine for reading the RX fifo
for i=0 to 3                      'read the 4 RXFIFO registers
   print i,hex$(pio(readfifo 1,0,i))  'verify they are correctly written
next
print

'run the PIO program. That should (continuously) write to reg 2 and 3 in the FIFO,
'but not alter register 0 and 1
pio start 1,0

'show the updated FIFO registers
print "3 RXFIFO registers after running the program"
for i=0 to 3      'read the 4 FIFO registers to see if the program works
   print i,hex$(pio(readfifo 1,0,i))
next
```

The expected output is:

```
3 RXFIFO registers before running the program
0      100
1      200
2      300
3      400

3 RXFIFO registers after running the program
0      100
1      200
2      3
3      4
```

EXAMPLE PROGRAM 5

Finally an example program showing how multiple state machines can work together, each one triggered from an interrupt from the other one. Note also the use of .LINE NEXT as starting point of the second program, and the use of x=PIO(NEXT LINE) to determine the starting point of the second program.

IRQ 1 sets and IRQ. When the other state machine is waiting WAIT 1 IRQ 1, it continues after IRQ 1 found set, and clears the IRQ 1 at same time.

```
'test for PIO IRQ's on PIO 0 state machines 0 and 1

  'uses GP0 and GP1
  setpin gp0,pio0
  setpin gp1,pio0
  pio clear 0

  'target frequency for PIO
  f=1e5 '100kHz

pio assemble 0
  .program sm0
  .line 0
    set pindirs,1                 'set GP0 out, see pinctrl
  .wrap target
    set pins,1 [31]               'set GP0 high, wait 31 cycles
    set pins,0 [31]               'set GP0 low, wait 31 cycles
    irq 0                         'set IRQ 0
    wait 1 irq 1                  'wait until sm1 sets IRQ 1
```

```
  .wrap
.end program list

  ln=pio(next line)                 'remember the line in the program
  p0=pio(pinctrl 0,1,,,,gp0)        'GP0 is pin for SET
  e0=pio(execctrl gp0,pio(.wrap target),pio(.wrap))

pio assemble 0
  .program sm1
  .line next
    set pindirs,1                   'set GP1 output
  .wrap target
    wait 1 irq 0                    'wait for IRQ 0
    set pins,1 [31]                 'set GP1 high and wait 31 cycles
    set pins,0 [31]                 'set GP1 low and wait 31 cycles
    irq 1                           'set IRQ 1
  .wrap
.end program list

  p1=pio(pinctrl 0,1,,,,gp1)
  e1=pio(execctrl gp0,pio(.wrap target),pio(.wrap))

  pio init machine 0,1,f,p1,e1,,ln  'start sm1 at line ln
  pio init machine 0,0,f,p0,e0,,0   'start sm0 at line 0

  pio start 0,1
  pio start 0,0

do:loop 'do nothing, let PIO generate the signals

end
```

# Appendix G
# Sprites
## VGA, HDMI and LCD FRAMEBUFFERS

You can create a sprite in various ways but essentially you are just storing an image in a buffer. The difference comes when you SHOW the sprite. In this case, first time in, the firmware stores the area of memory (or display real-estate) that will be replaced by the sprite and then draws the sprite in its place.

Subsequent SHOW commands replace the sprite with the stored background, store the background for the new location and finally draw the sprite. In this way you can move the sprite over the background without any extra code.

Collision detection then sits on top of this and looks for the rectangular boundaries of sprites touching to create an interrupt or a sprite touching the edge of the frame.

Sprites are ordered so the drawing order is held in a lifo. suppose you have sprite 1 overlapped by sprite 2 and then by sprite 3. If you simply moved sprite 1 then its background would overwrite bits of 2 and 3 – not what we want. SPRITE SHOW SAFE unwinds the LIFO by removing each sprite in reverse order, moves sprite 1 and then restores first 2 and then 3 on top of it. Finally there is the concept of layers (this is the 4th parameter in SPRITE SHOW).

The concept of the sprite implementation is as follows:

- Sprites are full colour and of any size. The collision boundary is the enclosing rectangle.
- Sprites are loaded to a specific number (1 to 64).
- Sprites are displayed using the SPRITE SHOW command.
- For each SHOW command the user must select a "layer". This can be between 0 and 10.
- Sprites collide with sprites on the same layer, layer 0, or the screen edge.
- Layer 0 is a special case and sprites on all other layers will collide with it.
- The SCROLL commands leave sprites on all layers except layer 0 unmoved.
- Layer 0 sprites scroll with the background and this can cause collisions.
- There is no practical limit on the number of collisions caused by SHOW or SCROLL commands.
- The SPRITE() function allows the user to fully interrogate the details of a collision.
- A SHOW command will overwrite the details of any previous collisions for that sprite.
- A SCROLL command will overwrite details of previous collisions for ALL sprites.
- To restore a screen to a previous state sprites should be removed in the opposite order to which they were written (ie, last in first out).

Because moving a sprite or, particularly, scrolling the background can cause multiple sprite collisions it is important to understand how they can be interrogated.

The best way to deal with a sprite collision is using the interrupt facility. A collision interrupt routine is set up using the SPRITE INTERRUPT command. Eg:

```
SPRITE INTERRUPT collision
```

The following is an example program for identifying all collisions that have resulted from either a SPRITE SHOW command or a SCROLL command

```
'
' This routine demonstrates a complete interrogation of collisions
'
SUB collision
  LOCAL INTEGER i
' First use the SPRITE(S) function to see what caused the interrupt
  IF SPRITE(S) <> 0 THEN 'collision of specific individual sprite
    'SPRITE(S) returns the sprite that moved to cause the collision
    PRINT "Collision on sprite ", SPRITE(S)
    process_collision(SPRITE(S))
```

```
    PRINT
  ELSE     '0 means collision of one or more sprites caused by background move
           ' SPRITE(C, 0) will tell us how many sprites had a collision
    PRINT "Scroll caused a total of ", SPRITE(C,0)," sprites to have collisions"
    FOR I = 1 TO SPRITE(C, 0)
      ' SPRITE(C, 0, i) will tell us the sprite number of the "I"th sprite
      PRINT "Sprite ", SPRITE(C, 0, i)
      process_collision(SPRITE(C, 0, i))
    NEXT i
    PRINT
  ENDIF
END SUB

' get details of the specific collisions for a given sprite
SUB process_collision(S AS INTEGER)
  LOCAL INTEGER i, j
  ' SPRITE(C, #n) returns the number of current collisions for sprite n
  PRINT "Total of " SPRITE(C, S) " collisions"
   FOR I = 1 TO SPRITE(C, S)
    ' SPRITE(C, S, i) will tell us the sprite number of the "I"th sprite
    j = SPRITE(C, S, i)
    IF j = &HF1 THEN
      PRINT "collision with left of screen"
    ELSE IF j = &HF2 THEN
      PRINT "collision with top of screen"
    ELSE IF j = &HF4 THEN
      PRINT "collision with right of screen"
    ELSE IF j = &HF8 THEN
      PRINT "collision with bottom of screen"
    ELSE
      ' SPRITE(C, #n, #m) returns details of the mth collision
      PRINT "Collision with sprite ", SPRITE(C, S, i)
    ENDIF
  NEXT i
END SUB
```

# Appendix H
# Turtle Graphics

This release contains a very comprehensive turtle graphics implementation for all versions except the WebMite RP2040 and WebMite RP2350.

The supported commands with "TURTLE " prepended are:

Where the name of a command includes COLOUR the American spelling COLOR is accepted as well, in both the two word and the one word form. For example TURTLE PEN COLOUR, TURTLE PEN COLOR, TURTLE PENCOLOUR, TURTLE PENCOLOR and TURTLE PC are all equivalent, and the same applies to FILL COLOUR and CURSOR COLOUR.

Most of the two word command names may also be written as a single word, and several have extra alternatives: BACK (BACKWARD), TURN LEFT and TURN RIGHT for LEFT and RIGHT, SET XY (SETXY, MOVE), SET X (SETX), SET Y (SETY), SET HEADING (SETHEADING, HEADING), PEN UP (PENUP), PEN WIDTH (PENWIDTH), ARC LEFT (ARCLEFT), ARC RIGHT (ARCRIGHT), FILL PATTERN (FILLPATTERN), NO FILL (NOFILL), BEGIN FILL (BEGIN_FILL), END FILL (END_FILL), SHOW TURTLE (SHOWTURTLE), HIDE TURTLE (HIDETURTLE) CURSOR SIZE (CURSORSIZE) and PEN DOWN (PENDOWN).

Movement Commands

| | |
|---|---|
| FORWARD distance | (FD) - Move forward by distance pixels |
| BACK distance | (BK) - Move backward by distance pixels |
| LEFT [angle] | (LT) - Turn left by angle degrees, 90 degrees if not specified |
| RIGHT [angle] | (RT) - Turn right by angle degrees, 90 degrees if not |

Position Commands

| | |
|---|---|
| SET XY x, y | - Move to absolute position (x,y) |
| SET X x | - Set X coordinate, keep Y |
| SET Y y | - Set Y coordinate, keep X |
| SET HEADING angle | (SETH) - Set absolute heading (0=up, 90=right) |
| HOME | - Return to center (MM.HRES\2,MM.VRES\2) heading 0 |

Pen Control Commands

| | |
|---|---|
| PEN UP | (PU) - Lift pen (stop drawing) |
| PEN DOWN | (PD) - Lower pen (start drawing) |
| PEN COLOUR color | (PC) - Set pen color |
| PEN WIDTH width | (PW) - Set pen line width |

Arc and Curve Commands

| | |
|---|---|
| ARC radius angle | - Draw arc with given radius and angle |
| ARCLEFT radius,angle | (ARCL) - Draw arc turning left |
| ARCRIGHT radius,angle | (ARCR) - Draw arc turning right |
| BEZIER cp1 , cp1angle, cp2, cp2angle, end, endangle | - Draw Bezier curve with control points |

Basic Shape Commands

| | |
|---|---|
| CIRCLE radius | - Draw circle at current position |
| DOT size | - Draw filled dot (default size=5) |
| FCIRCLE radius | - Draw filled circle |
| FRECTANGLE width,height | (FRECT) - Draw filled rectangle |
| ARECTANGLE width,height | (ARECT) - Draw rectangle outline |
| WEDGE radius start end | - Draw filled wedge/pie slice |

Fill Commands

| | |
|---|---|
| FILL COLOUR color | (FC) - Set fill color and enable filling |
| FILL PATTERN pattern | (FP) - Set fill pattern (0-31) |
| NO FILL | - Disable filling |

| | |
|---|---|
| BEGIN FILL | (BF) - Start recording polygon for fill |
| END FILL | (EF) - End recording and fill polygon |

Cursor Commands

| | |
|---|---|
| SHOW TURTLE | (ST) - Show turtle cursor |
| HIDE TURTLE | (HT) - Hide turtle cursor |
| CURSOR SIZE size | (CS) - Set cursor size |
| CURSOR COLOUR color | CC) - Set cursor color |
| STAMP | - Draw a turtle at the current x,y position |

State Management Commands

| | |
|---|---|
| RESET [show] | - clear screen and reset everything, show the turtle if show = 1 |
| PUSH | - Save current position and heading to stack |
| POP | - Restore position and heading from stack |

The code includes the ability to fill circles,rectangles and polygons with a textured fill. Run it in mode 2 on a RP2040 VGA system or mode 3 on a RP2350 VGA or HDMI.

Fill Patterns

0: Solid fill
1: Checkerboard
2: Vertical lines
3: Horizontal lines
4: Diagonal cross
5: Diagonal stripes
6: Crosshatch
7: Fine diagonal
8: Dense checkerboard
9: Diagonal right medium
10: Diagonal left medium
11: Vertical lines medium
12: Horizontal lines medium
13: Large checkerboard
14: Dotted vertical
15: Horizontal stripes tight
16: Grid
17: Weave pattern
18: Diamond
19: Gradient diagonal
20: Gradient diagonal reverse
21: Border/frame
22: Vertical split
23: Woven
24: Sparse dots
25: Diagonal very fine
26: Arrow up
27: Dense dots
28: Chevron
29: Diamond hollow

30: Circle
31: Circle filled

# Appendix I
## Special Keyboard Keys

MMBasic generates a single unique character for the function keys and other special keys on the keyboard. NB: The DE USB keyboard returns codes 200-209 for accented keyboard characters
with normal input (input$/inkey$)

The codes are shown in this table as hexadecimal and decimal numbers:

| Keyboard Key | Key Code (Hex) | Key Code (Decimal) |
|---|---|---|
| DEL | 7F | 127 |
| Up Arrow | 80 | 128 |
| Down Arrow | 81 | 129 |
| Left Arrow | 82 | 130 |
| Right Arrow | 83 | 131 |
| Insert | 84 | 132 |
| Home | 86 | 134 |
| End | 87 | 135 |
| Page Up | 88 | 136 |
| Page Down | 89 | 137 |
| Alt | 8B | 139 |
| F1/Shift F1 | 91/B1 | 145/177 |
| F2/Shift F2 | 92/B2 | 146/178 |
| F3/Shift F3 ** | 93/B3 | 147/179 |
| F4/Shift F4 ** | 94/B4 | 148/180 |
| F5/Shift F5 ** | 95/B5 | 149/181 |
| F6/Shift F6 ** | 96/B6 | 150/182 |
| F7/Shift F7 ** | 97/B7 | 151/183 |
| F8/Shift F8 ** | 98/B8 | 152/184 |
| F9/Shift F9 | 99/B9 | 153/185 |
| F10/Shift F10 | 9A/BA | 154/186 |
| F11/Shift F11 | 9B/BB | 155/187 |
| F12/Shift F12 | 9C/BC | 156/188 |
| PrtScr/SysRq | 9D | 157 |
| PAUSE/BREAK | 9E | 158 |
| SHIFT_TAB | 9F | 159 |
| SHIFT_DEL | A0 | 160 |
| SHIFT_DOWN_ARROW | A1 | 161 |
| SHIFT_LEFT_ARROW | A2 | 162 |
| SHIFT_RIGHT_ARROW | A3 | 163 |

| | | |
|---|---|---|
| SHIFT_UP_ARROW | A4 | 164 |

** indicates also work for VT100 emulators

For attached PS2 and USB keyboards, if the shift key is simultaneously pressed with the function keys F1 to F12 then 20 (hex) is added to the code (this is the equivalent of setting bit 5). For example Shift-F10 will generate BA (hex).

The shift modifier works with the function keys F1 to F12; it is ignored for the other keys except TAB, DEL and the four arrow keys as identified above. MMBasic will translate most VT100 escape codes generated by terminal emulators such as TeraTerm and Putty to these codes (the shift modifier only works for F3-F8). This means that a terminal emulator operating over a USB or a serial port opened as console will generate the same key codes as a directly attached keyboard.

# Appendix J
# Programming in BASIC - A Tutorial

The BASIC language was introduced in 1964 by Dartmouth College in the USA as a computer language for teaching programming and accordingly it is easy to use and learn. At the same time, it has proved to be a competent and powerful programming language and as a result it became very popular in the late 70s and early 80s. Even today some large commercial data systems are still written in the BASIC language (primarily Pick Basic).

The BASIC interpreter used in the PicoMite firmware's is called MMBasic and is a modern version of the BASIC language which loosely emulates the Microsoft BASIC interpreter that was popular years ago.

For a programmer the greatest advantage of BASIC is its ease of use. Some more modern languages such as C and C++ can be truly mind bending but with BASIC you can start with a one line program and get something sensible out of it. MMBasic is also powerful in that you can draw sophisticated graphics, manipulate the external I/O pins to control other devices and communicate with other devices using a range of built-in communications protocols.

## Command Prompt

Interaction with MMBasic is done via the console at the command prompt (ie, the greater than symbol (>) on the console). On startup MMBasic will issue the command prompt and wait for some command to be entered. It will also return to the command prompt if your program ends or if it generated an error message.

When the command prompt is displayed you have a wide range of commands that you can enter and execute. Typically they would list the program held in memory (LIST) or edit it (EDIT) or perhaps set some options (the OPTION command). Most times the command is just RUN which instructs MMBasic to run the program held in program memory.

Almost any command can be entered at the command prompt and this is often used to test a command to see how it works. A simple example is the PRINT command (more on this later), which you can test by entering the following at the command prompt:

```
PRINT 2 + 2
```

and not surprisingly MMBasic will print out the number 4 before returning to the command prompt.

This ability to test a command at the command prompt is useful when you are learning to program in BASIC, so it would be worthwhile having a Raspberry Pi Pico loaded with the PicoMite firmware handy for the occasional test while you are working through this tutorial.

## Structure of a BASIC Program

A BASIC program starts at the first line and continues until it runs off the end or hits an END command - at which point MMBasic will display the command prompt (>) on the console and wait for something to be entered.

A program consists of a number of statements or commands, each of which will cause the BASIC interpreter to do something (the words *statement* and *command* generally mean the same and are used interchangeable in this tutorial).

Normally each statement is on its own line but you can have multiple statements in the one line separated by the colon character (:).

For example;

```
A = 24.6 : PRINT A
```

Each line can start with a line number. Line numbers were mandatory in the early BASIC interpreters however modern implementations (such as MMBasic) do not need them. You can still use them if you wish but they have no benefit and generally just clutter up your programs.

This is an example of a program that uses line numbers:

```
50 A = 24.6
60 PRINT A
```

A line can also start with a label which can be used as the target for a program jump using the GOTO command. This will be explained in more detail when we cover the GOTO command but this is an example (the label name is JmpBack):

```
JmpBack: A = A + 1
PRINT A
GOTO JmpBack
```

### Comments

A comment is any text that follows the single quote character ('). A comment can be placed anywhere and extends to the end of the line. If MMBasic runs into a comment it will just skip to the end of it (ie, it does not take any action regarding a comment).

Comments should be used to explain non obvious parts of the program and generally inform someone who is not familiar with the program how it works and what it is trying to do. Remember that after only a few months a program that you have written will have faded from your mind and will look strange when you pick it up again. For this reason you will thank yourself later if you use plenty of comments.

The following are some examples of comments:

```
' calculate the hypotenuse
PRINT SQR(a * a + b * b)
INPUT var    ' get the temperature
```

Older BASIC programs used the command REM to start a comment and you can also use that if you wish but the single quote character is easier to use and more convenient.

### The PRINT Command

There are a number of common commands that are fundamental and we will cover them in this tutorial but arguably the most useful is the PRINT command. Its job is simple; to print something on the console. This is mostly used to output data for you to see (like the result of calculations) or provide informative messages.

PRINT is also useful when you are tracing a fault in your program; you can use it to print out the values of variables and display messages at key stages in the execution of the program.

In its simplest form the command will just print whatever is on its command line. So, for example:

```
PRINT 54
```

will display on the console the number 54 followed by a new line.

The data to be printed can be something simple like this or an expression, which means something to be calculated. We will cover expressions in more detail later but as an example the following:

```
> PRINT 3/21
 0.1428571429
>
```

would calculate the result of three divided by twenty one and display it. Note that the greater than symbol (>) is the command prompt produced by MMBasic – you do not type that in.

Other examples of the PRINT command include:

```
> PRINT "Wonderful World"
Wonderful World
> PRINT (999 + 1) / 5
 200
>
```

You can try these out at the command prompt.

The PRINT command will also work with multiple values at the same time, for example:

```
> PRINT "The first number is" 20+25 " and the second is" 18/3
The first number is 45 and the second is 6
>
```

Normally each value is separated by a space character as shown in the previous example but you can also separate values with a comma (,). The comma will cause a tab to be inserted between the two values. In MMBasic tabs in the PRINT command are eight characters apart.

To illustrate tabbing, the following command prints a tabbed list of numbers:

```
> PRINT 12, 34, 9.4, 1000
 12      34       9.4     1000
>
```

Note that there is a space printed before each number. This space is a place holder for the minus symbol (-) in case the value is negative. You can see the difference with the numbers 12 and 9.4 in this example:

```
> PRINT -12, 34, -9.4, 1000
-12      34      -9.4     1000
>
```

The print statement can be terminated with a semicolon (;). This will prevent the PRINT command from moving to a new line when it has printed all the text. For example:

```
PRINT "This will be";
PRINT " printed on a single line."
```

Will result in this output:

```
This will be printed on a single line.
```

The message would be look like this without the semicolon at the end of the first line:

```
This will be
 printed on a single line.
```

### Variables

Before we go much further we need to define what a "variable" is as they are fundamental to the operation of the BASIC language (in fact, most programming languages). A variable is simply a place to store an item of data (ie, its "value"). This value can be changed as the program runs which why it is called a "variable".

Variables in MMBasic can be one of three types. The most common is floating point and this is automatically assumed if the type of the variable is not specified. The other two types are integer and string and we will cover them later. A floating point number is an ordinary number which can contain a decimal point. For example 3.45 or -0.023 or 100.00 are all floating point numbers.

A variable can be used to store a number and it can then be used in the same manner as the number itself, in which case it will represent the value of the last number assigned to it.

As a simple example:

```
A = 3
B = 4
PRINT A + B
```

will display the number 7. In this case both A and B are variables and MMBasic used their current values in the PRINT statement. MMBasic will automatically create a variable when it first encounters it, so the statement `A = 3` both created a floating point variable (the default type) with the name of `A` and then it assigned the value of 3 to it.

The name of a variable must start with a letter while the remainder of the name can use letters, numbers, the underscore or the full stop (or period) characters. The name can be up to 31 characters long and the case (ie, capitals or not) is not important. Here are some examples:

```
Total_Count
ForeColour
temp3
count
x
ThisIsAVeryLongVariableName
increment.value
```

You can change the value of a variable anywhere in your program by using the assignment command, ie:

```
variable = expression
```

For example:

```
temp3 = 24.6
count = 5
CTemp = (FTemp - 32) * 0.5556
```

In the last example both CTemp and FTemp are variables and this line converts the value of FTemp (in degrees Fahrenheit) to degrees Celsius and stores the result in the variable CTemp.

## Expressions

We have met the term 'expression' before in this tutorial and in programming it has a specific meaning. It is a formula which can be resolved by the BASIC interpreter to a single number or value.

MMBasic will evaluate numeric expressions using the same rules that we learnt at school. For example, multiplication and division are performed first followed by addition and subtraction. These are called the rules of precedence and are fully described previously in this manual (see the chapter *Expressions and Operators*).

This means that MMBasic will resolve 2 + 3 * 6 by first multiplying 3 by 6 giving 18 then adding 2 resulting in a final value of 20. Similarly, both 5 * 4 and 10 + 4 * 3 – 2 will also resolve to 20.

If you want to force the interpreter to evaluate parts of the expression first you can surround that part of the expression with brackets. For example, (10 + 4) * (3 – 2) will resolve to 14 not 20 as would have been the case if the brackets were not used. Using brackets does not appreciably slow down the program so you should use them liberally if there is a chance that MMBasic will misinterpret your intention.

As pointed out earlier, you can use variables in an expression exactly the same as straight numbers. For example, this will increment the value of the variable `temp` by one:

```
temp = temp + 1
```

You can also use functions in expressions. These are special operations provided by MMBasic, for example to calculate trigonometric values.

An example of using a function is the following which will print the length of the hypotenuse of a right angled triangle. This uses the SQR() function which returns the square root of a number (a and b are variables holding the lengths of the other sides):

```
PRINT SQR(a * a + b * b)
```

MMBasic will first evaluate this expression by multiplying a by a, then multiplying b by b, then adding the results together. The resulting number is then passed to the SQR() function which will calculate the square root of that number (ie, the hypotenuse) and return it for the PRINT command to display.

Some other mathematical functions provided by MMBasic include:

SIN(r) – the sine of r
COS(r) – the cosine of r
TAN(r) – the tangent of r

There are many more functions available to you and they are all listed earlier in this manual.

Note that in the above trigonometric functions the value passed to the function (ie, 'r') is the angle in radians. In MMBasic you can use the function RAD(d) to convert an angle from degrees to radians ('d' is the angle in degrees).

Another feature of most programming languages (including BASIC) is that you can nest function calls within each other. For example, given the angle in degrees (ie, 'd') the sine of that angle can be found with this expression:

```
PRINT SIN(RAD(d))
```

In this case MMBasic will first take the value of d and convert it to radians using the RAD() function. The output of this function then becomes the input to the SIN() function.

### The IF Statement

Making decisions is at the core of most computer programs and in BASIC this is usually done with the IF statement. This is written almost like an English sentence:

```
IF condition THEN action
```

The *condition* is usually a comparison such as equals, less than, more than, etc.

For example:

```
IF Temp < 25 THEN PRINT "Cold"
```

Temp would be a variable holding the current temperature (in ºC) and PRINT "Cold" the action to be done.

There are a range of tests that you can make:

| | | | |
|---|---|---|---|
| = | equals | <> | not equal |
| < | less than | <= | less than or equals |
| > | greater than | >= | greater than or equals |

You can also add an ELSE clause which will be executed if the initial condition tested false:

```
IF condition THEN true-action ELSE false-action
```

For example, this will execute different actions when the temperature is under 25 or 25 or more:

```
IF Temp < 25 THEN PRINT "Cold" ELSE PRINT "Hot"
```

The previous examples all used single line IF statements but you can also use a multiline IF statement. They look like this:

```
IF condition THEN
    true-action
    true-action
ENDIF
```

```
IF condition THEN
    true-action
    true-action
ELSE
    false-action
    false-action
ENDIF
```

Unlike the single line IF statement you can have many true actions with each on their own line and similarly many false actions. Generally the single line IF statement is handy if you have a simple action that needs to be taken while the multiline version is much easier to understand if the actions are numerous and more complicated.

An example of a multiline IF statement with more than one action is:

```
IF Amount < 100 THEN
    PRINT "Too low"
    PRINT "Minimum value is 100"
ELSE
    PRINT "Input accepted"
    SaveToSDCard
    PRINT "Enter second amount"
ENDIF
```

Note that in the above example each action is indented to show what part of the IF structure it belongs to. Indenting is not mandatory but it makes a program much easier to understand for someone who is not familiar with it and therefore it is highly recommended.

In a multiline IF statement you can make additional tests using the ELSE IF command. This is best explained by using an example (the temperatures are all in ºC):

```
IF Temp < 0 THEN
    PRINT "Freezing"
ELSE IF Temp < 20 THEN
    PRINT "Cold"
ELSE IF Temp < 35 THEN
    PRINT "Warm"
ELSE
    PRINT "Hot"
ENDIF
```

The ELSE IF uses the same tests as an ordinary IF (ie, <, <=, etc) but that test will only be made if the preceding test was false. So, for example, you will only get the message *Warm* if `Temp < 0` failed, and `Temp < 20` failed but `Temp < 35` was true. The final ELSE will catch the case where all the tests were false.

An expression like `Temp < 20` is evaluated by MMBasic as either true or false with true having a value of one and false zero. You can see this if you entered the following at the console:

```
PRINT 30 > 20
```

MMBasic will print `1` meaning that the value of the expression is true.

Similarly the following will print `0` meaning that the expression evaluated to false.

```
PRINT 30 < 20
```

The IF statement does not really care about what the condition actually is, it just evaluates the condition and if the result is zero it will take that as false and if non zero it will take it as true.

This allows for some handy shortcuts. For example, if BalanceCorrect is a variable that is true (non zero) when some feature of the program is correct then the following can be used to make a decision based on that value:

```
IF BalanceCorrect THEN …do something…
```

### FOR Loops

Another common requirement in programming is repeating a set of actions. For instance, you might want to step through all seven days in the week and perform the same function for each day. BASIC provides the FOR loop construct for this type of job and it works like this:

```
FOR day = 1 TO 7
  Do something based on the value of 'day'
NEXT day
```

This starts by creating the variable day and assigning the value of 1 to it. The program will then execute the following statements until it comes to the NEXT statement. This tells the BASIC interpreter to increment the value of day, go back to the previous FOR statement and re-execute the following statements a second time. This will continue looping around until the value of day exceeds 7 and the program will then exit the loop and continue with the statements following the NEXT statement.

As a simple example, you can print the numbers from one to ten like this:

```
FOR nbr = 1 TO 10
  PRINT nbr,;
NEXT nbr
```

The comma at the end of the PRINT statement tells the interpreter to tab to the next tab column after printing the number and the semicolon will leave the cursor on this line rather than automatically moving to the next line. As a result, the numbers will be printed in neat columns across the page.

This is what you would see:

```
1       2       3       4       5       6       7       8       9       10
```

The FOR loop also has a couple of extra tricks up it sleeves. You can change the amount that the variable is incremented by using the STEP keyword. So, for example, the following will print just the odd numbers:

```
FOR nbr = 1 TO 10 STEP 2
  PRINT nbr,;
NEXT nbr
```

The value of the step (or increment value) defaults to one if the STEP keyword is not used but you can set it to whatever number you want.

When MMBasic is incrementing the variable it will check to see if the variable has exceeded the TO value and, if it has, it will exit from the loop. So, in the above example, the value of nbr will reach nine and it will be printed but on the next loop nbr will be eleven and at that point execution will leave the loop. This test is also applied at the start of the loop. For example, if in the beginning the value of the variable exceeds the TO value, the loop will never be executed, not even once.

By setting the STEP value to a negative number you can use the FOR loop to step down from a high number to low. In that case the starting number must be greater than the TO number.

For example, the following will print the numbers from 1 to 10 in reverse:

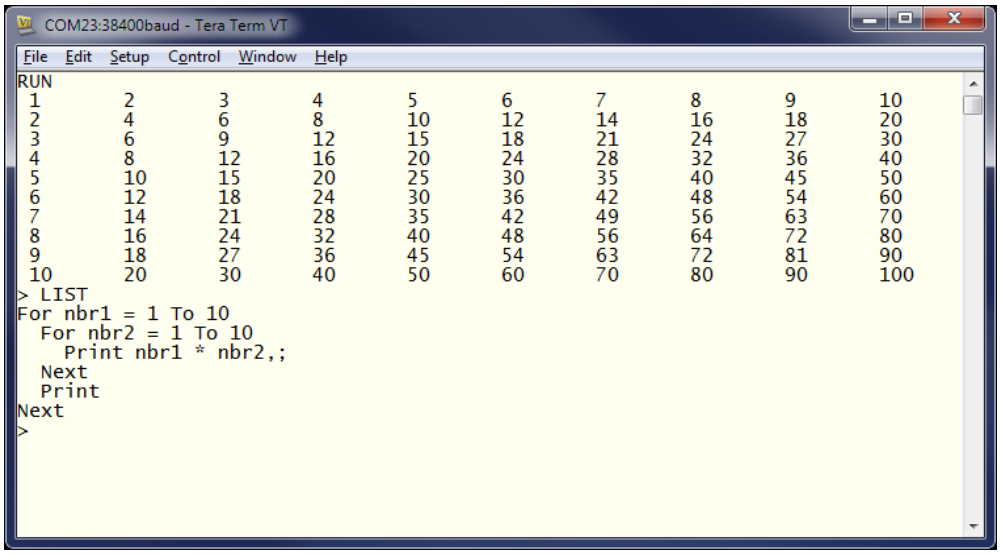
```
FOR nbr = 10 TO 1 STEP -1
  PRINT nbr,;
NEXT nbr
```

### Multiplication Table

To further illustrate how loops work and how useful they can be, the following short program uses two FOR loops to print out the multiplication table that we all learnt at school. The program for this is not complicated:

```
FOR nbr1 = 1 to 10
  FOR nbr2 = 1 to 10
    PRINT nbr1 * nbr2,;
  NEXT nbr2
  PRINT
NEXT nbr1
```

The output is shown in the following screen grab, which also shows a listing of the program.




You need to work through the logic of this example line by line to understand what it is doing. Essentially it consists of one loop inside another. The inner loop, which increments the variable `nbr2` prints one horizontal line of the table. When this loop has finished it will execute the following PRINT command which has nothing to print - so it will simply output a new line (ie, terminate the line printed by the inner loop).

The program will then execute another iteration of the outer loop by incrementing `nbr1` and re-executing the inner loop again. Finally, when the outer loop is exhausted (when `nbr1` exceeds 10) the program will reach the end and terminate.

One last point, you can omit the variable name from the NEXT statement and MMBasic will guess which variable you are referring to. However, it is good practice to include the name to make it easier for someone else who is reading the program to understand it. You can also terminate multiple loops using a comma separated list of variables in the NEXT statement. For example:

```
FOR var1 = 1 TO 5
  FOR var2 = 10 to 13
    PRINT var1 * var2
NEXT var1, var2
```

### DO Loops

Another method of looping is the DO…LOOP structure which looks like this:

```
DO WHILE condition
   <statement>
   <statement>
LOOP
```

This will start by testing the *condition* and if it is true the statements will be executed until the LOOP command is reached, at which point the program will return to DO statement and the *condition* will be tested again, and if it is still true the loop will execute again. The condition is the same as in the IF command (ie, `X < Y`).

For example, the following will keep printing the word "Hello" on the console for 4 seconds then stop:

```
Timer = 0
DO WHILE Timer < 4000
  PRINT "Hello"
LOOP
```

Note that `Timer` is a function within MMBasic which will return the time in milliseconds since the timer was reset. A reset is done by assigning zero to `Timer` (as done above) or when powering up the PicoMite.

A variation on the DO-LOOP structure is the following:

```
DO
   <statement>
   <statement>
LOOP UNTIL condition
```

In this arrangement the loop is first executed once, the *condition* is then tested and if the condition is false, the loop will be repeatedly executed until the *condition* becomes true. Note that the test in LOOP UNTIL is the inverse of DO WHILE.

For example, similar to the previous example, the following will also print "Hello" for four seconds:

```
Timer = 0
DO
  PRINT "Hello"
LOOP UNTIL Timer >= 4000
```

Both forms of the DO-LOOP essentially do the same thing, so you can use whatever structure fits with the logic that you wish to implement.

Finally, it is possible to have a DO Loop that has no conditions at all - ie,

```
DO
   <statement>
   <statement>
LOOP
```

This construct will continue looping forever and you, as the programmer, will need to provide a way to explicitly exit the loop (the EXIT DO command will do this). For example:

```
Timer = 0
DO
  PRINT "Hello"
  IF Timer >= 4000 THEN EXIT DO
LOOP
```

### Console Input

As well as printing data for the user to see your programs will also want to get input from the user. For that to work you need to capture keystrokes from the console and this can be done with the INPUT command. In its simplest form the command is:

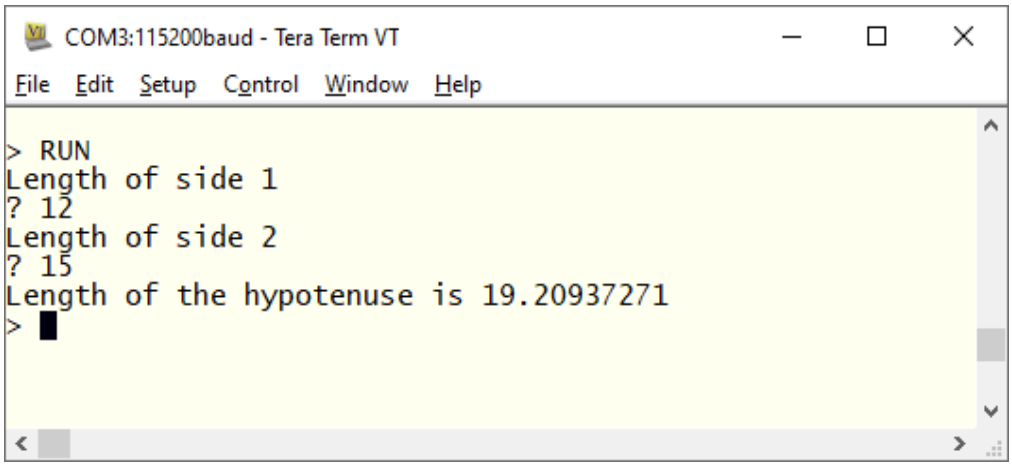
```
INPUT var
```

This command will print a question mark on the console's screen and wait for a number to be entered followed by the Enter key. That number will then be assigned to the variable `var`.

For example, the following program extends the expression for finding the hypotenuse of a triangle by allowing the user to enter the lengths of the other sides from the console.

```
PRINT "Length of side 1"
INPUT a
PRINT "Length of side 2"
INPUT b
PRINT "Length of the hypotenuse is" SQR(a * a + b * b)
```

This is a screen capture of a typical session:




The INPUT command can also print your prompt for you, so that you do not need a separate PRINT command. For example, this will work the same as the above program:

```
INPUT "Length of side 1"; a
INPUT "Length of side 2"; b
PRINT "Length of the hypotenuse is" SQR(a * a + b * b)
```

Finally, the INPUT command will allow you to input a series of numbers separated by commas with each number being saved in different variables.

For example:

```
INPUT "Enter the length of the two sides: ", a, b
PRINT "Length of the hypotenuse is" SQR(a * a + b * b)
```

If the user entered `12,15` the number `12` would be saved in the variable `a` and `15` in `b`.

Another method of getting input from the console is the LINE INPUT command. This will get the whole line as typed by the user and allocate it to a string variable. Like the INPUT command you can also specify a prompt. This is a simple example:

```
LINE INPUT "What is your name? ", s$
PRINT "Hello " s$
```

We will cover string variables later in this tutorial but for the moment you can think of them as a variable that holds a sequence of characters. If you ran the above program and typed in `John` when prompted the program would respond with `Hello John`.

Sometimes you do not want to wait for the user to hit the enter key, you want to get each character as it is typed in. This can be done with the INKEY$ function which will return the value of the character as a string consisting of just one character or an empty string (ie, contains no characters) if nothing has been entered.

### GOTO and Labels

One method of controlling the flow of the program is the GOTO command. This essentially tells MMBasic to jump to another part of the program and continue executing from there. The target of the GOTO is a label and this needs to be explained first.

A label is an identifier that marks part of the program. It must be the first thing on the line and it must be terminated with the colon (:) character. The name that you use can be up to 31 characters long and must follow the same rules as for a variable's name. For example, in the following program line LoopBack is a label:

```
LoopBack:  a = a + 1
```

When you use the GOTO command to jump to that particular part of the program you would use the command like this:

```
GOTO LoopBack
```

To put all this into context the following program will print out all the numbers from 1 to 10:

```
z = 0
LoopBack:  z = z + 1
PRINT z
IF z < 10 THEN GOTO LoopBack
```

The program starts by setting the variable `z` to zero then incrementing it to 1 in the next line. The value of `z` is printed and then tested to see if it is less than 10. If it is less than 10 the program execution will jump back to the label `LoopBack` where the process will repeat. Eventually the value of `z` will reach 10 and the program will run off the end and terminate.

Note that a FOR loop can do the same thing (and is simpler), so this example is purely designed to illustrate what the GOTO command can do.

In the past the GOTO command gained a bad reputation. This is because using GOTOs it is possible to create a program that continuously jumps from one point to another (often referred to as "spaghetti code") and that type of program is almost impossible for another programmer to understand. With constructs like the multiline IF statements the need for the GOTO statement has been reduced and it should be used only when there is no other way of changing the program's flow.

### Testing for Prime Numbers

The following is a simple program which brings together many of the programming features previously discussed.

```
DO
  InpErr:
  PRINT
  INPUT "Enter a number: "; a
  IF a < 2 THEN
    PRINT "Number must be equal or greater than 2"
    GOTO InpErr
  ENDIF

  Divs = 0
  FOR x = 2 TO SQR(a)
    r = a/x
    IF r = FIX(r) THEN Divs = Divs + 1
  NEXT x

  PRINT a " is ";
  IF Divs > 0 THEN PRINT "not ";
  PRINT "a prime number."
LOOP
```

This will first prompt (on the console) for a number and, when it has been entered, it will test if that number is a prime number or not and display a suitable message.

It starts with a DO Loop that does not have a condition – so it will continue looping forever. This is what we want. It means that when the user has entered a number, it will report if it is a prime number or not and then loop around and ask for another number. The way that the user can exit the program (if they wanted to) is by typing the break character (normally CTRL-C).

The program then prints a prompt for the user which is terminated with a semicolon character. This means that the cursor is left at the end of the prompt for the INPUT command which will get the number and store it in the variable a.

Following this the number is tested. If it is less than 2 an error message will be printed and the program will jump backwards and ask for the number again.

We are now ready to test if the number is a prime number. The program uses a FOR loop to step through the possible divisors testing if each one can divide evenly into the entered number. Each time it does the program will increment the variable `Divs`.

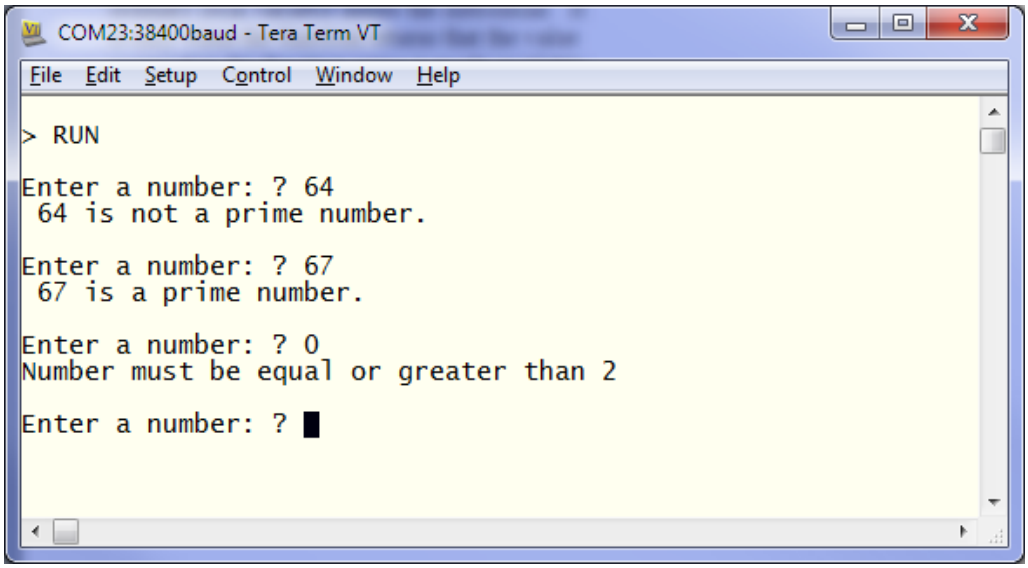
Note that the test is done with the function FIX(r) which simply strips off any digits after the decimal point. So, the condition `r = FIX(r)` will be true if `r` is an integer (ie, has no digits after the decimal point).

Finally, the program will construct the message for the user. The key part is that if the variable `Divs` is greater than zero it means that one or more numbers were found that could divide evenly into the test number. In that case the IF statement inserts the word "not" into the output message.

For example, if the entered number was 21 the user will see this response:

```
21 is not a prime number.
```

This is the result of running the program and some of the output:




You can test this program by using the editor (the EDIT command) to enter it.

Using your newly learnt skills you could then have a shot at making it more efficient. For example, because the program counts how many times a number can be divided into the test number it takes a lot longer than it should to detect a non prime number. The program would run much more efficiently if it jumped out of the FOR loop at the first number that divided evenly. You could use the GOTO command to do this or you could use the command EXIT FOR – that would cause the FOR loop to terminate immediately.

Other efficiencies include only testing the division with odd numbers (by using an initial test for an even number then starting the FOR loop at 3 and using STEP 2) or by only using prime numbers for the test (that would be much more complicated).

## Arrays




Arrays are something which you will probably not think of as useful at first glance but when you do need to use them you will find them very handy indeed.

An array is best thought of as a row of letterboxes for a block of units or condos as shown on the right. The letterboxes are all located at the same address and each box represents a unit or condo at that address. You can place a letter in the box for unit one, or unit two, etc.

Similarly an array in BASIC is a single variable with multiple sub units (called *elements* in BASIC) which are numbered. You can place data in element one, or element two, etc. In BASIC an array is created by the DIM command, for example:

```
DIM numarr(300)
```

This creates an array with the name of `numarr` containing 301 elements (think of them as letterboxes) ranging from 0 to 300. By default an array will start from zero so this is why there is an extra element making the total 301. To specify a specific element in the array (ie, a specific letterbox) you use an index which is simply the number of the array element that you wish to access. For example, if you want to set element number 100 in this array to (say) the number 876, you would do it this way:

```
numarr(100) = 876
```

Normally the index to an array is not a constant number as in this example (ie, 100) but a variable which can be changed to access different array elements.

As an example of how you might use an array, consider the case where you would like to record the maximum temperature for each day of the year and, at the end of the year, calculate the overall average. You could use ordinary variables to record the temperature for each day but you would need 365 of them and that would make your program quite unwieldy. Instead, you could define an array to hold the values like this:

```
DIM days(365)
```

Every day you would need to save the temperature in the correct location in the array. If the number of the day in the year was held in the variable `doy` and the maximum temperature was held in the variable `maxtemp` you would save the reading like this:

```
days(doy) = maxtemp
```

At the end of the year it would be simple to calculate the average for the year.
For example:

```
total = 0
FOR i = 1 to 365
  total = total + days(i)
NEXT i
PRINT "Average is:"  total / 365
```



This is much easier than adding up and averaging 365 individual variables.

The above array was single dimensioned but you can have multiple dimensions. Reverting to our analogy of letterboxes, an array with two dimensions could be thought of as a block of flats with multiple floors. A block could have a row of four letter boxes for level one, another row of four boxes for level two, and so on. To place a letter in a letterbox you need to specify the floor number and the unit number on that floor.

In BASIC such an array is specified using two indices separated by a comma. For example:

```
LetterBox(floor, unit)
```

As a practical example, assume that you needed to record the maximum temperature for each day over five years. To do this you could dimension the array as follows:

```
DIM days(365, 5)
```

The first index is the day in the year and the second is a number representing the year. If you wanted to set day 100 in year 3 to 24 degrees you would do it like this:

```
days(100, 3) = 24
```

In MMBasic for the PicoMite firmware, you can have up to six dimensions with the RP2040 processor or five dimensions with the RP2350 processor. The size of an array is limited only by the amount of free RAM that is available.

A complete array can be copied into another array with a single assignment by using the array names with empty brackets (eg, b() = a()). Both arrays must be the same type and hold the same total number of elements.

### Integers

So far all the numbers and variables that we have been using have been floating point. As explained before, floating point is handy because it will track digits after the decimal point and when you use division it will return a sensible result. So, if you just want to get things done and are not concerned with the details you should stick to floating point.

However, the limitation of floating point is that it stores numbers as an approximation with an accuracy of 14 digits in the PicoMite firmware. Most times this characteristic of floating point numbers is not a problem but there are some cases where you need to accurately store larger numbers.

As an example, let us say that you want to manipulate time accurately down to the microsecond so that you can compare two different date/times to work out which one is earlier. The easy way to do this is to convert the date/time to the number of microseconds since some date (say 1st Jan in year zero) - then finding the earliest of the two is just a matter of using an arithmetic compare in an IF statement.

The problem is that the number of microseconds since that date will exceed the accuracy range of floating point variables and this is where integer variables come in. An integer variable can accurately hold very large numbers up to nine million million million (or ±9223372036854775807 to be precise).

The downside of using an integer is that it cannot store fractions (ie, numbers after the decimal point). Any calculation that produces a fractional result will be rounded up or down to the nearest whole number when assigned to an integer. However this characteristic can be handy when dealing with money – for example, you don't want to send someone a bill for $100.13333333333.

It is easy to create an integer variable, just add the percent symbol (%) as a suffix to a variable name. For example, `sec%` is an integer variable. Within a program you can mix integers and floating point and MMBasic will make the necessary conversions but if you want to maintain the full accuracy of integers you should avoid mixing the two.

Just like floating point you can have arrays of integers, all you need to do is add the percent character as a suffix to the array name. For example: `days%(365, 5)`.

Beginners often get confused as to when they should use floating point or integers and the answer is simple… always use floating point unless you need an extremely high level of accuracy. This does not happen often but when you need them you will find that integers are quite useful.

### Strings

Strings are another variable type (like floating point and integers). Strings are used to hold a sequence of characters. For example, in the command:

```
PRINT "Hello"
```

The string "Hello" is a string constant. Note that a constant is something that does not change (as against a variable, which can) and that string constants are always surrounded by double quotes.

String variables names use the dollar symbol ($) as a suffix to identify them as a string instead of a normal floating point variable and you can use ordinary assignment to set their value. The following are examples (note that the second example uses an array of strings):

```
Car$ = "Holden"
Country$(12) = "India"
Name$ = "Fred"
```

You can also join strings using the plus operator:

```
Word1$ = "Hello"
Word2$ = "World"
Greeting$ = Word1$ + " " + Word2$
```

In which case the value of `Greeting$` will be "`Hello World`".

Strings can also be compared using operators such as = (equals), <> (not equals), < (less than), etc. For example:

```
IF Car$ = "Holden" THEN PRINT "Was an Aussie made car"
```

The comparison is made using the full ASCII character set so a space will come before a printable character. Also the comparison is case sensitive so 'holden' will not equal "Holden". Using the function UCASE() to convert the string to upper case you can then have a case insensitive comparison. For example:

```
IF UCASE$(Car$) = "HOLDEN" THEN PRINT "Was an Aussie made car"
```

You can have arrays of strings but you need to be careful when you declare them as you can rapidly run out of RAM (general memory used for storing variables, etc). This is because MMBasic will by default allocate 255 bytes of RAM for each element of the array. For example, a string array with 100 elements will by default use 25K of RAM.

To alleviate this you can use the LENGTH qualifier to limit the maximum size of each element. For instance, if you know that the maximum length of any string that will be stored in the array will be less than 20 characters you can use the following declaration to allocate just 20 bytes for each element:

```
DIM MyArray$(100) LENGTH 20
```

The resultant array will only use 2K of RAM.

You can use the LENGTH qualifier when declaring a normal (non array) string variable and it will save 256 bytes of RAM if the length is 9 or less (RP2040) or 15 or less (RP2350).

### Manipulating Strings

String handling is one of MMBasic's strengths and using a few simple functions you can pull apart and generally manipulate strings. The basic string functions are:

| | |
|---|---|
| LEFT$(string$, nbr ) | Returns a substring of *string$* with *nbr* of characters from the left (beginning) of the string. |
| RIGHT$(string$, nbr ) | Same as the above but return *nbr* of characters from the right (end) of the string. |
| MID$(string$, pos, nbr ) | Returns a substring of *string$* with *nbr* of characters starting from the character *pos* in the string (ie, the middle of the string). |

For example if `S$ = "This is a string"`

then: `R$ = LEFT$(S$, 7)` would result in the value of R$ being set to: "This is"

and: `R$ = RIGHT$(S$, 8)` would result in the value of R$ being set to: "a string"
finally: `R$ = MID$(S$, 6, 2)` would result in the value of R$ being set to: "is"

Note that in MID$() the first character position in a string is number 1, the second is number 2 and so on. So, counting the first character as one, the sixth position is the start of the word "is".

Another useful function is:

INSTR(string$, pattern$ ) Returns a number representing the position at which *pattern$* occurs in *string$*.

This can be used to search for a string inside another string. The number returned is the position of the substring inside the main string. Like with MID$() the start of the string is position 1.

For example if S$ = "This is a string"
Then: `pos = INSTR(S$, " ")`
would result in `pos` being set to the position of the first space in S$ (ie, 5).

INSTR() can be combined with other functions so this would return the first **word** in S$:

```
R$ = LEFT$(S$, INSTR(S$, " ") - 1)
```

There is also an extended version of INSTR():

INSTR(pos, string$, pattern$ ) Returns a number representing the position at which *pattern$* occurs in *string$* when starting the search at the character position *pos*.

So we can find the second word in S$ using the following:

```
pos1 = INSTR(S$, " ")
pos2 = INSTR(pos1 + 1, S$, " ")
r$ = MID$(S$, pos1 + 1, pos2 - pos1)
```

This last example is rather complicated so it might be worth working through it in detail so that you can understand how it works.

Note that INSTR() will return the number zero if the sub string is not found and that any string function will throw an error (and halt the program) if that is used as a character position. So, in a practical program you would first check for zero being returned by INSTR() before using that value.

For example:

```
r$ = ""
pos1 = INSTR(S$, " ")
if pos1 > 0 THEN
  pos2 = INSTR(pos1 + 1, S$, " ")
  if pos2 > 0 THEN r$ = MID$(S$, pos1 + 1, pos2 - pos1)
ENDIF
```

### Scientific Notation

Before we finish discussing data types we need to cover off the subject of floating point numbers and scientific notation.

Most numbers can be written normally, for example 11 or 24.5, but very large or small numbers are more difficult. For example, it has been estimated that the number of grains of sand on planet Earth is 7500000000000000000. The problem with this number is that you can easily lose track of how many zeros there are in the number and consequently it is difficult to compare this with a similar sized number.

A scientist would write this number as $7.5 \times 10^{18}$ which is called scientific notation and is much easier to comprehend.

When using the PRINT command MMBasic will automatically use scientific notation when printing very large or small floating point numbers. For example, if the above number was stored in a floating

point variable the PRINT command would display it as `7.5E+18` (this is BASIC's way of representing 7.5 x $10^{18}$). As another example, the number 0.0000000456 would display as `4.56E-8` which is the same as 4.56 x $10^{-8}$.

You can also use scientific notation when entering constant numbers in MMBasic. For example:

```
SandGrains = 7.5E+18
```

MMBasic only uses scientific notation for floating point numbers (not integers). For instance, if you assigned the number of grains of sand to an integer variable the PRINT command would display it as a normal number (with lots of zeros).

### DIM Command

We have used the DIM command before for defining arrays but it can also be used to create ordinary variables. For example, you can simultaneously create four string variables like this:

```
DIM STRING Car, Name, Street, City
```

Note that because these variables have been defined as strings using the DIM command we do not need the $ suffix, the definition alone is enough for MMBasic to identify their type. Similarly, when you use these variables in an expression you do not need the type suffix: For example:

```
City = "Sydney"
```

You can also use the keyword INTEGER to define a number of integer variables and FLOAT to do the same for floating point variables. This type of notation can similarly be used to define arrays.

For example:

```
DIM INTEGER seconds(200)
```

Another method of defining the variables type is to use the keyword AS. For example:

```
DIM Car AS STRING, Name AS STRING, Street AS STRING
```

This is the method used by Microsoft (MMBasic tries to maintain Microsoft compatibility) and it is useful if the variables have different types. For example:

```
DIM Car AS STRING, Age AS INTEGER, Value AS FLOAT
```

You can use any of these methods of defining a variable's type, they all act the same.

The advantage of defining variables using the DIM command is that they are clearly defined (preferably at the start of the program) and their type (float, integer or string) is not subject to misinterpretation.

You can strengthen this by using the following commands at the very top of your program:

```
OPTION EXPLICIT
OPTION DEFAULT NONE
```

The first specifies to MMBasic that all variables must be explicitly defined using DIM before they can be used. The second specifies that the type of all variables must be specified when they are created.

Why are these two commands important?

The first can help avoid a common programming error which is where you accidently misspell a variable's name. For example, your program might have the current temperature saved in a variable called `Temp` but at one point you accidently misspell it as `Tmp`. This will cause MMBasic to automatically create a variable called `Tmp` and set its value to zero.

This is obviously not what you want and it will introduce a subtle error which could be hard to find, even if you were aware that something was not right. On the other hand, if you used the OPTION EXPLICIT command at the start of your program MMBasic would refuse to automatically create the variable and instead would display an error thereby saving you from a probable headache.

The command OPTION DEFAULT NONE further helps because it tells MMBasic that the programmer must specifically specify the type of every variable when they are declared. It is easy to forget to specify the type and allowing MMBasic to automatically assume the type can lead to unexpected consequences.

For small, quick and dirty programs, it is fine to allow MMBasic to automatically create variables but in larger programs you should always disable this feature with OPTION EXPLICIT and strengthen it with OPTION DEFAULT NONE.

When a variable is created it is set to zero for float and integers and an empty string (ie, contains no characters) for a string variable. You can set its initial value to something else when it is created using DIM.

For example:

```
DIM FLOAT nbr = 12.56
DIM STRING Car = "Ford", City = "Perth"
```

You can also initialise arrays by placing the initialising values inside brackets like this:

```
DIM s$(2) = ("zero", "one", "two")
```

Note that because arrays start from zero by default this array actually has three elements with the index numbers of 0, 1 and 2. This is why we needed three string constants to initialise it.

### Constants

A common requirement in programming is to define an identifier that represents a value without the risk of the value being accidently changed - which can happen if variables were used for this purpose. These are called constants and they can represent I/O pin numbers, signal limits, mathematical constants and so on.

You can create a constant using the CONST command. This defines an identifier that acts like a variable but is set to a value that cannot be changed.

For example, if you wanted to check the voltage of a battery connected to pin 31 you could define the relevant values thus:

```
CONST BatteryVoltagePin = 31
CONST BatteryMinimum = 1.5
```

These constants can then be used in the program where they make more sense to the casual reader than simple numbers. For example:

```
SETPIN BatteryVoltagePin, AIN
IF PIN(BatteryVoltagePin) < BatteryMinimum THEN SoundAlarm
```

It is good programming practice to use constants for any fixed number that represents an important value. Normally they are defined at the start of a program where they are easy to see and conveniently located for another programmer to adjust (if necessary).

### Subroutines

A subroutine is a block of programming code which is self contained (like a module) and can be called from anywhere within your program. To your program it looks like a built in MMBasic command and can be used the same. For example, assume that you need a command that would signal an error by printing a message on the console. You could define the subroutine like this:

```
SUB ErrMsg
  PRINT "Error detected"
END SUB
```

With this subroutine embedded in your program all you have to do is use the command ErrMsg whenever you want to display the message. For example:

```
IF A < B THEN ErrMsg
```

The definition of a subroutine can be anywhere in the program but typically it is at the end. If MMBasic runs into the definition while running your program it will simply skip over it.

The above example is fine enough but it would be better if a more useful message could be displayed, one that could be customised every time the subroutine was called. This can be done by passing a string to the subroutine as an argument (sometimes called a parameter).

In this case the definition of the subroutine would look like this:

```
SUB ErrMsg  Msg$
  PRINT "Error: " + Msg$
END SUB
```

Then, when you call the subroutine, you can supply the string to be printed on the command line of the subroutine.

For example:

```
IF A < B THEN ErrMsg "Number too small"
```

When the subroutine is called like this the message "`Error: Number too small`" will be printed on the console. Inside the subroutine `Msg$` will have the value of "Number too small" when called like this and it will be concatenated in the PRINT statement to make the full error message.

A subroutine can have any number of arguments which can be float, integer or string with each argument separated by a comma.

Within the subroutine the arguments act like ordinary variables but they exist only within the subroutine and will vanish when the subroutine ends. You can have variables with the same name in the main program and they will be hidden within the subroutine and be different from arguments defined for the subroutine.

The type of the argument to be supplied can be specified with a type suffix (ie, $, % or ! for string, integer and float). For example, in the following the first argument must be a string and the second an integer:

```
SUB MySub Msg$, Nbr%
  …
END SUB
```

MMBasic will convert the supplied values if it can, so if your program supplied a floating point value as the second argument MMBasic will convert it to an integer. If MMBasic cannot convert the value it will display an error and return to the command prompt. For example, if you supplied a string for the second argument your program will stop with an error.

You do not have to use the type suffixes, you can instead define the type of the arguments using the AS keyword similar to the way it is used in the DIM command.

For example, the following is identical to the above example:

```
SUB MySub Msg AS STRING, Nbr AS INTEGER
  …
END SUB
```

Of course, if you used only one variable type throughout the program and used OPTION DEFAULT to set that type you could ignore the question of variable types completely.

When a subroutine is called with an argument that is a variable (ie, not a constant or expression) MMBasic will create a corresponding variable within the subroutine *that points back to this variable.* Any changes to the variable representing the argument inside the subroutine will also change the variable used in the call. This is called passing arguments by reference.

This is best explained by example:

```
DIM MyNumber = 5       ' set the variable to 5
CalcSquare MyNumber    ' the subroutine will square its value
PRINT MyNumber         ' this will print the number 25
END

SUB CalcSquare nbr
  nbr = nbr * nbr      ' square the argument and pass it back
END SUB
```

The subroutine CalcSquare will take its argument, square it and write it back to the variable representing the argument (`nbr`). Because the subroutine was called with a variable (`MyNumber`) the variable `nbr` will point back to `MyNumber` and any change to `nbr` will also change `MyNumber` accordingly. As a result the PRINT statement will output 25.

Passing arguments by reference is handy because it allows a subroutine to pass values back to the code that called it. However, it could lead to trouble if a subroutine used the variable representing an argument as a general purpose variable and changed its value. Then, if it were called with a variable as an argument, that variable would be inadvertently changed. To avoid this, you should prefix its definition with the keyword BYVAL. This instructs MMBasic to always use the value of the argument, even if it is a variable, and to never point back to the variable used in the call.

When you call a subroutine you can omit some (or all) of the parameters and they will take the value of zero (for floats or integers) or an empty string. This is handy as your subroutine can tell if a parameter is missing and act accordingly.

For example, here is our subroutine to generate an error message but this version can be used without specifying an error message as a parameter:

```
SUB ErrMsg  Msg$
  IF Msg$ = "" THEN
    PRINT "Error detected"
  ELSE
    PRINT "Error: " + Msg$
  ENDIF
END SUB
```

Within a subroutine you can use most features of MMBasic including calling other subroutines, IF…THEN commands, FOR…NEXT loops and so on. However, one thing that you cannot do is jump out of a subroutine using GOTO (if you do the result will be undefined and may cause your hair to turn grey).

Normally the subroutine will exit when the END SUB command is reached but you can also terminate the subroutine early by using the EXIT SUB command.

### Functions

Functions are similar to subroutines with the main difference being that a function is used to return a value in an expression. For example, if you wanted a function to convert a temperature from degrees Celsius to Fahrenheit you could define:

```
FUNCTION Fahrenheit(C)
  Fahrenheit = C * 1.8 + 32
END FUNCTION
```

Then you could use it in an expression:

```
Input "Enter a temperature in Celsius: ", t
PRINT "That is the same as" Fahrenheit(t) "F"
```

Or as another example:

```
IF Fahrenheit(temp) <= 32 THEN PRINT "Freezing"
```

You could also define the reverse:

```
FUNCTION Celsius(F)
  Celsius = (F - 32) * 0.5556
END FUNCTION
```

As you can see, the function name is used as an ordinary local variable inside the subroutine. It is only when the function returns that the value is made available to the expression that called it.

The rules for the argument list in a function are similar to subroutines. The only difference is that parentheses are always required around the argument list when you are calling a function, even if there are no arguments (parentheses are optional when calling a subroutine).

To return a value from the function you assign a value to the function's name within the function. If the function's name is terminated with a type suffix (ie, $, a % or a !) the function will return that type (string, integer or float), otherwise it will return whatever the OPTION DEFAULT is set to. For example, the following function will return a string:

```
FUNCTION LVal$(nbr)
  IF nbr = 0 THEN LVal$ = "False" ELSE LVal$ = "True"
END FUNCTION
```

You can explicitly specify the type of the function by using the AS keyword and then you do not need to use a type suffix (similar to defining a variable using DIM).

This is the above example rewritten to take advantage of this feature:

```
FUNCTION LVal(nbr) AS STRING
  IF nbr = 0 THEN LVal = "False" ELSE LVal = "True"
END FUNCTION
```

In this case the type returned by the function `LVal` will be a string.

A function is not limited to returning a single value - it can also return a whole array. This is described in the Subroutines and Functions chapter earlier in this manual.

As for subroutines you can use most features of MMBasic within functions. This includes FOR…NEXT loops, calling other functions and subroutines, etc. Also, the function will return to the expression that called it when the END FUNCTION command is reached but you can also return early by using the EXIT FUNCTION command.

### Local Variables

Variables that are created using DIM or that are just automatically created are called *global* variables. This means that they can be seen and used anywhere in the program including within subroutines and functions. However, inside a subroutine or function you will often need to use variables for various tasks that are internal to the subroutine/function. In portable code you do not want the name you chose for such a variable to clash with a global variable of the same name. To this end you can define a variable using the LOCAL command within the subroutine/function.

The syntax for LOCAL is identical to the DIM command, this means that the variable can be an array, you can set the type of the variable and you can initialise it to some value.

For example, this is our ErrMsg subroutine but this time it has been extended to use a local variable for joining the error message strings.

```
SUB ErrMsg  Msg$
  LOCAL STRING tstr
  tstr = "Error: " + Msg$
  PRINT tstr
END SUB
```

The variable tstr is declared as LOCAL within the subroutine, which means that (like the argument list) it will only exist within the subroutine and will vanish when the subroutine exits. You can have a global variable called tstr in your main program and it will be different from the variable tstr in the subroutine (in this case the global tstr will be hidden within the subroutine).

You should always use local variables for operations within your subroutine or function because they help make the module much more self contained and portable.

### Static Variables

LOCAL variables are reset to their initial values (normally zero or an empty string) every time the subroutine or function starts, however there are times when you would like the variable to retain its value between calls. This type of variable is defined with the STATIC command.

We can demonstrate how STATIC variables are useful by extending the ErrMsg subroutine to prevent duplicated calls to the subroutine repeatedly displaying the same message. For example, our program might call this subroutine from multiple places but if the message is the same in a number of subsequent calls we would like to see the message just once.

This is our new subroutine:

```
SUB ErrMsg  Msg$
  STATIC STRING lastmsg
  LOCAL STRING tstr
  IF Msg$ <> lastmsg THEN
    tstr = "Error: " + Msg$
    PRINT tstr
    lastmsg = Msg$
  ENDIF
END SUB
```

To keep track of the last message displayed we use a static variable called lastmsg. This will hold the text of the last message and we can compare it to the current message text to determine if it is different and therefore should be printed. This would give just one message every time a call is made with a duplicate message text.

The STATIC command uses exactly the same syntax as DIM. This means that you can define different types of static variables including arrays and you can also initialise them to some value.

The static variable is created on the first time the STATIC command is encountered and it is automatically set to zero (if a float or integer) or an empty string. On subsequent calls to the subroutine or function MMBasic will recognise that the variable has already been created and it will leave its value untouched (ie, whatever it was in the previous call). As with DIM you can also initialise a static variable to some value. For example:

```
STATIC INTEGER var = 123
```

On the first call (when the variable is created) it will be initialised to 123 but on subsequent calls it will keep whatever its value was previously set to.

Mostly static variables are used to keep track of the *state* of something while inside a subroutine or function. A *state* is a record of something that has happened previously.

Examples include:

- Has the COM port already been opened?
- What steps in a sequence have we completed?
- What text has already been displayed?

Normally you will use global variables (created using DIM) to track a *state* but sometimes you want this to be contained within a module and this is where static variables are valuable. Just like LOCAL the use of STATIC helps to make your subroutines and functions more self contained and portable.

### Calculate Days

We have covered a lot of programming commands and techniques so far in this tutorial and before we finish it would be worth giving an example of how they work together. The following is an example that uses many features of the BASIC language to calculate the number of days between two dates:

```
' Example program to calculate the number of days between two dates

OPTION EXPLICIT
OPTION DEFAULT NONE

DIM STRING s
DIM FLOAT d1, d2

DO
  ' main program loop
  PRINT : PRINT "Enter the date as  dd mmm yyyy"
  PRINT " First date";
  INPUT s
  d1 = GetDays(s)
  IF d1 = 0 THEN PRINT "Invalid date!" : CONTINUE DO
  PRINT "Second date";
  INPUT s
  d2 = GetDays(s)
  IF d2 = 0 THEN PRINT "Invalid date!" : CONTINUE DO
  PRINT "Difference is" ABS(d2 - d1) " days"
LOOP

' Calculate the number of days since 1/1/1900
FUNCTION GetDays(d$) AS FLOAT
  LOCAL STRING Month(11) =
("jan","feb","mar","apr","may","jun","jul","aug","sep","oct","nov","dec")
  LOCAL FLOAT Days(11) = (0,31,59,90,120,151,181,212,243,273,304,334)
  LOCAL FLOAT day, mth, yr, s1, s2

  ' Find the separating space character within a date
  s1 = INSTR(d$, " ")
  IF s1 = 0 THEN EXIT FUNCTION
  s2 = INSTR(s1 + 1, d$, " ")
  IF s2 = 0 THEN EXIT FUNCTION

  ' Get the day, month and year as numbers
  day = VAL(MID$(d$, 1, s2 - 1)) - 1
  IF day < 0 OR day > 30 THEN EXIT FUNCTION
  FOR mth = 0 TO 11
    IF LCASE$(MID$(d$, s1 + 1, 3)) = Month(mth) THEN EXIT FOR
  NEXT mth
  IF mth > 11 THEN EXIT FUNCTION
  yr = VAL(MID$(d$, s2 + 1)) - 1900
  IF yr < 1 OR yr >= 200 THEN EXIT FUNCTION

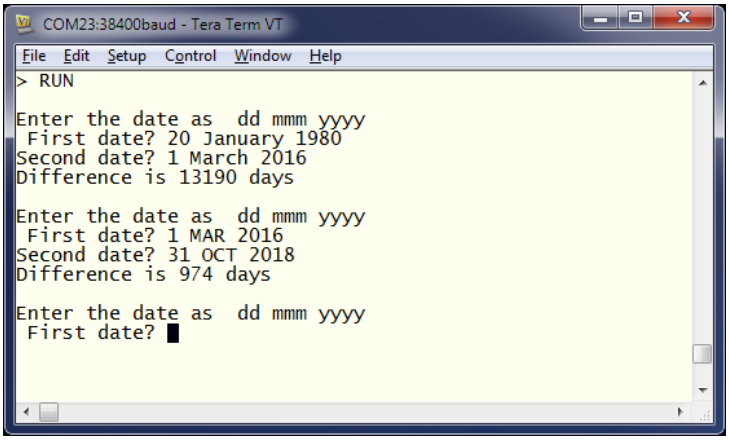
  ' Calculate the number of days including adjustment for leap years
  GetDays = (yr * 365) + FIX((yr - 1) / 4)
  IF yr MOD 4 = 0 AND mth >= 2 THEN GetDays = GetDays + 1
  GetDays = GetDays + Days(mth) + day
END FUNCTION
```

Note that the line starting `LOCAL STRING Month(11)` has been wrapped around because of the limited page width – it is one line as follows:

```
LOCAL STRING Month(11) = ("jan","feb","mar","apr","may","jun","jul","aug","sep","oct","nov","dec")
```

This program works by getting two dates from the user at the console and then converting them to the number of days since 1900. With these two numbers a simple subtraction will give the number of days between them.

When this program is run it will ask for the two dates to be entered and you need to use the form of: `dd mmm yyyy`.




The screen capture on the right shows what the running program will look like.

The main feature of the program is the defined function `GetDays()` which takes a string entered at the console, splits it into its day, month and year components then calculates the number of days since 1st January 1900.

This function is called twice, once for the first date and then again for the second date. It is then just a matter of subtracting one date (in days) from the other to get the difference in days.

We will not go into the detail of how the calculations are made (ie, handling leap years) as that can be left as an exercise for the reader. However, it is appropriate to point out some features of MMBasic that are used by the program.

It demonstrates how local variables can be used and how they can be initialised. In the function `GetDays()` two arrays are declared and initialised at the same time. These are just a convenient method of looking up the names of the months and the cumulative number of days for each month. Later in the function (the FOR loop) you can see how they make dealing with twelve different months quite efficient.

Another feature highlighted by this program is the string handling features of MMBasic. The INSTR() function is used to locate the two space characters in the date string and then later the MID$() function uses these to extract the day, month and year components of the date. The VAL() function is used to turn a string of digits (like the year) into a number that can be stored in a numeric variable.

Note that the value of a function is initialised to zero every time the function is called and unless it is set to some value it will return a zero value. This makes error handling easy because we can just exit the function if an error is discovered. It is then the responsibility of the calling program code to check for a return value of zero which signifies an error.

This program illustrates one of the benefits of using subroutines and functions which is that when written and fully tested they can be treated as a trusted "black box" that does not have to be opened. For this reason functions like this should be the properly tested and then, if possible, left untouched (in case you add some error).

There are a few features of this program that we have not covered before. The first is the MOD operator which will calculate the remainder of dividing one number into another. For example, if you divided 4 into 15 you would have a remainder of 3 which is exactly what the expression 15 MOD 4 will return. The ABS() function is also new and will return its argument as a positive number (eg, ABS(-15) will return +15 as will ABS(15)).

The EXIT FOR command will exit a FOR loop even though it has not reached the end of its looping, EXIT FUNCTION will immediately exit a function even though execution has not reached the end of the function and CONTINUE DO will immediately cause the program to jump to the end of a DO loop and execute it again.

Why would this program be useful? Well some people like to count their age in days, that way every day is a birthday! You can calculate your age in days, just enter the date that you were born and today's date. That is not particularly useful but the program itself is valuable as it demonstrates many of the characteristics of programming in MMBasic. So, work your way through the program and review each section until you understand it – it should be a rewarding experience.

## Appendix K – Firmware Version Applicability

Not every command, function and option is present in every firmware image. Where an entry in the Commands, Functions, Options or Predefined Read Only Variables listings is restricted, the restriction is printed as an underlined label at the start of its description. This appendix defines every label that is used.

A label is one of two kinds. A version label states which of the sixteen firmware images provide the feature and is drawn from the closed list in the first table below - no other wording is used. A condition states a hardware or configuration requirement (PSRAM fitted, a touch panel configured, and so on) and is listed in the second table. An entry may carry a version label, a condition, or both.

An entry with no label is available in all sixteen firmware images.

## The sixteen firmware images

Names are given without the version number and the .uf2 extension, so PicoMiteRP2350VGA means the file PicoMiteRP2350VGAV7.0.00.uf2.

| **RP2040 images** | **RP2350 images** |
|---|---|
| PicoMiteRP2040 | PicoMiteRP2350 |
| PicoMiteRP2040MIN | PicoMiteRP2350USB |
| PicoMiteRP2040USB | PicoMiteRP2350VGA |
| PicoMiteRP2040VGA | PicoMiteRP2350VGAUSB |
| PicoMiteRP2040VGAUSB | WebMiteRP2350 |
| WebMiteRP2040 | PicoMiteRP2350BT |
| | PicoMiteRP2350BTH |
| | PicoMiteHDMI |
| | PicoMiteHDMIUSB |
| | PicoMiteHDMIWEB |
| | PicoMiteHDMIBTH |
| | PicoMitePC3 |

## Version labels

| **Label** | **Firmware images** | **Examples** |
|---|---|---|
| RP2350 VERSIONS ONLY | PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | ASTRO, CMM2 LOAD, CMM2 RUN, FRAME, LOCATION, RAM, SLEW, STAR, STEPPER, TMC22xx |
| RP2350 VERSIONS EXCEPT WEBMITERP2350 | PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | RAY |
| RP2350 PICOMITE VERSIONS ONLY | PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350BT, PicoMiteRP2350BTH | MODE 400 / MODE 800, OPTION LCD SPI |
| RP2350 USB VERSIONS ONLY | PicoMiteRP2350USB, PicoMiteRP2350VGAUSB, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | FLASH UF2 |
| RP2350 NON-USB VERSIONS ONLY | PicoMiteRP2350, PicoMiteRP2350VGA, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI | YMODEM |
| RP2350 VGA USB AND HDMI USB VERSIONS ONLY | PicoMiteRP2350VGAUSB, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | KEYBOARD, OPTION SCREEN KEYBOARD |
| VGA VERSIONS ONLY | PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, | MAP (4-bit colour form), MODE 1/2/3 |

| | | |
|---|---|---|
| | PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB | |
| HDMI VERSIONS ONLY | PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | RESOLUTION, OPTION HDMI PINS |
| VGA AND HDMI VERSIONS ONLY | PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | GETSCANLINE, TILE |
| VGA AND ALL RP2350 VERSIONS ONLY | PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | MANDELBROT |
| VGA, HDMI AND RP2350 PICOMITE VERSIONS ONLY | PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMIBTH, PicoMitePC3 | MAP, MODE |
| NOT VGA OR HDMI VERSIONS | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040USB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH | BACKLIGHT, REFRESH, FRAMEBUFFER, CPU SPEED, OPTION CPUSPEED, OPTION LCDPANEL, OPTION TOUCH, OPTION SYSTEM SPI, GUI CALIBRATE |
| NOT VGA, HDMIWEB OR HDMIBTH VERSIONS | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040USB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMitePC3 | CAMERA |
| WIFI VERSIONS ONLY | WebMiteRP2040, WebMiteRP2350, PicoMiteHDMIWEB, PicoMitePC3 | WEB, JSON$(), OPTION WIFI, OPTION TCP SERVER PORT, OPTION UDP SERVER PORT, OPTION TELNET CONSOLE, OPTION TFTP, MM.ADDRESS$, MM.MESSAGE$, MM.TOPIC$ |
| NOT WIFI VERSIONS | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040USB, PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIBTH | OPTION PICO, SETPIN GP25 HEARTBEAT, MM.SUPPLY |
| USB VERSIONS ONLY | PicoMiteRP2040USB, PicoMiteRP2040VGAUSB, PicoMiteRP2350USB, PicoMiteRP2350VGAUSB, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | GAMEPAD, OPTION MOUSE SENSITIVITY, MM.USB |
| NON-USB VERSIONS ONLY | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040VGA, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350VGA, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI | OPTION MOUSE, OPTION KEYBOARD PINS, OPTION PS2 PINS, MM.PS2 |
| USB AND BLUETOOTH KEYBOARD VERSIONS ONLY | PicoMiteRP2040USB, PicoMiteRP2040VGAUSB, PicoMiteRP2350USB, PicoMiteRP2350VGAUSB, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteRP2350BTH, PicoMiteHDMIBTH, PicoMitePC3 | OPTION KEYBOARD REPEAT |
| BLUETOOTH HOST VERSIONS ONLY | PicoMiteRP2350BTH, PicoMiteHDMIBTH, PicoMitePC3 | BLUETOOTH, OPTION AUDIO BLUETOOTH, MM.INFO(BLUETOOTH) |

| | | |
|---|---|---|
| RP2350 WIFI AND BLUETOOTH VERSIONS ONLY | WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | OPTION CYW43 PINS |
| NOT BLUETOOTH HOST VERSIONS | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040USB, PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB | OPTION AUDIO (PWM) |
| GUI CONTROLS VERSIONS ONLY | PicoMiteRP2040, PicoMiteRP2040USB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | CTRLVAL(), MSGBOX(), GUI CLICK, GUI CURSOR, OPTION GUI CONTROLS |
| NOT PICOMITEMIN | PicoMiteRP2040, PicoMiteRP2040USB, PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | END TYPE, FM, HELP, STRUCT, TYPE, OPTION DISK SAVE, OPTION DISK LOAD |
| NOT PICOMITEMIN OR WEBMITE | PicoMiteRP2040, PicoMiteRP2040USB, PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | DRAW3D, TURTLE |
| NOT PICOMITEMIN OR BLUETOOTH HOST VERSIONS | PicoMiteRP2040, PicoMiteRP2040USB, PicoMiteRP2040VGA, PicoMiteRP2040VGAUSB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB | OPTION AUDIO SPI, OPTION AUDIO VS1053 |
| NOT RP2040 VGA | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040USB, PicoMiteRP2040VGAUSB, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | TOUCH() |
| NOT RP2040 VGA, RP2040 WEBMITE OR PICOMITEMIN | PicoMiteRP2040, PicoMiteRP2040USB, PicoMiteRP2040VGAUSB, PicoMiteRP2350, PicoMiteRP2350USB, PicoMiteRP2350VGA, PicoMiteRP2350VGAUSB, WebMiteRP2350, PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMI, PicoMiteHDMIUSB, PicoMiteHDMIWEB, PicoMiteHDMIBTH, PicoMitePC3 | CLICK() |
| PS2 KEYBOARD VERSIONS ONLY | PicoMiteRP2040, PicoMiteRP2040MIN, PicoMiteRP2040VGA, WebMiteRP2040, PicoMiteRP2350, PicoMiteRP2350VGA, WebMiteRP2350, | OPTION KEYBOARD (layout), OPTION KEYBOARD I2C, OPTION KEYBOARD DISABLE |

| | | |
|---|---|---|
| | PicoMiteRP2350BT, PicoMiteHDMI | |
| PICOMITEPC3 ONLY | PicoMitePC3 | OPTION RADIO |

## Conditions

| Condition | Meaning |
|---|---|
| RP2350B VERSIONS ONLY | Requires a module using the 80-pin RP2350B (eg, the Pimoroni Pico Plus 2), not a restriction on which firmware image is loaded. |
| RP2350 VERSIONS WITH PSRAM ENABLED ONLY | Requires PSRAM fitted and configured with OPTION PSRAM PIN. |
| WITH A BUFFERED DISPLAY DRIVER ONLY | Requires one of the buffered display drivers (a driver name ending in BUFF) to be configured with OPTION LCDPANEL. |
| WITH TOUCH SUPPORT ONLY | Requires a touch controller configured with OPTION TOUCH. |
| IN MODE 1 ONLY | Applies only while the display is in MODE 1. |
| PICOCALC ONLY | Applies to the Clockwork PicoCalc hardware. |
| ONLY ON RGB332 (256 COLOUR) DISPLAYS | Applies only in the 8-bit RGB332 colour modes. |

## Appendix L – Where Each Keyword Is Documented

The Commands, Functions and Options listings in this manual are the reference for the MMBasic language. A few feature families are large enough to have a manual of their own in the firmware download, and for those the listing gives only a summary. This appendix says where the full description of each of them is.

Any keyword not named below is documented in this manual, in the Commands, Functions, Options or Predefined Read Only Variables listing, or in the chapter covering its subject.

| Keyword | Documented in |
|---|---|
| ASTRO, LOCATION, SLEW, STAR | GPS_Astro_Reference.pdf |
| BITSTREAM | BITSTREAM_User_Manual.pdf |
| BLIT (beyond the entries in the Commands listing) | BLIT_User_Manual.pdf |
| CTRLVAL(), MSGBOX(), and the GUI controls - BARGAUGE, BUTTON, CAPTION, CHECKBOX, DISPLAYBOX, FORMATBOX, LED, LISTBOX, NUMBERBOX, RADIO, SLIDER, SPINBOX, SWITCH, TEXTBOX and the date, time and position controls | Advanced Graphics Functions.pdf |
| DRAW3D, DRAW3D() | 3D_Graphics_User_Manual.pdf |
| FM (the file manager) | FM_User_Manual.pdf |
| FRAME, FRAME() | FRAME_User_Manual.pdf |
| GAMEPAD, and game controller support generally | Game_Input_Devices_Manual.pdf |
| GPS(), OPTION GPS | Option_GPS_User_Manual.pdf |
| MEMORY SHARE | MEMORY_SHARE_User_Manual.pdf |
| OPTION PROFILING | option-profiling-cache.pdf |
| PIO | Appendix F of this manual |
| PLAY SAMPLE | PLAY_SAMPLE_User_Manual.pdf |
| RAY, RAY() | Raycaster_User_Manual.pdf |
| SPRITE, SPRITE() | SPRITE_User_Manual.pdf and Appendix G of this manual |
| STEPPER, TMC22xx, MM.CODE | Stepper_Reference.pdf |
| STRUCT, STRUCT(), TYPE, END TYPE | MMBasic_Structures_Manual.pdf |
| TILEMAP, TILEMAP() | TILEMAP_User_Manual.pdf |
| TURTLE | Appendix H of this manual |
| Serial devices on COM3 to COM6 | USB_CDC_Host_User_Manual.pdf |

## Appendix M – Companion Manuals

Some features are large enough to have a manual of their own rather than an entry in the listings. These are included in the firmware distribution zip file alongside this manual. Appendix L says which keyword is described in which of them; this appendix says what each one covers and which firmware versions it applies to.

| Manual | Covers | Applies to |
|---|---|---|
| Advanced Graphics Functions.pdf | The advanced graphic controls - buttons, switches, LEDs, check boxes, list boxes, sliders, spin boxes, text and number boxes and the date, time and position controls - together with CTRLVAL() and MSGBOX(). | Versions with GUI controls (see Appendix K) |
| 3D_Graphics_User_Manual.pdf | The DRAW3D command and function: cameras, objects, rotation and hidden line removal. | All except PicoMiteRP2040MIN and the WebMite versions |
| BITSTREAM_User_Manual.pdf | Generating precisely timed bit sequences with the BITSTREAM command, including IR transmit and PS2 output. | All versions |
| BLIT_User_Manual.pdf | Using the BLIT commands for sprites, animation and off-screen composition. | All versions |
| FM_User_Manual.pdf | The two panel file manager opened with the FM command. | All except PicoMiteRP2040MIN |
| FRAME_User_Manual.pdf | Building text based user interfaces with the FRAME command and function. | RP2350 versions only |
| Game_Development_Guide.pdf | A tutorial on writing games, bringing together sprites, tile maps, sound and game controller input. | Versions with video output |
| Game_Input_Devices_Manual.pdf | Reading game controllers - USB gamepads, the Wii nunchuck and classic controller, and keyboard and mouse input. | All versions; the GAMEPAD command needs a USB version |
| GPS_Astro_Reference.pdf | The ASTRO, STAR, LOCATION and SLEW commands for telescope pointing and navigation. | RP2350 versions only |
| MEMORY_SHARE_User_Manual.pdf | The MEMORY SHARE commands, which pass data between two Picos over a dedicated high speed link. | All versions |
| MMBasic_Structures_Manual.pdf | User defined types: TYPE, END TYPE, STRUCT and the STRUCT() function. | All except PicoMiteRP2040MIN |
| Option_GPS_User_Manual.pdf | Connecting a GPS receiver, the OPTION GPS command and the GPS() function. | All versions |
| PLAY_SAMPLE_User_Manual.pdf | Playing sampled sound effects with PLAY SAMPLE. | All versions |
| PicoMiteBT.pdf | The Bluetooth versions in detail - the console over Bluetooth LE on PicoMiteRP2350BT, and Bluetooth keyboards, mice and speakers on PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3. | PicoMiteRP2350BT, PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3 |
| Raycaster_User_Manual.pdf | Writing raycast games with the RAY command and function. | RP2350 versions except WebMiteRP2350 |
| SPRITE_User_Manual.pdf | The SPRITE commands and function in full - collision detection, layers and animation. | All versions |
| Stepper_Reference.pdf | Driving stepper motors: the STEPPER and TMC22xx commands, G-code, acceleration planning and hardware limit switches. | RP2350 versions only |
| TILEMAP_User_Manual.pdf | Building scrolling backgrounds with the TILEMAP command and function. | All versions; the TILEMAP() function is not available on the WebMite versions |

| | | |
|---|---|---|
| USB_CDC_Host_User_Manual.pdf | Using USB serial devices on COM3 to COM6. | USB versions only |
| mmb2csub.pdf | The mmb2csub tool (user-tools/mmb2csub.py), which turns a BASIC SUB or FUNCTION into a CSUB without writing any C, so that the busiest routines of a program run as machine code. | All versions |
| option-profiling-cache.pdf | OPTION PROFILING, and the optimisations the interpreter applies to every program. | All versions; its Build Availability table lists the versions that also report per-subroutine times |